

Apuntes de programación III

C#, .NET y herramientas de desarrollo



Ing. Alejandro Di Battista

Apuntes de Programación III

C#, .NET y herramientas de desarrollo

Ing. Alejandro Di Battista

29/06/2026

Nota sobre este material

Este material fue realizado como soporte al dictado de clase. Para su elaboración se empleó inteligencia artificial como ayuda en tareas de redacción y corrección; la selección, organización y revisión pedagógica corresponden al docente.

Índice

Fundamentos del ecosistema .NET	6
Variables y manejo de memoria en C	14
Tipos de datos y operadores en C	18
Enumeraciones en C	23
Strings en C	26
Tipos compuestos y colecciones en C	31
Clases y objetos en C	38
Null y tipos anulables en C	43
Funciones, Delegados y Eventos en C	46
Interfaces y contratos en C	50
Control de flujo y manejo de excepciones en C	54
Switch Expression y Pattern Matching en C	58
Manejo de errores — try / catch / finally	61
Iteradores en C: IEnumerable, IEnumerator y yield	64
LINQ — Language Integrated Query	68
Programación Asíncrona en C — Guía Práctica	72
Principios SOLID en C	77
Patrones en .NET para aplicar SRP	81
Patrón Decorador en C	85
Patrón Builder en C	88
Patrón Strategy y Patrón Visitor en C	91

IServiceProvider en .NET — Inyección de dependencias	95
JSON en C	99
Serialización y Repositorio en C	103
ADO.NET con SQLite en C	106
Repository y Unit of Work en C	109
ADO.NET en C — Referencia rápida	112
Dapper en C — Referencia completa	115
Entrada, salida y redirección en programas de consola	118
Terminal.Gui: apuntes para construir aplicaciones TUI en C	122
Entrada/salida por consola, archivos de texto y encoding	155
Cómo funciona la mini TUI...	160
Terminal.Gui: cómo se piensa una interfaz de usuario en la terminal	169
Mini CRM con TUI, ADO.NET y Dapper — Diseño por capas	197
Programación Web Backend con C, Minimal APIs, SQLite, ADO.NET y Dapper	201
Pruebas de APIs con cURL — Guía Práctica	240
Entity Framework Core con SQLite	244
Autenticación desde cero: login, hash, token y middleware	278
Sistema de Tickets con Minimal API y EF Core	287
Middleware y Endpoint Filters desde los primeros principios	308
Razor Pages	318
HTMX	329
Agenda CRUD con Razor Pages, HTMX y CSS propio	338
Apunte de clase: Componentes en Blazor desde primeros principios	360

EditForm en Blazor	375
Tutorial: Una agenda de contactos con Blazor	380
Cómo guardar API keys	390
El API compatible con OpenAI: completar texto como primitiva de procesamiento	396
Microsoft.Extensions.AI: del chat a un asistente con herramientas	403
Tutorial: asistente de programación mínimo con tools	416
Tutorial: bot de WhatsApp en C con wacli y Microsoft.Extensions.AI	424
Microsoft Agent Framework, desde los primeros principios	442
Model Context Protocol (MCP) en C	454
REST API vs. MCP: Entendiendo la Revolución de las APIs para Inteligencia Artificial	464
Tutorial de CLI en C: sortx en un solo archivo	467
Parser descendente recursivo y AST	486
Carrito de compras con EF Core y SQLite	494
Construyendo un agente con shell en C	515
Tutorial: Sitio web con ASP.NET Core y Razor Pages	530
Blazor en .NET 10 / C 14 — Tutorial completo en español argentino	544
Anexo: Terminal y Bash para desarrollo	566
Anexo: Use de Git y GitHub	582

CAPITULO 01

Fundamentos del ecosistema .NET

Por qué existe .NET

Antes de definir qué es .NET, conviene entender qué problema resuelve.

Cuando escribimos un programa en C#, hacen falta varias cosas para que ese programa efectivamente funcione:

- alguien que **traduzca** el código que escribimos a algo que la computadora pueda ejecutar,
- un **motor de ejecución** que corra ese código de forma segura y eficiente,
- una **biblioteca estándar** con clases ya hechas para tareas comunes: leer archivos, manejar fechas, trabajar con strings, hacer HTTP, etc.,
- **herramientas** para compilar, ejecutar, empaquetar, publicar y depurar,
- una forma ordenada de **incorporar dependencias** de terceros.

Si cada uno tuviera que armar todo eso por su cuenta, empezar a programar costaría enormemente. .NET es la plataforma que provee todas esas piezas juntas, integradas y funcionando en Windows, Linux y macOS.

Qué es .NET

.NET es una **plataforma de desarrollo**: un conjunto coordinado de partes que permiten escribir, compilar y ejecutar software.

Las piezas principales son:

- un **runtime** que ejecuta el código,
- un **SDK** con todas las herramientas de desarrollo,
- la **Base Class Library** (BCL): miles de clases listas para usar,
- un sistema de **paquetes** (NuGet) para sumar dependencias externas,
- un conjunto de **frameworks** especializados para distintos tipos de aplicación.

Con .NET se pueden construir aplicaciones de consola, servicios, APIs REST, sitios web, aplicaciones de escritorio, aplicaciones móviles y bibliotecas reutilizables.

Una analogía

Pensar .NET como una plataforma ayuda a no confundirlo con el lenguaje:

- **C#** es el idioma con el que escribimos,

- **.NET** es toda la infraestructura que permite que ese idioma se convierta en programas que efectivamente corren,
- los **frameworks** son ambientes especializados que viven sobre .NET para resolver tipos concretos de problema (web, datos, móvil, etc.).

C# y .NET: cómo se relacionan

C# es **un** lenguaje que corre sobre .NET. El runtime también ejecuta otros lenguajes:

- **F#**, lenguaje funcional,
- **VB.NET**, con presencia histórica fuerte en proyectos antiguos.

Todos comparten el mismo runtime, la misma BCL y el mismo ecosistema de paquetes. En esta materia trabajamos con C#, pero conviene saber que la plataforma no depende de un único lenguaje.

Las piezas en detalle

Runtime

El runtime es el **motor de ejecución**. Cuando compilamos un programa C# no se genera código de máquina directamente, sino un formato intermedio llamado IL (Intermediate Language). El runtime se encarga de:

- cargar ese código intermedio,
- traducirlo a instrucciones de la máquina en tiempo de ejecución (JIT, *Just In Time*),
- administrar la memoria (recolección de basura),
- gestionar tipos, hilos y excepciones.

Una computadora con solo el runtime instalado puede **ejecutar** aplicaciones .NET, pero no puede compilarlas.

SDK

El SDK (Software Development Kit) contiene todo lo necesario para **desarrollar**:

- el compilador de C# (Roslyn),
- la herramienta de línea de comandos dotnet,
- la BCL,
- el runtime (incluido dentro del SDK),
- plantillas de proyectos,
- herramientas para restaurar paquetes, correr tests, publicar binarios, etc.

Para programar alcanza con instalar el SDK. El runtime queda incluido.

Dicho de otra forma:

- **runtime** = ejecutar,
- **SDK** = desarrollar (que incluye ejecutar).

Base Class Library (BCL)

La BCL es el conjunto enorme de clases listas para usar que vienen con la plataforma. Algunos espacios de nombres muy frecuentes:

- `System.IO` — archivos y streams,
- `System.Text.Json` — serialización JSON,
- `System.Linq` — consultas sobre colecciones,
- `System.Net.Http` — clientes HTTP,
- `System.Collections.Generic` — colecciones genéricas,
- `System.Threading.Tasks` — programación asíncrona.

Antes de buscar una biblioteca externa para hacer algo, vale la pena revisar si la BCL ya lo resuelve.

NuGet

NuGet es el gestor oficial de paquetes. Permite incorporar bibliotecas de terceros (o publicar las propias) de forma versionada y reproducible. Cuando un proyecto declara que depende de un paquete, las herramientas descargan, instalan y mantienen actualizada esa dependencia automáticamente.

Sin un gestor de paquetes, cada equipo tendría que copiar manualmente archivos `.dll` y administrar versiones a mano: algo muy frágil y propenso a errores.

Frameworks vs bibliotecas

Estos dos términos a veces se mezclan, pero conviene distinguirlos.

Una **biblioteca** ofrece clases y funciones que llamamos cuando las necesitamos. *Nuestro código manda*: usa la biblioteca como una herramienta.

Un **framework** define una estructura general en la que nuestro código encaja. *El framework manda*: llama a nuestro código en los puntos que él define.

Frameworks importantes del ecosistema

- **ASP.NET Core** — sitios web, APIs REST y servicios backend.
- **Entity Framework Core** — ORM para acceso a datos: mapea clases a tablas relacionales.
- **.NET MAUI** — aplicaciones multiplataforma con interfaz gráfica (Windows, macOS, Android, iOS).

Estos frameworks no son parte obligatoria de `.NET`: se suman como paquetes cuando un proyecto los necesita.

Setup mínimo del ambiente de desarrollo

Para empezar a programar en C# alcanza con dos cosas:

1. el **SDK de .NET** instalado,
2. un editor: **Visual Studio Code**.

No hace falta Visual Studio (el IDE pesado). VS Code más el SDK es suficiente para toda la materia.

1. Instalar el SDK de .NET

Bajar la versión .NET 10 desde:

```
texto
https://dotnet.microsoft.com/download
```

Hay instaladores para Windows, Linux y macOS. En Linux suele estar disponible también vía gestor de paquetes (apt, dnf, etc.).

Para verificar que quedó instalado, abrir una terminal y correr:

```
bash
dotnet --version
```

Tiene que mostrar un número del estilo 10.0.x.

Para ver más detalles:

```
bash
dotnet --info
```

2. Instalar Visual Studio Code

Bajarlo desde:

```
texto
https://code.visualstudio.com/
```

Una vez abierto, instalar la extensión **C# Dev Kit** desde el panel de extensiones (Ctrl+Shift+X). Esa extensión agrega:

- soporte de lenguaje (autocompletado, navegación, refactors),
- depurador integrado,
- explorador de soluciones,
- integración con el SDK.

Con esto el ambiente queda listo para escribir, compilar y depurar código C#.

3. Probar la CLI

La CLI (dotnet) es la puerta de entrada principal a todo. Algunos comandos básicos que vamos a usar mucho:

```
bash
dotnet --version           # ver versión instalada
dotnet new console         # crear un proyecto consola tradicional
dotnet run                 # compilar y ejecutar
dotnet build               # solo compilar
dotnet add package <nombre> # agregar un paquete NuGet
```

Primer programa: archivos C# sueltos (file-based apps)

Históricamente, hasta el menor de los programas en C# necesitaba un **proyecto**: una carpeta con archivo .csproj, carpetas bin/ y obj/, etc. Eso tiene sentido para proyectos serios, pero es ruido innecesario para algo tan simple como "probar una idea" o "escribir un script de diez líneas".

A partir de .NET 10 y C# 14, se puede ejecutar un único archivo .cs **sin proyecto**. La feature se llama file-based apps.

Hola mundo en un solo archivo

Crear un archivo hola.cs con este contenido:

```
csharp
Console.WriteLine("¡Hola desde C# 14 y .NET 10!");
```

Ejecutarlo:

```
bash
dotnet run hola.cs
```

No hay .csproj. No hay bin/. No hay obj/. Solo un archivo y un comando.

Comparación con la forma tradicional

La forma tradicional sigue existiendo y es la que vamos a usar para proyectos más grandes:

```
bash
mkdir saludo
cd saludo
dotnet new console
dotnet run
```

Eso genera una carpeta completa con Program.cs, saludo.csproj, bin/ y obj/. Es lo correcto cuando el programa crece, tiene varios archivos, dependencias o tests.

Para arrancar, sin embargo, los archivos sueltos son mucho más cómodos.

Directivas dentro del archivo

Cuando el script necesita paquetes o configuraciones extra, se declaran al inicio del archivo con directivas #::

```
csharp
#:package Humanizer@2.14.1

using Humanizer;

Console.WriteLine("dotnet".Humanize(LetterCasing.Title));
```

Esto reemplaza al `dotnet add package` y al `.csproj` para casos simples.

Ejemplo completo: lista de estudiantes con notas

Veamos un programa más sustancial, también en un único archivo. Calcula el promedio individual de cada estudiante y el promedio general del curso.

Crear `estudiantes.cs`:

```
csharp
using System;
using System.Collections.Generic;
using System.Linq;

List<Estudiante> estudiantes = new() {
    new Estudiante("Ana", [8, 9, 7]),
    new Estudiante("Bruno", [6, 5, 7]),
    new Estudiante("Carla", [10, 9, 10])
};

Console.WriteLine("Promedio por estudiante:");
foreach (Estudiante e in estudiantes) {
    Console.WriteLine($" {e.Nombre,-8} {e.Promedio():F2}");
}

double promedioCurso = estudiantes.Average(e => e.Promedio());
Console.WriteLine();
Console.WriteLine($"Promedio del curso: {promedioCurso:F2}");

public class Estudiante {
    public string Nombre { get; }
    public List<int> Notas { get; }

    public Estudiante(string nombre, List<int> notas) {
        this.Nombre = nombre;
        this.Notas = notas;
    }

    public double Promedio() {
        return this.Notas.Average();
    }
}
```

Ejecutarlo:

```
bash
dotnet run estudiantes.cs
```

Salida esperada:

texto

Promedio por estudiante:

Ana 8.00

Bruno 6.00

Carla 9.67

Promedio del curso: 7.89

Qué hay en ese programa

Este ejemplo, en pocas líneas, muestra varias cosas que vamos a estudiar con detalle a lo largo del curso:

- **Top-level statements:** el código que arranca con `List<Estudiante> ...` no está dentro de un `Main` ni de una clase. Es el punto de entrada del programa.
- **Definición de clase** con propiedades de solo lectura y constructor (`Estudiante`).
- **Convención** `this.campo` para distinguir parámetros del constructor de los miembros del objeto.
- **Colecciones genéricas:** `List<Estudiante>`, `List<int>`.
- **Expresión de colección** `[...]: [8, 9, 7]` para crear una lista.
- **LINQ:** el método `Average` sobre la colección.
- **Interpolación de strings con formato:** `$"{...:F2}"` para mostrar dos decimales.
- **Todo en un único archivo**, sin proyecto.

A medida que el programa crezca y necesitemos varios archivos, tests, dependencias o publicación, vamos a migrar a la forma tradicional con proyecto `.csproj`. Pero el día a día del aula, sobre todo al principio, se resuelve perfectamente con archivos sueltos.

Resumen

Concepto	Idea clave
.NET	Plataforma de desarrollo: runtime + SDK + BCL + paquetes + frameworks
Runtime	Ejecuta el código intermedio (IL) y administra la memoria
SDK	Compilador, CLI, plantillas y herramientas para desarrollar
BCL	Biblioteca estándar de la plataforma: System.*
NuGet	Gestor oficial de paquetes
Biblioteca	Conjunto de clases que llamamos cuando las necesitamos
Framework	Estructura que llama a nuestro código en los puntos que él define
ASP.NET Core	Framework para web, APIs y backend
Entity Framework Core	ORM para acceso a datos
.NET MAUI	Aplicaciones multiplataforma con interfaz gráfica
CLI dotnet	Puerta de entrada a todo: crear, compilar, ejecutar, publicar
file-based apps	Archivos .cs sueltos ejecutables sin proyecto, desde .NET 10 / C# 14

CAPITULO 02

Variables y manejo de memoria en C#

Una variable es un nombre asociado a un valor que puede cambiar durante la ejecución.

```
csharp
int contador = 0;
contador = contador + 1;
contador += 1;
```

$x = x + 1$ no es una igualdad matemática: es una asignación. Significa "tomar el valor actual de x, sumarle 1 y guardar el resultado en x".

var e inferencia de tipos

El tipo se puede declarar de forma explícita o dejar que el compilador lo infiera:

```
csharp
int edad = 20;
var edad = 20; // también int
var precio = 19.99; // double
var alumnos = new List<string>();
```

var no es tipado dinámico: el tipo sigue siendo estático y se resuelve en compilación. Solo evita repetirlo cuando ya está claro a partir del valor inicial.

Conviene usar **var** cuando el tipo es evidente. Conviene usar tipo explícito cuando el valor inicial no lo deja claro:

```
csharp
var dato = ObtenerResultado(); // ¿qué devuelve esto?
```

const y readonly

Ambos expresan inmutabilidad, pero en momentos distintos.

const es para valores conocidos en tiempo de compilación:

```
csharp
const double Pi = 3.14159;
```

readonly es para campos que se fijan en la declaración o en el constructor:


```
csharp
class Persona {
    public readonly DateTime FechaCreacion;

    public Persona() {
        this.FechaCreacion = DateTime.Now;
    }
}
```

`DateTime.Now` no se conoce en compilación, así que no podría ser `const`.

Tipos por valor y por referencia

La distinción central: ¿una variable **contiene** el valor, o **referencia** un lugar donde el valor está guardado?

Tipos por valor copian el dato:

```
csharp
int a = 10;
int b = a;
b = 20;
// a sigue valiendo 10
```

Tipos por referencia copian la referencia, no el objeto:

```
csharp
int[] numeros = { 1, 2, 3 };
int[] copia = numeros;
copia[0] = 99;

Console.WriteLine(numeros[0]); // 99
```

`copia` y `numeros` apuntan al mismo arreglo. Para obtener una copia independiente hay que crearla explícitamente:

```
csharp
int[] copia = (int[])numeros.Clone();
```

Esta distinción aparece en todo: paso de parámetros, asignaciones, comparaciones, colecciones.

Stack y heap

Como primera aproximación:

- la **stack** se asocia con variables locales y llamadas a métodos: vida corta, administración automática.
- el **heap** se asocia con objetos creados dinámicamente: lo administra el recolector de basura.

La realidad del runtime es más matizada que "valores en stack, objetos en heap". Lo importante no es memorizar internals del CLR, sino entender que no toda la memoria se comporta igual.

Alcance y tiempo de vida

El **alcance** es dónde se puede usar la variable. El **tiempo de vida** es cuánto existe durante la ejecución.

Una variable declarada en un bloque solo es visible dentro de ese bloque:

```
csharp
void Metodo() {
    int x = 10;
    if (true) {
        int y = 20;
        Console.WriteLine(x); // ok
    }
    Console.WriteLine(y); // error: y no existe acá
}
```

Una variable local vive mientras se ejecuta su bloque. Una variable estática vive durante toda la ejecución del programa.

Sombreado

Una variable de alcance menor puede ocultar a otra de alcance mayor:

```
csharp
int x = 10;

void Metodo() {
    int x = 20; // oculta a la externa
    Console.WriteLine(x); // 20
}
```

"Globales" en C#

C# no tiene variables globales, pero un campo estático se puede usar de forma equivalente:

```
csharp
static class Global {
    public static int Contador = 0;
}

Global.Contador++;
```

Namespaces y clases estáticas

Un namespace agrupa tipos relacionados y evita conflictos de nombres:

```
csharp
namespace MiAplicacion.Utilidades;
```

Una clase estática agrupa miembros utilitarios que no dependen de una instancia:

csharp

```
static class ConstantesDelSistema {  
    public const int EdadMinima = 18;  
    public static readonly string Bienvenida = "Bienvenido";  
}
```

Una clase estática gigantesca con todo adentro suele ser mala idea: organizar no es amontonar prolijamente.

Resumen

Necesidad	Herramienta
Declarar sin repetir el tipo	var
Valor fijo conocido en compilación	const
Valor fijado al construir el objeto	readonly
Copiar el dato	tipo por valor
Copiar la referencia al objeto	tipo por referencia
Saber dónde se usa una variable	alcance
Saber cuánto existe una variable	tiempo de vida
Agrupar tipos relacionados	namespace
Agrupar utilidades sin instancia	static class

CAPITULO 03

Tipos de datos y operadores en C#

Tipos primitivos principales

Tipo	Qué representa	Ejemplo
int	Número entero	20, -5, 0
double	Número con decimales	3.14, 1.75
bool	Valor lógico	true, false
char	Un solo carácter	'A', 'ñ'
string	Texto	"Hola"

Literales: cómo reconocer el tipo

Si el literal...	Tipo
entre comillas dobles	string
entre comillas simples	char
es true o false	bool
solo dígitos enteros	int
tiene punto decimal	double
termina en m	decimal
termina en f	float
termina en L	long
empieza con 0x	entero hexadecimal
empieza con 0b	entero binario

```
csharp
long poblacion = 5_000_000_000L;
float temp     = 25.5f;
decimal saldo  = 1500.25m;
int color      = 0xFF_EC_DE;
int flags      = 0b1111_0000;
int millones   = 1_000_000; // _ es solo separador visual
```

Enteros

Tipo	Tamaño	Rango
byte	1 byte	0 a 255
short	2 bytes	-32.768 a 32.767
int	4 bytes	±2.1 mil millones
long	8 bytes	±9.2×10 ¹⁸

En la práctica cotidiana se usa casi siempre `int`.

```
csharp
Console.WriteLine(int.MaxValue); // 2.147.483.647
Console.WriteLine(int.MinValue); // -2.147.483.648

Math.Abs(-7);           // 7
Math.Max(3, 5);         // 5
Math.Pow(2, 10);        // 1024 (devuelve double)

int.Parse("42");         // lanza excepción si falla
int.TryParse("42", out int n); // versión segura, devuelve bool
```

Punto flotante y decimal

Tipo	Tamaño	Precisión	Uso
float	4 bytes	~7 dígitos	Gráficos, cuando el tamaño importa
double	8 bytes	~15 dígitos	Cálculo general
decimal	16 bytes	28-29 dígitos	Dinero, precisión exacta

`float` y `double` usan IEEE 754 y no representan exactamente todos los decimales:

```
csharp
Console.WriteLine(0.1 + 0.2); // 0.30000000000000004
```

`decimal` usa representación interna decimal y evita ese problema:

```
csharp
decimal precio = 19.99m;
decimal iva    = precio * 0.21m; // correcto para dinero
```

Métodos útiles:

```
csharp
Math.Abs(-3.75); // 3.75
Math.Round(3.75, 1); // 3.8
Math.Ceiling(3.2); // 4
Math.Floor(3.9); // 3
Math.Truncate(3.9); // 3 (descarta decimales, no redondea)
```

Booleanos

Solo puede ser true o false. Surge de comparaciones u operaciones lógicas:

```
csharp
bool esMayor = 10 > 5; // true

bool y = true && false; // false - AND
bool o = true || false; // true - OR
bool no = !true;        // false - NOT
```

&& y || tienen **cortocircuito**: si el primer operando determina el resultado, el segundo no se evalúa:

```
csharp
// Si x es 0, (10 / x) nunca se evalúa:
bool ok = (x != 0) && (10 / x > 1);
```

Caracteres (char)

Un solo carácter Unicode entre comillas simples. Distinto de string:

```
csharp
char letra = 'A';
string s = "A"; // tipo distinto
```

Secuencias de escape comunes: '\n' (nueva línea), '\t' (tabulación), '\\' (barra), '\'' (comilla simple).

Operadores

Aritméticos

```
csharp
10 + 3 // 13
10 - 3 // 7
10 * 3 // 30
10 / 3 // 3 - división entera (descarta decimales)
10 % 3 // 1 - resto

// Para obtener decimales, al menos un operando debe serlo:
(double)10 / 3 // 3.333...
```

Incremento y decremento

```
csharp
int a = 5;
int b = a++; // b = 5, a = 6 (post: usa el valor actual, luego incrementa)
int c = ++a; // c = 7, a = 7 (pre: incrementa primero, luego usa)
```

Asignación compuesta

```
csharp
x += 5;    // x = x + 5
x -= 3;    // x = x - 3
x *= 2;    // x = x * 2
x /= 4;    // x = x / 4
x %= 4;    // x = x % 4
```

Relacionales

```
csharp
a == b    // igual
a != b    // distinto
a > b     // mayor
a < b     // menor
a >= b    // mayor o igual
a <= b    // menor o igual
```

`==` sobre tipos por valor compara contenido. Sobre tipos por referencia compara identidad (salvo que el tipo lo sobrescriba, como `string` o `record`).

Bit a bit

```
csharp
int a = 0b1100; // 12
int b = 0b1010; // 10

a & b    // 0b1000 = 8 - AND bit a bit
a | b    // 0b1110 = 14 - OR bit a bit
a ^ b    // 0b0110 = 6 - XOR
~a       // invierte todos los bits
a << 1   // desplaza a la izquierda (× 2)
a >> 1   // desplaza a la derecha (÷ 2)
```

Tipos por valor y por referencia

Al copiar una variable de tipo **valor**, se copia el dato:

```
csharp
int a = 10;
int b = a;
b = 20;
Console.WriteLine(a); // 10 - no cambió
```

Al copiar una variable de tipo **referencia**, se copia la dirección (dos variables apuntan al mismo objeto):

```
csharp
var lista1 = new List<int> { 1, 2, 3 };
var lista2 = lista1;
lista2.Add(4);
Console.WriteLine(lista1.Count); // 4
```

`string` es un tipo por referencia pero se comporta como valor gracias a su inmutabilidad.

Operadores para null

```
csharp
string? nombre = null;

string texto = nombre ?? "Sin nombre"; // valor por defecto si null
nombre ??= "Valor default";           // asigna solo si era null
int? largo = nombre?.Length;          // accede solo si no es null
```

Conversión de tipos

```
csharp
// Implícita (sin pérdida de información)
int i = 10;
double d = i; // ok

// Explícita (puede haber pérdida)
double precio = 19.99;
int entero = (int)precio; // 19 - sin redondeo

// Desde texto
int.Parse("42"); // lanza excepción si falla
int.TryParse("42", out int n); // devuelve bool

// Entre referencias (devuelve null si falla, no excepción)
string? texto = dato as string;
```

Resumen

Necesidad	Herramienta
Enteros	int, long, short, byte
Decimales de uso general	double
Decimales para dinero	decimal
Verdadero/falso	bool
Un carácter	char
Ausencia de valor	null, int?, string?
Conversión segura desde texto	TryParse
Conversión segura de referencias	as

CAPITULO 04

Enumeraciones en C#

Un **enum** define un conjunto finito de constantes con nombre. Reemplaza números o cadenas mágicas por valores con significado explícito:

```
csharp
// Sin enum - ¿qué significa el 2?
if (orden.Estado == 2) { ... }

// Con enum - el código se explica solo
if (orden.Estado == EstadoOrden.Enviado) { ... }
```

Los enums son **tipos por valor** basados en un entero (int por defecto).

Declaración

```
csharp
// Valores asignados automáticamente: 0, 1, 2...
public enum DiaSemana { Lunes, Martes, Miercoles, Jueves, Viernes, Sabado, Domingo }

// Valores explícitos (no necesitan ser consecutivos)
public enum EstadoOrden {
    Pendiente = 0,
    Confirmado = 1,
    Enviado = 2,
    Cancelado = 99
}

// Tipo subyacente distinto de int
public enum CodigoError : short { Ninguno = 0, NoEncontrado = 1001 }
```

Uso básico

```
csharp
DiaSemana hoy = DiaSemana.Miercoles;

if (hoy == DiaSemana.Sabado || hoy == DiaSemana.Domingo)
    Console.WriteLine("Fin de semana");

string tipo = hoy switch {
    DiaSemana.Sabado or DiaSemana.Domingo => "Fin de semana",
    DiaSemana.Viernes                      => "¡Viernes!",
    -                                     => "Día hábil"
};
```

Conversión

```
csharp
int valor = (int)EstadoOrden.Enviado; // enum → int: 2
var estado = (EstadoOrden)2; // int → enum: Enviado

string nombre = DiaSemana.Lunes.ToString(); // "Lunes"

// string → enum seguro
if (Enum.TryParse<DiaSemana>("Viernes", ignoreCase: true, out DiaSemana d))
    Console.WriteLine(d);

// Verificar si un valor es válido (útil para datos externos)
bool valido = Enum.IsDefined<EstadoOrden>((EstadoOrden)50); // false
```

Reflexión

```
csharp
DiaSemana[] todos = Enum.GetValues<DiaSemana>();
string[] nombres = Enum.GetNames<DiaSemana>();

foreach (DiaSemana dia in Enum.GetValues<DiaSemana>())
    Console.WriteLine($"{(int)dia}: {dia}");
```

Flags — combinaciones de bits

Cuando un valor puede ser **combinación** de varios miembros, se usa [Flags]. Cada miembro debe ser potencia de 2:

```
csharp
[Flags]
public enum Permiso {
    Ninguno = 0b0000,
    Leer = 0b0001,
    Escribir = 0b0010,
    Ejecutar = 0b0100,
    Eliminar = 0b1000,
    Completo = Leer | Escribir | Ejecutar | Eliminar
}

Permiso editor = Permiso.Leer | Permiso.Escribir;

editor.HasFlag(Permiso.Escribir); // true
editor.HasFlag(Permiso.Eliminar); // false

editor |= Permiso.Ejecutar; // agregar
editor &= ~Permiso.Escribir; // quitar

Console.WriteLine(editor); // "Leer, Ejecutar"
```

Serialización JSON

```
csharp
// Por defecto: número
// {"Documento":1}

// Como string (más legible):
var opciones = new JsonSerializerOptions {
    Converters = { new JsonStringEnumConverter() }
};
// {"Documento":"Pasaporte"}
```

Buenas prácticas

Siempre definir un valor 0 con nombre (Ninguno o equivalente), ya que es el valor por defecto al hacer default:

```
csharp
// MAL
public enum Color { Rojo = 1, Verde = 2 }
Color c = default; // 0 - no está definido

// BIEN
public enum Color { Ninguno = 0, Rojo = 1, Verde = 2 }
Color c = default; // Color.Ninguno
```

Validar siempre datos externos antes de hacer cast:

```
csharp
if ( !Enum.IsDefined<EstadoOrden>((EstadoOrden)valorExterno))
    throw new ArgumentOutOfRangeException(...);
```

No usar enums para conjuntos que crecen con el tiempo (países, monedas): preferir constantes de string.

Resumen

Necesidad	Herramienta
Conjunto finito opciones con nombre	enum
Combinación de varias opciones	[Flags] + potencias de 2
Texto amigable por valor	[Description] + método de extensión
Iterar todos los valores	Enum.GetValues<T>()
Parsear string de forma segura	Enum.TryParse<T>() + Enum.IsDefined()
Serializar como texto en JSON	JsonStringEnumConverter

CAPITULO 05

Strings en C#

`string` representa texto. Es un tipo de referencia con comportamiento especial: `==` compara contenido (no identidad) y los strings son **inmutables**.

```
csharp
string nombre = "Ada";
```

Formas de escribir un string

```
csharp
// Concatenación
string a = "Hola, " + nombre;

// Interpolación: la forma más legible
string b = $"{nombre} tiene {edad} años, el doble es {edad * 2}";

// Verbatim: sin escapar barras, con saltos de línea literales
string c = @"C:\Users\Ada\Documentos";
string d = @"Línea 1
Línea 2";

// Combinada: verbatim + interpolación
string e = $"{@"C:\Users\Ada\{carpeta}"}";
```

Raw string literals ("\"")

Permiten escribir JSON, código u otro texto con comillas y saltos de línea sin escapar nada:

```
csharp
string json = "\"\"
{
  \"nombre\": \"Ada\",
  \"edad\": 20
}\"\";

```

Con `$$\"\"\"` se interpolan valores usando `{{ y }}`. Las llaves simples quedan como texto literal:

```
csharp
string json = $$\"\"\"
{
  \"nombre\": \"{{nombre}}\",
  \"edad\": {{edad}}
}\"\";

```

Inmutabilidad

Una vez creado, un string no se modifica. Cuando parece que se cambia, en realidad se crea uno nuevo:

```
csharp
string nombre = "Ana";
nombre = nombre + " María"; // crea un string nuevo, "Ana" sigue en memoria
```

StringBuilder

En bucles con muchas concatenaciones, cada + genera un objeto. Para esos casos StringBuilder trabaja sobre un buffer y solo materializa el string al terminar:

```
csharp
var sb = new StringBuilder();

for (int i = 0; i < 1000; i++) {
    sb.Append($"Línea {i}\n");
}

string resultado = sb.ToString();
```

Para uso cotidiano la concatenación e interpolación alcanzan. StringBuilder es una optimización para construcción intensiva.

Conversión automática

Cuando se necesita texto, C# llama a ToString() (heredado de object):

```
csharp
int numero = 42;
string mensaje = "El número es: " + numero; // llama numero.ToString()
```

En clases propias se sobrescribe para definir la representación textual:

```
csharp
class Punto {
    public int X { get; set; }
    public int Y { get; set; }

    public override string ToString() => $"({this.X}, {this.Y})";
}

var p = new Punto { X = 3, Y = 7 };
Console.WriteLine($"El punto es {p}"); // El punto es (3, 7)
```

Comparación

== compara contenido. Para ordenar, CompareTo devuelve negativo, cero o positivo:

```
csharp
"banana".CompareTo("manzana"); // negativo: "banana" va antes
```

`string.Compare` y `Equals` aceptan un `StringComparison` para controlar la comparación:

```
csharp
frase.Equals("hola", StringComparison.OrdinalIgnoreCase);
frase.Contains("ESTO", StringComparison.OrdinalIgnoreCase);
frase.StartsWith("ho", StringComparison.OrdinalIgnoreCase);
```

Las dos opciones más usadas:

- `OrdinalIgnoreCase`: para comparaciones técnicas (nombres de archivo, claves, comandos).
- `CurrentCultureIgnoreCase`: para texto visible al usuario, donde importan las reglas del idioma.

```
csharp
var config = new Dictionary<string, string>(StringComparer.OrdinalIgnoreCase);
config["Nombre"] = "Ada";
config["nombre"]; // "Ada"
```

Null, vacío y blanco

```
csharp
string? nulo = null;
string vacio = "";
string blanco = " ";

string.IsNullOrEmpty(nulo); // true
string.IsNullOrEmpty(vacio); // true
string.IsNullOrWhiteSpace(blanco); // true
```

`IsNullOrWhiteSpace` suele ser más útil: un string de solo espacios casi siempre representa "sin contenido".

Métodos comunes

Método	Qué hace
Length	Cantidad de unidades char
ToUpper() / ToLower()	Cambia mayúsculas/minúsculas
Trim() / TrimStart() / TrimEnd()	Elimina espacios en los extremos
Contains(sub)	Verifica si contiene un fragmento
StartsWith(sub) / EndsWith(sub)	Verifica inicio o final
IndexOf(sub)	Posición de la primera aparición; -1 si no existe
Replace(viejo, nuevo)	Reemplaza todas las apariciones
Split(sep)	Divide en array usando un separador
string.Join(sep, array)	Une un array en un string
Substring(inicio, largo) o s[i..j]	Extrae una parte
PadLeft(n, c) / PadRight(n, c)	Rellena hasta longitud n
string.IsNullOrEmpty(s)	true si es null o ""
string.IsNullOrWhiteSpace(s)	true si es null, "" o solo espacios

```
csharp
"informe_final.pdf".EndsWith(".pdf"); // true
"hola mundo".Replace("mundo", "Ada"); // "hola Ada"
"Ana,Luis,Pedro".Split(','); // { "Ana", "Luis", "Pedro" }
string.Join(" ", new[] { "a", "b", "c" }); // "a, b, c"
"42".PadLeft(6, '0'); // "000042"
```

Unicode y emojis

char en C# es 16 bits. Los emojis y símbolos fuera del rango básico usan dos char (par sustituto):

```
csharp
string emoji = "👍";
emoji.Length; // 2 - no 1
```

Para iterar correctamente usar Rune:

```
csharp
foreach (Rune r in "A👍B".EnumerateRunes())
    Console.Write($"[{r}] ");
// [A] [👍] [B]
```

Para contar caracteres visibles usar StringInfo:

```
csharp
new StringInfo("Hola 👍").LengthInTextElements; // 6
```

Resumen

Necesidad	Herramienta
Texto con valores intercalados	interpolación <code>\$"..."</code>
Multilínea o rutas Windows	verbatim <code>@"..."</code>
JSON / código con comillas	raw literal <code>"""..."""</code>
Concatenar mucho en bucle	<code>StringBuilder</code>
Igualdad ignorando mayúsculas	<code>StringComparison.OrdinalIgnoreCase</code>
Comparar respetando el idioma	<code>StringComparison.CurrentCultureIgnoreCase</code>
Diccionario con claves insensibles	<code>StringComparer.OrdinalIgnoreCase</code>
Detectar "sin contenido real"	<code>string.IsNullOrEmpty</code>
Iterar emojis correctamente	<code>Rune / EnumerateRunes()</code>
Contar caracteres visibles	<code>StringInfo.LengthInTextElements</code>

CAPITULO 06

Tipos compuestos y colecciones en C#

Tipo	Para qué	Clave
T[]	Secuencia de tamaño fijo	Acceso O(1) por índice
List<T>	Secuencia dinámica	Agrega y quita elementos
Dictionary<K,V>	Mapa clave → valor	Búsqueda O(1) por clave
HashSet<T>	Elementos únicos	Pertenencia eficiente
Tupla (T1, T2)	Agrupar valores temporalmente	Liviana, sin clase
record	Dato con identidad semántica	Igualdad por valor

Arrays

Tamaño fijo, índices desde 0, tipo homogéneo:

```
csharp
int[] notas = [7, 3, 9, 5, 8];

int primero = notas[0]; // 7
int ultimo = notas[^1]; // 8
var primeros = notas[0..3]; // [7, 3, 9]

foreach (int n in notas) Console.WriteLine(n);

Array.Sort(notas); // ordena
Array.Reverse(notas); // invierte
Array.IndexOf(notas, 7); // posición
Array.Exists(notas, n => n > 8); // true/false
var copia = (int[])notas.Clone(); // copia independiente
```

Arrays multidimensionales:

```
csharp
int[,] matriz = { { 1, 2 }, { 3, 4 } };
Console.WriteLine(matriz[1, 0]); // 3

int[][] jagged = new int[3][]; // cada fila puede tener distinto largo
```

Métodos comunes

Método	Qué hace
Length	Cantidad de elementos
arr[i] / arr[^1]	Acceso por índice (desde el final ^)
arr[a..b]	Sub-array (rango)
Array.Sort(arr)	Ordena in situ
Array.Reverse(arr)	Invierte el orden
Array.IndexOf(arr, x)	Posición de x; -1 si no está
Array.Exists(arr, pred)	true si algún elemento cumple
Array.Find(arr, pred)	Primer elemento que cumple
Array.FindAll(arr, pred)	Todos los que cumplen
Array.Fill(arr, v)	Rellena con un valor
arr.Clone()	Copia superficial independiente

List<T>

Dinámica, usa Count (no Length), expone métodos de instancia:

```
csharp
List<string> ciudades = ["Tucumán", "Córdoba", "Rosario"];

ciudades.Add("Mendoza");
ciudades.Insert(0, "Buenos Aires");
ciudades.AddRange(["Salta", "Jujuy"]);

ciudades.Remove("Córdoba");
ciudades.RemoveAt(0);
ciudades.RemoveAll(c => c.Length > 7);

bool existe = ciudades.Contains("Salta");
int pos     = ciudades.IndexOf("Salta");

ciudades.Sort();
ciudades.Reverse();
```

Con LINQ (se cubre en detalle en su propio apunte):

```
csharp
var resultado = ciudades
    .Where(c => c.Length > 5)
    .OrderBy(c => c)
    .ToList();
```

Métodos comunes

Método	Qué hace
Count	Cantidad de elementos
Add(x)	Agrega al final
Insert(i, x)	Inserta en la posición i
AddRange(col)	Agrega varios elementos
Remove(x)	Quita la primera aparición
RemoveAt(i)	Quita por índice
RemoveAll(pred)	Quita todos los que cumplen
Contains(x)	Verifica pertenencia
IndexOf(x)	Posición; -1 si no está
Find(pred) / FindAll(pred)	Busca el primero / todos
Sort() / Reverse()	Ordena / invierte
Clear()	Vacía la lista

Dictionary<K, V>

Pares clave → valor únicos. Acceso O(1):

```
csharp
var telefonos = new Dictionary<string, string>
{
    ["Ana"] = "381-123-4567",
    ["Carlos"] = "381-987-6543"
};

telefonos.Add("Diego", "381-111-2222"); // lanza si ya existe
telefonos["Ana"] = "381-999-8888"; // agrega o actualiza
telefonos.TryAdd("Ana", "otro"); // no hace nada si ya existe

// Acceso seguro (recomendado)
if (telefonos.TryGetValue("Pedro", out var tel))
    Console.WriteLine(tel);

var fallback = telefonos.GetValueOrDefault("Pedro", "Sin número");

telefonos.Remove("Carlos");

foreach (var (nombre, numero) in telefonos)
    Console.WriteLine($"{nombre}: {numero}");
```

Métodos comunes

Método	Qué hace
Count	Cantidad de pares
dict[k]	Lee o asigna (agrega o actualiza)
Add(k, v)	Agrega; lanza si la clave existe
TryAdd(k, v)	Agrega solo si no existe
TryGetValue(k, out v)	Acceso seguro sin excepción
GetValueOrDefault(k, def)	Valor asociado o un default
ContainsKey(k)	Verifica si existe la clave
Remove(k)	Quita el par
Keys / Values	Colección de claves / valores
Clear()	Vacía el diccionario

HashSet<T>

Elementos únicos, sin orden garantizado, Contains en O(1):

```
csharp
var frutas = new HashSet<string> { "manzana", "banana", "naranja" };
frutas.Add("banana"); // ignorado - ya existe

frutas.Contains("banana"); // true

var a = new HashSet<int> { 1, 2, 3, 4 };
var b = new HashSet<int> { 3, 4, 5, 6 };

a.UnionWith(b);      // { 1, 2, 3, 4, 5, 6 }
a.IntersectWith(b);  // { 3, 4 }
a.ExceptWith(b);     // { 1, 2 }
```

Métodos comunes

Método	Qué hace
Count	Cantidad de elementos
Add(x)	Agrega; <code>false</code> si ya existía
Contains(x)	Pertenencia en O(1)
Remove(x)	Quita el elemento
UnionWith(b)	Unión (agrega los de b)
IntersectWith(b)	Intersección (deja los comunes)
ExceptWith(b)	Diferencia (quita los de b)
SymmetricExceptWith(b)	Diferencia simétrica
IsSubsetOf(b) / IsSupersetOf(b)	Subconjunto / superconjunto
Overlaps(b)	<code>true</code> si comparten algún elemento

Tuplas

Agrupan valores sin definir una clase. Ideales para retornar múltiples valores:

```
csharp
var persona = (Nombre: "Ada", Edad: 20);
Console.WriteLine(persona.Nombre); // Ada

static (double Min, double Max) Rango(int[] datos) =>
    (datos.Min(), datos.Max());

var (min, max) = Rango([3, 7, 1, 9]);

// Descartar campo con _
var (_, soloMax) = Rango([3, 7, 1, 9]);
```

Operaciones comunes

Operación	Qué hace
t.Nombre	Acceso por nombre de campo
t.Item1, t.Item2	Acceso posicional
var (a, b) = t	Deconstrucción en variables
var (_, b) = t	Descarta un campo con <code>_</code>
t1 == t2	Igualdad por valor (campo a campo)

Records

Tipo especial para datos inmutables con igualdad por valor. El compilador genera constructor, Equals, GetHashCode, ToString y deconstrucción:

```
csharp
record Persona(string Nombre, int Edad);

var r1 = new Persona("Ana", 25);
var r2 = new Persona("Ana", 25);

Console.WriteLine(r1 == r2); // true (igualdad por valor)
Console.WriteLine(r1);       // Persona { Nombre = Ana, Edad = 25 }

// Copiar con cambios (expresión with)
var r3 = r1 with { Edad = 26 };
Console.WriteLine(r1.Edad); // 25 - no se modificó
Console.WriteLine(r3.Edad); // 26

// Deconstrucción
var (nombre, edad) = r1;
```

Records con herencia + pattern matching:

```
csharp
abstract record Figura;
record Circulo(double Radio) : Figura;
record Rectangulo(double Ancho, double Alto) : Figura;

double Area(Figura f) => f switch {
    Circulo { Radio: var r } => Math.PI * r * r,
    Rectangulo { Ancho: var a, Alto: var h } => a * h,
    _ => throw new ArgumentException()
};
```

Operaciones comunes

Operación	Qué hace
<code>r1 == r2</code>	Igualdad por valor
<code>r with { Prop = x }</code>	Copia con cambios (no muta el original)
<code>ToString()</code>	Representación legible (autogenerada)
<code>GetHashCode()</code>	Hash por valor (autogenerado)
<code>var (a, b) = r</code>	Deconstrucción (autogenerada)
<code>record struct</code>	Variante de tipo por valor

Resumen

Necesidad	Herramienta
Secuencia de tamaño fijo, acceso por índice	T[]
Secuencia que crece y se achica	List<T>
Buscar valores por una clave	Dictionary<K, V>
Evitar duplicados / operaciones de conjunto	HashSet<T>
Retornar varios valores sin crear una clase	Tupla (T1, T2)
Dato con igualdad por valor que viaja entre capas	record
Proyección temporal dentro de LINQ	tipo anónimo

CAPITULO 07

Clases y objetos en C#

Una **clase** define el estado (datos) y el comportamiento (métodos) de un tipo. Un **objeto** es una instancia de esa clase creada con new.

Sintaxis básica

```
csharp
public class Persona {
    private string nombre;
    private int edad;

    public Persona(string nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public string Nombre => this.nombre;
    public string Saludar() => $"Hola, soy {this.nombre} y tengo {this.edad} años.";
}

var p = new Persona("Ana", 25);
Console.WriteLine(p.Saludar());
```

Constructores

```
csharp
// Constructor delegado: un constructor llama a otro con :this(...)
public class Conexion {
    public string Host { get; }
    public int Puerto { get; }
    public bool Segura { get; }

    public Conexion(string host, int puerto, bool segura) {
        Host = host; Puerto = puerto; Segura = segura;
    }
    public Conexion(string host) : this(host, 443, true) { }
    public Conexion() : this("localhost") { }
}

// Primary constructor (C# 12+) - forma más compacta
public class Punto(double x, double y) {
    public double X => x;
    public double Y => y;
    public double Distancia => Math.Sqrt(x * x + y * y);
}

// Inicializador de objeto - asigna propiedades post-construcción
var e = new Empleado { Nombre = "Carlos", Departamento = "IT", Salario = 95_000m };
```


Propiedades

Sintaxis	Acceso	Cuándo usarla
{ get; set; }	Libre	Estado mutable
{ get; private set; }	Escritura interna	Estado con control
{ get; }	Solo lectura	Se asigna en constructor
{ get; init; }	Solo en new {}	DTOs, inicializadores
=> expresion	Solo lectura calculada	Propiedades derivadas
required { get; init; }	Obligatoria en new {}	Campos que no pueden omitirse

```
csharp
public class Artículo {
    public required string Nombre { get; init; }
    public decimal Precio { get; set; }
    public int Stock { get; private set; }
    public bool Disponible => Stock > 0; // calculada
}
```

Métodos y sobrecarga

```
csharp
public class Logger {
    public void Log(string mensaje) => Console.WriteLine($"[INFO] {mensaje}");
    public void Log(string mensaje, Exception ex) => Console.WriteLine($"[ERROR] {mensaje}: {ex.Message}");
    public void Log(string mensaje, string nivel) => Console.WriteLine($"[{nivel}] {mensaje}");
}
```

La sobrecarga se resuelve en **compilación**. Es distinta del `override`, que se resuelve en ejecución.

Miembros estáticos

Pertenecen a la clase, no a las instancias:

```
csharp
public class Contador {
    private static int total = 0;
    public int Id { get; }

    public Contador() { total++; Id = total; }

    public static int Total => total;
}

Console.WriteLine(Contador.Total); // sin instanciar
```

Clases completamente estáticas (no se instancian):

csharp

```
public static class MathEx {  
    public static bool EsPrimo(int n) {  
        if (n < 2) return false;  
        for (int i = 2; i * i <= n; i++)  
            if (n % i == 0) return false;  
        return true;  
    }  
}
```

Herencia

Una clase derivada adquiere los miembros de su clase base (relación "es un"):

csharp

```
public class Animal {  
    public string Nombre { get; }  
    public Animal(string nombre) { Nombre = nombre; }  
    public virtual string HacerSonido() => "...";  
}  
  
public class Perro : Animal {  
    public string Raza { get; }  
    public Perro(string nombre, string raza) : base(nombre) { Raza = raza; }  
    public override string HacerSonido() => "Guau!";  
}
```

`base(...)` encadena con el constructor de la clase padre. `protected` hace que los miembros sean visibles en las subclases pero no fuera.

Polimorfismo

`virtual + override` = despacho dinámico: el runtime decide qué implementación ejecutar según el tipo real del objeto, no el tipo declarado de la variable:

csharp

```
List<Animal> animales = [new Perro("Rex", "Labrador"), new Gato("Mia")];  
foreach (var a in animales)  
    Console.WriteLine(a.HacerSonido()); // cada uno ejecuta su override
```

`new` oculta el miembro base pero **no es polimórfico** — preferir siempre `override`.

Para verificar el tipo en runtime:

csharp

```
if (animal is Perro perro) Console.WriteLine(perro.Raza);  
var gato = animal as Gato; // null si falla, no excepción
```

Clases abstractas

No se pueden instanciar. Definen estructura compartida y obligan a implementar ciertos métodos:

csharp

```

public abstract class Figura {
    public abstract double Area();
    public abstract double Perimetro();
    public virtual string Describir() => $"{GetType().Name}: área={Area():F2}";
}

public class Circulo(double radio) : Figura {
    public override double Area() => Math.PI * radio * radio;
    public override double Perimetro() => 2 * Math.PI * radio;
}

```

	Clase abstracta	Interfaz
Puede tener estado	Sí	No
Herencia múltiple	No	Sí
Expresa "es un"	Sí	No necesariamente

Inmutabilidad

Un tipo inmutable no permite modificar su estado después de construirse. Las "mutaciones" devuelven un nuevo objeto:

csharp

```

public sealed class Dinero {
    public decimal Monto { get; }
    public string Moneda { get; }

    public Dinero(decimal monto, string moneda) { Monto = monto; Moneda = moneda; }
    public Dinero Sumar(Dinero otro) => new(Monto + otro.Monto, Moneda);
    public static Dinero operator +(Dinero a, Dinero b) => a.Sumar(b);
}

```

sealed impide que se herede la clase.

class vs. record vs. struct

	class	record	struct
Tipo	Referencia	Referencia	Valor
Igualdad	Por referencia	Por valor	Por valor
Ideal para	Entidades con estado mutable	DTOs, datos inmutables	Datos pequeños y simples

csharp

```

record Persona(string Nombre, int Edad); // igualdad por valor, ToString, deconstrucción

var p1 = new Persona("Ana", 25);
var p2 = p1 with { Edad = 26 };        // copia con cambio
Console.WriteLine(p1 == p2);           // false

```

Métodos de extensión

Agregan funcionalidad a tipos existentes sin heredar:

```
csharp
public static class StringExtensions {
    public static bool EsEmail(this string s) => s.Contains('@') && s.Contains('.');
}

"ana@ejemplo.com".EsEmail(); // true
```

Resumen

Necesidad	Herramienta
Estado + comportamiento propios	class
Inicialización compacta	primary constructor
Propiedad obligatoria	required { get; init; }
Comportamiento sin instancia	static
Especializar un tipo	herencia + override
Variar comportamiento por tipo	polimorfismo (virtual/override)
Forzar implementación	abstract
Datos inmutables entre capas	record
Datos pequeños por valor	struct
Extender tipos externos	métodos de extensión

CAPITULO 08

Null y tipos anulables en C#

null representa la **ausencia de valor**. Mal manejo produce el error más frecuente en .NET: `NullReferenceException`.

Desde C# 8 el compilador distingue referencias que pueden ser null (`string?`) de las que no (`string`). En proyectos .NET modernos el modo nullable está activado por defecto.

Tipos por valor anulables (T?)

Los tipos por valor (`int`, `bool`, etc.) no aceptan null por defecto. Con `?` sí:

```
csharp
int? edad = null; // Nullable<int>
bool? confirmado = null; // útil para "sí / no / sin responder"

edad.HasValue; // false si null
edad.Value; // lanza excepción si null
edad.GetValueOrDefault(); // 0 si null
edad.GetValueOrDefault(-1); // -1 si null
```

Comparaciones con null siempre dan `false` (no lanza excepción):

```
csharp
int? a = null;
Console.WriteLine(a > 5); // false
Console.WriteLine(a < 5); // false
```

Nullable reference types (C# 8+)

```
csharp
string nombre = "Ana"; // nunca null – el compilador lo verifica
string? apellido = null; // puede ser null – declarado explícitamente
```

El compilador analiza el flujo y avisa si se accede a algo potencialmente null:

```
csharp
string? texto = ObtenerTexto();
texto.Length; // Warning: posible null

if (texto != null) {
    texto.Length; // OK – el compilador lo sabe
}
```

Operadores

Operador	Nombre	Uso
?.	Null-conditional	Acceder a miembro solo si no es null
??	Null-coalescing	Valor de fallback si es null
??=	Null-coalescing assignment	Asignar solo si la variable es null
!	Null-forgiving	Suprimir warning (usar con cuidado)

? . — acceso condicional

```
csharp
string? nombre = null;
int? largo = nombre?.Length; // null, sin excepción

// Encadenable
string? ciudad = pedido?.Cliente?.Direccion?.Ciudad;

// Con métodos e indexadores
string? upper = s?.ToUpper();
char? c = s?[0];

// Con delegados
notificar?.Invoke("mensaje");
```

?? — valor de fallback

```
csharp
string display = nombre ?? "Anónimo"; // "Anónimo" si null
int segura = edad ?? 0;

// Encadenable
string resultado = a ?? b ?? "default"; // usa el primero no-null
```

??= — asignar solo si null

```
csharp
string? cache = null;
cache ??= "valor"; // asigna
cache ??= "otro"; // no hace nada – ya tiene valor

// Inicialización perezosa
List<string>? _items = null;
public List<string> Items { get { _items ??= []; return _items; } }
```

! — suprimir warning

```
csharp
int largo = texto!.Length; // el compilador acepta, pero explota si texto es null
```

Usarlo solo cuando hay certeza real de que el valor no es null.

Pattern matching con null

```
csharp
if (nombre is null) { ... }
if (nombre is not null) { ... }

string respuesta = input switch {
    null => "Nada ingresado",
    "" => "Cadena vacía",
    { Length: > 20 } => "Demasiado largo",
    _ => $"Ingresaste: {input}"
};
```

Buenas prácticas

Preferir `is null` sobre `== null` (no puede ser sobrecargado):

```
csharp
if (objeto is null) { ... } // siempre compara la referencia real
if (objeto is not null) { ... }
```

Validar parámetros al inicio del método:

```
csharp
static void Procesar(string nombre) {
    ArgumentNullException.ThrowIfNull(nombre); // C# 10+
    // acá el compilador sabe que nombre no es null
}
```

Filtrar nulls de colecciones:

```
csharp
List<string?> nombres = ["Ana", null, "Carlos"];
List<string> validos = nombres.OfType<string>().ToList();
```

Resumen

¿Acceder a miembro que puede ser null?	→ <code>obj?.Miembro</code>
¿Valor de fallback si es null?	→ <code>valor ?? "default"</code>
¿Asignar solo si es null?	→ <code>variable ??= nuevoValor</code>
¿Invocar delegado de forma segura?	→ <code>delegado?.Invoke(args)</code>
¿Verificar si algo es null?	→ <code>valor is null / is not null</code>
¿Suprimir warning (con certeza)?	→ <code>valor!</code>

CAPITULO 09

Funciones, Delegados y Eventos en C#

Las funciones en C# también son **valores**: se pueden guardar en variables, pasar como argumentos y devolver como resultado.

Parámetros

```
csharp
// Por valor (default): se copia, el original no cambia
static void Duplicar(int n) { n *= 2; }

// ref: alias real – los cambios se ven afuera
static void Incrementar(ref int valor, int cantidad) => valor += cantidad;
int x = 10; Incrementar(ref x, 5); // x = 15

// out: retorno adicional – el método DEBE asignarlo
static bool TryParseFecha(string texto, out DateOnly fecha) {
    if (DateOnly.TryParse(texto, out fecha)) return true;
    fecha = default; return false;
}
if (TryParseFecha("2025-06-15", out var fecha))
    Console.WriteLine(fecha);

// in: referencia de solo lectura (evita copia sin permitir modificación)
static double Traza(in Matriz4x4 m) => m.Datos[0] + m.Datos[5];

// Opcionales y nombrados
static string Formatear(string texto, bool mayusculas = false, int max = 100) { ... }
Formatear("texto", mayusculas: true, max: 50);

// params: número variable de argumentos
static int Sumar(params int[] nums) => nums.Sum();
Sumar(1, 2, 3, 4, 5); // 15
```

Funciones locales

Se declaran dentro de otro método. Solo son visibles ahí y pueden capturar variables del contexto:

```
csharp
static List<int> FiltrarPares(IEnumerable<int> fuente, int minimo) {
    bool EsValido(int n) => n >= minimo && n % 2 == 0; // local
    return fuente.Where(EsValido).ToList();
}
```

Recursión

Requiere siempre un **caso base** para no recurrir infinitamente:


```
csharp
static long Factorial(int n) {
    if (n <= 1) return 1;           // caso base
    return n * Factorial(n - 1);   // llamada recursiva
}
```

Delegados

Un delegado es una variable que guarda una función. Representa una firma (tipos de parámetros y retorno):

```
csharp
delegate int Operacion(int a, int b);

Operacion op = (a, b) => a + b;
Console.WriteLine(op(3, 4)); // 7

op = (a, b) => a * b;
Console.WriteLine(op(3, 4)); // 12
```

Delegado multicast: puede apuntar a varios métodos con +=:

```
csharp
delegate void Notificacion(string msg);

Notificacion n = msg => Console.WriteLine($"[Email] {msg}");
n += msg => Console.WriteLine($"[SMS] {msg}");

n("Pedido confirmado"); // invoca los dos
n -= ...; // desuscribir
```

Func<>, Action<> y Predicate<>

No hace falta declarar delegados propios:

Tipo	Firma	Uso
Func<T, TResult>	Recibe T, retorna TResult	Transformar
Func<T1, T2, TResult>	Recibe T1 y T2, retorna TResult	Combinar
Action<T>	Recibe T, no retorna	Efecto lateral
Action	Sin parámetros, no retorna	Callback sin datos
Predicate<T>	Recibe T, retorna bool	Filtrar/verificar

```
csharp
Func<int, int, int> sumar = (a, b) => a + b;
Func<string, int> largo = s => s.Length;
Action<string> imprimir = Console.WriteLine;
Predicate<int> esPar = n => n % 2 == 0;
```

Expresiones lambda

Función anónima compacta:

```
csharp
Func<int, int> cuadrado = x => x * x;
Func<int, int, int> maximo = (a, b) => a > b ? a : b;
Action saludar = () => Console.WriteLine("Hola");

// Con bloque
Func<int, int, int> max = (a, b) => {
    if (a > b) return a;
    return b;
};
```

Clausura: una lambda captura la **variable**, no su valor en ese momento:

```
csharp
int base_ = 10;
Func<int, int> sumarBase = n => n + base_;
base_ = 20;
Console.WriteLine(sumarBase(5)); // 25 - ve el nuevo valor

// Bug clásico en loops - usar copia local para evitarlo:
for (int i = 0; i < 3; i++) {
    int copia = i;
    acciones.Add(() => Console.WriteLine(copia)); // captura copia, no i
}
```

Funciones de orden superior

Funciones que reciben o devuelven otras funciones:

```
csharp
// Recibir función como parámetro
static List<T> Filtrar<T>(List<T> lista, Predicate<T> criterio) =>
    lista.Where(x => criterio(x)).ToList();

Filtrar(numeros, n => n % 2 == 0);

// Devolver función como resultado
static Func<int, int> CrearMultiplicador(int factor) => n => n * factor;

var doble = CrearMultiplicador(2);
doble(5); // 10
```

Eventos

Un **evento** permite a un objeto notificar a otros cuando algo ocurre. Es un delegado con restricciones: solo el dueño puede dispararlo; los suscriptores solo pueden usar += y -=.

```

csharp
// Patrón estándar
public class PedidoCreadoEventArgs(int id, string cliente) : EventArgs {
    public int Id => id;
    public string Cliente => cliente;
}

public class SistemaPedidos {
    public event EventHandler<PedidoCreadoEventArgs>? PedidoCreado;

    protected virtual void OnPedidoCreado(PedidoCreadoEventArgs e) =>
        PedidoCreado?.Invoke(this, e);

    public void CrearPedido(string cliente) {
        // ... lógica ...
        OnPedidoCreado(new PedidoCreadoEventArgs(1, cliente));
    }
}

// Suscribirse
var sistema = new SistemaPedidos();
sistema.PedidoCreado += (sender, e) =>
    Console.WriteLine($"Nuevo pedido de {e.Cliente}");

sistema.PedidoCreado -= handler; // desuscribir

```

	Delegado	Evento
Quién puede invocarlo	Cualquiera	Solo la clase dueña
Quién puede asignar (=)	Cualquiera	Solo la clase dueña
Suscripción (+/-=)	Cualquiera	Cualquiera

Resumen

Necesidad	Herramienta
Pasar por referencia real	ref
Retorno adicional (patrón TryXxx)	out
Referencia de solo lectura	in
N argumentos variables	params
Función auxiliar local	función local
Guardar una función que retorna valor	Func<>
Guardar una función sin retorno	Action<>
Guardar una función que devuelve bool	Predicate<T>
Función anónima en el lugar	lambda
Notificar cambios a múltiples observadores	evento con EventHandler<T>

CAPITULO 10

Interfaces y contratos en C#

Una **interfaz** define un **contrato**: dice qué puede hacer un tipo, sin decir cómo. Expresa "puede hacer..." en lugar de "es un..." (que es la herencia).

Sintaxis básica

```
csharp
public interface ILogger {
    void Log(string mensaje);
    void LogError(string mensaje, Exception? ex = null);
    bool EstaActivo { get; }
}

public class ConsoleLogger : ILogger {
    public bool EstaActivo => true;
    public void Log(string mensaje) => Console.WriteLine($"[INFO] {mensaje}");
    public void LogError(string mensaje, Exception? ex = null) => Console.WriteLine($"[ERROR] {mensaje}");
}

// Uso polimórfico – el código trabaja con la interfaz
ILogger log = new ConsoleLogger();
log.Log("Iniciando");
```

Una interfaz puede tener métodos, propiedades y eventos — sin campos ni constructores.

Implementación múltiple

Un tipo puede implementar **múltiples interfaces** (ventaja clave sobre clases abstractas):

```
csharp
public interface ILegible { string Leer(); }
public interface IEscribible { void Escribir(string s); }

public class Buffer : ILegible, IEscribible {
    private readonly StringBuilder sb = new();
    public string Leer() => sb.ToString();
    public void Escribir(string s) => sb.Append(s);
}
```

Métodos con implementación por defecto

Una interfaz puede proveer implementaciones opcionales que las clases pueden sobrescribir:

```
csharp
public interface IFormateador {
    string Formatear(string texto); // obligatorio

    // opcional - la clase puede sobreescribirlo o usar este
    string FormatearLista(IEnumerable<string> items) =>
        string.Join(", ", items.Select(Formatear));
}
```

Los métodos por defecto solo son accesibles a través del tipo de la interfaz, no de la clase concreta.

Interfaces vs. clases abstractas

	Interfaz	Clase abstracta
Herencia múltiple	Sí	No
Puede tener estado (campos)	No	Sí
Constructores	No	Sí
Expresa "puede hacer..."	Sí	Indirecto
Expresa "es un tipo de..."	No	Sí

```
texto
¿Comparten estado y lógica base?          → clase abstracta
¿Comparten solo un contrato de comportamiento? → interfaz
¿Necesita cumplir múltiples contratos?     → múltiples interfaces
```

Interfaces genéricas

```
csharp
public interface IRepository<T> {
    T? ObtenerPorId(int id);
    IList<T> ObtenerTodos();
    void Agregar(T entidad);
    void Eliminar(int id);
}

public class ProductoRepository : IRepository<Producto> {
    private readonly List<Producto> datos = [];

    public Producto? ObtenerPorId(int id) => datos.FirstOrDefault(p => p.Id == id);
    public IList<Producto> ObtenerTodos() => datos.AsReadOnly();
    public void Agregar(Producto p) => datos.Add(p);
    public void Eliminar(int id) => datos.RemoveAll(p => p.Id == id);
}
```

Restricciones en genéricos con where:

```
csharp
// T debe implementar IComparable<T>
static T Maximo<T>(T a, T b) where T : IComparable<T> =>
    a.CompareTo(b) >= 0 ? a : b;

// T debe ser clase y tener constructor sin parámetros
static T Crear<T>() where T : class, new() => new T();
```

Interfaces de .NET más usadas

Interfaz	Para qué
IEnumerable<T>	Poder recorrer con foreach
IComparable<T>	Comparación para ordenamiento
IComparer<T>	Criterio de comparación externo
IEquatable<T>	Igualdad por valor
IDisposable	Liberar recursos con using
IAsyncDisposable	Liberar recursos async

```
csharp
public class Gestor : IDisposable {
    private bool disposed = false;

    public void Dispose() {
        if (!disposed) {
            // liberar recursos
            disposed = true;
        }
    }
}

using var g = new Gestor(); // Dispose() se llama al salir del bloque
```

Ejemplo: clases abstracta + interfaces combinadas

```
csharp
public abstract class Animal(string nombre) {
    public string Nombre => nombre;
    public abstract void HacerSonido();
}

public interface IEntrenable { void Entrenar(); }
public interface IVacunable { void Vacunar(); }

// Perro ES un Animal Y PUEDE SER entrenado y vacunado
public class Perro(string nombre) : Animal(nombre), IEntrenable, IVacunable {
    public override void HacerSonido() => Console.WriteLine("Guau!");
    public void Entrenar() => Console.WriteLine($"{Nombre} aprende un truco");
    public void Vacunar() => Console.WriteLine($"{Nombre} fue vacunado");
}
```

Resumen

Necesidad	Herramienta
Definir comportamiento sin implementación	interfaz
Compartir estado y lógica base	clase abstracta
Cumplir múltiples contratos	múltiples interfaces
Contrato para un tipo genérico	interfaz genérica
Restringir tipo genérico	where T : IAlgo
Liberar recursos	IDisposable + using

CAPITULO 11

Control de flujo y manejo de excepciones en C#

Familia	Qué hace	Ejemplos
Condicionales	Elige qué ejecutar	if, switch
Iterativas	Repite un bloque	while, for, foreach
De salto	Altera el flujo	break, continue, return
De error	Maneja situaciones excepcionales	try/catch/finally

if / else

```
csharp
if (temperatura > 35) Console.WriteLine("Calor extremo");
else if (temperatura > 25) Console.WriteLine("Calor agradable");
else Console.WriteLine("Frío");

// Pattern matching con is
if (valor is int n and > 0) Console.WriteLine($"Entero positivo: {n}");
if (nombre is not null) Console.WriteLine(nombre.ToUpper());
```

Operador ternario ? :

```
csharp
string categoria = edad >= 18 ? "adulto" : "menor";

// Para múltiples casos, preferir switch expression
string nivel = nota switch {
    >= 9 => "Sobresaliente",
    >= 7 => "Notable",
    >= 6 => "Aprobado",
    _ => "Desaprobado"
};
```

switch expression (forma moderna)

Produce un valor, no necesita break:

```
csharp
string nombre = diaSemana switch {
    1 => "Lunes",
    2 => "Martes",
    6 or 7 => "Fin de semana",
    _ => "Otro día"
};
```


csharp - continuacion

```
// Con pattern matching de tipos y propiedades
static string Describir(object fig) => fig switch {
    Circulo c                => $"Círculo r={c.Radio}",
    Rectangulo { Ancho: var a, Alto: var h } => $"Rect {a}x{h}",
    null                     => "nulo",
    -                         => "desconocido"
};

// Pattern de tupla
static string Clima(int temp, bool llueve) => (temp, llueve) switch {
    (> 30, false) => "Soleado",
    (> 30, true)  => "Bochornoso",
    (-, true)    => "Lluvioso",
    -           => "Fresco"
};

// Pattern de lista
static string Analizar(int[] nums) => nums switch {
    []                => "vacía",
    [var x]           => $"un elemento: {x}",
    [var x, var y]    => $"dos: {x} y {y}",
    [var f, .., var u] => $"del {f} al {u}"
};
```

Bucles

csharp

```
// while – condición al inicio, puede no ejecutarse
while (intentos < 3) { ... }

// do while – condición al final, se ejecuta al menos una vez
do { Console.WriteLine("Contraseña: "); } while (input != "1234");

// for – con contador explícito
for (int i = 0; i < 10; i++) Console.WriteLine(i);
for (int i = 9; i >= 0; i--) Console.WriteLine(i); // hacia atrás

// foreach – forma idiomática para colecciones
foreach (var fruta in frutas) Console.WriteLine(fruta);

// con índice
foreach (var (fruta, i) in frutas.Select((f, i) => (f, i)))
    Console.WriteLine($"[{i}] {fruta}");

// async
await foreach (var item in ObtenerAsync()) { ... }
```

Sentencias de salto

```
csharp
break;    // salir del bucle o switch
continue; // pasar a la siguiente iteración
return;   // salir del método

// Ejemplo: salir al encontrar
for (int i = 0; i < nums.Length; i++) {
    if (nums[i] == buscar) { posicion = i; break; }
}

// Ejemplo: saltar elementos
foreach (var v in valores) {
    if (v is null) continue;
    Console.WriteLine(v);
}
```

Excepciones

```
csharp
try {
    int resultado = 10 / divisor;
} catch (DivideByZeroException ex) {
    Console.WriteLine($"División por cero: {ex.Message}");
} catch (Exception ex) {
    Console.WriteLine($"Error inesperado: {ex.Message}");
    throw; // relanzar sin perder el stack
} finally {
    // siempre se ejecuta, pase lo que pase
    Console.WriteLine("Operación finalizada");
}
```

Lanzar excepciones:

```
csharp
throw new ArgumentNullException(nameof(nombre));
throw new InvalidOperationException("Estado incorrecto");
ArgumentNullException.ThrowIfNull(nombre); // C# 10+
ArgumentOutOfRangeException.ThrowIfNegative(valor); // C# 8+
```

Jerarquía típica:

```
texto
Exception
■ ■ ■ ■ ArgumentException
■   ■ ■ ■ ■ ArgumentNullException
■   ■ ■ ■ ■ ArgumentOutOfRangeException
■ ■ ■ ■ InvalidOperationException
■ ■ ■ ■ NotSupportedException
■ ■ ■ ■ IOException
■ ■ ■ ■ FormatException
■ ■ ■ ■ OverflowException
```

Crear excepciones propias:

```
csharp
public class NegocioException(string mensaje, string codigo)
    : Exception(mensaje) {
    public string Codigo => codigo;
}

throw new NegocioException("Stock insuficiente", "STOCK_001");
```

Filtros de excepción:

```
csharp
catch (HttpRequestException ex) when (ex.StatusCode == HttpStatusCode.NotFound) {
    Console.WriteLine("Recurso no encontrado");
}
```

Resumen

Necesidad	Herramienta
Condición simple	if / else
Múltiples casos que producen un valor	switch expression
Repetir sin saber cuántas veces	while
Ejecutar al menos una vez	do while
Repetir con índice	for
Recorrer colección	foreach
Salir del bucle	break
Saltar iteración	continue
Manejar errores en runtime	try/catch/finally
Señalar estado inválido	throw new XxxException(...)

CAPITULO 12

Switch Expression y Pattern Matching en C#

El **pattern matching** permite examinar un valor y ramificar según su estructura, tipo o contenido. El **switch expression** es la forma principal: produce un valor directamente.

```
csharp
// Antes
string resultado;
if (figura is Circulo c) resultado = $"Círculo r={c.Radio}";
else if (figura is Rectangulo r) resultado = $"Rect {r.Ancho}x{r.Alto}";
else resultado = "desconocida";

// Ahora
string resultado = figura switch {
    Circulo c => $"Círculo r={c.Radio}",
    Rectangulo r => $"Rect {r.Ancho}x{r.Alto}",
    _ => "desconocida"
};
```

	switch instrucción	switch expression
Produce	Sentencias	Un valor
break	Obligatorio	No existe
default	Keyword	_ (wildcard)
Exhaustividad	No verificada	Verificada por compilador

Tipos de patrones

Constante

```
csharp
string msg = codigo switch {
    200 => "OK",
    404 => "No encontrado",
    500 => "Error del servidor",
    _ => $"Código: {codigo}"
};

// Con or - múltiples valores, mismo resultado
string resp = comando switch {
    "ayuda" or "help" or "?" => MostrarAyuda(),
    "salir" or "exit" => "Hasta luego",
    _ => "Comando desconocido"
};
```

Tipo (type pattern)

csharp

```
string desc = valor switch {
    int n => $"entero: {n}",
    double d => $"decimal: {d:F2}",
    string s => $"texto de {s.Length} chars",
    null => "nulo",
    _ => $"tipo: {valor.GetType().Name}"
};
```

Propiedad (property pattern)

csharp

```
decimal descuento = pedido switch {
    { Estado: "cancelado" } => 0,
    { Total: > 10_000, EsUrgente: false } => pedido.Total * 0.15m,
    { Total: > 5_000 } => pedido.Total * 0.10m,
    _ => 0
};
```

Relacional y lógico

csharp

```
string nivel = nota switch {
    >= 9 => "Sobresaliente",
    >= 7 and < 9 => "Notable",
    >= 6 and < 7 => "Aprobado",
    < 0 or > 10 => throw new ArgumentOutOfRangeException(),
    _ => "Desaprobado"
};
```

Tupla

csharp

```
string clima = (temp, llueve) switch {
    (> 30, false) => "Soleado",
    (> 30, true) => "Bochornoso",
    (_, true) => "Lluvioso",
    _ => "Fresco"
};
```

Lista (list pattern)

csharp

```
string analisis = nums switch {
    [] => "vacía",
    [var x] => $"un elemento: {x}",
    [var x, var y] => $"dos: {x} y {y}",
    [var f, .., var u] => $"del {f} al {u}"
};
```

Guardia when

csharp

```
string desc = figura switch {
    Circulo c when c.Radio == 0      => "Punto degenerado",
    Circulo c when c.Radio > 100    => "Circulo enorme",
    Circulo c                        => $"Circulo r={c.Radio}",
    Rectangulo { Ancho: var a, Alto: var h } when a == h => $"Cuadrado {a}",
    -                               => "Otra figura"
};
```

Pattern variable con is

csharp

```
if (valor is int n and > 0)
    Console.WriteLine($"Positivo: {n}");

if (nombre is { Length: > 0 } s)
    Console.WriteLine($"Nombre: {s.ToUpper()}");
```

Ejemplo completo — jerarquía de records

csharp

```
abstract record Figura(string Color);
record Circulo(string Color, double Radio) : Figura(Color);
record Rectangulo(string Color, double Ancho, double Alto) : Figura(Color);

static double Area(Figura f) => f switch {
    Circulo c => Math.PI * c.Radio * c.Radio,
    Rectangulo r => r.Ancho * r.Alto,
    -          => throw new ArgumentException($"No soportada: {f.GetType().Name}")
};
```

Resumen de patrones

Patrón	Sintaxis	Cuándo
Constante	valor => ...	Comparar contra un literal
Tipo	Tipo t => ...	Verificar y extraer tipo
Propiedad	{ Prop: valor }	Ramificar por propiedades
Relacional	> x, < x, >= x	Comparar rangos
Lógico	and, or, not	Combinar condiciones
Tupla	(v1, v2) => ...	Múltiples valores a la vez
Lista	[p1, p2, ...]	Estructura de colección
Guardia	patrón when condición	Condición extra
Variable	is Tipo v	Declarar variable inline

CAPITULO 13

Manejo de errores — try / catch / finally

Estructura básica

```
csharp
try {
    string contenido = File.ReadAllText("datos.txt");
    Console.WriteLine(contenido);
} catch (FileNotFoundException ex) {
    Console.WriteLine($"Archivo no encontrado: {ex.FileName}");
} catch (IOException ex) {
    Console.WriteLine($"Error de I/O: {ex.Message}");
} catch (Exception ex) {
    Console.WriteLine($"Error inesperado: {ex.Message}");
    throw; // relanzar sin perder el stack trace
} finally {
    // siempre se ejecuta, haya error o no
    Console.WriteLine("Operación finalizada.");
}
```

Las excepciones más específicas van **primero**; Exception al final.

finally y using

finally es para liberar recursos. using lo reemplaza automáticamente para IDisposable:

```
csharp
// Equivalentes – preferir using
try {
    using var lector = new StreamReader("datos.txt");
    Console.WriteLine(lector.ReadToEnd());
} catch (FileNotFoundException) {
    Console.WriteLine("No existe.");
}
// Dispose() se llama automáticamente
```

Lanzar excepciones

```
csharp
throw new ArgumentOutOfRangeException(nameof(n), "No puede ser negativo");
throw new InvalidOperationException("Estado incorrecto");

// Helpers de C# 10+
ArgumentNullException.ThrowIfNull(nombre);
ArgumentException.ThrowIfNullOrWhiteSpace(nombre);
ArgumentOutOfRangeException.ThrowIfNegative(valor);

// throw como expresión
string nombre = ObtenerNombre() ?? throw new InvalidOperationException("Requerido");

static double Dividir(double a, double b) =>
    b == 0 ? throw new DivideByZeroException() : a / b;
```

Filtros con when

```
csharp
catch (HttpRequestException ex) when (ex.StatusCode == HttpStatusCode.NotFound) {
    Console.WriteLine("No encontrado (404)");
} catch (HttpRequestException ex) when (ex.StatusCode == HttpStatusCode.Unauthorized) {
    Console.WriteLine("Sin autorización (401)");
}
```

Si la condición when es falsa, la excepción sigue propagándose al siguiente catch.

Excepciones personalizadas

```
csharp
public class SaldoInsuficienteException : Exception {
    public decimal SaldoActual { get; }
    public decimal MontoSolicitado { get; }

    public SaldoInsuficienteException(decimal saldo, decimal monto)
        : base($"Disponible: {saldo:N2}, solicitado: {monto:N2}") {
        SaldoActual = saldo; MontoSolicitado = monto;
    }
}

throw new SaldoInsuficienteException(cuenta.Saldo, montoSolicitado);
```

Buenas prácticas

Capturar excepciones específicas, no `Exception` genérica. No silenciar errores con `catch (Exception) { }`. Usar `TryParse` / `TryGetValue` para situaciones esperadas (más eficiente que `try/catch`):

```
csharp
// MAL: excepciones para control de flujo
try { var v = diccionario["clave"]; } catch (KeyNotFoundException) { }

// BIEN
if (diccionario.TryGetValue("clave", out var v)) Console.WriteLine(v);
```


Jerarquía de excepciones comunes

texto

```
Exception
  ■■■■ ArgumentException
  ■ ■■■■ ArgumentNullException
  ■ ■■■■ ArgumentOutOfRangeException
  ■■■■ InvalidOperationException
  ■■■■ NullReferenceException ← señal de null no manejado
  ■■■■ IOException
  ■ ■■■■ FileNotFoundException
  ■ ■■■■ UnauthorizedAccessException
  ■■■■ FormatException
```

Resumen

Necesidad	Herramienta
Código protegido ante fallos	try/catch
Limpieza garantizada	finally
Liberar recursos	using
Señalar estado inválido	throw new XxxException(...)
Propagar error original	throw; (sin argumentos)
Filtrar condición dentro de catch	catch (...) when (...)
Error del dominio propio	excepción personalizada
Conversión/búsqueda segura	TryParse / TryGetValue

CAPITULO 14

Iteradores en C#: IEnumerable, IEnumerator y yield

El problema

foreach funciona sobre cualquier colección porque estas implementan IEnumerable<T>. El foreach se traduce internamente a:

```
csharp
IEnumerator<T> e = coleccion.GetEnumerator();
try {
    while (e.MoveNext()) {
        var x = e.Current;
        // cuerpo del foreach
    }
} finally {
    e.Dispose();
}
```

Las interfaces

```
csharp
public interface IEnumerable<out T> {
    IEnumerator<T> GetEnumerator(); // devuelve un iterador nuevo
}

public interface IEnumerator<out T> : IDisposable {
    T Current { get; } // elemento actual
    bool MoveNext(); // avanza; false si no hay más
}
```

- IEnumerable = "se me puede recorrer" (la colección)
- IEnumerator = "llevo la cuenta del recorrido" (el iterador)

GetEnumerator() devuelve un objeto **nuevo** cada vez → se pueden tener varios recorridos simultáneos sin interferencia.

Implementación manual

```
csharp
public class RangoEntero : IEnumerable<int> {
    readonly int inicio, fin;
    public RangoEntero(int inicio, int fin) { this.inicio = inicio; this.fin = fin; }

    public IEnumerator<int> GetEnumerator() =>
        new RangoEnumerador(inicio, fin);
    IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();
}

public class RangoEnumerador : IEnumerator<int> {
    readonly int inicio, fin;
    int actual; bool iniciado;

    public RangoEnumerador(int i, int f) { inicio = i; fin = f; }
    public int Current => actual;
    object IEnumerator.Current => Current;
    public void Reset() { iniciado = false; }
    public void Dispose() { }

    public bool MoveNext() {
        if (iniciado) actual++;
        else { iniciado = true; actual = inicio; }
        return actual <= fin;
    }
}
```

Esto es mucho código. `yield return` lo simplifica enormemente.

`yield return` — la forma moderna

El compilador genera la clase enumerador por vos:

```
csharp
public class RangoEntero : IEnumerable<int> {
    readonly int inicio, fin;
    public RangoEntero(int i, int f) { inicio = i; fin = f; }

    public IEnumerator<int> GetEnumerator() {
        for (int i = inicio; i <= fin; i++)
            yield return i; // "pausar aquí, devolver i, continuar en el próximo MoveNext()"
    }

    IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();
}

foreach (int n in new RangoEntero(1, 5))
    Console.WriteLine(n); // 1 2 3 4 5
```

También como método estático que devuelve `IEnumerable<T>`:

```
csharp
static IEnumerable<int> Rango(int inicio, int fin) {
    for (int i = inicio; i <= fin; i++)
        yield return i;
}

// Secuencia infinita — solo consume lo que se necesita
static IEnumerable<int> NumerosNaturales() {
    int n = 1;
    while (true) yield return n++;
}
```

csharp - continuacion

```
// Tomar los primeros 5 pares
var pares = NumerosNaturales().Where(n => n % 2 == 0).Take(5);
```

`yield break` detiene el iterador antes de terminar:

csharp

```
static IEnumerable<int> HastaMaximo(IEnumerable<int> fuente, int max) {
    foreach (int n in fuente) {
        if (n > max) yield break; // detener
        yield return n;
    }
}
```

Evaluación diferida (lazy)

Los métodos con `yield` **no ejecutan nada al llamarlos**. Solo ejecutan cuando se itera el resultado. Esto permite trabajar con secuencias potencialmente infinitas y encadenar transformaciones sin materializar resultados intermedios:

csharp

```
var resultado = NumerosNaturales()
    .Where(n => n % 2 == 0) // no ejecuta aún
    .Select(n => n * n)     // no ejecuta aún
    .Take(5)               // no ejecuta aún
    .ToList();             // acá sí ejecuta todo
```

Cuándo usar `yield`

- Secuencias que se calculan sobre la marcha (no están guardadas)
- Secuencias potencialmente infinitas
- Evitar materializar toda la colección en memoria cuando solo se necesita una parte
- Recorrer fuentes externas (archivos, bases de datos) línea por línea

Resumen

	<code>IEnumerable<T></code>	<code>IEnumerator<T></code>
Rol	La colección	El iterador
Método principal	<code>GetEnumerator()</code>	<code>MoveNext()</code> / <code>Current</code>
Una instancia por...	Colección	Recorrido activo

texto

¿Quiero que mi clase sea recorrible con foreach? → implementar IEnumerable<T>
¿Quiero implementar el recorrido sin boilerplate? → usar yield return
¿Quiero generar una secuencia perezosa? → método que devuelve IEnumerable<T> con yield
¿Quiero detener el iterador antes? → yield break

CAPITULO 15

LINQ — Language Integrated Query

LINQ permite consultar y transformar colecciones de forma **declarativa** (qué resultado quiero) en lugar de imperativa (cómo hacerlo paso a paso):

```
csharp
// Imperativo – 20 líneas con bucles y lógica dispersa
// ...

// LINQ – 4 líneas que se leen como una descripción
var resultado = pedidos
    .Where(p => p.Estado == "aprobado" && p.Fecha >= DateTime.Today.AddDays(-30))
    .DistinctBy(p => p.ClienteId)
    .OrderByDescending(p => p.Monto)
    .Select(p => p.Monto)
    .ToList();
```

Evaluación diferida

La mayoría de los métodos LINQ **no ejecutan nada al llamarlos**. Solo construyen un plan. La ejecución ocurre cuando se itera el resultado:

```
csharp
var query = productos
    .Where(p => p.Precio > 1000) // no ejecuta
    .OrderBy(p => p.Nombre);     // no ejecuta

// La ejecución ocurre aquí:
foreach (var p in query) { ... }

// O al materializar:
var lista = query.ToList();     // ejecuta y guarda
```

Métodos que fuerzan la ejecución (materializan): `ToList()`, `ToArray()`, `ToDictionary()`, `Count()`, `First()`, `Sum()`, `Any()`, `All()`, `foreach`.

Métodos más usados

Filtrado

```
csharp
productos.Where(p => p.Precio > 1000 && p.Stock > 0)
productos.OfType<Electronico>() // filtra y castea
productos.Distinct()             // sin duplicados
productos.DistinctBy(p => p.Nombre) // sin duplicados por clave
productos.Take(10)               // primeros 10
productos.Skip(5)                 // saltar los primeros 5
productos.TakeWhile(p => p.Precio < 5000) // mientras se cumpla
```

Proyección

```
csharp
productos.Select(p => p.Nombre)           // transformar cada elemento
productos.Select(p => new { p.Nombre, p.Precio }) // tipo anónimo
productos.SelectMany(p => p.Etiquetas)    // aplanar colecciones anidadas
```

Ordenamiento

```
csharp
productos.OrderBy(p => p.Precio)
productos.OrderByDescending(p => p.Precio)
productos.OrderBy(p => p.Categoria).ThenBy(p => p.Nombre)
```

Agrupamiento

```
csharp
var grupos = productos.GroupBy(p => p.Categoria);

foreach (var g in grupos) {
    Console.WriteLine($"{g.Key}: {g.Count()} productos");
}

// Agrupar en diccionario
var dict = productos
    .GroupBy(p => p.Categoria)
    .ToDictionary(g => g.Key, g => g.ToList());
```

Joins

```
csharp
var result = clientes.Join(
    pedidos,
    c => c.Id,
    p => p.ClienteId,
    (c, p) => new { c.Nombre, p.Monto }
);

// Left join
var leftJoin = clientes
    .GroupJoin(pedidos, c => c.Id, p => p.ClienteId,
        (c, ps) => new { c.Nombre, Pedidos = ps })
    .SelectMany(x => x.Pedidos.DefaultIfEmpty(),
        (c, p) => new { c.Nombre, Monto = p?.Monto ?? 0 });
```

Agregación

```
csharp
productos.Count()
productos.Count(p => p.Stock > 0)
productos.Sum(p => p.Precio)
productos.Average(p => p.Precio)
productos.Min(p => p.Precio)
productos.Max(p => p.Precio)
productos.Aggregate(0m, (acc, p) => acc + p.Precio)
```

Búsqueda

```
csharp
productos.First()           // primero (excepción si vacío)
productos.FirstOrDefault()  // primero o null
productos.Single()          // exactamente uno (excepción si no)
productos.SingleOrDefault()
productos.Any()             // hay alguno?
productos.Any(p => p.Stock == 0)
productos.All(p => p.Precio > 0)
productos.Contains(elemento)
```

Conversión

```
csharp
.ToList()
.ToArray()
.ToDictionary(p => p.Id)
.ToDictionary(p => p.Id, p => p.Nombre)
.ToHashSet()
.ToLookup(p => p.Categoria) // como dictionary pero admite múltiples valores por clave
```

Sintaxis de query (alternativa)

```
csharp
// Equivalente al encadenamiento de métodos
var result = from p in productos
              where p.Precio > 1000
              orderby p.Precio descending
              select new { p.Nombre, p.Precio };
```

La sintaxis de métodos es más popular y soporta más operaciones. La de query es más legible para joins y agrupaciones complejas.

Ejemplo completo

csharp

```
record Producto(string Nombre, string Categoria, decimal Precio, int Stock);

List<Producto> productos = [
    new("Monitor", "Hardware", 450_000, 5),
    new("Teclado", "Hardware", 85_000, 20),
    new("Mouse", "Hardware", 35_000, 0),
    new("VS Code", "Software", 0, 99),
    new("Webcam", "Hardware", 55_000, 8),
];

var resumen = productos
    .Where(p => p.Stock > 0)
    .GroupBy(p => p.Categoria)
    .Select(g => new {
        Categoria = g.Key,
        Total = g.Sum(p => p.Precio),
        Promedio = g.Average(p => p.Precio),
        Cantidad = g.Count()
    })
    .OrderByDescending(g => g.Total);

foreach (var g in resumen)
    Console.WriteLine($"{g.Categoria}: {g.Cantidad} productos, total ${g.Total:N0}");
```

Resumen de operaciones

Operación	Métodos
Filtrar	Where, OfType, Distinct, Take, Skip
Transformar	Select, SelectMany
Ordenar	OrderBy, OrderByDescending, ThenBy
Agrupar	GroupBy, ToLookup
Combinar	Join, GroupJoin, Concat, Zip
Agregar	Count, Sum, Average, Min, Max, Aggregate
Buscar	First, FirstOrDefault, Single, Any, All, Contains
Materializar	ToList, ToArray, ToDictionary, ToHashSet

CAPÍTULO 16

Programación Asíncrona en C# — Guía Práctica

La programación asíncrona es fundamental en C# para desarrollar aplicaciones que no bloqueen el hilo de ejecución principal (por ejemplo, la interfaz de usuario o el hilo que atiende peticiones en un servidor web) mientras se realizan operaciones de entrada/salida (I/O), como consultar una base de datos, descargar un archivo de internet o leer del disco.

1. La Idea Central: I/O-Bound vs. CPU-Bound

La asincronía con `async` y `await` en C# está diseñada principalmente para **operaciones de entrada/salida (I/O-Bound)**:

- **Sincrónico (Bloqueante):** Tu programa le pide datos a la base de datos y el hilo de ejecución se queda cruzado de brazos esperando a que la red y el disco de la base de datos respondan. El hilo no puede hacer otra cosa mientras tanto.
- **Asíncrono (No bloqueante):** Tu programa inicia la petición a la base de datos y **libera el hilo** para que pueda hacer otras tareas (como responder a clics de UI o atender otras peticiones web). Cuando la base de datos responde, se retoma la ejecución en el punto donde se quedó.

[!NOTE] No creamos hilos nuevos manualmente. Delegamos la espera al sistema operativo o al hardware de red, optimizando drásticamente el uso de recursos.

2. Las Palabras Clave: `async` y `await`

- `async`: Es un modificador de método. Su única función es habilitar el uso de la palabra clave `await` dentro del método y permitir que el compilador transforme el método en una máquina de estados.
- `await`: Se coloca antes de una tarea (`Task`). Suspende la ejecución del método actual de manera no bloqueante hasta que la tarea finalice. Mientras espera, el hilo actual queda libre.

```
csharp
// Método sincrónico
void Procesar() {
    string datos = LeerArchivo("datos.txt"); // Bloquea el hilo aquí
    Console.WriteLine(datos);
}

// Método asíncrono equivalente
async Task ProcesarAsync() {
    string datos = await LeerArchivoAsync("datos.txt"); // Libera el hilo aquí mientras lee
    Console.WriteLine(datos);
}
```

3. Tipos de Retorno Asíncronos

Al escribir un método async, el tipo de retorno debe ser uno de los siguientes:

Tipo de Retorno	Cuándo usarlo	Ejemplo de firma
Task	El método hace un trabajo asíncrono pero no devuelve ningún valor (equivalente asíncrono a void).	async Task GuardarDatosAsync(...)
Task<T>	El método hace un trabajo asíncrono y devuelve un valor de tipo T al terminar.	async Task<Contacto> ObtenerContactoAsync(int id)
ValueTask<T>	Optimización de memoria para métodos que en la mayoría de los casos resuelven su resultado sincrónicamente y no necesitan instanciar una Task en el heap.	async ValueTask<int> LeerCacheAsync(...)
void	Únicamente para manejadores de eventos (<i>event handlers</i> en interfaces visuales).	async void OnClick(object sender, EventArgs e)

[!WARNING] Evita async void en métodos comunes. Si ocurre una excepción dentro de un método async void, esta no puede ser capturada por un bloque try-catch externo y tirará abajo la aplicación entera de forma abrupta.

4. Uso Práctico con Librerías Comunes

En el día a día, casi nunca crearás Task desde cero. En su lugar, **consumirás métodos asíncronos** provistos por .NET o por librerías externas. La convención en C# es que estos métodos terminen con el sufijo Async.

A. Consumo de APIs Web (HttpClient)

```
csharp
using System.Net.Http;
using System.Net.Http.Json;

var cliente = new HttpClient();

// Obtener contenido de texto de forma asíncrona
string html = await cliente.GetStringAsync("https://api.github.com/users/octocat");

// Obtener y deserializar JSON directamente a un objeto
Contacto? c = await cliente.GetFromJsonAsync<Contacto>("https://api.ejemplo.com/contactos/1");
```

B. Lectura y Escritura de Archivos (System.IO)

```
csharp
using System.IO;

string ruta = "archivo.txt";

// Escribir texto de forma asincrónica
await File.WriteAllTextAsync(ruta, "Hola, mundo asincrónico!");

// Leer texto de forma asincrónica
string contenido = await File.ReadAllTextAsync(ruta);
```

C. Persistencia con Dapper (Dapper)

Dapper provee extensiones asincrónicas equivalentes para todos sus métodos sobre `DbConnection`.

```
csharp
using System.Data;
using Microsoft.Data.Sqlite;
using Dapper;

using IDbConnection db = new SqliteConnection("Data Source=agenda.db");

// Consulta asincrónica de múltiples registros
var contactos = await db.QueryAsync<Contacto>("SELECT * FROM Contactos");

// Consulta asincrónica de un único registro
var contacto = await db.QuerySingleOrDefaultAsync<Contacto>(
    "SELECT * FROM Contactos WHERE Id = @id", new { id = 1 }
);

// Ejecutar comando asincrónico (INSERT/UPDATE/DELETE)
int filasAfectadas = await db.ExecuteAsync(
    "UPDATE Contactos SET Favorito = @fav WHERE Id = @id",
    new { fav = true, id = 5 }
);
```

5. Reglas de Oro y Buenas Prácticas

1. "Async All the Way" (Asincrónico de punta a punta)

La asincronía es contagiosa. Si un método es asincrónico, el método que lo llama también debería ser `async` y usar `await`, propagándose hasta el punto de entrada de la aplicación (`Main`, un endpoint de API o un controlador).

2. Nunca bloquee usando `.Result` o `.Wait()`

Si necesitás esperar un método asincrónico desde un contexto sincrónico, **no uses** las propiedades bloqueantes de `Task`:

```
csharp
// ✗ MAL - Bloquea el hilo actual y puede causar Deadlocks (bloqueos mutuos)
var datos = ObtenerDatosAsync().Result;
ObtenerDatosAsync().Wait();

// ✓ BIEN - Espera de forma no bloqueante usando await
var datos = await ObtenerDatosAsync();
```

Si estás obligado a llamar código asíncrono desde un método síncrono (por ejemplo, al implementar una interfaz síncrona de una librería externa que no podés modificar), podés usar descartes o correrlo en segundo plano, pero la regla general es siempre propagar el `await`.

3. El manejo de excepciones es transparente

Con `await`, el manejo de excepciones asíncronas funciona exactamente igual que el código síncrono mediante bloques `try-catch`:

```
csharp
try {
    string datos = await cliente.GetStringAsync("https://url-invalida.com");
} catch (HttpRequestException ex) {
    Console.WriteLine($"Error de red: {ex.Message}");
}
```

6. Ejecución en Paralelo Simultánea

A veces querés disparar múltiples tareas al mismo tiempo en lugar de esperar a que termine una para empezar la otra. Podés lograr esto guardando las tareas en variables (sin usar `await` inmediatamente) y luego esperándolas juntas con `Task.WhenAll`.

```
csharp
// Dispara las tres peticiones HTTP al mismo tiempo
Task<string> tarea1 = cliente.GetStringAsync("https://api.com/datos1");
Task<string> tarea2 = cliente.GetStringAsync("https://api.com/datos2");
Task<string> tarea3 = cliente.GetStringAsync("https://api.com/datos3");

// Espera a que las tres terminen (las operaciones corren en paralelo en la red)
string[] resultados = await Task.WhenAll(tarea1, tarea2, tarea3);

string resultado1 = resultados[0];
string resultado2 = resultados[1];
string resultado3 = resultados[2];
```

7. Cancelación de Operaciones (CancellationToken)

Las librerías asíncronas suelen aceptar un parámetro opcional llamado `CancellationToken`. Esto permite que tu aplicación aborte una operación larga (como una descarga o consulta pesada) si el usuario hace clic en "Cancelar" o si expira un tiempo límite (*timeout*).

csharp

```
using System.Threading;

// Crear una fuente de tokens de cancelación con un tiempo límite de 3 segundos
using var cts = new CancellationTokenSource(TimeSpan.FromSeconds(3));
CancellationToken token = cts.Token;

try {
    // Si la API tarda más de 3 segundos, GetStringAsync lanzará OperationCanceledException
    string html = await cliente.GetStringAsync("https://api.lenta.com/datos", token);
} catch (OperationCanceledException) {
    Console.WriteLine("La operación fue cancelada por timeout.");
}
```

CAPITULO 17

Principios SOLID en C#

Cinco reglas de diseño orientado a objetos que hacen el código más fácil de mantener, extender y testear:

Letra	Principio	Idea
S	Single Responsibility	Una clase, una razón para cambiar
O	Open/Closed	Abierto a extensión, cerrado a modificación
L	Liskov Substitution	Las subclases reemplazan a la base sin romper nada
I	Interface Segregation	Muchas interfaces pequeñas > una grande
D	Dependency Inversion	Depender de abstracciones, no de implementaciones

S — Single Responsibility

Una clase que hace demasiado (gestionar saldo + persistir + notificar + calcular puntos) tiene múltiples razones para cambiar. La solución: separar responsabilidades en clases propias.

```
csharp
// Solo modela la tarjeta
public class Tarjeta {
    public string Numero { get; set; }
    public decimal Saldo { get; set; }
    public int Puntos { get; set; }
    public string Tipo { get; set; }

    public void Acreditar(decimal monto) {
        if (monto <= 0) throw new MontoInvalidoException();
        Saldo += monto;
    }
    public void Debitar(decimal monto) {
        if (monto <= 0) throw new MontoInvalidoException();
        if (Saldo < monto) throw new SaldoInsuficienteException();
        Saldo -= monto;
    }
}

public class RegistroMovimientos {
    public void Registrar(string numero, string op, decimal monto) =>
        File.AppendAllText("movimientos.txt", $"{DateTime.Now}: {op} {monto} en {numero}\n");
}

public class NotificadorEmail {
    public void Notificar(string dest, string asunto, string cuerpo) { /* Smtplib */ }
}

public class CalculadorPuntos {
    public int Calcular(string tipo, decimal monto) => tipo switch {
        "clasica" => (int)(monto * 0.01m),
        "gold" => (int)(monto * 0.02m),
        "platinum" => (int)(monto * 0.05m),
    }
}
```

csharp - continuacion

```
}; - => 0
}
```

O — Open/Closed

Si agregar un tipo nuevo de tarjeta obliga a modificar `CalculadorPuntos`, la clase no está cerrada.

Solución: polimorfismo.

csharp

```
public abstract class Tarjeta {
    public decimal Saldo { get; protected set; }
    public int Puntos { get; protected set; }

    public void Debitar(decimal monto) {
        if (Saldo < monto) throw new SaldoInsuficienteException();
        Saldo -= monto;
        Puntos += CalcularPuntos(monto);
    }
    protected abstract int CalcularPuntos(decimal monto);
}

public class TarjetaClasica : Tarjeta { protected override int CalcularPuntos(decimal m) => (int)(m * 0.01m); }
public class TarjetaGold : Tarjeta { protected override int CalcularPuntos(decimal m) => (int)(m * 0.02m); }
public class TarjetaPlatinum : Tarjeta { protected override int CalcularPuntos(decimal m) => (int)(m * 0.05m); }

// Agregar TarjetaBlack no toca nada existente:
public class TarjetaBlack : Tarjeta { protected override int CalcularPuntos(decimal m) => (int)(m * 0.10m); }
```

L — Liskov Substitution

Si una subclase lanza excepciones inesperadas en métodos de la clase base, viola LSP:

csharp

```
// ✗ Mal - rompe el contrato de Tarjeta
public class TarjetaPrepaga : Tarjeta {
    public override void Acreditar(decimal monto) =>
        throw new OperacionNoSoportadaException();
}
```

Señal de violación: `if (tarjeta is TarjetaPrepaga)` diseminado en el código. Solución: rediseñar la jerarquía para que `Acreditar` solo exista donde tiene sentido.

I — Interface Segregation

Una interfaz enorme obliga a implementar métodos sin sentido. Mejor segregar:


```
csharp
public interface IAcreditable { void Acreditar(decimal monto); }
public interface IDEbitable { void Debitar(decimal monto); }
public interface IAcumulaPuntos { int Puntos { get; } }
public interface ICanjeablePuntos { void CanjearPuntos(int puntos); }

public class TarjetaDebito : IAcreditable, IDEbitable { /* solo lo que le corresponde */ }
public class TarjetaCredito : IDEbitable, IAcumulaPuntos, ICanjeablePuntos { /* ... */ }

// La terminal POS solo necesita debitar:
public class TerminalPOS {
    public void Cobrar(IDEbitable tarjeta, decimal monto) => tarjeta.Debitar(monto);
}
```

D — Dependency Inversion

ServicioTarjeta no debería instanciar directamente NotificadorEmail ni RegistroMovimientos — depende de detalles técnicos, no de abstracciones.

```
csharp
public interface IRegistroMovimientos {
    void Registrar(string numero, string op, decimal monto);
}
public interface INotificador {
    void Notificar(string dest, string asunto, string cuerpo);
}

// Módulo de alto nivel: depende de abstracciones
public class ServicioTarjeta(IRegistroMovimientos registro, INotificador notificador) {
    public void Debitar(Tarjeta tarjeta, decimal monto) {
        tarjeta.Debitar(monto);
        registro.Registrar(tarjeta.Numero, "Debito", monto);
        notificador.Notificar(tarjeta.EmailTitular, "Débito", $"Debitaste ${monto}");
    }
}

// Composición en Program.cs – cambiar correo por SMS es una línea:
var servicio = new ServicioTarjeta(new RegistroEnArchivo(), new NotificadorEmail());
```

Ganancia: ServicioTarjeta es testeable con mocks sin SMTP ni archivos reales.

Resumen

Principio	Señal de violación	Solución
SRP	Clase con múltiples razones para cambiar	Separar en clases con una responsabilidad
OCP	Agregar variante obliga a modificar código existente	Polimorfismo / herencia / estrategia
LSP	if (x is SubtipoConcreto) por doquier	Rediseñar jerarquía
ISP	OperacionNoSoportadaException en métodos heredados	Interfaces pequeñas y específicas
DIP	new de infraestructura dentro de servicios de negocio	Inyección de dependencias

CAPITULO 18

Patrones en .NET para aplicar SRP

SRP no significa "una clase, un método" — significa que una clase tiene **una razón clara para cambiar**. Cuando una clase cambia por motivos distintos (cambió la BD, cambió la UI, cambió el proveedor externo), mezcla responsabilidades.

Regla de oro: SRP detecta el problema; el patrón ayuda a ubicar la responsabilidad que sobra.

Mapa rápido

Si cambia...	Patrón útil
La creación/selección de objetos	Dependency Injection, Factory
La construcción compleja de un objeto	Builder
La forma de guardar o buscar datos	Repository, Unit of Work
Un criterio de búsqueda combinable	Specification
Un algoritmo o política de negocio	Strategy
El comportamiento según el estado interno	State
Una acción que debe ejecutarse o deshacerse	Command
Una cadena de validaciones o filtros	Chain of Responsibility
Una operación sobre una jerarquía de tipos	Visitor
Una API externa	Adapter
Logging, cache, retry, métricas	Decorator, Proxy
La coordinación entre varios objetos	Mediator, Facade
Reacciones a eventos	Observer / Events
Configuración externa	Options Pattern
Responsabilidades HTTP transversales	Middleware, Filters
Contrato externo vs modelo interno	DTO + Mapper

Dependency Injection

Sin DI, el servicio crea sus dependencias (queda acoplado a JSON, SQL, etc.). Con DI, recibe una abstracción:

```
csharp
public sealed class ContactoService(IContactoRepository repository) {
    public void Agregar(Contacto contacto) {
        var contactos = repository.ObtenerTodos();
        contactos.Add(contacto);
        repository.GuardarTodos(contactos);
    }
}

// Registro:
builder.Services.AddSingleton<IContactoRepository>(new JsonContactoRepository("contactos.json"));
builder.Services.AddScoped<ContactoService>();
```

Repository y Unit of Work

```
csharp
public interface IContactoRepository {
    List<Contacto> ObtenerTodos();
    void GuardarTodos(List<Contacto> contactos);
}

public sealed class JsonContactoRepository(string path) : IContactoRepository {
    public List<Contacto> ObtenerTodos() {
        if (!File.Exists(path)) return [];
        return JsonSerializer.Deserialize<List<Contacto>>(File.ReadAllText(path)) ?? [];
    }
    public void GuardarTodos(List<Contacto> contactos) =>
        File.WriteAllText(path, JsonSerializer.Serialize(contactos, new JsonSerializerOptions {
            WriteIndented = true }));
}

// Para tests: InMemoryContactoRepository : IContactoRepository { /* usa List<Contacto> */ }
```

En EF Core, DbContext cumple el rol de Repository + Unit of Work: rastrea cambios y los confirma con `SaveChangesAsync()`.

Strategy

Reemplaza `if/else` `if` sobre el mismo campo con clases intercambiables:

```
csharp
public interface IExportadorContactos { string Formato { get; } void Exportar(IEnumerable<Contacto> c, string path); }
public sealed class CsvExportadorContactos : IExportadorContactos { public string Formato => "csv"; /* ... */ }
public sealed class JsonExportadorContactos : IExportadorContactos { public string Formato => "json"; /* ... */ }

public sealed class ExportacionService(IEnumerable<IExportadorContactos> exportadores) {
    public void Exportar(string formato, IEnumerable<Contacto> c, string path) =>
        exportadores.Single(e => e.Formato == formato).Exportar(c, path);
}
```

Decorator

Agrega comportamiento transversal (logging, cache) sin modificar la clase original:

csharp

```
public sealed class LoggingContactoRepository(IContactoRepository inner, ILogger logger) :
    IContactoRepository {
    public List<Contacto> ObtenerTodos() {
        logger.LogInformation("Leyendo contactos.");
        return inner.ObtenerTodos();
    }
    public void GuardarTodos(List<Contacto> c) {
        logger.LogInformation("Guardando {n} contactos.", c.Count);
        inner.GuardarTodos(c);
    }
}
```

State

Elimina if/else sobre el mismo campo estado:

csharp

```
public abstract class EstadoPedido {
    public abstract EstadoPedido Pagar();
    public abstract EstadoPedido Cancelar();
}
public sealed class Pendiente : EstadoPedido {
    public override EstadoPedido Pagar() => new Pagado();
    public override EstadoPedido Cancelar() => new Cancelado();
}
public sealed class Pedido {
    public EstadoPedido Estado { get; private set; } = new Pendiente();
    public void Pagar() => Estado = Estado.Pagar();
}
```

Options Pattern (configuración tipada)

csharp

```
public sealed class AgendaOptions { public string ArchivoContactos { get; set; } = "contactos.json"; }

builder.Services.Configure<AgendaOptions>(builder.Configuration.GetSection("Agenda"));

public sealed class JsonContactoRepository(IOptions<AgendaOptions> options) : IContactoRepository {
    private readonly AgendaOptions opts = options.Value;
}
```

DTO + Mapper

Separa el contrato externo del modelo interno:

csharp

```
public sealed class ContactoDto { public int Id; public string NombreCompleto = ""; public string Email = ""; }

public static class ContactoMapper {
    public static ContactoDto ToDto(Contacto c) => new() {
        Id = c.Id, NombreCompleto = $"{c.Nombre} {c.Apellido}", Email = c.Email
    };
}
```

Reglas prácticas

Preguntas para detectar responsabilidades mezcladas: ¿La clase cambia si cambia la BD? ¿Si cambia la UI? ¿Si cambia el proveedor externo? Si varias respuestas son sí, hay mezcla.

Nombres que revelan responsabilidad: `JsonContactoRepository`, `CsvExportadorContactos`, `WhatsAppNotificador`. Nombres que esconden mezcla: `ContactoManager`, `AgendaHelper`, `Utils`, `GeneralService`.

CAPITULO 19

Patrón Decorador en C#

El Decorador agrega comportamiento a un objeto **envolviéndolo** con otro que implementa la **misma interfaz**:

```
csharp
decorador3(decorador2(decorador1(objetoOriginal)))
```

El problema

Una clase que empieza simple termina acumulando logging, validación, retry, cache, métricas... si todo eso se mete adentro, viola SRP y se vuelve imposible de mantener.

La solución

```
csharp
public interface INotificador { void Enviar(string mensaje); }

public class NotificadorEmail : INotificador {
    public void Enviar(string mensaje) => Console.WriteLine($"Email: {mensaje}");
}

// Decorador: misma interfaz, delega al inner
public class NotificadorConLogging : INotificador {
    private readonly INotificador inner;
    public NotificadorConLogging(INotificador inner) => this.inner = inner;

    public void Enviar(string mensaje) {
        Console.WriteLine($"[LOG] Enviando: {mensaje}");
        inner.Enviar(mensaje);
        Console.WriteLine("[LOG] Enviado");
    }
}

// Uso:
INotificador n = new NotificadorConLogging(new NotificadorEmail());
n.Enviar("Hola");
```

El consumidor no sabe si habla con el original o un decorador — solo ve `INotificador`.

Por qué no herencia

Con herencia necesitarías `EmailConLogging`, `EmailConRetry`, `EmailConLoggingYRetry`, `SmsConLogging`... — explosión combinatoria. Con decoradores tenés piezas combinables:

```
csharp
INotificador n = new LoggingNotificador(
    new RetryNotificador(
        new ValidacionNotificador(
            new NotificadorEmail())));
```

Casos de uso típicos

Logging:

```
csharp
public class ProcesadorConLogging : IProcesadorPedidos {
    private readonly IProcesadorPedidos inner;
    public ProcesadorConLogging(IProcesadorPedidos inner) => this.inner = inner;
    public void Procesar(int id) {
        Console.WriteLine($"[LOG] Inicio {id}");
        inner.Procesar(id);
        Console.WriteLine($"[LOG] Fin {id}");
    }
}
```

Cache:

```
csharp
public class BuscadorConCache : IBuscadorProductos {
    private readonly IBuscadorProductos inner;
    private readonly Dictionary<int, string> cache = new();

    public BuscadorConCache(IBuscadorProductos inner) => this.inner = inner;
    public string BuscarNombre(int id) {
        if (cache.TryGetValue(id, out var nombre)) return nombre;
        return cache[id] = inner.BuscarNombre(id);
    }
}
```

Retry:

```
csharp
public class ServicioConRetry : IServicio {
    private readonly IServicio inner;
    private readonly int intentos;
    public ServicioConRetry(IServicio inner, int intentos = 3) {
        this.inner = inner; this.intentos = intentos;
    }
    public void Ejecutar() {
        for (int i = 1; i <= intentos; i++) {
            try { inner.Ejecutar(); return; }
            catch when (i < intentos) { Console.WriteLine($"Intento {i} falló"); }
        }
    }
}
```

Con Dependency Injection

```
csharp
builder.Services.AddScoped<SmtpEmailSender>();
builder.Services.AddScoped<IEmailSender>(sp => {
    IEmailSender inner = sp.GetRequiredService<SmtpEmailSender>();
    inner = new RetryEmailSender(inner);
    inner = new LoggingEmailSender(inner);
    return inner;
});
```


El orden importa

Logging(Validacion(Servicio)) → primero loguea, después valida.

Validacion(Logging(Servicio)) → si falla la validación, ni llega al logging. Ambas pueden ser correctas dependiendo de la intención.

Ejemplo en .NET: Stream

```
csharp
using var archivo = new FileStream("datos.gz", FileMode.Open);
using var gzip    = new GZipStream(archivo, CompressionMode.Decompress);
using var lector  = new StreamReader(gzip, Encoding.UTF8);
```

FileStream → bytes; GZipStream → descompresión; StreamReader → texto. Cada capa agrega una responsabilidad sin modificar la anterior.

Comparación

	Decorador	Herencia
Combinable	Sí	No fácilmente
Desacoplado	Sí (usa interfaz)	No (acoplado a clase base)
Aplicable en runtime	Sí	No
SRP	Respetar	Suele violar

Cuándo usarlo

Cuando una clase acumula: Validar(); LogInicio(); MedirTiempo(); VerificarPermisos(); EjecutarReal(); LogFin(); — es señal de que esas capas deberían ser decoradores.

Usalo para: logging, cache, retry, validación, autorización, métricas, compresión, cifrado. No lo uses si la clase es trivial, si el orden de capas se vuelve confuso, o si estás decorando todo "porque patrones" sin necesidad real.

CAPITULO 20

Patrón Builder en C#

Builder construye objetos complejos **paso a paso**, evitando constructores con demasiados parámetros:

```
csharp
// Sin Builder - ilegible
var pizza = new Pizza("fina", "tomate", "mozzarella", true, false, true);

// Con Builder - intención explícita
Pizza pizza = new PizzaBuilder()
    .ConMasa("fina").ConSalsa("tomate").ConJamon().ConMorrón()
    .Build();
```

Implementación básica

```
csharp
public class PizzaBuilder {
    private string _masa = "media masa", _salsa = "tomate", _queso = "mozzarella";
    private bool _jamón, _aceitunas, _morrón;

    public PizzaBuilder ConMasa(string v) { _masa = v; return this; }
    public PizzaBuilder ConSalsa(string v) { _salsa = v; return this; }
    public PizzaBuilder ConJamon() { _jamón = true; return this; }
    public PizzaBuilder ConAceitunas() { _aceitunas = true; return this; }
    public PizzaBuilder ConMorrón() { _morrón = true; return this; }

    public Pizza Build() => new Pizza(_masa, _salsa, _queso, _jamón, _aceitunas, _morrón);
}
```

Cada método devuelve `this` → permite encadenar (Fluent Interface).

Cuándo usarlo

El Builder evita el **constructor telescópico** (muchas sobrecargas para cada combinación de parámetros opcionales) y los **booleanos ambiguos** (`true`, `false`, `true` — ¿qué es qué?). Conviene cuando:

- hay muchos parámetros opcionales o booleanos
- el objeto requiere validación antes de crearse
- hay colecciones internas o pasos de construcción
- se quiere un objeto final inmutable
- hay recetas o variantes predefinidas

```
csharp
// Con validación en Build()
public Pedido Build() {
    if (string.IsNullOrEmpty(_cliente))
        throw new InvalidOperationException("El cliente es obligatorio.");
    if (_productos.Count == 0)
        throw new InvalidOperationException("Debe agregar al menos un producto.");
    return new Pedido(_cliente, new List<string>(_productos), _metodoPago, _direccion, _esExpress);
}
```

Ejemplos prácticos

SQL dinámico:

```
csharp
string sql = new SqlQueryBuilder()
    .Select("Nombre").Select("Email")
    .From("Clientes")
    .Where("Activo = 1").Where("Edad >= 18")
    .OrderBy("Nombre")
    .Build();
// SELECT Nombre, Email FROM Clientes WHERE Activo = 1 AND Edad >= 18 ORDER BY Nombre
```

Configuración con lógica:

```
csharp
ServidorConfig config = new ServidorConfigBuilder()
    .ConHost("api.midominio.com")
    .UsarHttps() // también cambia el puerto a 443
    .ConTimeout(60)
    .Build();
```

Test Data Builder (muy útil en tests):

```
csharp
// Solo cambio lo que importa para cada test
Usuario admin = new UsuarioTestBuilder().ComoAdmin().Build();
Usuario inactivo = new UsuarioTestBuilder().Inactivo().Build();
Usuario sinEmail = new UsuarioTestBuilder().ConEmail("").Build();
```

Alternativas más simples

Antes de usar Builder, evaluar si no alcanza con:

```
csharp
// Record + with (C#)
var admin = usuarioBase with { Rol = "Admin" };

// Parámetros nombrados
var pizza = new Pizza(masa: "fina", salsa: "tomate", tieneJamon: true, tieneAceitunas: false,
    tieneMorrón: true);

// Object initializer (requiere setters)
var usuario = new Usuario { Nombre = "Ana", Email = "ana@x.com", Rol = "Admin", Activo = true };
```

Builder gana cuando hay validaciones, colecciones, valores por defecto complejos o construcción en múltiples pasos.

Builder en .NET

`WebApplication.CreateBuilder(args) + .Build()` es exactamente este patrón:

csharp

```
var builder = WebApplication.CreateBuilder(args);  
builder.Services.AddControllers();  
builder.Services.AddSwaggerGen();  
var app = builder.Build();
```

Relación con SOLID

SRP: el objeto final representa datos/comportamiento; el Builder se ocupa de construirlo — responsabilidades separadas. OCP: se pueden agregar variantes (`PedidoExpressBuilder`) sin tocar el objeto final.

CAPITULO 21

Patrón Strategy y Patrón Visitor en C#

texto

Strategy = variar el algoritmo.
Visitor = agregar operaciones sobre una jerarquía de objetos.

El problema

Un if/else if que crece con cada variante nueva viola OCP y mezcla lógica:

csharp

```
// * - cada tipo nuevo obliga a modificar esta clase
public decimal Calcular(string tipo, decimal total) => tipo switch {
    "Normal" => total,
    "Premium" => total * 0.90m,
    "Vip" => total * 0.80m,
    - => throw new ArgumentException()
};
```

La solución

Encapsular cada algoritmo en una clase con una interfaz común:

csharp

```
public interface IEstrategiaDescuento { decimal Aplicar(decimal total); }

public class DescuentoNormal : IEstrategiaDescuento { public decimal Aplicar(decimal t) => t; }
public class DescuentoPremium : IEstrategiaDescuento { public decimal Aplicar(decimal t) => t * 0.90m; }
public class DescuentoVip : IEstrategiaDescuento { public decimal Aplicar(decimal t) => t * 0.80m; }

// Contexto - no sabe qué estrategia usa
public class CalculadoraDescuento(IEstrategiaDescuento estrategia) {
    public decimal Calcular(decimal total) => estrategia.Aplicar(total);
}

// Agregar DescuentoBlackFriday no toca nada existente
var calc = new CalculadoraDescuento(new DescuentoPremium());
Console.WriteLine(calc.Calcular(1000)); // 900
```

Con delegates (para lógica simple)

csharp

```
public class CalculadoraDescuento(Func<decimal, decimal> estrategia) {
    public decimal Calcular(decimal total) => estrategia(total);
}

var calcVip = new CalculadoraDescuento(total => total * 0.80m);
```

Casos típicos

- **Exportadores:** CsvExportador, JsonExportador, PdfExportador → todos implementan IExportador
- **Pagos:** PagoConTarjeta, PagoTransferencia, PagoMercadoPago → todos implementan IEstrategiaPago
- **Ordenamiento:** OrdenarPorPrecio, OrdenarPorNombre → todos implementan IEstrategiaOrdenamiento
- **Impuestos:** ImpuestoArgentina, ImpuestoUruguay, ImpuestoExento

Cuándo usarlo / no usarlo

Usarlo: varios algoritmos intercambiables, if/switch repetidos, cambio de comportamiento en runtime, testeo aislado. No usarlo: una sola variante, el if es simple y local, crear clases empeora la legibilidad.

Parte 2: Visitor

El problema

Cuando hay una jerarquía estable de tipos y muchas operaciones externas que se necesitan sobre ella:

```
csharp
// ✗ – Parrafo termina conociendo HTML, Markdown, accesibilidad, estadísticas...
public class Parrafo {
    public string ToHtml() { ... }
    public string ToMarkdown() { ... }
    public int ContarPalabras() { ... }
}
```

La solución: double dispatch

```
csharp
// 1. Interfaz del visitante – un método por tipo
public interface IDocumentoVisitor {
    void Visit(Titulo t);
    void Visit(Parrafo p);
    void Visit(Imagen i);
}

// 2. Clases del modelo: solo exponen Accept
public abstract class Elemento { public abstract void Accept(IDocumentoVisitor v); }

public class Parrafo : Elemento {
    public string Texto { get; set; } = "";
    public override void Accept(IDocumentoVisitor v) => v.Visit(this); // this es Parrafo
}

// 3. Visitor concreto – opera sobre los tipos
public class HtmlVisitor : IDocumentoVisitor {
    public void Visit(Titulo t) => Console.WriteLine($"<h{t.Nivel}>{t.Texto}</h{t.Nivel}>");
    public void Visit(Parrafo p) => Console.WriteLine($"<p>{p.Texto}</p>");
    public void Visit(Imagen i) => Console.WriteLine($"<img src=\"{i.Url}\" alt=\"{i.Alt}\" />");
}

// Uso
var doc = new List<Elemento> { new Titulo { Texto = "Título", Nivel = 1 }, new Parrafo { Texto = "Hola" } };
var visitor = new HtmlVisitor();
foreach (var e in doc) e.Accept(visitor);
```

Para agregar MarkdownVisitor o WordCountVisitor: solo agregar una clase nueva, sin tocar el modelo.

Por qué hace falta Accept

Si llamaras `visitor.Visit(elemento)` directamente con `elemento` declarado como la clase base, el compilador elegiría el overload en tiempo de compilación basándose en el tipo base, no en el tipo real. `Accept` es el primer despacho; `visitor.Visit(this)` dentro de cada clase concreta es el segundo — **double dispatch**.

Visitor vs Pattern Matching moderno

Para jerarquías pequeñas, el pattern matching de C# puede ser más simple:

```
csharp
public static string ToMarkdown(Elemento e) => e switch {
    Titulo t => $"#{t.Nivel} {t.Texto}",
    Parrafo p => p.Texto,
    Imagen i => $"! [{i.Alt}] ({i.Url})",
    _ => throw new NotSupportedException()
};
```

Visitor conviene cuando la operación es grande, hay muchas, necesitás acumular estado, o trabajás con árboles complejos (AST, UI trees).

Comparación directa

Aspecto	Strategy	Visitor
Problema	Cambiar algoritmo	Agregar operaciones sobre jerarquía
Pregunta	¿Cómo hago esto?	¿Qué operación aplico a estos tipos?
Bueno para agregar	Nuevas estrategias	Nuevas operaciones
Malo para agregar	—	Nuevos tipos visitables
Cliente llama	<code>estrategia.Ejecutar()</code>	<code>elemento.Accept(visitor)</code>
Ejemplos	Descuentos, pagos, orden	Documentos, AST, UI tree

CAPITULO 22

IServiceProvider en .NET — Inyección de dependencias

El problema con new

Cuando una clase crea sus dependencias con new:

```
csharp
public class ProcesadorPedidos {
    private readonly EnviadorEmails emails = new EnviadorEmails(); // acoplado
    private readonly RepositorioPedidosSql pedidos = new RepositorioPedidosSql();
}
```

- No se puede testear sin enviar emails reales
- Cambiar el proveedor obliga a modificar la clase de negocio
- Las dependencias son invisibles desde afuera
- No hay control sobre cuántas instancias existen

La solución: inyección por constructor

```
csharp
public class ProcesadorPedidos(IEnviadorEmails emails, IRepositorioPedidos pedidos, IGatewayCobros cobros) {
    public void Procesar(Pedido p) {
        cobros.Cobrar(p);
        pedidos.Guardar(p);
        emails.EnviarConfirmacion(p);
    }
}
```

Las dependencias son explícitas, intercambiables y no las construye la clase. El problema se traslada: ¿quién las construye? → El contenedor de DI.

IServiceCollection + IServiceProvider

```
csharp
// Fase de configuración (antes del Build)
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddScoped<IRepositorioPedidos, RepositorioPedidosSql>();
builder.Services.AddSingleton<IEnviadorEmails, EnviadorEmailsSmtplib>();
builder.Services.AddTransient<CalculadoraImpuestos>();

// Forma con factory (cuando hace falta configuración)
builder.Services.AddSingleton<IEnviadorEmails>(sp => {
    var config = sp.GetRequiredService<IConfiguration>();
    return new EnviadorEmailsSmtplib(config["Smtplib:Host"]!);
});

// Fase de ejecución – el contenedor resuelve el grafo automáticamente
var app = builder.Build();
```

Los tres lifetimes

Lifetime	Cuándo crear	Cuándo usar
AddTransient	Nueva instancia cada vez	Livianos, sin estado, sin recursos
AddScoped	Una por scope (request HTTP)	Repositorios, DbContext, servicios de aplicación
AddSingleton	Una para toda la app	Caches, loggers, clientes HTTP, configuración

Regla de dependencias:

- Singleton solo puede depender de Singleton
- Scoped puede depender de Singleton y Scoped
- Transient puede depender de cualquiera

Captive dependency (bug clásico): un Singleton captura un Scoped en su constructor → el Scoped queda vivo para siempre, compartido entre requests. Activar `ValidateScopes = true` en desarrollo para detectarlo.

Uso correcto en código

El 95% del uso es simplemente declarar dependencias en el constructor:

```
csharp
public class ProcesadorPedidos(
    IRepositorioPedidos pedidos,
    IGatewayCobros cobros,
    ILogger<ProcesadorPedidos> log) {

    public async Task ProcesarAsync(Pedido p) {
        log.LogInformation("Procesando {Id}", p.Id);
        await cobros.CobrarAsync(p);
        await pedidos.GuardarAsync(p);
    }
}
```

csharp - continuacion

```
    }
}
```

Factory cuando el tipo depende del runtime

Único caso legítimo de inyectar `IServiceProvider` directamente:

csharp

```
public class FactoryEstrategiaPago(IServiceProvider sp) : IFactoryEstrategiaPago {
    public IEstrategiaPago Para(string metodo) => metodo switch {
        "tarjeta" => sp.GetRequiredService<EstrategiaTarjeta>(),
        "efectivo" => sp.GetRequiredService<EstrategiaEfectivo>(),
        _ => throw new ArgumentException($"Método desconocido: {metodo}")
    };
}
```

BackgroundService con servicios scoped

Un `BackgroundService` es singleton; no puede inyectar servicios scoped directamente. Solución: `IServiceScopeFactory`:

csharp

```
public class WorkerMails(IServiceScopeFactory scopeFactory) : BackgroundService {
    protected override async Task ExecuteAsync(CancellationToken ct) {
        while (!ct.IsCancellationRequested) {
            using var scope = scopeFactory.CreateScope();
            var repo = scope.ServiceProvider.GetRequiredService<IRepositorioPedidos>();
            await repo.ProcesarPendientesAsync(ct);
            await Task.Delay(TimeSpan.FromMinutes(1), ct);
        }
    }
}
```

Options Pattern (configuración tipada)

csharp

```
public class ConfigSmtip { public string Host { get; set; } = ""; public int Puerto { get; set; } }

// appsettings.json: { "Smtip": { "Host": "smtp.gmail.com", "Puerto": 587 } }
builder.Services.Configure<ConfigSmtip>(builder.Configuration.GetSection("Smtip"));

public class EnviadorEmailsSmtip(IOptions<ConfigSmtip> opt) : IEnviadorEmails {
    private readonly ConfigSmtip cfg = opt.Value;
    // usar cfg.Host, cfg.Puerto
}
```

Anti-patrones

Service Locator — `IServiceProvider` en código de negocio oculta las dependencias y dificulta los tests:

```
csharp
// X Mal
public class ProcesadorPedidos(IServiceProvider sp) {
    public void Procesar(Pedido p) {
        var repo = sp.GetRequiredService<IRepositorioPedidos>(); // dependencias ocultas
    }
}
```

Provider raíz como global estático — rompe los scopes, imposible de testear.

Inicialización al arrancar

```
csharp
var app = builder.Build();
using (var scope = app.Services.CreateScope()) {
    var db = scope.ServiceProvider.GetRequiredService<MiDbContext>();
    db.Database.EnsureCreated();
}
app.Run();
```

Resumen

```
texto
El contenedor responde dos preguntas:
1. ¿Cómo se construye cada dependencia? → IServiceCollection
2. ¿Cuánto tiempo vive? → Transient / Scoped / Singleton

Regla práctica:
Toca BD o infraestructura → Scoped
Sin estado, sin recursos → Transient
Global, thread-safe → Singleton
IServiceProvider en negocio → X (solo en factories e infraestructura)
```

CAPITULO 23

JSON en C#

JSON (*JavaScript Object Notation*) es el formato estándar para intercambiar datos entre programas. Convierte objetos en texto que cualquier lenguaje puede leer.

Los seis tipos de valores

```
json
"texto"          // string – siempre comillas dobles
42 / 3.14 / -7   // number
true / false     // boolean – minúsculas, sin comillas
null            // null – minúsculas, sin comillas
{ "clave": valor } // object – pares clave:valor entre llaves
[ valor, valor, valor ] // array – lista ordenada entre corchetes
```

Reglas de sintaxis frecuentemente olvidadas:

- Claves de objetos: siempre entre comillas dobles
- Sin coma después del último elemento
- true, false, null en minúsculas, sin comillas
- No hay comentarios en JSON

Estructura de objetos y arrays

```
json
{
  "id": 1,
  "nombre": "Ada Lovelace",
  "activa": true,
  "telefonos": ["+54 381 111-1111", "+54 381 222-2222"],
  "direccion": {
    "ciudad": "Tucumán",
    "pais": "Argentina"
  }
}
```

Array de objetos (estructura más común en APIs):

```
json
[
  { "id": 1, "texto": "Estudiar HTTP", "completada": false },
  { "id": 2, "texto": "Hacer el TP", "completada": true }
]
```

System.Text.Json en C#

```
csharp
using System.Text.Json;
```

Serializar (objeto → JSON)

```
csharp
record Tarea(int Id, string Texto, bool Completada);

var tarea = new Tarea(1, "Estudiar HTTP", false);

// Minificado (por defecto)
string json = JsonSerializer.Serialize(tarea);
// { "Id":1, "Texto": "Estudiar HTTP", "Completada":false }

// Con formato legible y camelCase
var opts = new JsonSerializerOptions {
    WriteIndented = true,
    PropertyNamingPolicy = JsonNamingPolicy.CamelCase
};
string json2 = JsonSerializer.Serialize(tarea, opts);
// { "id": 1, "texto": "Estudiar HTTP", "completada": false }
```

Deserializar (JSON → objeto)

```
csharp
string json = "{ \"id\": 1, \"texto\": \"Estudiar HTTP\", \"completada\": false }";
Tarea? tarea = JsonSerializer.Deserialize<Tarea>(json);

// Listas
List<Tarea>? tareas = JsonSerializer.Deserialize<List<Tarea>>(json);
```

`Deserialize<T>` devuelve `T?` porque puede ser `null` si el JSON es el literal `"null"`.

Mapeo de tipos

JSON	C#
string	string
number (entero)	int, long
number (decimal)	double, decimal
boolean	bool
null	null (tipo nullable T?)
object { }	clase o record
array []	List<T>, T[]

Casos frecuentes

Nombres diferentes entre C# (PascalCase) y JSON (camelCase):

```
csharp
// Opción 1: política de nombres
var opts = new JsonSerializerOptions { PropertyNamingPolicy = JsonNamingPolicy.CamelCase };

// Opción 2: insensible a mayúsculas al deserializar
var opts = new JsonSerializerOptions { PropertyNameCaseInsensitive = true };

// Opción 3: atributo por propiedad
record Tarea([property: JsonPropertyName("task_id")] int Id, string Texto);
```

Propiedades opcionales — si no están en el JSON, quedan con el valor por defecto del tipo:

```
csharp
string json = "{ \"id\": 5, \"texto\": \"Nueva\" }";
var t = JsonSerializer.Deserialize<Tarea>(json); // t.Completada == false
```

Enums como strings:

```
csharp
var opts = new JsonSerializerOptions { Converters = { new JsonStringEnumConverter() } };
// { "prioridad": "Alta" } en lugar de { "prioridad": 2 }
```

Propiedades extras en el JSON que no existen en la clase → se ignoran silenciosamente.

Ejemplo completo: agenda en JSON

```
csharp
using System.Text.Json;

const string archivo = "agenda.json";
var opts = new JsonSerializerOptions {
    PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
    PropertyNameCaseInsensitive = true,
    WriteIndented = true
};

var agenda = CargarAgenda(archivo);
agenda.Add(new Contacto(agenda.Count + 1, "Grace", "Hopper", ["+54 381 555-5555"]));
GuardarAgenda(agenda, archivo);

List<Contacto> CargarAgenda(string ruta) {
    if (!File.Exists(ruta)) return [];
    return JsonSerializer.Deserialize<List<Contacto>>(File.ReadAllText(ruta), opts) ?? [];
}

void GuardarAgenda(List<Contacto> c, string ruta) =>
    File.WriteAllText(ruta, JsonSerializer.Serialize(c, opts));

record Contacto(int Id, string Nombre, string Apellido, List<string> Telefonos);
```

Archivo generado:

```
json
[
  {
    "id": 1,
    "nombre": "Grace",
    "apellido": "Hopper",
    "telefonos": ["+54 381 555-5555"]
  }
]
```

Resumen

texto

```
Serializar    → JsonSerializer.Serialize(objeto, opts)
Deserializar  → JsonSerializer.Deserialize<Tipo>(json, opts)
```

Opciones útiles:

```
WriteIndented      = true    → legible
PropertyNamingPolicy = CamelCase → claves en camelCase
PropertyNameCaseInsensitive = true → flexible al deserializar
Converters = { new JsonStringEnumConverter() } → enums como strings
```


CAPITULO 24

Serialización y Repositorio en C#

Persistencia progresiva: texto plano → JSON/XML → base de datos. El patrón Repository oculta el medio de almacenamiento detrás de una interfaz.

IEntity y Repository genérico

```
csharp
public interface IEntity { int Id { get; set; } }

public class Repository<T> where T : class, IEntity {
    private readonly List<T> items;
    private readonly IFileSerializer<T> serializer;
    private readonly string path;

    public Repository(IFileSerializer<T> serializer, string path) {
        this.serializer = serializer;
        this.path = path;
        items = File.Exists(path) ? serializer.Load(path) : [];
    }

    public List<T> GetAll() => items;
    public T? GetById(int id) => items.FirstOrDefault(i => i.Id == id);

    public void Add(T item) {
        item.Id = items.Count == 0 ? 1 : items.Max(i => i.Id) + 1;
        items.Add(item);
        serializer.Save(items, path);
    }

    public void Update(T item) {
        var idx = items.FindIndex(i => i.Id == item.Id);
        if (idx >= 0) { items[idx] = item; serializer.Save(items, path); }
    }

    public void Delete(int id) {
        items.RemoveAll(i => i.Id == id);
        serializer.Save(items, path);
    }
}
```

IFileSerializer — abstraer JSON vs XML

```
csharp
public interface IFileSerializer<T> {
    List<T> Load(string path);
    void Save(List<T> items, string path);
}
```

JSON:

csharp

```
public class JsonSerializer<T> : IFileSerializer<T> {
    private readonly JsonSerializerOptions opts = new() {
        WriteIndented = true, PropertyNamingPolicy = JsonNamingPolicy.CamelCase
    };
    public List<T> Load(string path) =>
        JsonSerializer.Deserialize<List<T>>(File.ReadAllText(path), opts) ?? [];
    public void Save(List<T> items, string path) =>
        File.WriteAllText(path, JsonSerializer.Serialize(items, opts));
}
```

XML:

csharp

```
public class XmlFileSerializer<T> : IFileSerializer<T> {
    private readonly XmlSerializer xml = new(typeof(List<T>));
    public List<T> Load(string path) {
        using var f = File.OpenRead(path);
        return (List<T>?)xml.Deserialize(f) ?? [];
    }
    public void Save(List<T> items, string path) {
        using var f = File.Create(path);
        xml.Serialize(f, items);
    }
}
```

FileSerializerFactory

csharp

```
public static class FileSerializerFactory {
    public static IFileSerializer<T> Create<T>(string extension) => extension switch {
        ".json" => new JsonSerializer<T>(),
        ".xml" => new XmlFileSerializer<T>(),
        _ => throw new NotSupportedException($"Formato no soportado: {extension}")
    };
}
```

Uso:

csharp

```
var serializer = FileSerializerFactory.Create<Contacto>(".json");
var repo = new Repository<Contacto>(serializer, "agenda.json");
repo.Add(new Contacto { Nombre = "Grace", Apellido = "Hopper" });
```

Limitaciones del almacenamiento en archivos

Problema	Archivos	Base de datos
Búsquedas complejas	Filtrar todo en memoria	Consultas SQL indexadas
Concurrencia	Sin control	Transacciones y locks
Relaciones	Manual	Foreign keys
Rendimiento (escala)	Carga todo	Solo lo necesario

Los archivos JSON/XML son útiles para aplicaciones pequeñas o configuración. Para datos que crecen o requieren consultas complejas: SQLite o una base de datos real.

CAPITULO 25

ADO.NET con SQLite en C#

ADO.NET es la capa de acceso a datos de .NET. Trabaja con clases abstractas (DbConnection, DbCommand, DbDataReader) que cada motor implementa concretamente.

```
texto
Tu código → DbConnection (abstracto)
               ↓
           SQLiteConnection / SqlConnection / NpgsqlConnection ...
```

Setup SQLite

```
bash
dotnet add package Microsoft.Data.Sqlite
```

```
csharp
using Microsoft.Data.Sqlite;

const string connStr = "Data Source=agenda.db";
using var conn = new SQLiteConnection(connStr);
conn.Open();
```

using garantiza que la conexión se cierre aunque haya excepciones.

SQL básico — DDL y DML

```
sql
-- DDL: crear tabla
CREATE TABLE IF NOT EXISTS Contactos (
    Id      INTEGER PRIMARY KEY AUTOINCREMENT,
    Nombre  TEXT NOT NULL,
    Email   TEXT
);

-- DML: insertar, actualizar, eliminar
INSERT INTO Contactos (Nombre, Email) VALUES (@nombre, @email);
UPDATE Contactos SET Nombre = @nombre WHERE Id = @id;
DELETE FROM Contactos WHERE Id = @id;

-- DQL: consultar
SELECT Id, Nombre, Email FROM Contactos WHERE Id = @id;
```

CRUD completo en C#

INSERT:

csharp

```
using var cmd = conn.CreateCommand();
cmd.CommandText = "INSERT INTO Contactos (Nombre, Email) VALUES (@nombre, @email)";
cmd.Parameters.AddWithValue("@nombre", contacto.Nombre);
cmd.Parameters.AddWithValue("@email", contacto.Email ?? (object)DBNull.Value);
cmd.ExecuteNonQuery();
```

SELECT:

csharp

```
cmd.CommandText = "SELECT Id, Nombre, Email FROM Contactos";
using var reader = cmd.ExecuteReader();
var lista = new List<Contacto>();
while (reader.Read()) {
    lista.Add(new Contacto {
        Id = reader.GetInt32(0),
        Nombre = reader.GetString(1),
        Email = reader.IsDBNull(2) ? null : reader.GetString(2)
    });
}
```

Siempre usar parámetros — nunca concatenar strings. La concatenación permite SQL injection:

csharp

```
// ✗ Vulnerable
cmd.CommandText = $"SELECT * FROM Contactos WHERE Nombre = '{nombre}'";

// ✓ Seguro
cmd.CommandText = "SELECT * FROM Contactos WHERE Nombre = @nombre";
cmd.Parameters.AddWithValue("@nombre", nombre);
```

Transacciones

Una transacción agrupa operaciones: todas se aplican o ninguna.

csharp

```
using var tx = conn.BeginTransaction();
try {
    // múltiples comandos con cmd.Transaction = tx
    tx.Commit();
} catch {
    tx.Rollback();
    throw;
}
```

Clase Agenda completa

csharp

```

public class AgendaSqlite {
    private readonly string connStr;
    public AgendaSqlite(string db) {
        connStr = $"Data Source={db}";
        using var conn = new SqlConnection(connStr);
        conn.Open();
        using var cmd = conn.CreateCommand();
        cmd.CommandText = """
            CREATE TABLE IF NOT EXISTS Contactos (
                Id INTEGER PRIMARY KEY AUTOINCREMENT,
                Nombre TEXT NOT NULL,
                Email TEXT
            )
            """;
        cmd.ExecuteNonQuery();
    }

    public void Agregar(Contacto c) {
        using var conn = new SqlConnection(connStr);
        conn.Open();
        using var cmd = conn.CreateCommand();
        cmd.CommandText = "INSERT INTO Contactos (Nombre, Email) VALUES (@n, @e)";
        cmd.Parameters.AddWithValue("@n", c.Nombre);
        cmd.Parameters.AddWithValue("@e", c.Email ?? (object)DBNull.Value);
        cmd.ExecuteNonQuery();
    }

    public List<Contacto> ObtenerTodos() {
        using var conn = new SqlConnection(connStr);
        conn.Open();
        using var cmd = conn.CreateCommand();
        cmd.CommandText = "SELECT Id, Nombre, Email FROM Contactos";
        using var r = cmd.ExecuteReader();
        var lista = new List<Contacto>();
        while (r.Read())
            lista.Add(new Contacto { Id = r.GetInt32(0), Nombre = r.GetString(1),
                                     Email = r.IsDBNull(2) ? null : r.GetString(2) });
        return lista;
    }
}

```

Resumen

Clase ADO.NET	Responsabilidad
DbConnection	Abre/cierra la conexión a la BD
DbCommand	Ejecuta SQL (ExecuteNonQuery, ExecuteScalar, ExecuteReader)
DbDataReader	Itera fila por fila los resultados de un SELECT
DbTransaction	Agrupar operaciones — commit o rollback

Cambiar de SQLite a PostgreSQL = cambiar `SqlConnection` por `NpgsqlConnection` y el `connection string`. El resto del código ADO.NET no cambia.

CAPITULO 26

Repository y Unit of Work en C#

texto

Repository = interfaz uniforme para acceder a datos (oculta SQL/JSON/etc.)
Unit of Work = agrupa cambios y los persiste todos juntos al final

Repository — la interfaz

csharp

```
public interface IRepository<T, TId> {  
    T?      GetById(TId id);  
    List<T> GetAll();  
    void    Add(T entity);  
    void    Update(T entity);  
    void    Delete(TId id);  
}
```

El código de negocio solo habla con `IRepository`. No sabe si los datos vienen de un archivo, SQLite o una API.

Unit of Work — la interfaz

csharp

```
public interface IUnitOfWork : IDisposable {  
    IRepository<Contacto, int> Contactos { get; }  
    void SaveChanges();  
}
```

`SaveChanges()` persiste todos los cambios acumulados. Garantiza atomicidad: o todo se guarda, o nada.

Change tracking — EntityState

Para saber qué operación realizar al guardar, se rastrea el estado de cada entidad:

csharp

```
public enum EntityState { Unchanged, Added, Modified, Deleted }  
  
public class TrackedEntry<T> {  
    public T Entity { get; }  
    public EntityState State { get; set; }  
    public TrackedEntry(T entity, EntityState state) { Entity = entity; State = state; }  
}
```

Implementación JSON

```
csharp
public class JsonUnitOfWork : IUnitOfWork {
    private readonly List<TrackedEntry<Contacto>> tracked = [];
    private readonly string path;

    public JsonUnitOfWork(string path) {
        this.path = path;
        Contactos = new TrackedContactoRepository(tracked, path);
    }

    public IRepository<Contacto, int> Contactos { get; }

    public void SaveChanges() {
        var todos = JsonSerializer.Deserialize<List<Contacto>>(File.ReadAllText(path)) ?? [];
        foreach (var entry in tracked) {
            switch (entry.State) {
                case EntityState.Added: todos.Add(entry.Entity); break;
                case EntityState.Modified: var i = todos.FindIndex(c => c.Id == entry.Entity.Id);
                    if (i >= 0) todos[i] = entry.Entity; break;
                case EntityState.Deleted: todos.RemoveAll(c => c.Id == entry.Entity.Id); break;
            }
        }
        File.WriteAllText(path, JsonSerializer.Serialize(todos, new JsonSerializerOptions {
            WriteIndented = true }));
        tracked.Clear();
    }

    public void Dispose() { }
}
```

Implementación SQLite

```
csharp
public class SqliteUnitOfWork : IUnitOfWork {
    private readonly SqliteConnection conn;
    private SqliteTransaction? tx;

    public SqliteUnitOfWork(string connStr) {
        conn = new SqliteConnection(connStr);
        conn.Open();
        tx = conn.BeginTransaction();
        Contactos = new SqliteContactoRepository(conn, tx);
    }

    public IRepository<Contacto, int> Contactos { get; }

    public void SaveChanges() { tx?.Commit(); tx = conn.BeginTransaction(); }

    public void Dispose() { tx?.Rollback(); conn.Dispose(); }
}
```

Si `Dispose()` se llama sin `SaveChanges()`, el `Rollback()` deshace todo — comportamiento seguro por defecto.

Uso

```
csharp
// En el composition root (Program.cs)
using IUnitOfWork uow = new SqliteUnitOfWork("Data Source=app.db");

uow.Contactos.Add(new Contacto { Nombre = "Ada", Email = "ada@example.com" });
uow.Contactos.Add(new Contacto { Nombre = "Grace", Email = "grace@example.com" });
uow.SaveChanges(); // ambas se persisten juntas
```

Resumen

Concepto	Rol
IRepository<T, TId>	CRUD para un tipo de entidad
IUnitOfWork	Agrupar repositorios y persiste con SaveChanges()
EntityState	Unchanged / Added / Modified / Deleted
TrackedEntry<T>	Entidad + su estado pendiente
SaveChanges()	Aplica todos los cambios — commit o escritura a disco

Cambiar de JSON a SQLite = cambiar `new JsonUnitOfWork(...)` por `new SqliteUnitOfWork(...)`.
El código de negocio no cambia.

CAPITULO 27

ADO.NET en C# — Referencia rápida

ADO.NET es la capa de bajo nivel para acceder a bases de datos desde .NET. No es un ORM: el SQL lo escribimos nosotros, ADO.NET lo ejecuta.

texto

Aplicación C# → ADO.NET → Provider (SqlConnection, NpgsqlConnection, etc.) → Base de datos

Setup SQLite

bash

```
dotnet add package Microsoft.Data.Sqlite
```

csharp

```
using Microsoft.Data.Sqlite;
using var conn = new SqliteConnection("Data Source=miapp.db");
conn.Open();
```

using var garantiza que la conexión se cierre aunque haya excepciones.

Los tres métodos de ejecución

csharp

```
using var cmd = conn.CreateCommand();
cmd.CommandText = "...";

// Sin filas de resultado (CREATE, INSERT, UPDATE, DELETE)
int filasAfectadas = cmd.ExecuteNonQuery();

// Un único valor (COUNT, SUM, MAX...)
object? valor = cmd.ExecuteScalar();
int count = Convert.ToInt32(cmd.ExecuteScalar());

// Muchas filas (SELECT)
using var reader = cmd.ExecuteReader();
while (reader.Read()) { /* leer columnas */ }
```

Parámetros — siempre, nunca concatenar

csharp

```
// ✗ Vulnerable a SQL injection
cmd.CommandText = $"SELECT * FROM Contactos WHERE Nombre = '{nombre}'";

// ✓ Seguro
cmd.CommandText = "SELECT * FROM Contactos WHERE Nombre = $nombre";
cmd.Parameters.AddWithValue("$nombre", nombre);
```

CRUD completo

CREATE TABLE:

```
csharp
cmd.CommandText = ""
CREATE TABLE IF NOT EXISTS Contactos (
    Id      INTEGER PRIMARY KEY AUTOINCREMENT,
    Nombre  TEXT NOT NULL,
    Email   TEXT
)"";
cmd.ExecuteNonQuery();
```

INSERT:

```
csharp
cmd.CommandText = "INSERT INTO Contactos (Nombre, Email) VALUES ($n, $e); SELECT last_insert_rowid();";
cmd.Parameters.AddWithValue("$n", contacto.Nombre);
cmd.Parameters.AddWithValue("$e", contacto.Email ?? (object)DBNull.Value);
int nuevoId = Convert.ToInt32(cmd.ExecuteScalar());
```

SELECT → objetos:

```
csharp
cmd.CommandText = "SELECT Id, Nombre, Email FROM Contactos ORDER BY Nombre";
using var reader = cmd.ExecuteReader();
var lista = new List<Contacto>();
while (reader.Read()) {
    lista.Add(new Contacto {
        Id      = reader.GetInt32(0),
        Nombre  = reader.GetString(1),
        Email   = reader.IsDBNull(2) ? null : reader.GetString(2)
    });
}
```

UPDATE / DELETE:

```
csharp
cmd.CommandText = "UPDATE Contactos SET Nombre=$n WHERE Id=$id";
cmd.Parameters.AddWithValue("$n", contacto.Nombre);
cmd.Parameters.AddWithValue("$id", contacto.Id);
cmd.ExecuteNonQuery();

cmd.CommandText = "DELETE FROM Contactos WHERE Id=$id";
cmd.Parameters.Clear();
cmd.Parameters.AddWithValue("$id", id);
cmd.ExecuteNonQuery();
```

■ Sin WHERE en UPDATE/DELETE → se modifican **todas** las filas.

NULL en ADO.NET

```
csharp
// Al leer
string? email = reader.IsDBNull(2) ? null : reader.GetString(2);

// Al escribir
cmd.Parameters.AddWithValue("$email", contacto.Email ?? (object)DBNull.Value);
```

Transacciones

```
csharp
using var tx = conn.BeginTransaction();
try {
    cmd.Transaction = tx;
    // múltiples operaciones
    tx.Commit();
} catch {
    tx.Rollback();
    throw;
}
```

Cambiar de base de datos

Solo cambia el provider y el connection string. El código ADO.NET (`cmd.ExecuteNonQuery()`, `reader.Read()`, etc.) no cambia:

Base	Provider	Connection string
SQLite	Microsoft.Data.Sqlite	Data Source=app.db
PostgreSQL	Npgsql	Host=localhost;Database=midb;Username=...
SQL Server	Microsoft.Data.SqlClient	Server=...;Database=...;...

ADO.NET vs Dapper vs EF Core

Concepto	ADO.NET	Dapper	EF Core
SQL	Manual	Manual	Generado
Mapeo filas→objetos	Manual	Automático	Automático
Tracking	No	No	Sí
Cuándo usarlo	Aprender / control total	SQL propio + menos código	Proyectos grandes con relaciones

CAPITULO 28

Dapper en C# — Referencia completa

Dapper extiende `DbConnection` para mapear filas SQL a objetos C# automáticamente. El SQL lo escribís vos; Dapper hace el mapping.

```
bash
dotnet add package Dapper
dotnet add package Dapper.Contrib # CRUD sin SQL manual
dotnet add package Microsoft.Data.Sqlite
```

Cómo funciona el mapping

Dapper compara nombres de columnas con nombres de propiedades (insensible a mayúsculas):

```
sql
-- Tabla: columnas Id, Nombre, Apellido, Email
```

```
csharp
class Contacto {
    public int Id { get; set; }
    public string Nombre { get; set; } = "";
    public string Apellido { get; set; } = "";
    public string Email { get; set; } = "";
}
```

Los nombres coinciden → Dapper conecta automáticamente.

Métodos principales

```
csharp
using DbConnection db = new SqliteConnection("Data Source=app.db");
db.Open();

// Muchos resultados
IEnumerable<Contacto> todos = db.Query<Contacto>("SELECT * FROM Contactos");

// Un resultado o null
Contacto? c = db.QuerySingleOrDefault<Contacto>(
    "SELECT * FROM Contactos WHERE Id = @id", new { id = 5 });

// Un valor escalar
int total = db.QuerySingle<int>("SELECT COUNT(*) FROM Contactos");

// INSERT / UPDATE / DELETE
db.Execute("DELETE FROM Contactos WHERE Id = @id", new { id = 3 });
```

Parámetros — siempre @, nunca concatenar

```
csharp
// ✓ Seguro
db.Query<Contacto>(
    "SELECT * FROM Contactos WHERE Apellido = @apellido",
    new { apellido = "Díaz" });

// ✗ SQL injection
db.Query<Contacto>($"SELECT * FROM Contactos WHERE Apellido = '{apellido}'");
```

Se pueden pasar como objeto anónimo, diccionario, o instancia de la clase:

```
csharp
db.Execute("INSERT INTO Contactos (Nombre, Email) VALUES (@Nombre, @Email)", contacto);
```

Dapper.Contrib — CRUD sin SQL

```
csharp
using Dapper.Contrib.Extensions;

// INSERT → devuelve Id generado
long id = db.Insert(new Contacto { Nombre = "Elena", Email = "elena@mail.com" });
db.Insert(new List<Contacto> { ... }); // múltiples a la vez

// SELECT
Contacto? c = db.Get<Contacto>(id);
IEnumerable<Contacto> todos = db.GetAll<Contacto>();

// UPDATE / DELETE
c!.Email = "nuevo@mail.com";
db.Update(c);
db.Delete(c);
db.DeleteAll<Contacto>();
```

Convenciones: pluraliza la clase para la tabla (Contacto → Contactos), usa propiedad Id como PK.

Atributos para sobrescribir convenciones:

```
csharp
[Table("mis_contactos")]
class Contacto {
    [Key] // PK autogenerada (AUTOINCREMENT) – omitida en INSERT
    public int Id { get; set; }

    // [ExplicitKey] // PK asignada por nosotros – incluida en INSERT

    [Computed] // ignorada en INSERT/UPDATE (columna calculada)
    public string NombreCompleto => $"{Nombre} {Apellido}";
}
```

Filtrar en la base, no en memoria

```
csharp
// ✓ Eficiente – solo viajan las filas que coinciden
db.Query<Contacto>(
    "SELECT * FROM Contactos WHERE Nombre > @nombre", new { nombre = "Carlos" });

// ✗ Ineficiente – trae todo a memoria y filtra en C#
db.GetAll<Contacto>().Where(c => c.Nombre > "Carlos");
```

Ejemplo completo

```
csharp
using DbConnection db = new SqlConnection("Data Source=contactos.db");
db.Open();

// Crear tabla (ADO.NET puro para DDL)
using var cmd = db.CreateCommand();
cmd.CommandText = """
CREATE TABLE IF NOT EXISTS Contactos (
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    Nombre TEXT NOT NULL, Apellido TEXT NOT NULL, Email TEXT NOT NULL
)""";
cmd.ExecuteNonQuery();

// Insertar con Dapper.Contrib
db.Insert(new Contacto { Nombre = "Ana", Apellido = "García", Email = "ana@mail.com" });
db.Insert(new Contacto { Nombre = "Beto", Apellido = "López", Email = "beto@mail.com" });

// Listar
foreach (var c in db.GetAll<Contacto>())
    Console.WriteLine($"{c.Id}: {c.Nombre} {c.Apellido}");

// Consultar con Dapper puro (filtrado en servidor)
var resultado = db.Query<Contacto>(
    "SELECT * FROM Contactos WHERE Apellido = @ap", new { ap = "García" });

// Contar
int total = db.QuerySingle<int>("SELECT COUNT(*) FROM Contactos");
```

Cuándo usar qué

Situación	Herramienta
CRUD simple	Dapper.Contrib
JOINS, filtros, consultas complejas	Dapper puro (Query<T>)
Control total, sin abstracción	ADO.NET
Relaciones, tracking, migraciones	EF Core

Dapper no reemplaza ADO.NET — lo usa internamente. Entender ADO.NET ayuda a entender qué hace Dapper.

CAPITULO 29

Entrada, salida y redirección en programas de consola

Un programa de consola trabaja, en general, con tres flujos:

- `stdin`: entrada estándar. Normalmente es el teclado.
- `stdout`: salida estándar. Normalmente es la pantalla.
- `stderr`: salida de error. También suele verse en la pantalla.

En C#, esos flujos se usan a través de `Console.In`, `Console.Out` y `Console.Error`, o con los métodos más comunes:

```
csharp
string? linea = Console.ReadLine();
// linea = Console.In.ReadLine(); // equivalente a Console.ReadLine()
Console.WriteLine(linea);
// Console.Out.WriteLine(linea); // equivalente a Console.WriteLine(linea)
Console.Error.WriteLine("Mensaje de error");
```

La idea importante es esta: si un programa lee desde `Console.ReadLine()` y escribe con `Console.WriteLine()`, entonces la terminal puede redirigir su entrada y su salida sin cambiar el código.

1. Redirección desde la terminal

Pipe: |

El operador `|` conecta la salida estándar del comando de la izquierda con la entrada estándar del comando de la derecha.

```
bash
echo "hola" | dotnet mayusculas.cs
cat nombres.txt | dotnet mayusculas.cs
```

o en Windows:

```
cmd
echo hola | dotnet mayusculas.cs
type nombres.txt | dotnet mayusculas.cs
```

Interpretación:

- el programa de la izquierda escribe en `stdout`
- el programa de la derecha lee desde `stdin`

En otras palabras, `|` sirve para encadenar programas.

2. Redirección de entrada: `<`

El operador `<` hace que la entrada del programa venga desde un archivo en lugar del teclado.

```
bash
dotnet mayusculas.cs < nombres.txt
```

En ese caso:

- `Console.ReadLine()` ya no espera que el usuario escriba
- el programa va leyendo las líneas de `nombres.txt`

3. Redirección de salida: `>`

El operador `>` manda la salida del programa a un archivo en lugar de mostrarla en pantalla.

```
bash
dotnet mayusculas.cs > salida.txt
```

En ese caso:

- `Console.WriteLine()` no escribe en la terminal
- escribe dentro de `salida.txt`

Si el archivo ya existe, `>` lo reemplaza.

4. Combinando entrada y salida

Se pueden combinar ambas cosas:

```
bash
dotnet mayusculas.cs < nombres.txt > salida.txt
```

También se puede combinar con `|`:

```
bash
cat nombres.txt | dotnet mayusculas.cs > salida.txt
```

Resumen mental:

- `|`: conecta un programa con otro
- `<`: toma la entrada desde un archivo
- `>`: envía la salida a un archivo

5. Cómo se programa para que eso funcione

Para que un programa soporte bien la redirección desde la terminal, conviene que lea desde `Console.In` o `Console.ReadLine()` y escriba en `Console.Out` o `Console.WriteLine()`.

Ejemplo simple:

```
csharp
string? linea;
while ((linea = Console.ReadLine()) is not null) {
    Console.WriteLine(linea.ToUpper());
}
```

Ese mismo programa puede funcionar:

- interactivo, si el usuario escribe a mano
- con `<`, si la entrada viene de un archivo
- con `|`, si la entrada viene de otro programa
- con `>`, si la salida se guarda en un archivo

6. Redirección programática

Además de la redirección desde la terminal, también se puede hacer desde el propio código.

Para eso se usan `Console.SetIn(...)` y `Console.SetOut(...)`.

```
csharp
using System.IO;
using static System.Console;

SetIn(new StreamReader("entrada.txt"));
SetOut(new StreamWriter("salida.txt") { AutoFlush = true });

string? linea;
while ((linea = ReadLine()) is not null) {
    WriteLine(linea.ToUpper());
}
```

Qué hace este código:

- cambia la entrada estándar para que lea desde `entrada.txt`
- cambia la salida estándar para que escriba en `salida.txt`
- el resto del programa sigue usando `ReadLine()` y `WriteLine()` como siempre

El `AutoFlush = true` hace que cada `WriteLine()` se escriba inmediatamente en el archivo.

7. Detectar si la consola está redirigida

También se puede consultar si la entrada o la salida están redirigidas:

```
csharp
if (Console.IsInputRedirected) {
    Console.WriteLine("La entrada está redirigida");
} else {
    Console.WriteLine("La entrada no está redirigida");
}
```

Esto es útil si un programa necesita comportarse distinto cuando recibe datos desde teclado o desde un archivo.

8. Diferencia entre redirección externa e interna

Redirección desde la terminal:

- la decide quien ejecuta el programa
- el programa sigue siendo general y reutilizable
- ejemplos: |, <, >

Redirección programática:

- la decide el propio código
- sirve para demos, pruebas o automatización
- ejemplos: `Console.SetIn(...)`, `Console.SetOut(...)`

9. Idea clave para recordar

Si un programa usa bien `Console.ReadLine()` y `Console.WriteLine()`, entonces la terminal puede decidir de dónde viene la entrada y adónde va la salida.

Eso permite usar el mismo programa:

- de forma interactiva
- leyendo desde archivos
- escribiendo a archivos
- encadenado con otros programas

CAPITULO 30

Terminal.Gui: apuntes para construir aplicaciones TUI en C#

Estos apuntes usan **Terminal.Gui v2**, la librería de .NET para construir interfaces de terminal modernas. Vamos progresivamente desde el ejemplo más chico (dos paneles) hasta una agenda completa con persistencia en **SQLite** vía **Dapper.Contrib** e importación/exportación a **JSON**.

El objetivo es que al final puedas construir una aplicación básica de gestión con:

- ventanas y paneles (`Window`, `FrameView`);
- diálogos modales (`Dialog`, `Dialog<T>`);
- listas con búsqueda activa (`ListView`, `TextField`);
- vista de detalle con formato (`Markdown`);
- menú desplegable superior (`MenuBar`, `MenuItem`);
- manejo de teclado vía atajos en el menú;
- persistencia en SQLite usando `Dapper.Contrib` como mini-ORM;
- importación y exportación de los datos en JSON.

Archivos de clase

La progresión que vamos siguiendo está en estos archivos .cs autocontenidos (file-based, .NET 10):

Archivo	Tema principal	Sección
25.2-paneles.cs	Window + FrameView lado a lado	6
25.3-clases.cs	Subclase de Window + Dialog inline	7
25.4-clases-dialogo.cs	Dialog refactorizado a clase aparte	7
25.5-dialogo-contacto.cs	Dialog<T> tipado con validación	8
25.6-listview.cs	ListView + Markdown + búsqueda activa	11
25.7-menues.cs	MenuBar con Runnable como raíz	13

Cada archivo se ejecuta directo con `dotnet run archivo.cs`, sin proyecto ni .csproj.

El ejemplo final integrado está en `agenda-terminal-gui.cs` y se ejecuta con:

```
bash
dotnet run agenda-terminal-gui.cs
```

También se puede pasar la ruta del archivo SQLite:

```
bash
dotnet run agenda-terminal-gui.cs -- agenda.db
```

1. La idea: una interfaz gráfica dentro de la terminal

Una aplicación de consola tradicional imprime texto y lee texto:

```
csharp
Console.Write("Nombre: ");
string nombre = Console.ReadLine() ?? "";
```

Eso alcanza para programas chicos, pero se vuelve incómodo cuando hay muchas opciones, listas, formularios, búsqueda, edición y navegación.

Una **TUI** (*Terminal User Interface*) usa la terminal completa como superficie visual. En lugar de imprimir líneas una debajo de otra, dibuja ventanas, paneles, menús, botones, listas y formularios.

La terminal sigue siendo texto, pero la experiencia deja de ser una secuencia lineal de preguntas y respuestas.

Terminal.Gui resuelve varias tareas difíciles:

- dibujar correctamente en la pantalla;
- redibujar cuando cambia el tamaño de la terminal;
- manejar foco;
- capturar teclado y mouse;
- organizar componentes visuales;
- abrir diálogos modales;
- evitar que el programador tenga que usar `Console.SetCursorPosition` manualmente.

La idea central es esta:

```
texto
Application
  ■■■ Runnable / Window
    ■■■ MenuBar
    ■■■ FrameView
      ■ ■■■ ListView
      ■■■ FrameView
      ■ ■■■ Label
      ■■■ Dialog, cuando se abre uno
```

La aplicación no "imprime pantallas". Construye un árbol de objetos visuales. Terminal.Gui se encarga de dibujarlo, mover el foco, recibir eventos y redibujar cuando haga falta.

2. Filosofía de diseño

Terminal.Gui se parece más a WinForms, WPF o Avalonia que a `Console.WriteLine`.

Sus decisiones de diseño principales son:

Todo lo visible es un View

Un botón, una etiqueta, una lista, una ventana y un diálogo son View o derivan de View.

Eso significa que todos comparten conceptos:

- posición: X, Y;
- tamaño: Width, Height;
- padre visual: SuperView;
- hijos visuales: SubViews;
- foco;
- eventos;
- dibujo;
- layout.

Ejemplo:

```
csharp
Label label = new() {
    Text = "Buscar:",
    X = 1,
    Y = 2
};

TextField input = new() {
    X = Pos.Right(label) + 1,
    Y = Pos.Top(label),
    Width = Dim.Percent(40)
};
```

El layout se expresa, no se calcula a mano

En vez de calcular coordenadas con enteros todo el tiempo, Terminal.Gui usa:

- Pos para posiciones;
- Dim para tamaños.

Eso permite decir:

```
csharp
X = Pos.Right(label) + 1;
Width = Dim.Fill(2);
```

En castellano: "poné este campo una columna a la derecha del label y hacelo ocupar el resto del ancho, dejando dos columnas libres".

La aplicación es reactiva

No hay un `while` manual preguntando qué quiere hacer el usuario.

`TerminalGui` ejecuta un bucle de eventos:

1. dibuja la interfaz;
2. espera una tecla, mouse o evento interno;
3. manda ese evento al componente con foco;
4. ejecuta handlers;
5. redibuja lo que cambió.

Por eso escribimos código como este:

```
csharp
buscar.TextChanged += (_, _) => Actualizar();
btnGuardar.Accepting += (_, _) => Guardar();
```

El teclado es un ciudadano de primera clase

Una buena TUI se puede usar sin mouse. `TerminalGui` soporta:

- foco con `Tab`;
- menús con `F10` o `Alt+letra`;
- hotkeys con `_`;
- atajos como `Ctrl+S`;
- eventos de teclado;
- comandos.

La interfaz no debería contener la lógica de negocio

El `Button` no debería saber cómo se persiste un contacto. El botón llama a un método de la ventana, y ese método delega en una clase que maneja datos.

En el ejemplo final integrado hay cuatro responsabilidades separadas:

- `AgendaWindow` organiza la UI y coordina acciones;
- `Editar : Dialog<Contacto?>` edita un contacto;
- `SqliteAgendaStore` persiste contactos en SQLite con `Dapper.Contrib`;
- `JsonAgendaIO` lee y escribe JSON para importación y exportación;
- `Contacto` representa los datos.

3. Crear el proyecto

Para clase conviene una app C# de archivo único (file based) con .NET 10. Las dependencias se declaran al principio del .cs:

```
csharp
#!/usr/bin/env -S dotnet run
#:package Terminal.Gui@2.0.1
#:property PublishAot=false

using Terminal.Gui.App;           // Application, IApplication, Runnable
using Terminal.Gui.Drawing;       // LineStyle, colores y dibujo
using Terminal.Gui.Input;         // Key, Command
using Terminal.Gui.ViewBase;      // View, Pos, Dim
using Terminal.Gui.Views;         // Window, FrameView, Button, TextField, etc.
```

Luego se ejecuta:

```
bash
dotnet run agenda-terminal-gui.cs
```

O, si el archivo tiene el shebang `#!/usr/bin/env -S dotnet run` y permisos de ejecución, directamente:

```
bash
./agenda-terminal-gui.cs
```

4. Ciclo de vida de la aplicación

El ciclo de vida mínimo es:

```
csharp
using( IApplication app = Application.Create().Init()){
    Window mainWindow = new() { Title = "Tutorial Terminal.Gui v2" };
    app.Run(mainWindow);
}
```

Qué pasa ahí:

1. `Application.Create()` crea una instancia de aplicación.
2. `Init()` inicializa el driver de terminal, el loop principal y el manejo de input.
3. `Run(...)` ejecuta una pantalla, ventana o diálogo.
4. Cuando la pantalla pide detenerse, `Run` termina.
5. El `using` llama a `Dispose()` y restaura la terminal.

5. Configuración

Para una primera aplicación no es obligatorio configurar nada. Conviene concentrarse primero en layout, eventos y arquitectura.

Hay dos niveles de configuración que conviene conocer:

Nivel 1: ajustes puntuales en código. Son una o dos líneas antes de crear la aplicación, para cambiar defaults visuales:

```
csharp
Menu.DefaultBorderStyle = LineStyle.Rounded;
```

Eso le da bordes redondeados a todos los submenús sin tocar nada más. Es lo que usa el ejemplo de menús de la sección 13.

Nivel 2: ConfigurationManager para temas, colores y bindings globales:

```
csharp
using Terminal.Gui.Configuration;

ConfigurationManager.RuntimeConfig = ""
{
  "Theme": "Dark",
  "Application.DefaultKeyBindings": { "Quit": { "All": ["Ctrl+Q"] } }
}
"";

ConfigurationManager.Enable(ConfigLocations.Runtime);

using IApplication app = Application.Create().Init();
```

Regla práctica:

- si estás aprendiendo, no configures demasiado;
- si la app ya funciona, agregá tema, colores y bindings globales;
- si algo de configuración no se aplica, revisá que `ConfigurationManager.Enable(...)` ocurra **antes** de `Application.Create().Init()`.

6. Vistas como organizador visual

Una View puede ser:

- un control concreto: `Label`, `TextField`, `Button`, `CheckBox`;
- un contenedor: `FrameView`, `Window`, `Dialog`;
- una pantalla propia creada por nosotros.

Se agregan vistas con `Add`. Un primer ejemplo concreto, con dos paneles lado a lado:

```
csharp
using( IApplication app = Application.Create().Init()){
    Window mainWindow = new() { Title = "Tutorial Terminal.Gui v2" };

    FrameView maestro = new() {
        Title = "Maestro",
        X = 0, Y = 0,
        Width = Dim.Percent(25), Height = Dim.Fill()
    };
    FrameView detalle = new() {
        Title = "Detalle",
        X = Pos.Right(maestro) + 5,
        Width = Dim.Fill(), Height = Dim.Fill()
    };
    mainWindow.Add(maestro, detalle);

    app.Run( mainWindow);
}
```

Cada panel se vuelve el `SuperView` de lo que pongamos dentro. Lo que agreguemos al panel se posiciona dentro del área interna del panel, no contra toda la pantalla.

`Pos.Right(maestro) + 5` quiere decir "ubicá este panel cinco columnas a la derecha del borde derecho del panel maestro". Si después corremos el maestro o cambiamos su ancho, el detalle se reacomoda solo.

Esto permite pensar por capas:

```
texto
MainWindow
  MenuBar
  FrameView "Maestro"
    Label encabezado
    ListView
  FrameView "Detalle"
    Markdown o Label multilinea
    Label de estado
```

Esa estructura visual también ayuda a separar responsabilidades en el código.

7. Runners y pantallas ejecutables

En `Terminal.Gui` no se ejecuta cualquier objeto: se ejecuta un objeto que participa del ciclo de vida de la app. Para eso existe `IRunnable`; las clases cómodas para empezar son `Window` y `Runnable`.

Primer paso: todo en el constructor

Antes de empezar a separar en clases, conviene ver un ejemplo donde todo vive adentro del constructor de una `Window`. Aparece un botón centrado y, al apretarlo, se abre un diálogo de confirmación:

```
csharp
using( IApplication app = Application.Create().Init()){
    app.Run<Agenda>();
}

class Agenda : Window {
    public Agenda() {
        Title = "Agenda";
        Width = Dim.Fill();
    }
}
```

csharp - continuacion

```

    Height = Dim.Fill();

    Button boton = new() {
        Text = "¡Confirmar!",
        X = Pos.Center(),
        Y = Pos.Center()
    };

    Label resultado = new() {
        Text = "",
        X = Pos.Center(),
        Y = Pos.Bottom(boton) + 1
    };

    // Definimos un diálogo de confirmación que se mostrará al hacer click en el botón
    Dialog confirmar = new() {
        Title = "Confirmar acción",
        Width = 60,
        Height = 10
    };

    confirmar.Add(new Label() {
        Text = "¿Estás seguro de que quieres confirmar esta acción?",
        X = Pos.Center(),
        Y = 2
    });
    confirmar.AddButton(new Button(){Title = "Cancelar"}); // Resultado 0
    confirmar.AddButton(new Button(){Title = "Confirmar"}); // Resultado 1

    boton.Accepting += (_,_) => {
        // Apila el diálogo de confirmación y espera a que el usuario elija una opción
        App!.Run(confirmar);
        if (confirmar.Result == 1) {
            resultado.Text = "Resultado: ¡Confirmado!";
        } else {
            resultado.Text = "Resultado: Cancelado.";
        }
    };

    Add(boton, resultado);
}

```

Tres cosas importantes ya están acá:

- El diálogo se ejecuta con `App!.Run(confirmar)`. Eso apila el diálogo encima de la ventana y espera a que el usuario elija una opción.
- El orden en que se agregan los botones define el `Result`: el primero es 0, el segundo es 1. Ese resultado se lee después de que `Run` retorna.
- La ventana de atrás no recibe input mientras el diálogo está abierto.

Segundo paso: el diálogo como clase aparte

Cuando un diálogo crece o cuando se reutiliza, conviene sacarlo a su propia clase. El comportamiento es el mismo, pero el código de la ventana queda más limpio:

csharp

```

using( IApplication app = Application.Create().Init()){
    app.Run<Agenda>();
}

class Agenda : Window {
    public Agenda() {

```

csharp - continuacion

```

    Title = "Agenda";

    Button boton    = new() { Text = "¡Confirmar!", X = Pos.Center(), Y = Pos.Center() };
    Label resultado = new() { Text = "Resultado: ", X = Pos.Center(), Y = Pos.Bottom(boton) + 1 };

    boton.Accepting += (_,_) => {
        using Confirmar confirmar = new("¿Estás seguro de que quieres confirmar esta acción?");
        App!.Run(confirmar);
        resultado.Text = "Resultado " + (confirmar.Result==1 ? "¡Confirmado!" : "Cancelado.");
    };

    Add(boton);
    Add(resultado);
}

class Confirmar : Dialog {
    public Confirmar(string Mensaje){
        Title = "Confirmar acción";
        Width = 60; Height = 10;

        Add(new Label() { Text = Mensaje, X = Pos.Center(), Y = 2 });

        AddButton(new Button(){Title = "Cancelar" }); // Resultado 0
        AddButton(new Button(){Title = "Confirmar"}); // Resultado 1
    }
}

```

Mismo comportamiento, pero ahora Confirmar se puede reutilizar desde cualquier otra ventana y aceptar un mensaje distinto cada vez.

Tres formas equivalentes de correr una ventana

Las tres líneas siguientes producen el mismo resultado. Conviene conocer las tres porque aparecen mezcladas en código existente:

csharp

```

using (IApplication app = Application.Create().Init()) {
    app.Run(new Agenda());
}

```

csharp

```

using (IApplication app = Application.Create().Init()) {
    app.Run<Agenda>();
}

```

csharp

```

using IApplication app = Application.Create().Init();
using Agenda agenda = new();
app.Run(agenda);

```

La diferencia entre `app.Run(new Agenda())` y `app.Run<Agenda>()` es estética: en el segundo, la app construye la instancia por nosotros usando el constructor sin parámetros.

Cómo funciona la apertura de un diálogo

```
csharp
// Crea la caja de dialogo
Confirmar confirmar = new("¿Estás seguro de que quieres confirmar esta acción?");
// 'Apila' la caja de dialogo y espera a que el usuario elija una opción
App!.Run(confirmar);
// Al cerrar el diálogo, el control vuelve a esta línea, y se puede leer el resultado
resultado.Text = "Resultado " + (confirmar.Result==1 ? "¡Confirmado!" : "Cancelado.");
```

`App!.Run(dialog)` abre un loop modal encima del actual. Mientras el diálogo está abierto, la ventana de atrás no recibe input. Cuando el diálogo se cierra (porque el usuario apretó un botón sin marcar `e.Handled = true`), el control vuelve a la línea siguiente.

8. Ventanas, paneles y diálogos

Hay tres contenedores que conviene distinguir.

Window

Sirve como ventana principal tradicional. Tiene borde y título.

Runnable

Es útil como pantalla ejecutable sin marco. Puede ocupar todo el terminal y contener otros views. Se usa típicamente cuando se quiere combinar `MenuBar` con una `Window` adentro (ver sección sobre menús).

FrameView

Es un panel con borde y título, ideal para organizar áreas dentro de una pantalla.

```
csharp
FrameView detalle = new() {
    Title = "Detalle",
    X = Pos.Right(maestro) + 5,
    Width = Dim.Fill(), Height = Dim.Fill()
};
```

Dialog<T>

Es una ventana modal **tipada**: además de mostrarse en primer plano y bloquear lo de atrás, retorna un valor del tipo `T` cuando se cierra. Es el contenedor ideal para edición, confirmaciones o formularios cortos que necesitan devolver datos.

El ejemplo completo de un diálogo de edición de contactos:

csharp

```

using( IApplication app = Application.Create().Init()){
    Window mainWindow = new() { Title = "Tutorial Terminal.Gui v2" };

    Button editar = new() { Title = "Editar contacto", X = Pos.Center(), Y = Pos.Center() };
    Label resultado = new() { Text = "Resultado: Ninguno", X = Pos.Center(), Y = Pos.Bottom(editar) };

    editar.Accepting += (_,_) => {
        app.Run<Editar>();
        resultado.Text = $"Resultado: {app.GetResult<Contacto>}";
    };

    mainWindow.Add(editar);
    mainWindow.Add(resultado);

    app.Run( mainWindow);
}

record Contacto(string Nombre, string Telefono) {
    public override string ToString() => $"{Nombre} ({Telefono})";
};

class Editar : Dialog<Contacto?> {
    public Editar(){
        Title = "Editar Contacto";
        Width = 60; Height = 10;

        // Agrego el campo Nombre
        Label labelNombre = new() { Text = "Nombre:", X = 1, Y = 1 };
        TextField inputNombre = new() { X = Pos.Right(labelNombre) + 1, Y = 1, Width = 30 };
        Add(labelNombre, inputNombre);

        // Agrego el campo Teléfono
        Label labelTelefono = new() { Text = "Teléfono:", X = 1, Y = 3 };
        TextField inputTelefono = new() { X = Pos.Right(labelTelefono) + 1, Y = 3, Width = 15 };
        Add(labelTelefono, inputTelefono);

        // Si cancelo retorno null
        Button btnCancelar = new() { Title = "Cancelar" };
        btnCancelar.Accepting += (_,_) => Result = null;
        AddButton(btnCancelar);

        // Si confirmo, valido y retorno el contacto
        Button btnConfirmar = new() { Title = "Confirmar" };
        btnConfirmar.Accepting += (_, e) => {
            if(string.IsNullOrWhiteSpace(inputNombre.Text) || inputNombre.Text.Length < 3) {
                e.Handled = true; // Evito que el diálogo se cierre automáticamente
                inputNombre.SetFocus();
                return;
            }

            if(string.IsNullOrWhiteSpace(inputTelefono.Text) || !inputTelefono.Text.All( char.IsDigit ))
            {
                e.Handled = true; // Evito que el diálogo se cierre automáticamente
                inputTelefono.SetFocus();
                return;
            }

            Result = new Contacto(inputNombre.Text, inputTelefono.Text);
        };
        AddButton(btnConfirmar);
    }
}

```

Hay varias decisiones de diseño que valen la pena leer despacio:

- **El tipo del diálogo es `Dialog<Contacto?>`.** Eso obliga a setear `Result` antes de cerrar y permite que la ventana de afuera obtenga el contacto sin armar canales raros.
- **El botón Cancelar setea `Result = null`** y deja que el diálogo se cierre normalmente. Eso es lo que diferencia un cancel de un confirm sin escribir lógica extra.

- **La validación usa** `e.Handled = true` **solo cuando falla**. Si todos los campos son válidos, no se toca `e.Handled` y el diálogo se cierra solo. Si algún campo está mal, marcamos `e.Handled = true` para abortar el cierre y devolvemos el foco al campo problemático. No hay que llamar `RequestStop()` manualmente: el cierre lo dispara el botón cuando no se intercepta.
- **Las dos formas de leer el resultado**: una es usar la API tipada `app.GetResult<Contacto>()` después de `app.Run<Editar>()`. La otra es construir el diálogo a mano y leer la propiedad `Result`:

```
csharp
// Alternativa más explícita, equivalente:
editar.Accepting += (_,_) => {
    using Editar dialog = new();
    app.Run(dialog);
    if (dialog.Result is not null) {
        resultado.Text = $"Resultado: {dialog.Result}";
    }
};
```

La diferencia conceptual entre los tres contenedores:

- `FrameView` organiza una pantalla.
- `Dialog` (o `Dialog<T>`) interrumpe la pantalla para pedir una decisión o datos.
- `Runnable/Window` es lo que corre en `app.Run(...)` como pantalla principal.

9. Layout: ubicación con `Pos` y tamaño con `Dim`

La pantalla de terminal se mide en celdas de caracteres. Una coordenada no es un pixel; es una columna y fila de texto.

Cada `View` tiene:

```
csharp
X      // posición horizontal
Y      // posición vertical
Width  // ancho
Height // alto
```

Pero no conviene pensar que siempre son enteros. En `TerminalGui` son expresiones.

Posiciones absolutas

```
csharp
X = 1;
Y = 2;
```

Útil para cosas simples.

Posiciones relativas a otros views

```
csharp
X = Pos.Right(searchLabel) + 1;
Y = Pos.Top(searchLabel);
```

Esto dice: "a la derecha del label, en la misma fila".

Debajo de otro view

```
csharp
Y = Pos.Bottom(emailLabel) + 1;
```

Muy usado en formularios.

Anclar al final

```
csharp
Y = Pos.AnchorEnd(1);
```

Sirve para una barra de estado en la última fila.

Tamaño fijo

```
csharp
Width = 74;
Height = 22;
```

Es razonable para diálogos.

Porcentaje

```
csharp
Width = Dim.Percent(50);
```

Sirve para partir la pantalla en paneles.

Rellenar el espacio disponible

```
csharp
Width = Dim.Fill();
Height = Dim.Fill(2);
```

`Dim.Fill(2)` significa ocupar el espacio disponible dejando 2 celdas libres al final.

Ejemplo completo de formulario:


```
csharp
Label nameLabel = new() { Text = "Nombre:", X = 2, Y = 1 };

TextField nameField = new() {
    X = 14, Y = Pos.Top(nameLabel),
    Width = Dim.Fill(2)
};

Label phoneLabel = new() {
    Text = "Teléfono:",
    X = Pos.Left(nameLabel), Y = Pos.Bottom(nameLabel) + 1
};

TextField phoneField = new() {
    X = Pos.Left(nameField), Y = Pos.Top(phoneLabel),
    Width = Dim.Fill(2)
};
```

La ventaja de este estilo es que si movemos `nameLabel`, lo demás sigue acomodándose de manera coherente.

10. Componentes básicos de entrada

Label

Muestra texto fijo.

```
csharp
new Label { Text = "Buscar:", X = 1, Y = 2 };
```

TextField

Entrada de una sola línea.

```
csharp
TextField buscar = new() {
    X = 10, Y = 2,
    Width = Dim.Percent(42)
};

buscar.TextChanged += (_, _) => Actualizar();
```

`TextChanged` es lo que permite búsqueda activa: cada vez que cambia el texto, se vuelve a calcular la lista filtrada. (En `ListView`, el evento análogo cuando cambia la selección se llama `ValueChanged`, no confundir.)

TextView

Entrada o visualización de texto multilinea.

```
csharp
TextView notesField = new() {
    X = 14, Y = 11,
    Width = Dim.Fill(2), Height = 4
};
```

Button

Ejecuta una acción.

```
csharp
Button saveButton = new() {
    Text = "Guardar",
    IsDefault = true
};

saveButton.Accepting += (_, e) => {
    Save();
    e.Handled = true;
};
```

`IsDefault = true` permite que Enter active ese botón cuando el diálogo lo usa como acción principal.

CheckBox

Entrada booleana.

```
csharp
CheckBox favoriteField = new() {
    Text = "_Favorito",
    Value = contact.Favorito ? CheckState.Checked : CheckState.Unchecked
};
```

El estado se lee con `Value`:

```
csharp
bool favorito = favoriteField.Value == CheckState.Checked;
```

11. Listas, búsqueda activa y vista de detalle

Llegado este punto, ya sabemos armar paneles y diálogos. Toca combinar esos elementos en una pantalla de tipo "maestro-detalle": una lista a la izquierda, los datos del elemento seleccionado a la derecha, y una búsqueda arriba que filtra la lista en tiempo real.

Ejemplo completo

```
csharp
using System.Collections.ObjectModel;
using Terminal.Gui.App;           // Application, IApplication, Runnable
using Terminal.Gui.ViewBase;     // View, Pos, Dim
using Terminal.Gui.Views;       // Window, FrameView, Button, TextField, etc.

List<Contacto> agenda = [
    new("Ana",      "Andrade",  "(381)456-7890", 20),
    new("Bruno",    "Bianchi",   "(381)567-8901", 35),
    new("Carla",    "Cortés",    "(381)678-9012", 28),
    new("Diego",    "Díaz",      "(381)789-0123", 42),
    new("Elena",    "Espinosa",  "(381)890-1234", 30),
    new("Federico", "Fernández", "(381)901-2345", 45),
    new("Gabriela", "García",    "(381)012-3456", 25)
];

using( IApplication app = Application.Create().Init()){
    Window mainWindow= new() { Title = "Tutorial Terminal.Gui v2" };

    Label etiqueta = new() { Text = "Buscar:", X = 1, Y = 1 };
    TextField buscar = new() { X = Pos.Right(etiqueta), Y = 1, Width = 43 };

    ListView maestro = new() { X = 1, Y = 3, Width = 50, Height = Dim.Fill(4) };
    Markdown detalle = new() { Text = "", X = Pos.Right(maestro) + 1, Y = 3, Width = 40, Height = Dim.Fill(4) };

    List<Contacto> filtrados = [];

    void Actualizar() {
        string texto = buscar.Text ?? "";
        filtrados = agenda.Where(c => c.Coincide(texto)).ToList();
        maestro.SetSource(new ObservableCollection<string>(filtrados.Select(c => c.ToString()).ToList()));
    }

    buscar.TextChanged += (_, _) => Actualizar();
    Actualizar();

    maestro.ValueChanged += (_, _) => {
        int indice = maestro.SelectedItem ?? 0;
        if (indice >= 0 && indice < filtrados.Count) {
            Contacto contacto = filtrados[indice];
            detalle.Text = $"
            # Contacto

            **{contacto.NombreCompleto}**

            *Teléfono:* **{contacto.Telefono}**

            *Edad:* **{contacto.Edad}** años
            """;
        } else {
            detalle.Text = "_Sin contacto seleccionado._";
        }
    };

    mainWindow.Add(etiqueta, buscar, maestro, detalle);

    app.Run( mainWindow);
}

record Contacto(string Nombre, string Apellido, string Telefono, int Edad) {
    public string NombreCompleto => $"{Apellido}, {Nombre}";

    public bool Coincide(string texto) =>
        NombreCompleto.Contains(texto.Trim(), StringComparison.OrdinalIgnoreCase)
        || Telefono.Contains(texto.Trim());

    public override string ToString() => $" {NombreCompleto,-22} | {Telefono,10} | {Edad,2} años";
};
```

Cómo se piensa este código

El modelo es un record. El Contacto no es solo un contenedor de datos: tiene tres comportamientos que la UI necesita y que conviene que estén en el modelo, no en la ventana:

- `NombreCompleto` es la concatenación `Apellido`, `Nombre` usada en la lista.
- `Coincide(texto)` decide si un contacto matchea la búsqueda. Concentrar la regla de búsqueda en el modelo evita que la ventana sepa demasiado.
- `ToString()` formatea la fila de la lista con anchos fijos (`, -22, , 10, , 2`), que es lo que hace que las columnas queden alineadas en el `ListView`.

Dos listas en memoria. `agenda` es la lista completa; `filtrados` es la que está mostrándose en este momento. La búsqueda no toca `agenda`: solo recalcula `filtrados`. Eso permite limpiar la búsqueda y volver al estado original sin perder nada.

El `ListView` muestra strings, no objetos. `SetSource` recibe una `ObservableCollection<string>`. La selección se resuelve usando el mismo índice contra `filtrados`:

```
csharp
int indice = maestro.SelectedItem ?? 0;
Contacto contacto = filtrados[indice];
```

Ese detalle es importante: nunca conviene intentar reconstruir el contacto a partir de la fila visible. La fuente de verdad es siempre el objeto en `filtrados`.

`buscar.TextChanged` **es lo que hace que la búsqueda sea activa.** Cada vez que el usuario tipea una letra, se dispara `Actualizar`, que reconstruye `filtrados` y se lo pasa al `ListView`. No hay botón "Buscar" porque no hace falta.

`maestro.ValueChanged` **se dispara cuando cambia el ítem seleccionado** (con flechas, click o búsqueda). El handler regenera el texto del panel derecho.

El panel derecho es un Markdown. Esa vista interpreta ***negrita***, *itálica*, # título y demás. Para mostrar datos formateados ahorra mucho código respecto a armar `Labels` manualmente. El string usa raw strings (`$"" . . . ""`) para que las llaves y los saltos de línea queden legibles.

Por qué no consultar la base en cada `ValueChanged`. Aunque después agreguemos persistencia, conviene mantener la lista en memoria: la búsqueda activa filtra cientos de veces por segundo y no queremos golpear el almacenamiento con tantas consultas. La regla termina siendo: cada operación CRUD modifica el almacenamiento y la lista en memoria, en ese orden.

12. Diálogos de edición: separación de responsabilidades

El ejemplo de `Editar : Dialog<Contacto?>` de la sección 8 ilustra un principio que vale la pena explicitar para cuando los diálogos crezcan.

Una caja de diálogo de edición tiene tres responsabilidades, y nada más:

1. mostrar campos,

2. validar lo que el usuario ingresa,
3. devolver un resultado tipado.

Cosas que **no** tiene que hacer un diálogo:

- no toca la base de datos,
- no modifica la lista global de contactos en memoria,
- no decide si la operación se persiste o no.

La ventana principal decide qué hacer con el resultado:

```
csharp
editar.Accepting += (_, _) => {
    app.Run<Editar>();
    Contacto? nuevo = app.GetResult<Contacto>();
    if (nuevo is not null) {
        // primero: la ventana decide si persistir
        // segundo: la ventana actualiza la lista en memoria
        // tercero: la ventana refresca la UI
    }
};
```

Esa separación tiene una consecuencia práctica importante: el botón **Cancelar** del diálogo funciona "gratis", porque nada se persistió todavía. No hay nada que deshacer.

Cuando el diálogo edita un contacto existente (no uno nuevo), conviene pasarle una **copia** del contacto, no el original:

```
csharp
Contacto copia = original with { }; // record con copia por valor
```

Así, mientras el usuario está editando, el original sigue intacto. Si cancela, no pasó nada. Si confirma, la ventana sabe que tiene que reemplazar el original por la copia editada.

Para clases (no records) hace falta un método `Clone()`:

```
csharp
public Contacto Clone() => new() {
    Id      = this.Id,
    Nombre  = this.Nombre,
    Telefono = this.Telefono
};
```

13. Menús desplegables

Un menú superior se arma con tres tipos:

- `MenuBar`: la barra de menú entera, en la parte de arriba de la pantalla.
- `MenuBarItem`: cada menú principal (Archivo, Contacto, ...).
- `MenuItem`: cada acción concreta dentro de un menú (Salir, Agregar, ...).

Ejemplo completo

```
csharp
using Terminal.Gui.App;           // Application, IApplication, Runnable
using Terminal.Gui.Drawing;       // LineStyle
using Terminal.Gui.Input;         // Key
using Terminal.Gui.ViewBase;      // View, Pos, Dim
using Terminal.Gui.Views;         // Window, Label, MenuBar, MenuItem

using IApplication app = Application.Create().Init();

MenuBar.DefaultBorderStyle = LineStyle.Rounded;

Runnable raiz = new() { };

Label mensaje = new() {
    Text = "Seleccioná una opción desde la barra superior.",
    X = 1, Y = 1,
    Width = Dim.Fill(2), Height = 2,
};

MenuBar menu = new(new MenuItem[] {
    new("Archivo", new MenuItem[] {
        new("_Importar", "Importar desde JSON", () => mensaje.Text = "Archivo > Importar"),
        new("_Exportar", "Exportar a JSON", () => mensaje.Text = "Archivo > Exportar"),
        null!,
        new("_Salir", "Salir de la aplicación", () => app.RequestStop(),
Key.S.WithCtrl)
    })),
    new("Contacto", new MenuItem[] {
        new("_Agregar", "Agrega un nuevo contacto", () => mensaje.Text = "Contacto > Agregar",
Key.A.WithCtrl),
        new("_Editar", "Edita un contacto existente", () => mensaje.Text = "Contacto > Editar",
Key.E.WithCtrl),
        null!,
        new("_Borrar", "Borra un contacto existente", () => mensaje.Text = "Contacto > Borrar",
Key.B.WithCtrl)
    })
});

Window ventana = new() {
    Title = "Agenda",
    X = 0, Y = 1,
};

ventana.Add(mensaje);

raiz.Add(menu, ventana);
app.Run(raiz);
```

Cómo se piensa este código

Por qué hay un Runnable por encima de la Window. El MenuBar ocupa la fila $Y = 0$. Si pusiéramos todo dentro de la Window, su borde superior taparía el menú. La solución es tener un Runnable como raíz (sin borde), agregarle el MenuBar directamente en $Y = 0$, y poner la Window debajo, en $Y = 1$. El Runnable le no se ve: solo coordina.

El constructor de MenuBar recibe un array de MenuItem. Cada MenuItem toma un título y un array de MenuItem. No es un *initializer* { Menus = [. . .] }: es un parámetro posicional.

El constructor de MenuItem tiene cuatro variantes útiles. Las que aparecen acá son las dos más comunes:

```
csharp
new("_Importar", "Importar desde JSON", () => ...) // sin atajo
new("_Salir", "Salir de la aplicación", () => ..., Key.S.WithCtrl) // con atajo
```

El primer argumento es el texto visible. El guion bajo marca la letra rápida (`_Salir` habilita `Alt+S` dentro del menú abierto). El segundo es la ayuda que aparece en la barra de estado. El tercero es el callback. El cuarto, opcional, es el atajo global (`Ctrl+S`, `Ctrl+A`, etc.).

Los separadores son `null!`. Esa línea horizontal que divide grupos de opciones en un menú se obtiene insertando un `null!` (un `null` forzado para que C# no se queje) entre los `MenuItem`. Es feo pero es lo que la API espera.

`Menu.DefaultBorderStyle = LineStyle.Rounded` **antes de crear el menú.** Eso da bordes redondeados a todos los submenús que se abran. Es opcional, pero se ve mejor que el default. Tiene que ir antes de instanciar la `MenuBar`.

El menú no implementa la lógica. En este ejemplo los callbacks son tan chicos que están como lambdas inline, pero en una aplicación real cada `MenuItem` llamaría a un método con nombre:

```
csharp
new("_Importar", "Importar desde JSON", ImportarJson, Key.I.WithCtrl)
```

Y la lógica real vive en `ImportarJson()`. El menú solo conecta la opción visual con el método.

14. Manejo de teclado

Terminal.Gui ya maneja muchas teclas sin que tengamos que hacer nada:

- Tab cambia el foco entre controles.
- Las flechas navegan listas.
- Enter acepta botones o abre el ítem seleccionado.
- F9 (o Alt al solo) activa el menú.
- Escape cierra menús o diálogos según el contexto.

Atajos en el menú

La forma más natural de agregar un atajo es pasarlo como cuarto argumento del `MenuItem`:

```
csharp
new("_Salir", "Salir de la aplicación", () => app.RequestStop(), Key.S.WithCtrl)
```

Esto hace dos cosas a la vez:

- muestra `Ctrl+S` al lado del texto del menú,
- registra el atajo globalmente.

Ctrl+S queda disponible en cualquier pantalla mientras el MenuBar esté visible, sin que tengamos que tocar ningún evento de teclado.

La API de Key

Las teclas son objetos Key. Los modificadores se expresan con propiedades:

```
csharp
Key.S.WithCtrl    // Ctrl+S
Key.A.WithAlt     // Alt+A
Key.F.WithShift   // Shift+F
Key.N.WithCtrl.WithShift // Ctrl+Shift+N
```

Las teclas especiales tienen nombres propios: Key.F1, Key.Esc, Key.Enter, Key.Tab, Key.Delete, Key.PageUp, etc.

Cuándo conviene sobrescribir OnKeyDown

Para atajos que **no corresponden a una opción de menú** (por ejemplo, "F4 para enfocar el campo de búsqueda"), se puede sobrescribir OnKeyDown en la ventana:

```
csharp
protected override bool OnKeyDown(Key key) {
    if (key == Key.F4) {
        this.buscar.SetFocus();
        return true;
    }
    return base.OnKeyDown(key);
}
```

return true significa "esta tecla ya fue manejada"; base.OnKeyDown(key) deja que Terminal.Gui siga con su comportamiento por defecto.

La regla práctica: si la acción ya está en el menú, pasale el Key ahí. Si es una acción que solo tiene sentido como atajo (foco, navegación interna), usá OnKeyDown.

15. Persistencia con SQLite y Dapper.Contrib

Por qué este stack

Para una agenda no necesitamos un ORM completo como Entity Framework. Tampoco queremos escribir SQL crudo para cada operación CRUD. Dapper resuelve el medio:

- **ADO.NET** + SqlConnection abre la conexión a la base.
- **Dapper** convierte filas en objetos con Query<T> y ejecuta SQL con Execute.
- **Dapper.Contrib** agrega métodos de extensión GetAll, Insert, Update, Delete que generan el SQL básico por convención, basándose en atributos del modelo.

El resultado es un store muy chico: una clase, una conexión, y métodos de una línea para el 90% de los casos.

El modelo

El modelo se anota con atributos de `Dapper.Contrib`:

```
csharp
#:package Terminal.Gui@2.0.1
#:package Microsoft.Data.Sqlite@9.0.0
#:package Dapper@2.1.35
#:package Dapper.Contrib@2.0.78

using Microsoft.Data.Sqlite; // SqliteConnection
using Dapper;                // Execute, Query
using Dapper.Contrib.Extensions; // GetAll, Insert, Update, Delete, [Key], [Table]

using System.Text.Json;      // JsonSerializer
```

```
csharp
[Table("Contactos")]
public sealed class Contacto {
    [Key] public int Id { get; set; }
    public string Nombre { get; set; } = "";
    public string Telefono { get; set; } = "";
    public string Email { get; set; } = "";
    public string Notas { get; set; } = "";
    public bool Favorito { get; set; }

    public Contacto Clone() => new() {
        Id = this.Id,
        Nombre = this.Nombre,
        Telefono = this.Telefono,
        Email = this.Email,
        Notas = this.Notas,
        Favorito = this.Favorito
    };
}
```

Qué hace cada atributo:

- `[Table("Contactos")]` indica el nombre real de la tabla. Sin esto, `Dapper.Contrib` pluraliza el nombre de la clase, lo que no siempre acierta en español. Mejor explicitarlo siempre.
- `[Key]` marca la columna de clave primaria **auto-generada por la base** (en SQLite, `INTEGER PRIMARY KEY AUTOINCREMENT`). En `Insert`, esa propiedad se rellena con el id devuelto por la base.

El store

csharp

```
public sealed class SqliteAgendaStore {
    private readonly string connectionString;

    public SqliteAgendaStore(string dbPath) {
        this.connectionString = $"Data Source={dbPath}";
        EnsureSchema();
    }

    private SQLiteConnection Open() {
        SQLiteConnection connection = new(this.connectionString);
        connection.Open();
        return connection;
    }

    private void EnsureSchema() {
        using SQLiteConnection db = Open();
        db.Execute("""
            CREATE TABLE IF NOT EXISTS Contactos (
                Id            INTEGER PRIMARY KEY AUTOINCREMENT,
                Nombre        TEXT      NOT NULL,
                Telefono       TEXT      NOT NULL DEFAULT '',
                Email          TEXT      NOT NULL DEFAULT '',
                Notas          TEXT      NOT NULL DEFAULT '',
                Favorito       INTEGER NOT NULL DEFAULT 0
            );
        """);
    }

    public List<Contacto> GetAll() {
        using SQLiteConnection db = Open();
        return db.GetAll<Contacto>().ToList();
    }

    public int Insert(Contacto contact) {
        using SQLiteConnection db = Open();
        return (int)db.Insert(contact);
    }

    public bool Update(Contacto contact) {
        using SQLiteConnection db = Open();
        return db.Update(contact);
    }

    public bool Delete(int id) {
        using SQLiteConnection db = Open();
        return db.Delete(new Contacto { Id = id });
    }

    public void DeleteAll() {
        using SQLiteConnection db = Open();
        db.Execute("DELETE FROM Contactos;");
    }
}
```

Cómo se traduce cada operación

Método del store	Lo que hace Dapper.Contrib
db.GetAll<Contacto>()	SELECT * FROM Contactos
db.Get<Contacto>(id)	SELECT * FROM Contactos WHERE Id = @Id
db.Insert(c)	INSERT INTO Contactos (...) VALUES (...) y devuelve el id
db.Update(c)	UPDATE Contactos SET ... WHERE Id = @Id
db.Delete(c)	DELETE FROM Contactos WHERE Id = @Id

El detalle a recordar es que Delete recibe **el objeto**, no el id. Cuando solo tenemos el id (como en una eliminación desde la lista), construimos un contacto vacío con ese id:

```
csharp
db.Delete(new Contacto { Id = id });
```

Mapeo de tipos

SQLite no tiene tipos estrictos, pero Dapper resuelve el mapeo común:

Tipo C#	Tipo SQLite	Comentario
int	INTEGER	Directo
string	TEXT	Directo
bool	INTEGER	1 para true, 0 para false
DateTime	TEXT	Formato ISO 8601
decimal	TEXT	Para evitar pérdida de precisión

Por qué abrir y cerrar la conexión cada vez

Open() se llama en cada método y se libera con using. Para una app de escritorio con un solo usuario y una base SQLite local, esto es trivial en costo y elimina toda una categoría de bugs (conexiones colgadas, transacciones huérfanas). En aplicaciones servidor sí tendría sentido un pool, pero acá no.

Integración con la ventana

La ventana ya no tiene un flag hasUnsavedChanges. Cada operación CRUD persiste de inmediato. Reutilizando el patrón Dialog<Contacto?> de la sección 8:

```
csharp
private void NuevoContacto() {
    using Editar dialog = new(); // Editar : Dialog<Contacto?>
    App!.Run(dialog);

    if (dialog.Result is null) {
        return;
    }
}
```

csharp - continuacion

```
    Contacto nuevo = dialog.Result;
    int newId = this.store.Insert(nuevo);
    nuevo.Id = newId;

    this.contactos.Add(nuevo);
    SetStatus($"Contacto '{nuevo.Nombre}' creado.");
    Actualizar(seleccionado: newId);
}
```

El diálogo devuelve null si el usuario canceló y un Contacto válido si confirmó. El store se encarga de generar el Id nuevo en SQLite.

16. Importación y exportación JSON

La base de datos es la fuente de verdad. JSON sirve para dos cosas distintas:

- **Exportar:** dar al usuario un archivo legible para versionar, mandar por mail o respaldar.
- **Importar:** cargar contactos provistos en un archivo, por ejemplo al iniciar el sistema o al recibir datos de otro lado.

Estas dos operaciones no son persistencia: son **interoperabilidad**. Por eso viven en una clase separada del store.

Opciones en el menú

La importación y la exportación tienen que aparecer como acciones del menú Archivo, junto con recargar y salir. Usando la API que vimos en la sección 13:

csharp

```
new("Archivo", new MenuItem[] {
    new("_Importar JSON", "Importar desde JSON", ImportarJson, Key.I.WithCtrl),
    new("_Exportar JSON", "Exportar a JSON", ExportarJson, Key.E.WithCtrl),
    new("_Recargar", "Recargar desde la base", Recargar, Key.R.WithCtrl),
    null!,
    new("_Salir", "Salir de la aplicación", () => app.RequestStop(), Key.Q.WithCtrl)
})
```

El menú no implementa la operación. Solo conecta la opción visual con un método de la ventana principal:

- ImportarJson lee un archivo JSON y agrega esos contactos a la base.
- ExportarJson escribe los contactos actuales en un archivo JSON.

Los mismos métodos se invocan desde el teclado con Ctrl+I y Ctrl+E, sin escribir nada más, porque el atajo va como cuarto argumento del MenuItem.

El conversor JSON

csharp

```
public sealed class JsonAgendaIO {
    private static readonly JsonSerializerOptions JsonOptions = new() {
        WriteIndented = true,
        Encoder = JavaScriptEncoder.UnsafeRelaxedJsonEscaping
    };

    public List<Contacto> Read(string path) {
        if (!File.Exists(path)) {
            throw new FileNotFoundException($"Archivo no encontrado: '{path}'");
        }

        string json = File.ReadAllText(path, Encoding.UTF8);
        return JsonSerializer.Deserialize<List<Contacto>>(json, JsonOptions) ?? new();
    }

    public void Write(string path, IEnumerable<Contacto> contacts) {
        string json = JsonSerializer.Serialize(
            contacts.OrderBy(contact => contact.Id),
            JsonOptions);

        File.WriteAllText(path, json, Encoding.UTF8);
    }
}
```

UnsafeRelaxedJsonEscaping permite que las tildes y la ñ se escriban como tales en el archivo, en vez de aparecer como \u00f1. Para datos en español es lo que casi siempre se quiere.

Pedirle un path al usuario

Necesitamos un diálogo simple para que el usuario tipee la ruta del archivo. Se puede hacer con OpenFileDialog, pero un diálogo propio que extiende Dialog<string?> ya alcanza para el ejemplo y mantiene el mismo patrón de tipado que Editar:

csharp

```
class PedirRuta : Dialog<string?> {
    public PedirRuta(string titulo, string prompt, string rutaDefault){
        Title = titulo;
        Width = 60; Height = 8;

        Label label = new() { Text = prompt, X = 1, Y = 1 };
        TextField input = new() {
            Text = rutaDefault,
            X = 1, Y = Pos.Bottom(label) + 1,
            Width = Dim.Fill(2)
        };
        Add(label, input);

        Button btnCancelar = new() { Title = "Cancelar" };
        btnCancelar.Accepting += (_,_) => Result = null;
        AddButton(btnCancelar);

        Button btnAceptar = new() { Title = "Aceptar" };
        btnAceptar.Accepting += (_,_) => Result = input.Text ?? "";
        AddButton(btnAceptar);
    }
}
```

Y se usa así desde la ventana:

csharp

```
private string? PedirRutaArchivo(string titulo, string prompt, string rutaDefault) {
    using PedirRuta dialog = new(titulo, prompt, rutaDefault);
    App!.Run(dialog);
    return string.IsNullOrEmpty(dialog.Result) ? null : dialog.Result;
}
```

Exportar

Tomamos los contactos en memoria y los escribimos al archivo:

csharp

```
private void ExportarJson() {
    string? ruta = PedirRutaArchivo(
        "Exportar JSON",
        "Ruta de salida:",
        "agenda-export.json");

    if (ruta is null) {
        return;
    }

    try {
        this.json.Write(ruta, this.contactos);
        SetStatus($"Exportados {this.contactos.Count} contactos a '{ruta}'.");
    } catch (Exception ex) {
        MessageBox.ErrorQuery(App!, "Exportar", $"Error: {ex.Message}", "OK");
    }
}
```

Importar

Importar es más delicado: hay que decidir qué pasa con los contactos ya existentes. La estrategia más simple es **agregar**: cada contacto del JSON entra como nuevo registro en la base, con un id fresco asignado por SQLite.

csharp

```
private void ImportarJson() {
    string? ruta = PedirRutaArchivo(
        "Importar JSON",
        "Ruta del archivo a importar:",
        "agenda-export.json");

    if (ruta is null) {
        return;
    }

    try {
        List<Contacto> importados = this.json.Read(ruta);

        int respuesta = MessageBox.Query(
            App!,
            "Importar",
            $"Se agregarán {importados.Count} contactos del archivo a la base. ¿Continuar?",
            "No", "Sí") ?? 0;

        if (respuesta != 1) {
            return;
        }

        foreach (Contacto contacto in importados) {
            contacto.Id = 0; // SQLite asigna id nuevo
            int newId = this.store.Insert(contacto);
            contacto.Id = newId;
        }
    }
}
```

csharp - continuacion

```

    }

    CargarContactos();
    SetStatus($"{importados.Count} contactos importados desde '{ruta}'");
} catch (Exception ex) {
    MessageBox.ErrorQuery(App!, "Importar", $"Error: {ex.Message}", "OK");
}
}

```

Puntos clave:

- **Reseteamos Id = 0 antes de insertar** para que SQLite genere un id nuevo. Si dejáramos los ids del archivo, podríamos chocar con ids ya existentes en la base.
- **Confirmación explícita:** importar es una operación que agrega datos que después hay que limpiar si fue un error.
- **Recarga completa al final:** después de importar, refrescamos contactos desde la base para que la lista en memoria refleje exactamente lo que hay persistido (incluidos los ids nuevos).
- **Manejo de errores con MessageBox.ErrorQuery:** archivos inexistentes, JSON malformado o problemas de permisos no deben tirar el programa.

Dos formatos, un solo modelo

Lo importante es que el Contacto se serializa y deserializa **igual** en JSON y en SQLite. No hay dos versiones del modelo. Los atributos [Table] y [Key] no afectan la serialización JSON, y la propiedad Favorito se mapea a INTEGER en SQLite y a true/false en JSON sin que tengamos que hacer nada.

17. El CRUD final

El archivo agenda-terminal-gui.cs integra todas las piezas anteriores en una sola aplicación.

Partes principales:

texto

```

agenda-terminal-gui.cs
■■■ arranque de la app
■■■ AgendaWindow
■   ■■■ ConstruirLayout
■   ■■■ CargarContactos / Recargar
■   ■■■ NuevoContacto / EditarContacto / EliminarContacto
■   ■■■ ImportarJson / ExportarJson
■   ■■■ Actualizar (lista filtrada + detalle)
■   ■■■ PedirRuta
■■■ Editar : Dialog<Contacto?>
■■■ SqliteAgendaStore
■■■ JsonAgendaIO
■■■ Contacto

```

Arranque

```
csharp
string dbPath = args.Length > 0 ? args[0] : "agenda.db";

SQLiteAgendaStore store = new(dbPath);
JsonAgendaIO json = new();

using IApplication app = Application.Create().Init();
app.Run(new AgendaWindow(store, json));
```

Crear

```
csharp
private void NuevoContacto() {
    using Editar dialog = new(); // sin contacto preexistente
    App!.Run(dialog);

    if (dialog.Result is null) {
        return;
    }

    Contacto nuevo = dialog.Result;
    int newId = this.store.Insert(nuevo);
    nuevo.Id = newId;

    this.contactos.Add(nuevo);
    SetStatus($"Contacto '{nuevo.Nombre}' creado.");
    Actualizar(seleccionado: newId);
}
```

Leer

La lectura ocurre al arrancar y al recargar:

```
csharp
private void CargarContactos() {
    this.contactos = this.store.GetAll();
    Actualizar();
}

private void Recargar() {
    CargarContactos();
    SetStatus("Datos recargados desde la base.");
}
```


Actualizar

```
csharp
private void EditarContacto() {
    Contacto? seleccionado = ContactoSeleccionado();
    if (seleccionado is null) {
        MessageBox.Query(App!, "Editar", "Seleccioná un contacto.", "OK");
        return;
    }

    using Editar dialog = new(seleccionado.Clone());
    App!.Run(dialog);

    if (dialog.Result is null) {
        return;
    }

    Contacto editado = dialog.Result;
    this.store.Update(editado);
    int index = this.contactos.FindIndex(c => c.Id == seleccionado.Id);
    this.contactos[index] = editado;

    SetStatus($"Contacto '{editado.Nombre}' actualizado.");
    Actualizar(seleccionado: editado.Id);
}
```

El `Clone()` evita editar el objeto original mientras el usuario todavía puede cancelar.

Eliminar

```
csharp
private void EliminarContacto() {
    Contacto? seleccionado = ContactoSeleccionado();
    if (seleccionado is null) {
        return;
    }

    int respuesta = MessageBox.Query(
        App!,
        "Eliminar contacto",
        $"¿Eliminar a {seleccionado.Nombre}?",
        "No",
        "Si") ?? 0;

    if (respuesta == 1) {
        this.store.Delete(seleccionado.Id);
        this.contactos.RemoveAll(c => c.Id == seleccionado.Id);

        SetStatus("Contacto eliminado.");
        Actualizar();
    }
}
```

18. Orden recomendado para construir una TUI

El orden que seguimos en clase, replicable en cualquier TUI:

1. **Paneles** (25.2). Window con dos `FrameView` lado a lado. Sin lógica, solo layout.
2. **Subclase de Window** (25.3). Mover la pantalla a su propia clase con botón y diálogo de confirmación inline.
3. **Refactor del diálogo** (25.4). Sacar el `Dialog` a su propia clase para que sea reutilizable.

4. **Diálogo tipado con validación** (25.5). Pasar de `Dialog` a `Dialog<T>` y validar campos.
5. **ListView + búsqueda activa + detalle** (25.6). Vista maestro-detalle con `TextChanged` y `Markdown`.
6. **Menú superior** (25.7). `MenuBar` con atajos de teclado vía `Key.X.WithCtrl`.
7. **Modelo de datos** (Contacto con `[Table]` y `[Key]`) y store de persistencia (`SQLiteAgendaStore`).
Conviene verificarlo con un script chico antes de tocar la UI: que `Insert/GetAll/Update/Delete` funcionen.
8. **Conversor JSON** (`JsonAgendaIO`) y verificación con un round-trip de prueba.
9. **Acciones CRUD conectadas al store**: el menú dispara métodos de la ventana, que persisten y refrescan la lista en memoria.
10. **Importación y exportación JSON** desde el menú Archivo.
11. **Recargar** desde la base y **confirmar salida** si hay cambios pendientes.

Si se intenta hacer todo junto, los errores de layout, eventos y persistencia se mezclan. Especialmente el store: probarlo desde un archivo aparte antes de integrarlo a la UI evita muchas horas de depuración cruzada.

19. Errores comunes

Olvidar `[Table]` o `[Key]` en el modelo

Síntoma: `Insert` falla con un error sobre tabla inexistente, o el id no se rellena después de insertar.

Solución: anotar la clase con `[Table("Contactos")]` y la propiedad clave con `[Key]`. Sin `[Key]`, `Dapper.Contrib` busca una propiedad llamada `Id` por convención, pero ser explícito evita sorpresas.

Delete recibe el objeto, no el id

`db.Delete(5)` no compila o no hace lo que parece. Hay que pasar un objeto con el id seteado:

```
csharp
db.Delete(new Contacto { Id = id });
```

Conexión no cerrada

Síntoma: el archivo `.db` queda bloqueado, o las pruebas dejan procesos colgados.

Solución: siempre usar `using SqlConnection db = Open();`. El `using` garantiza que la conexión se cierra incluso si la operación lanza excepción.

Importar JSON conservando los ids del archivo

Si el JSON viene con ids 1, 2, 3 y la base ya tiene contactos con esos ids, `Insert` falla por clave duplicada. Conviene resetear `contact.Id = 0` antes de insertar y dejar que `SQLite` asigne ids frescos.

Mantener la lista en memoria desactualizada

Después de cada operación CRUD, hay que actualizar **dos** lugares: la base con el store, y la lista contactos en memoria. Si solo se actualiza uno, la UI queda inconsistente con la base hasta el próximo Recargar. La regla: persistir primero, actualizar memoria después, refrescar la lista por último.

Guardar directamente desde el diálogo

No conviene. El diálogo debe devolver datos (vía `Result`). La ventana decide cuándo persistir. Esto es lo que hace posible que el botón "Cancelar" del diálogo funcione: no hay nada para deshacer porque nada se persistió todavía.

Perder la selección al filtrar

Si al refrescar la lista siempre seleccionás el primer elemento, la UI se siente errática. Conviene mantener el Id seleccionado:

```
csharp
int? idAMantener = idSolicitado ?? ContactoSeleccionado()?.Id;
```

Intentar parsear el texto de la fila

No uses la fila formateada como fuente de verdad. La fuente de verdad es el objeto `Contacto` en `filtrados`.

20. Ejercicios para alumnos

1. Agregar un campo `FechaNacimiento` al contacto. Hace falta tocar el modelo, el schema (`ALTER TABLE` o regenerar la base), el diálogo y la fila de la lista.
2. Validar que el email contenga `@` antes de guardar.
3. Agregar un menú `Ver -> Solo favoritos` que filtre la lista.
4. Agregar un botón `Limpiar búsqueda`.
5. Implementar la importación JSON con una opción extra: "Reemplazar todo" además de "Agregar". Usar `store.DeleteAll()` antes de insertar.
6. Reemplazar `ListView` por `TableView` para mostrar columnas.
7. Agregar importación y exportación CSV, reusando `Dapper.Contrib` para la lectura de la base.
8. Envolver la importación JSON en una transacción de SQLite (`BEGIN/COMMIT/ROLLBACK`) para que sea todo o nada.
9. Usar `db.Get<Contacto>(id)` para refrescar el detalle desde la base cada vez que cambia la selección, en vez de leer la lista en memoria.
10. Agregar un campo `FechaUltimaModificacion` que se actualice automáticamente en `Insert` y `Update`, mostrarlo en una columna del `TableView`.

21. Fuentes oficiales consultadas

- Terminal.Gui — inicio: <https://gui-cs.github.io/Terminal.Gui/docs/getting-started.html>
- Terminal.Gui — arquitectura de aplicación: <https://gui-cs.github.io/Terminal.Gui/docs/application.html>
- Terminal.Gui — configuración: <https://gui-cs.github.io/Terminal.Gui/docs/config.html>
- Terminal.Gui — layout: <https://gui-cs.github.io/Terminal.Gui/docs/layout.html>
- Terminal.Gui — View: <https://gui-cs.github.io/Terminal.Gui/docs/View.html>
- Terminal.Gui — menús: <https://gui-cs.github.io/Terminal.Gui/docs/menus.html>
- Terminal.Gui — teclado: <https://gui-cs.github.io/Terminal.Gui/docs/keyboard.html>
- Microsoft.Data.Sqlite: <https://learn.microsoft.com/dotnet/standard/data/sqlite/>
- Dapper: <https://github.com/DapperLib/Dapper>
- Dapper.Contrib: <https://github.com/DapperLib/Dapper.Contrib>
- System.Text.Json: <https://learn.microsoft.com/dotnet/standard/serialization/system-text-json/>

CAPITULO 31

Entrada/salida por consola, archivos de texto y encoding

En C#, la entrada/salida más directa aparece en dos lugares:

- `Console`, para leer y escribir en la terminal;
- `File`, `Directory` y `Path`, para trabajar con archivos, carpetas y rutas.

Encoding atraviesa ambos mundos: define cómo se convierten bytes en texto y texto en bytes.

Console: leer y escribir

`Console` es una clase estática para leer y escribir en la terminal, controlar colores y cursor, y trabajar con los streams estándar.

```
csharp
// Escribir
Console.Write("Sin salto");
Console.WriteLine("Con salto");

// Leer
string? linea = Console.ReadLine(); // espera Enter
ConsoleKeyInfo tecla = Console.ReadKey(); // sin Enter, captura la tecla
```

Colores y cursor

```
csharp
Console.ForegroundColor = ConsoleColor.Cyan;
Console.BackgroundColor = ConsoleColor.DarkBlue;
Console.WriteLine("Texto con color");
Console.ResetColor();

Console.Clear();
Console.CursorVisible = false;
Console.SetCursorPosition(col: 10, top: 5);
var (col, row) = Console.GetCursorPosition();
```

Streams estándar

Un programa de consola no tiene una sola "salida". Tiene tres streams estándar:

Stream	Descripción
<code>Console.In</code>	Entrada estándar: teclado o pipe
<code>Console.Out</code>	Salida estándar: pantalla o redirección

Stream	Descripción
<code>Console.Error</code>	Salida de error separada de <code>stdout</code>

```
csharp
// Leer toda la entrada (útil con pipe)
string todo = Console.In.ReadToEnd();

// Escribir errores sin contaminar la salida normal
Console.Error.WriteLine("Algo salió mal");
```

Esta separación importa cuando un programa se usa desde scripts: la salida normal puede procesarse con otro comando y los errores pueden ir por otro camino.

Redirección desde shell

La terminal puede conectar los streams estándar con archivos u otros programas:

```
bash
dotnet run < entrada.txt          # stdin desde archivo
dotnet run > salida.txt           # stdout a archivo
dotnet run 2> error.txt           # stderr a archivo
dotnet run < entrada.txt | grep "x"
```

El programa no necesita saber si `Console.In` viene del teclado o de un archivo. Lee el mismo stream.

File: leer y escribir archivos de texto

`File` opera sobre archivos. Para texto, los métodos más usados son `ReadAllText`, `WriteAllText` y `AppendAllText`.

```
csharp
using System.IO;
using System.Text;

// Leer todo el archivo
string texto = File.ReadAllText("datos.txt");
string textoUtf8 = File.ReadAllText("datos.txt", Encoding.UTF8);

// Escribir (reemplaza el contenido)
File.WriteAllText("salida.txt", "Hola mundo", Encoding.UTF8);

// Agregar al final
File.AppendAllText("log.txt", "Nueva línea\n", Encoding.UTF8);

// Verificar existencia y borrar
if (File.Exists("viejo.txt")) {
    File.Delete("viejo.txt");
}
```

Los paths relativos, como `"datos.txt"` o `"datos/entrada.txt"`, se interpretan desde el directorio de trabajo actual del proceso. En proyectos de consola, ese directorio no siempre coincide con la carpeta donde está el `.cs`.

Directory: carpetas

Directory opera sobre carpetas.

```
csharp
using System.IO;

// Crear (no falla si ya existe)
Directory.CreateDirectory("datos/salidas/2026");

// Listar archivos
string[] todos = Directory.GetFiles("mis-archivos");
string[] soloTxt = Directory.GetFiles("mis-archivos", "*.txt");

// Verificar y borrar
if (Directory.Exists("temp")) {
    Directory.Delete("temp", recursive: true);
}
```

recursive: true borra también el contenido de la carpeta. Hay que usarlo con cuidado.

Path: construir rutas

Path no lee ni escribe archivos. Construye y analiza rutas de forma portable entre sistemas operativos.

```
csharp
using System.IO;

// Usar Combine en vez de concatenar con "/" o "\"
string ruta = Path.Combine("datos", "clientes", "enero.txt");

string archivo = Path.GetFileName(ruta);           // "enero.txt"
string nombre = Path.GetFileNameWithoutExtension(ruta); // "enero"
string ext = Path.GetExtension(ruta);              // ".txt"
string? dir = Path.GetDirectoryName(ruta);         // carpeta contenedora
string abs = Path.GetFullPath(ruta);               // ruta absoluta
string bak = Path.ChangeExtension(ruta, ".bak");    // cambia extensión
string tmp = Path.GetTempPath();                   // carpeta temporal del sistema
```

La regla práctica es: construir rutas con `Path.Combine`, no con strings pegados a mano.

Encoding

Un archivo es bytes. El encoding es la regla para convertir bytes en caracteres y caracteres en bytes. Si lees un UTF-8 como Latin1, o al revés, aparecen caracteres rotos: mañana puede verse como maÑ±ana. Ese fenómeno se llama **mojibake**.

```
csharp
using System.Text;

// Regla práctica: mismo encoding al escribir y al leer
File.WriteAllText("archivo.txt", contenido, Encoding.UTF8);
string texto = File.ReadAllText("archivo.txt", Encoding.UTF8);

// Para la consola: poner al inicio del programa
Console.InputEncoding = Encoding.UTF8;
Console.OutputEncoding = Encoding.UTF8;
```

Encoding	Cuándo usarlo
Encoding.UTF8	Por defecto hoy en día; soporta cualquier idioma
Encoding.Latin1	Archivos heredados en español/europeo
Encoding.Unicode	UTF-16; frecuente en archivos de Windows
Encoding.ASCII	Solo inglés sin acentos

Ejemplo completo

Este programa lee un archivo, transforma su contenido y escribe otro archivo. Usa `Console.Out` para informar el resultado y `Console.Error` para avisar errores.

```
csharp
using System.IO;
using System.Text;

Console.InputEncoding = Encoding.UTF8;
Console.OutputEncoding = Encoding.UTF8;

string carpeta = "datos";
string entrada = Path.Combine(carpeta, "entrada.txt");
string salida = Path.Combine(carpeta, "salida.txt");

Directory.CreateDirectory(carpeta);

if (File.Exists(entrada)) {
    string texto = File.ReadAllText(entrada, Encoding.UTF8);
    File.WriteAllText(salida, texto.ToUpper(), Encoding.UTF8);
    Console.WriteLine("Procesado.");
} else {
    Console.Error.WriteLine("No existe el archivo de entrada.");
}
```


Resumen

Necesidad	Método
Leer línea	<code>Console.ReadLine()</code>
Leer tecla sin Enter	<code>Console.ReadKey()</code>
Escribir sin salto	<code>Console.Write(...)</code>
Escribir con salto	<code>Console.WriteLine(...)</code>
Mensajes de error	<code>Console.Error.WriteLine(...)</code>
Leer todo stdin	<code>Console.In.ReadToEnd()</code>
Limpiar pantalla	<code>Console.Clear()</code>
Cambiar color	<code>Console.ForegroundColor,</code> <code>Console.BackgroundColor</code>
Restaurar color	<code>Console.ResetColor()</code>
Mover cursor	<code>Console.SetCursorPosition(col, top)</code>
Leer archivo	<code>File.ReadAllText(path, encoding)</code>
Escribir archivo	<code>File.WriteAllText(path, texto, encoding)</code>
Agregar al final	<code>File.AppendAllText(path, texto, encoding)</code>
Existe archivo	<code>File.Exists(path)</code>
Borrar archivo	<code>File.Delete(path)</code>
Crear carpeta	<code>Directory.CreateDirectory(path)</code>
Listar archivos	<code>Directory.GetFiles(path, "*.ext")</code>
Existe carpeta	<code>Directory.Exists(path)</code>
Borrar carpeta	<code>Directory.Delete(path, recursive: true)</code>
Unir rutas	<code>Path.Combine(parte1, parte2, ...)</code>

CAPITULO 32

Cómo funciona la mini TUI...

Este tutorial explica el programa [23.6-consola-nano-tui.cs](#), una TUI mínima hecha solo con funciones de Console.

La meta del programa es simple:

- mantener una agenda en memoria
- mostrar un menú principal
- navegar con teclado
- listar contactos
- cargar un contacto nuevo con validación

No usa librerías externas. Toda la interfaz se arma con:

- `Clear()`
- `Write()` y `WriteLine()`
- `ReadKey(true)`
- `ReadLine()`
- `SetCursorPosition()`
- `ForegroundColor` y `ResetColor()`
- `CursorTop`, `CursorLeft` y `WindowWidth`

1. Estructura general

El programa define primero una agenda de contactos:

```
csharp
List<Contacto> agenda = [
    new("Analia", "Altamira", 5551234, 36),
    new("Beatriz", "Benitez", 5555678, 41),
    new("Carlos", "Córdoba", 5559012, 16),
    new("Diego", "Dominguez", 5553456, 28),
    new("Elena", "Espósito", 5557890, 22),
    new("Federico", "Fernandez", 5552345, 31),
    new("Gabriela", "Garcia", 5556789, 27),
    new("Hector", "Hernandez", 5550123, 45),
];
```

Luego entra en el bucle principal:

```
csharp
while(true) {
    Clear();
    Titulo("Menú Principal");
    int opcion = Menu("Listar Contactos", "Agregar Contacto", "Editar Contacto", "Borrar Contacto",
"Salir");
    switch(opcion) {
        case 0: ListarContactos(); break;
        case 1: AgregarContacto(); break;
        case 2: EditarContacto(); break;
        case 3: BorrarContacto(); break;
        case 4:
        case -1:
            WriteLine("\nChau!");
            return;
    }
}
```

Ese bucle organiza toda la interfaz:

- limpia la pantalla actual
- dibuja el título
- muestra el menú
- ejecuta la opción elegida

2. Preparación de la consola

Antes del bucle principal, el programa configura la salida Unicode:

```
csharp
Console.OutputEncoding = System.Text.Encoding.UTF8;
```

Eso permite que caracteres como Córdoba, Espósito o Menú se vean bien en pantalla.

3. Título de cada pantalla

Cada pantalla usa la misma función:

```
csharp
void Titulo(string mensaje) {
    ForegroundColor = ConsoleColor.DarkBlue;
    WriteLine($"\\n=== {mensaje} ===");
    ResetColor();
}
```

Acá se usan dos funciones importantes de Console:

- ForegroundColor para cambiar el color del título
- WriteLine() para imprimirlo

4. Menú con teclado

La función Menu dibuja las opciones y espera la tecla del usuario:

```
csharp
int Menu(params string[] opciones) {
    int y = CursorTop;
    int pos = 0;
    while(true) {
        SetCursorPosition(0, y);
        for(int i = 0; i < opciones.Length; i++) {
            MostrarCursor($"{i + 1}. {opciones[i]}", i == pos);
        }

        var key = ReadKey(true).Key;
        if(key == ConsoleKey.Escape) return -1;
        if(key == ConsoleKey.Enter) return pos;

        MoverCursor(key, opciones.Length, ref pos);
    }
}
```

Lo importante acá es:

- CursorTop guarda la fila inicial del menú
- SetCursorPosition(0, y) vuelve siempre al mismo bloque de líneas
- ReadKey(true) lee teclas sin mostrarlas

5. Cómo se marca la opción actual

Cada línea del menú se imprime con MostrarCursor:

```
csharp
void MostrarCursor(string texto, bool seleccionada) {
    ForegroundColor = seleccionada ? ConsoleColor.Green : ConsoleColor.Gray;
    WriteLine($" {(seleccionada ? ">" : " ")} {texto}");
    ResetColor();
}
```

Esta función define el aspecto de la selección:

- verde para la opción activa
- gris para las demás
- un prefijo > como cursor visual

6. Cómo se mueve la selección

La lógica de navegación está separada en MoverCursor:

```
csharp
void MoverCursor(ConsoleKey key, int cantidad, ref int actual) {
    actual = key switch {
        ConsoleKey.UpArrow => actual - 1,
        ConsoleKey.DownArrow => actual + 1,
        >= ConsoleKey.D0 and <= ConsoleKey.D9 => key - ConsoleKey.D1,
        _ => 0
    };
    actual = Math.Clamp(actual, 0, cantidad - 1);
}
```

Eso permite dos modos de uso:

- flechas arriba y abajo
- teclas numéricas

7. Pantalla de listado

La función ListarContactos muestra una lista navegable:

```
csharp
void ListarContactos() {
    Clear();
    Titulo("LISTA DE CONTACTOS");

    var y = CursorTop;
    var pos = 0;
    while(true){
        Console.SetCursorPosition(0, y);
        for(int i = 0; i < agenda.Count; i++) {
            var c = agenda[i];
            MostrarCursor($"{i + 1}. {c.NombreCompleto.PadRight(30)} | {c.Telefono.ToString("000-0000").PadRight(15)} | {c.Edad.ToString().PadLeft(3)} años", i == pos);
        }

        var key = ReadKey(true).Key;
        if(key == ConsoleKey.Escape || key == ConsoleKey.Enter) { return; }
        MoverCursor(key, agenda.Count, ref pos);
    }
}
```

Acá se combinan varias técnicas:

- Clear() para entrar a otra pantalla
- CursorTop para recordar desde qué fila redibujar
- SetCursorPosition() para volver siempre al inicio de la lista
- PadRight() y PadLeft() para alinear columnas

8. Alta de contacto

La función AgregarContacto usa un lector genérico para pedir los datos:

```

csharp
void AgregarContacto() {
    Clear();
    Titulo("Agregar Contacto");

    Contacto nuevo = new(
        Nombre: Leer<string>("Nombre", TryParseNombre),
        Apellido: Leer<string>("Apellido", TryParseNombre),
        Telefono: Leer<int>("Teléfono", TryParseNumero),
        Edad: Leer<int>("Edad", TryParseEdad)
    );

    agenda.Add(nuevo);
    WriteLine();
    MostrarContacto(nuevo);
    Pausa("Contacto agregado. \n\nPresione una tecla para continuar...");
}

```

Lo interesante es que el mismo lector sirve para distintos tipos (string e int).

9. Lector genérico

La función Leer<T> arma el formulario campo por campo:

```

csharp
T Leer<T>(string mensaje, Convertidor<T> convertir) {
    Console.ForegroundColor = ConsoleColor.Cyan;
    Write($"{mensaje.PadLeft(12)}: ");
    ResetColor();

    int y = CursorTop;

    while (true) {
        Console.SetCursorPosition(14, y);
        LimpiarLinea();

        var linea = ReadLine() ?? "";
        if (convertir(linea, out var valor)) {
            LimpiarLinea();
            return valor;
        }
    }
}

```

Paso a paso:

1. Escribe la etiqueta del campo.
2. Guarda la fila actual con CursorTop.
3. Mueve el cursor a la columna 14, donde empieza la entrada.
4. Limpia la línea actual.
5. Lee con ReadLine().
6. Intenta convertir el valor.
7. Si la conversión es válida, devuelve el resultado.

10. Limpiar una línea

Para reutilizar la misma zona de la pantalla, el programa usa LimpiarLinea:

```
csharp
void LimpiarLinea() {
    int x = CursorLeft, y = CursorTop;
    Write(new string(' ', WindowWidth - 1));
    Console.SetCursorPosition(x, y);
}
```

Esto borra la línea actual sin perder la posición del cursor.

11. Mensajes de error

Los errores se escriben debajo del campo actual:

```
csharp
void MostrarError(string mensaje = "") {
    Console.SetCursorPosition(14, CursorTop);
    LimpiarLinea();
    ForegroundColor = ConsoleColor.Red;
    Write(mensaje);
    ResetColor();
}
```

La idea es:

- mover el cursor a la zona del mensaje
- limpiar el contenido anterior
- escribir el error en rojo

12. Validadores

Cada tipo de dato tiene su propia validación.

Para nombre:

```
csharp
bool TryParseNombre(string texto, out string valor) {
    valor = texto.Trim().ToTitle();
    if( !string.IsNullOrEmpty(valor)) { return true; }
    MostrarError("El nombre no puede estar vacío.");
    return false;
}
```

Para teléfono:

csharp

```
bool TryParseNumero(string texto, out int valor) {  
    if (int.TryParse(texto, out valor)) { return true; }  
    MostrarError("Por favor, ingrese un número válido.");  
    return false;  
}
```

Para edad:

csharp

```
bool TryParseEdad(string texto, out int valor) {  
    if (int.TryParse(texto, out valor) && valor >= 18 && valor <= 65) { return true; }  
    MostrarError("Por favor, ingrese una edad válida (entre 18 y 65).");  
    return false;  
}
```

13. Mostrar un contacto

Después de dar de alta un contacto, el programa lo vuelve a mostrar:

csharp

```
void MostrarContacto(Contacto contacto) {  
    WriteLine($"    Nombre: {contacto.Nombre}");  
    WriteLine($"    Apellido: {contacto.Apellido}");  
    WriteLine($"    Teléfono: {contacto.Telefono.ToTelefono()}");  
    WriteLine($"    Edad: {contacto.Edad}");  
}
```

14. Pausa entre pantallas

La función Pausa detiene el flujo hasta que el usuario presione una tecla:

csharp

```
void Pausa(string mensaje = "Presione una tecla para continuar...") {  
    WriteLine();  
    ForegroundColor = ConsoleColor.DarkGray;  
    Write(mensaje);  
    ResetColor();  
    ReadKey(true);  
    LimpiarLinea();  
}
```

Esto se usa después de mostrar resultados o mensajes temporales.

15. Extensiones de formato

El programa usa dos extensiones para normalizar la salida:


```
csharp
static class Formatos {
    public static string ToTitle(this string texto) {
        CultureInfo cultura = new("es-AR");
        return cultura.TextInfo.ToTitleCase(texto.ToLower(cultura));
    }

    public static string ToTelefono(this int valor) {
        return valor.ToString("000-0000");
    }
}
```

16. Delegado y modelo

El lector genérico se apoya en este delegado:

```
csharp
delegate bool Convertidor<T>(string texto, out T valor);
```

Y los datos se representan con este record:

```
csharp
record Contacto(string Nombre, string Apellido, int Telefono, int Edad) {
    public string NombreCompleto => $"{Apellido}, {Nombre}";
    public override string ToString() => $"{Nombre} {Apellido} - {Telefono.ToTelefono()} - {Edad} años";
}
```

17. Estado actual del programa

En el código actual, estas funciones todavía están como placeholder:

```
csharp
void EditarContacto() {
    Titulo("Editar Contacto");
    Pausa("Funcionalidad no implementada. \n\nPresione una tecla para continuar...");
}

void BorrarContacto() {
    Titulo("Borrar Contacto");
    Pausa("Funcionalidad no implementada. \n\nPresione una tecla para continuar...");
}
```

Es importante marcarlo porque este tutorial está sincronizado con el código tal como está hoy, sin explicar funcionalidades que todavía no están implementadas.

18. Cómo ejecutarlo

```
bash
dotnet 23.6-consola-nano-tui.cs
```

19. Qué enseña este ejemplo

Este programa muestra cómo construir una TUI mínima usando solo funciones de consola:

- pantalla principal
- menú navegable
- listado con cursor visual
- formulario con validación
- mensajes de error en contexto

Si querés ver el código completo, está en [23.6-consola-nano-tui.cs](https://github.com/23.6-consola-nano-tui.cs).

CAPITULO 33

Terminal.Gui: cómo se piensa una interfaz de usuario en la terminal

Estos apuntes son sobre **diseñar interfaces** con **Terminal.Gui**, la librería de .NET para construir TUIs modernas. El ejemplo que recorreremos es un editor de una pequeña base de productos (código, descripción, precio, existencias), pero el foco no está en el programa: está en la interfaz. Vamos a esbozar las ideas, los componentes y las decisiones de diseño. **No vamos a escribir un programa terminado**. La persistencia, las validaciones de negocio y la importación/exportación las mencionamos solo cuando sirven para explicar una decisión de UI.

El objetivo es que, después de leer estos apuntes, sepas pensar la pantalla antes de tipear código y reconozcas los bloques con los que se arma una TUI:

- ventanas, paneles y diálogos;
- menús desplegables y atajos de teclado;
- formularios con campos de texto, números y checks;
- listas tipadas y búsqueda activa;
- separación entre la UI y la lógica del dominio;
- mensajes al usuario, validación y foco.

1. La idea: una interfaz gráfica dentro de la terminal

Una aplicación de consola tradicional imprime texto y lee texto:

```
csharp
Console.Write("Código: ");
string codigo = Console.ReadLine() ?? "";
```

Para un programa chico alcanza. Pero cuando hay muchas opciones, listas, formularios, búsqueda, edición y navegación, esa forma se vuelve incómoda muy rápido.

Una **TUI** (*Terminal User Interface*) usa la terminal entera como superficie visual. En lugar de imprimir líneas una debajo de otra, dibuja ventanas, paneles, menús, botones, listas y formularios. La terminal sigue siendo texto, pero la experiencia deja de ser una secuencia lineal de preguntas y respuestas.

Terminal.Gui resuelve por nosotros las tareas difíciles:

- dibujar correctamente en la pantalla;
- redibujar cuando cambia el tamaño de la terminal;
- manejar el foco entre componentes;

- capturar teclado y mouse;
- organizar componentes visuales;
- abrir diálogos modales;
- evitar que tengamos que usar `Console.SetCursorPosition` a mano.

La idea central es que **una aplicación no "imprime pantallas"**. Construye un árbol de objetos visuales y `TerminalGui` se encarga de dibujarlo, mover el foco, recibir eventos y redibujar cuando haga falta:

```

texto
Application
  ■■■■ ProductWindow
    ■■■■ MenuBar
    ■■■■ controles de búsqueda
    ■■■■ FrameView "Productos"
    ■ ■■■■ ListView<Producto>
    ■■■■ FrameView "Detalle"
    ■ ■■■■ Label
    ■■■■ Label de estado
  
```

Esto es lo primero que conviene internalizar: dejamos de pensar en "imprimo, leo, imprimo, leo" y empezamos a pensar en **un árbol de vistas**.

2. Filosofía de diseño

`TerminalGui` se parece más a WinForms, WPF o Avalonia que a `Console.WriteLine`.

Todo lo visible es un View

Un botón, una etiqueta, una lista, una ventana y un diálogo son `View` o derivan de `View`. Eso significa que todos comparten los mismos conceptos:

- posición: `X, Y`;
- tamaño: `Width, Height`;
- padre visual: `Superview`;
- hijos visuales: `Subviews`;
- foco;
- eventos;
- dibujo;
- layout.

El layout se expresa, no se calcula a mano

En vez de calcular coordenadas con enteros, `TerminalGui` usa:

- `Pos` para posiciones;
- `Dim` para tamaños.

Eso permite decir cosas como "pone este campo una columna a la derecha del label" o "que ocupe el resto del ancho dejando dos columnas libres", sin tener que recalcular nada cuando la terminal cambie de tamaño.

La aplicación es reactiva

No hay un `while` manual preguntando al usuario qué quiere hacer. `TerminalGui` ejecuta un bucle de eventos:

1. dibuja la interfaz;
2. espera una tecla, un click o un evento interno;
3. manda ese evento al componente con foco;
4. ejecuta los handlers;
5. redibuja lo que cambió.

Nuestro código casi no llama a nada activamente; **reacciona**. Se suscribe a eventos como `ValueChanged`, `Accepting`, `OpenSelectedItem`, y deja que el framework dispare la lógica cuando corresponda.

El teclado es un ciudadano de primera clase

Una buena TUI se puede usar sin mouse. `TerminalGui` ofrece foco con `Tab`, menús con `F10` o `Alt+letra`, hotkeys con `_`, atajos como `Ctrl+N`, eventos de teclado y comandos. Antes de agregar mouse, conviene que todo se pueda hacer con el teclado.

La interfaz no debería contener la lógica de negocio

El botón "Guardar" no tiene que saber cómo se persiste un producto. El botón llama a un método de la ventana, y ese método delega en una clase que se ocupa de los datos.

En nuestro ejemplo separamos cuatro responsabilidades:

- `ProductsWindow` organiza la UI y coordina las acciones;
- `ProductDialog` edita un producto;
- un **store** (cualquiera: `SQLite`, archivos, memoria) persiste productos;
- `Producto` representa los datos del dominio.

Esa separación es lo que nos permite cambiar el almacenamiento sin tocar la pantalla, y al revés.

3. Crear el proyecto

Para clase conviene una app C# de archivo único con .NET 10. Las dependencias se declaran al principio del `.cs`. Para una TUI básica alcanza con:

```
csharp
#:package TerminalGui@2.0.1
#:property PublishAot=false
```

Los using más usados son:

```
csharp
using Terminal.Gui.App;           // Application, IApplication, Runnable
using Terminal.Gui.Drawing;       // LineStyle, colores y dibujo
using Terminal.Gui.Input;         // Key, Command
using Terminal.Gui.ViewBase;      // View, Pos, Dim
using Terminal.Gui.Views;         // Window, FrameView, Button, TextField, etc.

using System.Collections.ObjectModel; // ObservableCollection
using System.Globalization;         // CultureInfo (formato de moneda/fecha)
```

Si más adelante el ejemplo necesita persistir en SQLite, JSON o lo que sea, eso se agrega aparte. No es parte de la TUI.

4. Ciclo de vida de la aplicación

El ciclo mínimo es:

```
csharp
using IApplication app = Application.Create().Init();
app.Run(new ProductsWindow(store));
```

Qué pasa ahí:

1. `Application.Create()` crea una instancia de aplicación.
2. `Init()` inicializa el driver de terminal, el loop principal y el manejo de input.
3. `Run(...)` ejecuta una pantalla, ventana o diálogo.
4. Cuando la pantalla pide detenerse, `Run` termina.
5. El `using` llama a `Dispose()` y restaura la terminal.

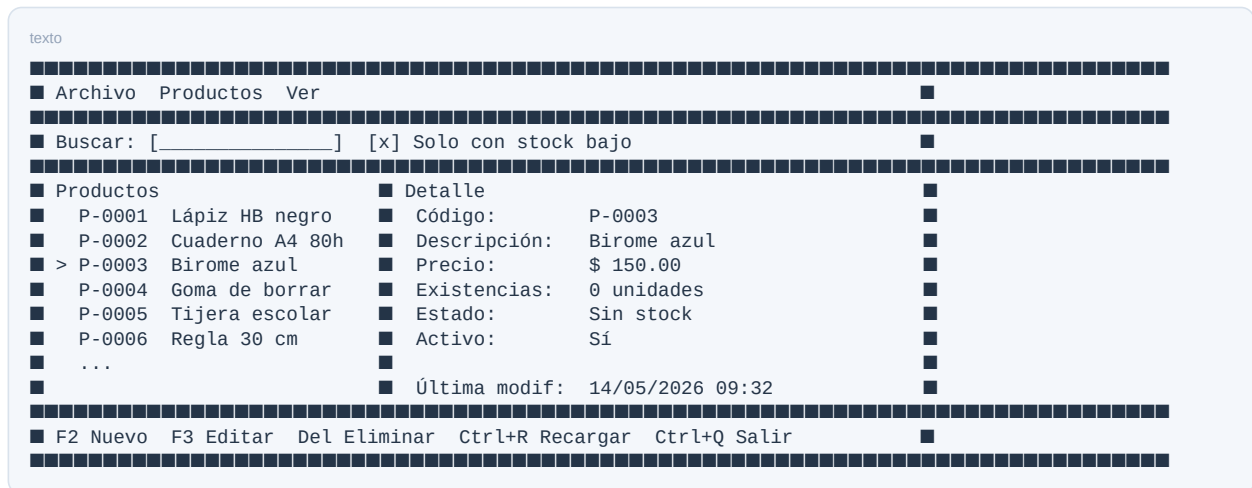
Para cerrar una ventana o diálogo se usa:

```
csharp
App!.RequestStop();
```

`RequestStop` no mata el proceso. Le pide al loop activo que termine de forma ordenada.

5. Pensar la pantalla antes de codear

Antes de tipear una línea, conviene dibujar la pantalla en papel o en un comentario ASCII. Para nuestro editor de productos imaginamos algo así:



Decisiones tomadas en este sketch:

- el **menú** vive arriba y agrupa acciones por categoría;
- la **búsqueda** está a la vista, no escondida en un submenú, porque es lo que más se usa;
- la **lista (maestro)** solo muestra código y descripción: lo justo para identificar y elegir;
- el **detalle** a la derecha muestra **todos** los datos del producto seleccionado, con formato amable (precio con símbolo de moneda, fecha legible, estado calculado);
- abajo, una **barra de ayuda** recuerda los atajos.

La razón pedagógica de ese reparto es importante: **la lista no es un reporte**. La lista sirve para *encontrar* un producto. El detalle sirve para *verlo entero*. Si metemos precio y stock en la lista, el usuario empieza a leer columnas en vez de buscar; y si encima los productos tienen 10 campos, la lista se vuelve ilegible. **Maestro escueto, detalle completo**.

Este sketch vale más que media hora de prueba y error con Pos y Dim. Una vez que sabemos qué queremos dibujar, el resto es traducción.

6. Vistas como organizador visual

Una View puede ser:

- un control concreto: Label, TextField, Button, CheckBox, ListView;
- un contenedor con borde y título: FrameView, Window, Dialog;
- una pantalla propia creada por nosotros (heredando de Runnable).

Las vistas se agregan a su contenedor con Add. Por ejemplo, el panel "Productos" tendrá adentro un ListView, no contra toda la pantalla sino contra el área interna del panel. Eso permite mover el panel y que la lista lo siga.

La estructura visual de nuestra pantalla refleja exactamente el sketch:

```

texto
ProductsWindow
    ■■■ MenuBar
    ■■■ Label "Buscar:"
    ■■■ TextField searchField
    ■■■ CheckBox lowStockOnly
    ■■■ FrameView "Productos"
    ■ ■■■ ListView<Producto> productsList
    ■■■ FrameView "Detalle"
    ■ ■■■ Label detailsLabel
    ■■■ Label statusBar

```

Cuando esta estructura está clara, el código casi se escribe solo.

7. Runners y pantallas ejecutables

En `Terminal.Gui` no se ejecuta cualquier objeto: se ejecuta uno que participa del ciclo de vida de la app. Para eso existe `IRunnable`; la clase cómoda para empezar es `Runnable`.

Esquema típico:

```

csharp
public sealed class ProductsWindow : Runnable {
    public ProductsWindow(IProductStore store) {
        Title = "Productos - Terminal.Gui";
        Width = Dim.Fill();
        Height = Dim.Fill();

        BuildLayout();
        LoadProducts();
    }
}

```

Cuando una pantalla necesita abrir un diálogo:

```

csharp
ProductDialog dialog = new("Editar producto", selected.Clone());
App!.Run(dialog);

if (dialog.Saved && dialog.Product is not null) {
    // aplicar cambios
}

```

`App!.Run(dialog)` abre un loop modal encima del actual. Mientras el diálogo está abierto, la ventana de atrás no recibe input. Cuando el diálogo llama a `App!.RequestStop()`, el control vuelve a la línea siguiente.

8. Ventanas, paneles y diálogos

Conviene distinguir cuatro contenedores:

Contenedor	Cuándo usarlo
Runnable	Pantalla ejecutable que pasa a <code>app.Run(...)</code> . Nuestra raíz.
Window	Ventana principal tradicional con borde y título.
FrameView	Panel con borde y título, dentro de una pantalla.
Dialog	Ventana modal para edición, confirmación o formularios.

En la pantalla de productos:

- `ProductsWindow` es un `Runnable` (raíz);
- los paneles "Productos" y "Detalle" son `FrameView`;
- el diálogo de edición es un `Dialog`;
- las confirmaciones de borrado las hace `MessageBox`, que internamente es otro `Dialog`.

La diferencia conceptual:

- `FrameView` **organiza** una pantalla;
- `Dialog` **interrumpe** la pantalla para pedir una decisión o datos;
- `Runnable/Window` es lo que corre en `app.Run(...)`.

9. Layout: ubicación con `Pos` y tamaño con `Dim`

La pantalla de terminal se mide en celdas de caracteres. Una coordenada no es un pixel; es una columna y una fila de texto.

Cada `View` tiene `X`, `Y`, `Width` y `Height`. La clave es entender que **no son enteros, son expresiones**.

Posiciones absolutas

```
csharp
X = 1;
Y = 2;
```

Útil para cosas simples cerca del borde superior izquierdo.

Posiciones relativas a otros views

```
csharp
X = Pos.Right(searchLabel) + 1;
Y = Pos.Top(searchLabel);
```

"A la derecha del label, en la misma fila". Esto es lo que se usa la mayor parte del tiempo en formularios.

Debajo de otro view

```
csharp
Y = Pos.Bottom(priceLabel) + 1;
```

Muy útil para apilar campos.

Anclar al final

```
csharp
Y = Pos.AnchorEnd(1);
```

La barra de estado va siempre en la última fila, sin importar el alto de la terminal.

Tamaños

Expresión	Significado
Width = 30	Ancho fijo de 30 columnas.
Width = Dim.Percent(60)	El 60% del ancho del padre.
Width = Dim.Fill()	Lo que sobre.
Width = Dim.Fill(2)	Lo que sobre, dejando 2 columnas libres al final.

La ventaja de este estilo es que si movemos un control, los demás se acomodan solos. Y si el usuario agranda la terminal, la pantalla se redibuja sin que tengamos que recalculr nada.

10. Componentes básicos de entrada

Para un editor de productos vamos a necesitar cinco tipos de componentes de entrada.

Label

Texto fijo. Se usa para títulos de campos y mensajes cortos.

```
csharp
new Label { Text = "Código:", X = 2, Y = 1 };
```

TextField

Entrada de una sola línea. Es la base de los campos de código y descripción.

```
csharp
TextField codeField = new() { X = 14, Y = 1, Width = 14 };
codeField.ValueChanged += (_, _) => /* validar formato */ ;
```

ValueChanged se dispara con cada cambio de texto. Para el campo `searchField` esto habilita la **búsqueda activa**: cada tecla refina la lista.

Campos numéricos

Terminal.Gui 2.x ofrece un `NumericTextField` (o se puede usar un `TextField` validando manualmente). Para el dominio de productos hay dos campos numéricos clave:

- **Precio**: decimal, con dos decimales, no negativo.
- **Existencias**: int, no negativo.

Pedagógicamente vale la pena distinguirlos: el precio tiene parte fraccionaria, las existencias no. La UI debería:

- alinear los números **a la derecha**, no a la izquierda como un texto cualquiera;
- mostrar el precio con formato (\$ 120,00 o 120.00, según corresponda);
- impedir tipear letras o signos no esperados.

Button

Ejecuta una acción.

csharp

```
Button saveButton = new() { Text = "Guardar", IsDefault = true };
saveButton.Accepting += (_, e) => { Save(); e.Handled = true; };
```

`IsDefault = true` hace que Enter active ese botón cuando el diálogo está enfocado. Es la acción principal.

CheckBox

Entrada booleana. En nuestro caso, para marcar "Solo con stock bajo" arriba, y "Activo" en el diálogo de edición.

csharp

```
CheckBox active = new() { Text = "_Activo", Value = CheckState.Checked };
```

El guion bajo marca la letra de atajo (Alt+A).

11. El patrón maestro-detalle

Llegamos al corazón de la pantalla. Dos paneles que se hablan:

- el **maestro** (panel izquierdo): una lista corta y escaneable. **Solo código y descripción**.
- el **detalle** (panel derecho): toda la información del producto seleccionado, con formato amable.

El usuario navega el maestro con las flechas; el detalle se actualiza solo.

El modelo del producto

Vamos a tener algo así:

```
csharp
public sealed class Producto {
    public int Id { get; set; }
    public string Codigo { get; set; } = "";
    public string Descripcion { get; set; } = "";
    public decimal Precio { get; set; }
    public int Existencias { get; set; }
    public bool Activo { get; set; } = true;
    public DateTime UltimaModif { get; set; } = DateTime.Now;

    public Producto Clone() => new() {
        Id = this.Id,
        Codigo = this.Codigo,
        Descripcion = this.Descripcion,
        Precio = this.Precio,
        Existencias = this.Existencias,
        Activo = this.Activo,
        UltimaModif = this.UltimaModif
    };

    // Solo código y descripción: este es el secreto del maestro escueto
    public override string ToString() {
        string cod = (this.Codigo ?? "").PadRight(8);
        string desc = this.Descripcion ?? "";
        if (desc.Length > 40) {
            desc = desc.Substring(0, 39) + "...";
        }
        return $"{cod} {desc}";
    }
}
```

La decisión clave está en `ToString()`. **El maestro no es un reporte**. Si querés ver precio o stock, lo ves en el detalle. La lista queda limpia, los códigos se alinean visualmente, y el ojo escanea rápido.

Los dos paneles

En el método `BuildLayout()` de la ventana, los paneles del maestro y el detalle se arman como dos `FrameView` lado a lado:

```
csharp
// Panel izquierdo: el maestro
FrameView listPanel = new() {
    Title = "Productos",
    X = 0,
    Y = 4, // debajo del menú y la búsqueda
    Width = Dim.Percent(42), // ocupa menos: solo código + descripción
    Height = Dim.Fill(2) // hasta arriba de la barra de estado
};

this.productsList = new ListView<Producto>() {
    X = 0, Y = 0,
    Width = Dim.Fill(),
    Height = Dim.Fill()
};

listPanel.Add(this.productsList);

// Panel derecho: el detalle
FrameView detailsPanel = new() {
    Title = "Detalle",
```

csharp - continuacion

```

X      = Pos.Right(listPanel),    // pegado al maestro
Y      = Pos.Top(listPanel),
Width  = Dim.Fill(),
Height = Dim.Fill(2)
};

this.detailsLabel = new Label() {
    X      = 1, Y = 0,
    Width  = Dim.Fill(1),
    Height = Dim.Fill()
};

detailsPanel.Add(this.detailsLabel);

Add(listPanel, detailsPanel);

```

Dos cosas que merecen atención:

- `Pos.Right(listPanel)` posiciona el panel derecho **a la derecha del izquierdo, sea cual sea el ancho del izquierdo**. Si mañana cambiamos `Dim.Percent(42)` por `Dim.Percent(35)`, el detalle se reacomoda solo.
- el `Label` del detalle puede contener varias líneas separadas por `\n`. No hace falta un control especial para mostrar varios campos.

La sincronización: el evento que las une

Para que el detalle se actualice cuando el usuario navega la lista, nos suscribimos al evento de cambio de selección del `ListView<Producto>`:

csharp

```

this.productsList.SelectedItemChanged += (_, _) => RefreshDetails();

```

Ese `RefreshDetails()` es el que escribe el contenido del panel derecho. Lo escribimos completo porque es el ejemplo concreto que pediste:

csharp

```

private void RefreshDetails() {
    Producto? selected = this.productsList.Value;

    if (selected is null) {
        this.detailsLabel.Text = " (sin selección)";
        return;
    }

    string estado = (selected.Existencias, selected.Activo) switch {
        (_, false) => "Producto inactivo",
        (0, _)     => "Sin stock",
        (< 5, _)   => "Stock bajo",
        _          => "OK"
    };

    string activo = selected.Activo ? "Sí" : "No";

    // Cultura argentina para el formato de moneda y fecha
    CultureInfo ar = CultureInfo.GetCultureInfo("es-AR");

    this.detailsLabel.Text =
        $"""
        Código:      {selected.Codigo}
        Descripción: {selected.Descripcion}
        """

```

csharp - continuacion

```

        Precio:      {selected.Precio.ToString("C2", ar)}
        Existencias: {selected.Existencias} unidad
        Estado:      {estado}

        Activo:      {activo}
        Última modif: {selected.UltimaModif.ToString("dd/MM/yyyy HH:mm", ar)}
    """;
}

```

Los puntos pedagógicos:

- **switch por patrones** para calcular el estado. Es un caso clásico donde C# moderno gana legibilidad: el orden de los casos importa (primero `inactivo`, después `sin stock`, después `stock bajo`, después `OK`) y leer las reglas como una tabla es más claro que cinco `if`.
- **Formato de moneda con cultura**: `Precio.ToString("C2", ar)` da \$ 150,00. Si dejamos la cultura por defecto en una terminal angloparlante, sale \$150.00 o algo más raro. Para apps argentinas, vale la pena fijarla.
- **Espacios en blanco al inicio** (" Código: ..."): el `Label` no tiene padding propio. Dos espacios manuales evitan que el texto se pegue al borde del panel.
- **Líneas en blanco** entre grupos lógicos: identificación, datos comerciales, metadatos. Eso hace que el detalle se lea como una ficha, no como un párrafo.

El flujo completo cuando el usuario navega

Vale la pena seguirlo mentalmente:

1. el usuario aprieta la flecha abajo;
2. el `ListView<Producto>` mueve el cursor a la siguiente fila;
3. internamente, dispara `SelectedItemChanged`;
4. nuestro handler llama a `RefreshDetails()`;
5. `RefreshDetails()` lee `productsList.Value` y vuelve a armar el texto del label de detalle;
6. `Terminal.Gui` redibuja el panel derecho.

Y todo eso sin un solo `while` ni un `Console.ReadKey`. Esa es la diferencia entre programar para consola y programar para una TUI.

Una nota sobre `SetSource`

Cuando recargamos productos, hay que pasarle la colección al `ListView`. La forma idiomática en `Terminal.Gui 2.x` es con `SetSource` sobre una colección observable:

csharp

```

this.productsList.SetSource(
    new ObservableCollection<Producto>(this.filteredProducts));

```

Eso garantiza que cuando la fuente cambia, el control se redibuja. Después de cualquier filtro, recarga o edición, llamamos a este `SetSource` con la lista filtrada actual y el `ListView` se actualiza.

12. Búsqueda en caliente

"Búsqueda en caliente" o "búsqueda activa" significa que **no hay botón Buscar**: el usuario tipea en un campo y la lista se filtra con cada tecla. Es la forma estándar en TUIs modernas y, una vez que se prueba, se vuelve adictiva.

Para implementarla bien hacen falta tres ingredientes: dos listas en memoria, un handler de cambio de texto y una rutina de refresco que respete la selección.

Dos listas en memoria

El primer instinto sería filtrar la lista original, pero eso es un error: si filtramos y después borramos el texto del campo de búsqueda, ya no tenemos los datos originales. La solución es mantener dos colecciones:

```
csharp
private List<Producto> products = new(); // todos los productos
private List<Producto> filteredProducts = new(); // los visibles ahora mismo
```

`products` se modifica al insertar, editar o eliminar (y al recargar desde el store). `filteredProducts` se recalcula cada vez que cambia el filtro. El `ListView` **siempre se alimenta de `filteredProducts`**, nunca de `products` directamente.

Cargar productos desde el store

La carga inicial llena `products` y dispara el primer refresco:

```
csharp
private void LoadProducts() {
    this.products = this.store.GetAll();
    RefreshProducts();
}
```

Suscribirse al campo de búsqueda

Suponiendo que en `BuildLayout()` creamos así el campo de búsqueda:

```
csharp
Label searchLabel = new() {
    Text = "Buscar:",
    X = 1, Y = 2
};

this.searchField = new TextField() {
    X = Pos.Right(searchLabel) + 1,
    Y = Pos.Top(searchLabel),
    Width = Dim.Percent(40)
};

this.lowStockOnly = new CheckBox() {
    Text = "Solo con _stock bajo",
    X = Pos.Right(this.searchField) + 2,
    Y = Pos.Top(this.searchField),
    Value = CheckState.Unchecked
};
```

csharp - continuacion

```
Add(searchLabel, this.searchField, this.lowStockOnly);
```

... el cableado de la búsqueda en caliente es de dos líneas:

csharp

```
this.searchField.TextChanged += (_, _) => RefreshProducts();  
this.lowStockOnly.CheckedStateChanged += (_, _) => RefreshProducts();
```

TextChanged se dispara con cada cambio de texto en el campo. No esperamos a que el usuario apriete Enter. No hay timer. No hay debounce. Cada tecla refresca.

Nota. En Terminal.Gui 2.x el evento se llama TextChanged. Versiones anteriores usaron ValueChanged. Si encontrás un nombre y no funciona, probá el otro.

El predicado de filtrado

Antes del refresco, definimos qué significa "coincide con la búsqueda". Lo aislamos en un método chico para que después se pueda extender:

csharp

```
private static bool Matches(Producto product, string search, bool onlyLowStock) {  
    if (onlyLowStock && product.Existencias > 5) {  
        return false;  
    }  
  
    if (string.IsNullOrWhiteSpace(search)) {  
        return true;  
    }  
  
    search = search.Trim().ToLowerInvariant();  
  
    return product.Codigo.ToLowerInvariant().Contains(search)  
        || product.Descripcion.ToLowerInvariant().Contains(search);  
}
```

Tres detalles deliberados:

- ToLowerInvariant en ambos lados: la búsqueda es **case-insensitive**, como espera cualquier usuario.
- Contains, **no** StartsWith: el usuario puede tipear "birome" o "azul" y encontrar "Birome azul". Buscar por prefijo se siente mezquino.
- **El filtro de stock se evalúa primero**: si está activo y el producto no califica, ni hace falta mirar el texto.

El refresco que respeta la selección

Acá está el método que junta todo. Es probablemente el método más cargado de la ventana, pero cada línea cumple un propósito:


```

csharp
private void RefreshProducts(int? selectedId = null) {
    // 1. Recordar qué producto estaba seleccionado, si no nos lo pasaron
    selectedId ??= this.productsList.Value?.Id;

    // 2. Leer el estado actual del filtro
    string search = this.searchField.Text?.ToString()?.Trim() ?? "";
    bool onlyLowStock = this.lowStockOnly.Value == CheckState.Checked;

    // 3. Recalcular la lista filtrada
    this.filteredProducts = this.products
        .Where(p => Matches(p, search, onlyLowStock))
        .OrderBy(p => p.Codigo)
        .ToList();

    // 4. Pasarle al ListView la nueva fuente
    this.productsList.SetSource(
        new ObservableCollection<Producto>(this.filteredProducts));

    // 5. Reposicionar el cursor sobre el producto que estaba seleccionado
    int index;
    if (this.filteredProducts.Count == 0) {
        index = -1;
    } else if (selectedId is null) {
        index = 0;
    } else {
        int found = this.filteredProducts.FindIndex(p => p.Id == selectedId.Value);
        index = found >= 0 ? found : 0;
    }
    this.productsList.Index = index;

    // 6. Forzar el refresco del detalle (por si cambió el seleccionado)
    RefreshDetails();

    // 7. Mensaje para el usuario
    SetStatus($"{this.filteredProducts.Count} de {this.products.Count} productos.");
}

```

El paso 5 es **lo que evita la sensación errática**. Sin él, cada vez que el usuario tipea una letra el cursor salta al primer producto. Con él, el cursor se queda sobre el producto que el usuario estaba mirando, **mientras siga apareciendo en los resultados**. Si deja de aparecer, el cursor cae elegantemente al primero.

Lo que el usuario ve

Imaginemos que en el maestro hay 50 productos y el usuario tipea, letra por letra, b - i - r:

Tecla	Filtro	Productos visibles
(vacío)	""	50 productos
b	"b"	12 (los que tienen una "b" en algún lado)
bi	"bi"	4
bir	"bir"	2 (Birome azul, Birome negra)

Y todo esto sucede **mientras tipea**, sin pulsar Enter, sin abrir un diálogo, sin clickear nada. Esa es la sensación que vale la pena entregarle al usuario.

Variantes que conviene conocer

- **Búsqueda con debounce:** en aplicaciones donde cada filtro cuesta caro (porque pega a una base remota), conviene esperar 200-300 ms desde la última tecla antes de filtrar. En nuestro ejemplo, con los datos en memoria, es innecesario.
- **Resaltar coincidencias:** pintar en otro color las letras que matchean dentro de la fila. Vistoso, pero requiere dibujar la fila a mano, en vez de delegar en `ToString()`.
- **Búsqueda por más campos:** si el día de mañana se quiere buscar también por proveedor, basta con extender `Matches` con un `||` más. La rutina de refresco no cambia.

13. Cajas de diálogo para edición

Una caja de diálogo de edición tiene tres responsabilidades, ni una más:

1. **mostrar campos** con los valores actuales;
2. **validar** lo que el usuario tipeó;
3. **devolver un resultado** (guardó o canceló).

Lo que **no** hace:

- no toca la base de datos;
- no modifica la lista global;
- no decide qué hacer con el resultado.

La ventana principal es la que decide qué pasa con el producto editado. Esta separación es lo que hace posible que el botón "Cancelar" funcione: no hay nada para deshacer porque nada se persistió todavía.

El sketch del diálogo

Antes de codear, dibujamos:

[illegible]

Decisiones:

- los campos se apilan de arriba abajo, con líneas en blanco para agrupar (identificación, datos comerciales, metadatos);

- los **labels van alineados a la izquierda** y los campos en una columna alineada, para que el ojo siga una sola línea vertical;
- los **botones van abajo a la derecha**, "Guardar" como acción principal y "Cancelar" al lado;
- Enter activa "Guardar" (es el botón por defecto), Escape activa "Cancelar".

El esqueleto de la clase

```
csharp
public sealed class ProductDialog : Dialog {
    private readonly TextField codeField;
    private readonly TextField descriptionField;
    private readonly TextField priceField;
    private readonly TextField stockField;
    private readonly CheckBox activeField;

    // Resultado que la ventana principal va a leer
    public bool Saved { get; private set; }
    public Producto? Product { get; private set; }

    public ProductDialog(string title, Producto product) {
        this.Product = product;

        this.Title = title;
        this.Width = 60;
        this.Height = 14;

        // ... crear campos, botones, suscribir eventos ...
    }
}
```

Notar dos cosas:

- el diálogo **recibe el producto desde afuera**, no lo crea él. Esto es lo que permite reutilizarlo para Nuevo (producto vacío) y para Editar (producto clonado).
- Saved y Product son las dos propiedades que la ventana principal va a consultar al cerrarse el diálogo. Es la única forma de comunicación de adentro hacia afuera.

Armar el layout dentro del constructor

Acá viene la parte más larga, pero es directa: una pareja Label + TextField por cada campo, posicionado en relación al anterior con Pos.Bottom. Todos los campos arrancan en la misma columna X para que se vean alineados.

```
csharp
// Campo: Código
Label codeLabel = new() {
    Text = "Código:",
    X = 2, Y = 1
};

this.codeField = new TextField() {
    Text = product.Codigo,
    X = 18, // columna fija para alinear
    Y = Pos.Top(codeLabel),
    Width = 12 // los códigos son cortos
};

// Campo: Descripción
Label descLabel = new() {
```

csharp - continuacion

```

    Text = "Descripción:",
    X = Pos.Left(codeLabel),
    Y = Pos.Bottom(codeLabel) + 1
};

this.descriptionField = new TextField() {
    Text = product.Descripcion,
    X = Pos.Left(this.codeField),
    Y = Pos.Top(descLabel),
    Width = Dim.Fill(2) // hasta el borde del diálogo
};

// Campo: Precio
Label priceLabel = new() {
    Text = "Precio:",
    X = Pos.Left(codeLabel),
    Y = Pos.Bottom(descLabel) + 2 // un renglón en blanco para separar
};

CultureInfo ar = CultureInfo.GetCultureInfo("es-AR");

this.priceField = new TextField() {
    Text = product.Precio.ToString("F2", ar), // formato 0.00
    X = Pos.Left(this.codeField),
    Y = Pos.Top(priceLabel),
    Width = 14
};

// Campo: Existencias
Label stockLabel = new() {
    Text = "Existencias:",
    X = Pos.Left(codeLabel),
    Y = Pos.Bottom(priceLabel) + 1
};

this.stockField = new TextField() {
    Text = product.Existencias.ToString(),
    X = Pos.Left(this.codeField),
    Y = Pos.Top(stockLabel),
    Width = 14
};

// CheckBox: Activo
this.activeField = new CheckBox() {
    Text = "_Activo",
    X = Pos.Left(codeLabel),
    Y = Pos.Bottom(stockLabel) + 2,
    Value = product.Activo ? CheckState.Checked : CheckState.Unchecked
};

Add(codeLabel, this.codeField,
    descLabel, this.descriptionField,
    priceLabel, this.priceField,
    stockLabel, this.stockField,
    this.activeField);

```

Hay varios patrones interesantes acá:

- `Pos.Left(codeLabel)` en todos los labels hace que la columna izquierda se alinee sola. Si cambiamos `X = 2` por `X = 3`, los seis labels se mueven juntos.
- `Pos.Left(this.codeField)` en todos los campos hace lo mismo con la columna de la derecha. Una sola decisión de diseño, replicada.
- `Pos.Bottom(...)` + 1 apila los campos uno bajo el otro. +2 cuando queremos una línea en blanco visible (separa grupos).
- `Width = Dim.Fill(2)` en la descripción la hace flexible hasta el borde derecho, mientras que `Width = 14` deja los campos numéricos cortos. **Eso es comunicación visual**: el ancho del campo le sugiere al

usuario qué se espera adentro.

- **El precio se inicializa formateado** con `ToString("F2", ar)`: si el producto vale 150, el campo arranca con "150,00", no con "150". Pequeño, pero da sensación de pulcritud.

Los botones

Un `Dialog` tiene un método `AddButton` que ubica los botones en la fila inferior automáticamente. No nos preocupamos por las coordenadas:

```
csharp
Button saveButton = new() {
    Text = "_Guardar",
    IsDefault = true // Enter dispara este botón
};

saveButton.Accepting += (_, e) => {
    Save(); // método que valida y cierra
    e.Handled = true;
};

Button cancelButton = new() {
    Text = "_Cancelar"
};

cancelButton.Accepting += (_, e) => {
    this.Saved = false;
    App!.RequestStop(); // cerrar sin guardar nada
    e.Handled = true;
};

AddButton(saveButton);
AddButton(cancelButton);
```

Tres cosas para subrayar:

- `IsDefault = true` en `Guardar` hace que `Enter`, desde cualquier campo del diálogo, dispare el guardado. Es lo que el usuario espera de un formulario.
- **El guion bajo** ("`_Guardar`", "`_Cancelar`") marca la letra del atajo: `Alt+G` y `Alt+C`.
- `e.Handled = true` corta la propagación del evento. Sin eso, ciertas situaciones disparan el handler dos veces.

El método `Save` con validación

Acá vive toda la lógica del diálogo. Es el único método largo de la clase, pero su estructura es la misma para cualquier formulario: validar campo por campo, mostrar error si falla, llenar el modelo si todo está bien.

```
csharp
private void Save() {
    // 1. Limpiar y validar el código
    string code = (this.codeField.Text?.ToString() ?? "").Trim();
    if (string.IsNullOrEmpty(code)) {
        MessageBox.ErrorQuery(App!, "Validación",
            "El código es obligatorio.", "OK");
        this.codeField.SetFocus();
        return;
    }

    // 2. Limpiar y validar la descripción
```

csharp - continuacion

```

string desc = (this.descriptionField.Text?.ToString() ?? "").Trim();
if (string.IsNullOrWhiteSpace(desc)) {
    MessageBox.ErrorQuery(App!, "Validación",
        "La descripción es obligatoria.", "OK");
    this.descriptionField.SetFocus();
    return;
}

// 3. Parsear y validar el precio (decimal, no negativo)
CultureInfo ar = CultureInfo.GetCultureInfo("es-AR");
string priceText = (this.priceField.Text?.ToString() ?? "").Trim();
if (!decimal.TryParse(priceText, NumberStyles.Number, ar, out decimal price)
    || price < 0) {
    MessageBox.ErrorQuery(App!, "Validación",
        "El precio debe ser un número mayor o igual a cero.", "OK");
    this.priceField.SetFocus();
    return;
}

// 4. Parsear y validar las existencias (entero, no negativo)
string stockText = (this.stockField.Text?.ToString() ?? "").Trim();
if (!int.TryParse(stockText, out int stock) || stock < 0) {
    MessageBox.ErrorQuery(App!, "Validación",
        "Las existencias deben ser un entero mayor o igual a cero.", "OK");
    this.stockField.SetFocus();
    return;
}

// 5. Todo OK: volcar al modelo
this.Product!.Codigo = code;
this.Product.Descripcion = desc;
this.Product.Precio = price;
this.Product.Existencias = stock;
this.Product.Activo = this.activeField.Value == CheckState.Checked;
this.Product.UltimaModif = DateTime.Now;

this.Saved = true;
App!.RequestStop();
}

```

Los puntos pedagógicos:

- El patrón "validar - mostrar - enfocar - return" se repite para cada campo. Es deliberadamente verboso: cada bloque es independiente, fácil de leer y fácil de modificar. No conviene meterlo en un loop sobre un array de validadores hasta que haya 15 campos.
- `decimal.TryParse` con `CultureInfo argentina`: acepta "150,00" como precio. Si usáramos la cultura por defecto, en una máquina con locale en inglés "150,00" se rechazaría. Es una decisión de UX: queremos que el usuario tipee como tipea normalmente.
- `int.TryParse` para las existencias: rechaza "0.5" o "abc" sin necesidad de un try/catch. `TryParse` siempre es mejor que `Parse` en código de UI.
- **Mantener el foco**: después de mostrar el error, `SetFocus()` lleva el cursor al campo culpable. Esto es lo que separa un formulario amable de uno que castiga al usuario.
- `UltimaModif = DateTime.Now` se setea acá, en el lugar único donde se acepta un cambio. Así nunca queda desincronizado con la edición real.

Edición sobre una copia

Cuando abrimos el diálogo para editar un producto existente, le pasamos una **copia** del original, no el original:

```

csharp
ProductDialog dialog = new("Editar producto", selected.Clone());
App!.Run(dialog);

if (dialog.Saved && dialog.Product is not null) {
    // recién acá decidimos qué hacer con el producto editado
    this.store.Update(dialog.Product);
    int index = this.products.FindIndex(p => p.Id == dialog.Product.Id);
    this.products[index] = dialog.Product;
    SetStatus($"Producto '{dialog.Product.Codigo}' actualizado.");
    RefreshProducts(dialog.Product.Id);
}

```

Si el usuario edita y cancela, el original queda intacto sin necesidad de "deshacer" nada: la copia se descarta y listo. Si el usuario guarda, recién entonces la ventana principal:

1. persiste con el store (primero la base);
2. reemplaza el objeto en la lista en memoria (después la memoria);
3. da feedback en la barra de estado;
4. refresca la lista pidiendo que mantenga seleccionado el producto recién editado.

El **orden importa**: si la base falla (por ejemplo, código duplicado), no queremos haber actualizado la memoria. Y si todo sale bien, el refresco final asegura que la fila del producto editado se vea con sus datos nuevos.

El mismo diálogo, dos usos

El mismo `ProductDialog` sirve para **crear** y para **editar**. La única diferencia está en lo que la ventana hace después:

```

csharp
private void NewProduct() {
    Producto product = new(); // sin Id, sin datos

    ProductDialog dialog = new("Nuevo producto", product);
    App!.Run(dialog);

    if (!dialog.Saved || dialog.Product is null) {
        return;
    }

    int newId = this.store.Insert(dialog.Product); // primero la base
    dialog.Product.Id = newId;
    this.products.Add(dialog.Product); // después la memoria
    SetStatus($"Producto '{dialog.Product.Codigo}' creado.");
    RefreshProducts(newId);
}

```

Es el mismo diálogo, la misma validación, la misma experiencia visual. Cambia solo el título y lo que se hace al cerrarse: `Insert` para Nuevo, `Update` para Editar. Esa simetría es lo que hace que la UI se sienta consistente: el usuario aprende a editar y ya sabe crear.

14. Menús desplegables

Un menú superior se arma con tres clases:

- `MenuBar`: la barra completa arriba;
- `MenuItem`: cada menú principal (Archivo, Productos, Ver);
- `MenuItem`: cada acción dentro del menú.

Para nuestro editor de productos:

```
csharp
MenuBar menu = new() {
    Menus = [
        new MenuItem("_Archivo", [
            new MenuItem("_Recargar", "Ctrl+R", ReloadProducts),
            new MenuItem("_Salir", "Ctrl+Q", RequestExit) ]),
        new MenuItem("_Productos", [
            new MenuItem("_Nuevo", "F2", NewProduct),
            new MenuItem("_Editar", "F3", EditSelectedProduct),
            new MenuItem("_Eliminar", "Del", DeleteSelectedProduct) ]),
        new MenuItem("_Ver", [
            new MenuItem("Solo _stock bajo", "", ToggleLowStockOnly) ]
    ]
};
```

Algunas reglas que merece la pena fijar:

- el guion bajo marca la letra rápida (_Archivo habilita Alt+A);
- el segundo texto ("Ctrl+R") es **ayuda visual** para el usuario, no implementa el atajo;
- el menú no contiene lógica; **delega** en métodos de la ventana;
- las opciones se agrupan por categoría, no por orden de aparición histórica.

15. Manejo de teclado

Terminal.Gui ya maneja muchas teclas por nosotros:

- Tab cambia el foco;
- las flechas navegan la lista;
- Enter acepta botones o abre el ítem seleccionado;
- F10 activa el menú;
- Escape cierra menús o diálogos.

Para los atajos propios sobreescibimos `OnKeyDown` en la ventana:


```
csharp
protected override bool OnKeyDown(Key key) {
    bool Handle(Action action) {
        action();
        return true;
    }

    return key switch {
        var k when k == Key.F2      => Handle(NewProduct),
        var k when k == Key.F3      => Handle(EditSelectedProduct),
        var k when k == Key.Delete  => Handle>DeleteSelectedProduct),
        var k when k == Key.R.WithCtrl => Handle(ReloadProducts),
        _ => base.OnKeyDown(key)
    };
}
```

- `return true` significa "esta tecla ya fue manejada, no la sigas propagando";
- `return base.OnKeyDown(key)` permite que `Terminal.Gui` haga lo suyo (mover foco, scrolllear, etc.).

Atajos sugeridos para el editor de productos:

texto	
F2	Nuevo producto
F3	Editar producto
Enter	Editar, si la lista tiene foco
Del	Eliminar producto
Ctrl+R	Recargar
Ctrl+Q	Salir
F4	Llevar foco al campo de búsqueda

Una buena TUI **se puede usar entera sin tocar el mouse**. Si al usar la app vos misma sentís que tenés que estirar la mano al mouse, falta un atajo.

16. Mensajes al usuario

Tres canales para hablarle al usuario, cada uno con su propósito:

Barra de estado

Un `Label` en `Y = Pos.AnchorEnd(1)` que muestra un mensaje corto: "Producto 'Lápiz HB' creado.", "5 productos cargados.", "Sin resultados."

Es la forma menos invasiva de dar feedback. No interrumpe, no requiere clickear OK. Se usa para resultados de operaciones que salieron bien.

`MessageBox.Query`

Para confirmaciones y decisiones binarias:

```
csharp
int answer = MessageBox.Query( App!, "Eliminar producto", $"¿Eliminar el producto '{selected.Descripcion}'?", "No", "Sí") ?? 0;

if (answer == 1) {
    // borrar
}
```

Devuelve el índice del botón apretado. Para evitar bugs por culpa de un click distraído, **el botón por defecto** ("No") debería ser el seguro.

MessageBox.ErrorQuery

Para errores reales: archivo no encontrado, conexión rota, validación fallida.

```
csharp
MessageBox.ErrorQuery(App!, "Validación", "El precio no puede ser negativo.", "OK");
```

Regla práctica: **un error nunca debería tirar el programa**. Si una operación puede fallar, va en un try/catch y el error se muestra acá.

17. Validación: dónde y cuándo

En el dominio de productos hay reglas naturales:

- el **código** no puede estar vacío y, según el negocio, podría tener formato fijo (P-9999, por ejemplo);
- la **descripción** no puede estar vacía;
- el **precio** no puede ser negativo;
- las **existencias** no pueden ser negativas;
- el **código** debería ser único.

Acá entra una decisión de diseño: **dónde valida cada cosa**.

Regla	Dónde valida
Tipo (que el precio sea un número)	En el campo, mientras se tipea (impide letras).
Formato (que el código sigue patrón)	En el diálogo, al apretar Guardar.
No vacío, no negativo	En el diálogo, al apretar Guardar.
Código único	En el store o en la ventana, antes de insertar.

La validación **inmediata** (al tipear) sirve para errores obvios y de bajo costo. La validación **al guardar** sirve para reglas que dependen de varios campos a la vez o de información que no está en el formulario (¿existe ya un producto con ese código?).

No querés validar la unicidad del código en cada pulsación de tecla: sería tirar una consulta a la base por cada letra. Pero sí querés impedir que el usuario tipee letras en el campo de precio.

18. Separación entre UI y datos

La ventana no conoce SQLite. No conoce JSON. No conoce archivos. La ventana habla con un **store** a través de una interfaz mínima:

```
csharp
public interface IProductStore {
    List<Producto> GetAll();
    int          Insert(Producto product);
    bool         Update(Producto product);
    bool         Delete(int id);
}
```

Detrás de esa interfaz puede haber:

- una implementación en memoria (InMemoryProductStore) para probar la UI sin tener una base;
- una implementación con SQLite (SqliteProductStore);
- una implementación que lea/escriba un JSON (JsonProductStore);
- una implementación que llame a una API HTTP.

La ventana no se entera de cuál es. Esta es la **inyección de dependencias** más sencilla que existe: el constructor recibe un IProductStore, y arriba de todo decidimos qué pasar:

```
csharp
IProductStore store = new InMemoryProductStore(); // o el que sea
app.Run(new ProductsWindow(store));
```

Esto es lo que hace que la UI sea testeable, evolucionable y reutilizable. Si después decidimos persistir en otra cosa, no tocamos ni una línea de la pantalla.

19. Visualización condicional: stock bajo

El dominio de productos da una oportunidad pedagógica linda: **resaltar visualmente** los productos con poco stock. Hay tres formas principales y conviene conocer las tres.

1. Indicador en el texto del maestro

Lo más simple. En el ToString() del producto, agregamos un caracter al principio:

```
csharp
public override string ToString() {
    string flag = (this.Existencias <= 5) ? "!" : " ";
    string cod  = (this.Codigo ?? "").PadRight(8);
    string desc = this.Descripcion ?? "";
    if (desc.Length > 38) { desc = desc.Substring(0, 37) + "..."; }
    return $"{flag} {cod} {desc}";
}
```

Resultado en la lista:

```
texto
P-0001  Lápiz HB negro
! P-0002  Cuaderno A4 80h
P-0003  Birome azul
! P-0004  Goma de borrar
```

Funciona en cualquier terminal, no requiere colores, y respeta la regla del maestro escueto: ni precio ni cantidad invaden la lista. Es el equivalente al asterisco de los favoritos en una agenda.

2. Color en la fila del maestro

TerminalGui permite dibujar con `ColorScheme` propios. Una fila con stock bajo puede pintarse en rojo. Es vistoso, pero hay que cuidarlo: necesita ser legible en terminales con tema claro y oscuro, y conviene no abusar para que no parezca un árbol de Navidad.

3. Panel de detalle con campo "Estado"

Es lo que ya hicimos en `RefreshDetails`: el campo Estado se calcula con un `switch` sobre existencias y Activo, y se muestra como una línea más en la ficha. Es el modo más conservador y suele ser el más útil en aplicaciones de negocio: el usuario ve el estado **cuando elige el producto**, no necesita escanear toda la lista.

Las tres se pueden combinar: indicador discreto en la fila + texto explícito en el detalle. Cada una refuerza la otra sin saturar.

20. Orden recomendado para construir una TUI

Para no perderse, conviene construir en este orden:

1. **Sketch en papel o ASCII** de la pantalla principal.
2. **Modelo de datos** (Producto) con los campos básicos.
3. **Store en memoria** que cumpla `IProductStore`, con dos o tres productos de prueba hardcodeados.
4. **Ventana principal vacía** con título y borde.
5. **Menú superior**, aunque las acciones no hagan nada todavía.
6. **Panel izquierdo con ListView** mostrando los productos de prueba.
7. **Panel derecho con detalle** del producto seleccionado.
8. **Búsqueda activa** que filtre la lista.
9. **Diálogo de edición** que se abra y vuelva, sin todavía persistir.
10. **Acciones CRUD** conectadas al store en memoria.
11. **Atajos de teclado**.
12. **Recargar y salida limpia**.
13. **Cambiar el store a uno persistente** (SQLite, archivos, lo que toque).

Si se intenta hacer todo junto, los errores de layout, eventos y datos se mezclan y no se sabe dónde está la falla. Empezar con un store en memoria es lo que permite enfocarse en la UI primero.

21. Errores comunes

Querer parsear el texto de la fila

La fila se dibuja con `ToString()`, pero la fuente de verdad sigue siendo el objeto `Producto`. Si necesitamos saber qué seleccionó el usuario, leemos `productsList.Value`, no el texto.

Validar dos veces lo mismo (y diferente)

Si el diálogo valida que el precio no sea negativo pero el store no lo hace, una llamada directa al store puede meter datos malos. Si el store valida pero el diálogo no, el usuario apreta Guardar y recién ahí se entera del error. La regla: **el diálogo valida para que el usuario no se frustre, el store valida para garantizar la integridad de los datos**. Ambos, no uno solo.

Editar el objeto original antes de Guardar

Si el diálogo modifica directamente el `Producto` que vino de la lista, no hay forma de cancelar. Hay que trabajar sobre una copia (`Clone()`) y solo aplicar los cambios cuando el usuario confirma.

Mantener la lista en memoria desactualizada

Después de cada operación CRUD hay que actualizar **dos** lugares: el store, y la lista en memoria que alimenta al `ListView`. Si solo actualizamos uno, la UI queda inconsistente. La regla: persistir primero, actualizar memoria después, refrescar la lista por último.

Confiar en el orden visual

Si el usuario ordena la lista por precio descendente y nosotros guardamos "el producto en el índice 3", a la próxima ordenada eso ya no significa nada. Siempre que necesitemos referenciar un producto, usamos su `Id`, no su posición.

Botones por defecto peligrosos

Un diálogo de confirmación de borrado cuyo botón por defecto es "Sí" es una trampa. El default tiene que ser el seguro, en general "No" o "Cancelar".

Olvidarse del teclado

Toda funcionalidad de menú o botón debería tener un atajo de teclado. Probar la app **sin tocar el mouse** es el mejor test de usabilidad de una TUI.

Sobrecargar el sketch inicial

Empezar con 12 botones y 8 paneles. La pantalla cabe, pero el usuario se pierde. Es mejor empezar con poco y agregar lo que falte cuando un caso de uso lo pida.

22. Fuentes oficiales consultadas

- Terminal.Gui — inicio: <https://gui-cs.github.io/Terminal.Gui/docs/getting-started.html>
- Terminal.Gui — arquitectura de aplicación: <https://gui-cs.github.io/Terminal.Gui/docs/application.html>
- Terminal.Gui — configuración: <https://gui-cs.github.io/Terminal.Gui/docs/config.html>
- Terminal.Gui — layout: <https://gui-cs.github.io/Terminal.Gui/docs/layout.html>
- Terminal.Gui — View: <https://gui-cs.github.io/Terminal.Gui/docs/View.html>
- Terminal.Gui — menús: <https://gui-cs.github.io/Terminal.Gui/docs/menus.html>
- Terminal.Gui — teclado: <https://gui-cs.github.io/Terminal.Gui/docs/keyboard.html>

CAPITULO 34

Mini CRM con TUI, ADO.NET y Dapper — Diseño por capas

Un mini CRM (contactos + oportunidades + actividades) que combina Terminal.Gui, SQLite y Dapper.

Arquitectura en cuatro capas

texto

Presentación (TUI)	→ CrmWindow, diálogos, listas
Aplicación	→ CrmWindow coordina acciones del usuario
Persistencia	→ CrmDatabase (SQL con Dapper)
Dominio	→ Contact, Opportunity, Activity

Dominio — las entidades

csharp

```
public class Contact {
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public string Company { get; set; } = "";
    public string Email { get; set; } = "";
    public string Phone { get; set; } = "";
    public string Status { get; set; } = "Lead"; // Lead / Active / Inactive
    public string Notes { get; set; } = "";
    public DateTime CreatedAt { get; set; } = DateTime.Now;
}

public class Opportunity {
    public int Id { get; set; }
    public int ContactId { get; set; }
    public string Title { get; set; } = "";
    public string Stage { get; set; } = "Prospecting";
    public decimal Value { get; set; }
    public DateTime ExpectedCloseDate { get; set; }
    public string Notes { get; set; } = "";
}

public class Activity {
    public int Id { get; set; }
    public int ContactId { get; set; }
    public string Type { get; set; } = "Call"; // Call / Email / Meeting
    public string Subject { get; set; } = "";
    public DateTime DueDate { get; set; }
    public bool IsDone { get; set; }
    public string Notes { get; set; } = "";
}
```

Persistencia — CrmDatabase con Dapper

csharp

```

public class CrmDatabase {
    private readonly string connStr;

    public CrmDatabase(string dbPath) {
        connStr = $"Data Source={dbPath}";
        Initialize();
    }

    private IDbConnection Conn() => new SqlConnection(connStr);

    private void Initialize() {
        using var conn = Conn();
        conn.Execute("""
            CREATE TABLE IF NOT EXISTS Contacts (
                Id INTEGER PRIMARY KEY AUTOINCREMENT,
                Name TEXT NOT NULL, Company TEXT, Email TEXT,
                Phone TEXT, Status TEXT, Notes TEXT,
                CreatedAt TEXT DEFAULT (datetime('now'))
            );
            CREATE TABLE IF NOT EXISTS Opportunities (
                Id INTEGER PRIMARY KEY AUTOINCREMENT,
                ContactId INTEGER REFERENCES Contacts(Id) ON DELETE CASCADE,
                Title TEXT, Stage TEXT, Value REAL,
                ExpectedCloseDate TEXT, Notes TEXT
            );
            CREATE TABLE IF NOT EXISTS Activities (
                Id INTEGER PRIMARY KEY AUTOINCREMENT,
                ContactId INTEGER REFERENCES Contacts(Id) ON DELETE CASCADE,
                Type TEXT, Subject TEXT, DueDate TEXT,
                IsDone INTEGER DEFAULT 0, Notes TEXT
            );
            """);
    }

    // Búsqueda
    public List<Contact> SearchContacts(string q) {
        using var conn = Conn();
        return conn.Query<Contact>(
            "SELECT * FROM Contacts WHERE Name LIKE @q OR Email LIKE @q OR Company LIKE @q",
            new { q = $"%{q}%" }).ToList();
    }

    // CRUD contactos
    public void AddContact(Contact c) {
        using var conn = Conn();
        conn.Execute("INSERT INTO Contacts (Name, Company, Email, Phone, Status, Notes) " +
            "VALUES (@Name, @Company, @Email, @Phone, @Status, @Notes)", c);
    }

    public void UpdateContact(Contact c) {
        using var conn = Conn();
        conn.Execute("UPDATE Contacts SET Name=@Name, Company=@Company, Email=@Email, " +
            "Phone=@Phone, Status=@Status, Notes=@Notes WHERE Id=@Id", c);
    }

    public void DeleteContact(int id) {
        using var conn = Conn();
        conn.Execute("DELETE FROM Contacts WHERE Id=@id", new { id });
    }

    // Oportunidades y actividades por contacto
    public List<Opportunity> GetOpportunities(int contactId) {
        using var conn = Conn();
        return conn.Query<Opportunity>(
            "SELECT * FROM Opportunities WHERE ContactId=@contactId", new { contactId }).ToList();
    }

    public List<Activity> GetActivities(int contactId) {
        using var conn = Conn();
        return conn.Query<Activity>(
            "SELECT * FROM Activities WHERE ContactId=@contactId ORDER BY DueDate", new { contactId
        }).ToList();
    }
}

```


csharp - continuacion

```

var acts = db.GetActivities(selectedContact.Id);
activitiesList.SetSource(acts.Select(a =>
    $"[{a.IsDone ? "✓" : " "}] {a.Subject, -30} {a.DueDate:dd/MM}").ToList());
    }
}

```

Flujo de datos al editar

texto

```

Usuario presiona F2
↓
CrmWindow abre ContactDialog (modal)
↓
Application.Run(dialog) – bloquea hasta que cierra
↓
si !dialog.Cancelado → db.UpdateContact(dialog.Resultado)
↓
RefreshContacts() – la lista refleja el cambio

```

Decisiones de diseño

Decisión	Razón
ON DELETE CASCADE en BD	Al borrar un contacto, sus datos hijos se eliminan automáticamente
Datos semilla al inicio	Permite probar la app inmediatamente
Un archivo, capas conceptuales	Simplicidad didáctica sin perder separación de responsabilidades
SQL explícito con Dapper	Se ve qué hace la app sin abstracciones de ORM
Diálogos modales para edición	Aíslan la tarea; el usuario no pierde contexto
RefreshX() centralizado	Evita duplicar lógica de actualización en cada evento

CAPITULO 35

Programación Web Backend con C#, Minimal APIs, SQLite, ADO.NET y Dapper

Programación III - UTN Tucumán C# 14 / .NET 10 - Minimal APIs - SQLite - ADO.NET - Dapper - Dapper.Contrib - HttpClient

Este tutorial unifica y ordena los materiales anteriores en una secuencia simple:

1. entender qué problema resuelve un backend;
2. entender HTTP y REST;
3. construir endpoints con Minimal APIs;
4. persistir datos con SQLite;
5. usar ADO.NET, Dapper y Dapper.Contrib;
6. consumir la API desde C# y desde HTML;
7. agregar validaciones, logging, archivos estáticos, CORS y middleware.

La idea no es memorizar código. La idea es poder leer una request, entender qué endpoint la atiende, qué datos usa, qué respuesta devuelve y por qué.

1. Qué problema resuelve un backend

Una aplicación de consola corre en una computadora. Sus datos viven ahí. Si otra aplicación necesita leer o modificar esos datos, hace falta una forma clara de comunicarse.

Un **backend web** es un programa que:

1. escucha pedidos HTTP;
2. interpreta qué quiere el cliente;
3. ejecuta lógica de aplicación;
4. consulta o modifica datos;
5. devuelve una respuesta HTTP.

Esquema general:

texto

```
Cliente HTTP
  ↓ request
Backend ASP.NET Core
  ↓ lógica
Base de datos SQLite
  ↑ datos
Backend ASP.NET Core
  ↑ response
Cliente HTTP
```

El cliente puede ser:

- un navegador;
- una app mobile;
- otro backend;
- un programa de consola;
- un test automatizado;
- una página HTML con JavaScript.

El contrato entre cliente y servidor es **HTTP**.

2. HTTP como protocolo de mensajes

HTTP es un protocolo basado en mensajes. En cada interacción hay dos mensajes:

texto

```
Request → pedido que envía el cliente
Response → respuesta que devuelve el servidor
```

Request

Una request tiene esta estructura conceptual:

texto

```
Método + ruta + versión
Headers
Body opcional
```

Ejemplo:

http

```
POST /productos HTTP/1.1
Host: localhost:5000
Content-Type: application/json
Accept: application/json

{
  "nombre": "Monitor",
  "precio": 250000
}
```

Se lee así:

texto

Quiero crear algo dentro de /productos.
Te envío datos en JSON.
Espero una respuesta JSON.

Response

Una response tiene esta estructura conceptual:

texto

Versión + status code
Headers
Body opcional

Ejemplo:

http

HTTP/1.1 201 Created
Content-Type: application/json
Location: /productos/7

```
{  
  "id": 7,  
  "nombre": "Monitor",  
  "precio": 250000  
}
```

Se lee así:

texto

El recurso fue creado correctamente.
La representación está en JSON.
El nuevo recurso está en /productos/7.

Métodos HTTP

El método indica la intención del pedido.

Método	Intención principal
GET	Leer datos.
POST	Crear un recurso o ejecutar una operación de creación.
PUT	Reemplazar un recurso completo.
PATCH	Modificar parcialmente un recurso.
DELETE	Eliminar un recurso.

GET normalmente no lleva body. POST, PUT y PATCH normalmente sí.

Status codes principales

El status code comunica el resultado de la operación.

Código	Nombre	Uso típico
200	OK	La operación salió bien y devuelve contenido.
201	Created	Se creó un recurso.
204	No Content	La operación salió bien pero no devuelve body.
400	Bad Request	El cliente mandó datos inválidos.
401	Unauthorized	Falta autenticación.
403	Forbidden	Está autenticado, pero no tiene permiso.
404	Not Found	El recurso no existe.
409	Conflict	Hay conflicto con el estado actual.
422	Unprocessable Entity	El JSON está bien formado, los datos no son válidos para regla de negocio.
500	Internal Server Error	Falló el servidor.

Una API no solo devuelve datos. Devuelve datos más significado.

texto

404 no significa "no hay nada".
404 significa "el recurso solicitado no existe".

3. JSON

JSON es texto plano para representar datos.

Ejemplo:

json

```
{
  "id": 5,
  "nombre": "Mouse",
  "precio": 10000,
  "activo": true
}
```

Tipos comunes:

Tipo	Ejemplo
Número	42, 3.14

Tipo	Ejemplo
Texto	"Hola"
Booleano	true, false
Nulo	null
Objeto	{ "clave": "valor" }
Array	[1, 2, 3]

ASP.NET Core convierte objetos C# a JSON automáticamente cuando una API devuelve objetos.

4. REST

REST es una forma de diseñar APIs alrededor de **recursos**.

Un recurso es algo con identidad dentro del sistema:

```
texto
Producto
Cliente
Factura
Turno
Alumno
Inscripción
```

En REST no diseñamos URLs como acciones:

```
texto
/crearProducto
/borrarProducto
/listarProductos
```

Diseñamos URLs como recursos:

```
texto
/productos
/productos/5
```

Y usamos el método HTTP para expresar la operación:

Request	Significado
GET /productos	Leer la colección de productos.
GET /productos/5	Leer el producto con id 5.
POST /productos	Crear un producto nuevo.
PUT /productos/5	Reemplazar el producto 5.
PATCH /productos/5	Modificar parte del producto 5.

Request	Significado
DELETE /productos/5	Eliminar el producto 5.

Colección y recurso individual

```
texto
/productos
```

representa la colección completa.

```
texto
/productos/5
```

representa un producto concreto.

Representación

El recurso real no viaja por la red. Lo que viaja es una representación.

El producto vive en la base de datos. La API devuelve una representación JSON:

```
json
{
  "id": 5,
  "nombre": "Mouse",
  "precio": 10000
}
```

Conviene no confundir:

```
texto
Tabla de base de datos ≠ entidad C# ≠ DTO ≠ JSON público
```

A veces se parecen. No son lo mismo.

Métodos seguros e idempotentes

GET es seguro: no debería modificar el estado del servidor.

PUT y DELETE son idempotentes: repetir la misma operación debería dejar el sistema en el mismo estado final.

Ejemplo:

```
texto
DELETE /productos/5
DELETE /productos/5
DELETE /productos/5
```

Después de la primera llamada, el producto ya no existe. Repetir la request no debería eliminar productos distintos.

POST no es necesariamente idempotente:

```
texto
POST /productos
POST /productos
POST /productos
```

puede crear tres productos distintos.

PUT vs PATCH

PUT reemplaza el recurso completo.

```
http
PUT /productos/5
Content-Type: application/json

{
  "nombre": "Mouse inalámbrico",
  "precio": 18000
}
```

PATCH modifica parcialmente.

```
http
PATCH /productos/5
Content-Type: application/json

{
  "precio": 18000
}
```

Para un primer CRUD alcanza con PUT. PATCH se puede dejar como mejora.

5. Primera Minimal API

Crear un archivo `app.cs`:

```
csharp
#sdk Microsoft.NET.Sdk.Web

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/", () => "API funcionando");

app.MapGet("/saludo", () => "Hola desde el servidor");

app.Run();
```

Ejecutar:

```
bash
dotnet run app.cs
```

La línea importante:

```
csharp
app.MapGet("/saludo", () => "Hola desde el servidor");
```

Se lee así:

```
texto
Cuando llegue un GET a /saludo,
ejecutá esta función
y devolvé el resultado como response HTTP.
```

Un **endpoint** es una función disponible por HTTP.

6. Rutas y parámetros

Ahora agregamos un parámetro en la ruta:

```
csharp
#:sdk Microsoft.NET.Sdk.Web

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/productos/{id:int}", (int id) => {
    return $"Producto solicitado: {id}";
});

app.Run();
```

Request:

```
http
GET /productos/10
```

Respuesta:

```
texto
Producto solicitado: 10
```

La ruta:

```
csharp
"/productos/{id:int}"
```

indica que `id` viene desde la URL y debe poder convertirse a `int`.

Si llega:

```
texto
/productos/abc
```

no coincide con esa ruta porque abc no es entero.

También podemos recibir datos desde la **query string**.

En Minimal APIs, si un parámetro simple no está en la ruta, ASP.NET Core lo busca en la query string.

Ejemplo:

```
csharp
app.MapGet("/productos/buscar", (string texto) => {
    return $"Buscando productos que contengan: {texto}";
});
```

Request:

```
http
GET /productos/buscar?texto=mouse
```

Respuesta:

```
texto
Buscando productos que contengan: mouse
```

En este caso:

```
texto
/productos/buscar?texto=mouse
    ■■■ texto llega al parámetro string texto
```

La ruta identifica la operación:

```
texto
/productos/buscar
```

La query string manda el criterio de búsqueda:

```
texto
?texto=mouse
```

También se pueden recibir varios valores:

```
csharp
app.MapGet("/productos", (int pagina, int tamaño) => {
    return $"Página {pagina}, tamaño {tamaño}";
});
```

Request:

```
http
GET /productos?pagina=2&tamaño=20
```

Si un parámetro de query string puede faltar, conviene declararlo como nullable o darle un valor por defecto:

```
csharp
app.MapGet("/productos", (string? texto, int pagina = 1) => {
    return $"Texto: {texto}, página: {pagina}";
});
```

Request válida:

```
http
GET /productos?texto=teclado
```

Como pagina no llegó en la query string, toma el valor por defecto 1.

También podemos recibir valores desde las **cabeceras** o **headers** de la request.

Para eso usamos el atributo [FromHeader].

Primero hay que agregar este using:

```
csharp
using Microsoft.AspNetCore.Mvc;
```

Ejemplo:

```
csharp
app.MapGet("/perfil", ([FromHeader(Name = "Authorization")] string authorization) => {
    return $"Header Authorization recibido: {authorization}";
});
```

Request:

```
http
GET /perfil HTTP/1.1
Host: localhost:5000
Authorization: Bearer abc123
```

Respuesta:

```
texto
Header Authorization recibido: Bearer abc123
```

En este caso:

```
texto
Authorization: Bearer abc123
■■■ llega al parámetro string authorization
```

Si el nombre del parámetro coincide con el nombre del header, se puede escribir más corto:

```
csharp
app.MapGet("/cliente", ([FromHeader] string origen) => {
    return $"Origen: {origen}";
});
```

Request:

```
http
GET /cliente HTTP/1.1
Host: localhost:5000
Origen: web
```

Pero algunos headers tienen guiones, como User-Agent. En C# no podemos tener un parámetro llamado user-agent, por eso hay que indicar el nombre real del header:

```
csharp
app.MapGet("/cliente", ([FromHeader(Name = "User-Agent")] string userAgent) => {
    return $"User-Agent: {userAgent}";
});
```

Si un header puede faltar, conviene declararlo como nullable:

```
csharp
app.MapGet("/perfil", ([FromHeader(Name = "Authorization")] string? authorization) => {
    if (authorization is null) {
        return Results.Unauthorized();
    }
    return Results.Ok($"Header recibido: {authorization}");
});
```

Otra forma es recibir el **contexto HTTP** completo.

El contexto representa toda la request y toda la response. Desde ahí podemos leer la ruta, la query string, los headers, el body, el usuario autenticado, la IP del cliente y otros datos del pedido.

Ejemplo leyendo un header desde HttpContext:

```
csharp
app.MapGet("/perfil", (HttpContext context) => {
    string authorization = context.Request.Headers["Authorization"].ToString();
    return $"Header Authorization recibido: {authorization}";
});
```

Request:

```
http
GET /perfil HTTP/1.1
Host: localhost:5000
Authorization: Bearer abc123
```

El valor se lee desde:

```
csharp
context.Request.Headers["Authorization"]
```

Si el header puede faltar, es mejor verificarlo:

```
csharp
app.MapGet("/perfil", (HttpContext context) => {
    if (!context.Request.Headers.TryGetValue("Authorization", out var authorization)) {
        return Results.Unauthorized();
    }

    return Results.Ok($"Header recibido: {authorization}");
});
```

También se pueden leer valores de la query string desde el contexto:

```
csharp
app.MapGet("/productos/buscar", (HttpContext context) => {
    string texto = context.Request.Query["texto"].ToString();

    return $"Buscando productos que contengan: {texto}";
});
```

Request:

```
http
GET /productos/buscar?texto=mouse
```

El valor se lee desde:

```
csharp
context.Request.Query["texto"]
```

En general, conviene usar parámetros directos cuando el endpoint es simple:

```
csharp
app.MapGet("/productos/buscar", (string texto) => {
    return $"Buscando productos que contengan: {texto}";
});
```

Y conviene usar `HttpContext` cuando necesitamos acceder a información más general de la request:

```
csharp
app.MapGet("/diagnostico", (HttpContext context) => {
    string metodo = context.Request.Method;
    string ruta = context.Request.Path;
    string userAgent = context.Request.Headers["User-Agent"].ToString();

    return Results.Ok(new {
        Metodo = metodo,
        Ruta = ruta,
        UserAgent = userAgent
    });
});
```

Regla práctica:

texto

Ruta → identifica el recurso.
Query string → manda filtros, orden, paginación u opciones simples.
Headers → mandan metadatos del pedido, como autenticación o formato esperado.
Body → manda datos principales para crear o modificar recursos.
HttpContext → permite acceder manualmente a toda la request y la response.

Parámetros que vienen de servicios

No todos los parámetros de un endpoint salen de la request.

ASP.NET Core también puede completar parámetros usando objetos registrados como servicios.

Ejemplo:

csharp

```
var builder = WebApplication.CreateBuilder(args);

DatabaseOptions databaseOptions = new DatabaseOptions {
    ConnectionString = "Data Source=productos.db"
};

builder.Services.AddSingleton(databaseOptions);
builder.Services.AddScoped<ProductoRepositorio>();

var app = builder.Build();

app.MapGet("/productos", (ProductoRepositorio repositorio) => {
    IEnumerable<Producto> productos = repositorio.ObtenerTodos();
    return Results.Ok(productos);
});
```

En este endpoint:

csharp

```
(ProductoRepositorio repositorio) =>
```

repositorio no viene de la ruta, ni de la query string, ni del body.

Viene del contenedor de servicios porque antes fue registrado con:

csharp

```
builder.Services.AddScoped<ProductoRepositorio>();
```

La configuración también se puede registrar como servicio.

csharp

```
DatabaseOptions databaseOptions = new DatabaseOptions {
    ConnectionString = "Data Source=productos.db"
};

builder.Services.AddSingleton(databaseOptions);
```

Y otro servicio puede recibir esa configuración por constructor:


```
csharp
public class ProductoRepositorio {
    private readonly string connectionString;

    public ProductoRepositorio(DatabaseOptions options) {
        connectionString = options.ConnectionString;
    }
}

public class DatabaseOptions {
    public string ConnectionString { get; set; } = "";
}
```

También se pueden mezclar parámetros que vienen de lugares distintos:

```
csharp
app.MapGet("/productos/{id:int}", (int id, ProductoRepositorio repositorio) => {
    Producto? producto = repositorio.ObtenerPorId(id);

    if (producto is null) {
        return Results.NotFound();
    }

    return Results.Ok(producto);
});
```

En ese caso:

```
texto
int id                → viene de la ruta /productos/{id:int}
ProductoRepositorio repositorio → viene de los servicios registrados
```

Regla práctica:

```
texto
Parámetros simples    → ruta, query string o headers.
Objetos del body      → JSON enviado por el cliente.
Servicios registrados → contenedor de servicios de ASP.NET Core.
```

7. CRUD en memoria

Antes de usar base de datos, armamos una API en memoria. Esto permite entender HTTP, REST, DTOs y status codes sin mezclar todavía SQL.

```
csharp
#::sdk Microsoft.NET.Sdk.Web

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

List<Producto> productos = [
    new Producto(1, "Teclado", 15000),
    new Producto(2, "Mouse", 10000)
];

app.MapGet("/productos", () => {
    return Results.Ok(productos);
});
```

csharp - continuacion

```
app.MapGet("/productos/{id:int}", (int id) => {
    Producto? producto = productos.FirstOrDefault(p => p.Id == id);

    if (producto is null) { return Results.NotFound(); }

    return Results.Ok(producto);
});

app.MapPost("/productos", (CrearProductoDto dto) => {
    if (string.IsNullOrEmpty(dto.Nombre)) { return Results.BadRequest("El nombre es obligatorio."); }
    if (dto.Precio <= 0) { return Results.BadRequest("El precio debe ser mayor que cero."); }

    int nuevoId = productos.Count == 0 ? 1 : productos.Max(p => p.Id) + 1;

    Producto producto = new Producto(nuevoId, dto.Nombre, dto.Precio);
    productos.Add(producto);

    return Results.Created($" /productos/{producto.Id}", producto);
});

app.MapPut("/productos/{id:int}", (int id, ActualizarProductoDto dto) => {
    if (string.IsNullOrEmpty(dto.Nombre)) { return Results.BadRequest("El nombre es obligatorio."); }
    if (dto.Precio <= 0) { return Results.BadRequest("El precio debe ser mayor que cero."); }

    int indice = productos.FindIndex(p => p.Id == id);

    if (indice == -1) {
        return Results.NotFound();
    }

    Producto productoActualizado = new Producto(id, dto.Nombre, dto.Precio);
    productos[indice] = productoActualizado;

    return Results.Ok(productoActualizado);
});

app.MapDelete("/productos/{id:int}", (int id) => {
    Producto? producto = productos.FirstOrDefault(p => p.Id == id);

    if (producto is null) {
        return Results.NotFound();
    }

    productos.Remove(producto);

    return Results.NoContent();
});

app.Run();

public record Producto(int Id, string Nombre, decimal Precio);
public record CrearProductoDto(string Nombre, decimal Precio);
public record ActualizarProductoDto(string Nombre, decimal Precio);
```

Este ejemplo ya tiene la estructura de una API real:

texto

```
GET colección
GET por id
POST creación
PUT actualización
DELETE eliminación
DTOS de entrada
status codes correctos
JSON automático
```

8. API completa con SQLite y repositorio

Crear app.cs:

```
csharp
#:sdk Microsoft.NET.Sdk.Web
#:package Microsoft.Data.Sqlite@*
#:package Dapper@*
#:package Dapper.Contrib@*

using Dapper;
using Dapper.Contrib.Extensions;
using Microsoft.Data.Sqlite;

var builder = WebApplication.CreateBuilder(args);

DatabaseOptions databaseOptions = new DatabaseOptions {
    ConnectionString = "Data Source=productos.db"
};

builder.Services.AddSingleton(databaseOptions);
builder.Services.AddScoped<ProductoRepositorio>();

var app = builder.Build();

ProductoRepositorio.InicializarBaseDeDatos(databaseOptions);

app.MapGet("/", () => "API de productos");

app.MapGet("/productos", (ProductoRepositorio repositorio) => {
    IEnumerable<Producto> productos = repositorio.ObtenerTodos();

    return Results.Ok(productos);
});

app.MapGet("/productos/{id:int}", (int id, ProductoRepositorio repositorio) => {
    Producto? producto = repositorio.ObtenerPorId(id);

    if (producto is null) {
        return Results.NotFound();
    }

    return Results.Ok(producto);
});

app.MapGet("/productos/buscar", (string texto, ProductoRepositorio repositorio) => {
    IEnumerable<Producto> productos = repositorio.Buscar(texto);

    return Results.Ok(productos);
});

app.MapPost("/productos", (CrearProductoDto dto, ProductoRepositorio repositorio) => {
    List<string> errores = ValidarProducto(dto.Nombre, dto.Precio);

    if (errores.Count > 0) {
        return Results.BadRequest(errores);
    }

    Producto producto = repositorio.Crear(dto.Nombre, dto.Precio);

    return Results.Created($"productos/{producto.Id}", producto);
});

app.MapPut("/productos/{id:int}", (int id, ActualizarProductoDto dto, ProductoRepositorio repositorio)
=> {
    List<string> errores = ValidarProducto(dto.Nombre, dto.Precio);

    if (errores.Count > 0) {
        return Results.BadRequest(errores);
    }

    Producto? producto = repositorio.Actualizar(id, dto.Nombre, dto.Precio);
});
```

csharp - continuacion

```

        if (producto is null) {
            return Results.NotFound();
        }

        return Results.Ok(producto);
    });

app.MapDelete("/productos/{id:int}", (int id, ProductoRepositorio repositorio) => {
    bool eliminado = repositorio.Eliminar(id);

    if (!eliminado) {
        return Results.NotFound();
    }

    return Results.NoContent();
});

app.Run();

static List<string> ValidarProducto(string nombre, decimal precio) {
    List<string> errores = [];

    if (string.IsNullOrWhiteSpace(nombre)) {
        errores.Add("El nombre es obligatorio.");
    }

    if (precio <= 0) {
        errores.Add("El precio debe ser mayor que cero.");
    }

    return errores;
}

public class ProductoRepositorio {
    private readonly string connectionString;

    public ProductoRepositorio(DatabaseOptions options) {
        connectionString = options.ConnectionString;
    }

    public static void InicializarBaseDeDatos(DatabaseOptions options) {
        using SqlConnection connection = new SqlConnection(options.ConnectionString);

        string sql = """
            CREATE TABLE IF NOT EXISTS Productos (
                Id INTEGER PRIMARY KEY AUTOINCREMENT,
                Nombre TEXT NOT NULL,
                Precio REAL NOT NULL
            );
            """;

        connection.Execute(sql);
    }

    public IEnumerable<Producto> ObtenerTodos() {
        using SqlConnection connection = new SqlConnection(connectionString);

        return connection.GetAll<Producto>();
    }

    public Producto? ObtenerPorId(int id) {
        using SqlConnection connection = new SqlConnection(connectionString);

        return connection.Get<Producto>(id);
    }

    public IEnumerable<Producto> Buscar(string texto) {
        using SqlConnection connection = new SqlConnection(connectionString);

        string sql = """
            SELECT Id, Nombre, Precio

```

csharp - continuacion

```

        FROM Productos
        WHERE Nombre LIKE @Patron
        ORDER BY Nombre;
        """;

        return connection.Query<Producto>(
            sql,
            new { Patron = $"%{texto}%" }
        );
    }

    public Producto Crear(string nombre, decimal precio) {
        using SqlConnection connection = new SqlConnection(connectionString);

        Producto producto = new Producto {
            Nombre = nombre,
            Precio = precio
        };

        long id = connection.Insert(producto);
        producto.Id = (int)id;

        return producto;
    }

    public Producto? Actualizar(int id, string nombre, decimal precio) {
        using SqlConnection connection = new SqlConnection(connectionString);

        Producto? producto = connection.Get<Producto>(id);

        if (producto is null) {
            return null;
        }

        producto.Nombre = nombre;
        producto.Precio = precio;

        connection.Update(producto);

        return producto;
    }

    public bool Eliminar(int id) {
        using SqlConnection connection = new SqlConnection(connectionString);

        Producto? producto = connection.Get<Producto>(id);

        if (producto is null) {
            return false;
        }

        connection.Delete(producto);

        return true;
    }
}

[Table("Productos")]
public class Producto {
    [Key]
    public int Id { get; set; }

    public string Nombre { get; set; } = "";

    public decimal Precio { get; set; }
}

public record CrearProductoDto(string Nombre, decimal Precio);

public record ActualizarProductoDto(string Nombre, decimal Precio);

public class DatabaseOptions {

```

csharp - continuacion

```
public string ConnectionString { get; set; } = "";
```

Roles de cada parte:

Herramienta	Para qué sirve
SQLite	Base de datos liviana en un archivo local.
ADO.NET	Infraestructura base de conexión y comandos SQL.
Dapper	Ejecutar SQL y mapear resultados a objetos.
Dapper.Contrib	CRUD simple con métodos como Get, Insert, Update, Delete.
Repositorio	Concentrar el acceso a datos fuera de los endpoints.
DatabaseOptions	Guardar la configuración de conexión registrada como servicio.

Los endpoints reciben HTTP, validan datos y devuelven status codes.

El repositorio habla con SQLite.

texto

```
Endpoint      → recibe HTTP, valida datos, devuelve status codes.
Repositorio   → habla con SQLite.
Configuración → guarda el connection string.
```

No conviene compartir una única `SqlConnection` para toda la aplicación. Una API puede recibir varias requests al mismo tiempo. Si todas comparten la misma conexión, aparecen problemas de estado, concurrencia y errores difíciles de seguir.

Por eso cada operación del repositorio crea su propia conexión:

csharp

```
using SqlConnection connection = new SqlConnection(connectionString);
```

La conexión se usa durante esa operación y después se cierra automáticamente al salir del using.

Ejecutar:

bash

```
dotnet run app.cs
```

Endpoints disponibles:

Método	Ruta	Acción
GET	/	Verificar que la API funciona.
GET	/productos	Listar productos.
GET	/productos/{id}	Obtener un producto por id.

Método	Ruta	Acción
GET	/productos/buscar?texto=...	Buscar productos por nombre.
POST	/productos	Crear producto.
PUT	/productos/{id}	Actualizar producto completo.
DELETE	/productos/{id}	Eliminar producto.

Regla práctica:

texto

No compartir una misma `SqlConnection` entre requests.
Sí compartir configuración, fábricas o repositorios.
Crear y cerrar la conexión cerca de donde se usa.

9. Probar la API

Con la API ejecutándose, se puede probar desde otra terminal.

Listar

bash

```
curl http://localhost:5000/productos
```

Crear

bash

```
curl -X POST http://localhost:5000/productos \
-H "Content-Type: application/json" \
-d '{"nombre":"Monitor","precio":250000}'
```

Obtener por id

bash

```
curl http://localhost:5000/productos/1
```

Buscar

bash

```
curl "http://localhost:5000/productos/buscar?texto=mon"
```

Actualizar

```
bash
curl -X PUT http://localhost:5000/productos/1 \
-H "Content-Type: application/json" \
-d '{"nombre":"Monitor 24","precio":280000}'
```

Eliminar

```
bash
curl -X DELETE http://localhost:5000/productos/1
```

10. Leer un endpoint completo

Endpoint:

```
csharp
app.MapGet("/productos/{id:int}", (int id, ProductoRepositorio repositorio) => {
    Producto? producto = repositorio.ObtenerPorId(id);

    if (producto is null) {
        return Results.NotFound();
    }

    return Results.Ok(producto);
});
```

Lectura línea por línea:

```
csharp
app.MapGet("/productos/{id:int}", ...)
```

Define un endpoint para GET /productos/5.

```
csharp
(int id, ProductoRepositorio repositorio) =>
```

ASP.NET Core toma el valor de la ruta, lo convierte a entero y también inyecta un `ProductoRepositorio`.

```
csharp
Producto? producto = repositorio.ObtenerPorId(id);
```

El endpoint le pide el producto al repositorio. El endpoint no necesita saber cómo se abre la conexión ni qué SQL se ejecuta.

```
csharp
if (producto is null) {
    return Results.NotFound();
}
```


Si no existe, devuelve 404.

```
csharp
return Results.Ok(producto);
```

Si existe, devuelve 200 con JSON.

11. Consumir la API con HttpClient

Crear otro archivo: cliente.cs.

```
csharp
using System.Net.Http.Json;

HttpClient http = new HttpClient {
    BaseAddress = new Uri("http://localhost:5000")
};

Console.WriteLine("Listado inicial:");

List<Producto>? productos = await http.GetFromJsonAsync<List<Producto>>("/productos");

MostrarProductos(productos);

CrearProductoDto nuevoProducto = new CrearProductoDto("Auriculares", 35000);

HttpResponseMessage respuestaCreacion = await http.PostAsJsonAsync("/productos", nuevoProducto);

Console.WriteLine($"POST /productos -> {(int)respuestaCreacion.StatusCode} {respuestaCreacion.StatusCode}");

Producto? productoCreado = await respuestaCreacion.Content.ReadFromJsonAsync<Producto>();

Console.WriteLine($"Creado: {productoCreado?.Id} - {productoCreado?.Nombre}");

Console.WriteLine("Listado final:");

productos = await http.GetFromJsonAsync<List<Producto>>("/productos");

MostrarProductos(productos);

static void MostrarProductos(List<Producto>? productos) {
    if (productos is null) {
        Console.WriteLine("No se pudo leer la respuesta.");
        return;
    }

    foreach (Producto producto in productos) {
        Console.WriteLine($"{producto.Id}: {producto.Nombre} - ${producto.Precio}");
    }
}

public record Producto(int Id, string Nombre, decimal Precio);

public record CrearProductoDto(string Nombre, decimal Precio);
```

Ejecutar la API en una terminal:

```
bash
dotnet run app.cs
```

Ejecutar el cliente en otra terminal:

```
bash
dotnet run cliente.cs
```

Líneas importantes:

```
csharp
List<Producto>? productos = await http.GetFromJsonAsync<List<Producto>>("/productos");
```

hace un GET, lee JSON y lo convierte a objetos C#.

```
csharp
HttpResponseMessage respuestaCreacion = await http.PostAsJsonAsync("/productos", nuevoProducto);
```

hace un POST, serializa el objeto como JSON y lo manda en el body.

12. Manejo de errores desde el cliente

Un cliente no debería asumir que todo salió bien.

Ejemplo:

```
csharp
HttpResponseMessage response = await http.GetAsync("/productos/9999");

if (response.IsSuccessStatusCode) {
    Producto? producto = await response.Content.ReadFromJsonAsync<Producto>();
    Console.WriteLine(producto?.Nombre);
} else if (response.StatusCode == System.Net.HttpStatusCode.NotFound) {
    Console.WriteLine("El producto no existe.");
} else {
    string error = await response.Content.ReadAsStringAsync();
    Console.WriteLine($"Error: {(int)response.StatusCode} {error}");
}
```

Regla práctica:

```
texto
La API comunica con status codes.
El cliente decide qué hacer con cada status code.
```

13. Consumir la API desde HTML con fetch

ASP.NET Core puede servir archivos estáticos desde una carpeta wwwroot.

Crear:

```
texto
app.cs
wwwroot/
  index.html
```

Archivo `wwwroot/index.html`:

```
html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Productos</title>
</head>
<body>
  <h1>Productos</h1>

  <form id="formulario">
    <input id="nombre" placeholder="Nombre">
    <input id="precio" placeholder="Precio" type="number">
    <button>Guardar</button>
  </form>

  <ul id="lista"></ul>

  <script>
    const lista = document.getElementById("lista");
    const formulario = document.getElementById("formulario");
    const nombre = document.getElementById("nombre");
    const precio = document.getElementById("precio");

    async function cargarProductos() {
      const respuesta = await fetch("/productos");
      const productos = await respuesta.json();

      lista.innerHTML = "";

      for (const producto of productos) {
        const item = document.createElement("li");
        item.textContent = `${producto.nombre} - ${producto.precio}`;
        lista.appendChild(item);
      }
    }

    formulario.addEventListener("submit", async evento => {
      evento.preventDefault();

      await fetch("/productos", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          nombre: nombre.value,
          precio: Number(precio.value)
        })
      });

      nombre.value = "";
      precio.value = "";

      await cargarProductos();
    });

    cargarProductos();
  </script>
</body>
</html>
```

Para servir ese archivo desde ASP.NET Core, agregar antes de mapear endpoints:

csharp

```
app.UseDefaultFiles();  
app.UseStaticFiles();
```

Ejemplo:

csharp

```
var app = builder.Build();  
  
app.UseDefaultFiles();  
app.UseStaticFiles();  
  
app.MapGet("/productos", () => {  
    // ...  
});
```

Ahora el navegador puede abrir:

texto

```
http://localhost:5000
```

Como el HTML y la API salen del mismo backend, no hace falta CORS.

14. CORS

CORS aparece cuando el frontend y el backend están en orígenes distintos.

Ejemplo:

texto

```
Frontend: http://localhost:5500  
Backend:  http://localhost:5000
```

Para el navegador son orígenes distintos.

Configuración:

csharp

```
builder.Services.AddCors(options => {  
    options.AddPolicy("Frontend", policy => {  
        policy  
            .WithOrigins("http://localhost:5500")  
            .AllowAnyHeader()  
            .AllowAnyMethod();  
    });  
});
```

Activación:

csharp

```
app.UseCors("Frontend");
```

Ubicación típica:

```
csharp
var app = builder.Build();

app.UseCors("Frontend");

app.MapGet("/productos", () => {
    // ...
});
```

Para clase conviene servir `index.html` desde `wwwroot` usando el mismo backend. Así CORS no aparece hasta que realmente haga falta.

15. Middleware

Una request no entra directamente al endpoint. Pasa por una tubería.

```
texto
Request
↓
Middleware
↓
Middleware
↓
Endpoint
↓
Response
```

Cada middleware puede:

- leer la request;
- modificar la request;
- decidir si la ejecución sigue;
- ejecutar código antes del endpoint;
- ejecutar código después del endpoint;
- modificar la response.

Ejemplo simple para ver el flujo:

```
csharp
app.Use(async (context, next) => {
    Console.WriteLine($"Entró {context.Request.Method} {context.Request.Path}");

    await next();

    Console.WriteLine($"Salió {context.Response.StatusCode}");
});
```

Si llega:

```
http
GET /productos
```

la consola muestra algo parecido a:

```
texto
Entró GET /productos
Salió 200
```

El orden del pipeline importa. En general:

```
csharp
var app = builder.Build();

app.UseCors("Frontend");
app.UseDefaultFiles();
app.UseStaticFiles();

app.Use(async (context, next) => {
    Console.WriteLine($"{context.Request.Method} {context.Request.Path}");
    await next();
});

app.MapGet("/productos", () => {
    // ...
});
```

Configurar y usar middleware

Un middleware se configura agregándolo al pipeline con `app.Use( . . . )`.

La forma general es:

```
csharp
app.Use(async (context, next) => {
    // Código antes del endpoint.

    await next();

    // Código después del endpoint.
});
```

`context` es el `HttpContext`. Permite acceder a la request y a la response.

`next` representa el próximo paso de la tubería. Si llamamos a `await next()`, la request sigue avanzando. Si no lo llamamos, el middleware corta la ejecución y el endpoint no se ejecuta.

```
texto
Con await next() → la request sigue.
Sin await next() → la request se corta ahí.
```

Esto sirve para validaciones generales que se repiten en muchos endpoints.

Ejemplo: validar usuario con un header

Supongamos que la API espera recibir un usuario en el header `X-Usuario`.

Request válida:

```
http
GET /productos HTTP/1.1
Host: localhost:5000
X-Usuario: ana
```

Request inválida:

```
http
GET /productos HTTP/1.1
Host: localhost:5000
```

Podemos validar ese header con un middleware:

```
csharp
app.Use(async (context, next) => {
    if (!context.Request.Headers.TryGetValue("X-Usuario", out var usuario)) {
        context.Response.StatusCode = 401;
        await context.Response.WriteAsync("Falta el header X-Usuario.");
        return;
    }

    if (string.IsNullOrEmpty(usuario)) {
        context.Response.StatusCode = 401;
        await context.Response.WriteAsync("El usuario no puede estar vacío.");
        return;
    }

    await next();
});
```

Se lee así:

```
texto
Si no viene X-Usuario, responder 401 y cortar.
Si X-Usuario viene vacío, responder 401 y cortar.
Si X-Usuario es válido, continuar con el endpoint.
```

Código completo:

```
csharp
#sdk Microsoft.NET.Sdk.Web

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.Use(async (context, next) => {
    if (!context.Request.Headers.TryGetValue("X-Usuario", out var usuario)) {
        context.Response.StatusCode = 401;
        await context.Response.WriteAsync("Falta el header X-Usuario.");
        return;
    }

    if (string.IsNullOrEmpty(usuario)) {
        context.Response.StatusCode = 401;
        await context.Response.WriteAsync("El usuario no puede estar vacío.");
        return;
    }

    await next();
});

app.MapGet("/productos", () => {
    return Results.Ok(new[] {
```

csharp - continuacion

```
        "Teclado",  
        "Mouse",  
        "Monitor"  
    });  
};  
  
app.Run();
```

Si probamos sin header:

http

GET /productos

Respuesta:

texto

```
401 Unauthorized  
Falta el header X-Usuario.
```

Si probamos con header:

http

```
GET /productos  
X-Usuario: ana
```

Respuesta:

json

```
[  
  "Teclado",  
  "Mouse",  
  "Monitor"  
]
```

Usar el usuario dentro del endpoint

Además de validar, el middleware puede guardar datos en `context.Items`.

`context.Items` es una bolsa de datos que vive durante una sola request.

csharp

```
app.Use(async (context, next) => {  
    if (!context.Request.Headers.TryGetValue("X-Usuario", out var usuario)) {  
        context.Response.StatusCode = 401;  
        await context.Response.WriteAsync("Falta el header X-Usuario.");  
        return;  
    }  
  
    context.Items["Usuario"] = usuario.ToString();  
  
    await next();  
});
```

Luego el endpoint puede leer ese dato desde el contexto:


```
csharp
app.MapGet("/perfil", (HttpContext context) => {
    string usuario = context.Items["Usuario"]?.ToString() ?? "";
    return Results.Ok(new { Usuario = usuario });
});
```

Flujo:

```
texto
Request
↓
Middleware valida X-Usuario
↓
Middleware guarda Usuario en context.Items
↓
Endpoint lee context.Items["Usuario"]
↓
Response
```

Cuidar qué rutas protege el middleware

Un middleware se aplica a los endpoints que están después en el pipeline.

Ejemplo:

```
csharp
app.MapGet("/", () => "API funcionando");

app.Use(async (context, next) => {
    if (!context.Request.Headers.TryGetValue("X-Usuario", out var usuario)) {
        context.Response.StatusCode = 401;
        await context.Response.WriteAsync("Falta el header X-Usuario.");
        return;
    }

    context.Items["Usuario"] = usuario.ToString();

    await next();
});

app.MapGet("/productos", () => {
    return Results.Ok("Listado de productos");
});
```

En este ejemplo:

```
texto
/           → no queda protegido por este middleware.
/productos → sí queda protegido por este middleware.
```

En aplicaciones reales, la autenticación suele hacerse con middleware ya provisto por ASP.NET Core, como `UseAuthentication()` y `UseAuthorization()`. El ejemplo anterior sirve para entender la idea: un middleware puede inspeccionar la request, decidir si sigue y dejar datos preparados para los endpoints.

16. Logging con ILogger

ASP.NET Core trae infraestructura de logging.

Se puede pedir un logger en un endpoint:

```
csharp
app.MapPost("/productos", (CrearProductoDto dto, ILogger<Program> logger) => {
    logger.LogInformation("Intentando crear producto {Nombre} con precio {Precio}", dto.Nombre,
        dto.Precio);

    if (string.IsNullOrEmpty(dto.Nombre)) {
        logger.LogWarning("Intento de crear producto con nombre vacio");
        return Results.BadRequest("El nombre es obligatorio.");
    }

    if (dto.Precio <= 0) {
        logger.LogWarning("Intento de crear producto con precio inválido: {Precio}", dto.Precio);
        return Results.BadRequest("El precio debe ser mayor que cero.");
    }

    // insertar producto...

    logger.LogInformation("Producto creado correctamente");

    return Results.Ok();
});
```

Niveles comunes:

Nivel	Uso
Trace	Detalle extremo.
Debug	Información de depuración.
Information	Operación normal.
Warning	Algo raro pero controlado.
Error	Falló una operación.
Critical	Fallo grave.

Esta forma:

```
csharp
logger.LogInformation("Producto creado con id {Id}", producto.Id);
```

es preferible a:

```
csharp
logger.LogInformation($"Producto creado con id {producto.Id}");
```

porque conserva el mensaje estructurado.

17. Response compression

La compresión reduce el tamaño de respuestas HTTP.

Sirve para:

- JSON grandes;
- HTML;
- CSS;
- JavaScript.

Con `#:sdk Microsoft.NET.Sdk.Web`, el middleware ya está disponible. No hace falta agregar un paquete extra para este caso.

Configurar servicios:

```
csharp
builder.Services.AddResponseCompression(options => {
    options.EnableForHttps = true;
});
```

Activar middleware:

```
csharp
app.UseResponseCompression();
```

Ubicación típica:

```
csharp
var app = builder.Build();

app.UseResponseCompression();
app.UseDefaultFiles();
app.UseStaticFiles();

app.MapGet("/productos", () => {
    // ...
});
```

Cuando el cliente manda:

```
http
Accept-Encoding: gzip, br
```

el servidor puede responder comprimido.

18. Organización mínima del código

Al principio, un solo `app.cs` ayuda a ver todo junto.

Cuando el archivo crece, se puede separar:

```
texto
BackendProductos/
  app.cs
  Data/
    BaseDeDatos.cs
  Models/
    Producto.cs
  Dtos/
    CrearProductoDto.cs
    ActualizarProductoDto.cs
  Endpoints/
    ProductosEndpoints.cs
  wwwroot/
    index.html
```

Regla práctica:

```
texto
Primero entender.
Después organizar.
```

No conviene separar en muchas carpetas antes de entender qué hace cada parte.

Una separación mínima útil es mover endpoints a un método de extensión:

```
csharp
public static class ProductosEndpoints {
    public static void MapProductosEndpoints(this WebApplication app) {
        app.MapGet("/productos", () => {
            // ...
        });

        app.MapPost("/productos", () => {
            // ...
        });
    }
}
```

Y en `app.cs`:

```
csharp
app.MapProductosEndpoints();
```

19. Versión final con middleware útil

Esta versión integra:

- API REST;
- SQLite;
- Dapper;
- Dapper.Contrib;
- repositorio;

- validación;
- logging;
- archivos estáticos;
- compresión;
- middleware simple.

csharp

```
#:sdk Microsoft.NET.Sdk.Web
#:package Microsoft.Data.Sqlite@*
#:package Dapper@*
#:package Dapper.Contrib@*

using Dapper;
using Dapper.Contrib.Extensions;
using Microsoft.Data.Sqlite;

var builder = WebApplication.CreateBuilder(args);

DatabaseOptions databaseOptions = new DatabaseOptions {
    ConnectionString = "Data Source=productos.db"
};

builder.Services.AddResponseCompression(options => { options.EnableForHttps = true; });
builder.Services.AddSingleton(databaseOptions);
builder.Services.AddScoped<ProductoRepositorio>();

var app = builder.Build();

app.UseResponseCompression();
app.UseDefaultFiles();
app.UseStaticFiles();

app.Use(async (context, next) => {
    Console.WriteLine($"Entró {context.Request.Method} {context.Request.Path}");

    await next();

    Console.WriteLine($"Salió {context.Response.StatusCode}");
});

ProductoRepositorio.InicializarBaseDeDatos(databaseOptions);

app.MapGet("/", () => "API de productos");

app.MapGet("/productos", (ProductoRepositorio repositorio, ILogger<Program> logger) => {
    IEnumerable<Producto> productos = repositorio.ObtenerTodos();

    logger.LogInformation("Se consultó el listado de productos");

    return Results.Ok(productos);
});

app.MapGet("/productos/{id:int}", (int id, ProductoRepositorio repositorio, ILogger<Program> logger) => {
    Producto? producto = repositorio.ObtenerPorId(id);

    if (producto is null) {
        logger.LogWarning("Producto {Id} no encontrado", id);
        return Results.NotFound();
    }

    return Results.Ok(producto);
});

app.MapGet("/productos/buscar", (string texto, ProductoRepositorio repositorio) => {
    IEnumerable<Producto> productos = repositorio.Buscar(texto);

    return Results.Ok(productos);
});
```

csharp - continuacion

```
});

app.MapPost("/productos", (CrearProductoDto dto, ProductoRepositorio repositorio, ILogger<Program>
logger) => {
    List<string> errores = ValidarProducto(dto.Nombre, dto.Precio);

    if (errores.Count > 0) {
        logger.LogWarning("Producto inválido: {Errores}", string.Join(" | ", errores));
        return Results.BadRequest(errores);
    }

    Producto producto = repositorio.Crear(dto.Nombre, dto.Precio);

    logger.LogInformation("Producto creado: {Id} {Nombre}", producto.Id, producto.Nombre);

    return Results.Created($" /productos/{producto.Id}", producto);
});

app.MapPut("/productos/{id:int}", (int id, ActualizarProductoDto dto, ProductoRepositorio repositorio,
ILogger<Program> logger) => {
    List<string> errores = ValidarProducto(dto.Nombre, dto.Precio);

    if (errores.Count > 0) {
        return Results.BadRequest(errores);
    }

    Producto? producto = repositorio.Actualizar(id, dto.Nombre, dto.Precio);

    if (producto is null) {
        logger.LogWarning("No se pudo actualizar. Producto {Id} no encontrado", id);
        return Results.NotFound();
    }

    logger.LogInformation("Producto actualizado: {Id}", producto.Id);

    return Results.Ok(producto);
});

app.MapDelete("/productos/{id:int}", (int id, ProductoRepositorio repositorio, ILogger<Program> logger)
=> {
    bool eliminado = repositorio.Eliminar(id);

    if (!eliminado) {
        logger.LogWarning("No se pudo eliminar. Producto {Id} no encontrado", id);
        return Results.NotFound();
    }

    logger.LogInformation("Producto eliminado: {Id}", id);

    return Results.NoContent();
});

app.Run();

static List<string> ValidarProducto(string nombre, decimal precio) {
    List<string> errores = [];

    if (string.IsNullOrWhiteSpace(nombre)) {
        errores.Add("El nombre es obligatorio.");
    }

    if (precio <= 0) {
        errores.Add("El precio debe ser mayor que cero.");
    }

    return errores;
}

public class ProductoRepositorio {
    private readonly string connectionString;

    public ProductoRepositorio(DatabaseOptions options) {
```

csharp - continuacion

```
        connectionString = options.ConnectionString;
    }

    public static void InicializarBaseDeDatos(DatabaseOptions options) {
        using SQLiteConnection connection = new SQLiteConnection(options.ConnectionString);

        string sql = """
            CREATE TABLE IF NOT EXISTS Productos (
                Id INTEGER PRIMARY KEY AUTOINCREMENT,
                Nombre TEXT NOT NULL,
                Precio REAL NOT NULL
            );
            """;

        connection.Execute(sql);
    }

    public IEnumerable<Producto> ObtenerTodos() {
        using SQLiteConnection connection = new SQLiteConnection(connectionString);

        return connection.GetAll<Producto>();
    }

    public Producto? ObtenerPorId(int id) {
        using SQLiteConnection connection = new SQLiteConnection(connectionString);

        return connection.Get<Producto>(id);
    }

    public IEnumerable<Producto> Buscar(string texto) {
        using SQLiteConnection connection = new SQLiteConnection(connectionString);

        string sql = """
            SELECT Id, Nombre, Precio
            FROM Productos
            WHERE Nombre LIKE @Patron
            ORDER BY Nombre;
            """;

        return connection.Query<Producto>(
            sql,
            new { Patron = $"%{texto}%" }
        );
    }

    public Producto Crear(string nombre, decimal precio) {
        using SQLiteConnection connection = new SQLiteConnection(connectionString);

        Producto producto = new Producto {
            Nombre = nombre,
            Precio = precio
        };

        long id = connection.Insert(producto);
        producto.Id = (int)id;

        return producto;
    }

    public Producto? Actualizar(int id, string nombre, decimal precio) {
        using SQLiteConnection connection = new SQLiteConnection(connectionString);

        Producto? producto = connection.Get<Producto>(id);

        if (producto is null) {
            return null;
        }

        producto.Nombre = nombre;
        producto.Precio = precio;

        connection.Update(producto);
    }
}
```

csharp - continuacion

```
        return producto;
    }

    public bool Eliminar(int id) {
        using SqlConnection connection = new SqlConnection(connectionString);

        Producto? producto = connection.Get<Producto>(id);

        if (producto is null) {
            return false;
        }

        connection.Delete(producto);

        return true;
    }
}

[Table("Productos")]
public class Producto {
    [Key]
    public int Id { get; set; }
    public string Nombre { get; set; } = "";
    public decimal Precio { get; set; }
}

public record CrearProductoDto(string Nombre, decimal Precio);
public record ActualizarProductoDto(string Nombre, decimal Precio);

public class DatabaseOptions {
    public string ConnectionString { get; set; } = "";
}
```


20. Resumen

Necesidad	Herramienta
Exponer endpoints HTTP simples	Minimal APIs con <code>app.MapGet</code> , <code>app.MapPost</code> , <code>app.MapPut</code> , <code>app.MapDelete</code>
Identificar recursos	rutas como <code>/productos</code> y <code>/productos/{id:int}</code>
Recibir parámetros de ruta	parámetros del handler como <code>(int id)</code>
Recibir filtros u opciones	query string como <code>?texto=mouse&pagina=2</code>
Recibir metadatos del pedido	headers con <code>[FromHeader]</code> o <code>context.Request.Headers</code>
Acceder manualmente a toda la request	<code>HttpContext</code>
Enviar datos para crear o modificar	body JSON con DTOs
Devolver respuestas HTTP claras	<code>Results.Ok</code> , <code>Results.Created</code> , <code>Results.BadRequest</code> , <code>Results.NotFound</code> , <code>Results.NoContent</code>
Validar datos de entrada	funciones como <code>ValidarProducto</code>
Guardar datos en SQLite	<code>Microsoft.Data.Sqlite</code>
Ejecutar SQL simple	ADO.NET con <code>SqlConnection</code> y <code>SqlCommand</code>
Simplificar consultas SQL	Dapper con <code>Query</code> , <code>Execute</code> y parámetros
Simplificar CRUD básico	Dapper.Contrib con <code>GetAll</code> , <code>Get</code> , <code>Insert</code> , <code>Update</code> , <code>Delete</code>
Evitar inyección SQL	parámetros como <code>@Patron</code> , no concatenación de strings
Ordenar acceso a datos	repositorios o servicios inyectados
Configurar servicios	<code>builder.Services.Add...</code>
Agregar lógica común a todas las requests	middleware con <code>app.Use(...)</code>
Cortar una request inválida	no llamar a <code>await next()</code> y responder con status code
Validar usuario de forma simple	header <code>X-Usuario</code> leído desde middleware
Compartir datos dentro de una request	<code>context.Items</code>
Servir HTML, CSS y JS desde la API	<code>app.UseDefaultFiles()</code> y <code>app.UseStaticFiles()</code>
Permitir frontend/backend distintos orígenes	CORS con <code>builder.Services.AddCors</code> y <code>app.UseCors</code>
Registrar eventos de la API	<code>ILogger<T></code>
Comprimir respuestas	<code>builder.Services.AddResponseCompression</code> y <code>app.UseResponseCompression</code>
Consumir una API desde C#	<code>HttpClient</code> y <code>System.Net.Http.Json</code>
Consumir una API desde HTML	<code>fetch</code>

CAPITULO 36

Pruebas de APIs con cURL — Guía Práctica

cURL (Client URL) es una herramienta de línea de comandos utilizada para transferir datos a través de diversos protocolos de red, principalmente HTTP y HTTPS. Es la herramienta por excelencia para desarrolladores backend al momento de probar, depurar e inspeccionar endpoints de APIs web desde la terminal sin necesidad de usar herramientas visuales como Postman.

1. Sintaxis Básica y Parámetros Más Comunes

La estructura básica de un comando cURL es:

```
bash
curl [opciones] [URL]
```

Opciones (Flags) Indispensables:

Flag	Nombre Completo	Propósito	Ejemplo
-X	--request	Especifica el método HTTP (GET, POST, PUT, DELETE).	-X POST
-H	--header	Define cabeceras HTTP (Headers) para enviar metadatos o autenticación.	-H "Content-Type: application/json"
-d	--data	Envía datos en el cuerpo de la petición (Payload).	-d '{"id": 1}'
-i	--include	Incluye las cabeceras de la respuesta en la salida (útil para ver códigos de estado).	curl -i http://...
-I	--head	Realiza una petición HEAD (solo descarga las cabeceras de respuesta, no el cuerpo).	curl -I http://...
-v	--verbose	Modo detallado. Muestra todo el flujo (conexión TCP, TLS, cabeceras enviadas y recibidas).	curl -v http://...
-L	--location	Sigue redirecciones HTTP automáticamente (si la web redirige de HTTP a HTTPS).	curl -L http://...

2. Ejemplos de Peticiones HTTP Básicas

Utilizaremos la API pública de pruebas **JSONPlaceholder** para los siguientes ejemplos.

A. Petición GET (Obtener Datos)

Por defecto, si no se especifica el flag `-X`, cURL realiza una petición GET:

```
bash
# Petición GET simple
curl https://jsonplaceholder.typicode.com/posts/1

# Petición GET inspeccionando las cabeceras de respuesta (como Content-Type y Status Code)
curl -i https://jsonplaceholder.typicode.com/posts/1
```

B. Petición POST (Crear Datos con JSON)

Para enviar datos JSON, debemos configurar el método en POST, indicar la cabecera `Content-Type` y pasar la cadena JSON en `-d`:

```
bash
curl -X POST https://jsonplaceholder.typicode.com/posts \
-H "Content-Type: application/json" \
-d '{"title": "Nuevo Post", "body": "Contenido del post", "userId": 1}'
```

C. Petición PUT (Actualizar Datos)

El método PUT se utiliza para reemplazar completamente un recurso existente:

```
bash
curl -X PUT https://jsonplaceholder.typicode.com/posts/1 \
-H "Content-Type: application/json" \
-d '{"id": 1, "title": "Post Modificado", "body": "Cuerpo modificado", "userId": 1}'
```

D. Petición DELETE (Eliminar Datos)

Para borrar un recurso específico:

```
bash
curl -X DELETE https://jsonplaceholder.typicode.com/posts/1
```

3. Pruebas Avanzadas y Autenticación

Envío de Tokens de Autenticación (Bearer Token)

Muy común al proteger APIs con JWT:

```
bash
curl https://api.miservicio.com/usuarios \
-H "Authorization: Bearer TU_TOKEN_AQUI"
```

Descargar el Resultado a un Archivo

Si quieres guardar la respuesta de la API directamente en un archivo físico en lugar de imprimirlo en la terminal:

```
bash
# Guarda la respuesta en 'salida.json'
curl -o salida.json https://jsonplaceholder.typicode.com/posts
```

4. Ejemplo Práctico: Probando nuestro Servidor MCP HTTP

Podemos probar el servidor MCP minimalista que construimos en la clase anterior directamente usando cURL en nuestra máquina local.

Asegúrate de que tu servidor esté corriendo localmente (por ejemplo, en el puerto 5002) y ejecuta los siguientes comandos en tu terminal:

Paso 1: Enviar solicitud de Inicialización (Initialize)

```
bash
curl -i -X POST http://localhost:5002/mcp \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "protocolVersion": "2025-06-18",
    "capabilities": {},
    "clientInfo": { "name": "curl-client", "version": "1.0" }
  }
}'
```

Paso 2: Consultar la lista de Herramientas (List Tools)

```
bash
curl -X POST http://localhost:5002/mcp \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/list"
}'
```

Paso 3: Ejecutar la herramienta saludar (Call Tool)

bash

```
curl -X POST http://localhost:5002/mcp \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "id": 3,
  "method": "tools/call",
  "params": {
    "name": "saludar",
    "arguments": {
      "nombre": "Ada Lovelace"
    }
  }
}'
```

CAPITULO 37

Entity Framework Core con SQLite

Tutorial completo — agenda de contactos y profundización

C# 14 / .NET 10 / EF Core 10

¿Qué vamos a construir?

Una agenda de contactos de línea de comandos donde cada contacto puede tener:

- Un **nombre**.
- Una **categoría** (Familia, Trabajo, Amigos...).
- Varias **direcciones**, donde una dirección puede ser un teléfono o un email.

Nos enfocamos a propósito en consola: sin formularios, sin UI, sin frameworks web. Queremos que toda la atención vaya al **modelado de datos y al acceso a la base**. Una vez que entendés EF Core en consola, usarlo en MAUI, ASP.NET o lo que sea es *exactamente el mismo código* — cambia lo de alrededor.

El objetivo de esta clase es entender **qué es un ORM y cómo trabaja con relaciones**. Para eso elegimos un modelo con tres tablas relacionadas: es lo suficientemente simple para entenderlo de un vistazo, y lo suficientemente rico para ver aparecer las relaciones 1:N y N:1, las foreign keys, la carga de datos relacionados, y el borrado en cascada.

Lo que queremos entender al final:

- Qué es un ORM y cuándo conviene usarlo.
- Cómo se traduce una clase C# en una tabla SQL.
- Cómo se traducen las relaciones entre clases en foreign keys.
- Qué es un DbContext y por qué su ciclo de vida importa.
- Cómo funciona el *change tracking*.
- Cómo se cargan datos relacionados con Include.
- Dónde está el límite entre "dejá que EF Core decida" y "necesito pensar el SQL".

Una aclaración sobre migraciones. En las partes prácticas del proyecto (partes 1 a 8) NO vamos a usar migraciones — vamos a dejar que EF Core cree la base automáticamente con `EnsureCreated()`. Las migraciones son una pieza fundamental cuando un proyecto evoluciona, pero introducirlas hoy en el contexto del ejercicio agrega ruido que nos aleja del objetivo central. Las cubrimos en profundidad en la **Parte 12** de este mismo documento.

Parte 1 — Primeros principios

Antes de tocar código, necesitás tener claros algunos conceptos. Si te los saltás, vas a terminar copiando código sin entender qué hace.

1.1 — El problema del impedance mismatch

Tu programa piensa en **objetos**: un Contacto con propiedades, referencias a otros objetos, listas, herencia. Tu base de datos piensa en **tablas**: filas y columnas, claves primarias, foreign keys, sin polimorfismo.

Los dos modelos no coinciden. Este desajuste tiene un nombre clásico: *object-relational impedance mismatch*. Sin un ORM, el código de acceso a datos termina siendo así:

```
csharp
// Sin ORM - SQL a mano, mapeo manual, propenso a errores
static Contacto? BuscarContacto(int id) {
    using var cmd = new SqlCommand(
        "SELECT Id, Nombre FROM Contactos WHERE Id = @id", conn);
    cmd.Parameters.AddWithValue("@id", id);
    using var reader = cmd.ExecuteReader();
    if (reader.Read()) {
        var contacto = new Contacto {
            Id = reader.GetInt32(0),
            Nombre = reader.GetString(1),
        };
        // ahora otra query para las direcciones...
        // y otra para la categoria...
        return contacto;
    }
    return null;
}
```

Multiplicá eso por cada tabla, cada operación, cada cambio de schema, cada join. Los cambios en la base exigen rastrear cada query afectado, y no hay forma de verificar en compilación que una consulta es válida.

Un **ORM** (Object-Relational Mapper) automatiza esa traducción. Vos trabajás con objetos; la librería traduce a SQL por vos, y sabe cómo seguir las referencias entre ellos.

1.2 — Qué es Entity Framework Core

EF Core es el ORM oficial de Microsoft para .NET. Hace cuatro cosas principales:

1. **Mapea clases a tablas.** Una clase Contacto se convierte en la tabla Contactos. Sus propiedades se vuelven columnas. EF Core infiere los tipos, las claves, los nombres.
2. **Traduce LINQ a SQL.** Escribís `db.Contactos.Where(c => c.Nombre.StartsWith("A")).ToList()` y EF Core genera el `SELECT ... WHERE Nombre LIKE 'A%'` por vos.
3. **Entiende relaciones.** Si Contacto tiene una lista de Direccion, EF Core sabe generar la foreign key, sabe hacer el JOIN cuando las pedís juntas, y sabe borrarlas en cascada cuando borrás el contacto.
4. **Rastrea cambios.** Cargás un contacto, le modificás el nombre, llamás `SaveChanges()`, y EF Core genera el UPDATE correcto — solo para las columnas que cambiaron.

EF Core 10 salió en noviembre 2025 y es release LTS soportado hasta noviembre 2028. Corre sobre .NET 10.

1.3 — Por qué SQLite

SQLite es un motor de base de datos **embebido**: toda la base vive en un único archivo. No hay servidor, no hay conexiones de red, no hay cuentas de usuario.

Ventajas para nuestro caso:

- **Cero configuración.** No hay que instalar nada. El paquete NuGet ya trae el motor.
- **Portable.** El archivo `.db` lo podés copiar, mover, versionar, compartir.
- **Multiplataforma.** Windows, Linux, macOS, sin cambios.
- **Real.** No es una simulación — es un motor SQL completo, usado en producción por Android, iOS, Firefox, y básicamente toda la industria.

Desventajas (que no nos afectan en este ejercicio):

- Mala concurrencia de escritura: si varios procesos escriben a la vez, se traban.
- No soporta stored procedures ni triggers avanzados.
- No escala horizontalmente.

Para una app de escritorio o una agenda personal, estas desventajas no existen. Para una app web con 10000 usuarios escribiendo al mismo tiempo, cambiás a PostgreSQL o SQL Server **sin tocar el código de EF Core** — solo el provider. Ese es parte del valor del ORM.

1.4 — Code First

Hay dos formas de empezar un proyecto con EF Core:

- **Database First:** ya existe una base de datos y vos generás clases C# a partir de ella. Típico cuando heredás un sistema legacy.
- **Code First:** escribís las clases en C# y EF Core genera la base. Es el enfoque moderno por defecto.

Nosotros usamos **Code First**. Nuestras clases C# son la fuente de verdad; la base se deriva de ellas. Los cambios al modelo se pueden expresar como **migraciones** que EF Core convierte en `ALTER TABLE`, `CREATE TABLE`, etc. — pero eso lo veremos en la Parte 12.

1.5 — Las relaciones, traducidas

Esta es la parte central de la clase. Pensemos cómo se traducen las relaciones entre objetos a tablas SQL.

Relación N:1 (muchos contactos pertenecen a una categoría):

En objetos:

```
csharp
class Contacto { Categoria Categoria; }
class Categoria { string Nombre; }
```


En tablas:

```
texto
Contactos(Id, Nombre, CategoriaId → Categorías.Id)
Categorías(Id, Nombre)
```

La clase Contacto guarda una *referencia* a Categoria. La tabla Contactos guarda una *foreign key* a la fila de Categorías.

Relación 1:N (un contacto tiene muchas direcciones):

En objetos:

```
csharp
class Contacto { List<Direccion> Direcciones; }
class Direccion { Contacto Contacto; }
```

En tablas:

```
texto
Contactos(Id, Nombre)
Direcciones(Id, Valor, ContactoId → Contactos.Id)
```

La lista vive del lado de Contacto en el modelo de objetos. En el modelo relacional, la foreign key vive del lado "muchos" — es decir, cada Direccion recuerda a qué Contacto pertenece. Esa es la forma natural de expresarlo en SQL: en una base relacional no hay "listas", solo relaciones.

Relación N:N (un libro tiene muchos géneros y un género aplica a muchos libros):

En objetos:

```
csharp
class Libro { List<Genero> Generos; }
class Genero { List<Libro> Libros; }
```

En tablas SQL se necesita una **tabla de unión**:

```
texto
Libros(Id, Titulo)
Generos(Id, Nombre)
LibroGeneros(LibroId → Libros.Id, GeneroId → Generos.Id)
```

EF Core puede crear la tabla de unión automáticamente, o podés definirla explícitamente si necesitás guardar datos extras en la relación (como fecha de publicación o rol del autor).

Relación 1:1 (un libro tiene un único detalle editorial):

En tablas, la FK vive en la tabla del lado "dependiente":

```
texto
Libros(Id, Titulo)
DetallesEditoriales(Id, ISBN, LibroId → Libros.Id)
```

Cubrimos los tres tipos con ejemplos completos en la **Parte 11**.

1.6 — El DbContext: puerta de entrada a todo

Todo lo que hagas con EF Core pasa por un objeto `DbContext`. Es la "unidad de trabajo" (*unit of work*) — representa una sesión con la base. Sus responsabilidades son:

- Abre y cierra la conexión a la base.
- Contiene las `DbSet<T>`, que son las colecciones consultables (una por cada tabla).
- Rastrea los cambios que hacés sobre las entidades cargadas.
- Traduce operaciones LINQ a SQL.
- Gestiona transacciones.
- Ejecuta las operaciones pendientes cuando llamás `SaveChanges()`.

Regla importantísima: **el DbContext no es thread-safe y es barato de crear**. Creás uno, lo usás para una unidad de trabajo (una operación del menú), lo desechás. No lo compartas entre hilos. No lo hagas `static`. No lo tengas vivo horas.

Usaremos `using` para que se desechen solos.

Parte 2 — Preparación del proyecto

2.1 — Requisitos

- .NET 10 SDK instalado (`dotnet --version` debería mostrar 10.x).
- Un editor: VS Code, Visual Studio 2026 o Rider.

2.2 — Crear el proyecto

```
bash
dotnet new console -n AgendaEF -f net10.0
cd AgendaEF
```

2.3 — Instalar los paquetes NuGet

Dos paquetes, cada uno con su razón de ser:

```
bash
dotnet add package Microsoft.EntityFrameworkCore --version 10.*
dotnet add package Microsoft.EntityFrameworkCore.Sqlite --version 10.*
```

- `Microsoft.EntityFrameworkCore`: el core del ORM. Las clases `DbContext`, `DbSet`, el change tracker, el query pipeline.

- `Microsoft.EntityFrameworkCore.Sqlite`: el *provider* de SQLite. Sin esto EF Core sabe hablar LINQ pero no sabe a qué base traducir.

2.4 — Alternativa: configuración file-based (.NET 10)

Nota — programas basados en archivo. .NET 10 introduce los *file-based programs*: podés escribir y ejecutar un programa C# **sin crear un proyecto ni un** `.csproj`, declarando las dependencias directamente dentro del archivo `.cs` con directivas `#:`.

En lugar del flujo `dotnet new` → `dotnet add package`, el archivo declara sus propias dependencias en el encabezado:

```
csharp
#:package Microsoft.EntityFrameworkCore@10.*
#:package Microsoft.EntityFrameworkCore.Sqlite@10.*

// todo el código del programa a continuación...
using Microsoft.EntityFrameworkCore;

var db = new AgendaRapidaContext();
db.Database.EnsureCreated();

// ... resto del programa
```

Y se ejecuta directamente:

```
bash
dotnet run agenda.cs
```

El SDK descarga los paquetes, compila y corre — sin carpeta de proyecto, sin `bin/`, sin `obj/`.

¿Cuándo usar file-based y cuándo el proyecto tradicional?

Situación	Enfoque recomendado
Script de prueba rápida, exploración de una API, ejemplo para clase	File-based — menos fricción
App con múltiples archivos, capas (Models/, Data/, Services/)	Proyecto tradicional con <code>.csproj</code>
Necesitás migraciones (<code>dotnet ef migrations add</code>)	Proyecto tradicional — las tools de EF requieren <code>.csproj</code>
Querés distribuir o versionar el proyecto	Proyecto tradicional

Para **este tutorial** usamos el proyecto tradicional porque la arquitectura en capas (Models, Data, Services) y las migraciones de la Parte 12 requieren un `.csproj`. Pero si querés hacer una prueba rápida de una query o experimentar con el `DbContext` antes de estructurar el proyecto, el enfoque file-based es ideal.

Parte 3 — Arquitectura del proyecto

texto

```

AgendaEF/
  ■■■ AgendaEF.csproj
  ■■■ Program.cs           ← menú interactivo (UI de consola)
  ■■■ Models/
    ■ ■■■ Categoria.cs
    ■ ■■■ Contacto.cs
    ■ ■■■ Direccion.cs
    ■ ■■■ TipoDireccion.cs ← enum
  ■■■ Data/
    ■ ■■■ AgendaContext.cs ← el DbContext
  ■■■ Services/
    ■■■ ContactoService.cs ← la lógica CRUD
  
```

Por qué separamos en capas

Podríamos poner todo en `Program.cs`. Sería más corto. Pero quiero que practiquemos separación de responsabilidades desde el día uno:

Capa	Responsabilidad	Qué NO debe hacer
Models	Representar los datos del dominio	Saber de SQL, consola, o EF Core
Data	Configurar EF Core y el mapeo a la base	Tener lógica de negocio
Services	Operaciones CRUD y reglas de negocio	Leer de Console ni imprimir
Program	Menú y entrada/salida por consola	Hablar directo con la base

Esta separación no es un capricho académico. Es lo que te permite, mañana, reutilizar `ContactoService` desde una app MAUI o una Web API sin cambiar una línea. El CRUD no "sabe" que hoy lo llama una consola.

Decisiones de diseño explícitas

Decisión	Por qué
SQLite con archivo local <code>agenda.db</code>	Cero infra para el ejercicio, portable
<code>EnsureCreated()</code> en lugar de migraciones	Nos enfocamos en el ORM, no en la evolución del schema
Tabla única <code>Direcciones</code> con enum <code>Tipo</code>	Evita duplicar estructura "teléfono/email" en dos tablas casi idénticas
<code>DbContext</code> nuevo por operación	Respetar el ciclo de vida correcto, evita memory leaks
Id autoincremental en todas las tablas	Convención de EF Core, cero configuración
Operaciones síncronas	Simplicidad pedagógica; async se verá en la Parte 9
Service sin <code>Console.WriteLine</code>	La lógica no depende de la UI; es testeable

Parte 4 — Los modelos (el corazón del diseño)

Acá está la parte más importante del tutorial. **Cómo modelás las clases determina cómo se ven las tablas y cómo fluye todo el resto del código.**

4.1 — El enum TipoDireccion

Archivo: Models/TipoDireccion.cs

```
csharp
namespace AgendaEF.Models;

public enum TipoDireccion {
    Telefono = 1,
    Email    = 2,
}
```

Qué está pasando:

- Un enum es la forma más segura de representar un conjunto cerrado de valores. El compilador te obliga a usar solo los valores permitidos; no podés tipear "telefono" con minúscula y romper todo.
- Los valores numéricos (= 1, = 2) los pongo explícitos: así, si mañana alguien agrega un valor en el medio de la lista, los que ya existen no cambian su número. Esto importa porque **EF Core guarda el enum en la base como número por defecto**. Si los valores cambian, la base queda inconsistente.
- No uso 0. Es buena costumbre: el 0 es el valor por defecto de cualquier enum, y que represente un estado "sin asignar" ayuda a detectar bugs.

4.2 — La entidad Categoria

Archivo: Models/Categoria.cs

```
csharp
using System.ComponentModel.DataAnnotations;

namespace AgendaEF.Models;

public class Categoria {
    public int Id { get; set; }

    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;

    // Navegación inversa: una categoría puede tener muchos contactos
    public List<Contacto> Contactos { get; set; } = [];
}
```

Puntos clave:

- Id sin atributos. EF Core usa convención: una propiedad llamada Id se asume clave primaria. En SQLite, un int PK se auto-incrementa.
- [Required] en Nombre → se traduce a NOT NULL.
- [MaxLength(50)] documenta la intención y se respeta en providers estrictos.

- `List<Contacto> Contactos` es una **propiedad de navegación**. No es una columna en la tabla — es la forma que tiene el modelo de objetos de expresar "una categoría conoce a todos los contactos que la usan". EF Core la usa para hacer joins cuando pedís categoría + contactos juntos.
- `= []` inicializa la lista vacía. Evita `NullReferenceException` cuando accedés a una categoría recién creada antes de `SaveChanges()`.

4.3 — La entidad Contacto

Archivo: `Models/Contacto.cs`

```
csharp
using System.ComponentModel.DataAnnotations;

namespace AgendaEF.Models;

public class Contacto {
    public int Id { get; set; }

    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;

    public DateTime CreadoEn { get; set; } = DateTime.UtcNow;

    // Relación N:1 con Categoria
    public int CategoriaId { get; set; }
    public Categoria Categoria { get; set; } = null!;

    // Relación 1:N con Direccion
    public List<Direccion> Direcciones { get; set; } = [];
}
```

Este es el centro de gravedad del modelo. Miralo despacio.

La relación con Categoria:

Tenemos dos propiedades que representan lo mismo desde ángulos distintos:

- `CategoriaId` es la **foreign key** — el número que en la tabla `Contactos` apunta a una fila de `Categorias`. Es un dato crudo.
- `Categoria` es la **propiedad de navegación** — el objeto `Categoria` cargado en memoria. Lo usás cuando querés acceder a datos de la categoría desde el contacto (ej: `contacto.Categoria.Nombre`).

¿Por qué tener las dos? Porque resuelven problemas distintos:

- Si solo querés *asignar* una categoría (ej: "este contacto ahora es de Trabajo, cuyo id es 2"), con `CategoriaId = 2` alcanza. No necesitás traer el objeto `Categoria` de la base.
- Si querés *leer* datos de la categoría, usás la propiedad de navegación.

El patrón "FK + navegación" es una convención fuerte en EF Core. Aprendelo bien. El nombre `CategoriaId` es clave: EF Core por convención busca una propiedad con ese nombre para mapearla como la FK de la navegación `Categoria`.

`El = null!` en `Categoria`:

Es una promesa al compilador: "sé que esta propiedad parece que podría ser null, pero en la práctica EF Core la va a poblar cuando cargue el contacto, así que ignorá el warning". Es un truco común con propiedades de navegación no-opcionales.

La relación con Direcciones:

Del lado del Contacto, la relación 1:N se expresa como una `List<Direccion>`. Del lado de la Direccion vive la foreign key `ContactoId`. EF Core ve este par y entiende "ajá, uno a muchos".

4.4 — La entidad Direccion

Archivo: `Models/Direccion.cs`

```
csharp
using System.ComponentModel.DataAnnotations;

namespace AgendaEF.Models;

public class Direccion {
    public int Id { get; set; }

    [Required]
    public TipoDireccion Tipo { get; set; }

    [Required]
    [MaxLength(150)]
    public string Valor { get; set; } = string.Empty;

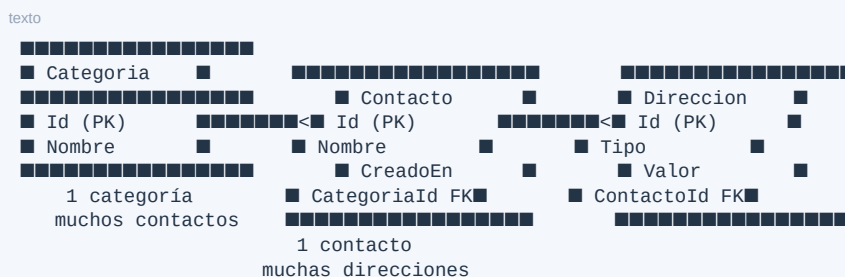
    // Relación N:1 con Contacto
    public int ContactoId { get; set; }
    public Contacto Contacto { get; set; } = null!;
}
```

Qué observar:

- Tipo es nuestro enum. EF Core lo guardará como INTEGER (los valores 1 o 2). Simple y eficiente.
- Valor es el string que contiene el número de teléfono o el email. Un solo campo para ambos casos. El tipo del contenido lo indica Tipo.
- Otra vez el par `ContactoId + Contacto`, pero esta vez del lado "muchos" de la relación. La FK vive acá porque en una base relacional, **la FK siempre vive en el lado "muchos"** de una relación 1:N.

4.5 — Diagrama mental

Pongamos todo junto:



Las flechas ■■■■■< indican "uno a muchos": el lado del < es el "muchos". Los tres Id son claves primarias autoincrementales. Las dos FK viven del lado "muchos" de cada relación.

Parte 5 — El DbContext

Archivo: Data/AgendaContext.cs

```
csharp
using AgendaEF.Models;
using Microsoft.EntityFrameworkCore;

namespace AgendaEF.Data;

public class AgendaContext : DbContext {
    public DbSet<Contacto> Contactos => Set<Contacto>();
    public DbSet<Categoria> Categorias => Set<Categoria>();
    public DbSet<Direccion> Direcciones => Set<Direccion>();

    protected override void OnConfiguring(DbContextOptionsBuilder options) {
        string path = Path.Combine(AppContext.BaseDirectory, "agenda.db");
        options.UseSqlite($"Data Source={path}");
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder) {
        // Índice para búsquedas rápidas por nombre de contacto
        modelBuilder.Entity<Contacto>()
            .HasIndex(c => c.Nombre);

        // Cada categoría tiene un nombre único
        modelBuilder.Entity<Categoria>()
            .HasIndex(c => c.Nombre)
            .IsUnique();
    }

    /// <summary>
    /// Crea la base si no existe, con las tablas según el modelo.
    /// Llamar una vez al arrancar la app.
    /// </summary>
    public static void InicializarBase() {
        using var db = new AgendaContext();
        db.Database.EnsureCreated();

        // Semilla: si no hay categorías, crear las típicas
        if (!db.Categorias.Any()) {
            db.Categorias.AddRange(
                new Categoria { Nombre = "Familia" },
                new Categoria { Nombre = "Trabajo" },
                new Categoria { Nombre = "Amigos" },
                new Categoria { Nombre = "Otros" }
            );
            db.SaveChanges();
        }
    }
}
```

Qué está pasando:

- Tres DbSet, uno por tabla. EF Core necesita verlos para saber qué clases son entidades.
- OnConfiguring configura la conexión. Para SQLite, la connection string es simplemente Data Source=ruta/al/archivo.
- OnModelCreating es donde se configura el modelo con la **Fluent API**. Pusimos dos índices: uno común sobre Contacto.Nombre (vamos a buscar por nombre seguido), y uno único sobre Categoria.Nombre

(no queremos dos categorías llamadas "Trabajo").

- `EnsureCreated()` mira el modelo actual y, si la base no existe, la crea con todas las tablas, índices y foreign keys. Si ya existe, no hace nada.
- **La semilla inicial** se ejecuta una sola vez gracias al `check if (!db.Categorias.Any())`.

*¿Por qué EnsureCreated() y no migraciones? EnsureCreated() es perfecto para el caso que nos ocupa: queremos que la base aparezca automáticamente para poder enfocarnos en el ORM. Pero tiene un límite grave: **no soporta cambios al modelo después de la primera ejecución**. Si mañana agregás una propiedad a Contacto, EnsureCreated() ve que la base existe y se queda tranquilo — aunque le falte la columna nueva. El programa después va a explotar. **Regla práctica para este ejercicio**: si cambiás el modelo, borra el archivo agenda.db y dejá que el programa lo cree de nuevo. El manejo correcto de cambios al schema — con **migraciones** — lo cubrimos en la Parte 12.*

Fluent API vs Data Annotations

Hay dos formas de configurar el modelo:

- **Data Annotations:** atributos en la entidad ([Required], [MaxLength]). Simples, viven junto a la propiedad. Mezclan responsabilidades: la clase de dominio sabe cosas de la base de datos.
- **Fluent API:** código en `OnModelCreating` usando `modelBuilder`. Más expresivo, puede hacer cosas que los atributos no pueden (índices únicos, relaciones complejas, configuración específica del provider). Mantiene las clases de dominio limpias.

Regla: lo básico va en atributos, lo avanzado en Fluent API. Coexisten sin problema; si hay conflicto, gana Fluent API.

Para proyectos más grandes hay una forma más escalable: delegar la configuración de cada entidad a su propia clase. Lo vemos en la **Parte 13**.

Parte 6 — El servicio CRUD

Archivo: Services/ContactoService.cs

Acá viven las operaciones de negocio. Una por cada operación del menú.

[illegible]

6.2 — AsNoTracking() en las lecturas

Cuando cargás una entidad con EF Core, por defecto el context la **trackea** — registra su estado para detectar cambios después. Si solo querés leer (no modificar), esto es desperdicio: consume memoria y CPU para nada.

.AsNoTracking() le dice a EF Core "esto es solo lectura, no te molestes en trackear". Resultado: consultas más rápidas, menos memoria. **Usalo siempre en lecturas puras.**

En los métodos Actualizar y Eliminar *no* usamos AsNoTracking, porque justamente necesitamos que EF Core trackee los cambios para generar el UPDATE / DELETE correcto.

6.3 — .Include(): la clave para cargar relaciones

Esta es la nueva estrella del tutorial:

```
csharp
db.Contactos
    .Include(c => c.Categoria)
    .Include(c => c.Direcciones)
```

Sin Include, si cargás un contacto y accedés a contacto.Categoria, te va a venir null. EF Core por defecto **no trae las relaciones** — las cargás explícitamente. Esto es *eager loading*: decís por adelantado qué relaciones querés.

El SQL que genera EF Core con estos Include es algo como:

```
sql
SELECT c.Id, c.Nombre, c.CategoriaId,
       cat.Id, cat.Nombre,
       d.Id, d.Tipo, d.Valor, d.ContactoId
FROM Contactos c
LEFT JOIN Categorias cat ON cat.Id = c.CategoriaId
LEFT JOIN Direcciones d ON d.ContactoId = c.Id
ORDER BY c.Nombre
```

EF Core después arma los objetos a partir de estas filas: agrupa las direcciones por contacto y las mete en la lista correspondiente.

¿Por qué Include y no cargar automáticamente todo? Porque traer relaciones es caro. EF Core te obliga a pedir explícitamente lo que querés: es un default conservador y correcto.

El problema N+1: si tuvieras *lazy loading* activado (que no activamos), cada acceso a c.Categoria dispararía una query nueva. 1000 contactos = 1001 queries. Include lo evita: una sola query. Es de los problemas de performance más comunes con ORMs.

6.4 — Find vs FirstOrDefault

- Find(id): busca por clave primaria. Primero revisa el change tracker (si la entidad ya está cargada en memoria, la devuelve sin ir a la base). Solo funciona con la PK. **No soporta Include.**
- FirstOrDefault(c => ...): busca con cualquier condición LINQ. Siempre va a la base. Acepta Include.

Por eso en `Eliminar` usamos `FirstOrDefault` con `Include`: necesitamos que EF Core cargue las direcciones junto al contacto para poder borrarlas en cascada. En `Actualizar`, como solo vamos a tocar propiedades del contacto, `Find` alcanza y es más rápido.

6.5 — Borrado en cascada

Cuando borramos un contacto, sus direcciones quedarían huérfanas. EF Core detecta automáticamente la relación 1:N y configura **cascade delete** por defecto — cuando borrás el contacto, sus direcciones se borran también.

Por eso cargamos el contacto con `Include(c => c.Direcciones)` antes de eliminarlo: para que EF Core las trackee y sepa que tiene que borrarlas. La categoría, en cambio, *no* se borra — está compartida por muchos contactos.

6.6 — Agregar direcciones en dos momentos distintos

Al crear el contacto (`Agregar`), pasamos la lista ya construida:

```
csharp
var contacto = new Contacto {
    Nombre = nombre,
    CategoriaId = categoriaId,
    Direcciones = direcciones, // ya tiene la lista adentro
};
db.Contactos.Add(contacto);
db.SaveChanges();
```

EF Core hace un INSERT al contacto, y después INSERTs a cada dirección usando el Id recién generado. Todo en una sola transacción.

Al agregar una dirección a un contacto existente (`AgregarDireccion`), pasamos el `ContactoId`:

```
csharp
var direccion = new Direccion {
    ContactoId = contactoId, // apuntamos al contacto existente
    Tipo = tipo,
    Valor = valor,
};
db.Direcciones.Add(direccion);
db.SaveChanges();
```

No necesitamos cargar el contacto entero. Con saber su id, creamos una nueva fila en `Direcciones` apuntando a él. Esta es la magia de separar FK y navegación.

Parte 7 — El menú principal

Archivo: `Program.cs`

El menú delega todo al service. No sabe nada de EF Core.

csharp - continuacion

```
var c = service.Agregar(nombre, categoriaId, direcciones);
Console.WriteLine($"Agregado con id {c.Id}.\n");
}

void Listar() {
    var contactos = service.ListarTodos();
    if (contactos.Count == 0) {
        Console.WriteLine("(Agenda vacía.)\n");
        return;
    }

    foreach (var c in contactos)
        MostrarResumen(c);
    Console.WriteLine();
}

void Buscar() {
    string termino = LeerObligatorio("Buscar nombre que contenga: ");
    var resultados = service.BuscarPorNombre(termino);

    if (resultados.Count == 0) {
        Console.WriteLine("(Sin resultados.)\n");
        return;
    }

    Console.WriteLine($"Encontrados {resultados.Count}:");
    foreach (var c in resultados)
        MostrarResumen(c);
    Console.WriteLine();
}

void VerDetalle() {
    int id = LeerId();
    var c = service.BuscarPorId(id);
    if (c is null) {
        Console.WriteLine("No existe ese contacto.\n");
        return;
    }

    Console.WriteLine($"[{c.Id}] {c.Nombre}");
    Console.WriteLine($"Categoría: {c.Categoria.Nombre}");
    Console.WriteLine($"Creado: {c.CreadoEn:yyyy-MM-dd HH:mm}");
    Console.WriteLine("Direcciones:");
    if (c.Direcciones.Count == 0) {
        Console.WriteLine(" (ninguna)");
    } else {
        foreach (var d in c.Direcciones)
            Console.WriteLine($" [{d.Id}] {d.Tipo}: {d.Valor}");
    }
    Console.WriteLine();
}

void Editar() {
    int id = LeerId();
    var actual = service.BuscarPorId(id);
    if (actual is null) {
        Console.WriteLine("No existe ese contacto.\n");
        return;
    }

    Console.WriteLine($"Editando: {actual.Nombre} ({actual.Categoria.Nombre})");
    string nombre = LeerOpcional($"Nombre [{actual.Nombre}]: ") ?? actual.Nombre;
    Console.WriteLine($"Categoría actual: {actual.Categoria.Nombre}");
    int categoriaId = ElegirCategoria();

    bool ok = service.Actualizar(id, nombre, categoriaId);
    Console.WriteLine(ok ? "Actualizado.\n" : "No se pudo actualizar.\n");
}

void AgregarDireccion() {
    int id = LeerId();
    var c = service.BuscarPorId(id);
```


csharp - continuación

```

    }
    Console.WriteLine("Inválido.");
}
}

static string LeerObligatorio(string prompt) {
    while (true) {
        Console.Write(prompt);
        string? v = Console.ReadLine()?.Trim();
        if (!string.IsNullOrEmpty(v))
            return v;
        Console.WriteLine("Requerido.");
    }
}

static string? LeerOpcional(string prompt) {
    Console.Write(prompt);
    string? v = Console.ReadLine()?.Trim();
    return string.IsNullOrEmpty(v) ? null : v;
}

static int LeerId() {
    while (true) {
        Console.Write("Id: ");
        if (int.TryParse(Console.ReadLine(), out int id))
            return id;
        Console.WriteLine("Número inválido.");
    }
}
}

```

Qué observar:

- **El menú no sabe nada de EF Core.** Solo habla con ContactoService. Si mañana cambiás de SQLite a PostgreSQL, o pasás de EF Core a Dapper, este archivo no se toca.
- try/catch **general** captura errores inesperados sin que el programa se caiga.
- **Confirmación antes de borrar.** Pequeña cortesía que evita desastres accidentales.

Parte 8 — Probarlo

bash

dotnet run**Flujo sugerido:**

texto

- 1) Agregar contacto
 - Nombre: Ada Lovelace
 - Categoría: 2 (Trabajo)
 - Tipo: 1 (Teléfono), Valor: 381-5551234
 - Tipo: 2 (Email), Valor: ada@utn.edu.ar
 - Tipo: 1 (Teléfono), Valor: 011-45551122
 - (vacío para terminar)
- 1) Agregar contacto
 - Nombre: Alan Turing
 - Categoría: 3 (Amigos)
 - Tipo: 2 (Email), Valor: turing@cripto.com
- 2) Listar → ves ambos contactos con su categoría y cantidad de direcciones

texto - continuación

- 4) Ver detalle id 1 → ves las tres direcciones de Ada completas
- 6) Agregar dirección al id 1 → agregás un email más a Ada
- 7) Eliminar dirección id 2 → borrás un teléfono de Ada
- 8) Eliminar contacto id 1 → confirma y borra Ada + todas sus direcciones
- 2) Listar → solo queda Alan

Salí con 9, volvé a entrar, listá de nuevo. Los contactos siguen ahí. El archivo agenda.db está al lado del ejecutable. **La persistencia funciona.**

Ver el SQL generado

Una de las críticas más frecuentes a los ORMs es que *ocultan el SQL que generan*. EF Core te permite verlo. Agregá esto al OnConfiguring del AgendaContext:

```
csharp
options.UseSqlite($"Data Source={path}")
    .LogTo(Console.WriteLine, Microsoft.Extensions.Logging.LogLevel.Information);
```

Cosas que descubrí mirando el SQL:

- El Include(c => c.Categoria) se traduce como LEFT JOIN Categorías.
- El Where se traduce como WHERE, no como filtrado en memoria.
- Los índices que definimos en OnModelCreating aparecen al crear la base.
- Podés detectar queries inesperadamente costosas o el problema N+1 antes de que lleguen a producción.

Este superpoder es la mejor forma de usar un ORM sin volverte dependiente de él.

Parte 9 — Consultas LINQ en profundidad

El servicio del ejercicio usa las operaciones más comunes. Acá expandimos el repertorio que vas a necesitar en proyectos reales.

9.1 — Ejecución diferida vs inmediata

EF Core **no ejecuta la consulta hasta que se itera**. Hasta entonces es un objeto IQueryable<T> que describe *qué* querés, no *cuándo* buscarlo.

```
csharp
// IQueryable – consulta construida, NO ejecutada todavía
IQueryable<Contacto> query = db.Contactos.Where(c => c.CategoriaId == 2);

// Podés seguir refinando sin ir a la base
query = query.OrderBy(c => c.Nombre);
query = query.Take(10);

// Acá se genera y ejecuta el SQL
List<Contacto> resultado = query.ToList();
// SELECT TOP 10 * FROM Contactos WHERE CategoriaId = 2 ORDER BY Nombre
```

Métodos que ejecutan inmediatamente: `ToList()`, `ToArray()`, `FirstOrDefault()`, `Count()`, `Any()`, `Sum()`, `foreach`.

Este comportamiento permite construir queries condicionalmente sin duplicar código:

```
csharp
IQueryable<Contacto> q = db.Contactos.AsNoTracking();

if (!string.IsNullOrEmpty(filtroNombre))
    q = q.Where(c => c.Nombre.Contains(filtroNombre));

if (filtroCategoria.HasValue)
    q = q.Where(c => c.CategoriaId == filtroCategoria);

return q.OrderBy(c => c.Nombre).ToList();
```

9.2 — Proyecciones con Select

En lugar de cargar la entidad completa, proyectás solo las columnas que necesitás. Más eficiente, menos datos transferidos.

```
csharp
// Sin proyección – trae todas las columnas (incluyendo las que no usás)
List<Contacto> todos = db.Contactos.ToList();

// Con proyección a tipo anónimo – solo las columnas necesarias
var resumen = db.Contactos
    .Select(c => new { c.Id, c.Nombre, c.CategoriaId })
    .ToList();

// Proyección a un DTO (record) – más tipado, más limpio
record ContactoResumen(int Id, string Nombre, string Categoria);

List<ContactoResumen> dtos = db.Contactos
    .AsNoTracking()
    .Include(c => c.Categoria)
    .Select(c => new ContactoResumen(c.Id, c.Nombre, c.Categoria.Nombre))
    .ToList();
```

Las proyecciones siempre van del lado del servidor (SQL), no en memoria. EF Core genera el SELECT con solo las columnas proyectadas.

9.3 — ThenInclude: relaciones anidadas

Cuando una relación incluida tiene a su vez relaciones propias, usás `ThenInclude`:

```
csharp
// En un sistema hipotético donde Categoria tiene una lista de Tags
var contactos = db.Contactos
    .Include(c => c.Categoria)
        .ThenInclude(cat => cat.Tags) // relaciones de la relación
    .Include(c => c.Direcciones)
    .ToList();
```

En nuestro modelo de agenda no hace falta porque las relaciones no tienen más de dos niveles, pero en modelos más ricos (pedidos → ítems → productos → categorías) `ThenInclude` es esencial.

9.4 — Consultas comunes

```
csharp
// Paginación – fundamental en cualquier listado
int pagina = 2;
int tamaño = 10;

var paginados = db.Contactos
    .AsNoTracking()
    .OrderBy(c => c.Nombre)
    .Skip((pagina - 1) * tamaño)
    .Take(tamaño)
    .ToList();

// Conteos y agregaciones
int total = db.Contactos.Count();
int soloFamilia = db.Contactos.Count(c => c.CategoriaId == 1);
bool hayAlguno = db.Contactos.Any();
bool existeAda = db.Contactos.Any(c => c.Nombre == "Ada Lovelace");

// Agrupación: cuántos contactos por categoría
var porCategoria = db.Contactos
    .AsNoTracking()
    .Include(c => c.Categoria)
    .GroupBy(c => c.Categoria.Nombre)
    .Select(g => new {
        Categoria = g.Key,
        Cantidad = g.Count(),
    })
    .OrderByDescending(x => x.Cantidad)
    .ToList();

// Búsqueda insensible a mayúsculas (SQLite es insensible por defecto con ASCII)
var resultados = db.Contactos
    .Where(c => EF.Functions.Like(c.Nombre, "%lovelace%"))
    .ToList();
```

9.5 — Consultas asíncronas

Todas las operaciones terminales tienen su variante `async`. En apps con UI (MAUI, ASP.NET), `async` es obligatorio para no bloquear el hilo principal.

```
csharp
// Equivalentes async de las operaciones básicas
List<Contacto> contactos = await db.Contactos.ToListAsync();
Contacto? uno = await db.Contactos.FirstOrDefaultAsync(c => c.Id == id);
int total = await db.Contactos.CountAsync();
bool existe = await db.Contactos.AnyAsync(c => c.Nombre == "Ada");

await db.SaveChangesAsync();
```

El uso en un servicio `async`:

```
csharp
public async Task<List<Contacto>> ListarTodosAsync() {
    using var db = new AgendaContext();

    return await db.Contactos
        .AsNoTracking()
        .Include(c => c.Categoria)
        .Include(c => c.Direcciones)
        .OrderBy(c => c.Nombre)
        .ToListAsync();
}
```

Parte 10 — Operaciones de escritura avanzadas

10.1 — ExecuteUpdate: actualización masiva sin cargar entidades

El patrón estándar (cargar entidad → modificar → SaveChanges) tiene un costo: trae filas de la base al servidor para modificarlas y enviarlas de vuelta. Para operaciones sobre muchas filas, hay una forma más eficiente:

```
csharp
// Forma estándar - carga cada contacto en memoria
var contactos = db.Contactos.Where(c => c.CategoriaId == 4).ToList();
foreach (var c in contactos)
    c.CategoriaId = 3;
db.SaveChanges();

// ExecuteUpdate - genera un solo UPDATE en la base, sin cargar nada
await db.Contactos
    .Where(c => c.CategoriaId == 4)
    .ExecuteUpdateAsync(setter =>
        setter.SetProperty(c => c.CategoriaId, 3));
// SQL: UPDATE Contactos SET CategoriaId = 3 WHERE CategoriaId = 4
```

Podés encadenar varios SetProperty para actualizar múltiples columnas a la vez.

10.2 — ExecuteDelete: borrado masivo

```
csharp
// Forma estándar - carga, marca, guarda
var viejas = db.Direcciones.Where(d => d.Tipo == TipoDireccion.Email).ToList();
db.Direcciones.RemoveRange(viejas);
db.SaveChanges();

// ExecuteDelete - un solo DELETE, sin cargar nada
await db.Direcciones
    .Where(d => d.Tipo == TipoDireccion.Email)
    .ExecuteDeleteAsync();
// SQL: DELETE FROM Direcciones WHERE Tipo = 2
```

Importante: ExecuteUpdate y ExecuteDelete **no usan el change tracker** — bypassan la unidad de trabajo. Se ejecutan inmediatamente, como SQL crudo. No esperan SaveChanges(). Úsalos cuando la operación es masiva y no necesitás los objetos en memoria.

10.3 — Transacciones explícitas

Por defecto, cada SaveChanges() es una transacción automática. Para operaciones que involucran múltiples SaveChanges() y necesitás que todo se confirme o revierta junto:

```
csharp
using var db = new AgendaContext();
using var tx = await db.Database.BeginTransactionAsync();

try {
    // Primera operación
    var categoria = new Categoria { Nombre = "VIP" };
    db.Categorias.Add(categoria);
    await db.SaveChangesAsync();

    // Segunda operación que depende de la primera
    var contacto = new Contacto {
```

csharp - continuación

```

        Nombre = "Ada Lovelace",
        CategoriaId = categoria.Id,
    };
    db.Contactos.Add(contacto);
    await db.SaveChangesAsync();

    await tx.CommitAsync(); // todo o nada
} catch {
    await tx.RollbackAsync();
    throw;
}

```

Parte 11 — Relaciones avanzadas

El modelo de la agenda solo usa 1:N y N:1. Los proyectos reales también necesitan N:N y 1:1.

11.1 — N:N simple (tabla de unión automática)

Un libro puede tener muchos géneros; un género puede aplicar a muchos libros.

csharp

```

public class Libro {
    public int Id { get; set; }
    public string Titulo { get; set; } = string.Empty;

    public List<Genero> Generos { get; set; } = [];
}

public class Genero {
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;

    public List<Libro> Libros { get; set; } = [];
}

```

Configuración en `OnModelCreating` (EF Core crea la tabla de unión automáticamente):

csharp

```

modelBuilder.Entity<Libro>()
    .HasMany(l => l.Generos)
    .WithMany(g => g.Libros)
    .UsingEntity(j => j.ToTable("libro_generos"));

```

Uso:

csharp

```

using var db = new LibreriaContext();

var novela = new Genero { Nombre = "Novela" };
var historica = new Genero { Nombre = "Histórica" };
db.Generos.AddRange(novela, historica);

var libro = new Libro {
    Titulo = "La guerra y la paz",
    Generos = [novela, historica],
};
db.Libros.Add(libro);
db.SaveChanges();

```

csharp - continuacion

```
// Leer con géneros incluidos
var libros = db.Libros
    .Include(l => l.Generos)
    .ToList();
```

11.2 — N:N con tabla de unión explícita

Cuando la relación tiene **atributos propios** (por ejemplo, un autor puede ser "principal" o "coautor" de un libro), definís la tabla de unión como entidad:

csharp

```
public class LibroAutor {
    public int LibroId { get; set; }
    public int AutorId { get; set; }
    public string Rol { get; set; } = "principal";
    public int Orden { get; set; } = 1;

    public Libro Libro { get; set; } = null!;
    public Autor Autor { get; set; } = null!;
}

public class Libro {
    public int Id { get; set; }
    public string Titulo { get; set; } = string.Empty;
    public List<LibroAutor> LibroAutores { get; set; } = [];
}

public class Autor {
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public List<LibroAutor> LibroAutores { get; set; } = [];
}
```

Configuración:

csharp

```
modelBuilder.Entity<LibroAutor>(entity => {
    entity.HasKey(la => new { la.LibroId, la.AutorId }); // PK compuesta

    entity.HasOne(la => la.Libro)
        .WithMany(l => l.LibroAutores)
        .HasForeignKey(la => la.LibroId);

    entity.HasOne(la => la.Autor)
        .WithMany(a => a.LibroAutores)
        .HasForeignKey(la => la.AutorId);
});
```

Consulta con `ThenInclude` para navegar por la tabla de unión:

csharp

```
var libros = db.Libros
    .Include(l => l.LibroAutores)
    .ThenInclude(la => la.Autor)
    .ToList();

foreach (var libro in libros) {
    var autores = libro.LibroAutores
        .OrderBy(la => la.Orden)
        .Select(la => $"{la.Autor.Nombre} ({la.Rol})");
    Console.WriteLine($"{libro.Titulo} - {string.Join(", ", autores)}");
}
```

```
csharp - continuacion
```

```
}
```

11.3 — Relación 1:1

Un libro tiene un único detalle editorial; ese detalle pertenece a un único libro.

```
csharp
```

```
public class DetalleEditorial {  
    public int Id { get; set; }  
    public string ISBN { get; set; } = string.Empty;  
    public string Editorial { get; set; } = string.Empty;  
    public int Paginas { get; set; }  
  
    public int LibroId { get; set; }  
    public Libro Libro { get; set; } = null!;  
}  
  
public class Libro {  
    public int Id { get; set; }  
    public string Titulo { get; set; } = string.Empty;  
  
    // Navegación hacia el detalle (puede ser null si no se cargó)  
    public DetalleEditorial? Detalle { get; set; }  
}
```

Configuración:

```
csharp
```

```
modelBuilder.Entity<Libro>()  
    .HasOne(l => l.Detalle)  
    .WithOne(d => d.Libro)  
    .HasForeignKey<DetalleEditorial>(d => d.LibroId);
```

La FK vive del lado del "dependiente" (DetalleEditorial), igual que en 1:N.

Parte 12 — Migraciones

EnsureCreated() crea la base una sola vez y no soporta cambios. En proyectos reales necesitás **migraciones**: clases generadas automáticamente que describen la diferencia entre el modelo C# actual y el schema de la base, y que se pueden aplicar de forma incremental y reversible.

12.1 — Paquetes adicionales

Para usar las herramientas de migraciones:

```
bash
```

```
dotnet add package Microsoft.EntityFrameworkCore.Tools --version 10.*  
dotnet add package Microsoft.EntityFrameworkCore.Design --version 10.*
```


12.2 — Flujo de trabajo

```
bash
# 1. Definir o modificar el modelo C#

# 2. Crear la migración
dotnet ef migrations add CrearAgenda

# 3. Revisar el archivo generado (siempre — antes de aplicar)
# Migrations/20251201120000_CrearAgenda.cs

# 4. Aplicar a la base
dotnet ef database update
```

Cada migración tiene dos métodos: `Up()` (aplica el cambio) y `Down()` (lo revierte). EF Core los genera automáticamente comparando el modelo actual con el snapshot anterior.

Comandos útiles:

```
bash
dotnet ef migrations list           # ver todas las migraciones
dotnet ef database update NombreMigracion # ir a una migración específica
dotnet ef migrations remove        # eliminar la última (si no se aplicó)
dotnet ef database update 0        # revertir todo
```

12.3 — Qué contiene una migración

```
csharp
// Migrations/20251201120000_CrearAgenda.cs — generado por EF Core
public partial class CrearAgenda : Migration {
    protected override void Up(MigrationBuilder migrationBuilder) {
        migrationBuilder.CreateTable(
            name: "Categorias",
            columns: table => new {
                Id = table.Column<int>(type: "INTEGER", nullable: false)
                    .Annotation("Sqlite:Autoincrement", true),
                Nombre = table.Column<string>(maxLength: 50, nullable: false),
            },
            constraints: table => {
                table.PrimaryKey("PK_Categorias", x => x.Id);
            });

        // ... más tablas ...
    }

    protected override void Down(MigrationBuilder migrationBuilder) {
        migrationBuilder.DropTable(name: "Contactos");
        migrationBuilder.DropTable(name: "Categorias");
        // ...
    }
}
```

Señales de advertencia que deben hacer pausar antes de aplicar:

- `DropTable` o `DropColumn` — se pierden datos permanentemente.
- `AddColumn NOT NULL` sin `defaultValue` sobre una tabla con datos existentes — la base puede rechazar la operación.
- `RenameColumn` donde esperabas `AddColumn` — EF Core a veces interpreta mal los renames.

12.4 — Datos semilla via HasData

Para datos que siempre deben existir (catálogos, configuraciones iniciales), en lugar de insertar a mano podés declarar semilla en la configuración del modelo:

```
csharp
// En la configuración de Categoria:
entity.HasData(
    new Categoria { Id = 1, Nombre = "Familia" },
    new Categoria { Id = 2, Nombre = "Trabajo" },
    new Categoria { Id = 3, Nombre = "Amigos" },
    new Categoria { Id = 4, Nombre = "Otros" }
);
```

Los IDs en HasData son **obligatorios y fijos**. EF Core genera la migración con los INSERT correspondientes y los aplica solo una vez.

12.5 — Aplicar migraciones al arrancar la app

En lugar de EnsureCreated(), una app con migraciones hace:

```
csharp
using var db = new AgendaContext();
await db.Database.MigrateAsync(); // aplica todas las migraciones pendientes
```

MigrateAsync() es seguro de llamar siempre: si no hay migraciones pendientes, no hace nada. Si la base no existe, la crea. Es el reemplazo correcto de EnsureCreated() en proyectos que evolucionan.

12.6 — Nombrar bien las migraciones

```
bash
# Descriptivo – cada nombre dice qué cambió
dotnet ef migrations add CrearTablaContactos
dotnet ef migrations add AgregarCampoEmailContacto
dotnet ef migrations add AgregarIndiceNombreContacto
dotnet ef migrations add SeedCategoriasIniciales

# Genérico – inútil para entender el historial
dotnet ef migrations add Migration1
dotnet ef migrations add Cambios
```

Parte 13 — Configuración escalable con IEntityTypeConfiguration

Para proyectos con muchas entidades, mezclar toda la configuración en OnModelCreating lo vuelve inmanejable. La solución es delegar la configuración de cada entidad a su propia clase:

```
csharp
// Data/Configurations/ContactoConfiguration.cs
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

namespace AgendaEF.Data.Configurations;
```

csharp - continuacion

```

public class ContactoConfiguration : IEntityTypeConfiguration<Contacto> {
    public void Configure(EntityTypeBuilder<Contacto> entity) {
        entity.HasKey(c => c.Id);

        entity.Property(c => c.Nombre)
            .IsRequired()
            .HasMaxLength(100);

        entity.HasIndex(c => c.Nombre);

        entity.HasOne(c => c.Categoria)
            .WithMany(cat => cat.Contactos)
            .HasForeignKey(c => c.CategoriaId)
            .OnDelete(DeleteBehavior.Restrict);

        entity.HasMany(c => c.Direcciones)
            .WithOne(d => d.Contacto)
            .HasForeignKey(d => d.ContactoId)
            .OnDelete(DeleteBehavior.Cascade);
    }
}

```

csharp

```

// Data/Configurations/CategoriaConfiguration.cs
public class CategoriaConfiguration : IEntityTypeConfiguration<Categoria> {
    public void Configure(EntityTypeBuilder<Categoria> entity) {
        entity.HasKey(c => c.Id);

        entity.Property(c => c.Nombre)
            .IsRequired()
            .HasMaxLength(50);

        entity.HasIndex(c => c.Nombre)
            .IsUnique();
    }
}

```

El contexto queda limpio y aplica todas las configuraciones automáticamente:

csharp

```

public class AgendaContext : DbContext {
    public DbSet<Contacto> Contactos => Set<Contacto>();
    public DbSet<Categoria> Categorias => Set<Categoria>();
    public DbSet<Direccion> Direcciones => Set<Direccion>();

    protected override void OnConfiguring(DbContextOptionsBuilder options) {
        string path = Path.Combine(AppContext.BaseDirectory, "agenda.db");
        options.UseSqlite($"Data Source={path}");
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder) {
        // Aplica automáticamente todas las IEntityTypeConfiguration del assembly
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(AgendaContext).Assembly);
    }
}

```

Esta estructura escala a proyectos con 20, 50, 100 entidades sin que el contexto crezca.

Parte 14 — Para apps con inyección de dependencias

En una app con DI (ASP.NET Core, MAUI con Microsoft.Extensions.DependencyInjection), el contexto se configura en el contenedor y se inyecta donde se necesita. No se instancia manualmente.

14.1 — Configurar el contexto con DI

csharp

```
// Program.cs (ASP.NET Core o app con DI)
builder.Services.AddDbContext<AgendaContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("Agenda")));
```

json

```
// appsettings.json
{
  "ConnectionStrings": {
    "Agenda": "Data Source=agenda.db"
  }
}
```

El contexto recibe las opciones por constructor:

csharp

```
public class AgendaContext : DbContext {
    public AgendaContext(DbContextOptions<AgendaContext> options) : base(options) { }

    public DbSet<Contacto> Contactos => Set<Contacto>();
    public DbSet<Categoria> Categorias => Set<Categoria>();
    public DbSet<Direccion> Direcciones => Set<Direccion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder) {
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(AgendaContext).Assembly);
    }
}
```

14.2 — El patrón Repositorio

Con DI, el servicio recibe el contexto por constructor en lugar de instanciarlo:

csharp

```
public class ContactoService {
    private readonly AgendaContext _db;

    public ContactoService(AgendaContext db) {
        this._db = db;
    }

    public async Task<List<Contacto>> ListarTodosAsync() =>
        await this._db.Contactos
            .AsNoTracking()
            .Include(c => c.Categoria)
            .Include(c => c.Direcciones)
            .OrderBy(c => c.Nombre)
            .ToListAsync();

    public async Task<Contacto?> BuscarPorIdAsync(int id) =>
        await this._db.Contactos
            .AsNoTracking()
            .Include(c => c.Categoria)
            .Include(c => c.Direcciones)
            .FirstOrDefaultAsync(c => c.Id == id);

    public async Task<Contacto> AgregarAsync(string nombre, int categoriaId, List<Direccion> direcciones) {
        bool categoriaExiste = await this._db.Categorias.AnyAsync(c => c.Id == categoriaId);
        if (!categoriaExiste)
            throw new ArgumentException($"La categoría {categoriaId} no existe.");

        var contacto = new Contacto {
            Nombre = nombre.Trim(),
```

csharp - continuacion

```

        CategoriaId = categoriaId,
        Direcciones = direcciones,
    };

    this._db.Contactos.Add(contacto);
    await this._db.SaveChangesAsync();

    return contacto;
}

```

Registro en el contenedor:

csharp

```
builder.Services.AddScoped<ContactoService>();
```

AddScoped significa que el contenedor crea una instancia por request (web) o por scope (consola). El ciclo de vida del DbContext lo maneja el contenedor automáticamente.

Parte 15 — Principios de diseño para llevar a cualquier proyecto

Cerramos con los principios transferibles. Estos aplican a cualquier app de acceso a datos que construyas en el futuro, independientemente del ORM.

15.1 — El DbContext tiene ciclo de vida corto

Un DbContext es como una conversación con la base. Abrís la conversación, hacés lo tuyo, cortás. Razones concretas:

- **Memory leaks:** el change tracker acumula entidades a medida que consultás. Un contexto largo crece indefinidamente.
- **Datos stale:** si otro proceso modifica la base, tu contexto largo sigue viendo la versión vieja.
- **Transacciones confusas:** un contexto largo con muchos saves puede dejar la base en estados inconsistentes si algo falla.

Regla operativa: **un DbContext por unidad de trabajo** (una operación del menú, una request web, una pantalla de MAUI).

csharp

```

// ✓ Correcto - contexto corto
async Task ActualizarNombre(int id, string nombre) {
    using var db = new AgendaContext();
    var c = await db.Contactos.FindAsync(id);
    if (c is null) return;
    c.Nombre = nombre;
    await db.SaveChangesAsync();
}

// ✗ Incorrecto - contexto de vida larga
public class Servicio {
    private readonly AgendaContext _db = new(); // vive toda la vida del servicio
}

```

```
csharp - continuacion  
}
```

15.2 — Separá lectura y escritura mentalmente

Las operaciones de lectura y escritura son fundamentalmente distintas.

- **Lecturas:** usá `AsNoTracking`. Son idempotentes. No requieren transacción.
- **Escrituras:** sin `AsNoTracking` (necesitás tracking para `SaveChanges`). No son idempotentes. Pueden requerir transacción.

Esta distinción es el germen del patrón **CQRS** (Command Query Responsibility Segregation). Para una agenda no hace falta llegar a CQRS, pero la mentalidad sí.

15.3 — FK + propiedad de navegación: el patrón cardinal

Cuando modelás una relación, casi siempre querés ambas:

```
csharp  
public int CategoriaId { get; set; } // la FK cruda - para asignar  
public Categoria Categoria { get; set; } // la navegación - para leer
```

- **Asignar es barato:** `CategoriaId = 2` y listo. No traés el objeto `Categoria` de la base.
- **Leer es cómodo:** `contacto.Categoria.Nombre` en lugar de queries manuales.
- **EF Core los relaciona automáticamente** por convención de nombre.

15.4 — Include explícito es la forma correcta

EF Core por defecto **no carga las relaciones automáticamente**. Vos decidís qué traer en cada query. Consecuencia: si accedés a `contacto.Categoria` sin `Include`, obtenés `null`. No es un bug del ORM; es que olvidaste pedirlo.

15.5 — Proyectar cuando solo necesitás parte de los datos

```
csharp  
// ✗ Trae todo para usar solo el nombre  
var lista = db.Contactos.ToList().Select(c => c.Nombre);  
  
// ✓ El SQL solo trae la columna necesaria  
var nombres = db.Contactos.Select(c => c.Nombre).ToList();
```

15.6 — Pensá el modelo relacional, no solo el de objetos

Antes de escribir la clase, dibujá el diagrama de tablas. Si el diagrama es raro, el modelo de objetos también lo va a ser.

Pregunta de chequeo: *¿dónde vive la foreign key?* La respuesta es siempre "en el lado muchos" de una relación 1:N, o "en una tabla intermedia" en N:N.

15.7 — Revisá siempre las migraciones antes de aplicar

Las migraciones son SQL que va a producción. `DropTable`, `DropColumn`, `AddColumn NOT NULL` sin default sobre datos existentes — son operaciones que destruyen datos si no las revisás con cuidado.

15.8 — Mirá el SQL generado

Nunca uses un ORM sin saber qué SQL está generando. Detectás el problema N+1. Te volvés mejor en SQL. Cuando el ORM genera algo subóptimo, podés saltarlo con `FromSqlRaw` o `ExecuteSqlRaw`. El ORM es una comodidad, no un reemplazo del conocimiento de bases de datos.

15.9 — Validación en capas

Hay tres lugares donde validar que un campo es requerido:

1. **En la UI** (antes de llamar al service): feedback inmediato.
2. **En el service** (regla de negocio): garantía aunque la UI falle.
3. **En la base** (`NOT NULL`): última línea de defensa.

Parece redundante, pero cada capa protege de un escenario distinto. *Defense in depth*.

Cierre

Lo que acabamos de cubrir va de un extremo al otro de EF Core: desde el proyecto más simple (agenda en consola con tres tablas) hasta las herramientas que usás en proyectos de producción (migraciones, `IEntityTypeConfiguration`, `ExecuteUpdate/ExecuteDelete`, transacciones, repositorios con DI).

El hilo conductor es siempre el mismo: **entender el por qué**. Por qué `AsNoTracking`, por qué `Include`, por qué contextos cortos, por qué tanto FK como navegación, por qué revisar el SQL. El cómo lo vas a recordar cinco minutos; el por qué te sirve para los próximos diez años.

Y lo más importante: los principios no son específicos de EF Core. El impedance mismatch, el unit of work, las relaciones 1:N con FK del lado muchos, el problema N+1, la separación lectura/escritura — todo esto vale con Dapper, con SQLAlchemy, con Hibernate, con cualquier ORM que uses en los próximos 20 años. Los frameworks pasan; los principios quedan.

CAPITULO 38

Autenticación desde cero: login, hash, token y middleware

Tutorial breve para **Programación III** — C# 14 / .NET 10 Acompaña al Sistema de Tickets con Minimal API y EF Core. Acá frenamos sobre las cuatro piezas de seguridad y las explicamos **desde los primeros principios**: qué problema resuelve cada una y por qué está hecha así. No hay código nuevo: es el mismo del sistema, mirado de cerca.

1. El problema de fondo

Una API en internet recibe pedidos de cualquiera. Antes de dejar que alguien lea o modifique tickets, hay que responder dos preguntas distintas:

- **¿Quién sos?** → eso es **autenticación**.
- **¿Podés hacer esto?** → eso es **autorización**.

Este tutorial es solo sobre la primera. Y resolverla bien implica encadenar cuatro problemas más chicos:

texto

- | | |
|---|-----------------------|
| 1. Guardar la contraseña sin guardarla | → hash + salt |
| 2. Comprobar la contraseña al entrar | → login |
| 3. No volver a pedirla en cada request | → token de sesión |
| 4. Reconocer al usuario en cada request | → middleware + filtro |

Cada pieza existe porque la anterior dejó un cabo suelto. Vamos en ese orden.

2. Por qué no se guarda la contraseña

El instinto es crear una columna Password y guardar ahí lo que el usuario escribió. **Es el error clásico de seguridad**. El día que alguien se filtra la base —y pasa todo el tiempo— se lleva las contraseñas de todos en texto plano. Y como la gente reusa contraseñas, ese robo se contagia a sus correos, bancos y todo lo demás.

La idea central es contraintuitiva pero simple:

No necesitamos guardar la contraseña. Necesitamos poder reconocerla cuando vuelva.

Para eso usamos una **función de hash**: agarra cualquier texto y devuelve una huella de tamaño fijo. Tiene dos propiedades que la hacen útil acá:

- Es **determinista**: la misma entrada da siempre la misma huella.

- Es **irreversible**: de la huella no se puede volver al texto original.

Entonces guardamos la huella, no la contraseña. Cuando el usuario vuelve a escribir su clave, le calculamos la huella otra vez y comparamos huella con huella. Si la base se filtra, el atacante se lleva huellas inservibles.

3. El hash con salt, paso a paso

Un hash pelado todavía tiene dos agujeros. El salt y el PBKDF2 los tapan. Mirá la clase y después desarmamos cada decisión:

```
csharp
using System.Security.Cryptography;

namespace TicketSystem.Security;

public static class PasswordHasher {
    private const int SaltSize = 16;           // 128 bits
    private const int HashSize = 32;           // 256 bits
    private const int Iterations = 100_000;

    public static (string Hash, string Salt) Hashear(string password) {
        byte[] salt = RandomNumberGenerator.GetBytes(SaltSize);
        byte[] hash = Pbkdf2(password, salt);

        return (Convert.ToBase64String(hash), Convert.ToBase64String(salt));
    }

    public static bool Verificar(string password, string hashGuardado, string saltGuardado) {
        byte[] salt = Convert.FromBase64String(saltGuardado);
        byte[] hash = Pbkdf2(password, salt);
        byte[] esperado = Convert.FromBase64String(hashGuardado);

        return CryptographicOperations.FixedTimeEquals(hash, esperado);
    }

    private static byte[] Pbkdf2(string password, byte[] salt) {
        return Rfc2898DeriveBytes.Pbkdf2(
            password,
            salt,
            Iterations,
            HashAlgorithmName.SHA256,
            HashSize);
    }
}
```

Agujero 1: dos personas con la misma clave dan el mismo hash

Si Ana y Beto usan ambos `clave1234`, un hash simple les da **la misma huella**. Un atacante que ve dos huellas iguales ya sabe que comparten contraseña. Peor: puede armar un diccionario gigante (huella → palabra), las famosas *rainbow tables*, y resolver millones de hashes de un saque.

La solución es el **salt**: un valor aleatorio distinto para cada usuario, que se mezcla con la contraseña antes de hashear.

```
texto
hash = Hash("clave1234" + salt_de_Ana)   → huella A
hash = Hash("clave1234" + salt_de_Beto)  → huella B   (¡distinta!)
```

El salt **no es secreto**: se guarda al lado del hash, en su propia columna. Su trabajo no es esconderse, sino hacer que cada hash sea único. Eso inutiliza las rainbow tables: el atacante tendría que armar una tabla nueva por cada salt. En el código, `RandomNumberGenerator.GetBytes(SaltSize)` genera esos 16 bytes aleatorios.

Agujero 2: hashear una vez es demasiado rápido

Las funciones de hash comunes están diseñadas para ser **veloces**. Para verificar una contraseña eso suena bien, pero le sirve igual al atacante: con la base filtrada, puede probar miles de millones de combinaciones por segundo contra cada hash.

La solución es **PBKDF2**: aplicar el hash muchas veces seguidas, encadenado. Acá, `Iterations = 100_000` significa cien mil vueltas.

Por qué eso ayuda: para el servidor, hashear una vez al hacer login es imperceptible (milisegundos). Para el atacante que prueba fuerza bruta, multiplicar cada intento por cien mil convierte un ataque de horas en uno de años. La lentitud es el rasgo deseado, no un defecto.

`Rfc2898DeriveBytes.Pbkdf2` viene en .NET, sin paquetes externos. Le pasamos la contraseña, el salt, las iteraciones, el algoritmo base (SHA256) y cuántos bytes de salida queremos (32 → 256 bits).

El detalle final: comparar en tiempo fijo

En `Verificar`, lo natural sería comparar las dos huellas con `==`. Pero un `==` sobre bytes **corta apenas encuentra la primera diferencia**. Eso abre un ataque sutil: midiendo cuánto tarda la comparación en fallar, un atacante puede deducir el hash byte por byte (un *timing attack*).

`CryptographicOperations.FixedTimeEquals` compara **siempre en el mismo tiempo**, recorra o no toda la secuencia. Le saca al atacante esa información de tiempo.

Resumen de la pieza: guardamos (hash, salt). El hash es irreversible, el salt lo hace único, PBKDF2 lo hace lento de atacar, y la comparación en tiempo fijo no filtra pistas. La contraseña original nunca toca el disco.

4. El login: verificar sin conocer

Ahora que sabemos guardar la huella, el login es directo. Esta es la parte de `AuthService`:

```
csharp
public async Task<string?> LoginAsync(string email, string password) {
    var usuario = await this.repositorio.ObtenerPorEmailAsync(email);
    if (usuario is null) {
        return null;
    }

    bool valido = PasswordHasher.Verificar(
        password, usuario.PasswordHash, usuario.PasswordSalt);

    if (!valido) {
        return null;
    }
}
```

csharp - continuacion

```
usuario.Token = GenerarToken();
usuario.TokenExpira = DateTime.Now.Add(DuracionSesion);
await this.repositorio.GuardarCambiosAsync();

return usuario.Token;
}
```

El flujo es: buscar al usuario por email, recalcular la huella de la contraseña que mandó y compararla con la guardada. Fijate que el servidor **nunca conoce la contraseña real**: solo confirma que la huella coincide.

Por qué return null y no un mensaje distinto: el método devuelve null tanto si el email no existe como si la contraseña está mal. Es a propósito. Si dijéramos "ese email no existe", le estaríamos confirmando a un atacante qué cuentas son válidas para después atacarlas. Una sola respuesta vaga para los dos casos no regala esa información.

Si todo da bien, no devolvemos al usuario: devolvemos un **token**. Eso nos lleva a la pieza siguiente.

5. El token: no preguntar la clave en cada request

HTTP no tiene memoria. Cada request llega solo, sin saber nada de los anteriores. Si protegemos /tickets, el servidor necesita reconfirmar quién sos **en cada pedido**.

Pedir la contraseña en cada request sería absurdo y peligroso (más veces viaja la clave, más chances de que se filtre). La solución es el **token de sesión**: una credencial temporal que reemplaza a la contraseña una vez que ya probaste quién sos.

csharp

```
private static string GenerarToken() {
    byte[] bytes = RandomNumberGenerator.GetBytes(32);
    return Convert.ToBase64String(bytes);
}
```

El token es simplemente 32 bytes aleatorios convertidos a texto. Al hacer login se genera, se guarda en el usuario junto con su vencimiento (TokenExpira) y se le entrega al cliente. De ahí en más, el cliente lo manda en cada request y el servidor lo reconoce sin volver a tocar la contraseña.

RandomNumberGenerator, **nunca** Random: el token tiene que ser **impredecible**. Random produce secuencias que un atacante puede anticipar si adivina la semilla; sirve para barajar cartas, no para seguridad. RandomNumberGenerator es criptográficamente seguro: aunque veas mil tokens, no podés predecir el próximo.

Validar y cerrar la sesión son el reverso:

```

csharp
public async Task<Usuario?> ValidarTokenAsync(string token) {
    var usuario = await this.repositorio.ObtenerPorTokenAsync(token);
    if (usuario is null) {
        return null;
    }

    if (usuario.TokenExpira is null || usuario.TokenExpira < DateTime.Now) {
        return null;
    }

    return usuario;
}

public async Task LogoutAsync(string token) {
    var usuario = await this.repositorio.ObtenerPorTokenAsync(token);
    if (usuario is not null) {
        usuario.Token = null;
        usuario.TokenExpira = null;
        await this.repositorio.GuardarCambiosAsync();
    }
}

```

ValidarTokenAsync chequea **dos** cosas: que el token exista y que no haya vencido. Un token sin fecha o vencido vale lo mismo que ninguno.

***Por qué los tokens vencen:** si un token durara para siempre, robarlo una vez daría acceso eterno. El vencimiento a 8 horas limita el daño de un token filtrado. LogoutAsync hace lo mismo de forma manual: borra el token, así deja de servir aunque alguien lo tenga copiado.*

6. El middleware: identificar al que toca la puerta

Tenemos tokens, pero falta el mecanismo que los lea **en cada request, automáticamente**, antes de que el endpoint corra. Ese es el trabajo del **middleware**.

Un middleware es una función que se mete en el camino de **todos** los requests. En Program.cs:

```

csharp
// 2° Bearer: si viene un token valido, deja el usuario en HttpContext.Items.
app.Use(async (context, next) => {
    var auth = context.RequestServices.GetRequiredService<AuthService>();
    var header = context.Request.Headers.Authorization.ToString();

    if (header.StartsWith("Bearer ", StringComparison.OrdinalIgnoreCase)) {
        var token = header["Bearer ".Length..].Trim();
        var usuario = await auth.ValidarTokenAsync(token);

        if (usuario is not null) {
            context.Items["Usuario"] = usuario;
        }
    }

    await next(context);
});

```

Paso a paso: lee el header Authorization, que por convención llega como Bearer <token>. Si está, le recorta el prefijo "Bearer " y valida lo que queda. Si el token es válido, **deja al usuario en HttpContext.Items** y sigue con await next(context).

Qué es Bearer: es el esquema estándar de HTTP para "el que porta este token tiene acceso" (bearer = portador). No es magia de .NET, es una convención de la web; por eso el cliente manda `Authorization: Bearer abc123...`

`HttpContext.Items` **es un casillero por request:** un diccionario que nace y muere con cada pedido. El middleware deja ahí al usuario y cualquier endpoint posterior lo encuentra, sin volver a validar el token. El helper `ctx.UsuarioActual()` no es más que leer ese casillero.

Clave: el middleware NO rechaza a nadie. Si no hay token o es inválido, simplemente **no** deja usuario en `Items` y deja pasar el request igual. Suena raro, pero es a propósito: identificar y bloquear son trabajos distintos. El middleware solo identifica. Bloquear es del filtro.

`context.RequestServices.GetRequiredService<AuthService>()`: los delegates de `app.Use` no son *scoped*, pero `AuthService` sí lo es (arrastra el `DbContext`). Pedirlo así, dentro del delegate, respeta el ciclo de vida por request en lugar de capturar una instancia equivocada.

7. El filtro: cerrar la puerta

El middleware dejó (o no) un usuario en el casillero. Falta quien **decida** si el endpoint corre o se rechaza con 401. Ese es el trabajo de un **filtro de endpoint**.

Qué es un filtro

Un filtro de endpoint es una función que se mete **entre el routing y el handler**: corre justo antes de que se ejecute el código del endpoint, y puede dejarlo correr o cortar antes. Su forma es siempre la misma: recibe el `context` del request y un `next`, y decide si llama a `next` (seguir) o devuelve algo sin llamarlo (cortar).

Para nuestra autorización, la lógica es de tres líneas: miramos el casillero `Items["Usuario"]` que dejó el middleware Bearer; si está vacío, cortamos con 401; si hay usuario, dejamos pasar.

Filtro vs middleware, la diferencia clave: el middleware corre en **todos** los requests del sistema. El filtro corre solo en los endpoints donde uno lo engancha. Por eso el middleware Bearer (que identifica a todos) es middleware, y la autorización (que protege solo `/tickets`) es un filtro: queremos elegir a qué rutas aplicarlo.

Un filtro es, en el fondo, una función

La forma más directa de escribir un filtro es una **lambda** pasada a `AddEndpointFilter`. Sin clases ni ceremonia: la función `(context, next) => ...` es el filtro.

```
csharp
app.MapGroup("/tickets")
    .AddEndpointFilter(async (context, next) => {
        var usuario = context.HttpContext.Items["Usuario"] as Usuario;
        if (usuario is null) {
            return Results.Unauthorized();
        }
    })
```

csharp - continuacion

```
return await next(context);
});
```

Eso ya funciona y protege el grupo entero. Pero tiene un problema: el grupo `/tickets` **no es el único** que necesita protección. El endpoint `POST /auth/logout` también la pide (hay que estar logueado para cerrar sesión). Si dejáramos la lambda inline, tendríamos que **copiar y pegar** esas mismas tres líneas en cada lugar protegido. Y código duplicado es código que algún día vas a corregir en un lado y olvidar en el otro.

Un nombre para la función: el método de extensión

La solución es darle a esa función un **nombre reutilizable**. Un método de extensión envuelve el `AddEndpointFilter` con la lambda adentro, y lo expone como un método que se lee natural sobre cualquier ruta o grupo.

Creá `Api/AuthExtensions.cs`:

csharp

```
using TicketSystem.Domain;

namespace TicketSystem.Api;

public static class AuthExtensions {
    public static TBuilder RequiresAuth<TBuilder>(this TBuilder builder)
        where TBuilder : IEndpointConventionBuilder {
        return builder.AddEndpointFilter(async (context, next) => {
            var usuario = context.HttpContext.Items["Usuario"] as Usuario;
            if (usuario is null) {
                return Results.Unauthorized();
            }

            return await next(context);
        });
    }
}
```

Ahora proteger una ruta es escribir `.RequiresAuth()`, y la lógica vive en **un solo lugar**:

csharp

```
var ticketsGroup = app.MapGroup("/tickets")
    .RequiresAuth();
```

Por qué `IEndpointConventionBuilder` genérico: `MapGroup` devuelve un `RouteGroupBuilder`, pero `MapPost/MapGet` devuelven un `RouteHandlerBuilder`. Son tipos distintos. Si tipáramos el método para uno solo, no andaría en el otro. Como **ambos** implementan `IEndpointConventionBuilder`, el genérico con ese `where` cubre los dos. Y devolver `TBuilder` (en vez de la interfaz) conserva el tipo original, así podés seguir encadenando métodos después de `.RequiresAuth()`.

Por qué lambda y no una clase: un filtro también puede escribirse como una clase que implementa `IEndpointFilter`. Eso conviene cuando el filtro necesita **dependencias inyectadas por constructor** (un servicio, un logger). El nuestro no necesita ninguna: solo lee `Items["Usuario"]`, que el middleware ya dejó servido. Sin dependencias que inyectar, la lambda dentro de la extensión es más liviana y igual de reutilizable. Como dato: esto que escribimos es, en miniatura, lo mismo que hace el método `RequireAuthorization()` que trae ASP.NET Core —también es un método de extensión sobre `IEndpointConventionBuilder`—, solo que el del framework arrastra todo un sistema de esquemas y claims que para aprender el concepto es de más.

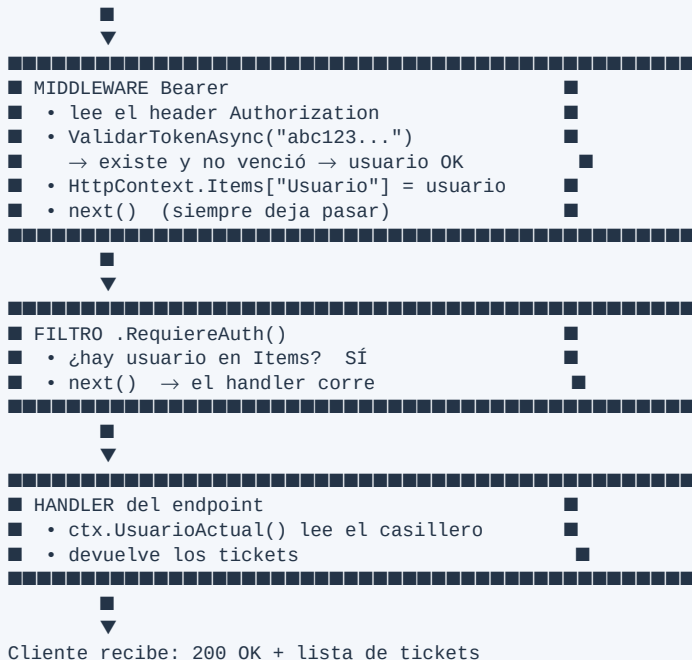
Por qué separar middleware y filtro: dividir "identificar" de "autorizar" deja al grupo /auth (login, registro) **público** —no lleva `.RequiereAuth()`, cualquiera puede loguearse— y a /tickets **protegido**. Si el middleware bloqueara por su cuenta, no habría forma de tener rutas públicas: nadie podría siquiera pedir un token. Separar las dos responsabilidades es lo que hace que el login pueda ser abierto y los tickets cerrados, con el mismo mecanismo.

8. El recorrido completo de un request

Juntemos las cuatro piezas siguiendo un GET /tickets con token válido:

texto

Cliente manda: GET /tickets
Authorization: Bearer abc123...



Ahora el mismo GET /tickets **sin** token, o con uno vencido:

texto

MIDDLEWARE: no hay token válido → NO deja usuario en Items → next()
FILTRO: ¿hay usuario en Items? NO → Results.Unauthorized()
Cliente recibe: 401 Unauthorized (el handler nunca corrió)

Y así cierra el circuito que arrancó en la sección 1:

Problema	Pieza que lo resuelve
Guardar la clave sin guardarla	PasswordHasher (hash + salt)

Problema	Pieza que lo resuelve
Comprobar la clave al entrar	<code>AuthService.LoginAsync</code>
No pedir la clave en cada request	Token de sesión + vencimiento
Identificar al usuario en cada request	Middleware Bearer + Items
Dejar entrar solo a los identificados	Filtro <code>.RequiresAuth()</code>

Cada pieza tapa el cabo que dejó la anterior. Esa es toda la autenticación del sistema, de la contraseña al 401.

Programación III — Tecnicatura Universitaria en Programación — UTN Facultad Regional Tucumán

CAPITULO 39

Sistema de Tickets con Minimal API y EF Core

Tutorial paso a paso para **Programación III** — C# 14 / .NET 10 Construimos un sistema de tickets minimalista de punta a punta: entidades, acceso a datos con EF Core, repositorios, servicios, autenticación con hash de contraseña + token de sesión, y una API REST con Minimal API, middleware y agrupación de rutas.

1. Qué vamos a construir

Un sistema donde **clientes y usuarios internos comparten una misma tabla**, pueden iniciar sesión, y cada ticket registra:

- **Quién lo origina y quién es el responsable.**
- Un **estado** (Abierto, EnProceso, Cerrado).
- Una **bitácora de acciones**, donde cada acción tiene una fecha: si la fecha ya pasó, es algo que se hizo; si es futura, es un **compromiso pendiente**.

La arquitectura sigue un patrón en capas clásico:

```
texto
HTTP (Minimal API + middleware)
  ▼
Servicios ← lógica del sistema
  ▼
Repositorios ← acceso a datos (interface + implementación)
  ▼
DbContext (EF Core) ← SQLite
```

Por qué capas: cada capa solo conoce a la de abajo a través de una interface. Eso nos deja cambiar SQLite por otra base, o testear los servicios con un repositorio falso, sin tocar el resto. No es sobre-ingeniería: es la separación mínima para que el código no se vuelva una bola de barro.

2. Crear el proyecto

Abrí una terminal y ejecutá:

```
bash
dotnet new web -n TicketSystem
cd TicketSystem
```

Agregamos el paquete de EF Core con proveedor SQLite:

```
bash
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
```

dotnet new web crea el template más liviano de ASP.NET Core, ideal para Minimal API: no trae controladores ni vistas, solo un Program.cs con lo justo. Como vamos a trabajar **code-first sin migraciones** (la base se crea sola a partir de las entidades), nos alcanza con el proveedor SQLite: no hace falta EntityFrameworkCore.Design ni la herramienta dotnet-ef.

Vamos a organizar el código en carpetas. Creálas ahora:

```
bash
mkdir Domain Data Repositories Security Services Api
```

Cada carpeta es un namespace y una responsabilidad:

Carpeta	Contiene
Domain	Las entidades (Usuario, Ticket, Accion) y enums
Data	El DbContext
Repositories	Interfaces e implementaciones de acceso a datos
Security	El hasher de contraseñas
Services	La lógica del sistema
Api	DTOs, filtros y helpers de la capa HTTP

3. Las entidades del dominio

Las entidades son las clases que EF Core mapea a tablas. Empezamos por los **enums** y la entidad Usuario.

Creá el archivo Domain/Usuario.cs:

```
csharp
namespace TicketSystem.Domain;

public enum TipoUsuario {
    Cliente,
    Interno
}

public class Usuario {
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public TipoUsuario Tipo { get; set; }

    // Autenticación
    public string PasswordHash { get; set; } = string.Empty;
    public string PasswordSalt { get; set; } = string.Empty;
    public string? Token { get; set; }
    public DateTime? TokenExpira { get; set; }
}
```

csharp - continuacion

```
// Navegación
public ICollection<Ticket> TicketsOriginados { get; set; } = new List<Ticket>();
public ICollection<Ticket> TicketsAsignados { get; set; } = new List<Ticket>();
}
```

= string.Empty evita advertencias del compilador por valores nulos: una propiedad string no anulable tiene que arrancar con algo. El Token y TokenExpira sí son anulables (string?, DateTime?) porque un usuario recién creado todavía no inició sesión.

Ahora Domain/Ticket.cs:

csharp

```
namespace TicketSystem.Domain;

public enum EstadoTicket {
    Abierto,
    EnProceso,
    Cerrado
}

public class Ticket {
    public int Id { get; set; }
    public string Titulo { get; set; } = string.Empty;
    public string? Descripcion { get; set; }
    public int OriginadoPorId { get; set; }
    public int? ResponsableId { get; set; }
    public EstadoTicket Estado { get; set; } = EstadoTicket.Abierto;
    public DateTime FechaCreacion { get; set; } = DateTime.Now;

    // Navegación
    public Usuario OriginadoPor { get; set; } = null!;
    public Usuario? Responsable { get; set; }
    public ICollection<Accion> Acciones { get; set; } = new List<Accion>();
}
```

Dos FK al mismo tipo: OriginadoPorId y ResponsableId apuntan ambas a Usuario. El responsable es anulable (int?) porque un ticket puede entrar sin asignar. El = null! en OriginadoPor le dice al compilador "confía, EF lo va a llenar", evitando la advertencia de nulo sin hacer la propiedad anulable.

Y por último Domain/Accion.cs:

csharp

```
namespace TicketSystem.Domain;

public class Accion {
    public int Id { get; set; }
    public int TicketId { get; set; }
    public string Descripcion { get; set; } = string.Empty;
    public int RegistradaPorId { get; set; }
    public DateTime Fecha { get; set; }
    public bool Realizada { get; set; }

    // Navegación
    public Ticket Ticket { get; set; } = null!;
    public Usuario RegistradaPor { get; set; } = null!;
}
```

El corazón del requerimiento "acciones pasadas o futuras": el campo `Fecha` resuelve todo sin tablas extra. `Fecha pasada` → ya se hizo. `Fecha futura` → compromiso. El flag `Realizada` permite marcar cuándo un compromiso futuro efectivamente se cumplió.

4. El DbContext

El `DbContext` es el puente entre las entidades y la base. Cada `DbSet<T>` es una tabla.

Creá `Data/TicketContext.cs`:

```
csharp
using Microsoft.EntityFrameworkCore;
using TicketSystem.Domain;

namespace TicketSystem.Data;

public class TicketContext : DbContext {
    public DbSet<Usuario> Usuarios => Set<Usuario>();
    public DbSet<Ticket> Tickets => Set<Ticket>();
    public DbSet<Accion> Acciones => Set<Accion>();

    public TicketContext(DbContextOptions<TicketContext> options)
        : base(options) {
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder) {
        modelBuilder.Entity<Ticket>(entity => {
            entity.HasOne(t => t.OriginadoPor)
                .WithMany(u => u.TicketsOriginados)
                .HasForeignKey(t => t.OriginadoPorId)
                .OnDelete(DeleteBehavior.Restrict);

            entity.HasOne(t => t.Responsable)
                .WithMany(u => u.TicketsAsignados)
                .HasForeignKey(t => t.ResponsableId)
                .OnDelete(DeleteBehavior.Restrict);
        });

        modelBuilder.Entity<Accion>(entity => {
            entity.HasOne(a => a.Ticket)
                .WithMany(t => t.Acciones)
                .HasForeignKey(a => a.TicketId)
                .OnDelete(DeleteBehavior.Cascade);

            entity.HasOne(a => a.RegistradaPor)
                .WithMany()
                .HasForeignKey(a => a.RegistradaPorId)
                .OnDelete(DeleteBehavior.Restrict);
        });
    }
}
```

Por qué configuramos las relaciones a mano: cuando una entidad tiene **dos FK** hacia el mismo tipo (`Ticket` → `Usuario`, por origen y por responsable), EF Core no puede adivinar qué propiedad de navegación corresponde a cada una. Hay que decirse explícito con `HasOne` / `WithMany` / `HasForeignKey`. **Restrict vs Cascade:** con `Restrict`, EF no te deja borrar un usuario si tiene tickets (te protege de borrados en cadena accidentales). Con `Cascade` en las acciones, sí: borrar un ticket borra sus acciones, porque una acción no tiene sentido sin su ticket.

5. Los repositorios

Un repositorio aísla el acceso a datos detrás de una interface. El resto del sistema pide datos sin saber que abajo hay EF Core.

Repositorio de Ticket

Creá Repositories/ITicketRepository.cs:

```
csharp
using TicketSystem.Domain;

namespace TicketSystem.Repositories;

public interface ITicketRepository {
    Task<Ticket?> ObtenerPorIdAsync(int id);
    Task<IEnumerable<Ticket>> ObtenerTodosAsync();
    Task<IEnumerable<Ticket>> ObtenerPorEstadoAsync(EstadoTicket estado);
    Task AgregarAsync(Ticket ticket);
    Task AgregarAccionAsync(Accion accion);
    Task GuardarCambiosAsync();
}
```

Y su implementación, Repositories/TicketRepository.cs:

```
csharp
using Microsoft.EntityFrameworkCore;
using TicketSystem.Data;
using TicketSystem.Domain;

namespace TicketSystem.Repositories;

public class TicketRepository : ITicketRepository {
    private readonly TicketContext context;

    public TicketRepository(TicketContext context) {
        this.context = context;
    }

    public async Task<Ticket?> ObtenerPorIdAsync(int id) {
        return await this.context.Tickets
            .Include(t => t.OriginadoPor)
            .Include(t => t.Responsable)
            .Include(t => t.Acciones)
            .ThenInclude(a => a.RegistradaPor)
            .FirstOrDefaultAsync(t => t.Id == id);
    }

    public async Task<IEnumerable<Ticket>> ObtenerTodosAsync() {
        return await this.context.Tickets
            .Include(t => t.OriginadoPor)
            .Include(t => t.Responsable)
            .ToListAsync();
    }

    public async Task<IEnumerable<Ticket>> ObtenerPorEstadoAsync(EstadoTicket estado) {
        return await this.context.Tickets
            .Include(t => t.OriginadoPor)
            .Include(t => t.Responsable)
            .Where(t => t.Estado == estado)
            .ToListAsync();
    }

    public async Task AgregarAsync(Ticket ticket) {
        await this.context.Tickets.AddAsync(ticket);
    }
}
```

csharp - continuacion

```

public async Task AgregarAccionAsync(Accion accion) {
    await this.context.Acciones.AddAsync(accion);
}

public async Task GuardarCambiosAsync() {
    await this.context.SaveChangesAsync();
}
}

```

Include / ThenInclude: EF Core por defecto NO trae las propiedades de navegación (las relaciones). Si querés que ticket.OriginadoPor venga lleno, hay que pedirlo con Include. El ThenInclude baja un nivel más: de la acción a quién la registró. **Por qué GuardarCambiosAsync está separado:** el repo no persiste en cada método. Acumula cambios y los guarda cuando el servicio lo decide. Así varias operaciones pueden ir en una sola transacción.

Repositorio de Usuario

Creá Repositories/IUsuarioRepository.cs:

csharp

```

using TicketSystem.Domain;

namespace TicketSystem.Repositories;

public interface IUsuarioRepository {
    Task<Usuario?> ObtenerPorIdAsync(int id);
    Task<Usuario?> ObtenerPorEmailAsync(string email);
    Task<Usuario?> ObtenerPorTokenAsync(string token);
    Task<IEnumerable<Usuario>> ObtenerTodosAsync();
    Task AgregarAsync(Usuario usuario);
    Task GuardarCambiosAsync();
}

```

Y Repositories/UsuarioRepository.cs:

csharp

```

using Microsoft.EntityFrameworkCore;
using TicketSystem.Data;
using TicketSystem.Domain;

namespace TicketSystem.Repositories;

public class UsuarioRepository : IUsuarioRepository {
    private readonly TicketContext context;

    public UsuarioRepository(TicketContext context) {
        this.context = context;
    }

    public async Task<Usuario?> ObtenerPorIdAsync(int id) {
        return await this.context.Usuarios.FindAsync(id);
    }

    public async Task<Usuario?> ObtenerPorEmailAsync(string email) {
        return await this.context.Usuarios
            .FirstOrDefaultAsync(u => u.Email == email);
    }

    public async Task<Usuario?> ObtenerPorTokenAsync(string token) {
        return await this.context.Usuarios
            .FirstOrDefaultAsync(u => u.Token == token);
    }
}

```

csharp - continuacion

```

public async Task<IEnumerable<Usuario>> ObtenerTodosAsync() {
    return await this.context.Usuarios.ToListAsync();
}

public async Task AgregarAsync(Usuario usuario) {
    await this.context.Usuarios.AddAsync(usuario);
}

public async Task GuardarCambiosAsync() {
    await this.context.SaveChangesAsync();
}
}

```

FindAsync vs FirstOrDefaultAsync: FindAsync *busca por clave primaria y, si la entidad ya está en memoria, ni va a la base.* FirstOrDefaultAsync *lo usamos cuando buscamos por otra columna (email, token) con una condición.*

6. El helper de contraseñas

Nunca se guarda una contraseña en texto plano. Guardamos un *hash* (resultado irreversible) junto con un *salt* (valor aleatorio que hace único cada hash).

Creá Security/PasswordHasher.cs:

csharp

```

using System.Security.Cryptography;

namespace TicketSystem.Security;

public static class PasswordHasher {
    private const int SaltSize = 16; // 128 bits
    private const int HashSize = 32; // 256 bits
    private const int Iterations = 100_000;

    public static (string Hash, string Salt) Hashear(string password) {
        byte[] salt = RandomNumberGenerator.GetBytes(SaltSize);
        byte[] hash = Pbkdf2(password, salt);

        return (Convert.ToBase64String(hash), Convert.ToBase64String(salt));
    }

    public static bool Verificar(string password, string hashGuardado, string saltGuardado) {
        byte[] salt = Convert.FromBase64String(saltGuardado);
        byte[] hash = Pbkdf2(password, salt);
        byte[] esperado = Convert.FromBase64String(hashGuardado);

        return CryptographicOperations.FixedTimeEquals(hash, esperado);
    }

    private static byte[] Pbkdf2(string password, byte[] salt) {
        return Rfc2898DeriveBytes.Pbkdf2(
            password,
            salt,
            Iterations,
            HashAlgorithmName.SHA256,
            HashSize);
    }
}

```

PBKDF2 aplica el hash 100.000 veces a propósito: hace que probar contraseñas por fuerza bruta sea lentísimo para un atacante. Viene en .NET, sin paquetes externos. **El salt** es por qué dos usuarios con la misma contraseña terminan con hashes distintos: se mezcla un valor aleatorio antes de hashear. `FixedTimeEquals` compara siempre en el mismo tiempo, aunque los bytes difieran desde el primero. Un `==` común "corta" apenas encuentra una diferencia, y midiendo cuánto tarda un atacante podría deducir el hash byte a byte (ataque de timing).

7. Los servicios

Los servicios contienen la **lógica del sistema**. Hablan con los repositorios (vía interface) y exponen operaciones de negocio.

Servicio de autenticación

Creá `Services/AuthService.cs`:

```
csharp
using System.Security.Cryptography;
using TicketSystem.Domain;
using TicketSystem.Repositories;
using TicketSystem.Security;

namespace TicketSystem.Services;

public class AuthService {
    private readonly IUserarioRepository repositorio;
    private static readonly TimeSpan DuracionSesion = TimeSpan.FromHours(8);

    public AuthService(IUserarioRepository repositorio) {
        this.repositorio = repositorio;
    }

    public async Task<Usuario> RegistrarAsync(
        string nombre,
        string email,
        string password,
        TipoUsuario tipo) {
        var existente = await this.repositorio.ObtenerPorEmailAsync(email);
        if (existente is not null) {
            throw new InvalidOperationException("Ya existe un usuario con ese email.");
        }

        var (hash, salt) = PasswordHasher.Hashear(password);

        var usuario = new Usuario {
            Nombre = nombre,
            Email = email,
            Tipo = tipo,
            PasswordHash = hash,
            PasswordSalt = salt
        };

        await this.repositorio.AgregarAsync(usuario);
        await this.repositorio.GuardarCambiosAsync();
        return usuario;
    }

    public async Task<string?> LoginAsync(string email, string password) {
        var usuario = await this.repositorio.ObtenerPorEmailAsync(email);
        if (usuario is null) {
            return null;
        }
    }
}
```


csharp - continuacion

```

        bool valido = PasswordHasher.Verificar(
            password, usuario.PasswordHash, usuario.PasswordSalt);

        if (!valido) {
            return null;
        }

        usuario.Token = GenerarToken();
        usuario.TokenExpira = DateTime.Now.Add(DuracionSesion);
        await this.repositorio.GuardarCambiosAsync();

        return usuario.Token;
    }

    public async Task<Usuario?> ValidarTokenAsync(string token) {
        var usuario = await this.repositorio.ObtenerPorTokenAsync(token);
        if (usuario is null) {
            return null;
        }

        if (usuario.TokenExpira is null || usuario.TokenExpira < DateTime.Now) {
            return null;
        }

        return usuario;
    }

    public async Task LogoutAsync(string token) {
        var usuario = await this.repositorio.ObtenerPorTokenAsync(token);
        if (usuario is not null) {
            usuario.Token = null;
            usuario.TokenExpira = null;
            await this.repositorio.GuardarCambiosAsync();
        }
    }

    private static string GenerarToken() {
        byte[] bytes = RandomNumberGenerator.GetBytes(32);
        return Convert.ToBase64String(bytes);
    }
}

```

El flujo de sesión: LoginAsync verifica la contraseña, y si está bien genera un token aleatorio con su vencimiento (8 horas). ValidarTokenAsync chequea las dos cosas: que el token exista y que no haya vencido. LogoutAsync simplemente borra el token. RandomNumberGenerator, **no** Random: el token tiene que ser impredecible. Random es predecible (sirve para juegos, no para seguridad); RandomNumberGenerator es criptográficamente seguro.

Servicio de tickets

Creá Services/TicketService.cs:

csharp

```

using TicketSystem.Domain;
using TicketSystem.Repositories;

namespace TicketSystem.Services;

public class TicketService {
    private readonly ITicketRepository repositorio;

    public TicketService(ITicketRepository repositorio) {
        this.repositorio = repositorio;
    }
}

```

csharp - continuacion

```
public async Task<Ticket> CrearTicketAsync(
    string titulo,
    string? descripcion,
    int originadoPorId,
    int? responsableId = null) {
    var ticket = new Ticket {
        Titulo = titulo,
        Descripcion = descripcion,
        OriginadoPorId = originadoPorId,
        ResponsableId = responsableId,
        Estado = EstadoTicket.Abierto,
        FechaCreacion = DateTime.Now
    };

    await this.repositorio.AgregarAsync(ticket);
    await this.repositorio.GuardarCambiosAsync();
    return ticket;
}

public async Task<Ticket?> ObtenerTicketAsync(int id) {
    return await this.repositorio.ObtenerPorIdAsync(id);
}

public async Task<IEnumerable<Ticket>> ListarTicketsAsync() {
    return await this.repositorio.ObtenerTodosAsync();
}

public async Task<IEnumerable<Ticket>> ListarPorEstadoAsync(EstadoTicket estado) {
    return await this.repositorio.ObtenerPorEstadoAsync(estado);
}

public async Task CambiarEstadoAsync(int ticketId, EstadoTicket nuevoEstado) {
    var ticket = await this.repositorio.ObtenerPorIdAsync(ticketId)
        ?? throw new InvalidOperationException($"No existe el ticket {ticketId}.");

    ticket.Estado = nuevoEstado;
    await this.repositorio.GuardarCambiosAsync();
}

public async Task AsignarResponsableAsync(int ticketId, int responsableId) {
    var ticket = await this.repositorio.ObtenerPorIdAsync(ticketId)
        ?? throw new InvalidOperationException($"No existe el ticket {ticketId}.");

    ticket.ResponsableId = responsableId;
    await this.repositorio.GuardarCambiosAsync();
}

public async Task<Accion> RegistrarAccionAsync(
    int ticketId,
    string descripcion,
    int registradaPorId,
    DateTime fecha) {
    var ticket = await this.repositorio.ObtenerPorIdAsync(ticketId)
        ?? throw new InvalidOperationException($"No existe el ticket {ticketId}.");

    var accion = new Accion {
        TicketId = ticketId,
        Descripcion = descripcion,
        RegistradaPorId = registradaPorId,
        Fecha = fecha,
        Realizada = fecha <= DateTime.Now
    };

    await this.repositorio.AgregarAccionAsync(accion);
    await this.repositorio.GuardarCambiosAsync();
    return accion;
}

public async Task MarcarAccionRealizadaAsync(int ticketId, int accionId) {
    var ticket = await this.repositorio.ObtenerPorIdAsync(ticketId)
        ?? throw new InvalidOperationException($"No existe el ticket {ticketId}.");
```

csharp - continuacion

```

var accion = ticket.Acciones.FirstOrDefault(a => a.Id == accionId)
?? throw new InvalidOperationException($"No existe la acción {accionId} en el ticket.");

accion.Realizada = true;
await this.repositorio.GuardarCambiosAsync();
}
}

```

El ?? throw: si ObtenerPorIdAsync devuelve null, lanzamos la excepción en la misma línea. Es el operador de coalescencia nula combinado con throw como expresión: corto y claro. Realizada = fecha <= DateTime.Now: acá se materializa la regla. Si registrás una acción con fecha pasada, queda como ya hecha. Si la fecha es futura, queda pendiente (compromiso), y después se cierra con MarcarAccionRealizadaAsync.

8. Los DTOs

Los **DTOs** (Data Transfer Objects) son los objetos que viajan por HTTP. Usamos record porque son inmutables y concisos: ideal para datos que solo transportan.

Creá Api/Dtos.cs:

csharp

```

using TicketSystem.Domain;

namespace TicketSystem.Api;

public record RegistroDto(string Nombre, string Email, string Password, TipoUsuario Tipo);
public record LoginDto(string Email, string Password);
public record CrearTicketDto(string Titulo, string? Descripcion, int? ResponsableId);
public record CambiarEstadoDto(EstadoTicket Estado);
public record AsignarResponsableDto(int ResponsableId);
public record RegistrarAccionDto(string Descripcion, DateTime Fecha);

```

Por qué DTOs y no las entidades directo: nunca exponemos la entidad Usuario en la API, porque tiene el PasswordHash. Los DTOs definen exactamente qué entra y qué sale, sin filtrar datos sensibles ni provocar ciclos infinitos al serializar las navegaciones.

9. El helper y la verificación de autenticación

Necesitamos dos piezas para la capa HTTP: un **helper** para leer el usuario logueado y una **extensión reusable** para proteger rutas.

Creá Api/AuthHelper.cs:

csharp

```

using TicketSystem.Domain;

namespace TicketSystem.Api;

public static class AuthHelper {
    public static Usuario? UsuarioActual(this HttpContext context) {

```

csharp - continuacion

```

    }
    return context.Items["Usuario"] as Usuario;
}

```

`HttpContext.Items` es un diccionario que vive solo durante el request actual. El middleware Bearer va a dejar ahí al usuario, y cualquier endpoint lo lee con `ctx.UsuarioActual()`. Es un método de extensión (`this HttpContext`) para que se lea natural.

Creá `Api/AuthExtensions.cs`:

csharp

```

using TicketSystem.Domain;

namespace TicketSystem.Api;

public static class AuthExtensions {
    public static TBuilder RequiereAuth<TBuilder>(this TBuilder builder)
        where TBuilder : IEndpointConventionBuilder {
        return builder.AddEndpointFilter(async (context, next) => {
            var usuario = context.HttpContext.Items["Usuario"] as Usuario;
            if (usuario is null) {
                return Results.Unauthorized();
            }

            return await next(context);
        });
    }
}

```

Qué hace la extensión: envuelve un filtro inline y le da un nombre reutilizable. Si no hay usuario en `HttpContext.Items` (porque no vino token o era inválido), corta con 401 Unauthorized y el handler ni se ejecuta. Al exponerlo como `.RequiereAuth()`, la misma verificación sirve tanto para un grupo entero como para un endpoint aislado, sin copiar y pegar la lambda.

10. El seeder de datos (solo desarrollo)

Para no registrar todo a mano y, sobre todo, para tener material variado con que probar los filtros por estado, precargamos varios usuarios y tickets en distintos estados, cada uno con acciones pasadas y futuras mezcladas. El seeder **es solo para desarrollo**: en producción NUNCA se siembran usuarios con contraseñas conocidas ni datos de prueba.

Creá `Data/SeedData.cs`:

csharp

```

using TicketSystem.Domain;
using TicketSystem.Security;

namespace TicketSystem.Data;

// ATENCION: este seeder es EXCLUSIVAMENTE para desarrollo y pruebas.
// Crea usuarios con contraseñas conocidas. NUNCA debe ejecutarse en produccion.
public static class SeedData {
    public static async Task SembrarAsync(TicketContext context) {
        // Si ya hay usuarios, asumimos que la base ya fue sembrada y salimos.
        // Esto hace que el seeder sea idempotente: correrlo dos veces no duplica.
    }
}

```

csharp - continuacion

```

    if (context.Usuarios.Any()) {
        return;
    }

    // --- Usuarios ---
    // Dos internos (los que resuelven) y tres clientes (los que abren tickets).
    var ana = CrearUsuario("Soporte Ana", "ana@empresa.com", "clave1234", TipoUsuario.Interno);
    var beto = CrearUsuario("Soporte Beto", "beto@empresa.com", "clave1234", TipoUsuario.Interno);
    var clara = CrearUsuario("Clara Cliente", "clara@cliente.com", "clave5678",
TipoUsuario.Cliente);
    var diego = CrearUsuario("Diego Cliente", "diego@cliente.com", "clave5678",
TipoUsuario.Cliente);
    var elsa = CrearUsuario("Elsa Cliente", "elsa@cliente.com", "clave5678", TipoUsuario.Cliente);

    await context.Usuarios.AddRangeAsync(ana, beto, clara, diego, elsa);
    await context.SaveChangesAsync();

    // A partir de aca los Id ya estan asignados por la base.
    var hoy = DateTime.Now;

    // --- Tickets ---
    // Usamos una fecha base relativa a "hoy" para que las acciones pasadas
    // y futuras queden siempre coherentes, corras esto cuando lo corras.

    // 1) CERRADO: ya resuelto, todas sus acciones en el pasado.
    var t1 = CrearTicket(
        "No llegan los mails de aviso",
        "Deje de recibir notificaciones por correo",
        originadoPor: clara, responsable: ana,
        estado: EstadoTicket.Cerrado, creado: hoy.AddDays(-10));

    // 2) EN PROCESO: arrancado, con trabajo hecho y un compromiso pendiente.
    var t2 = CrearTicket(
        "La factura sale con el total mal",
        "El IVA se calcula sobre el monto equivocado",
        originadoPor: diego, responsable: beto,
        estado: EstadoTicket.EnProceso, creado: hoy.AddDays(-3));

    // 3) EN PROCESO: otro mas, para que el filtro devuelva varios.
    var t3 = CrearTicket(
        "La app se cierra al exportar PDF",
        "Al tocar 'Exportar' la pantalla se va a blanco",
        originadoPor: elsa, responsable: ana,
        estado: EstadoTicket.EnProceso, creado: hoy.AddDays(-1));

    // 4) ABIERTO: recién entrado, sin responsable todavía.
    var t4 = CrearTicket(
        "Quiero cambiar mi correo de contacto",
        "Necesito actualizar el email de mi cuenta",
        originadoPor: clara, responsable: null,
        estado: EstadoTicket.Abierto, creado: hoy.AddHours(-2));

    await context.Tickets.AddRangeAsync(t1, t2, t3, t4);
    await context.SaveChangesAsync();

    // --- Acciones ---
    // Mezcla deliberada de pasadas (Realizada se infiere true) y futuras
    // (compromisos, Realizada false). Replicamos la regla del servicio aca.

    var acciones = new List<Accion> {
        // Ticket 1 (cerrado): historia completa, todo en el pasado.
        CrearAccion(t1, "Reproduce el problema en el servidor de prueba", ana, hoy.AddDays(-9)),
        CrearAccion(t1, "Detecte el filtro de spam mal configurado", ana, hoy.AddDays(-8)),
        CrearAccion(t1, "Aplique el fix y confirme con la clienta", ana, hoy.AddDays(-7)),

        // Ticket 2 (en proceso): algo hecho + un compromiso a futuro.
        CrearAccion(t2, "Revise la formula de calculo del IVA", beto, hoy.AddDays(-2)),
        CrearAccion(t2, "Reunion con contabilidad para validar la alicuota", beto, hoy.AddDays(2)),

        // Ticket 3 (en proceso): solo un compromiso pendiente.
        CrearAccion(t3, "Analizar los logs de la exportacion", ana, hoy.AddDays(1)),
    }

```

csharp - continuacion

```

        // Ticket 4 (abierto): un compromiso de primer contacto.
        CrearAccion(t4, "Llamar a la clienta para verificar identidad", ana, hoy.AddDays(1)),
    };

    await context.Acciones.AddRangeAsync(acciones);
    await context.SaveChangesAsync();
}

// --- Helpers privados: arman las entidades sin repetir codigo ---

private static Usuario CrearUsuario(
    string nombre, string email, string password, TipoUsuario tipo) {
    var (hash, salt) = PasswordHasher.Hashear(password);
    return new Usuario {
        Nombre = nombre,
        Email = email,
        Tipo = tipo,
        PasswordHash = hash,
        PasswordSalt = salt
    };
}

private static Ticket CrearTicket(
    string titulo, string descripcion,
    Usuario originadoPor, Usuario? responsable,
    EstadoTicket estado, DateTime creado) {
    return new Ticket {
        Titulo = titulo,
        Descripcion = descripcion,
        OriginadoPorId = originadoPor.Id,
        ResponsableId = responsable?.Id,
        Estado = estado,
        FechaCreacion = creado
    };
}

private static Accion CrearAccion(
    Ticket ticket, string descripcion, Usuario registradaPor, DateTime fecha) {
    return new Accion {
        TicketId = ticket.Id,
        Descripcion = descripcion,
        RegistradaPorId = registradaPor.Id,
        Fecha = fecha,
        // Misma regla que el servicio: pasado -> hecho, futuro -> pendiente.
        Realizada = fecha <= DateTime.Now
    };
}
}

```

Esto deja la base con un panorama realista: **5 usuarios** (2 internos, 3 clientes) y **4 tickets** repartidos en Cerrado (1), EnProceso (2) y Abierto (1), con un total de 7 acciones donde conviven historial pasado y compromisos a futuro.

Por qué fechas relativas a hoy (`hoy.AddDays(-9)`, `hoy.AddDays(2)`): si pusiéramos fechas fijas tipo "2026-05-30", con el correr de las semanas todas quedarían en el pasado y perderíamos los casos de "compromiso futuro". Anclando todo a `DateTime.Now`, la mezcla pasado/futuro se mantiene siempre, sin importar cuándo corras el seeder. **Por qué los helpers privados** (`CrearUsuario`, `CrearTicket`, `CrearAccion`): sin ellos, armar 5 usuarios + 4 tickets + 7 acciones sería un muro de inicializadores repetidos. Cada helper concentra el patrón de creación en un solo lugar; el cuerpo del seeder queda legible, casi como leer los datos en una tabla. **Por qué pasamos los objetos** (`originadoPor: clara`) **y no los Id**: es más legible y a prueba de errores. Como sembramos usuarios y guardamos primero, cuando los helpers leen `.Id` ya están asignados. El responsable? `.Id` con el `?` permite el caso del ticket abierto sin responsable (`null`). **Por qué dos bloques de `SaveChangesAsync` encadenados**: mismo motivo que antes, escalado. Guardamos usuarios → obtenemos sus `Id` → creamos tickets que los referencian → guardamos tickets → obtenemos sus `Id` → creamos acciones que los referencian. Cada nivel necesita los `Id` del anterior. **El comentario de advertencia en el código** no es decorativo: deja explícito que esto jamás va a producción. Un seeder con contraseñas conocidas en un sistema real es un agujero de seguridad.

11. Armar la API en Program.cs

Acá juntamos todo: registro de servicios, middleware como funciones inline, y los endpoints agrupados.

Reemplazá **todo** el contenido de `Program.cs` por esto:

```
csharp
using System.Diagnostics;
using System.Text.Json.Serialization;
using Microsoft.EntityFrameworkCore;
using TicketSystem.Api;
using TicketSystem.Data;
using TicketSystem.Domain;
using TicketSystem.Repositories;
using TicketSystem.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.ConfigureHttpJsonOptions(options => {
    options.SerializerOptions.Converters.Add(new JsonStringEnumConverter());
});

// --- Servicios (inyección de dependencias) ---
builder.Services.AddDbContext<TicketContext>(options =>
    options.UseSqlite("Data Source=tickets.db"));

builder.Services.AddScoped<ITicketRepository, TicketRepository>();
builder.Services.AddScoped<IUsuarioRepository, UsuarioRepository>();
builder.Services.AddScoped<TicketService>();
builder.Services.AddScoped<AuthService>();

var app = builder.Build();

// --- Base de datos: code-first sin migraciones ---
// EnsureCreated() crea el archivo y las tablas a partir de las entidades
// si todavía no existen. Además, SOLO en desarrollo, sembramos datos de prueba.
using (var scope = app.Services.CreateScope()) {
    var context = scope.ServiceProvider.GetRequiredService<TicketContext>();
    await context.Database.EnsureCreatedAsync();

    if (app.Environment.IsDevelopment()) {
        await SeedData.SembrarAsync(context);
    }
}
```

```

csharp - continuacion
// ===== MIDDLEWARE COMO FUNCIONES =====

// 1° Logging: mide tiempo y registra método, ruta y status.
app.Use(async (context, next) => {
    var logger = context.RequestServices
        .GetRequiredService<ILogger<Program>>();
    var cronometro = Stopwatch.StartNew();

    logger.LogInformation("-> {Metodo} {Ruta}",
        context.Request.Method, context.Request.Path);

    await next(context);

    cronometro.Stop();
    logger.LogInformation("<- {Metodo} {Ruta} respondio {Status} en {Ms}ms",
        context.Request.Method, context.Request.Path,
        context.Response.StatusCode, cronometro.ElapsedMilliseconds);
});

// 2° Bearer: si viene un token valido, deja el usuario en HttpContext.Items.
app.Use(async (context, next) => {
    var auth = context.RequestServices.GetRequiredService<AuthService>();
    var header = context.Request.Headers.Authorization.ToString();

    if (header.StartsWith("Bearer ", StringComparison.OrdinalIgnoreCase)) {
        var token = header["Bearer ".Length..].Trim();
        var usuario = await auth.ValidarTokenAsync(token);

        if (usuario is not null) {
            context.Items["Usuario"] = usuario;
        }
    }

    await next(context);
});

// ===== GRUPO PUBLICO: /auth =====

var authGroup = app.MapGroup("/auth");

authGroup.MapPost("/registro", async (RegistroDto dto, AuthService auth) => {
    try {
        var u = await auth.RegistrarAsync(dto.Nombre, dto.Email, dto.Password, dto.Tipo);
        return Results.Created($"/usuarios/{u.Id}",
            new { u.Id, u.Nombre, u.Email, u.Tipo });
    }
    catch (InvalidOperationException ex) {
        return Results.BadRequest(new { error = ex.Message });
    }
});

authGroup.MapPost("/login", async (LoginDto dto, AuthService auth) => {
    var token = await auth.LoginAsync(dto.Email, dto.Password);
    return token is null
        ? Results.Unauthorized()
        : Results.Ok(new { token });
});

authGroup.MapPost("/logout", async (HttpContext ctx, AuthService auth) => {
    var usuario = ctx.UsuarioActual();
    await auth.LogoutAsync(usuario.Token!);
    return Results.Ok(new { mensaje = "Sesion cerrada." });
})
.RequiereAuth();

// ===== GRUPO PROTEGIDO: /tickets =====

var ticketsGroup = app.MapGroup("/tickets")
    .RequiereAuth();

ticketsGroup.MapGet("/", async (TicketService svc) => {
    var tickets = await svc.ListarTicketsAsync();

```


csharp - continuacion

```

    return Results.Ok(tickets.Select(MapearTicket));
});

ticketsGroup.MapGet("/{id:int}", async (int id, TicketService svc) => {
    var ticket = await svc.ObtenerTicketAsync(id);
    return ticket is null ? Results.NotFound() : Results.Ok(MapearTicket(ticket));
});

ticketsGroup.MapGet("/estado/{estado}", async (EstadoTicket estado, TicketService svc) => {
    var tickets = await svc.ListarPorEstadoAsync(estado);
    return Results.Ok(tickets.Select(MapearTicket));
});

ticketsGroup.MapPost("/", async (CrearTicketDto dto, HttpContext ctx, TicketService svc) => {
    var usuario = ctx.UsuarioActual();
    var ticket = await svc.CrearTicketAsync(
        dto.Titulo, dto.Descripcion, usuario.Id, dto.ResponsableId);
    return Results.Created($" /tickets/{ticket.Id}", MapearTicket(ticket));
});

ticketsGroup.MapPut("/{id:int}/estado", async (int id, CambiarEstadoDto dto, TicketService svc) => {
    try {
        await svc.CambiarEstadoAsync(id, dto.Estado);
        return Results.NoContent();
    }
    catch (InvalidOperationException ex) {
        return Results.NotFound(new { error = ex.Message });
    }
});

ticketsGroup.MapPut("/{id:int}/responsable", async (int id, AsignarResponsableDto dto, TicketService
svc) => {
    try {
        await svc.AsignarResponsableAsync(id, dto.ResponsableId);
        return Results.NoContent();
    }
    catch (InvalidOperationException ex) {
        return Results.NotFound(new { error = ex.Message });
    }
});

ticketsGroup.MapPost("/{id:int}/acciones", async (
    int id, RegistrarAccionDto dto, HttpContext ctx, TicketService svc) => {
    var usuario = ctx.UsuarioActual();
    try {
        var accion = await svc.RegistrarAccionAsync(
            id, dto.Descripcion, usuario.Id, dto.Fecha);
        return Results.Created($" /tickets/{id}/acciones/{accion.Id}",
            new { accion.Id, accion.Descripcion, accion.Fecha, accion.Realizada });
    }
    catch (InvalidOperationException ex) {
        return Results.NotFound(new { error = ex.Message });
    }
});

ticketsGroup.MapPut("/{ticketId:int}/acciones/{accionId:int}/realizada", async (
    int ticketId, int accionId, TicketService svc) => {
    try {
        await svc.MarcarAccionRealizadaAsync(ticketId, accionId);
        return Results.NoContent();
    }
    catch (InvalidOperationException ex) {
        return Results.NotFound(new { error = ex.Message });
    }
});

app.Run();

// --- Mapeo entidad -> DTO de respuesta (evita ciclos de navegacion) ---
static object MapearTicket(Ticket t) => new {
    t.Id,
    t.Titulo,

```

```
csharp - continuacion
```

```
t.Descripcion,  
t.Estado,  
t.FechaCreacion,  
OriginadoPor = t.OriginadoPor?.Nombre,  
Responsable = t.Responsable?.Nombre,  
Acciones = t.Acciones.Select(a => new { a.Id, a.Descripcion, a.Fecha, a.Realizada })  
};
```

Tres decisiones de diseño que conviene entender:

El orden del pipeline importa. El logging va primero para registrar absolutamente todo, incluso requests que después fallan en autenticación. El Bearer va segundo, así cuando un endpoint corre, el usuario ya está identificado.

Servicios scoped dentro de un middleware. Los delegates de `app.Use` no son scoped, así que para usar `AuthService` (que sí lo es, porque arrastra el `DbContext`) lo pedimos con `context.RequestServices.GetRequiredService<>()` dentro del delegate. Eso respeta el ciclo de vida por request.

`.RequiresAuth()` **evita duplicación.** La autorización sigue siendo un filtro de endpoint, pero envuelta en un método de extensión reutilizable. Eso deja la protección con una sintaxis corta y consistente tanto en `POST /auth/logout` como en todo el grupo `/tickets`.

Enums como texto en JSON. `JsonStringEnumConverter` hace que el cliente pueda mandar y recibir "EnProceso", "Abierto" o "Cerrado" en vez de números como 1 o 2. Sin esa línea, el `PUT /tickets/{id}/estado` con un body como `{ "estado": "EnProceso" }` termina en 400 `BadRequest` porque el binder intenta leer el enum solo como entero.

`MapearTicket` **aplana la entidad.** Si devolvieras un `Ticket` directo, el serializador entraría en loop: `Ticket` → `Usuario` → `TicketsOriginados` → `Ticket`... El mapeo a un objeto plano corta el ciclo y, de paso, oculta el `PasswordHash`.

12. Crear la base con code-first

No usamos migraciones. La base se crea sola: el bloque `EnsureCreatedAsync()` que pusimos en `Program.cs` inspecciona las entidades al arrancar y genera el archivo `tickets.db` con todas las tablas si todavía no existe. No hay comandos extra que correr.

Simplemente levanta la API:

```
bash  
dotnet run
```

En el primer arranque pasan tres cosas, en orden: se crea `tickets.db` con sus tablas, y como estamos en entorno de desarrollo, el seeder precarga los 5 usuarios, los 4 tickets y sus acciones. La consola te va a mostrar la URL (algo como `http://localhost:5000`). Dejela corriendo.

Code-first con EnsureCreated vs migraciones: EnsureCreated es perfecto para aprender y prototipar porque es cero configuración: crea el esquema y listo. Su límite es que **no sabe evolucionar** una base ya creada. Si después cambiás una entidad, EnsureCreated no toca la tabla existente; tendrías que borrar `tickets.db` para que la regenere (perdiendo los datos). Para un sistema real que evoluciona, ahí sí se usan migraciones. Para esta práctica, code-first directo es lo más simple. **¿Querés empezar de cero?** Borrá el archivo `tickets.db` y volvé a correr `dotnet run`: se recrea y se vuelve a sembrar.

13. Probar el sistema

Ya tenés usuarios sembrados, así que podés saltar el registro e ir directo al login. Creá un archivo `prueba.http` (VS Code con la extensión REST Client lo ejecuta) y dejalo así. Si `dotnet run` te muestra otro puerto, cambiá `@base` antes de ejecutar los requests:

```
http
@base = http://localhost:5263

### Acceso sin token: debe devolver 401
GET {{base}}/tickets

### Login con usuario sembrado
# @name login
POST {{base}}/auth/login
Content-Type: application/json

{
  "email": "clara@cliente.com",
  "password": "clave5678"
}

### Token obtenido del login
@token = {{login.response.body.$.token}}

### Listar tickets
GET {{base}}/tickets
Authorization: Bearer {{token}}

### Listar tickets en proceso
GET {{base}}/tickets/estado/EnProceso
Authorization: Bearer {{token}}

### Ver ticket 2 con acciones sembradas
GET {{base}}/tickets/2
Authorization: Bearer {{token}}

### Crear un ticket nuevo
# @name crearTicket
POST {{base}}/tickets
Authorization: Bearer {{token}}
Content-Type: application/json

{
  "titulo": "Prueba desde REST Client",
  "descripcion": "Ticket creado desde prueba.http",
  "responsableId": 1
}

### Id del ticket recién creado
@ticketId = {{crearTicket.response.body.$.id}}

### Registrar una acción pasada
# @name accionPasada
POST {{base}}/tickets/{{ticketId}}/acciones
Authorization: Bearer {{token}}
```

http - continuacion

Content-Type: application/json

```
{
  "descripcion": "Accion pasada de prueba",
  "fecha": "2026-05-30T10:00:00"
}
```

```
### Registrar una accion futura
# @name accionFutura
POST {{base}}/tickets/{{ticketId}}/acciones
Authorization: Bearer {{token}}
Content-Type: application/json
```

```
{
  "descripcion": "Accion futura de prueba",
  "fecha": "2026-12-31T15:00:00"
}
```

```
### Id de la accion futura
@accionFuturaId = {{accionFutura.response.body.$.id}}
```

```
### Cambiar estado a EnProceso
PUT {{base}}/tickets/{{ticketId}}/estado
Authorization: Bearer {{token}}
Content-Type: application/json
```

```
{
  "estado": "EnProceso"
}
```

```
### Marcar la accion futura como realizada
PUT {{base}}/tickets/{{ticketId}}/acciones/{{accionFuturaId}}/realizada
Authorization: Bearer {{token}}
```

```
### Reasignar responsable del ticket creado
PUT {{base}}/tickets/{{ticketId}}/responsable
Authorization: Bearer {{token}}
Content-Type: application/json
```

```
{
  "responsableId": 2
}
```

```
### Ver el ticket creado actualizado
GET {{base}}/tickets/{{ticketId}}
Authorization: Bearer {{token}}
```

```
### Logout
POST {{base}}/auth/logout
Authorization: Bearer {{token}}
```

```
### Reintentar con el mismo token: debe devolver 401
GET {{base}}/tickets
Authorization: Bearer {{token}}
```

Qué observar: el primer GET /tickets sin token responde 401. Después del login, el archivo captura el token automáticamente y ya no hay que copiarlo a mano. El GET /tickets devuelve los 4 tickets sembrados, GET /tickets/estado/EnProceso devuelve 2, y el PUT /tickets/{{ticketId}}/estado acepta "EnProceso" como texto gracias a JsonStringEnumConverter. Al final, POST /auth/logout invalida el token y el último GET /tickets vuelve a dar 401. **Si un request sigue devolviendo un comportamiento viejo:** corta y relanza dotnet run. En cambios de Program.cs, es muy común dejar una instancia anterior del servidor corriendo y terminar probando contra una configuración desactualizada.

Resumen: cómo encajan las piezas

Capa	Archivos	Responsabilidad
Dominio	Domain/*.cs	Entidades y enums
Datos	Data/TicketContext.cs	Mapeo a SQLite, relaciones
Seed (dev)	Data/SeedData.cs	Datos de prueba, solo desarrollo
Repositorios	Repositories/*.cs	Acceso a datos tras interface
Seguridad	Security/PasswordHasher.cs	Hash + salt de contraseñas
Servicios	Services/*.cs	Lógica del sistema
API	Api/*.cs + Program.cs	HTTP, middleware, rutas

Árbol de decisión rápido para cuando extiendas el sistema:

- ¿Agregar un dato a una entidad? → tocás Domain/. Como usamos EnsureCreated, borra tickets.db para que se regenere con el campo nuevo (en un sistema real, ahí pasarías a migraciones).
- ¿Una nueva consulta a la base? → método nuevo en la interface del repo + su implementación.
- ¿Una regla de negocio? → va en el Service, nunca en el endpoint.
- ¿Un nuevo endpoint? → al grupo que corresponda (/auth público, /tickets protegido).

Eso es todo: el sistema completo, de la entidad al endpoint, listo para correr y para que lo amplíes.

CAPITULO 40

Middleware y Endpoint Filters desde los primeros principios

1. El punto de partida: una petición es un viaje

Cuando llega una petición HTTP, no salta directo a tu código y vuelve. Hace un **viaje** a través de una serie de etapas, en orden, antes de llegar a la lógica que la atiende, y otro viaje de vuelta con la respuesta. Cada etapa puede mirar la petición, modificarla, decidir frenarla, o dejarla pasar a la siguiente.

ASP.NET Core organiza ese viaje como un **pipeline**: una tubería compuesta por piezas encadenadas, donde la salida de una es la entrada de la siguiente. La petición entra por un extremo, atraviesa las piezas una por una, llega al destino (tu código), y la respuesta hace el camino inverso atravesando las mismas piezas.

Todo lo que sigue son dos formas de meter piezas propias en ese viaje. El **middleware** son piezas en el tramo *general* del viaje, antes de saber siquiera qué código va a atender la petición. Los **endpoint filters** son piezas en el tramo *final*, ya envolviendo el código concreto que la atiende. El primer principio es ese: **son dos puntos distintos del mismo viaje, y por eso ven cosas distintas**.

2. Qué es el middleware

Un **middleware** es un componente que se sienta en el pipeline de la aplicación y procesa *todas* las peticiones que pasan por él. Es lo primero que toca una petición que entra, y lo último que toca una respuesta que sale.

La palabra describe literalmente su posición: está en el **medio**, entre el servidor que recibe el byte crudo del HTTP y el código de negocio que produce la respuesta. Ejemplos típicos de cosas que se resuelven con middleware: redirigir HTTP a HTTPS, servir archivos estáticos (CSS, imágenes), autenticación, registrar cada petición en un log, capturar excepciones no manejadas.

En `Program.cs`, el pipeline se arma con una serie de llamadas `app.Use...`, y **el orden en que las escribís es el orden del viaje**:

```
csharp
var app = builder.Build();

app.UseHttpsRedirection(); // 1ª pieza
app.UseStaticFiles();      // 2ª pieza
app.UseRouting();          // 3ª pieza
app.UseAuthentication();   // 4ª pieza
app.UseAuthorization();     // 5ª pieza

app.MapRazorPages();       // destino final

app.Run();
```

Cada Use . . . agrega una pieza a la tubería. La petición las atraviesa de arriba hacia abajo; la respuesta vuelve de abajo hacia arriba.

*El principio: **el middleware no sabe ni le importa qué código va a atender la petición**. Opera al nivel más bajo y general, sobre la petición cruda. Por eso sirve para cosas que valen para toda la app por igual, sin importar si después la atiende una página Razor, una API o un archivo estático.*

3. La forma de un middleware: el patrón de la cebolla

Un middleware no es solo "código que corre antes". Es código que **envuelve** al resto del pipeline, con una parte que corre a la ida y otra a la vuelta. La forma canónica es esta:

```
csharp
app.Use(async (context, next) => {
    // (A) Código de IDA: corre ANTES de las piezas siguientes
    Console.WriteLine($"Entrando: {context.Request.Path}");

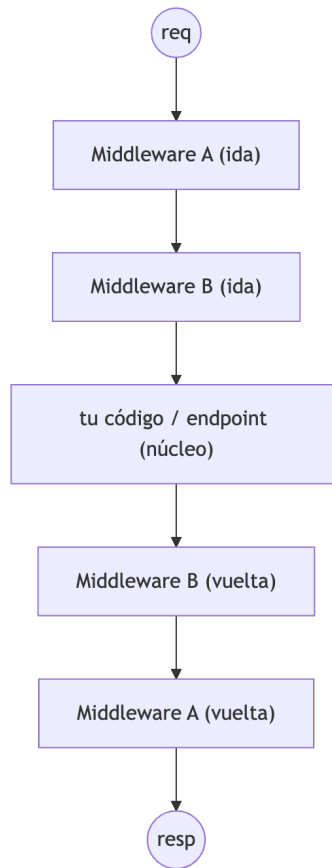
    await next(context); // ← le pasa el control a la siguiente pieza

    // (B) Código de VUELTA: corre DESPUÉS de que las siguientes terminaron
    Console.WriteLine($"Saliendo: {context.Response.StatusCode}");
});
```

Hay dos elementos clave:

- `context` es el `HttpContext`: el objeto que representa *toda* la petición y la respuesta. Por ahí accedés a la URL, los headers, el cuerpo, el usuario autenticado, y ahí escribís la respuesta. Es el "expediente" que viaja por todo el pipeline.
- `next` es el resto del pipeline empaquetado en una función. Llamar a `await next(context)` significa "ahora le toca a la pieza siguiente". Todo lo que escribís *antes* de esa línea corre a la ida; todo lo que escribís *después* corre a la vuelta, cuando las piezas internas ya terminaron.

A esto se lo llama el **patrón de la cebolla**: cada middleware es una capa que envuelve a la de adentro. La petición atraviesa las capas hacia el centro (la parte de ida de cada una), llega al núcleo, y la respuesta sale atravesando las capas en reversa (la parte de vuelta de cada una).



Diagrama

*El principio: **un middleware ve la petición dos veces** —una a la ida, otra a la vuelta— porque rodea a todo lo que está más adentro. Por eso puede, por ejemplo, arrancar un cronómetro a la ida y detenerlo a la vuelta para medir cuánto tardó toda la petición.*

4. El orden importa (y por qué)

Como el pipeline es una secuencia, **el orden de los `app.Use`...** **cambia el comportamiento**, no es decorativo. Y los errores de orden son una fuente clásica de bugs sutiles.

El ejemplo más claro es autenticación y autorización:

```
csharp
app.UseAuthentication(); // PRIMERO: ¿quién sos? (identifica al usuario)
app.UseAuthorization();  // DESPUÉS: ¿podés hacer esto? (chequea permisos)
```

Si las invirtieras, la autorización correría antes de que nadie haya identificado al usuario, así que siempre vería un usuario anónimo y rechazaría todo. El orden codifica una dependencia: no podés preguntar "¿podés?" antes de saber "¿quién sos?".

Otro caso: `UseStaticFiles()` suele ir temprano. Si una petición es para `/css/site.css`, ese middleware la atiende y **cortocircuita** ahí mismo (lo vemos en la sección que sigue), sin molestar al resto del

pipeline. Ponerlo temprano evita trabajo innecesario para los archivos estáticos.

El principio: el orden del pipeline expresa dependencias y prioridades. Pensar "¿qué necesita estar resuelto antes de que esta pieza corra?" te dice dónde ubicarla. No es un detalle: es parte de la lógica.

5. Cortocircuitar: cuando un middleware no llama al siguiente

Hasta acá, cada middleware llamaba a `await next(context)` para pasar el control. Pero un middleware **no está obligado** a hacerlo. Si decide no llamar a `next`, el viaje se detiene ahí: las piezas de adentro nunca corren, y la respuesta empieza a volver desde ese punto. Eso es **cortocircuitar** el pipeline.

```
csharp
app.Use(async (context, next) => {
    if (EstaEnMantenimiento()) {
        context.Response.StatusCode = 503;
        await context.Response.WriteAsync("En mantenimiento, volvé más tarde.");
        return; // ← NO llama a next: corta el pipeline acá
    }

    await next(context); // si no, sigue normalmente
});
```

Acá, si la app está en mantenimiento, el middleware responde él mismo y nada de lo que está más adentro (ni el ruteo, ni tus páginas) llega a ejecutarse. Cortocircuitar es la herramienta para frenar peticiones que no deben continuar: un usuario no autenticado, un archivo estático ya servido, una petición bloqueada por un límite de tasa.

El principio: next es una decisión, no un trámite. Llamarlo deja pasar; no llamarlo corta. Esa simple elección —seguir o frenar— es lo que le da al middleware el poder de actuar como guardián de todo lo que viene después.

6. Tres formas de escribir un middleware

El delegado inline (`app.Use(async (context, next) => ...)`) es la forma más directa, pero hay tres maneras de escribir middleware, de menor a mayor formalidad:

1. Inline con `app.Use` — para lógica chica y puntual, como vimos arriba.

2. Por convención (una clase) — para middleware reutilizable. Es una clase normal (no implementa ninguna interfaz) que sigue una convención de nombres: recibe el `next` en el constructor y expone un método `InvokeAsync`.

```
csharp
public class LoggingMiddleware {
    private readonly RequestDelegate next;

    public LoggingMiddleware(RequestDelegate next) {
        this.next = next;
    }
}
```

csharp - continuación

```
public async Task InvokeAsync(HttpContext context) {  
    Console.WriteLine($"→ {context.Request.Method} {context.Request.Path}");  
    await this.next(context);  
    Console.WriteLine($"← {context.Response.StatusCode}");  
}
```

Se enchufa con `app.UseMiddleware<LoggingMiddleware>()`;

3. Basado en fábrica (IMiddleware) — una clase que implementa la interfaz `IMiddleware`. Conviene cuando el middleware necesita dependencias inyectadas *por petición* (scoped), porque se resuelve desde el contenedor de DI en cada request en vez de una sola vez al arrancar.

*El principio: **las tres hacen lo mismo conceptualmente** —envuelven el pipeline con la lógica de ida/vuelta de la sección 3—. La elección es de empaquetado: inline para lo puntual, clase por convención para lo reutilizable, `IMiddleware` cuando entran en juego dependencias scoped.*

7. El problema que el middleware no resuelve bien

El middleware es potente justamente porque es general: corre antes de saber qué va a atender la petición. Pero esa misma generalidad es su límite.

Imaginá que querés validar los datos de entrada de *una* operación concreta —digamos, que el `id` que llega sea mayor que cero antes de ejecutar el handler que crea un producto. Con middleware esto es incómodo:

- El middleware corre **antes del ruteo** (o al menos antes de que se resuelvan los argumentos del handler), así que **no conoce los parámetros ya convertidos** que va a recibir tu código. Solo ve la petición cruda: tendría que hurgar a mano en la query string o el cuerpo, reparseando lo que el framework va a parsear igual más adelante.
- El middleware corre para **todas** las peticiones. Aplicar una validación que solo tiene sentido en un endpoint significaría meter ahí dentro condiciones del tipo "si la ruta es tal y el método es cual...", lo que ensucia y no escala.
- El middleware trabaja con el `HttpContext` y la respuesta cruda, **no con el valor de retorno** de tu handler. Si quisieras transformar el resultado de un endpoint, tendrías que interceptar el cuerpo de la respuesta ya serializado, que es engorroso.

En resumen: el middleware está demasiado *afuera* y demasiado *temprano* para tareas que dependen de **qué endpoint específico** atiende la petición y de **sus argumentos ya procesados**. Para eso existe la otra herramienta.

8. Qué es un endpoint filter

Un **endpoint filter** es código que se ejecuta *envolviendo a un endpoint concreto* (o a un grupo de endpoints), no a toda la aplicación. Es la pieza que faltaba: corre **después** de que el ruteo ya decidió qué código atiende la petición y de que los argumentos ya fueron convertidos, pero **antes** de que el handler corra (y también a la vuelta, después de que corre).

La diferencia de posición lo cambia todo. Mientras el middleware vive en el pipeline general y ve peticiones crudas, el endpoint filter vive pegado al endpoint y por eso tiene acceso a:

- **Los argumentos ya convertidos** que va a recibir el handler (el `int id` ya parseado, el objeto ya bindeado), no la petición cruda.
- **El valor de retorno** del handler, que puede inspeccionar o transformar a la vuelta.
- **Solo el endpoint (o grupo) al que lo asociaste**, no toda la app.

Los endpoint filters son la herramienta recomendada en .NET 10 para las llamadas *cross-cutting concerns* (validación, logging, autorización fina) **a nivel de ruta**: cosas que querés aplicar a un endpoint o a un conjunto de ellos, aprovechando que ya sabés qué endpoint es y con qué argumentos.

*El principio: **el endpoint filter sabe lo que el middleware ignora**. Está más adentro y más tarde en el viaje, así que conoce el destino concreto y sus datos ya procesados. Esa cercanía es exactamente su razón de ser.*

9. La forma de un endpoint filter

Un endpoint filter implementa la interfaz `IEndpointFilter`, que define un único método: `InvokeAsync`. La forma es deliberadamente parecida a la del middleware —tiene la misma estructura de ida/vuelta con un `next`— pero trabaja con otros objetos:

```
csharp
public class ValidacionIdFilter : IEndpointFilter {
    public async ValueTask<object?> InvokeAsync(
        EndpointFilterInvocationContext context,
        EndpointFilterDelegate next) {

        // (A) IDA: inspeccionar los ARGUMENTOS ya convertidos del handler
        int id = context.GetArgument<int>(0); // el primer argumento del handler
        if (id <= 0) {
            return Results.BadRequest("El id debe ser mayor que cero.");
            // ← cortocircueta: el handler nunca corre
        }

        // pasa el control al handler (o al siguiente filtro)
        object? resultado = await next(context);

        // (B) VUELTA: puede inspeccionar o transformar el RESULTADO del handler
        return resultado;
    }
}
```

Compará pieza por pieza con el middleware:

Middleware	Endpoint filter
recibe HttpContext context	recibe EndpointFilterInvocationContext
next(context) → resto del pipeline	next(context) → handler o siguiente filtro
trabaja con la petición/respuesta cruda	trabaja con los argumentos y el valor de retorno
devuelve Task (nada)	devuelve ValueTask<object?> (el resultado)

Las dos diferencias que más importan:

- `context.GetArgument<T>(indice)` te da los argumentos del handler ya convertidos a su tipo. No reparseás nada: el framework ya hizo el binding y vos lo recibís listo. Esto es justo lo que el middleware no podía hacer cómodo.
- **El filtro devuelve el resultado** (object?). A la ida podés devolver algo *en lugar de* llamar al handler (cortocircuito); a la vuelta podés tomar lo que devolvió el handler y transformarlo antes de que se convierta en respuesta.

Se asocia a un endpoint o a un grupo con `AddEndpointFilter`:

```
csharp
// a un endpoint puntual
app.MapPost("/productos/{id}", CrearProducto)
    .AddEndpointFilter<ValidacionIdFilter>();

// a todo un grupo de endpoints
var grupo = app.MapGroup("/productos");
grupo.AddEndpointFilter<ValidacionIdFilter>(); // aplica a todos los del grupo
```

*El principio: **mismo patrón de cebolla, distinto material**. El filtro envuelve al handler igual que el middleware envuelve al pipeline, pero en vez de manipular bytes crudos manipula los argumentos tipados de entrada y el valor de retorno de salida. Por eso es la herramienta natural para validar entradas y dar forma a salidas.*

10. Encadenar filtros y cortocircuitar

Igual que con el middleware, podés poner **varios filtros sobre un mismo endpoint**, y forman su propia cadena. Se ejecutan en el orden en que los agregás (a la ida) y en orden inverso (a la vuelta), exactamente como las capas de la cebolla:

```
csharp
app.MapPost("/productos/{id}", CrearProducto)
    .AddEndpointFilter<LoggingFilter>() // capa externa
    .AddEndpointFilter<ValidacionIdFilter>(); // capa interna
```

A la ida corre primero `LoggingFilter`, después `ValidacionIdFilter`, después el handler; a la vuelta, al revés.

Y como el middleware, **cualquier filtro puede cortocircuitar**: si devuelve un resultado sin llamar a `next(context)`, ni los filtros internos ni el handler llegan a correr. Eso es justo lo que hace el ejemplo de validación de la sección anterior: con un `id` inválido, devuelve `BadRequest` y el handler nunca se ejecuta.

*El principio: **la decisión de seguir o frenar vuelve a estar en** next. Toda la familia de mecanismos de pipeline de ASP.NET Core comparte esta mecánica: llamar a next deja pasar al siguiente; devolver sin llamarlo, corta. Aprendido una vez, sirve para los dos.*

11. Middleware vs. endpoint filter: la diferencia esencial

Ahora que viste los dos, la distinción se puede resumir en una idea: **a qué altura del viaje viven, y por eso qué pueden ver.**

Aspecto	Middleware	Endpoint filter
Alcance	Toda la app (todas las peticiones)	Un endpoint o un grupo
Cuándo corre	Temprano, antes de saber el endpoint	Tarde, ya conocido el endpoint y sus argumentos
Qué ve	Petición/respuesta crudas (HttpContext)	Argumentos ya convertidos y valor de retorno
Sirve para	Concerns globales: HTTPS, archivos estáticos, auth general, logging, manejo de errores	Concerns por ruta: validar entradas, transformar resultados, lógica específica del endpoint
Se enchufa con	<code>app.Use...</code> en <code>Program.cs</code>	<code>.AddEndpointFilter<T>()</code> sobre el endpoint/grupo
Funciona con	Cualquier estilo (Razor Pages, MVC, APIs, Blazor)	Endpoints (Minimal APIs, y también MVC/Razor Pages)

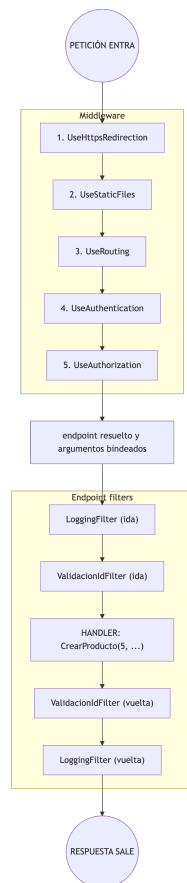
La forma más simple de elegir: **si lo que querés hacer vale para toda la aplicación y no necesita saber qué endpoint atiende ni sus argumentos, es middleware. Si depende del endpoint concreto o de sus parámetros ya procesados, es un endpoint filter.**

Una imagen que ayuda: el middleware es el **control de seguridad del aeropuerto** —todos pasan por él, sin importar el vuelo—; el endpoint filter es el **chequeo en la puerta de embarque** de tu vuelo específico, que ya sabe a dónde vas y revisa tu tarjeta concreta.

*El principio: **no compiten, se complementan**. Casi toda app real usa los dos: middleware para lo transversal y general, filtros para lo específico de cada ruta. No es "cuál es mejor" sino "cuál es la altura correcta del viaje para este trabajo".*

12. El recorrido completo de una petición

Cerremos atando todo en un viaje de punta a punta. Llega `POST /productos/5` con un cuerpo JSON:



Diagrama

Leído así, el modelo es uno solo y consistente: **un viaje con capas anidadas, donde cada capa corre a la ida y a la vuelta, y donde llamar a next deja pasar y no llamarlo corta.** El middleware son las capas externas y generales; los endpoint filters, las capas internas y específicas de cada endpoint. Entendido el patrón de la cebolla una vez, las dos herramientas se vuelven la misma idea aplicada a dos alturas distintas del recorrido.

Resumen de los principios

Concepto	Regla en una frase
Pipeline	La petición hace un viaje por capas anidadas: ida hacia el núcleo, vuelta hacia afuera.
Middleware	Capa general que ve la petición cruda; corre para toda la app, antes de saber el endpoint.
Patrón de la cebolla	El código antes de <code>next</code> corre a la ida; el código después, a la vuelta.
Orden	La secuencia de <code>app</code> . <code>Use . . .</code> expresa dependencias y prioridades, no es decorativa.
Cortocircuito	No llamar a <code>next</code> frena el viaje y empieza el regreso desde esa capa.
Endpoint filter	Capa pegada a un endpoint que ve los argumentos convertidos y el valor de retorno.
Elección	Global y sin conocer el endpoint → middleware. Por ruta o según argumentos → endpoint filter.

Programación III — Tecnicatura Universitaria en Programación — UTN, Facultad Regional Tucumán.

CAPITULO 41

Razor Pages

1. El punto de partida: qué hace un servidor web

Antes de Razor, antes de C#, está el hecho crudo: un servidor web recibe **peticiones HTTP** y devuelve **respuestas HTTP**. Una petición es básicamente tres cosas: un **verbo** (GET, POST, etc.), una **URL** (/contactos?id=5) y opcionalmente un **cuerpo** (los datos de un formulario). La respuesta es un texto (casi siempre HTML) más algunos metadatos llamados *headers*.

Todo lo que viene después es maquinaria para responder esa pregunta: "llegó esta petición, ¿qué texto le devuelvo?". Razor Pages es una forma de organizar esa maquinaria donde la unidad de trabajo es **la página**.

2. Qué es una página Razor

Una página Razor es un archivo `.cshtml` que mezcla HTML con C#. El `.cshtml` se compone de dos lenguajes entrelazados por el carácter `@`: todo lo que no está marcado con `@` es HTML literal que sale tal cual; lo que empieza con `@` es código C# cuyo *resultado* se inserta en el HTML.

```
razor
@page

@{
    string nombre = "Ana";
    int hora = DateTime.Now.Hour;
}

<h1>Hola, @nombre</h1>
@if (hora < 12) {
    <p>Buenos días</p>
} else {
    <p>Buenas tardes</p>
}
```

Cuando alguien pide esta página, el servidor *ejecuta* ese código y produce HTML plano:

```
html
<h1>Hola, Ana</h1>
<p>Buenas tardes</p>
```

El navegador nunca ve C# ni `@if`: ve el resultado. Esto es lo que se llama **renderizar del lado del servidor**. El primer principio fundamental es ese: **el HTML se arma en el servidor ejecutando código, y al navegador llega solo el texto final**.

La diferencia entre un `.cshtml` cualquiera y una página Razor "de verdad" es la primera línea: `@page`. Esa directiva es la que le dice al framework "este archivo es una página accesible por URL", no un simple fragmento. Sin `@page`, el archivo existe pero no responde a ninguna petición por sí mismo.

3. El ruteo: de una URL a un archivo

El **ruteo** (routing) es el mecanismo que decide, dada una URL, qué página la atiende. En Razor Pages la regla base es deliciosamente simple: **la ruta es la ubicación del archivo dentro de la carpeta Pages/**.

texto

```
Pages/Index.cshtml      → /
Pages/Contactos.cshtml  → /Contactos
Pages/Cientes/Lista.cshtml → /Cientes/Lista
```

Index es un nombre especial: corresponde a la raíz de su carpeta (igual que `index.html` en un sitio estático). Por eso `Pages/Index.cshtml` responde en `/` y no en `/Index`.

No hay un archivo de configuración que diga "la URL `/Contactos` va a tal página". La estructura de carpetas es la tabla de rutas. Esto es lo que se llama **routing por convención**: el sistema deduce la regla de la ubicación, en vez de que vos la declares.

Hay una pieza en `Program.cs` que enciende todo esto: `app.MapRazorPages()`. Esa línea le dice a la app "examiná la carpeta `Pages/` y armá las rutas". Sin ella, ningún `.cshtml` responde.

Handlers: varias acciones dentro de una misma página

Una URL puede pedirse con distintos verbos, y una misma página puede querer responder a varias acciones. Acá entra la segunda regla del ruteo: dentro de una página, los métodos de C# que responden a peticiones se llaman **handlers**, y su nombre se arma con una fórmula fija:

texto

On + Verbo + NombreDelHandler

Petición HTTP	Método que se ejecuta
GET <code>/Contactos</code>	<code>OnGet()</code>
GET <code>/Contactos?handler=Ver&id=5</code>	<code>OnGetVer(int id)</code>
POST <code>/Contactos?handler=Guardar</code>	<code>OnPostGuardar(...)</code>

Tres observaciones que aclaran de dónde sale cada parte:

- El **verbo** (Get/Post) viene de cómo se hizo la petición: un enlace o teclear la URL es GET; el envío de un formulario es POST.
- El **nombre del handler** viene del parámetro especial `handler` en la query string. Sin `?handler=...`, se usa el handler "por defecto": `OnGet` para GET, `OnPost` para POST.
- Si no escribís ningún handler, igual existe un `OnGet` implícito que simplemente renderiza la página. Por eso una página con solo `@page` y HTML ya funciona: tiene un `OnGet` invisible que no hace nada más que mostrar el HTML.

Entonces el ruteo opera en dos niveles: **qué archivo** (por la URL/carpeta) y **qué método dentro de ese archivo** (por el verbo + handler).

4. Los parámetros y el binding: de texto a objetos C#

Acá viene uno de los conceptos más importantes y más fáciles de subestimar. Una petición HTTP es **puro texto**. Cuando llega GET /Contactos?id=5&buscar=ana, el id y el buscar son cadenas de caracteres dentro de la URL. Pero tu handler de C# quiere trabajar con un `int id` y un `string buscar`, no con texto crudo.

El **model binding** es el proceso que tiende ese puente: mira el nombre de cada parámetro de tu método y busca un valor que se llame igual en la petición, lo convierte al tipo que pediste, y te lo entrega listo.

```
csharp
public IActionResult OnGetVer(int id, string buscar) {
    // Razor vio "id" en la URL (id=5), lo convirtió a int → id == 5
    // Razor vio "buscar" en la URL (buscar=ana) → buscar == "ana"
}
```

El binding une por **nombre**, no por posición ni por orden. Si tu parámetro se llama `id`, busca `id`; si se llama `contactoId`, busca `contactoId`. De dónde lo busca:

- **Query string:** `?id=5` (lo que va después del `?` en la URL).
- **Datos de ruta:** si la ruta tiene segmentos con nombre.
- **Cuerpo del formulario:** los campos de un `<form>` que se envió por POST.

El binding también funciona con **objetos**, no solo con valores sueltos. Supongamos un formulario con varios campos:

```
html
<input name="Nombre" value="Ana" />
<input name="Apellido" value="Gómez" />
```

Y un handler que recibe un objeto completo:

```
csharp
public IActionResult OnPostGuardar(ContactoInput input) { ... }
```

¿Cómo pasa de varios `<input name="...">` a un solo objeto? Cuando el parámetro es una clase o record, Razor toma cada propiedad (Nombre, Apellido, ...) y busca un campo del formulario con ese mismo nombre para llenarla. Por eso los `name` de los `<input>` tienen que coincidir con los nombres de las propiedades del objeto. El binding arma el objeto pieza por pieza.

*El principio acá: **el binding es el traductor entre el mundo de texto del HTTP y el mundo tipado de C#**. Y la regla que lo gobierna es siempre "unir por nombre". Si algo "no llega" a tu handler, el 90% de las veces es porque el nombre del campo HTML no coincide con el nombre del parámetro o de la propiedad.*

5. Los modelos: las formas que toman tus datos

"Modelo" es una palabra sobrecargada, así que vale la pena separarla en los roles concretos que cumple. En el fondo, un modelo es solo una clase que representa datos; lo que cambia es *qué datos y para qué*.

Aparecen típicamente tres tipos distintos, y la distinción no es burocracia: cada uno protege una frontera.

El modelo de dominio / entidad representa cómo vive una cosa *adentro* de tu aplicación. Suele ser la clase que un ORM (como EF Core) mapea a una tabla de la base. Tiene todo lo que la cosa "es", incluido el Id que asigna la base.

```
csharp
public class Contacto {
    public int Id { get; set; }
    public string Nombre { get; set; } = "";
    public string Apellido { get; set; } = "";
    // ...
}
```

El DTO de entrada (*Data Transfer Object*) representa qué datos *aceptás de afuera*, desde un formulario. Es la frontera entre el mundo externo (que no controlás) y tu dominio (que sí). Se lo separa de la entidad por seguridad y claridad: si bindearas el formulario directo contra la entidad, alguien podría mandar campos que no querés que toque. El DTO declara explícitamente "estos son los únicos campos válidos como entrada".

```
csharp
public record ContactoInput(int Id, string Nombre, string Apellido, string Telefono);
```

El view model representa qué necesita *una vista* para dibujarse, que a veces es más que una sola entidad. Por ejemplo, una lista puede necesitar los elementos y cuál está seleccionado para resaltarlos. Eso no es un dato del dominio (un contacto no "sabe" si está seleccionado); es un dato de la pantalla. El view model los junta.

```
csharp
public record ListViewModel(List<Contacto> Contactos, int? SeleccionadoId);
```

El principio: cada modelo protege una frontera distinta. El dominio es lo que vive adentro; el DTO controla qué entra; el view model arma lo que la pantalla necesita ver. En apps chicas a veces se colapsan en uno solo, pero entender por qué existen por separado te evita acoplar cosas que después cuesta desacoplar.

Un detalle de Razor sobre la palabra "modelo": una página o un partial pueden declarar `@model TipoX` arriba. Eso le dice a Razor "el objeto que me pasan para renderizar es de tipo TipoX", y dentro del archivo lo accedés con `Model`. Es el modelo *de esa vista*, no necesariamente una entidad de base de datos: puede ser una entidad, un DTO o un view model, según qué necesite la vista.

6. Los formularios: el ciclo completo de captura de datos

Un formulario es la forma en que el usuario *manda* datos al servidor. Ya vimos las dos mitades sueltas —el binding convierte los campos en un objeto (sección 4), y el DTO define qué campos aceptamos (sección 5)—. Acá las juntamos en el ciclo completo, que tiene una estructura muy reconocible una vez que la ves entera: **mostrar el formulario** → **recibir y validar** → **guardar o volver a mostrar con errores** → **redirigir**.

El formulario es un viaje de ida y vuelta

Lo primero que conviene entender es que un formulario involucra **dos peticiones distintas**, no una:

1. Un **GET** que muestra el formulario vacío (o precargado, si es una edición). El usuario lo ve y lo completa.
2. Un **POST** que envía los datos cuando el usuario hace clic en "Guardar". Acá el servidor los procesa.

Esas dos peticiones caen en dos handlers distintos de la misma página: `OnGet` arma el formulario, `OnPost` lo recibe. Tenelo presente, porque toda la mecánica que sigue gira alrededor de esos dos momentos.

[BindProperty]: el modelo que es de la página

En la sección 4 el handler recibía el objeto como *argumento* (`OnPostGuardar(ContactoInput input)`). Funciona, pero Razor Pages ofrece una forma más cómoda y más usada: declarar una **propiedad** de la página marcada con `[BindProperty]`.

```
csharp
public class CrearModel : PageModel {
    [BindProperty]
    public ContactoInput Entrada { get; set; } = new();

    public void OnGet() {
        // El formulario se muestra vacío (Entrada recién creado).
    }

    public IActionResult OnPost() {
        // Al llegar el POST, Razor YA llenó Entrada con los campos del formulario.
        // ...
    }
}
```

La diferencia con recibir el objeto por argumento es de *alcance*: una propiedad `[BindProperty]` vive en la página y está disponible tanto al renderizar (el GET la usa para dibujar) como al recibir (el POST la encuentra llena). Eso es justo lo que necesitás cuando un formulario falla la validación y querés volver a mostrarlo con lo que el usuario ya había escrito: los datos siguen ahí, en la propiedad.

Por defecto, `[BindProperty]` se llena **solo en POST** (no en GET), que es lo que casi siempre querés. Si necesitás que también se complete en GET —por ejemplo, un filtro que viaja en la URL—, se usa `[BindProperty(SupportsGet = true)]`.

Los tag helpers: HTML que conoce tu modelo

En la sección 4 escribíamos los `<input>` a mano, cuidando que el `name` coincidiera con la propiedad. Razor ofrece **tag helpers**: atributos especiales que generan ese HTML a partir del modelo, sin que tengas que repetir nombres ni valores.

razor

```

@page
@model CrearModel

<form method="post">
  <label asp-for="Entrada.Nombre">Nombre</label>
  <input asp-for="Entrada.Nombre" />
  <span asp-validation-for="Entrada.Nombre"></span>

  <label asp-for="Entrada.Email">Email</label>
  <input asp-for="Entrada.Email" />
  <span asp-validation-for="Entrada.Email"></span>

  <button type="submit">Guardar</button>
</form>

```

El tag helper `asp-for="Entrada.Nombre"` hace tres cosas solo: pone el name correcto (`Entrada.Nombre`, para que el binding lo reconozca al volver), pone el value actual (para que en una edición el campo aparezca precargado), y deduce atributos de tipo a partir de la propiedad (un `Email` genera `type="email"`, un campo requerido agrega `required`). El `asp-validation-for` es el lugar donde aparecerá el mensaje de error de ese campo, si lo hay.

El principio: el tag helper ata el HTML al modelo en un solo lugar. En vez de coordinar a mano el name del input con la propiedad de C# (y arriesgarte a que dejen de coincidir), declarás la propiedad una vez y el helper genera el campo consistente con ella. El binding de vuelta funciona porque los nombres los puso el mismo sistema que los va a leer.

La validación: ModelState y las anotaciones

Antes de guardar, casi siempre hay que validar: que el nombre no esté vacío, que el email tenga forma de email. Razor integra esto con **anotaciones de datos**: atributos que ponés sobre las propiedades del DTO declarando las reglas.

csharp

```

public class ContactoInput {
    public int Id { get; set; }

    [Required(ErrorMessage = "El nombre es obligatorio.")]
    public string Nombre { get; set; } = "";

    [Required]
    public string Apellido { get; set; } = "";

    [EmailAddress(ErrorMessage = "El email no tiene un formato válido.")]
    public string Email { get; set; } = "";
}

```

Cuando llega el POST, Razor revisa esas reglas **automáticamente** y deja el resultado en una propiedad llamada `ModelState`. Tu handler solo consulta si todo dio bien:

csharp

```

public IActionResult OnPost() {
    if (!ModelState.IsValid) {
        // Algo no validó: volvemos a mostrar el MISMO formulario.
        // Entrada conserva lo que el usuario escribió, y los
        // asp-validation-for muestran los mensajes de error.
        return Page();
    }
}

```

csharp - continuación

```

repositorio.Guardar(Entrada);           // todo bien: persistimos
return RedirectToPage("Index");         // y redirigimos (ver más abajo)
}

```

El patrón es siempre el mismo y vale la pena memorizarlo: **si ModelState no es válido, devolvé Page()** para re-mostrar el formulario con errores; si es válido, **procesá y redirigí**. Page() vuelve a renderizar la página actual —con la propiedad [BindProperty] todavía llena, así el usuario no pierde lo que escribió, y con los mensajes de error visibles en los asp-validation-for.

El principio: la validación vive en el modelo, no en el handler. Las reglas se declaran como anotaciones sobre el DTO (un solo lugar, junto a los datos que describen), y Razor las aplica solo. El handler no chequea campo por campo: solo pregunta ModelState.IsValid y decide entre re-mostrar o seguir.

POST-Redirect-GET: por qué un POST exitoso siempre redirige

Mirá que en el ejemplo, tras guardar con éxito, el handler hace RedirectToPage(...) en vez de devolver HTML directamente. Esto es deliberado y tiene nombre: el patrón **POST-Redirect-GET** (PRG).

El motivo es un problema concreto del navegador. Si un POST respondiera directamente con la página de "guardado con éxito", esa página quedaría asociada a un POST. Si el usuario después aprieta F5 (refrescar), el navegador **reenvía el POST** —y guardás el contacto otra vez—. Quizás viste alguna vez el cartel "¿Reenviar formulario?": es exactamente esto.

La solución del PRG: el POST no devuelve HTML, sino una **redirección** a una URL. El navegador, al recibir el redirect, hace un **GET** a esa URL. Ahora la página que el usuario ve está asociada a un GET inofensivo: si refresca, repite el GET (que no cambia nada), no el POST.

texto

```

GET /crear           → muestra el formulario vacío
    ■ el usuario completa y envía
POST /crear          → OnPost valida y guarda
    ■ return RedirectToPage("Index") ← NO devuelve HTML, redirige
GET /                → el navegador pide la página de destino
    ■ el usuario ve el resultado; si refresca, repite este GET inofensivo

```

El principio: un POST que modifica datos siempre termina en redirección, no en HTML. Así el resultado queda bajo una URL "de solo lectura" que se puede refrescar sin repetir el cambio. La regla mental: GET muestra, POST modifica y redirige a un GET.

El ciclo completo de un formulario

Juntando todo, un formulario de Razor Pages recorre siempre estos pasos:

texto

1. GET → OnGet arma el formulario ([BindProperty] vacío o precargado)
los tag helpers (asp-for) generan los campos atados al modelo
2. El usuario completa y envía
3. POST → Razor llena la propiedad [BindProperty] con los campos (binding)
y aplica las anotaciones de validación → ModelState
4. ¿ModelState.IsValid?

texto - continuación

```
NO → return Page()           (re-muestra el form con datos y errores)
SÍ → procesar + RedirectToPage(...) (PRG: redirige a un GET)
```

Cada pieza que ya conocías encuentra su lugar acá: el binding (paso 3), el DTO y sus anotaciones (paso 3-4), los modelos (la propiedad que viaja), y los handlers OnGet/OnPost (los dos extremos del viaje). El formulario no es un tema aparte: es la coreografía de todo lo anterior alrededor de capturar datos del usuario.

7. El layout: el marco compartido

Cada página produce un fragmento de HTML (su <h1>, sus formularios). Pero una página HTML completa necesita siempre lo mismo alrededor: <!DOCTYPE html>, el <head> con los estilos, las etiquetas <body>. Repetir eso en cada página sería absurdo.

El **layout** es una plantilla que contiene todo ese envoltorio común una sola vez, con un hueco donde se inserta el contenido de cada página:

razor

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>@ViewData["Title"]</title>
  <link rel="stylesheet" href="/css/site.css" />
</head>
<body>
  <main>
    @RenderBody()
  </main>
</body>
</html>
```

@RenderBody() es el hueco: ahí Razor mete el HTML que generó la página individual. Cuando una página dice `Layout = "_Layout";`, le está pidiendo a Razor "envolveme con esa plantilla". El flujo mental es: la página produce su fragmento, el layout lo recibe en @RenderBody(), y juntos forman el documento HTML completo que se envía.

@ViewData["Title"] es un pequeño canal de comunicación de la página hacia el layout: la página escribe `ViewData["Title"] = "Mi título"` y el layout lo lee para poner el título. Es un diccionario compartido para esos datos sueltos que la página quiere pasarle al marco.

*El principio: **el layout invierte la relación**. Uno tiende a pensar "la página contiene el HTML"; en realidad el layout contiene el documento y la página le aporta el relleno del medio. Por eso los estilos y scripts comunes viven en el layout: están una vez y aplican a todas las páginas.*

8. Los partials: fragmentos reutilizables

Si el layout es el marco grande compartido entre páginas, los **partials** son piezas chicas reutilizables *dentro* de las páginas. Un partial es un `.cshtml` (por convención con nombre que empieza en `_`, como `_Detalle.cshtml`) que renderiza un trozo de HTML y normalmente recibe un modelo.

```
razor
@* _Detalle.cshtml *@
@model Contacto

<h2>@Model.Nombre @Model.Apellido</h2>
<p>@Model.Telefono</p>
```

Lo usás de dos formas, y la distinción es central:

Desde otra vista, con la etiqueta `<partial>`:

```
razor
<partial name="_Detalle" model="contacto" />
```

Razor renderiza ese partial pasándole `contacto` como su `Model`, y pega el resultado donde está la etiqueta. Así componés una pantalla a partir de fragmentos.

Desde un handler, con `Partial(...)`:

```
csharp
public IActionResult OnGetDetalle(int id) {
    Contacto? contacto = repositorio.ObtenerPorId(id);
    return Partial("_Detalle", contacto);
}
```

La diferencia clave: un handler normal renderiza una página entera (con su layout). Pero `Partial(...)` devuelve **solo el fragmento**, sin layout, sin `<html>` ni `<head>`. Es exactamente un pedacito de HTML aislado.

El principio: el mismo partial sirve para componer una página completa (vía `<partial>`) y para responder un fragmento suelto (vía `Partial(...)`). Esa dualidad permite que la presentación viva en un solo lugar y se reutilice en distintos contextos sin duplicar HTML.

9. La inyección de dependencias: cómo las piezas se encuentran

Queda una pregunta natural: si un handler usa, por ejemplo, un repositorio de datos, pero nunca escribís `new Repositorio(...)`, ¿de dónde sale esa instancia?

Acá entra la **inyección de dependencias** (DI). El primer principio es separar dos roles: hay objetos que *necesitan* colaboradores para funcionar (un handler necesita un repositorio; el repositorio necesita una conexión a la base), y hay un mecanismo central —el **contenedor** de DI— que se encarga de *crearlos* y *entregárselos*. En vez de que cada objeto fabrique sus propias dependencias, las *pide* y el contenedor se las da. De ahí "inyección": las dependencias se inyectan desde afuera.

El proceso tiene dos mitades. Primero, en `Program.cs`, **registrás** qué servicios existen y cómo crearlos:

```
csharp
builder.Services.AddRazorPages();
builder.Services.AddScoped<Repositorio>();
```


Cada `Add . . .` le dice al contenedor "cuando alguien necesite un `Repositorio`, sabé cómo dárselo". Después, **consumís** ese servicio pidiéndolo donde lo necesitás. En una página Razor se pide con `@inject`:

```
razor
@inject Repositorio Repositorio
```

Esa línea le dice a Razor "necesito un `Repositorio`, conseguímelo del contenedor". Razor lo busca, lo crea (con sus propias dependencias resueltas en cadena: si el repositorio necesita una conexión, el contenedor la crea primero), y te lo deja disponible como una variable que tus handlers pueden usar.

El `Add . . .` también define el **ciclo de vida** —cuánto vive cada instancia—, y esto importa:

- `AddScoped`: una instancia por petición HTTP. Apropiado para objetos que no guardan estado entre peticiones; uno fresco por request es lo natural.
- `AddSingleton`: una sola instancia para toda la vida de la aplicación, compartida por todas las peticiones. Apropiado para servicios sin estado mutable o con estado seguro de compartir.
- `AddTransient`: una instancia nueva cada vez que se pide. Apropiado para objetos livianos y sin estado.

La elección depende de si el objeto tiene estado y de si es seguro compartirlo. Elegir mal acá es una fuente clásica de errores sutiles (por ejemplo, compartir como singleton un objeto que asume ser de una sola petición).

*El principio: **DI invierte quién manda en la creación de objetos**. Sin DI, cada objeto fabrica y conoce a sus colaboradores, lo que los amarra entre sí. Con DI, los objetos solo declaran qué necesitan, y un mecanismo externo decide cómo construirlo. Eso hace que puedas cambiar la implementación (por ejemplo, una versión falsa para tests) sin tocar el código que la usa.*

10. Cómo encaja todo en una sola petición

Cerremos con el recorrido completo de una acción, que ata los conceptos. El usuario hace clic en "ver contacto 5":

1. **Ruteo (URL → archivo)**. Llega `GET /?handler=Detalle&id=5`. La URL / mapea a `Pages/Index.cshtml`.
2. **Ruteo (verbo + handler → método)**. Verbo `GET` + `handler=Detalle` → se ejecuta `OnGetDetalle`.
3. **Binding**. `id=5` (texto en la URL) se convierte al `int id` del método.
4. **Inyección**. El handler usa el repositorio que la página recibió por `@inject`, ya construido por el contenedor.
5. **Modelo**. `repositorio.ObtenerPorId(5)` devuelve un `Contacto` (entidad de dominio).
6. **Partial**. El handler hace `return Partial("_Detalle", contacto)`: renderiza el fragmento con ese modelo.
7. **Respuesta**. Ese HTML vuelve al navegador, que lo muestra.

Cada concepto es una regla chica y predecible. La sensación de "magia" de Razor desaparece cuando ves que es siempre la misma cadena: **una URL elige un método por su nombre, el binding le llena los argumentos, la inyección le presta los colaboradores, el método produce un modelo, y un partial (o la página) lo convierte en HTML.**

Resumen de los principios

Concepto	Regla en una frase
Página	El HTML se arma en el servidor ejecutando código; al navegador llega el texto final.
Ruteo	La URL elige el archivo (por carpeta) y el verbo + <code>handler</code> eligen el método.
Binding	Los datos de texto del HTTP se asignan a los argumentos C# uniendo por nombre.
Modelos	Cada modelo protege una frontera: dominio (adentro), DTO (entrada), view model (pantalla).
Formularios	GET muestra el form (tag helpers atan el HTML al modelo); POST valida con <code>ModelState</code> y redirige (PRG).
Layout	Una plantilla común contiene el documento; la página aporta el relleno en <code>@RenderBody()</code> .
Partials	Fragmentos reutilizables; sirven para componer páginas y para devolver HTML suelto.
Inyección	Los objetos declaran qué necesitan; un contenedor los crea y se los entrega.

CAPITULO 42

HTMX

1. El punto de partida: cómo funciona la web clásica

Antes de cualquier framework, la web funcionaba con un mecanismo simple y robusto. El navegador muestra una página HTML. Dentro de esa página hay dos cosas que pueden iniciar una nueva petición al servidor:

- Un **enlace** (`<a href="...">`): al hacer clic, el navegador pide esa URL por GET.
- Un **formulario** (`<form action="..." method="post">`): al enviarlo, el navegador manda sus campos a esa URL.

En ambos casos pasa lo mismo: el navegador hace la petición, el servidor responde con **una página HTML completa y nueva**, y el navegador **descarta la página actual y dibuja la nueva** de cero.

A este modelo se lo llama **hipertexto**: documentos HTML enlazados entre sí, donde navegar es reemplazar un documento por otro. Es el modelo para el que se diseñó la web, y tiene virtudes enormes: es simple, no requiere JavaScript, funciona en cualquier navegador, y el estado vive en un solo lugar (el servidor). El primer principio a retener es ese: **en la web clásica, solo dos elementos —enlaces y formularios— pueden hablar con el servidor, solo responden a dos eventos —clic y envío—, y la respuesta siempre reemplaza la página entera.**

2. El problema: recargar toda la página es un desperdicio

Ese modelo tiene una incomodidad concreta. Imaginá una pantalla con una lista a la izquierda y un panel de detalle a la derecha. Hacés clic en un elemento de la lista para ver su detalle. En la web clásica, eso pide una página nueva y **redibuja todo**: la lista de la izquierda —que no cambió— se descarta y se vuelve a descargar y pintar, igual que el encabezado, el menú y todo lo demás.

Las consecuencias se sienten:

- **Parpadeo**: la pantalla se pone en blanco un instante mientras llega y se dibuja la página nueva.
- **Se pierde el estado visual**: la posición del scroll, el foco, lo que el usuario tenía a medio escribir.
- **Se transfiere de más**: el servidor manda HTML de zonas que ya estaban iguales.

La pregunta natural es: *si lo único que cambió es el panel de la derecha, ¿por qué no actualizar solo ese panel y dejar el resto intacto?* Esa es exactamente la mejora que queremos. El problema es que las herramientas de la web clásica no la permiten: un enlace o un formulario reemplazan **todo**, no saben hacer un reemplazo parcial.

3. La solución de las SPA (y su costo)

La industria resolvió esto con las **Single Page Applications** (SPA): aplicaciones donde el navegador carga una vez una base, y a partir de ahí JavaScript se encarga de todo. La mecánica es:

1. El usuario hace algo (un clic). Código JavaScript lo intercepta.
2. Ese código hace una petición al servidor con `fetch`, y el servidor responde con **datos crudos**, normalmente en formato **JSON**.
3. Más código JavaScript toma esos datos, **construye el HTML** correspondiente en el navegador, y lo inserta en el lugar justo del documento.

Esto resuelve el problema: solo se actualiza la parte que cambió, sin recargar nada. Pero tiene un costo grande, y entenderlo es clave para entender por qué existe HTMX:

- **La lógica de presentación se duplica.** El servidor sabe cómo son los datos; el navegador sabe cómo convertirlos en HTML. Las reglas de "cómo se ve un contacto" viven ahora en *dos* lados.
- **Hay estado en dos lugares.** El servidor tiene la verdad de los datos, pero el navegador mantiene su propia copia para dibujar, y mantener ambas sincronizadas es una fuente constante de bugs.
- **Hace falta mucho JavaScript** y un andamiaje considerable (frameworks, herramientas de build, etc.) para algo que conceptualmente era simple.

El principio: la SPA gana interactividad parcial a cambio de duplicar la presentación y el estado. Ese intercambio vale la pena para apps muy interactivas (editores colaborativos, juegos), pero es carísimo para la enorme mayoría de pantallas, que son formularios, listas y detalles.

4. La idea central de HTMX: generalizar el hipertexto

Acá entra HTMX, con una observación tan simple que parece tramposa. Recordá las tres limitaciones de la web clásica (sección 1):

1. Solo **enlaces y formularios** pueden hablar con el servidor.
2. Solo responden a **clic y envío**.
3. La respuesta siempre **reemplaza la página entera**.

HTMX pregunta: ¿y si quitamos esas tres limitaciones arbitrarias? ¿Qué pasaría si **cualquier** elemento pudiera hacer una petición, ante **cualquier** evento, y la respuesta pudiera reemplazar **cualquier** parte de la página? Eso es, literalmente, todo lo que hace HTMX. No inventa un paradigma nuevo: **generaliza el que la web ya tenía**.

HTMX es una librería de JavaScript chica (unos 14 kb) que lees en el `<head>` de tu página. Una vez cargada, lee **atributos HTML** que vos escribís —los que empiezan con `hx`— y, por debajo, hace el `fetch` y el reemplazo del DOM que vos habrías tenido que programar a mano en una SPA. La diferencia es que vos no escribís ese JavaScript: lo declarás con atributos.

```
html
<!-- web clásica: solo un enlace, reemplaza todo -->
<a href="/contactos/5">Ver contacto</a>

<!-- HTMX: cualquier elemento, cualquier evento, reemplazo parcial -->
<button hx-get="/contactos/5" hx-target="#detalle">Ver contacto</button>
```

El botón de HTMX hace una petición GET a `/contactos/5`, recibe HTML del servidor, y lo mete dentro del elemento `#detalle` —sin tocar el resto de la página—. Mismo espíritu que el enlace, pero sin las tres limitaciones.

*El principio: **HTMX es hipertexto sin restricciones**. Si entendés cómo funcionaba un `<a href>`, ya entendés HTMX; solo que ahora el "enlace" puede ser cualquier elemento, dispararse con cualquier evento, y actualizar solo un pedazo de la pantalla.*

5. Los cuatro atributos que lo explican casi todo

Toda interacción de HTMX se arma combinando, como mucho, cuatro decisiones. Cada una corresponde a un atributo, y juntas responden a las cuatro preguntas de cualquier interacción:

Pregunta	Atributo	Responde...
¿Qué pido?	<code>hx-get</code> / <code>hx-post</code>	a qué URL, con qué verbo HTTP
¿Cuándo lo pido?	<code>hx-trigger</code>	qué evento dispara la petición
¿Dónde lo pongo?	<code>hx-target</code>	en qué elemento del DOM va la respuesta
¿Cómo lo pongo?	<code>hx-swap</code>	de qué forma se inserta el HTML recibido

Esa es la columna vertebral. Casi todo lo demás de HTMX son variaciones o comodidades sobre estas cuatro. Las vemos una por una.

6. `hx-get` y `hx-post`: qué pedir

Estos atributos dicen **a qué URL** hacer la petición y **con qué verbo HTTP**. Hay uno por cada verbo: `hx-get`, `hx-post`, `hx-put`, `hx-patch`, `hx-delete`.

```
html
<button hx-get="/contactos/5">Ver</button>
<button hx-post="/contactos">Crear</button>
<button hx-delete="/contactos/5">Borrar</button>
```

La elección del verbo sigue la semántica de siempre: GET para *leer* sin cambiar nada, POST (o PUT/PATCH/DELETE) para *modificar* el estado del servidor. Tener ese atributo en un elemento es lo que lo convierte en "activo": le dice a HTMX "este elemento puede hacer peticiones". El valor es la URL, igual que el `href` de un enlace o el `action` de un formulario.

*El principio: `hx-get/hx-post` **reemplazan al** `href` **y al** `action`. Es la misma idea de siempre —una URL y un verbo— pero ahora aplicable a cualquier elemento, no solo a enlaces y formularios.*

7. `hx-trigger`: cuándo pedir

En la web clásica, los eventos estaban fijos: un enlace dispara con el clic, un formulario con el envío. `hx-trigger` rompe esa fijeza: vos elegís **qué evento** del DOM dispara la petición.

```
html
<!-- al hacer clic (es el default para un botón, podés omitirlo) -->
<button hx-get="/datos" hx-trigger="click">Cargar</button>

<!-- al cargarse el elemento en la página -->
<div hx-get="/lista" hx-trigger="load">Cargando...</div>

<!-- al cambiar el valor de un input -->
<input hx-get="/buscar" hx-trigger="change" />
```

Cada elemento tiene un evento por defecto razonable (clic para botones, `submit` para formularios, `change` para inputs), así que muchas veces ni hace falta escribir `hx-trigger`. Pero cuando lo escribís, podés afinarlo con **modificadores**, que son la parte realmente potente:

```
html
<input
  hx-get="/buscar"
  hx-trigger="input changed delay:300ms" />
```

Esto se lee: disparará cuando llegue el evento `input` (cada tecla), pero **solo si el valor cambió** (`changed`, evita disparar si tocaste una flecha sin cambiar texto), y **esperá 300 ms desde la última tecla** antes de mandar la petición (`delay:300ms`). Ese `delay` es el clásico *debounce*: en vez de pedir al servidor en cada tecla, espera a que el usuario haga una pausa. Con un atributo conseguís un comportamiento que a mano sería bastante código.

Otros modificadores útiles: `once` (dispara una sola vez), `from:<selector>` (escuchar el evento en *otro* elemento, no en este), y `every <tiempo>` (para *polling*: pedir periódicamente).

*El principio: `hx-trigger` **libera el "cuándo"**. La web clásica te daba un evento fijo por elemento; HTMX te deja elegirlo y refinarlo (*debounce*, una sola vez, escuchar a otro elemento, repetir en intervalos), todo de forma declarativa.*

8. `hx-target`: dónde poner la respuesta

Por defecto, HTMX inserta la respuesta **en el mismo elemento que disparó la petición**. Pero casi siempre querés ponerla en otro lado: hacés clic en un botón de la lista y querés que el resultado aparezca en el panel de detalle. `hx-target` dice **en qué elemento** va la respuesta, usando un selector CSS:

```
html
<button hx-get="/contactos/5" hx-target="#detalle">Ver</button>
<div id="detalle"></div>
```

El valor es un selector CSS (#detalle, .panel, etc.), pero HTMX agrega palabras clave relativas muy cómodas para no depender de IDs fijos:

- `this` — el propio elemento que dispara (es el default).
- `closest <selector>` — el ancestro más cercano que coincide (ej. `closest tr` apunta a la fila de tabla que contiene al botón).
- `find <selector>` — el primer descendiente que coincide.
- `next <selector>` / `previous <selector>` — el siguiente/anterior elemento que coincide en el DOM.

```
html
<!-- pone la respuesta en la fila de tabla que contiene este botón -->
<button hx-get="/refrescar" hx-target="closest tr">Refrescar fila</button>
```

*El principio: `hx-target` **separa "quién dispara" de "qué se actualiza"**. En la web clásica eran lo mismo (el enlace que clickeás recarga todo). HTMX desacopla las dos cosas: un elemento puede disparar un cambio en cualquier otro.*

9. hx-swap: cómo insertarla

Sabiendo *dónde* va la respuesta (`hx-target`), falta decidir *cómo* se inserta respecto a ese target. Eso es `hx-swap`. Las opciones se basan en nombres estándar del DOM:

Valor	Qué hace
<code>innerHTML</code>	Reemplaza el contenido del target (es el default).
<code>outerHTML</code>	Reemplaza el target entero , incluido él mismo.
<code>beforeend</code>	Agrega la respuesta al final del contenido del target.
<code>afterbegin</code>	Agrega la respuesta al principio del contenido del target.
<code>beforebegin</code>	Inserta antes del target (como hermano previo).
<code>afterend</code>	Inserta después del target (como hermano siguiente).
<code>delete</code>	Elimina el target (ignora la respuesta).
<code>none</code>	No inserta nada (útil si solo te interesan los efectos colaterales).

La distinción más usada es `innerHTML` vs `outerHTML`: el primero deja el contenedor y cambia su interior; el segundo reemplaza el contenedor mismo. Y `beforeend` es típico para "agregar a una lista" sin tocar lo que ya estaba (pensá en cargar más resultados al final).

```
html
<!-- agrega cada comentario nuevo al final de la lista, sin recargar los anteriores -->
<form hx-post="/comentarios" hx-target="#lista" hx-swap="beforeend">
  <textarea name="texto"></textarea>
  <button type="submit">Comentar</button>
</form>
<div id="lista"></div>
```

hx-swap también acepta modificadores de tiempo, como `swap: 200ms` (esperar antes de reemplazar) o `transition: true` (usar la View Transitions API del navegador para animar el cambio), útiles para sincronizar con animaciones CSS.

*El principio: hx-target y hx-swap **son una pareja**: uno elige el elemento de referencia, el otro la operación del DOM respecto a ese elemento. Entre los dos cubren cualquier forma de meter HTML en la página.*

10. Enviar datos: parámetros, formularios y hx-include

Pedir HTML está bien, pero muchas veces necesitás **mandar datos** al servidor. HTMX recoge datos de varias formas, sin que vos los serialices a mano:

- **Desde un formulario:** si el elemento con `hx-post` es un `<form>` (o está adentro), HTMX recoge automáticamente todos sus campos y los manda, igual que un formulario clásico.
- **En la URL:** podés poner parámetros directamente en el atributo, como `hx-get="/buscar?activo=true"`.
- **Con hx-include:** este es el atributo que da flexibilidad. Le indicás, con un selector CSS, **qué otros campos incluir** en la petición aunque no estén en el mismo formulario.

```
html
<input id="busqueda" name="q" />

<!-- este botón no está en el form de búsqueda, pero incluye su campo -->
<button hx-get="/buscar" hx-include="#busqueda" hx-target="#resultados">
  Buscar
</button>
```

hx-include también acepta las palabras relativas que vimos (`closest` form incluye los campos del formulario más cercano, por ejemplo). Es la herramienta para juntar, en una petición, datos que viven en distintas partes de la pantalla.

*El principio: **HTMX reusa el mecanismo de envío de los formularios** (campos con name, recogidos automáticamente) y lo extiende con `hx-include` para componer la petición con campos de donde haga falta. No hay serialización manual: nombrás los campos y HTMX arma el envío.*

11. El servidor manda HTML, no JSON

Acá está la diferencia filosófica con las SPA, y conviene detenerse. En una SPA, el servidor responde **datos** (JSON) y el navegador los convierte en HTML. En HTMX, el servidor responde **HTML ya armado**, y HTMX simplemente lo coloca donde indica `hx-target`.

```
html
<!-- Lo que el servidor devuelve a una petición HTMX NO es esto: -->
{ "id": 5, "nombre": "Ana", "telefono": "381..." }

<!-- ...sino directamente esto: -->
<div class="detalle">
  <h2>Ana Gómez</h2>
  <p>381...</p>
</div>
```

Esto tiene una consecuencia profunda que resuelve el costo de las SPA (sección 3): **la lógica de presentación vive en un solo lugar, el servidor**. No hay plantillas duplicadas en el navegador, no hay estado de UI que mantener sincronizado. El servidor es la única fuente de verdad, tanto de los datos como de cómo se ven. Por eso con HTMX se escribe poco o nada de JavaScript propio: la pantalla la arma el servidor con sus plantillas habituales (las de tu framework de backend), y HTMX solo transporta esos fragmentos y los inserta.

Esto también explica por qué HTMX es **agnóstico del backend**: como solo necesita que el servidor devuelva fragmentos de HTML, funciona igual con cualquier lenguaje o framework que sepa generar HTML.

*El principio: **HTMX devuelve la presentación al servidor**. La SPA repartía el "cómo se ve" entre cliente y servidor; HTMX lo concentra del lado del servidor y deja al navegador un rol mínimo: pedir un fragmento y pegarlo. Una sola fuente de verdad, cero JavaScript que mantener.*

12. Comunicación de vuelta: los headers HX- * y los eventos

Hasta acá la conversación fue en una dirección: el cliente pide, el servidor manda HTML. Pero a veces el servidor necesita decirle algo *extra* al cliente más allá del fragmento: "además de esto, refrescá tal otra zona" o "redirigí a tal página". Para eso, HTMX usa **headers HTTP** en la respuesta.

El más característico es `HX-Trigger`: con él, el servidor puede **disparar un evento** en el navegador. Esto resuelve un problema típico: una petición HTMX apunta a *un solo* `hx-target`, pero a veces una acción debería actualizar *dos* zonas. La técnica es:

1. Un elemento escucha un evento personalizado usando `hx-trigger` con el modificador `from:body`:

```
html
<div id="lista"
  hx-get="/lista"
  hx-trigger="load, datosCambiaron from:body">
</div>
```

Esto dice: cargá la lista al inicio (`load`) y cada vez que en el `<body>` se dispare un evento llamado `datosCambiaron`.

2. El servidor dispara ese evento devolviendo un header en la respuesta de *otra* petición (por ejemplo, al guardar):

```
texto
HX-Trigger: datosCambiaron
```

Cuando HTMX recibe una respuesta con ese header, dispara el evento `datosCambiaron` en el navegador. El `#lista`, que lo está escuchando, se refresca solo. Así, una acción de guardar que devuelve el detalle puede *además* avisar "los datos cambiaron", y la lista se actualiza sin que las dos plantillas se conozcan entre sí.

El modificador `from:body` es importante acá: el evento del header se dispara a nivel del `<body>`, así que el elemento que quiere reaccionar tiene que escuchar desde ahí, no desde sí mismo.

Hay otros headers de respuesta útiles: `HX-Redirect` (forzar una redirección del navegador), `HX-Refresh` (recargar la página entera), `HX-Retarget` (cambiar a último momento dónde va la respuesta). Y en la *petición*, HTMX manda headers como `HX-Request: true`, que le permiten al servidor saber que la pidió HTMX (y devolver un fragmento) en vez de un navegador normal (y devolver la página completa).

*El principio: **los headers** `HX-*` **son el canal de vuelta**. El cuerpo de la respuesta lleva el HTML; los headers llevan instrucciones de control. Con `HX-Trigger`, el servidor puede orquestar actualizaciones de varias zonas disparando eventos, sin acoplar las plantillas entre sí.*

13. El recorrido completo de una interacción

Cerremos atando todo. El usuario está en una pantalla con una lista y un panel de detalle, y hace clic en un contacto:

```
texto
El usuario hace clic en un contacto de la lista
■ el elemento tiene: hx-get="/contactos/5"
■                      hx-trigger="click"      (¿CUÁNDO? al clic)
■                      hx-target="#detalle"    (¿DÓNDE? en #detalle)
■                      hx-swap="innerHTML"     (¿CÓMO? reemplaza su interior)
▼
HTMX hace GET /contactos/5
■ (manda también el header HX-Request: true)
▼
El servidor ejecuta su lógica y renderiza un FRAGMENTO de HTML
(no una página entera, no JSON): el detalle del contacto 5
■ opcionalmente agrega headers, ej. HX-Trigger: datosCambiaron
▼
HTMX recibe el HTML
■ lo inserta DENTRO de #detalle (innerHTML)
■ el resto de la página (la lista, el menú) NO se toca
■ si vino HX-Trigger, dispara ese evento → otras zonas que lo
■ escuchan se refrescan solas
▼
La pantalla se actualizó parcialmente, sin recargar, sin parpadeo,
sin que escribiéramos una línea de JavaScript
```

Visto así, HTMX no es un framework nuevo que haya que aprender de cero: es la web clásica con tres restricciones levantadas. **Cualquier** elemento puede pedir (`hx-get/hx-post`), ante **cualquier** evento

(`hx-trigger`), poniendo la respuesta en **cualquier** lugar (`hx-target`) de **cualquier** forma (`hx-swap`). El servidor sigue mandando HTML como siempre, y por eso la presentación y el estado siguen viviendo en un solo lado. Toda la complejidad de las SPA —el JSON, las plantillas duplicadas, el estado sincronizado— desaparece, y a cambio recuperás la interactividad parcial que la web clásica no tenía.

Resumen de los principios

Concepto	Regla en una frase
Hipertexto	La web clásica: solo enlaces/formularios piden, y la respuesta reemplaza todo.
Idea de HTMX	Levantar tres restricciones: cualquier elemento, cualquier evento, reemplazo parcial.
<code>hx-get/hx-post</code>	Qué pedir: la URL y el verbo HTTP, en cualquier elemento.
<code>hx-trigger</code>	Cuándo pedir: qué evento dispara, con modificadores como <code>delay</code> (debounce).
<code>hx-target</code>	Dónde poner la respuesta: un selector CSS, desacoplado de quién dispara.
<code>hx-swap</code>	Cómo insertarla: reemplazar el interior, el elemento, agregar al final, etc.
HTML, no JSON	El servidor manda presentación ya armada; una sola fuente de verdad.
Headers HX- *	El canal de vuelta: HX-Trigger dispara eventos para orquestar varias zonas.

CAPITULO 43

Agenda CRUD con Razor Pages, HTMX y CSS propio

1. Qué vamos a construir

Una agenda de contactos con las operaciones de un CRUD: listar, buscar, ver detalle, crear, editar y borrar. La pantalla es de tipo **maestro-detalle**: a la izquierda la lista de contactos, a la derecha un panel que cambia según lo que estés haciendo (ver, crear o editar).

Lo que define esta arquitectura es que **no escribimos JavaScript propio para la interacción**. HTMX es una pequeña librería que lee atributos HTML —hx-get, hx-post, hx-trigger, hx-target, hx-swap— para decidir qué evento dispara una petición, qué método HTTP usa y qué parte del HTML reemplaza. El servidor responde con fragmentos de HTML ya renderizados por Razor, y HTMX los inserta en su lugar. No hay API JSON, no hay framework de frontend, no hay estado duplicado en el navegador.

Las técnicas que vas a aprender y poder reutilizar en otros proyectos:

- **Renderizar HTML parcial en el servidor** y reemplazar regiones de la página con HTMX, sin escribir JavaScript.
- **Separar el dominio de la entrada** con un modelo y un DTO distintos.
- **Estructurar el acceso a datos en capas** (repositorio + servicio) con interfaces e inyección de dependencias.
- **Persistir con EF Core + SQLite** usando contextos efímeros por operación.
- **Mantener la protección antiforgery activa** mientras las peticiones viajan por HTMX.
- **Refrescar una región de la pantalla desde el servidor** emitiendo un evento por cabecera HTTP (HX-Trigger), la técnica central del tutorial.
- **Construir una lista seleccionable accesible** con radios navegables por teclado.

La estructura de carpetas queda así:

```
texto
AgendaRazorHtmx/
  Program.cs          <- arranque y registro de servicios
  Data/
    AgendaDbContext.cs <- DbContext de EF Core
    AgendaDbSeeder.cs  <- semilla de datos
  Models/
    Contact.cs         <- entidad de dominio
  Dtos/
    ContactInput.cs    <- contrato de entrada de formularios
  Repositories/
    IContactRepository.cs <- acceso a datos con EF Core
    ContactRepository.cs
  Services/
    IContactService.cs  <- lógica de negocio
    ContactService.cs
```

texto - continuación

```

■■■■ ViewModels/
■ ■■■■ ContactListViewModel.cs      <- lista + selección + señales de refresco
■ ■■■■ ContactListItemViewModel.cs  <- un ítem de la lista
■■■■ Pages/
■ ■■■■ Index.cshtml                <- pantalla principal + handlers inline
■ ■■■■ _ContactList.cshtml         <- lista (panel maestro)
■ ■■■■ _ContactListItem.cshtml     <- un ítem de la lista
■ ■■■■ _ContactDetail.cshtml       <- detalle (solo lectura)
■ ■■■■ _ContactForm.cshtml        <- alta y edición
■ ■■■■ Shared/
■ ■■■■ _Layout.cshtml              <- plantilla base: HTMX + CSS propio
■■■■ wwwroot/
■ ■■■■ css/site.css
■ ■■■■ js/htmx.min.js

```

Al ejecutar la app se crea `agenda.db` en la raíz del proyecto (excluido de git por `.gitignore`, junto con `agenda.db-shm` y `agenda.db-wal`).

Principio: una carpeta por responsabilidad. Cada tipo vive cerca de sus pares y tiene una sola razón para cambiar. El dominio en `Models/`, el contrato de entrada en `Dtos/`, el acceso a datos en `Repositories/`, la lógica de negocio en `Services/` y las clases de presentación en `ViewModels/`. Esta organización escala: cuando el proyecto crece, sabés exactamente dónde va cada pieza nueva.

2. Cómo fluye una interacción

Antes de escribir código conviene fijar el modelo mental. Cada vez que el usuario hace algo —un clic, escribir en el buscador— el recorrido es el mismo:

texto

```

Navegador
  ■ HTMX dispara un GET o POST
  ▼
Index.cshtml (un handler: OnGetDetail, OnPostSave, ...)
  ■
  ▼
ContactService (lógica de negocio)
  ■
  ▼
ContactRepository → EF Core → SQLite (agenda.db)

```

Y la respuesta vuelve convertida en HTML:

texto

```

SQLite → Repository → Service → Handler → Partial Razor → HTML
                                     ■
                                     ▼
                                HTMX inserta ese HTML en una región de la página

```

La idea central de HTMX (hipermedia). El servidor no devuelve datos crudos (JSON) que el navegador deba interpretar y convertir en pantalla. Devuelve **HTML ya armado**, y HTMX lo coloca en el lugar indicado. La consecuencia es profunda: la lógica de "qué se ve" vive en un solo lado, el servidor. No hay plantillas duplicadas ni estado sincronizado entre cliente y servidor. Esto es lo que hace que no necesites escribir JavaScript.

3. Crear el proyecto

Abrí una terminal y ejecutá:

```
bash
dotnet new webapp -n AgendaRazorHtmx
cd AgendaRazorHtmx
```

`dotnet new webapp` crea un proyecto *Razor Pages* (distinto de `dotnet new web`, que es el mínimo para *Minimal API*). Trae la estructura de `Pages/` y la configuración base. HTMX lo servimos como **archivo local** desde `wwwroot/js/` y los estilos los escribimos en `wwwroot/css/site.css`, así que no hay paquetes de frontend que instalar.

Agregá el paquete de EF Core para SQLite, lo único extra que necesitamos:

```
bash
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
```

El template trae páginas de ejemplo en `Pages/`. Las vamos a reemplazar por las nuestras.

4. Razor Pages y los handlers inline

Razor Pages genera HTML en el servidor. Una página es un archivo `.cshtml` que mezcla HTML con C#, y la directiva `@page` al inicio la convierte en una URL accesible:

```
razor
@page

<h1>Hola Razor</h1>
```

La lógica de una página puede vivir en un archivo aparte (`Index.cshtml.cs`, el *code-behind*) o, como acá, en **handlers inline** dentro del mismo `.cshtml`, en un bloque `@functions`:

```
razor
@functions {
    public IActionResult OnGetDetail(int id) {
        // Se ejecuta con: GET /?handler=Detail&id=1
    }
}
```

La convención de Razor traduce la URL al nombre del método combinando el verbo HTTP con el parámetro `handler`:

Petición HTTP	Método que se ejecuta
GET /	OnGet()
GET /?handler=Detail&id=1	OnGetDetail(int id)
POST /?handler=Save	OnPostSave(...)

Los parámetros restantes de la URL (`id=1`) se asignan a los argumentos del método por nombre: eso es **model binding**, y funciona igual con tipos complejos (un `record` o una clase) armándolos a partir de los campos de un formulario.

Cuándo usar handlers inline. Concentrar los handlers en el `.cshtml` deja toda la lógica de una pantalla en un archivo, fácil de leer de corrido. Es ideal cuando la página es el punto de orquestación y delega el trabajo pesado a servicios (como acá). Si la lógica de la página creciera mucho, el `code-behind` separado vuelve a ser preferible. La regla práctica: *inline* mientras la página solo orqueste; *code-behind* cuando empiece a contener reglas de negocio.

5. CSS propio en vez de un framework

Escribimos un `wwwroot/css/site.css` propio en lugar de usar Bootstrap, Tailwind o Pico. La pantalla maestro-detalle tiene requisitos concretos —dos paneles con scroll independiente, lista seleccionable con radios, altura completa de la ventana— que resultan más limpios con un puñado de reglas a medida que adaptando los defaults de un framework.

El CSS se apoya en tres ideas que conviene conocer porque se reutilizan en cualquier hoja de estilos:

```
css
:root {
  --color-surface: #ffffff;
  --color-selected: #d9e1f2;
  --color-focus: #2556a3;
  --radius: 8px;
  --space-4: 1rem;
  /* ... */
}
```

- **Variables CSS** (`--color-*`, `--space-*`): un solo lugar para definir el tema y el espaciado. Cambiás una variable y se actualiza toda la app.
- **Layout con Grid** para el maestro-detalle (`grid-template-columns: 320px 1fr`), con scroll contenido en cada panel.
- **Accesibilidad de fábrica:** estilos de `:focus-visible`, una utilidad `.visually-hidden`, y bloques `@media (prefers-reduced-motion)` y `@media (forced-colors)` que respetan las preferencias del sistema operativo.

El archivo completo está en [wwwroot/css/site.css](#). No hace falta memorizarlo; alcanza con reconocer qué hace cada clase cuando aparezca en los `partials` (`.panel`, `.contact-item`, `.detail-grid`, `.actions`, `.primary`, `.danger`, `.muted`, `.visually-hidden`).

Cuándo escribir CSS propio. Un framework sin clases es ideal para formularios y páginas convencionales. Cuando necesitás control fino sobre el layout —un grid de dos columnas con scroll independiente, el resaltado preciso de una fila seleccionada— suele salir más limpio escribir las reglas vos. El costo es mantenerlo; la ventaja es que no hay clases ajenas que pelean entre sí ni dependencias externas.

6. HTMX: el modelo de hipermedia

HTMX amplía el HTML de forma declarativa. En vez de programar clics, fetch y reemplazo del DOM, agregás atributos al markup que describen el comportamiento:

```
html
<button
  hx-get="/?handler=Detail&id=2"
  hx-target="#detail"
  hx-swap="innerHTML">
  Ver detalle
</button>
```

Al hacer clic, HTMX hace tres cosas: pide por GET a esa URL, recibe el HTML que devuelve el servidor, y lo inserta dentro de `#detail`. Toda la lógica está declarada en atributos HTML.

Estos son los atributos y técnicas que usamos:

Atributo / técnica	Para qué sirve
hx-get / hx-post	Hacer una petición GET o POST
hx-target	A qué elemento aplicar la respuesta
hx-swap	Cómo insertar el HTML (<code>innerHTML</code> reemplaza el contenido)
hx-trigger	Qué evento dispara la petición, con modificadores como <code>delay:250ms</code>
hx-include	Enviar campos extra junto con la petición (un token, el buscador)
hx-vals	Adjuntar pares clave-valor fijos a la petición
hx-confirm	Mostrar un cuadro de confirmación antes de enviar
hx-trigger="load"	Disparar una petición apenas el elemento entra al DOM
hx-trigger="evento from:body"	Escuchar en el <code><body></code> un evento personalizado
Cabecera HX-Trigger-After-Swap	El servidor ordena a HTMX disparar un evento tras insertar el HTML

Las dos últimas filas son la base de la técnica central (sección 14): el servidor puede emitir un evento desde una respuesta HTTP, y cualquier elemento de la página que escuche ese evento reacciona. Eso permite que una mutación actualice una región distinta de la que apuntó la petición.

Por qué HTML parcial y no JSON. En una SPA tradicional, el servidor manda JSON y el navegador tiene código que lo convierte en HTML; la lógica de presentación queda duplicada en dos lados. Con HTMX, el servidor manda el HTML ya armado por Razor y el navegador solo lo coloca: una sola fuente de verdad para la presentación, cero JavaScript que mantener. Esta es la decisión arquitectónica que sostiene todo el proyecto.

7. El modelo y el DTO de entrada

Empecemos por los dos tipos que representan un contacto: uno hacia adentro (el dominio) y otro hacia afuera (la entrada).

El **modelo** vive en `Models/Contact.cs`. Representa cómo existe un contacto dentro de la aplicación y es también la entidad que EF Core mapea a una tabla:

```
csharp
// Models/Contact.cs
namespace AgendaRazorHtmx.Models;

public class Contact {
    public int Id { get; set; }
    public string Nombre { get; set; } = "";
    public string Apellido { get; set; } = "";
    public string Telefono { get; set; } = "";
    public string Email { get; set; } = "";
    public string Notas { get; set; } = "";
}
```

El **DTO de entrada** vive en `Dtos/ContactInput.cs`. Define qué datos acepta un formulario. Es un record porque es inmutable y se declara en pocas líneas:

```
csharp
// Dtos/ContactInput.cs
namespace AgendaRazorHtmx.Dtos;

public record ContactInput(
    int Id,
    string Nombre,
    string Apellido,
    string Telefono,
    string Email,
    string Notas
);
```

Principio: separar el dominio de la entrada. El modelo es lo que la app persiste y manipula; el DTO es la frontera de lo que aceptamos de afuera. Mantenerlos separados impide que un formulario toque propiedades que no debería y deja explícito qué campos son válidos como entrada. Es un patrón que se aplica en cualquier capa de borde: lo que entra no es lo mismo que lo que guardás. **Un solo DTO para crear y editar.** Usamos el Id como señal: si vale 0, es un contacto nuevo; si trae un valor real, es una edición. El formulario es el mismo en ambos casos y manda un campo oculto Id. Esto colapsa dos pantallas en una sola: un único `_ContactForm` y un único handler `OnPostSave`. El servicio decide internamente si inserta o actualiza.

Los **view models** viven en `ViewModels/` y existen para pasarle a cada vista exactamente lo que necesita, ni más ni menos:

```
csharp
// ViewModels/ContactListViewModel.cs
public record ContactListViewModel(
    List<Contact> Contacts,
    int? SelectedId,
    int? DetailIdToLoad = null,
    bool ClearDetail = false
);

// ViewModels/ContactListItemViewModel.cs
public record ContactListItemViewModel(Contact Contact, bool Selected);
```

ContactListViewModel combina los contactos con cuál está seleccionado y dos señales (DetailIdToLoad, ClearDetail) que veremos en acción cuando una búsqueda cambia la selección. ContactListItemViewModel le entrega a cada ítem solo su contacto y si está seleccionado.

Por qué un view model y no el modelo a secas. Una vista que necesita "el contacto más si está seleccionado" no debería recibir el modelo crudo y calcular la selección por su cuenta. El view model empaqueta exactamente lo que la vista consume. Mantiene las vistas tontas (solo muestran) y la decisión de qué mostrar en el handler.

8. El acceso a datos: repositorio y servicio

El acceso a datos está en dos capas, cada una con su interfaz. El **repositorio** habla con EF Core; el **servicio** contiene la lógica de negocio. La página depende del servicio, el servicio depende del repositorio, y ninguno conoce los detalles del otro.

El DbContext

El puente entre las clases C# y SQLite, en Data/AgendaDbContext.cs:

```
csharp
// Data/AgendaDbContext.cs
using AgendaRazorHtmx.Models;
using Microsoft.EntityFrameworkCore;

namespace AgendaRazorHtmx.Data;

public class AgendaDbContext(DbContextOptions<AgendaDbContext> options) : DbContext(options) {
    public DbSet<Contact> Contacts => Set<Contact>();
}
```

El repositorio

Repositories/ContactRepository.cs implementa IContactRepository con operaciones atómicas sobre la base. Cada método crea y cierra su propio DbContext:

```
csharp
// Repositories/IContactRepository.cs
public interface IContactRepository {
    List<Contact> GetAll(string? search = null);
    Contact? GetById(int id);
    Contact Add(Contact contact);
    Contact Update(Contact contact);
    void Delete(int id);
}
```

```
csharp
// Repositories/ContactRepository.cs
public class ContactRepository(IDbContextFactory<AgendaDbContext> dbContextFactory) : IContactRepository
{
    public List<Contact> GetAll(string? search = null) {
        using AgendaDbContext dbContext = dbContextFactory.CreateDbContext();
        IQueryable<Contact> query = dbContext.Contacts
            .OrderBy(contact => contact.Apellido)
            .ThenBy(contact => contact.Nombre);

        if (!string.IsNullOrEmpty(search)) {

```

csharp - continuacion

```

        string term = search.Trim();

        query = query.Where(contact =>
            EF.Functions.Like(contact.Nombre, $"%{term}%")
            || EF.Functions.Like(contact.Apellido, $"%{term}%")
            || EF.Functions.Like(contact.Telefono, $"%{term}%")
            || EF.Functions.Like(contact.Email, $"%{term}%")
            || EF.Functions.Like(contact.Notas, $"%{term}%")
        );
    }

    return query.ToList();
}

public Contact? GetById(int id) {
    using AgendaDbContext dbContext = dbContextFactory.CreateDbContext();
    return dbContext.Contacts.SingleOrDefault(contact => contact.Id == id);
}

public Contact Add(Contact contact) {
    using AgendaDbContext dbContext = dbContextFactory.CreateDbContext();
    dbContext.Contacts.Add(contact);
    dbContext.SaveChanges();
    return contact;
}

public Contact Update(Contact contact) {
    using AgendaDbContext dbContext = dbContextFactory.CreateDbContext();
    dbContext.Contacts.Update(contact);
    dbContext.SaveChanges();
    return contact;
}

public void Delete(int id) {
    using AgendaDbContext dbContext = dbContextFactory.CreateDbContext();
    Contact? contact = dbContext.Contacts.SingleOrDefault(c => c.Id == id);
    if (contact is null) { return; }
    dbContext.Contacts.Remove(contact);
    dbContext.SaveChanges();
}
}

```

GetAll unifica listar y buscar: sin search devuelve todo ordenado por apellido; con texto, filtra con `EF.Functions.Like`, que EF Core traduce al LIKE de SQL. Así hay un solo camino para poblar la lista, con o sin filtro.

El servicio

`Services/ContactService.cs` implementa `IContactService` e interpreta el DTO: si el Id es 0 agrega; si no, busca el contacto, le aplica los cambios y lo actualiza.

csharp

```

// Services/IContactService.cs
public interface IContactService {
    List<Contact> GetAll(string? search = null);
    Contact? GetById(int id);
    Contact Save(ContactInput input);
    void Delete(int id);
}

```

```

csharp
// Services/ContactService.cs
public class ContactService(IContactRepository contactRepository) : IContactService {
    public List<Contact> GetAll(string? search = null) =>
        contactRepository.GetAll(search);

    public Contact? GetById(int id) =>
        contactRepository.GetById(id);

    public Contact Save(ContactInput input) {
        if (input.Id == 0) {
            Contact newContact = FromInput(input);
            return contactRepository.Add(newContact);
        }

        Contact contact = contactRepository.GetById(input.Id)
            ?? throw new InvalidOperationException($"No existe el contacto {input.Id}.");

        contact.Nombre = input.Nombre;
        contact.Apellido = input.Apellido;
        contact.Telefono = input.Telefono;
        contact.Email = input.Email;
        contact.Notas = input.Notas;

        return contactRepository.Update(contact);
    }

    public void Delete(int id) =>
        contactRepository.Delete(id);

    private static Contact FromInput(ContactInput input) => new() {
        Nombre = input.Nombre,
        Apellido = input.Apellido,
        Telefono = input.Telefono,
        Email = input.Email,
        Notas = input.Notas
    };
}

```

Principio: separar acceso a datos de lógica de negocio. El repositorio solo sabe leer y escribir filas —no interpreta el DTO ni decide qué operación corresponde. El servicio sabe qué hacer con un `ContactInput` —no sabe nada de EF Core. Cada capa tiene una razón de cambio independiente: si cambiás el ORM, tocás el repositorio; si cambia una regla de negocio, tocás el servicio. Las interfaces (`IContactRepository`, `IContactService`) hacen explícito el contrato y permiten reemplazar cualquier capa por un doble en tests. IDbContextFactory y un contexto por operación. En vez de compartir un `DbContext`, inyectamos una fábrica y creamos un contexto efímero en cada método (using lo libera al terminar). Un `DbContext` acumula estado y no es seguro entre hilos; uno de vida corta por operación evita problemas de concurrencia y de datos rancios, y encaja con peticiones breves como las de HTMX.

9. La semilla de datos

Para no arrancar con la agenda vacía, sembramos contactos al iniciar, **solo si la tabla está vacía**. Para no repetir `new Contact { Nombre = ..., Apellido = ... }` dieciocho veces, guardamos los datos como un array de **tuplas con nombre** y los proyectamos a `Contact` con un `Select`:

```

csharp
// Data/AgendaDbSeeder.cs
public static class AgendaDbSeeder {
    public static void Seed(AgendaDbContext dbContext) {
        if (dbContext.Contacts.Any()) { return; }

        (string Nombre, string Apellido, string Telefono, string Email, string Notas)[] sampleContacts =
        {
            ("Ana", "Gómez", "3815551001", "ana@mail.com", "Cliente frecuente"),
            ("Luis", "Pérez", "3815551002", "luis@mail.com", "Prefiere WhatsApp"),
            ("María", "López", "3815551003", "maria@mail.com", "Contactar por la tarde"),
            ("Jorge", "Martínez", "3815551004", "jorge@mail.com", "Contactar por la mañana"),
            ("Sofía", "García", "3815551005", "sofia@mail.com", "Contactar por la noche"),
            ("Carlos", "Rodríguez", "3815551006", "carlos@mail.com", "Contactar por la noche"),
            ("Lucía", "Fernández", "3815551007", "lucia@mail.com", "Contactar por la mañana"),
            ("Diego", "Sánchez", "3815551008", "diego@mail.com", "Contactar por la noche"),
            ("Valentina", "Ruiz", "3815551009", "valentina@mail.com", "Enviar presupuesto este
viernes"),
            ("Mateo", "Castro", "3815551010", "mateo@mail.com", "Llamar luego de las 18 hs"),
            ("Camila", "Herrera", "3815551011", "camila@mail.com", "Consulta por plan anual"),
            ("Franco", "Morales", "3815551012", "franco@mail.com", "Pide recordatorio por
email"),
            ("Julieta", "Navarro", "3815551013", "julieta@mail.com", "Revisar datos fiscales"),
            ("Tomás", "Silva", "3815551014", "tomas@mail.com", "Prefiere atención
presencial"),
            ("Agustina", "Vega", "3815551015", "agustina@mail.com", "Seguimiento la próxima
semana"),
            ("Bruno", "Ortega", "3815551016", "bruno@mail.com", "Interesado en servicio
premium"),
            ("Renata", "Molina", "3815551017", "renata@mail.com", "Derivada por un cliente
actual"),
            ("Simón", "Medina", "3815551018", "simon@mail.com", "Confirmar reunión del
martes")
        };

        dbContext.Contacts.AddRange(sampleContacts.Select(c => new Contact {
            Nombre = c.Nombre, Apellido = c.Apellido, Telefono = c.Telefono, Email = c.Email, Notas =
c.Notas
        }));
        dbContext.SaveChanges();
    }
}

```

Tuplas con nombre para datos tabulares. El tipo `(string Nombre, string Apellido, ...)` describe cada fila con campos nombrados, así el array literal queda en una línea por contacto —compacto y alineado como una tabla— sin perder claridad sobre qué es cada valor. El `Select` final convierte cada tupla en la entidad real. Es un patrón cómodo para cualquier dato de ejemplo o de configuración que tenga forma de tabla. **La guarda** `if (Any()) return` **hace la siembra idempotente**. Podés arrancar la app muchas veces sin duplicar datos. Para volver al estado inicial, borrá `agenda.db` y volvé a correr: se recrea y se siembra. Los datos persisten entre ejecuciones porque viven en el archivo `SQLite`.

10. Program.cs: dependencias y arranque

`Program.cs` registra los servicios, configura el pipeline y crea/siembrala base al iniciar.

```

csharp
using AgendaRazorHtmx.Data;
using AgendaRazorHtmx.Repositories;
using AgendaRazorHtmx.Services;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

```

csharp - continuacion

```

string connectionString = "Data Source=./agenda.db";

builder.Services.AddRazorPages();
builder.Services.AddDbContextFactory<AgendaDbContext>(options => options.UseSqlite(connectionString));
builder.Services.AddScoped<IContactRepository, ContactRepository>();
builder.Services.AddScoped<IContactService, ContactService>();

var app = builder.Build();

using (IServiceScope scope = app.Services.CreateScope()) {
    IDbContextFactory<AgendaDbContext> dbContextFactory =
        scope.ServiceProvider.GetRequiredService<IDbContextFactory<AgendaDbContext>>();

    using AgendaDbContext dbContext = dbContextFactory.CreateDbContext();
    dbContext.Database.EnsureCreated();
    AgendaDbSeeder.Seed(dbContext);
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();

app.MapRazorPages();

app.Run();

```

Registrar interfaces, no implementaciones. `AddScoped<IContactService, ContactService>()` le dice al contenedor: cuando alguien pida `IContactService`, entregá un `ContactService`. La página pide `IContactService` y nunca menciona la clase concreta. Esto es **inversión de dependencias**: el código de alto nivel depende de abstracciones, lo que te deja cambiar la implementación o inyectar un doble en tests sin tocar a quien la consume. **Antiforgery queda activado.** `AddRazorPages()` exige un token en cada POST para protegerse de ataques CSRF. No lo desactivamos: HTMX puede mandar el token. Cada formulario incluye `@Html.AntiForgeryToken()` (un input oculto); en el guardado, `hx-post` está en el `<form>` y HTMX envía sus campos solo; en el borrado, el botón usa `hx-include="closest form"`. Tenés la protección sin renunciar a HTMX. `AddDbContextFactory` y `EnsureCreated()`. Registramos una fábrica porque el repositorio crea contextos efímeros (sección 8). Al arrancar, `EnsureCreated()` crea el archivo y la tabla si no existen, y `Seed()` carga los datos. `EnsureCreated` alcanza para un tutorial; en producción con un esquema que evoluciona se usan **migraciones** de EF Core.

11. El layout

El layout es la plantilla que envuelve todas las páginas: el `<head>` con los estilos y el script de HTMX, y la estructura base.

Descargamos HTMX a un archivo local para que la app funcione sin internet y siempre con la misma versión:

bash

```

curl -fsSL https://unpkg.com/htmx.org@2.0.4/dist/htmx.min.js -o wwwroot/js/htmx.min.js

```

Creá `Pages/Shared/_Layout.cshtml`:

razor

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>@ViewData["Title"] - Agenda Razor HTMX</title>
  <link rel="icon" href="data:," />
  <script src="/js/htmx.min.js"></script>
  <link rel="stylesheet" href="/css/site.css" />
</head>
<body>
  <main class="container">
    @RenderBody()
  </main>
</body>
</html>

```

`<script src="/js/htmx.min.js">` carga el archivo local que sirve `app.UseStaticFiles()`. `<link rel="icon" href="data:," />` evita el pedido automático de `/favicon.ico` (y su 404 en consola) sin crear un ícono. `@RenderBody()` es donde se inserta el contenido de cada página.

12. La página principal con handlers inline

Esta es la pantalla central: el HTML del maestro-detalle más todos los handlers en un bloque `@functions`.

Creá `Pages/Index.cshtml`:

razor

```

@page
@using AgendaRazorHtmx.Dtos
@using AgendaRazorHtmx.Models
@using AgendaRazorHtmx.Services
@using AgendaRazorHtmx.ViewModels
@using Microsoft.AspNetCore.Mvc
@inject IContactService ContactService

@{
  ViewData["Title"] = "Agenda";
  Layout = "_Layout";
}

<h1 id="agenda-title">Agenda</h1>

<div class="agenda">
  <section class="panel master-panel" aria-labelledby="agenda-title">
    <button
      class="primary action-button"
      hx-get="/?handler=New"
      hx-target="#detail"
      hx-swap="innerHTML">
      Agregar
    </button>

    <form id="search-form">
      <label class="visually-hidden" for="search">Buscar contactos</label>
      <input
        id="search"
        name="search"
        type="search"
        placeholder="Buscar por nombre, teléfono o email"
        autocomplete="off"
        hx-get="/?handler=List"
      >
    </form>
  </section>
</div>

```

razor - continuación

```

        hx-trigger="input changed delay:250ms, search"
        hx-include="#search-form, #current-selection"
        hx-vals='{"autoSelect":"true"}'
        hx-target="#master"
        hx-swap="innerHTML" />
    </form>

    <div
        id="master"
        class="list-scroll"
        hx-get="/?handler=List"
        hx-trigger="refresh-master from:body"
        hx-include="#search-form, #current-selection"
        hx-vals='{"autoSelect":"false"}'
        hx-swap="innerHTML">
    </div>

    @* Carga inicial: autoselecciona el primer contacto visible. *@
    <div
        class="visually-hidden"
        hx-get="/?handler=List"
        hx-trigger="load"
        hx-target="#master"
        hx-include="#search-form, #current-selection"
        hx-vals='{"autoSelect":"true"}'
        hx-swap="innerHTML">
    </div>
</section>

<section class="panel">
    <div id="detail" aria-live="polite" aria-atomic="true">
        <div id="current-selection"></div>
        <p>Seleccioná un contacto.</p>
    </div>
</section>
</div>

@functions {

    public IActionResult OnGetList(string? search, int? selectedId, bool autoSelect) {
        List<Contact> contacts = ContactService.GetAll(search);
        int? effectiveSelectedId = contacts.Any(c => c.Id == selectedId) ? selectedId : null;
        int? detailIdToLoad = null;
        bool clearDetail = false;

        if (autoSelect && effectiveSelectedId is null) {
            Contact? firstContact = contacts.FirstOrDefault();

            if (firstContact is null) {
                clearDetail = true;
            } else {
                effectiveSelectedId = firstContact.Id;
                detailIdToLoad = firstContact.Id;
            }
        }

        return Partial("_ContactList", new ContactListViewModel(contacts, effectiveSelectedId,
            detailIdToLoad, clearDetail));
    }

    public IActionResult OnGetNew() {
        TriggerMasterRefresh();
        return Partial("_ContactForm", new ContactInput(0, "", "", "", "", ""));
    }

    public IActionResult OnGetEmpty() {
        return Content("<<div id='current-selection'></div> <p>Seleccioná un contacto.</p>>>",
            "text/html");
    }

    public IActionResult OnGetDetail(int id) {
        Contact? contact = ContactService.GetById(id);

```


razor - continuacion

```

        if (contact is null) { return NotFound(); }
        return Partial("_ContactDetail", contact);
    }

    public IActionResult OnGetEdit(int id) {
        Contact? contact = ContactService.GetById(id);
        if (contact is null) { return NotFound(); }
        return Partial("_ContactForm", ToInput(contact));
    }

    public IActionResult OnPostSave(ContactInput input) {
        Contact savedContact = ContactService.Save(input);
        TriggerMasterRefresh();
        return Partial("_ContactDetail", savedContact);
    }

    public IActionResult OnPostDelete(int id) {
        ContactService.Delete(id);
        TriggerMasterRefresh();
        return Content("""
            <div id="current-selection"></div>
            <p>Contacto eliminado.</p>
            """, "text/html");
    }

    private void TriggerMasterRefresh() {
        // HTMX dispara este evento en el body tras actualizar #detail; #master lo escucha.
        Response.Headers["HX-Trigger-After-Swap"] = "refresh-master";
    }

    private static ContactInput ToInput(Contact contact) => new(
        contact.Id,
        contact.Nombre,
        contact.Apellido,
        contact.Telefono,
        contact.Email,
        contact.Notas
    );
}

```

La página concentra el HTML de la pantalla, la inyección del servicio con `@inject` y los handlers. No contiene el modelo, ni el DTO, ni el acceso a datos: todo eso vive en sus carpetas (secciones 7 y 8).

Dos regiones tienen `id` para que HTMX sepa a cuál apuntar: `#master` (la lista) y `#detail` (ver/crear/editar). Dentro de `#detail` vive siempre `#current-selection`, un contenedor con el input oculto que guarda el `selectedId` del contacto visible.

El buscador, en detalle. `hx-trigger="input changed delay:250ms, search"` dispara cuando el valor cambia (`input changed`), pero espera 250 ms desde la última tecla (`delay:250ms`, un `debounce` que evita una petición por pulsación), y también ante el evento `search` (cuando el usuario limpia el campo con la "x"). `hx-include="#search-form, #current-selection"` manda el texto y, si hay un contacto a la vista, su `id`. `hx-vals='{ "autoSelect": "true" }'` le avisa al servidor que esta petición puede seleccionar el primer resultado. `OnGetList` **resuelve listar, filtrar y auto-seleccionar**. Trae los contactos (con o sin filtro). Si el seleccionado sigue visible, lo conserva. Si `autoSelect` es `true` y no hay selección válida, elige el primero (`DetailIdToLoad`) o, si la búsqueda no devolvió nada, pide limpiar el detalle (`ClearDetail`). Toda la decisión de selección vive en un solo lugar. `TriggerMasterRefresh()` **centraliza la sincronización de la lista**. Los handlers que mutan datos —`OnGetNew` (abre el formulario de alta), `OnPostSave`, `OnPostDelete`— lo llaman antes de devolver su respuesta. Los que solo leen —`OnGetDetail`, `OnGetEdit`— no, porque no cambian la lista. La sección 14 explica cómo ese método refresca `#master`.

13. Los partials

Los partials son fragmentos de HTML reutilizables. Cada handler devuelve uno (o un Content simple) y HTMX lo inserta.

Lista (el panel maestro)

Pages/_ContactList.cshtml:

```

razor
@using AgendaRazorHtmx.Models
@using AgendaRazorHtmx.ViewModels
@model ContactListViewModel

<form class="contact-list">
    <fieldset class="contact-group">
        <legend class="visually-hidden">Contactos</legend>

        @if (Model.Contacts.Count == 0) {
            <p class="muted">No hay contactos para esa búsqueda.</p>
        } else {
            foreach (Contact contact in Model.Contacts) {
                @await Html.PartialAsync(
                    "_ContactListItem",
                    new ContactListItemViewModel(contact, Model.SelectedId == contact.Id)
                )
            }
        }
    </fieldset>
</form>

@if (Model.DetailIdToLoad is not null) {
    <div
        class="visually-hidden"
        hx-get="/?handler=Detail&id=@Model.DetailIdToLoad"
        hx-trigger="load"
        hx-target="#detail"
        hx-include="this"
        hx-swap="innerHTML">
    </div>
} else if (Model.ClearDetail) {
    <div
        class="visually-hidden"
        hx-get="/?handler=Empty"
        hx-trigger="load"
        hx-target="#detail"
        hx-include="this"
        hx-swap="innerHTML">
    </div>
}

```

La lista delega cada ítem a un partial. En vez de escribir el markup del radio inline, llama a `Html.PartialAsync` con un `ContactListItemViewModel`. Cada partial tiene una sola responsabilidad y se compone con los demás. Es el equivalente Razor de descomponer una UI en componentes pequeños. **Los disparadores al final cargan el detalle cuando la búsqueda mueve la selección.** Marcar un radio como checked desde el servidor no dispara su evento change en el navegador. Por eso, cuando `OnGetList` eligió un contacto nuevo (`DetailIdToLoad`) o vació la búsqueda (`ClearDetail`), la lista incluye un elemento oculto con `hx-trigger="load"` que pide el detalle correspondiente apenas entra al DOM. Solo aparece en respuesta a búsquedas que cambian la selección.

Ítem de la lista

Pages/_ContactListItem.cshtml:

```

razor
@using AgendaRazorHtmx.ViewModels
@model ContactListItemViewModel

@{
    string inputId = $"contact-{Model.Contact.Id}";
}

<div
    id="contact-option-@Model.Contact.Id"
    class="contact-option">
    <input
        id="@inputId"
        class="contact-radio"
        type="radio"
        name="selectedId"
        value="@Model.Contact.Id"
        checked="@Model.Selected"
        hx-get="/?handler=Detail&id=@Model.Contact.Id"
        hx-trigger="change"
        hx-target="#detail"
        hx-include="this"
        hx-swap="innerHTML" />

    <label class="contact-item" for="@inputId">
        <strong>@Model.Contact.Apellido, @Model.Contact.Nombre</strong>
        <br />
        <span class="muted">@Model.Contact.Telefono</span>
    </label>
</div>

```

Por qué radios y no enlaces. Cada contacto es un `<input type="radio">` (oculto con CSS) con su `<label>` estilado como tarjeta. Un grupo de radios da, sin código extra, **navegación por teclado**: con las flechas te movés entre opciones, y al cambiar la selección (`hx-trigger="change"`) HTMX pide el detalle. El `<fieldset>` con `<legend>` oculto los agrupa de forma accesible. El radio puede sentirse forzado para una lista, pero es una elección deliberada: el **radio (o checkbox) es el único control nativo de HTML que da selección única más navegación con flechas sin JavaScript**. Las alternativas no encajan con la premisa de cero JS de este proyecto: una lista de `<a>/<button>` con `aria-current` tiene markup más natural pero pierde el movimiento con flechas (quedás con Tab + Enter); el patrón ARIA `role="listbox"` es semánticamente ideal, pero su navegación por teclado **exige JavaScript** (roving tabindex o `aria-activedescendant`). Cuando necesites una lista seleccionable, accesible y navegable por teclado sin escribir JS, los radios te dan la semántica y el teclado gratis. `hx-include="this"` limita lo que se envía al valor del propio radio, sin arrastrar otros campos de la página.

Detalle (solo lectura)

Pages/_ContactDetail.cshtml:

```

razor
@using AgendaRazorHtmx.Models
@model Contact

@{
    Contact contact = Model;
}

<div class="detail-view">
    <div id="current-selection">

```

razor - continuación

```

<input type="hidden" name="selectedId" value="@contact.Id" />
</div>

<div class="detail-header">
  <div>
    <h2>@contact.Nombre @contact.Apellido</h2>
    <p class="detail-phone">@contact.Telefono</p>
  </div>
</div>

<dl class="detail-grid">
  <div class="detail-field">
    <dt>Nombre</dt>
    <dd>@contact.Nombre</dd>
  </div>
  <div class="detail-field">
    <dt>Apellido</dt>
    <dd>@contact.Apellido</dd>
  </div>
  <div class="detail-field">
    <dt>Teléfono</dt>
    <dd>@(string.IsNullOrEmpty(contact.Telefono) ? "Sin teléfono" : contact.Telefono)</dd>
  </div>
  <div class="detail-field">
    <dt>Email</dt>
    <dd>@(string.IsNullOrEmpty(contact.Email) ? "Sin email" : contact.Email)</dd>
  </div>
  <div class="detail-field detail-notes">
    <dt>Notas</dt>
    <dd>@(string.IsNullOrEmpty(contact.Notas) ? "Sin notas" : contact.Notas)</dd>
  </div>
</dl>

<form class="actions" hx-target="#detail" hx-swap="innerHTML">
  @Html.AntiForgeryToken()

  <button
    type="button"
    class="primary action-button"
    hx-get="/?handler=Edit&id=@contact.Id">
    Editar
  </button>

  <button
    type="button"
    class="danger action-button"
    hx-post="/?handler=Delete&id=@contact.Id"
    hx-include="closest form"
    hx-confirm="¿Eliminar este contacto?">
    Eliminar
  </button>
</form>
</div>

```

#current-selection **mantiene al buscador informado**. Mientras el detalle es visible, este contenedor guarda el id del contacto mostrado. El buscador lo incluye (hx-include="#search-form, #current-selection"), así el servidor sabe qué contacto conservar seleccionado al filtrar. El partial del detalle no conoce ni a #master ni al buscador: solo expone su id en un lugar conocido. **El token antiforgery viaja con** hx-include="closest form". El botón "Eliminar" hace POST y necesita el token. Está dentro de un <form> con @Html.AntiForgeryToken(), y hx-include="closest form" incluye los campos de ese formulario (entre ellos el token oculto). hx-confirm muestra un cuadro de confirmación antes de enviar.

Alta y edición (un solo formulario)

Pages/_ContactForm.cshtml:

razor

```

@using AgendaRazorHtmx.Dtos
@model ContactInput

<div class="contact-form-view">
    <div id="current-selection">
        @if (Model.Id > 0) {
            <input type="hidden" name="selectedId" value="@Model.Id" />
        }
    </div>

    <div class="form-header">
        <h2>
            @(Model.Id == 0 ? "Nuevo contacto" : $"Editar {Model.Nombre} {Model.Apellido}")
        </h2>
        <p class="muted">Los campos marcados con <span class="required-mark" aria-hidden="true">*</span>
son obligatorios.</p>
    </div>

    <form
        class="contact-form"
        hx-post="/?handler=Save"
        hx-target="#detail"
        hx-swap="innerHTML">

        @Html.AntiForgeryToken()

        <input type="hidden" name="Id" value="@Model.Id" />

        <div class="form-grid">
            <div class="form-field">
                <label for="Nombre">Nombre <span class="required-mark"
aria-hidden="true">*</span></label>
                <input id="Nombre" name="Nombre" value="@Model.Nombre" autocomplete="given-name"
required autofocus />
            </div>
            <div class="form-field">
                <label for="Apellido">Apellido <span class="required-mark"
aria-hidden="true">*</span></label>
                <input id="Apellido" name="Apellido" value="@Model.Apellido" autocomplete="family-name"
required />
            </div>
            <div class="form-field">
                <label for="Telefono">Teléfono</label>
                <input id="Telefono" name="Telefono" value="@Model.Telefono" autocomplete="tel"
inputmode="tel" />
            </div>
            <div class="form-field">
                <label for="Email">Email</label>
                <input id="Email" name="Email" type="email" value="@Model.Email" autocomplete="email"
inputmode="email" />
            </div>
            <div class="form-field form-field-wide">
                <label for="Notas">Notas</label>
                <textarea id="Notas" name="Notas">@Model.Notas</textarea>
            </div>
        </div>

        <div class="actions form-actions">
            <button class="primary action-button" type="submit">Guardar</button>

            @if (Model.Id == 0) {
                <button
                    type="button"
                    class="action-button"
                    hx-get="/?handler=Empty"
                    hx-target="#detail"
                    hx-swap="innerHTML">
                    Cancelar
                </button>
            } else {
                <button
                    type="button"

```

razor - continuación

```

        class="action-button"
        hx-get="/?handler=Detail&id=@Model.Id"
        hx-target="#detail"
        hx-swap="innerHTML">
        Cancelar
    </button>
    }
</div>
</form>
</div>

```

Un formulario para crear y editar. El campo oculto Id decide: 0 al crear, el id real al editar (lo prepara OnGetEdit con ToInput). El título y el botón Cancelar se adaptan con un @if. Al enviar, OnPostSave recibe el ContactInput por model binding y el servicio inserta o actualiza según el Id. **Mejoras de formulario que valen en cualquier proyecto.** required y autofocus en Nombre; autocomplete e inputmode apropiados por campo, para que el teclado del móvil y el autocompletado del navegador ayuden; el asterisco marcando los obligatorios. En edición, #current-selection guarda el id editado para que el buscador lo respete.

14. La técnica clave: eventos del servidor para refrescar otra zona

Acá está el corazón del tutorial. Cuando **guardás** o **borrás**, cambian *dos* regiones a la vez:

- Al guardar: #detail muestra el contacto, y #master puede haber cambiado (nombre nuevo, orden distinto, un contacto más).
- Al borrar: #detail muestra "Contacto eliminado", y el contacto debe desaparecer de #master.

Pero una petición HTMX apunta a **un solo** elemento (hx-target). ¿Cómo actualizamos el otro?

El mecanismo: el servidor emite un evento

HTMX define un canal para esto. Si una respuesta HTTP trae la cabecera HX-Trigger-After-Swap: nombre-del-evento, HTMX —después de insertar el HTML en el DOM— **dispara un evento del DOM con ese nombre** en el <body>. Cualquier elemento que escuche ese evento reacciona.

Son dos piezas que encajan:

1. El servidor emite el evento desde cualquier handler que mute datos:

```

csharp
private void TriggerMasterRefresh() {
    Response.Headers["HX-Trigger-After-Swap"] = "refresh-master";
}

```

2. El elemento que debe refrescarse escucha el evento con hx-trigger="evento from:body":

```
html
<div
  id="master"
  hx-get="/?handler=List"
  hx-trigger="refresh-master from:body"
  hx-include="#search-form, #current-selection"
  hx-vals='{ "autoSelect": "false" }'
  hx-swap="innerHTML">
</div>
```

El `from:body` indica que el listener escuche el evento en el `<body>` (donde HTMX lo emite), no solo en el propio elemento. Cuando llega `refresh-master`, `#master` hace su GET `/?handler=List` y se reconstruye, incluyendo el filtro actual y el `selectedId` de `#current-selection`. Usa `autoSelect: false` porque la selección ya la determina el id que viaja en `#current-selection`.

Por qué esta técnica es valiosa

Desacopla quién responde de qué se actualiza. El partial que devuelve un handler (`_ContactDetail`, el mensaje de borrado) no necesita saber que existe `#master`. Solo el handler, que tiene la vista de conjunto, decide "esto fue una mutación, refrescá la lista". Si mañana renombrás `#master` o cambiás su URL, tocás un solo lugar. **Centraliza la regla de sincronización.** "Cuándo refrescar la lista" vive en un único método, `TriggerMasterRefresh()`. Agregar una nueva mutación es una línea: llamarlo. La lógica no se dispersa por las plantillas. **Es un patrón general de HTMX.** Cualquier app donde una acción deba actualizar regiones distintas a la apuntada puede usar `HX-Trigger` (o `HX-Trigger-After-Swap`) para emitir eventos y dejar que cada región se suscriba. Un evento, varios oyentes: el mismo modelo de los eventos del navegador, manejado desde el servidor.

La carga inicial

`#master` escucha un evento, así que no se llena solo al cargar la página. De eso se encarga un `<div>` aparte con `hx-trigger="load"`, que se ejecuta una vez al entrar al DOM y manda `autoSelect: true` para seleccionar el primer contacto y mostrar su detalle:

```
html
@* Carga inicial: autoselecciona el primer contacto visible. *@
<div
  class="visually-hidden"
  hx-get="/?handler=List"
  hx-trigger="load"
  hx-target="#master"
  hx-include="#search-form, #current-selection"
  hx-vals='{ "autoSelect": "true" }'
  hx-swap="innerHTML">
</div>
```

Requisito de la técnica. El elemento que escucha (`#master`) debe existir en el DOM cuando el evento se dispara. Acá siempre existe porque es parte del HTML estático de `Index.cshtml`. Si un elemento que escucha eventos pudiera ser reemplazado por otra operación, perdería su listener: tenelo presente al elegir qué región escucha.

15. Los dos flujos de mutación

Con la técnica clara, los dos flujos que cambian datos se leen de un vistazo. En ambos, la respuesta actualiza `#detail` y el evento `refresh-master` actualiza `#master`.

Guardar (alta o edición)

texto

Enviás el formulario → `hx-post /?handler=Save`



`OnPostSave(input)`

```

■■ ContactService.Save(input)  guarda en SQLite (Id==0 inserta; si no, actualiza)
■■ TriggerMasterRefresh()      cabecera HX-Trigger-After-Swap: refresh-master
■■ devuelve _ContactDetail

```



HTMX inserta el detalle en `#detail` (`#current-selection` queda con el id guardado) y dispara `refresh-master`



`#master` pide la lista y queda con el contacto recién guardado seleccionado

Borrar

texto

Clic en "Eliminar" → `hx-confirm` → `hx-post /?handler=Delete&id=...`



`OnPostDelete(id)`

```

■■ ContactService.Delete(id)
■■ TriggerMasterRefresh()      cabecera HX-Trigger-After-Swap: refresh-master
■■ devuelve el mensaje + #current-selection vacío

```



HTMX inserta el mensaje en `#detail` y dispara `refresh-master`



`#master` pide la lista; el contacto ya no está

Abrir el formulario de alta sigue el mismo patrón: `OnGetNew()` llama a `TriggerMasterRefresh()` antes de devolver `_ContactForm`, así la lista se refresca y queda sin selección mientras creás un contacto.

16. Probar la aplicación

Levantá la app:

bash

```
dotnet run
```

Abrí la URL que muestre la consola (por ejemplo `https://localhost:7xxx`) y recorré todo:

1. **Listar.** Al cargar, ves los contactos ordenados por apellido, con el primero seleccionado y su detalle visible.
2. **Buscar.** Escribí "ana" o "gómez": la lista se filtra tras una pausa breve, sin recargar la página.

3. **Navegar con teclado.** Hacé foco en la lista y movete con las flechas: el detalle cambia con la selección.
4. **Ver detalle.** Clic en un contacto: el panel derecho muestra sus datos.
5. **Crear.** "Agregar", completá el formulario, "Guardar": el detalle muestra el contacto nuevo **y** aparece en la lista.
6. **Editar.** Desde un detalle, "Editar", cambiá algo, "Guardar": vuelve al detalle actualizado y la lista se refresca con ese contacto seleccionado.
7. **Borrar.** Desde un detalle, "Eliminar", confirmá: el contacto desaparece de la lista.

***Si la página aparece sin estilos:** verificá que `Index.cshtml` tenga `Layout = "_Layout"`; y que exista `wwwroot/css/site.css`. Sin el layout no se cargan ni los estilos ni el script de HTMX. **Si un POST (guardar, borrar) devuelve 400:** casi seguro falta el token antiforgery. Confirmá que el formulario tenga `@Html.AntiForgeryToken()`. En el guardado, `hx-post` está en el `<form>`; en el borrado, el botón usa `hx-include="closest form"`. **Si la lista no se refresca al guardar o borrar:** verificá que `TriggerMasterRefresh()` se llame en ese handler y que `#master` tenga `hx-trigger="refresh-master from:body"`. El nombre del evento en la cabecera y en el `hx-trigger` debe coincidir exactamente. **Los datos persisten.** Al frenar y volver a correr `dotnet run`, los contactos que creaste siguen ahí (viven en `agenda.db`). Para volver al estado inicial, borrá `agenda.db` y arrancá de nuevo.*

CAPITULO 44

Apunte de clase: Componentes en Blazor desde primeros principios

Objetivo

Entender cómo se construyen componentes en Blazor partiendo de un único modelo mental:

```
texto
estado + plantilla = interfaz renderizada
```

Vamos a usar **una sola aplicación como hilo conductor**: una agenda de contactos. Cada concepto nuevo agrega una pieza a esa misma app, en vez de empezar un ejemplo distinto cada vez.

Temas:

- Qué es un componente y cómo crear archivos `.razor`.
- Parámetros con `[Parameter]` (padre → hijo).
- Componentes anidados y reutilización del modelo de datos.
- `ChildContent` y `RenderFragment` (contenido interno).
- Cascading parameters (datos transversales).
- Estilos: globales, aislados y parametrizados.
- Data binding con `@bind`.
- Comunicación hijo → padre con `EventCallback<T>`.
- Two-way binding entre componentes propios.
- Métodos del ciclo de vida.
- Ejemplo integrador maestro-detalle.

Un componente combina markup Razor, estado, código C#, parámetros de entrada, eventos y, opcionalmente, estilos propios.

La idea central es una sola:

```
texto
UI = f(estado)
```

El ciclo es:

```

texto
estado + plantilla Razor
↓
Blazor construye la UI
↓
el usuario hace algo (click, input, submit)
↓
cambia el estado
↓
Blazor vuelve a renderizar solo lo necesario

```

No es magia: es una función con buena prensa. Todo lo que sigue son distintas formas de mover datos hacia ese estado y hacia la plantilla.

2. El modelo de datos compartido

Antes de crear componentes, definimos el dato que toda la app va a manejar. En una aplicación real **el modelo vive en un solo lugar**, no duplicado dentro de cada componente.

```

csharp
// Models/Contact.cs

namespace AgendaBlazor.Models;

public record Contact(
    int Id,
    string FullName,
    string Email,
    string Phone
);

```

Cualquier componente que lo use solo necesita:

```

razor
@using AgendaBlazor.Models

```

A lo largo del apunte asumimos que este using está disponible (por ejemplo, agregado en `_Imports.razor`).

3. Crear un componente mínimo

Un componente vive en un archivo `.razor` y tiene dos partes: la visual y la lógica.

```

razor
<!-- Components/ContactCard.razor -->

<article class="card">
    <h3>@Contact.FullName</h3>
    <p>@Contact.Email</p>
    <p>@Contact.Phone</p>
</article>

@code {

```

razor - continuación

```
[Parameter]
public Contact Contact { get; set; } = default!;
}
```

La anotación `[Parameter]` significa que esa propiedad **puede recibir un valor desde afuera**. Se usa así:

razor

```
<ContactCard Contact="ana" />
```

Conceptualmente es como llamar a una función, pero en lugar de devolver un valor, produce UI:

csharp

```
ContactCard(Contact: ana) // → markup renderizado
```

Se puede recibir el contacto entero (Contact) o campos sueltos (FullName, Email...). Pasar el objeto suele ser más limpio cuando van juntos.

La regla más importante de Blazor aparece ya acá:

texto

El padre suele ser dueño del estado.
El hijo recibe datos y renderiza.

4. Componentes anidados

Un componente puede usar otros adentro (*nested components*). Nuestra agenda tendrá esta jerarquía:

texto

```
Contacts.razor
  ■■■ ContactList.razor
    ■■■ ContactCard.razor (x N)
```

La página dueña de los datos:

razor

```
<!-- Pages/Contacts.razor -->

@page "/contacts"

<h1>Agenda</h1>

<ContactList Contacts="contacts" />

@code {
    private List<Contact> contacts =
    [
        new(1, "Ana Gómez", "ana@email.com", "3815551111"),
        new(2, "Luis Díaz", "luis@email.com", "3815552222"),
        new(3, "María Ruiz", "maria@email.com", "3815553333")
    ];
}
```

La lista, que recibe los datos y crea una tarjeta por contacto:

```

razor
<!-- Components/ContactList.razor -->

<ul>
    @foreach (var contact in Contacts) {
        <li @key="contact.Id">
            <ContactCard Contact="contact" />
        </li>
    }
</ul>

@code {
    [Parameter]
    public IReadOnlyList<Contact> Contacts { get; set; } = [];
}

```

@key="contact.Id" ayuda a Blazor a identificar cada elemento de la lista entre renders, lo que evita reordenamientos incorrectos.

El flujo de datos siempre es **padre** → **hijo**: el padre tiene el estado, el hijo lo muestra.

5. ChildContent: componentes que reciben contenido interno

A veces queremos un componente que envuelva contenido escrito por quien lo usa:

```

razor
<Panel Title="Datos del contacto">
    <p>Nombre: Ana Gómez</p>
    <p>Teléfono: 3815551111</p>
</Panel>

```

Para lograrlo, el componente expone una propiedad ChildContent de tipo RenderFragment:

```

razor
<!-- Components/Panel.razor -->

<section class="panel">
    <header><h2>@Title</h2></header>
    <main>@ChildContent</main>
</section>

@code {
    [Parameter]
    public string Title { get; set; } = "";

    [Parameter]
    public RenderFragment? ChildContent { get; set; }
}

```

texto

RenderFragment = un pedazo de interfaz que otro componente puede renderizar

Esto permite componentes estructurales reutilizables:

razor

```
<Modal Title="Editar contacto">
  <ContactEditor Contact="selectedContact" />
</Modal>
```

6. Cascading parameters: datos transversales

Imaginá que muchos componentes necesitan el mismo dato (el tema visual):

texto

ContactsPage → ContactList → ContactCard

Pasarlo manualmente nivel por nivel (Theme="@Theme" en cada uno) produce **parameter drilling**: componentes intermedios reciben parámetros que no usan, solo para reenviarlos.

La solución: un ancestro **publica** un valor y cualquier descendiente lo **consume** sin que los intermedios lo toquen.

El valor a compartir:

csharp

```
// Models/AppTheme.cs

namespace AgendaBlazor.Models;

public class AppTheme {
    public string Mode { get; set; } = "light";
    public string AccentColor { get; set; } = "#2563eb";
}
```

El ancestro publica con <CascadingValue>:

razor

```
@page "/contacts"

<CascadingValue Value="theme">
  <h1>Agenda</h1>
  <ContactList Contacts="contacts" />
</CascadingValue>

@code {
    private AppTheme theme = new() { Mode = "dark", AccentColor = "#7c3aed" };

    private List<Contact> contacts =
    [
        new(1, "Ana Gómez", "ana@email.com", "3815551111"),
        new(2, "Luis Díaz", "luis@email.com", "3815552222")
    ];
}
```

El descendiente consume con [CascadingParameter] (sin que ContactList haga nada):

```

razor
<!-- Components/ContactCard.razor -->

<article class="card" style="border-color:@Theme?.AccentColor">
  <h3>@Contact.FullName</h3>
  <p>@Contact.Email</p>
  <small>Tema: @Theme?.Mode</small>
</article>

@code {
  [Parameter]
  public Contact Contact { get; set; } = default!;

  [CascadingParameter]
  public AppTheme? Theme { get; set; }
}

```

Resumen:

```

texto
[Parameter]          → dato explícito del padre al hijo directo
[CascadingParameter] → dato compartido de un ancestro a sus descendientes

```

Conviene para datos transversales: tema visual, usuario actual, idioma/cultura, estado global de layout. **No** para datos comunes: si todo es cascading, la app termina pareciendo una búsqueda del tesoro con bugs incluidos.

7. Estilos de componentes

7.1 CSS global

En `wwwroot/app.css`:

```

css
.card {
  border: 1px solid #ddd;
  border-radius: 12px;
  padding: 1rem;
}

```

Afecta a toda la app. Sirve para body, tipografía, layout principal, botones base, variables CSS y utilitarios comunes.

7.2 CSS aislado por componente

Blazor soporta **CSS isolation**: un archivo `.razor.css` con el mismo nombre que el componente.

```

texto
Components/
  ■■■ ContactCard.razor
  ■■■ ContactCard.razor.css

```

```

CSS
/* Components/ContactCard.razor.css */
.card {
    border: 1px solid #ddd;
    border-radius: 12px;
    padding: 1rem;
    background: white;
}

.card h3 { margin: 0 0 .5rem; }
.card p { margin: .25rem 0; }

```

Los estilos quedan asociados al componente y se reducen las colisiones con el resto de la app.

7.3 Estilos parametrizados

A veces el padre necesita pasar una clase o estilo:

```

razor
<article class="card @Class" style="@Style">
    <h3>@Contact.FullName</h3>
</article>

@code {
    [Parameter] public Contact Contact { get; set; } = default!;
    [Parameter] public string? Class { get; set; }
    [Parameter] public string? Style { get; set; }
}

```

Uso:

```

razor
<ContactCard Contact="ana" Class="selected" />

```

No abusos de Style inline: si cada componente recibe CSS suelto, la app empieza a parecer una mudanza sin cajas etiquetadas.

8. Data binding

Data binding es vincular un dato C# con un elemento visual. Se usa la directiva @bind:

```

razor
<input @bind="name" />
<p>Hola, @name</p>

@code {
    private string name = "";
}

```

Cuando cambia el input cambia la variable; cuando cambia la variable cambia la UI.

Actualizar en cada tecla. Por defecto muchos inputs actualizan con onchange (al perder el foco). Para reaccionar en cada tecla:

razor

```
<input @bind="name" @bind:event="oninput" />
```

Binding con objetos. Funciona igual sobre propiedades de un objeto:

razor

```
<input @bind="contact.FullName" @bind:event="oninput" />
<input @bind="contact.Email" @bind:event="oninput" />

<ContactCard Contact="contact" /> <!-- vista previa en vivo -->

@code {
    private Contact contact = new(1, "Ana Gómez", "ana@email.com", "3815551111");
}
```

El flujo: el input cambia → cambia `contact.FullName` → Blazor renderiza → `ContactCard` recibe el nuevo valor.

`@bind:get` y `@bind:set`. Para controlar lectura y escritura por separado (por ejemplo, limitar a 20 caracteres):

razor

```
<input @bind:get="name" @bind:set="SetName" @bind:event="oninput" />

@code {
    private string name = "";

    private void SetName(string value) =>
        name = value.Length <= 20 ? value : value[..20];
}
```

texto

`@bind:get` → de dónde sale el valor actual
`@bind:set` → qué hago cuando el usuario lo cambia

9. Comunicación entre componentes

Las cuatro vías, que ya vimos por separado:

texto

```
Padre → Hijo: [Parameter]
Hijo → Padre: EventCallback<T>
Ancestro → Descendientes: [CascadingParameter]
Input ↔ Estado: @bind
```

Falta la que va de **hijo a padre**.

9.1 Hijo → padre con `EventCallback<T>`

Cuando el hijo quiere avisarle algo al padre (por ejemplo, "me seleccionaron"), recibe un callback como parámetro y lo invoca:

```

razor
<!-- Components/ContactListItem.razor -->

<li>
    <button @onclick="Select">@Contact.FullName</button>
</li>

@code {
    [Parameter]
    public Contact Contact { get; set; } = default!;

    [Parameter]
    public EventCallback<Contact> OnSelected { get; set; }

    private async Task Select() => await OnSelected.InvokeAsync(Contact);
}

```

El padre pasa el dato y el callback, y reacciona:

```

razor
@page "/communication-demo"

<ul>
    @foreach (var contact in contacts) {
        <ContactListItem Contact="@contact" OnSelected="HandleSelected" />
    }
</ul>

@if (selectedContact is not null) {
    <ContactCard Contact="@selectedContact" />
}

@code {
    private Contact? selectedContact;

    private List<Contact> contacts =
    [
        new(1, "Ana Gómez", "ana@email.com", "3815551111"),
        new(2, "Luis Díaz", "luis@email.com", "3815552222")
    ];

    private void HandleSelected(Contact contact) => selectedContact = contact;
}

```

Flujo:

```

texto
Padre pasa datos + callback → usuario hace click en el hijo
→ el hijo invoca el callback → el padre actualiza su estado → Blazor re-renderiza

```

Este patrón es central: **el hijo avisa, el padre decide.**

9.2 Two-way binding entre componentes propios

@bind también funciona con componentes propios. La convención es exponer una propiedad X y un EventCallback<T> llamado XChanged:

```

razor
<!-- Components/SearchBox.razor -->

<input value="@Value" @oninput="HandleInput" placeholder="@Placeholder" />

@code {
    [Parameter]
    public string Value { get; set; } = "";

    [Parameter]
    public EventCallback<string> ValueChanged { get; set; }

    [Parameter]
    public string Placeholder { get; set; } = "Buscar...";

    private async Task HandleInput(ChangeEventArgs e) =>
        await ValueChanged.InvokeAsync(e.Value?.ToString() ?? "");
}

```

Gracias al par Value / ValueChanged, se usa así:

```

razor
<SearchBox @bind-Value="searchText" Placeholder="Buscar contacto..." />

```

Que es equivalente a escribir a mano:

```

razor
<SearchBox Value="@searchText"
    ValueChanged="value => searchText = value" />

```

10. Métodos del ciclo de vida

Un componente nace, recibe parámetros, se renderiza y muere. Blazor permite engancharse en cada momento. Los métodos principales:

```

csharp
OnInitialized() / OnInitializedAsync()    // nace
OnParametersSet() / OnParametersSetAsync() // recibe o cambian parámetros
ShouldRender()                          // ¿debe re-renderizar?
OnAfterRender(bool firstRender) / ...Async // ya se renderizó
Dispose() / DisposeAsync()               // se limpia al morir

```

OnInitialized / OnInitializedAsync — inicializar estado al nacer. La versión async sirve para cargar datos:

```

razor
@if (contacts is null) {
    <p>Cargando...</p>
} else {
    <ul>@foreach (var c in contacts) { <li>@c.FullName</li> }</ul>
}

@code {
    private List<Contact>? contacts;

    protected override async Task OnInitializedAsync() {
        // En una app real: contacts = await ContactService.GetAllAsync();
    }
}

```

razor - continuacion

```

    await Task.Delay(500);
    contacts = [ new(1, "Ana Gómez", "ana@email.com", "3815551111") ];
}
}

```

OnParametersSet — recalculan estado derivado cuando llegan o cambian parámetros:

razor

```

<h3>@DisplayName</h3>

@code {
    [Parameter] public string FirstName { get; set; } = "";
    [Parameter] public string LastName { get; set; } = "";

    private string DisplayName { get; set; } = "";

    protected override void OnParametersSet() =>
        DisplayName = $"{LastName}, {FirstName}";
}

```

ShouldRender — decide si re-renderizar. Casi nunca hace falta; es optimización fina. Si lo usás sin medir, estás afinando un violín con una llave inglesa.

OnAfterRender(bool firstRender) — corre después del render, cuando el DOM ya existe. Sirve para interactuar con JavaScript, medir elementos, o inicializar librerías externas:

razor

```

@code {
    protected override void OnAfterRender(bool firstRender) {
        if (firstRender)
            Console.WriteLine("Primer render completo");
    }
}

```

texto

```

firstRender == true → primera renderización
firstRender == false → renderizaciones posteriores

```

Dispose — limpiar recursos. Si creás timers, suscripciones o recursos externos, tenés que liberarlos:

razor

```

@implements IDisposable

<p>Hora: @currentTime</p>

@code {
    private Timer? timer;
    private string currentTime = "";

    protected override void OnInitialized() {
        timer = new Timer(_ => {
            currentTime = DateTime.Now.ToLongTimeString();
            InvokeAsync(StateHasChanged);
        }, null, 0, 1000);
    }

    public void Dispose() => timer?.Dispose();
}

```

Si no limpiás, el componente desaparece de la pantalla pero deja procesos vivos: fantasmas con consumo de RAM.

11. Ejemplo integrador: agenda maestro-detalle

Ahora unimos todo en una mini agenda con búsqueda, lista seleccionable y panel de detalle.

```
texto
Components/
  ■■■ SearchBox.razor      (ya lo tenemos: §9.2)
  ■■■ ContactList.razor   (selección con EventCallback)
  ■■■ ContactDetails.razor (panel de detalle)
Pages/
  ■■■ Contacts.razor      (dueña del estado)
```

11.1 ContactList con selección

```
razor
<!-- Components/ContactList.razor -->

<ul class="contact-list">
  @foreach (var contact in Contacts) {
    <li @key="contact.Id">
      <button
        class="contact-list-item @(contact.Id == SelectedContactId ? "selected" : "")"
        onclick="() => OnSelected.InvokeAsync(contact)">
        @contact.FullName
      </button>
    </li>
  }
</ul>

@code {
  [Parameter] public IReadOnlyList<Contact> Contacts { get; set; } = [];
  [Parameter] public int? SelectedContactId { get; set; }
  [Parameter] public EventCallback<Contact> OnSelected { get; set; }
}
```

11.2 ContactDetails (puramente visual)

```
razor
<!-- Components/ContactDetails.razor -->

@if (Contact is null) {
  <section class="empty-details"><p>Seleccioná un contacto.</p></section>
} else {
  <section class="contact-details">
    <h2>@Contact.FullName</h2>
    <p><strong>Email:</strong> @Contact.Email</p>
    <p><strong>Teléfono:</strong> @Contact.Phone</p>
  </section>
}

@code {
  [Parameter] public Contact? Contact { get; set; }
}
```

11.3 Página Contacts (orquesta todo)

```

razor
<!-- Pages/Contacts.razor -->

@page "/"contacts"

<h1>Agenda</h1>

<div class="layout">
    <aside>
        <SearchBox @bind-Value="searchText" Placeholder="Buscar contacto..." />
        <ContactList
            Contacts="filteredContacts"
            SelectedContactId="selectedContact?.Id"
            OnSelected="SelectContact" />
    </aside>

    <main>
        <ContactDetails Contact="selectedContact" />
    </main>
</div>

@code {
    private string searchText = "";
    private Contact? selectedContact;

    private List<Contact> contacts =
    [
        new(1, "Ana Gómez", "ana@email.com", "3815551111"),
        new(2, "Luis Díaz", "luis@email.com", "3815552222"),
        new(3, "María Ruiz", "maria@email.com", "3815553333"),
        new(4, "Carlos Medina", "carlos@email.com", "3815554444")
    ];

    private IReadOnlyList<Contact> filteredContacts =>
        contacts
            .Where(c =>
                c.FullName.Contains(searchText, StringComparison.OrdinalIgnoreCase) ||
                c.Email.Contains(searchText, StringComparison.OrdinalIgnoreCase) ||
                c.Phone.Contains(searchText, StringComparison.OrdinalIgnoreCase))
            .ToList();

    protected override void OnInitialized() =>
        selectedContact = contacts.FirstOrDefault();

    private void SelectContact(Contact contact) => selectedContact = contact;
}

```

Cada concepto del apunte aparece acá:

```

texto
Anidación:      Contacts → SearchBox / ContactList / ContactDetails
Padre → hijo:   Contacts pasa Contacts, SelectedContactId, Contact
Hijo → padre:   ContactList avisa con OnSelected
Two-way binding: SearchBox con @bind-Value
Lifecycle:      OnInitialized elige el contacto inicial
Estado:         searchText y selectedContact
Estado derivado: filteredContacts (se recalcula al escribir)

```

11.4 Estilos aislados

CSS

```
/* Pages/Contacts.razor.css */
.layout { display: grid; grid-template-columns: 320px 1fr; gap: 1rem; }
aside { border-right: 1px solid #ddd; padding-right: 1rem; }
main { padding-left: 1rem; }
```

CSS

```
/* Components/SearchBox.razor.css */
.search-box {
    width: 100%; padding: .75rem;
    border: 1px solid #ccc; border-radius: .75rem;
    margin-bottom: 1rem;
}
```

CSS

```
/* Components/ContactList.razor.css */
.contact-list { list-style: none; padding: 0; margin: 0; }
.contact-list-item {
    width: 100%; text-align: left; border: 0;
    padding: .75rem; border-radius: .75rem;
    background: transparent; cursor: pointer;
}
.contact-list-item:hover { background: #f3f4f6; }
.contact-list-item.selected { background: #e0ecff; font-weight: 600; }
```

CSS

```
/* Components/ContactDetails.razor.css */
.contact-details { border: 1px solid #ddd; border-radius: 1rem; padding: 1rem; }
.empty-details { color: #777; font-style: italic; }
```

(Para que `.search-box` aplique, agregá `class="search-box"` al `<input>` de `SearchBox.razor`.)

12. Cómo diseñar un componente

Antes de escribirlo, respondé:

texto

1. ¿Qué muestra?
2. ¿Qué datos necesita?
3. ¿Quién es dueño del estado?
4. ¿Tiene estado interno o todo viene de afuera?
5. ¿Debe avisar eventos al padre?
6. ¿Debe recibir contenido interno?
7. ¿Sus estilos son propios o globales?

Aplicado a `ContactCard`:

texto

- Muestra un contacto.
- Recibe `Contact` desde afuera.
- No carga datos por su cuenta ni decide la selección.
- Puede avisar `OnSelected`.
- Puede tener CSS aislado.

13. Regla de oro

texto

El padre posee el estado.
El hijo muestra datos.
El hijo avisa eventos.
El padre decide qué hacer.

Las cuatro vías de comunicación:

texto

Padre → Hijo: [Parameter]
Hijo → Padre: EventCallback<T>
Ancestro → Descendientes: [CascadingParameter]
Input ↔ Estado: @bind

El corazón conceptual es: **estado, parámetros, eventos, renderizado y composición**. Lo que sigue (todo construido sobre esto) sería: formularios con `EditForm` y validaciones, servicios inyectados, render modes, componentes genéricos, templates con `RenderFragment<T>`, optimización de renderizado y JS interop.

14. Referencias oficiales

- Componentes: <https://learn.microsoft.com/en-us/aspnet/core/blazor/components/>
- Cascading values and parameters: <https://learn.microsoft.com/en-us/aspnet/core/blazor/components/cascading-values-and-parameters>
- CSS isolation: <https://learn.microsoft.com/en-us/aspnet/core/blazor/components/css-isolation>
- Data binding: <https://learn.microsoft.com/en-us/aspnet/core/blazor/components/data-binding>
- Event handling: <https://learn.microsoft.com/en-us/aspnet/core/blazor/components/event-handling>
- Lifecycle: <https://learn.microsoft.com/en-us/aspnet/core/blazor/components/lifecycle>

CAPITULO 45

EditForm en Blazor

Programación III — Tecnicatura Universitaria en Programación

1. ¿Por qué existe EditForm?

Antes de aprender la sintaxis, pensemos el problema que resuelve.

Un formulario web "crudo" en HTML te da un `<form>` con inputs, pero vos tenés que resolver a mano todo lo demás:

- Leer cada valor del input y meterlo en tu objeto.
- Validar que el email tenga formato de email, que la edad sea un número, que el nombre no esté vacío.
- Mostrar los mensajes de error al lado del campo correcto.
- Evitar que el formulario se mande si hay errores.
- Volver a poblar los inputs cuando editás un registro existente.

Eso es mucho código repetitivo y propenso a errores. `EditForm` es el componente de Blazor que te da todo eso **enganchado a un objeto C#**: vos le pasás un modelo, decorás las propiedades con atributos de validación, y Blazor se encarga del binding, la validación y el flujo de envío.

La idea central: **el formulario está atado a un objeto del dominio, no a strings sueltos**. Vos pensás en términos de Contacto, no de "el valor del input nombre".

2. Conceptos centrales

`EditForm` se apoya en tres piezas que conviene tener claras desde el arranque.

2.1. El modelo (Model)

Es el objeto C# que el formulario edita. Cada propiedad pública se puede atar a un input. La validación vive acá, como atributos sobre las propiedades (Data Annotations):

```
csharp
using System.ComponentModel.DataAnnotations;

public class Contacto {
    [Required(ErrorMessage = "El nombre es obligatorio")]
    [StringLength(50, ErrorMessage = "Máximo 50 caracteres")]
    public string Nombre { get; set; } = "";

    [Required(ErrorMessage = "El email es obligatorio")]
```

csharp - continuación

```
[EmailAddress(ErrorMessage = "Email inválido")]
public string Email { get; set; } = "";

[Range(0, 120, ErrorMessage = "Edad entre 0 y 120")]
public int Edad { get; set; }
}
```

2.2. El EditContext

Acá está la parte que casi nadie explica y que conviene entender, porque es lo que hace que el formulario "reaccione".

Cuando le pasás un `Model` a `EditForm`, internamente Blazor crea un `EditContext`. El `EditContext` es un objeto que **envuelve al modelo y lleva la cuenta del estado del formulario**: qué campos fueron tocados, qué campos son válidos, si hubo algún cambio. Es el cerebro que coordina todo.

Tenés dos formas de trabajar:

- Pasarle `Model` y dejar que Blazor cree el `EditContext` solo (lo más común).
- Crear vos el `EditContext` y pasárselo con el parámetro `EditContext` (cuando necesitás control fino: validar a mano, escuchar cambios de campo, etc.).

No pases los dos a la vez. O `Model`, o `EditContext`, nunca ambos.

2.3. Los componentes de input

Blazor trae inputs propios que se enganchan al `EditContext`: `InputText`, `InputNumber`, `InputCheckbox`, `InputSelect`, `InputDate`, `InputTextArea`, etc. Cada uno se ata a una propiedad del modelo con `@bind-Value`:

razor

```
<InputText @bind-Value="contacto.Nombre" />
```

Esto es binding bidireccional: si el usuario escribe, la propiedad se actualiza; si la propiedad cambia desde código, el input se actualiza. La diferencia con un `<input>` común es que estos avisan al `EditContext` cuándo cambiaron, y por eso la validación funciona.

3. Dinámica de funcionamiento

Veamos el flujo completo, paso a paso, desde que se renderiza hasta que se envía.

1. **Render inicial.** `EditForm` recibe el `Model`, crea el `EditContext` y renderiza los inputs. Cada `InputText`/`InputNumber` se registra contra el `EditContext` y muestra el valor actual de su propiedad.
1. **El usuario escribe.** Al cambiar un input, `@bind-Value` actualiza la propiedad del modelo y el input le avisa al `EditContext` que ese campo se modificó. El `EditContext` lo marca como "tocado".

1. **Validación.** Esto lo dispara el `<DataAnnotationsValidator />` (un componente que tenés que poner dentro del form). Él escucha al `EditContext` y, cuando un campo cambia o cuando se intenta enviar, corre los atributos de validación y guarda los resultados.
1. **Feedback visual.** `<ValidationMessage For="..." />` muestra el mensaje de error de un campo puntual; `<ValidationSummary />` muestra todos juntos. Además, los inputs reciben clases CSS (`valid / invalid, modified`) que podés estilar.
1. **Envío.** Cuando el usuario aprieta el botón submit, `EditForm` revalida todo y dispara **uno** de tres callbacks:
 - `OnValidSubmit` — el modelo pasó todas las validaciones.
 - `OnInvalidSubmit` — hubo errores.
 - `OnSubmit` — se dispara siempre, y vos validás a mano (no uses este junto con los otros dos).

En la práctica, el 90% de las veces usás `OnValidSubmit` y listo: si llega ahí, el modelo es válido.

4. Ejemplo completo

Formulario de alta de contacto. Notá cómo el componente solo se preocupa por el modelo y el callback; toda la maquinaria de validación está declarada.

```

razor
@page "/contacto-nuevo"

@using System.ComponentModel.DataAnnotations

<h3>Nuevo contacto</h3>

<EditForm Model="this.contacto" OnValidSubmit="this.Guardar">
  <DataAnnotationsValidator />
  <ValidationSummary />

  <div class="mb-3">
    <label class="form-label">Nombre</label>
    <InputText class="form-control" @bind-Value="this.contacto.Nombre" />
    <ValidationMessage For="@(() => this.contacto.Nombre)" />
  </div>

  <div class="mb-3">
    <label class="form-label">Email</label>
    <InputText class="form-control" @bind-Value="this.contacto.Email" />
    <ValidationMessage For="@(() => this.contacto.Email)" />
  </div>

  <div class="mb-3">
    <label class="form-label">Edad</label>
    <InputNumber class="form-control" @bind-Value="this.contacto.Edad" />
    <ValidationMessage For="@(() => this.contacto.Edad)" />
  </div>

  <button type="submit" class="btn btn-primary">Guardar</button>
</EditForm>

@if (this.guardado) {
  <div class="alert alert-success mt-3">
    Contacto guardado: @this.contacto.Nombre (@this.contacto.Email)
  </div>
}

```

razor - continuacion

```

@code {
    private Contacto contacto = new();
    private bool guardado = false;

    private void Guardar() {
        // Si llegamos acá, el modelo ya pasó todas las validaciones.
        this.guardado = true;
        // Acá iría la llamada al repositorio / DbContext.
    }

    public class Contacto {
        [Required(ErrorMessage = "El nombre es obligatorio")]
        [StringLength(50, ErrorMessage = "Máximo 50 caracteres")]
        public string Nombre { get; set; } = "";

        [Required(ErrorMessage = "El email es obligatorio")]
        [EmailAddress(ErrorMessage = "Email inválido")]
        public string Email { get; set; } = "";

        [Range(18, 120, ErrorMessage = "La edad debe estar entre 18 y 120")]
        public int Edad { get; set; }
    }
}

```

Qué mirar de este ejemplo

- El `Model` apunta a `this.contacto`, una instancia viva. Blazor escribe directo sobre ese objeto.
- `DataAnnotationsValidator` es lo que conecta los atributos `[Required]`, `[EmailAddress]`, etc. con el `EditContext`. **Sin él, no se valida nada.** Es el olvido más común.
- `ValidationMessage For="@((() => this.contacto.Nombre))"` usa una expresión lambda para identificar el campo. Esa sintaxis rara con `() =>` no es decorativa: Blazor analiza la expresión para saber a qué propiedad apunta.
- `OnValidSubmit` solo corre si todo validó. Por eso adentro de `Guardar()` no hace falta volver a chequear nada.

5. Validación manual con `EditContext` (cuando lo necesitás)

A veces querés validar fuera del submit (por ejemplo, habilitar el botón solo si el form es válido). Ahí creás vos el `EditContext`:

razor

```

<EditForm EditContext="this.editContext" OnValidSubmit="this.Guardar">
    <DataAnnotationsValidator />
    <!-- inputs... -->
    <button type="submit" class="btn btn-primary" disabled="@(!this.esValido)">
        Guardar
    </button>
</EditForm>

@code {
    private Contacto contacto = new();
    private EditContext editContext = default!;
    private bool esValido = false;

    protected override void OnInitialized() {
        this.editContext = new EditContext(this.contacto);
    }
}

```

razor - continuación

```
this.editContext.OnFieldChanged += (sender, args) => {  
    this.esValido = this.editContext.Validate();  
};  
  
private void Guardar() { /* ... */ }
```

Acá enganchás el evento `OnFieldChanged` del `EditContext` para revalidar cada vez que un campo cambia, y usás el resultado para habilitar o deshabilitar el botón. Esto es exactamente lo que `EditContext` te permite y un `<form>` crudo no.

6. Errores comunes

- **Olvidar** `<DataAnnotationsValidator />`. El formulario "funciona" pero nunca valida. Siempre llega a `OnValidSubmit`.
- **Pasar** `Model` y `EditContext` **juntos**. Blazor tira excepción. Elegí uno.
- **Usar** `<input>` **de HTML en vez de** `InputText`. Los inputs comunes no avisan al `EditContext`, así que la validación los ignora.
- **Mezclar** `OnSubmit` **con** `OnValidSubmit/OnInvalidSubmit`. O usás el manual, o usás los automáticos.
- **Modelo con propiedades que no son** `{ get; set; }` **públicas**. El binding necesita acceso de lectura y escritura.

7. Resumen mental

Pensalo así: `EditForm` es un `<form>` que sabe de C#. Le das un objeto, declarás las reglas como atributos, ponés el `DataAnnotationsValidator` para que esas reglas se apliquen, y dejás que el `EditContext` orqueste el estado. Tu código solo se ocupa de qué hacer cuando el modelo ya es válido.

CAPITULO 46

Tutorial: Una agenda de contactos con Blazor

Vamos a desarmar pieza por pieza esta aplicación para entender cómo se construye una app Blazor de verdad: desde el modelo de datos hasta los componentes visuales. La app es una agenda de contactos con un layout *maestro-detalle*: a la izquierda la lista de contactos con un buscador, a la derecha el detalle o el formulario de edición.

La idea no es solo "que funcione", sino ver **las técnicas** que Blazor nos da para organizar el código: inyección de dependencias, componentes, parámetros, eventos, binding, validación y parámetros en cascada. Cada sección usa un archivo del proyecto como ejemplo, así que tené el código a mano.

1. El modelo de datos y el servicio

El modelo: Models/Contacto.cs

Todo arranca con una clase común y corriente (un POCO) que representa un contacto:

```
csharp
public class Contacto {
    public int Id { get; set; }

    [Required(ErrorMessage = "El nombre es obligatorio")]
    public string Nombre { get; set; } = "";

    [Required(ErrorMessage = "El teléfono es obligatorio")]
    [Phone(ErrorMessage = "El teléfono no es válido")]
    public string Telefono { get; set; } = "";

    [Required(ErrorMessage = "El email es obligatorio")]
    [EmailAddress(ErrorMessage = "El email no es válido")]
    public string Email { get; set; } = "";
    // ... Empresa, Cargo, Direccion, FechaNacimiento, Notas
}
```

Fijate en las **anotaciones de validación** ([Required], [Phone], [EmailAddress]). Vienen de System.ComponentModel.DataAnnotations y son *declarativas*: en lugar de escribir código que valide, **describimos las reglas sobre la propiedad** y dejamos que el framework las aplique. Más adelante vamos a ver que el formulario las usa automáticamente — no hay ni un `if` de validación escrito a mano en toda la app.

El modelo también tiene un truco que vamos a necesitar:

```
csharp
public Contacto Clonar() => (Contacto)MemberwiseClone();
```

¿Para qué clonar un contacto? Paciencia, lo vemos en la sección 2.

El servicio: Services/ContactoService.cs

Acá está la regla de oro: **los componentes no acceden a los datos directamente**. Toda la lógica de datos vive en una clase de servicio:

```
csharp
public class ContactoService {
    private readonly IDbContextFactory<AgendaContext> dbFactory;

    public ContactoService(IDbContextFactory<AgendaContext> dbFactory) =>
        this.dbFactory = dbFactory;

    public async Task<IEnumerable<Contacto>> ListarAsync(string? busqueda = null) { ... }
    public async Task GuardarAsync(Contacto contacto) { ... }
    public async Task EliminarAsync(Contacto contacto) { ... }
}
```

Tres operaciones: listar (con búsqueda opcional), guardar y eliminar. El componente que lo use no tiene idea de que abajo hay Entity Framework y SQLite — y eso es exactamente lo que queremos. Mañana cambiamos SQLite por una API REST y los componentes ni se enteran.

Detalle lindo: GuardarAsync resuelve alta y modificación en un solo método mirando el Id:

```
csharp
if (contacto.Id == 0)
    db.Contactos.Add(contacto); // Id 0 → es nuevo
else
    db.Contactos.Update(contacto); // tiene Id → ya existe
```

Inyección de dependencias

¿Y cómo llega el servicio al componente? **Nunca hacemos new ContactoService()**. Lo registramos en Program.cs:

```
csharp
builder.Services
    .AddDbContextFactory<AgendaContext>(
        options => options.UseSqlite("Data Source=contactos.db"));

builder.Services
    .AddScoped<ContactoService>();
```

Y lo pedimos donde lo necesitamos, en Home.razor:

```
razor
@Inject ContactoService Servicio
```

Esto es **inyección de dependencias (DI)**: el contenedor de ASP.NET Core crea las instancias y las "inyecta" a quien las pida. Miralo en cadena: ContactoService pide en su constructor un IDbContextFactory<AgendaContext>... que también está registrado. El contenedor arma todo el árbol de dependencias solo. Nosotros declaramos *qué* necesitamos, el framework resuelve *cómo* conseguirlo.

¿Por qué AddScoped? En Blazor Server, *scoped* significa "una instancia por circuito de usuario": cada visitante tiene su propio servicio mientras dura su conexión.

2. La lógica de la aplicación: Pages/Home.razor

Home.razor es el cerebro de la app. Todo su estado se reduce a cuatro variables:

```
csharp
private IEnumerable<Contacto> contactos = []; // la lista visible
private Contacto? seleccionado;             // el que se muestra en el detalle
private Contacto? editando;                 // el que se está editando (si hay)
private string busqueda = "";               // el texto del buscador
```

Y acá viene una idea clave de Blazor: **la UI es una función del estado**. No escribimos código que "muestre el formulario" o que "oculte el detalle"; escribimos qué se ve para cada estado y Blazor redibuja solo:

```
razor
@if (editando is not null) {
    <FormularioContacto Contacto="@editando" OnGuardar="@Guardar" OnCancelar="@Cancelar" />
} else if (seleccionado is not null) {
    <DetalleContacto Contacto="@seleccionado" OnEditar="@Editar" OnEliminar="@Eliminar" />
} else {
    <p>Seleccioná un contacto para ver su detalle.</p>
}
```

Leelo como una tabla de decisión: ¿hay algo en editando? → formulario. ¿No, pero hay seleccionado? → detalle. ¿Ninguno? → estado vacío. **Cambiar de "modo" es simplemente asignar variables:**

```
csharp
private void Seleccionar(Contacto c) {
    seleccionado = c;
    editando = null; // si estaba editando, salgo del modo edición
}

private void Agregar() {
    editando = new Contacto(); // contacto nuevo, Id = 0
    seleccionado = null;
}

private void Editar() => editando = seleccionado?.Clonar();
private void Cancelar() => editando = null;
```

¿Por qué Clonar()?

Acá se resuelve el misterio de la sección 1. Si hiciéramos `editando = seleccionado`, ambas variables apuntarían **al mismo objeto**: cada letra que el usuario tipea en el formulario modificaría el contacto de la lista en vivo. Y si después cancela... el daño ya está hecho.

Editando una **copia**, "Cancelar" es gratis (tiro la copia y listo) y "Guardar" es explícito:

```
csharp
private async Task Guardar() {
    await Servicio.GuardarAsync(editando!); // persistir la copia
    await Cargar();                         // recargar la lista desde la base
    seleccionado = contactos.FirstOrDefault(c => c.Id == editando!.Id);
    editando = null;                        // volver al modo detalle
}
```

Notá que después de guardar **recargamos la lista** y volvemos a buscar el contacto guardado para dejarlo seleccionado. La base de datos es la fuente de verdad; la lista en memoria es solo una vista de ella.

La carga inicial

```
csharp
protected override async Task OnInitializedAsync() {
    tema.Cambio += StateHasChanged; // esto lo vemos en la sección 7
    await Cargar();
}

private async Task Cargar() => contactos = await Servicio.ListarAsync(busqueda);
```

`OnInitializedAsync` es parte del **ciclo de vida** del componente: Blazor lo llama una vez, cuando el componente arranca. Es el lugar natural para cargar datos.

3. RenderFragment: Controles/Pantalla.razor

Mirá `Home.razor` de nuevo. ¿Notaste que casi no tiene HTML de layout? Nada de `<div class="row">`, columnas, scroll... Todo eso vive en `Pantalla.razor`, un componente que define la estructura visual y deja **tres huecos** para rellenar:

```
razor
<div class="container-fluid d-flex flex-column vh-100 ...">
  <div class="card rounded-4 h-100 ...">
    <div class="card-body ... border-bottom">
      @Cabecera                                     @* hueco 1 *@
    </div>
    <div class="row g-0 flex-grow-1 ...">
      <div class="col-4 h-100 border-end">
        @Maestro                                     @* hueco 2 *@
      </div>
      <div class="col-8 h-100">
        @Detalle                                     @* hueco 3 *@
      </div>
    </div>
  </div>
</div>

@code {
    [Parameter] public RenderFragment? Cabecera { get; set; }
    [Parameter] public RenderFragment? Maestro { get; set; }
    [Parameter] public RenderFragment? Detalle { get; set; }
}
```

Un `RenderFragment` **es un parámetro cuyo valor es... contenido Razor**. Así como un parámetro `string` recibe texto, un `RenderFragment` recibe un pedazo de UI. El que usa el componente lo llena con etiquetas que coinciden con el nombre del parámetro:

```
razor
<Pantalla>
  <Cabecera>
    <h1>Agenda</h1>
    <BotonTema />
  </Cabecera>
  <Maestro>
    <Buscador ... />
    @* la lista de contactos *@
  </Maestro>
  <Detalle>
    @* formulario o detalle según el modo *@
  </Detalle>
</Pantalla>
```

razor - continuación

```
</Detalle>
</Pantalla>
```

El resultado es una separación de responsabilidades buenísima: *Pantalla* sabe **dónde** va cada cosa (y pelea ella sola con el CSS), *Home* sabe **qué** va en cada lugar. Si mañana queremos el maestro a la derecha, tocamos un solo archivo.

■ Si el componente tiene un solo hueco, conviene llamarlo `ChildContent`: es el nombre mágico que Blazor llena automáticamente con todo lo que pongas entre las etiquetas, sin tag extra.

4. Parámetros y eventos: `Contactos/ItemContacto.razor`

Cada fila de la lista de contactos es un `ItemContacto`. Es el ejemplo perfecto de la anatomía de un componente Blazor: **datos que entran, eventos que salen**.

razor

```
<button class="list-group-item ... @(Activo ? "active" : "")"
  @onclick="@OnSeleccionar">
  <span class="badge ..." @Contacto.Iniciales</span>
  <span>
    <span class="d-block fw-semibold">@Contacto.Nombre @Contacto.Apellido</span>
    <small>@Contacto.Empresa</small>
  </span>
</button>

@code {
  [Parameter] public Contacto Contacto { get; set; } = default!;
  [Parameter] public bool Activo { get; set; }
  [Parameter] public EventCallback OnSeleccionar { get; set; }
}
```

Tres parámetros, tres roles:

- `Contacto` — los datos a mostrar. Fluyen *del padre al hijo*.
- `Activo` — un flag visual: ¿soy el contacto seleccionado? Notá que el componente **no decide** si está activo; se lo dicen. Él solo se pinta.
- `OnSeleccionar` — un `EventCallback`: la forma de avisarle al padre que pasó algo. Fluye *del hijo al padre*.

Y así lo usa *Home*:

razor

```
@foreach (var c in contactos) {
  <ItemContacto Contacto="@c"
    Activo="@((c.Id == seleccionado?.Id))"
    OnSeleccionar="() => Seleccionar(c)" />
}
```

Este es el patrón fundamental de Blazor (y de casi todo framework de componentes): **los datos bajan, los eventos suben**. `ItemContacto` no sabe que existe una lista, ni quién está seleccionado, ni qué pasa al hacer click. Solo dice "me clickearon" y el padre decide. Eso lo hace tonto... y por eso, reutilizable y fácil de

razonar.

Bonus de EventCallback: cuando el padre maneja el evento, Blazor re-renderiza automáticamente. Por eso al hacer click la fila se marca como activa sin que nadie llame a nada.

5. Binding bidireccional: Controles/Buscador.razor

En Home el buscador se usa así:

```
razor
<Buscador @bind-Value="@busqueda" OnBuscar="@Cargar" />
```

Ese @bind-Value es **binding bidireccional**: la variable busqueda del padre y el texto del input quedan sincronizados en los dos sentidos. ¿Magia? No: es azúcar sintáctico. @bind-Value se expande a *dos* parámetros:

```
razor
<Buscador Value="@busqueda" ValueChanged="@v => busqueda = v)" ... />
```

Para que eso funcione, el componente tiene que cumplir una **convención**: exponer un parámetro X y su compañero XChanged:

```
razor
<input class="form-control ps-5" placeholder="Buscar contacto..."
value="@Value" @oninput="@OnInput" />

@code {
    [Parameter] public string Value { get; set; } = "";
    [Parameter] public EventCallback<string> ValueChanged { get; set; }
    [Parameter] public EventCallback OnBuscar { get; set; }

    private async Task OnInput(ChangeEventArgs e) {
        Value = e.Value?.ToString() ?? "";
        await ValueChanged.InvokeAsync(Value); // aviso al padre el nuevo valor
        await OnBuscar.InvokeAsync();         // y le aviso que busque
    }
}
```

Seguí el ciclo completo de una tecla:

1. El usuario tipea → se dispara @oninput del <input>.
2. OnInput toma el texto y hace ValueChanged.InvokeAsync(Value) → la variable busqueda del padre se actualiza.
3. OnBuscar.InvokeAsync() → el padre ejecuta Cargar(), que llama al servicio con el filtro.
4. La lista se re-renderiza con los resultados.

Todo eso, **en cada tecla** (por eso usamos @oninput y no @onchange, que esperaría a que el input pierda el foco). Búsqueda en vivo con diez líneas de código.

Fijate además que Buscador separa dos cosas que parecen una: *sincronizar el valor* (ValueChanged, parte del binding) y *pedir la acción* (OnBuscar, un evento de negocio). El binding no obliga al padre a hacer nada;

el evento sí.

6. Formularios y validación:

Contactos/FormularioContacto.razor

Llegó el momento en que las anotaciones de la sección 1 pagan el esfuerzo. El formulario completo:

```

razor
<EditForm Model="Contacto" OnValidSubmit="@OnGuardar">
  <DataAnnotationsValidator />

  <Boton Texto="Cancelar" Estilo="outline-secondary" OnClick="@OnCancelar" />
  <Boton Icono="save" Texto="Guardar cambios" />  @* sin OnClick → submit *@

  <div class="row g-3">
    <Entrada Etiqueta="Nombre *" @bind-Valor="Contacto.Nombre" />
    <Entrada Etiqueta="Apellido *" @bind-Valor="Contacto.Apellido" />
    <Entrada Etiqueta="Teléfono *" @bind-Valor="Contacto.Telefono" />
    <Entrada Etiqueta="Email *" @bind-Valor="Contacto.Email" />
    @* ... *@
    <EntradaFecha Etiqueta="Fecha de nacimiento" @bind-Valor="Contacto.FechaNacimiento" />
  </div>

  <ValidationSummary class="text-danger mt-3" />
</EditForm>

```

Tres piezas trabajan en equipo:

- `<EditForm Model="Contacto">` — el contenedor. Crea un *contexto de edición* alrededor del objeto Contacto que rastrea qué campos se tocaron y si son válidos.
- `<DataAnnotationsValidator />` — el conector. Le dice al EditForm: "para validar, lee las anotaciones del modelo". Acá se enchufan los [Required], [Phone] y [EmailAddress] que escribimos en Contacto.cs. Una línea, cero lógica duplicada.
- `<ValidationSummary />` — muestra la lista de errores (con los ErrorMessage que definimos en el modelo).

Y la jugada maestra: OnValidSubmit. No es "cuando aprieten el botón" sino "cuando aprieten el botón **y el modelo sea válido**". Si falta el nombre o el email está mal formado, OnGuardar directamente no se ejecuta. La validación quedó declarada una sola vez, en el modelo, y el formulario la respeta sin que escribamos un solo if.

Los Inputs

Adentro de un EditForm no se usa `<input>` pelado sino los componentes de Blazor: InputText, InputDate, InputTextArea, etc. ¿Por qué? Porque ellos se registran en el contexto de edición: marcan el campo como "modificado", participan de la validación y se pintan con clases CSS según su estado.

En este proyecto los envolvimos en un control propio, Entrada.razor, para no repetir el `<label>` y la columna de grilla en cada campo:

```

razor
<div class="@Columnna">
  <label class="form-label fw-semibold">@Etiqueta</label>
  <InputText class="form-control" Value="@Valor"
    ValueChanged="@ValorChanged" ValueExpression="@ValorExpression" />
</div>

@code {
  [Parameter] public string Etiqueta { get; set; } = "";
  [Parameter] public string? Valor { get; set; }
  [Parameter] public EventCallback<string?> ValorChanged { get; set; }
  [Parameter] public Expression<Func<string?>>? ValorExpression { get; set; }
}

```

¿Reconocés el patrón `Valor / ValorChanged` de la sección 5? Es la misma convención de binding, y por eso en el formulario podemos escribir `@bind-Valor="Contacto.Nombre"`. La novedad es `ValorExpression`: el sistema de validación no necesita solo el *valor* del campo, necesita saber *qué propiedad del modelo* es, para encontrar sus anotaciones. `@bind-Valor` completa los tres parámetros automáticamente.

Detalle del botón Guardar: `Boton.razor` decide su type solo:

```

razor
<button type="@((OnClick.HasDelegate ? "button" : "submit"))" ...>

```

Si le pasaron `OnClick`, es un botón común. Si no (como "Guardar cambios"), es `type="submit"` y dispara el submit del `EditForm` que lo contiene. Por eso Cancelar cancela y Guardar guarda, usando el mismo componente.

7. CascadingParameter: el tema claro/oscuro

Último desafío: el botón de tema (■/■) está en la cabecera, pero el atributo `data-bs-theme` que cambia los colores está en `Pantalla`, y el estado lo maneja `Home`. ¿Pasamos el tema por parámetro a cada componente del camino? Se puede, pero es tedioso (el famoso *prop drilling*). Blazor tiene algo mejor: **parámetros en cascada**.

El estado: `Controles/Tema.cs`

```

csharp
public class Tema {
  public bool Oscuro { get; private set; } = true;
  public string Modo => Oscuro ? "dark" : "light";
  public event Action? Cambio;
  public void Alternar() { Oscuro = !Oscuro; Cambio?.Invoke(); }
}

```

Una clase simple: el estado (`Oscuro`), una acción (`Alternar`) y un evento (`Cambio`) para avisar cuando algo cambió.

El proveedor: Home.razor

razor

```
<CascadingValue Value="@tema">
  <Pantalla> ... </Pantalla>
</CascadingValue>
```

CascadingValue hace que el objeto tema esté **disponible para todos los componentes que estén adentro**, a cualquier profundidad. Como un Wi-Fi: no hace falta cablear cada habitación.

Los consumidores

Cualquier descendiente lo recibe declarando un [CascadingParameter] — Blazor lo encuentra por tipo, sin que nadie se lo pase explícitamente:

razor

```
@* BotonTema.razor — lee el estado y lo cambia *@
<Boton Icono="@(<Tema!.Oscuro ? "sun" : "moon-stars")" OnClick="@Tema.Alternar" />

@code {
    [CascadingParameter] public Tema? Tema { get; set; }
}
```

razor

```
@* Pantalla.razor — usa el estado para pintar *@
<div class="..." data-bs-theme="@(<Tema?.Modo ?? "dark")">

@code {
    [CascadingParameter] public Tema? Tema { get; set; }
}
```

Míralo bien: BotonTema y Pantalla **se comunican sin conocerse**. Uno está en la cabecera, el otro es el contenedor de toda la app; no hay ningún parámetro entre ellos. Comparten el mismo objeto Tema que cae en cascada desde Home.

La pieza final: el evento Cambio

Queda un problema sutil. Cuando clickeás BotonTema, se ejecuta Tema.Alternar()... pero Blazor re-renderiza al componente donde ocurrió el evento (BotonTema), no a Pantalla, que es quien tiene que cambiar el color. ¿Cómo se enteraría Home de que tiene que redibujar todo?

Con el evento que pusimos en Tema:

csharp

```
// Home.razor
protected override async Task OnInitializedAsync() {
    tema.Cambio += StateHasChanged; // "cuando el tema cambie, redibujame"
    await Cargar();
}

public void Dispose() => tema.Cambio -= StateHasChanged;
```

Home se suscribe: cada vez que el tema cambia, llama a StateHasChanged() (el "che Blazor, redibujá manual) y como Home contiene a todo lo demás, la app entera se actualiza. Y como buen ciudadano, se **desuscribe en Dispose** (por eso el @implements IDisposable arriba del archivo) para no dejar referencias colgadas.

Este trío — *objeto de estado* + *CascadingValue* + *evento de cambio* — es la versión mínima de un patrón que vas a ver en todas las apps Blazor grandes para manejar estado compartido.

Recapitulando

Técnica	Dónde verla	La idea en una frase
Inyección de dependencias	<code>Program.cs</code> , <code>Home.razor</code>	Declarás qué necesitás; el framework lo provee.
<code>DataAnnotations</code>	<code>Contacto.cs</code>	Las reglas de validación viven en el modelo, una sola vez.
UI como función del estado	<code>Home.razor</code>	No mostrás/ocultás cosas: cambiás variables y Blazor redibuja.
<code>RenderFragment</code>	<code>Pantalla.razor</code>	Parámetros que reciben UI: layouts con huecos.
Parámetros y <code>EventCallback</code>	<code>ItemContacto.razor</code>	Los datos bajan, los eventos suben.
Binding bidireccional	<code>Buscador.razor</code>	Convención <code>Value + ValueChanged = @bind-Value</code> .
<code>EditForm</code> y validación	<code>FormularioContacto.razor</code>	<code>OnValidSubmit</code> solo se dispara si el modelo es válido.
<code>CascadingParameter</code>	<code>Tema.cs</code> , <code>BotonTema.razor</code> , <code>Pantalla.razor</code>	Estado compartido sin pasar parámetros mano a mano.

Ahora sí: abrí los fuentes, rompé algo a propósito (sacá el `Clonar()`, cambiá `@oninput` por `@onchange`, borrá el `DataAnnotationsValidator`) y mirá qué pasa. No hay mejor forma de entender por qué cada pieza está donde está. ■

CAPITULO 47

Cómo guardar API keys

Una API key es una credencial: una cadena de texto que identifica y autoriza a tu programa frente a un servicio externo (Gemini, Anthropic, OpenAI, etc.). Quien tiene la key puede usar el servicio **a tu nombre y a tu costo**. Por eso el problema no es técnico sino de *secreto*: ¿dónde guardamos algo que el programa necesita leer pero que nadie más debería ver?

La regla de oro: una API key nunca va escrita en el código fuente. Si está en el código, viaja con el código. Y el código termina en GitHub, en un repo compartido, en la pantalla de un proyector durante una clase. Una vez expuesta, la key está comprometida para siempre.

El problema, desde el principio

Imaginá la solución ingenua:

```
csharp
// MAL. Nunca hagas esto.
string apiKey = "AIzaSyD-1234567890abcdefg";
```

Esto funciona. El programa anda. Pero acabás de meter un secreto dentro de un archivo que vas a versionar con Git y subir a GitHub. Bots automáticos escanean GitHub buscando exactamente este patrón, y en cuestión de minutos alguien puede estar gastando tu cuota.

La idea central para resolverlo es **separar la configuración del código**. El código describe *qué hacer*; la configuración (incluida la key) describe *con qué credenciales y en qué entorno hacerlo*. El programa, en tiempo de ejecución, lee la key desde un lugar externo que vive *fuera* del repositorio.

Ese "lugar externo" tiene varias formas. Las vemos de la más fundamental a la más cómoda.

Variables de entorno

Una **variable de entorno** es un par NOMBRE=valor que el sistema operativo le entrega a cada proceso cuando arranca. Es el mecanismo más antiguo y universal: existe en Windows, Mac y Linux, y todo lenguaje sabe leerlas. No dependen de tu proyecto ni de tu lenguaje; viven en el sistema operativo.

Cuando tu programa arranca, hereda una copia de las variables de entorno del proceso que lo lanzó (la terminal, el IDE). Por eso, si definís una variable en una terminal, solo los programas lanzados *desde esa misma terminal* la verán.

Leerlas en C# es directo:

```
csharp
string apiKey = Environment.GetEnvironmentVariable("GEMINI_API_KEY");
```


Fijate que el código ya no contiene el secreto: contiene el nombre de dónde buscarlo. El código es público, el valor es privado.

En Windows

En PowerShell hay dos niveles. El primero es temporal: la variable vive solo mientras esa ventana de terminal esté abierta.

```
powershell
# Solo para la sesión actual de esta terminal
$env:GEMINI_API_KEY = "tu-key-aca"
```

El segundo es persistente: queda guardada en el registro de Windows para tu usuario, y estará disponible en todas las terminales futuras.

```
powershell
# Persistente para tu usuario (sobrevive reinicios)
[Environment]::SetEnvironmentVariable("GEMINI_API_KEY", "tu-key-aca", "User")
```

Después de la versión persistente tenés que **cerrar y reabrir** la terminal (y a veces el IDE) para que tome el cambio, porque las variables se heredan en el arranque del proceso.

También podés hacerlo por interfaz gráfica: buscá "Editar variables de entorno del sistema" en el menú inicio.

Para **averiguar qué hay guardado** listás todas las variables o leés una puntual:

```
powershell
# Listar todas
Get-ChildItem Env:

# Ver una sola
$env:GEMINI_API_KEY
```

Para **borrar**, distinguí los dos niveles. La de la sesión actual se borra asignándole `$null`; la persistente se borra pasándole `$null` al mismo método con que la creaste:

```
powershell
# Borrar de la sesión actual
$env:GEMINI_API_KEY = $null

# Borrar la persistente del usuario (registro)
[Environment]::SetEnvironmentVariable("GEMINI_API_KEY", $null, "User")
```

En Mac (y Linux)

En la terminal, lo temporal se hace con `export`:

```
bash
# Solo para la sesión actual de esta terminal
export GEMINI_API_KEY="tu-key-aca"
```

Para hacerlo persistente, agregás ese mismo `export` al archivo de configuración de tu shell. En Mac moderno el shell por defecto es **zsh**, así que el archivo es `~/.zshrc` (si usás `bash`, sería `~/.bashrc`):

```
bash
# Agregar al final de ~/.zshrc
echo 'export GEMINI_API_KEY="tu-key-aca"' >> ~/.zshrc
source ~/.zshrc # recarga el archivo sin reabrir la terminal
```

Ese archivo se ejecuta cada vez que abrís una terminal nueva, así que la variable queda disponible siempre.

Para **averiguar qué hay guardado** listás o lees una puntual:

```
bash
# Listar todas (o filtrar con grep)
printenv
printenv | grep GEMINI

# Ver una sola
echo $GEMINI_API_KEY
```

Para **borrar** de la sesión actual usás `unset`. Pero ojo: si la hiciste persistente agregándola al `~/.zshrc`, `unset` solo la saca de la terminal actual; **vuelve a aparecer** la próxima vez que abras una terminal. Para borrarla de verdad tenés que editar el `~/.zshrc` y eliminar la línea del `export`.

```
bash
# Borrar de la sesión actual
unset GEMINI_API_KEY

# Borrar la persistente: abrir el archivo y eliminar la línea export
nano ~/.zshrc
source ~/.zshrc # recargar para que tome el cambio
```

Moraleja: borra del mismo lugar donde guardaste. Si la persististe en un archivo de configuración, `unset` no alcanza; hay que tocar el archivo.

El archivo `.env`

Las variables de entorno persistentes tienen un problema práctico: son globales a tu usuario, no a tu proyecto. Si trabajás en cinco proyectos con cinco keys distintas, ensuciás el sistema entero y se vuelve difícil saber qué necesita cada proyecto.

La convención `.env` resuelve esto. Es simplemente un archivo de texto, en la raíz del proyecto, con las variables que ese proyecto necesita:

```
env
GEMINI_API_KEY=tu-key-aca
DB_CONNECTION=Data Source=app.db
```

La key sigue estando *fuera del código*, pero ahora vive *junto al proyecto* y es fácil ver de un vistazo qué credenciales requiere.

El punto crítico: el archivo `.env` **nunca se versiona**. Por eso lo primero que hacés es agregarlo al `.gitignore`.

```
gitignore
# .gitignore
.env
```

La práctica habitual es versionar un `.env.example` con los *nombres* de las variables pero sin los valores, para que otro desarrollador sepa qué tiene que completar:

```
env
# .env.example (este SÍ se versiona)
GEMINI_API_KEY=
DB_CONNECTION=
```

En el ecosistema .NET, el `.env` no se lee solo: un paquete como `DotNetEnv` carga ese archivo y vuelca su contenido a las variables de entorno del proceso al arrancar.

```
bash
dotnet add package DotNetEnv
```

```
csharp
DotNetEnv.Env.Load(); // lee .env y lo carga como variables de entorno
string apiKey = Environment.GetEnvironmentVariable("GEMINI_API_KEY");
```

Así cerramos el círculo: el `.env` alimenta a las variables de entorno, y el código lee variables de entorno como siempre.

Acá **averiguar qué hay y borrar** es lo más simple de todo: el `.env` es un archivo de texto. Ver qué contiene es abrirlo; borrar una key es eliminar (o vaciar) su línea y guardar. No hay registro escondido ni magia.

```
bash
cat .env          # ver qué hay
nano .env         # editar y borrar la línea que quieras
```

La forma específica de .NET: User Secrets

.NET trae su propia solución, pensada justamente para no andar con `.env` en desarrollo: el **Secret Manager** (User Secrets). La idea es elegante: guarda los secretos en un archivo JSON **fuera de la carpeta del proyecto**, en tu perfil de usuario. Así es *físicamente imposible* commitarlo por accidente, porque no está dentro del repo.

Lo activás una vez por proyecto:

```
bash
dotnet user-secrets init
```

Esto agrega un `<UserSecretsId>` (un GUID) a tu archivo `.csproj`. Después guardás cada secreto:

bash

```
dotnet user-secrets set "GeminiApiKey" "tu-key-aca"
```

El valor se guarda en un `secrets.json` ubicado en:

- **Windows:** %APPDATA%\Microsoft\UserSecrets\<id>\secrets.json
- **Mac/Linux:** ~/.microsoft/usersecrets/<id>/secrets.json

Para leerlo, .NET integra User Secrets en el sistema de **configuración** (IConfiguration). En desarrollo, el secreto se mezcla automáticamente con `appsettings.json`, variables de entorno, etc.:

csharp

```
var builder = WebApplication.CreateBuilder(args);  
string apiKey = builder.Configuration["GeminiApiKey"];
```

Por qué esto es lo idiomático en .NET: el código pide `Configuration["GeminiApiKey"]` sin saber de dónde sale el valor. En tu máquina sale de User Secrets; en el servidor de producción saldrá de una variable de entorno o de un servicio como Azure Key Vault. El código no cambia: cambia solo la fuente de configuración según el entorno.

Para **averiguar qué hay y borrar**, el Secret Manager tiene comandos propios, que es una de sus ventajas frente a las variables de entorno:

bash

```
# Listar todo lo guardado en el proyecto  
dotnet user-secrets list  
  
# Borrar uno por su clave  
dotnet user-secrets remove "GeminiApiKey"  
  
# Borrar todos los del proyecto  
dotnet user-secrets clear
```

Recordá que estos comandos operan sobre el `secrets.json` asociado al <UserSecretsId> del .csproj. Si los corrés desde otra carpeta de proyecto, vas a estar mirando otros secretos (o ninguno).

En resumen

Mecanismo	Alcance	Cuándo usarlo	Cómo borrar
Variable de entorno	Sistema / usuario	Universal, producción, scripts	<code>\$env:X=\$null / unset</code> o editar <code>~/.zshrc</code>
<code>.env</code> + <code>DotNetEnv</code>	Por proyecto	Desarrollo, equipos, multi-lenguaje	Eliminar la línea del archivo
User Secrets	Por proyecto (fuera del repo)	Desarrollo en .NET, lo idiomático	<code>dotnet user-secrets remove / clear</code>

Las tres comparten el mismo principio fundamental: **el secreto vive fuera del código y fuera del control de versiones**, y el programa lo lee en tiempo de ejecución. La diferencia está en la comodidad y en qué tan

difícil es equivocarse y exponerlo.

UTN — Facultad Regional Tucumán · Tecnicatura Universitaria en Programación · Programación III

CAPITULO 48

El API compatible con OpenAI: completar texto como primitiva de procesamiento

El problema

Queremos hablarle a un modelo de lenguaje desde nuestro código. No desde un chat web, no desde una librería que esconde todo, sino mandando un pedido HTTP nosotros mismos y leyendo la respuesta. La idea es entender qué viaja por el cable, porque una vez que lo entendés, te das cuenta de que **toda la potencia del modelo se reduce a una sola operación: completar texto**. Y sobre esa operación, con un buen prompt, podés construir traductores, extractores de datos, generadores de código y consultores sobre tus propios datos.

Este apunte recorre el archivo `40.2-openai-httpclient.cs`, un programa de un solo archivo (.NET 10, file-based) que hace exactamente eso con `HttpClient` puro, sin SDK de por medio.

El "API compatible con OpenAI"

OpenAI definió un formato de pedido y respuesta para su endpoint de chat. Ese formato se volvió tan común que muchísimos otros proveedores (Gemini, Groq, DeepSeek, Ollama corriendo local, etc.) ofrecen un endpoint que **habla el mismo dialecto**. A eso le decimos "compatible con OpenAI".

La consecuencia práctica es enorme: escribís el cliente una sola vez y cambiás de proveedor tocando nada más que tres variables de entorno. Mirá cómo arranca el programa:

```
csharp
string proveedor = (args.Length > 0 ? args[0] : "openai").ToUpper();

string URL      = Environment.GetEnvironmentVariable($"{proveedor}_API_URL") ?? "";
string? API_KEY = Environment.GetEnvironmentVariable($"{proveedor}_API_KEY");
string MODELO   = Environment.GetEnvironmentVariable($"{proveedor}_MODEL") ?? "gpt-5.5";
```

Pasás `openai` o `gemini` por línea de comando y el programa arma los nombres de variable (`OPENAI_API_URL`, `GEMINI_API_URL`, etc.) sobre la marcha. El `.env` se carga con `DotNetEnv.Env.Load()` al inicio. Un `.env` típico:

```
texto
OPENAI_API_URL=https://api.openai.com/v1/chat/completions
OPENAI_API_KEY=sk-...
OPENAI_MODEL=gpt-5.5

GEMINI_API_URL=https://generativelanguage.googleapis.com/v1beta/openai/chat/completions
GEMINI_API_KEY=...
```

texto - continuación

```
GEMINI_MODEL=gemini-2.5-flash
```

Por qué variables de entorno y no la clave en el código. La API key es un secreto. Si la escribís en el `.cs` y subís el archivo a GitHub, regalaste tu clave (y tu plata, porque estos servicios se pagan por uso). El `.env` queda fuera del control de versiones con un `.gitignore`.

Antes de correr el ejemplo: configurar tus claves

El programa **no funciona sin un archivo** `.env`, porque de ahí saca la URL, la clave y el modelo. Por eso el `.env` real no se versiona; en su lugar se sube un `.env.ejemplo` que sirve de plantilla. El primer paso es copiarlo:

bash

```
cp .env.ejemplo .env
```

Después abrís el `.env` y completás las claves. Cada proveedor te da la suya en su propio panel:

- **OpenAI** → creás la clave en <https://platform.openai.com/api-keys>. Es de pago: necesitás cargar saldo en *Billing* antes de que funcione.
- **Gemini (Google)** → la conseguís gratis en <https://aistudio.google.com/apikey>. Tiene un nivel gratuito generoso, ideal para probar en clase.
- **Groq** → clave gratuita en <https://console.groq.com/keys>, muy rápido para modelos abiertos.

Tu `.env` queda parecido a esto (poné la clave que conseguiste donde dice `sk-...`):

texto

```
OPENAI_API_URL=https://api.openai.com/v1/chat/completions
OPENAI_API_KEY=sk-tu-clave-real-aca
OPENAI_MODEL=gpt-5.5

GEMINI_API_URL=https://generativelanguage.googleapis.com/v1beta/openai/chat/completions
GEMINI_API_KEY=tu-clave-de-gemini-aca
GEMINI_MODEL=gemini-2.5-flash
```

Recién con el `.env` completo podés ejecutar el programa. Sin argumentos usa OpenAI; pasándole el nombre elegís otro proveedor:

bash

```
dotnet run 40.2-openai-httpclient.cs # usa OPENAI_*
dotnet run 40.2-openai-httpclient.cs gemini # usa GEMINI_*
```

Tratá tu clave como una contraseña. No la pegues en chats, capturas de pantalla ni la subas a un repo. Si se te escapó, andá al panel del proveedor y revocala (regenera una nueva): la vieja deja de servir al instante.

El formato básico del pedido

El cuerpo del pedido es un JSON con una forma muy simple. Esta es la pieza central del apunte, así que vale la pena mirarla con calma:

```
json
{
  "model": "gpt-5.5",
  "messages": [
    {
      "role": "user",
      "content": "traduce al inglés: hola mundo"
    }
  ],
  "temperature": 1
}
```

Tres campos importan:

- `model`: qué modelo querés usar. Lo decide el proveedor.
- `messages`: una lista de mensajes. Cada mensaje tiene un `role` y un `content`. Los roles son `system` (instrucciones generales de comportamiento), `user` (lo que dice el usuario) y `assistant` (lo que respondió el modelo en turnos anteriores). En el caso más simple mandamos un solo mensaje `user`.
- `temperature`: cuánta "creatividad" o aleatoriedad. Cerca de 0 = respuestas más determinísticas y repetibles; más alto = más variadas. Para extraer datos conviene bajarla; para generar texto creativo, subirla.

Lo justo y necesario. El pedido más simple posible es solo `model` y `messages`; el resto de los campos son opcionales y la API usa un valor por defecto si no los mandás. Por eso el ejemplo manda lo mínimo: pide la completación y nada más.

Armar el JSON a mano (y por qué hay que escapar)

El programa construye el JSON con un *raw string literal* interpolado (`$$"..."`):

```
csharp
var json = $$"""
{
  "model": "{{Json(MODELO)}}",
  "messages": [
    {
      "role": "user",
      "content": "{{Json(prompt)}}"
    }
  ],
  "temperature": {{temperatura}}
}
""";
```

Fijate que el `prompt` pasa por una función `Json(...)`:

```
csharp
static string Json(string texto) => JsonEncodedText.Encode(texto).ToString();
```

Esto es **imprescindible**. Si el `prompt` del usuario contiene comillas, saltos de línea o backslashes, y los metés crudos dentro del JSON, rompés el JSON y el pedido falla (o peor, abrís una puerta a inyección).

`JsonEncodedText.Encode` convierte el `dió "hola"` en el `dió \"hola\"`. Es la versión chiquita y casera de lo que hace un serializador.

El pedido HTTP

Con el JSON listo, el envío es directo:

```
csharp
using var http = new HttpClient();
if (API_KEY is not null) {
    http.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", API_KEY);
}

using var contenido = new StringContent(json, Encoding.UTF8, "application/json");
using var respuesta = await http.PostAsync(URL, contenido);
```

Tres cosas que viajan en el pedido:

1. **El método y la URL:** un POST al endpoint de chat completions.
2. **El header** `Authorization: Bearer <API_KEY>`: así el servidor sabe quién sos y a quién cobrarle. Es el esquema de autenticación estándar de estas APIs.
3. **El cuerpo:** el JSON, con `Content-Type: application/json`.

El formato básico de la respuesta

El servidor devuelve también un JSON. La parte que nos interesa está anidada acá:

```
json
{
  "choices": [
    {
      "message": {
        "role": "assistant",
        "content": "translate to english: hello world"
      }
    ]
  }
}
```

`choices` es una lista porque podés pedir varias respuestas alternativas; nosotros tomamos la primera. El texto que generó el modelo está en `choices[0].message.content`. El código lo extrae navegando el árbol con `JsonNode`:

```
csharp
return JsonNode.Parse(cuerpo)?["choices"]?[0]?["message"]?["content"]?.ToString() ?? "";
```

Y antes maneja el caso de error, que también vale la pena tener:

csharp

```
if (!respuesta.IsSuccessStatusCode) {  
    return $"Error HTTP {respuesta.StatusCode}:{respuesta.ReasonPhrase}\n{cuerpo}";  
}
```

Cuando algo sale mal (clave inválida, modelo inexistente, sin saldo) el cuerpo del error suele explicar bastante bien qué pasó. No te lo guardes: mostralo.

La idea central: una sola operación, muchos usos

Todo lo anterior se condensa en una función: `Completar(prompt)`. Recibe texto, se lo manda al modelo, devuelve texto. **Eso es todo lo que el modelo sabe hacer.** No "traduce", no "resume", no "programa". Sólo continúa el texto que le diste de la forma más plausible.

La magia no está en el modelo: está en **cómo redactás el prompt**. Cada función del programa es un prompt distinto envuelto en una firma de C# que le da nombre a la intención:

csharp

```
async Task<string> Traducir(string texto, string idioma) {  
    return await Completar($"traduce el siguiente texto al {idioma}: {texto}");  
}  
  
async Task<string> ExtraerNombre(string texto) {  
    return await Completar($"extrae el nombre del siguiente texto en formato <apellido>, <nombre>:  
{texto}");  
}  
  
async Task<string> ExtraerFecha(string texto) {  
    return await Completar($"Hoy es {DateTime.Now:yyyy-MM-dd}. Extrae la fecha relativa del siguiente  
texto: {texto}");  
}  
  
async Task<string> Resumir(string texto) {  
    return await Completar($"resume el siguiente texto en una frase: {texto}");  
}
```

Mirá lo que está pasando: programar contra el modelo es **escribir instrucciones en castellano y pegarles tu dato adelante o atrás**. La función `ExtraerFecha` es particularmente astuta: le inyecta la fecha de hoy (`DateTime.Now`) al prompt, porque el modelo no sabe qué día es. "El próximo lunes" sólo se puede resolver si le decís desde cuándo contar. Ese es un patrón clave: **vos aportás el contexto que el modelo no tiene; el modelo aporta el procesamiento del lenguaje.**

De texto a artefactos

Dos funciones dan un paso más: en vez de devolver texto para leer, generan un archivo:

csharp

```
async Task<string> Programar(string texto) {  
    var resultado = await Completar($"Escribe un programa en c# que {texto}. Solo dame el codigo sin  
explicaciones.");  
    File.WriteAllText("./40.0-programa.cs", resultado, Encoding.UTF8);  
    return resultado;  
}  
  
async Task<string> PaginaWeb(string texto) {  
    var resultado = await Completar($"Escribe una pagina web autocontenida en html que {texto}. Solo
```

csharp - continuacion

```
dame el codigo sin explicaciones.");
File.WriteAllText("./40.0-pagina.html", resultado, Encoding.UTF8);
return resultado;
}
```

El "Solo dame el codigo sin explicaciones" es importante: el modelo tiende a charlar ("¡Claro! Acá tenés tu programa..."), y eso ensucia un archivo que querés ejecutar. El prompt acota la salida.

Consultar tus propios datos

La función más interesante combina el modelo con un dato local:

csharp

```
async Task<string> Consultar(string texto) {
    var alumnos = File.ReadAllText("../alumnos/alumnos.md");

    return await Completar($"Actua como un asistente de programacion y responde a la siguiente pregunta:
{texto}\n\nTen en cuenta esta informacion de los alumnos:\n{alumnos}");
}
```

Acá lees un archivo de tu disco, lo pegás dentro del prompt y le pedís al modelo que responda **a partir de ese contenido**. El modelo no tiene tus datos; vos se los das en el momento. Así podés preguntar "¿cuántos alumnos varones y mujeres hay?" o "¿a qué alumnos les falta el TP5?" sobre tu propia planilla. Esta es la semilla de lo que después se formaliza como **RAG** (Retrieval-Augmented Generation): traer datos relevantes y meterlos en el contexto antes de preguntar.

Ideas para experimentar

Una vez que tenés `Completar`, agregar capacidades es agregar funciones de una línea. Algunas que valen la pena probar en clase:

csharp

```
// Clasificar el sentimiento de un comentario
async Task<string> Sentimiento(string texto) =>
    await Completar($"Clasifica el sentimiento como POSITIVO, NEGATIVO o NEUTRO. Responde solo la
palabra: {texto}");

// Corregir ortografía sin cambiar el contenido
async Task<string> Corregir(string texto) =>
    await Completar($"Corrige la ortografia y gramatica del siguiente texto, sin cambiar el sentido ni
agregar nada: {texto}");

// Extraer datos estructurados como JSON (listo para deserializar)
async Task<string> ExtraerContacto(string texto) =>
    await Completar($"Extrae nombre, telefono y email del texto y responde SOLO un JSON con esas claves,
sin markdown: {texto}");

// Generar preguntas de examen sobre un tema
async Task<string> Examen(string tema) =>
    await Completar($"Genera 5 preguntas de opcion multiple sobre {tema}, con 4 opciones cada una y la
respuesta correcta marcada.");

// Explicar código que no entendés
async Task<string> Explicar(string codigo) =>
    await Completar($"Explica en castellano, paso a paso y para un principiante, que hace este
codigo:\n{codigo}");
```

```
csharp - continuacion
```

```
// Clasificar tickets de soporte en categorías fijas
```

```
async Task<string> Clasificar(string ticket) =>
```

```
    await Completar($"Clasifica este ticket en una de estas categorías: FACTURACION, TECNICO, VENTAS, OTRO. Responde solo la categoría:\n{ticket}");
```

El ejemplo de `ExtraerContacto` cierra un arco lindo: si le pedís al modelo que devuelva **JSON puro**, podés deserializar esa respuesta con `JsonSerializer` y conectarla directo a tu `Contacto` de la agenda. El modelo se vuelve un parser de texto libre a datos tipados. Ojo: como no es determinístico, conviene bajar la temperatura y validar siempre lo que vuelve.

Lo que hay que llevarse

El modelo expone **una sola operación: completar texto**. Todo lo demás — traducir, resumir, extraer, programar, clasificar, consultar tus datos — es esa operación más un prompt bien escrito. El "API compatible con OpenAI" es nada más que un formato JSON acordado para pedir esa completación por HTTP: `messages` adentro, `choices[0].message.content` de vuelta. Entendido el formato crudo, los SDK dejan de ser magia y pasan a ser comodidad: encapsulan exactamente este pedido y esta respuesta que acabás de armar a mano.

UTN — Facultad Regional Tucumán · Tecnicatura Universitaria en Programación · Programación III

CAPITULO 49

Microsoft.Extensions.AI: del chat a un asistente con herramientas

Programación III · Tecnicatura Universitaria en Programación · C# 14 / .NET 10

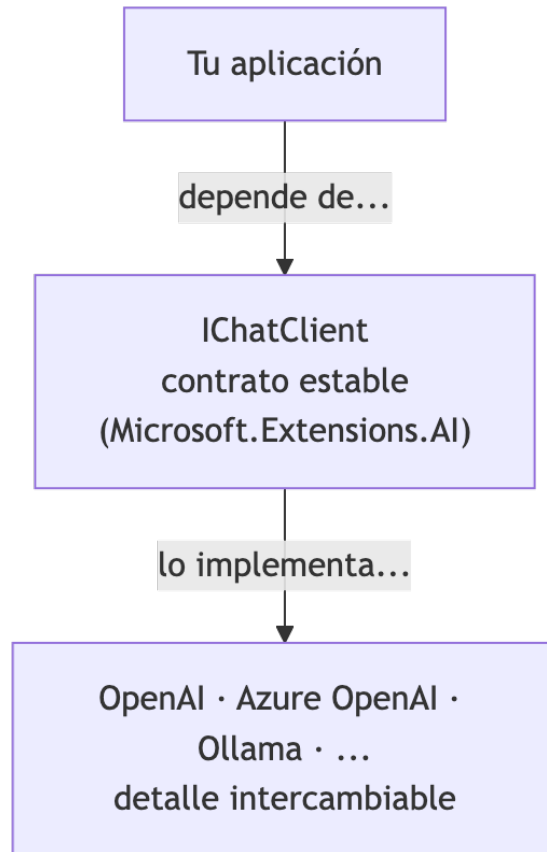
1. ¿Qué problema resolvemos?

Antes de meternos con APIs, pensemos qué pasaba **sin** esta librería.

Cada proveedor de modelos (OpenAI, Azure OpenAI, Anthropic, Ollama, etc.) trae su propio SDK con su propia forma de llamar al modelo. Si tu aplicación hablaba directamente con el SDK de OpenAI y mañana querías mover todo a Azure, o probar en local con Ollama, tenías que reescribir cada lugar donde hacías una llamada al modelo.

`Microsoft.Extensions.AI` (de acá en más, **MEAI**) ataca eso con una idea vieja y conocida en .NET: **programar contra una interfaz, no contra una implementación concreta**. Es exactamente el mismo principio que ya usás con `ILogger`, `IConfiguration` o `DbContext`: tu código depende de un contrato estable, y quién lo implementa lo decidís en un solo lugar.

La interfaz central se llama `IChatClient`. Tu aplicación habla con `IChatClient`. Quién está atrás —OpenAI, Ollama, lo que sea— es un detalle que vive en una sola línea de configuración.



Diagrama

Esto te da tres cosas concretas:

1. **Portabilidad:** cambiás de proveedor tocando una línea.
2. **Testeabilidad:** podés inyectar un `IChatClient` falso en los tests.
3. **Composición:** podés envolver el cliente con capas (logging, caché, telemetría, invocación de funciones) sin tocar tu lógica.

2. Los tipos que tenés que conocer

MEAI es chico de vocabulario. Con cuatro o cinco tipos te movés en el 90% de los casos.

Tipo	Para qué sirve
<code>IChatClient</code>	El cliente. Le mandás mensajes, te devuelve respuestas.
<code>ChatMessage</code>	Un mensaje individual de la conversación.
<code>ChatRole</code>	Quién habla: <code>System</code> , <code>User</code> , <code>Assistant</code> , <code>Tool</code> .
<code>ChatResponse</code>	La respuesta completa del modelo. Su propiedad <code>.Text</code> trae el texto.
<code>ChatOptions</code>	Opciones de la llamada: modelo, temperatura, herramientas, etc.

Un punto importante sobre los **roles**, porque es la base de todo lo que sigue:

- **System**: instrucciones de comportamiento. "Sos un asistente que responde en español y de forma concisa." Lo escribís vos, el usuario no lo ve.
- **User**: lo que escribe la persona.
- **Assistant**: lo que respondió el modelo en turnos anteriores.
- **Tool**: el resultado de ejecutar una herramienta (lo vemos en la parte 3).

El modelo **no tiene memoria propia**. Cada vez que lo llamás, le mandás toda la conversación de nuevo. Esto te va a parecer raro al principio, pero es clave: la "memoria" del chat es simplemente una lista de `ChatMessage` que vos mantenés y le reenviás.

3. Instalación

Necesitás dos paquetes para el caso típico (acá uso OpenAI como proveedor, pero el patrón es idéntico para los demás):

```
bash
dotnet add package Microsoft.Extensions.AI
dotnet add package Microsoft.Extensions.AI.OpenAI
```

- `Microsoft.Extensions.AI`: las abstracciones (`IChatClient`, etc.) **más** las utilidades de alto nivel (invocación de funciones, caché, telemetría).
- `Microsoft.Extensions.AI.OpenAI`: el adaptador concreto que convierte el SDK de OpenAI en un `IChatClient`.

Con esos dos paquetes alcanza para **todos** los proveedores que usamos en este apunte (OpenAI, Gemini, Groq, Ollama, OpenRouter), porque todos exponen un endpoint compatible con OpenAI. Lo vemos en detalle en la sección 6.

4. La llamada más simple posible (una sola pregunta)

Empecemos por lo mínimo: mandar un texto, recibir un texto. Sin conversación, sin herramientas.

```
csharp
using Microsoft.Extensions.AI;
using OpenAI.Chat;

// 1. Creamos el cliente concreto del proveedor...
// 2. ...y lo convertimos a IChatClient con AsIChatClient().
IChatClient client =
    new ChatClient(
        model: "gpt-5.4-mini",
        apiKey: Environment.GetEnvironmentVariable("OPENAI_API_KEY"))
        .AsIChatClient();
```

csharp - continuacion

```
// 3. Mandamos un único texto y leemos la respuesta.
ChatResponse respuesta = await client.GetResponseAsync("¿Qué es la recursividad?");

Console.WriteLine(respuesta.Text);
```

Fijate en la secuencia de tres pasos:

1. `new ChatClient(...)` crea el cliente nativo de OpenAI. La API key la sacamos de una variable de entorno (nunca la quemes en el código).
2. `.AsIChatClient()` es el método de extensión clave: toma ese cliente nativo y lo "viste" como un `IChatClient`. A partir de acá, tu código ya no sabe que atrás hay OpenAI.
3. `GetResponseAsync("...")` tiene una sobrecarga cómoda que acepta un `string` suelto. Por debajo, lo arma como un único `ChatMessage` con rol `User`.

Encapsulando en una función reutilizable

En la práctica no vas a tener el cliente suelto en `Main`. Lo razonable es envolverlo en una función (o un método de un servicio). Acá una versión que recibe una pregunta y un mensaje de sistema opcional:

csharp

```
public class AsistenteSimple {
    private readonly IChatClient client;

    public AsistenteSimple(IChatClient client) {
        this.client = client;
    }

    public async Task<string> PreguntarAsync(string pregunta, string? sistema = null) {
        // Armos la lista de mensajes a mano para tener control del rol System.
        List<ChatMessage> mensajes = [];

        if (sistema is not null) {
            mensajes.Add(new ChatMessage(ChatRole.System, sistema));
        }

        mensajes.Add(new ChatMessage(ChatRole.User, pregunta));

        ChatResponse respuesta = await this.client.GetResponseAsync(mensajes);
        return respuesta.Text;
    }
}
```

Uso:

csharp

```
var asistente = new AsistenteSimple(client);

string r = await asistente.PreguntarAsync(
    pregunta: "Explicá qué es un puntero, en una oración.",
    sistema: "Respondés en español rioplatense, breve y sin rodeos.");

Console.WriteLine(r);
```

¿Por qué pasar de la versión de una línea a esta? Porque ahora `AsistenteSimple` depende de `IChatClient` por constructor. Eso es **inyección de dependencias** lista para usar: en producción le pasás el cliente real, en los tests le pasás uno falso, y la clase ni se entera.

Manteniendo una conversación

Si querés varios turnos, la idea es la que adelantamos: el historial es una lista que vos vas acumulando.

```
csharp
List<ChatMessage> historial = [
    new(ChatRole.System, "Sos un tutor de programación paciente.")
];

while (true) {
    Console.Write("Vos: ");
    string? entrada = Console.ReadLine();
    if (string.IsNullOrEmpty(entrada)) break;

    historial.Add(new ChatMessage(ChatRole.User, entrada));

    ChatResponse respuesta = await client.GetResponseAsync(historial);
    Console.WriteLine($"IA: {respuesta.Text}");

    // Clave: agregamos la respuesta al historial para el próximo turno.
    historial.AddMessages(respuesta);
}
```

El método `AddMessages(respuesta)` agrega los mensajes que generó el modelo de vuelta a la lista. Así, en la siguiente vuelta, el modelo "recuerda" lo que ya dijo... porque se lo estás reenviando entero.

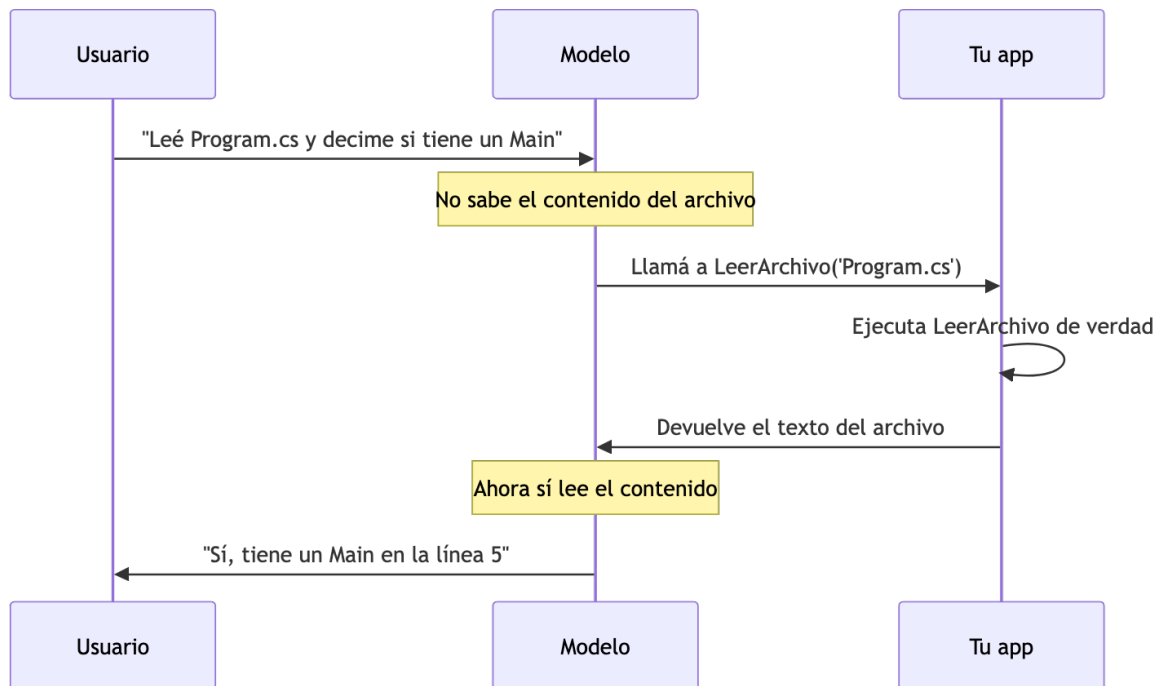
5. Herramientas (tools): que el modelo haga cosas, no solo hable

Hasta acá el modelo solo produce texto. Pero un **asistente de programación** de verdad necesita *actuar*: leer un archivo, escribir otro, listar un directorio. Eso se logra con **herramientas** (en inglés, *tools* o *function calling*).

La idea de fondo

El modelo **no ejecuta tu código**. Lo que hace es distinto y más sutil:

1. Vos le describís qué funciones existen (nombre, qué hace, qué parámetros recibe).
2. El modelo, si le conviene para responder, en vez de contestar con texto, contesta: *"quiero llamar a LeerArchivo con ruta = 'Program.cs'"*.
3. **Tu programa** ejecuta esa función de verdad y le devuelve el resultado al modelo.
4. El modelo recibe el resultado y sigue (puede contestar, o pedir otra herramienta).



Diagrama

Lo bueno de MEAI: ese ida y vuelta (pasos 2 a 4), que es tedioso de programar a mano, **lo automatiza** por vos con un componente de la pipeline. Vos solo escribís las funciones normales de C#.

Paso 1: escribir las herramientas como funciones C# normales

Son métodos comunes y corrientes. La única diferencia es que les ponés [Description] para que el modelo entienda qué hacen. Esas descripciones **son parte del prompt**: cuanto más claras, mejor decide el modelo.

```

csharp
using System.ComponentModel;

public static class HerramientasArchivos {
    // Acotamos todo a una carpeta de trabajo para no dejar que el modelo
    // toque cualquier cosa del disco. Esto es una decisión de seguridad.
    private static readonly string RaízTrabajo =
        Path.Combine(Directory.GetCurrentDirectory(), "workspace");

    [Description("Lee el contenido de texto de un archivo dentro del workspace y lo devuelve.")]
    public static string LeerArchivo(
        [Description("Ruta relativa del archivo, por ejemplo 'src/Program.cs'.")] string ruta) {
        string completa = ResolverRutaSegura(ruta);

        if (!File.Exists(completa)) {
            return $"ERROR: el archivo '{ruta}' no existe.";
        }

        return File.ReadAllText(completa);
    }

    [Description("Escribe (crea o sobrescribe) un archivo de texto dentro del workspace.")]
    public static string EscribirArchivo(
        [Description("Ruta relativa del archivo a escribir.")] string ruta,
        [Description("Contenido completo que se va a guardar en el archivo.")] string contenido) {
        string completa = ResolverRutaSegura(ruta);

        Directory.CreateDirectory(Path.GetDirectoryName(completa));
        File.WriteAllText(completa, contenido);
    }
}
  
```

csharp - continuacion

```

        return $"OK: se escribieron {contenido.Length} caracteres en '{ruta}'.";
    }

    [Description("Lista los archivos y carpetas del workspace.")]
    public static string ListarArchivos() {
        var entradas = Directory
            .EnumerateFileSystemEntries(RaizTrabajo, "*", SearchOption.AllDirectories)
            .Select(p => Path.GetRelativePath(RaizTrabajo, p));

        return string.Join("\n", entradas);
    }

    // Evita que una ruta tipo "../..//algo" se escape del workspace.
    private static string ResolverRutaSegura(string ruta) {
        string completa = Path.GetFullPath(Path.Combine(RaizTrabajo, ruta));

        if (!completa.StartsWith(RaizTrabajo, StringComparison.Ordinal)) {
            throw new InvalidOperationException("Ruta fuera del workspace.");
        }

        return completa;
    }
}

```

Tres detalles pedagógicos importantes acá:

- **Las descripciones son para el modelo, no para vos.** Pensalas como documentación que el modelo lee para decidir cuándo y cómo usar cada función. Una descripción floja = un modelo que usa mal la herramienta.
- **Devolvemos string siempre, incluso los errores.** El modelo necesita poder leer qué pasó. Si la función tira una excepción cruda, mejor devolver un mensaje de error legible que el modelo pueda interpretar y reintentar.
- ResolverRutaSegura **no es opcional**. Le estás dando a un modelo la capacidad de escribir en el disco. Acotalo siempre a un sandbox. Un asistente que puede escribir en cualquier lado es un agujero de seguridad.

Paso 2: registrar la invocación automática de funciones

Acá entra la **pipeline** de MEAI. Recordá que IChatClient se puede envolver en capas. Para que el ida y vuelta de herramientas sea automático, agregamos la capa UseFunctionInvocation() con el ChatClientBuilder:

```

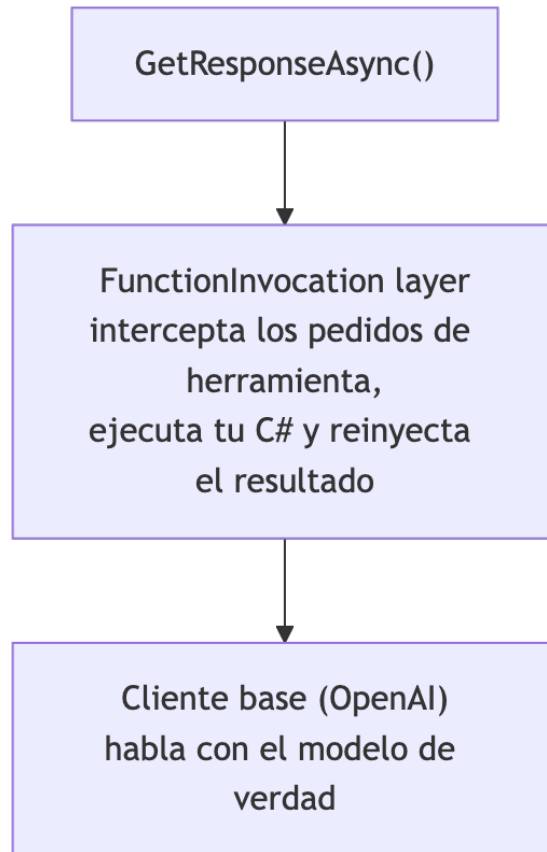
csharp
using Microsoft.Extensions.AI;
using OpenAI.Chat;

IChatClient baseClient =
    new ChatClient("gpt-5.4-mini", Environment.GetEnvironmentVariable("OPENAI_API_KEY"))
        .AsIChatClient();

// Envolvemos el cliente base con la capa que ejecuta las funciones por nosotros.
IChatClient client = new ChatClientBuilder(baseClient)
    .UseFunctionInvocation()
    .Build();

```

Visualmente, la pipeline quedó así:



Diagrama

Paso 3: declarar las herramientas y llamar

Convertimos cada método en un `AIFunction` con `AIFunctionFactory.Create`, y los pasamos por `ChatOptions.Tools`:

```
csharp
ChatOptions opciones = new() {
    Tools =
    [
        AIFunctionFactory.Create(HerramientasArchivos.LeerArchivo),
        AIFunctionFactory.Create(HerramientasArchivos.EscribirArchivo),
        AIFunctionFactory.Create(HerramientasArchivos.ListarArchivos),
    ]
};

List<ChatMessage> historial =
[
    new(ChatRole.System,
        """
        Sos un asistente de programación. Trabajás sobre los archivos del workspace.
        Cuando el usuario te pida ver, crear o modificar código, usá las herramientas
        disponibles en lugar de inventar el contenido. Respondés en español.
        """)
];

historial.Add(new ChatMessage(ChatRole.User,
    "Creá un archivo Saludo.cs con una clase que tenga un método que imprima 'Hola'.");

ChatResponse respuesta = await client.GetResponseAsync(historial, opciones);

Console.WriteLine(respuesta.Text);
```

Cuando corras esto, internamente puede pasar lo siguiente sin que vos escribas una línea más:

1. El modelo decide que necesita EscribirArchivo.
2. La capa UseFunctionInvocation ejecuta tu método EscribirArchivo, que crea el archivo de verdad.
3. El resultado ("OK: se escribieron ...") vuelve al modelo.
4. El modelo te responde algo como *"Listo, creé Saludo.cs con la clase Saludo."*

Todo ese baile ocurre **dentro** de la única llamada a GetResponseAsync. Eso es lo que la capa te ahorra.

El bucle de conversación completo del asistente

Juntando todo, un asistente de programación interactivo se ve así:

```
csharp
Directory.CreateDirectory(
    Path.Combine(Directory.GetCurrentDirectory(), "workspace"));

while (true) {
    Console.WriteLine("\nVos: ");
    string? entrada = Console.ReadLine();
    if (string.IsNullOrEmpty(entrada)) break;

    historial.Add(new ChatMessage(ChatRole.User, entrada));

    ChatResponse respuesta = await client.GetResponseAsync(historial, opciones);

    Console.WriteLine($"Asistente: {respuesta.Text}");
    historial.AddMessages(respuesta);
}
```

Probá pedirle cosas como:

- *"Listá los archivos del proyecto."*
- *"Leé Saludo.cs y agregale un comentario explicando qué hace."*
- *"Creá un archivo README.md con una descripción del workspace."*

En cada caso el modelo elige solo qué herramienta usar, tu programa la ejecuta, y el resultado se reintegra a la conversación.

6. Una factory para elegir el proveedor

Acá se ve el pago de toda la abstracción. **Nada de lo que escribimos en las secciones 4 y 5 cambia.** Tu clase AsistenteSimple, tus HerramientasArchivos, tu ChatOptions, tu bucle de conversación: todo queda igual, porque todo depende de IChatClient. Lo único que cambia es **cómo construís el cliente base**.

La clave que nos permite unificar todos los proveedores en un solo lugar es esta: OpenAI, Gemini, Groq, Ollama y OpenRouter exponen todos un endpoint **compatible con el protocolo de OpenAI** (/v1/chat/completions). Entonces, en vez de usar un SDK distinto por cada uno, usamos el **mismo** paquete Microsoft.Extensions.AI.OpenAI y solo le cambiamos tres datos: la URL base, el nombre del modelo y de dónde sacar la API key.

Paso 1: un record que describe a cada proveedor

Empezamos modelando "qué necesito saber de un proveedor" como un dato. Son exactamente esos tres campos (más la key, que puede faltar en el caso local de Ollama):

```
csharp
public record Proveedor(string Endpoint, string Modelo, string? VariableApiKey = null) {
    public static Proveedor Crear(string proveedor) => proveedor.ToLower() switch {
        // "openai" => new("https://api.openai.com/v1/chat/completions",
        "gpt-5.4-mini", "OPENAI_API_KEY"),
        "openai" => new("https://api.openai.com/v1/chat/completions",
        "gpt-5.5", "OPENAI_API_KEY"),
        "gemini" => new("https://generativelanguage.googleapis.com/v1beta/openai/chat/completions",
        "gemini-2.5-flash", "GEMINI_API_KEY"),
        "groq" => new("https://api.groq.com/openai/v1/chat/completions",
        "qwen/qwen3.6-27b", "GROQ_API_KEY"),
        "ollama" => new("http://localhost:11434/v1/chat/completions",
        "qwen2.5-coder:7b"),
        "openrouter" => new("https://openrouter.ai/api/v1/chat/completions",
        "openrouter/auto", "OPENROUTER_API_KEY"),
        _ => throw new InvalidOperationException($"Proveedor desconocido '{proveedor}'.")
    };
}
```

Fijate en un par de cosas pedagógicas:

- Usamos un `switch expression` sobre el string normalizado a minúsculas. Es la forma idiomática en C# moderno de mapear un valor a una configuración, sin una pila de `if/else`.
- **Ollama no tiene** `VariableApiKey` (queda en `null`): corre local y no pide credencial. Modelamos esa diferencia en el tipo, no con un `if` suelto más adelante.
- El `_` (descarte) cubre el caso de un proveedor mal escrito y falla rápido con un mensaje claro. Mejor explotar al arrancar que mandar requests a un endpoint inexistente.

Paso 2: convertir un Proveedor en un IChatClient

Ahora la fábrica de verdad. Recibe el nombre del proveedor, resuelve su configuración y arma el `IChatClient`. El detalle fino está en el endpoint: el SDK de OpenAI espera la **URL base** (hasta `/v1`) y él solito le agrega `/chat/completions`. Como nuestros datos vienen con la ruta completa, se la recortamos.

```
csharp
using System.ClientModel;
using Microsoft.Extensions.AI;
using OpenAI;
using OpenAI.Chat;

public static class FabricaChatClient {
    public static IChatClient Crear(string nombreProveedor) {
        Proveedor p = Proveedor.Crear(nombreProveedor);

        // El SDK agrega "/chat/completions" por su cuenta: le pasamos solo la base.
        string baseUrl = p.Endpoint.Replace("/chat/completions", "");

        // Si el proveedor no necesita key (Ollama), mandamos una ignorada;
        // si la necesita, la leemos de la variable de entorno y validamos que exista.
        string apiKey = p.VariableApiKey is null
            ? "no-requiere-key"
            : Environment.GetEnvironmentVariable(p.VariableApiKey)
            ?? throw new InvalidOperationException(
                $"Falta la variable de entorno '{p.VariableApiKey}'");

        OpenAIClientOptions opciones = new() {
```

csharp - continuacion

```
Endpoint = new Uri(baseUrl)
};

return new OpenAIClient(new ApiKeyCredential(apiKey), opciones)
    .GetChatClient(p.Modelo)
    .AsIChatClient();
}
```

Paso 3: usarla

Ahora elegir proveedor es una sola línea, y absolutamente todo lo demás del apunte sigue funcionando sin tocarse:

csharp

```
// Cambiás solo este string para saltar de un proveedor a otro.
IChatClient baseClient = FabricaChatClient.Crear("gemini");

// El resto es idéntico a las secciones 4 y 5.
IChatClient client = new ChatClientBuilder(baseClient)
    .UseFunctionInvocation()
    .Build();

ChatResponse respuesta = await client.GetResponseAsync("¿Qué es la recursividad?");
Console.WriteLine(respuesta.Text);
```

Una mejora típica es no quemar el string: leerlo de la configuración o de una variable de entorno, así cambiás de proveedor sin recompilar.

csharp

```
string proveedor = Environment.GetEnvironmentVariable("IA_PROVEEDOR") ?? "ollama";
IChatClient baseClient = FabricaChatClient.Crear(proveedor);
```

Qué proveedor conviene para qué

Proveedor	Modelo del ejemplo	Necesita key	Bueno para...
openai	gpt-5.5	Sí	Calidad alta y <i>function calling</i> sólido.
gemini	gemini-2.5-flash	Sí	Buen costo/rendimiento, contexto grande.
groq	qwen/qwen3.6-27b	Sí	Inferencia muy rápida sobre modelos abiertos.
ollama	qwen2.5-coder:7b	No (local)	Gratis, privado, ideal para la cátedra.
openrouter	openrouter/auto	Sí	Un único endpoint que rutea a decenas de modelos.

Para que Ollama funcione, primero hay que tenerlo corriendo y haber bajado el modelo:

```
bash
ollama pull qwen2.5-coder:7b
ollama serve # levanta el servidor local en http://localhost:11434
```

Un detalle honesto para los alumnos: que todos hablen `IChatClient` no garantiza que todos soporten **lo mismo**. El *function calling* de la sección 5 depende de que el modelo concreto sepa pedir herramientas. Los modelos grandes de OpenAI, Gemini y Groq lo hacen bien; con Ollama elegí un modelo que las soporte (qwen2.5-coder y llama3.1, por ejemplo, sí). La abstracción te unifica la *API*, no las *capacidades* del modelo.

Sobre los nombres de modelo: los identificadores de este ejemplo (gpt-5.5, qwen/qwen3.6-27b, etc.) son los que cargaste vos. Cambian seguido; antes de dar la clase conviene verificar el nombre exacto vigente en la documentación de cada proveedor, porque un modelo dado de baja hace fallar la llamada con un 404.

7. Por qué esto importa (y dónde seguir)

Lo que armamos es, en esencia, **el corazón de un agente**: un modelo + un conjunto de herramientas que son métodos reales de tu sistema. La frase para recordar es:

*Las herramientas del agente **son** los métodos de tu dominio.*

Si en lugar de leer/escribir archivos esas funciones fueran `BuscarCliente`, `CrearPedido` o `ConsultarStock` (los métodos de un repositorio), tendrías un agente que opera sobre tu base de datos. El mecanismo es idéntico; solo cambian las funciones que registrás en `ChatOptions.Tools`.

Algunas capas más de la pipeline que vale la pena conocer cuando avancen:

Capa	Qué agrega
<code>UseFunctionInvocation()</code>	Ejecuta tus herramientas automáticamente (la que usamos).
<code>UseDistributedCache()</code>	Cachea respuestas para no repetir llamadas idénticas.
<code>UseOpenTelemetry()</code>	Trazas y métricas de cada llamada al modelo.
<code>UseLogging()</code>	Loguea cada request/response a un <code>ILogger</code> .

Todas se encadenan en el `ChatClientBuilder` igual que `UseFunctionInvocation`, y todas operan sin que tu lógica de negocio se entere. Es el mismo patrón de middleware que ya conocés de ASP.NET Core, aplicado a las llamadas al modelo.

8. Resumen en una carilla

- MEAI te da `IChatClient`, una **interfaz estable** para hablar con cualquier proveedor.
- Pasás del SDK concreto a `IChatClient` con `.AsIChatClient()`.

-
- Una llamada simple: `await client.GetResponseAsync("texto")` → `respuesta.Text`.
 - El modelo **no tiene memoria**: el historial es una `List<ChatMessage>` que vos mantenés y reenviás.
 - Los **roles** (System, User, Assistant, Tool) estructuran la conversación.
 - Las **herramientas** son métodos C# normales con `[Description]`, registrados con `AIFunctionFactory.Create` en `ChatOptions.Tools`.
 - `UseFunctionInvocation()` automatiza el ciclo pedido-de-herramienta → ejecución → resultado.
 - **Cambiar de proveedor** (OpenAI, Gemini, Groq, Ollama, OpenRouter) se reduce a un string en una factory, porque todos hablan el protocolo compatible con OpenAI. La abstracción unifica la API, no las capacidades del modelo.
 - Acotá siempre lo que las herramientas pueden hacer (sandbox). Darle al modelo acceso al disco sin límites es peligroso.
-

CAPITULO 50

Tutorial: asistente de programación mínimo con tools

En este tutorial vamos a construir, desde cero, un asistente de programación que se ejecuta en una consola y puede usar tres herramientas:

- `listar-archivos`: conoce qué archivos existen.
- `leer-archivo`: obtiene el contenido de un archivo.
- `escribir-archivo`: crea o reemplaza un archivo.

El código completo se encuentra en [40.8-asistente-tools-minimo.cs](#).

1. Qué es realmente un asistente con tools

Un modelo de lenguaje solamente recibe datos y genera una respuesta. No puede leer nuestro disco por sí mismo.

Si le preguntamos:

¿Qué contiene programa.cs?

el modelo no conoce ese archivo. Para resolverlo, nuestra aplicación le ofrece una función llamada `leer-archivo`.

El ciclo completo es el siguiente:

1. La aplicación envía al modelo el mensaje del usuario y la descripción de las tools disponibles.
2. El modelo decide si necesita usar una tool.
3. Si necesita una, responde con el nombre de la tool y sus argumentos. Por ejemplo: `leer-archivo(ruta:"programa.cs")`.
4. La aplicación ejecuta la función C# correspondiente.
5. La aplicación envía el resultado de la función al modelo.
6. El modelo utiliza ese resultado para redactar la respuesta final.

La decisión de usar una tool pertenece al modelo. La ejecución de la tool pertenece a nuestro programa.

Este flujo coincide con el proceso de function calling documentado por [OpenAI](#). En este ejemplo, `Microsoft.Extensions.AI` se ocupa automáticamente de repetir las llamadas necesarias entre los pasos 2 y 6.

2. Estructura mínima

Vamos a utilizar un archivo ejecutable de C# de .NET 10. Las primeras líneas declaran el intérprete y las dependencias:

```
csharp
#!/usr/bin/env -S dotnet run
#:package DotNetEnv@*
#:package Microsoft.Extensions.AI@10.4.0
#:package Microsoft.Extensions.AI.OpenAI@10.4.0
```

Cada dependencia cumple una función concreta:

Paquete	Responsabilidad
DotNetEnv	Carga variables desde el archivo <code>.env</code> .
Microsoft.Extensions.AI	Define el cliente, los mensajes, las tools y la invocación automática.
Microsoft.Extensions.AI.OpenAI	Adapta el cliente oficial de OpenAI a las abstracciones de <code>Microsoft.Extensions.AI</code> .

Importamos los espacios de nombres necesarios:

```
csharp
using System.ComponentModel;
using Microsoft.Extensions.AI;
using OpenAIChatClient = OpenAI.Chat.ChatClient;
```

`DescriptionAttribute`, definido en `System.ComponentModel`, permite describir los parámetros de las tools. Esas descripciones ayudan al modelo a construir argumentos correctos.

3. Configurar la clave de API

La aplicación necesita una clave de la API de OpenAI. La clave no debe escribirse dentro del código.

En el mismo directorio desde el que se ejecutará el programa, crear un archivo `.env`:

```
env
OPENAI_API_KEY=la_clave_personal
```

Agregar `.env` al archivo `.gitignore` para evitar publicarlo:

```
gitignore
.env
```

El programa carga y valida la variable:

```
csharp
DotNetEnv.Env.Load();

var apiKey = Environment.GetEnvironmentVariable("OPENAI_API_KEY");
if (string.IsNullOrEmpty(apiKey)) {
    throw new InvalidOperationException("Falta OPENAI_API_KEY en el archivo .env.");
}
```

La aplicación se detiene inmediatamente si no encuentra una clave. Es preferible este error concreto a permitir que la primera consulta falle con un mensaje menos claro.

4. Definir un workspace

Una tool de archivos no debería operar libremente sobre todo el disco. Vamos a limitarla a un directorio llamado workspace:

```
csharp
var workspace = Path.GetFullPath("workspace");
Directory.CreateDirectory(workspace);
```

`Path.GetFullPath` transforma la ruta relativa en una ruta absoluta. Las tres tools utilizarán siempre este mismo directorio como raíz.

Si el programa se ejecuta desde `clases`, los archivos administrados quedarán dentro de:

```
texto
clases/workspace/
```

Esta separación evita que una prueba normal modifique el código del propio asistente.

5. Crear el cliente y habilitar las tools

Primero construimos el cliente básico de OpenAI. Después agregamos una etapa que ejecuta automáticamente las funciones solicitadas por el modelo:

```
csharp
IChatClient chat = new OpenAIChatClient("gpt-5.5", apiKey)
    .AsIChatClient()
    .AsBuilder()
    .UseFunctionInvocation()
    .Build();
```

Cada llamada tiene un propósito:

1. `OpenAIChatClient` crea el cliente concreto para el modelo `gpt-5.5`.
2. `AsIChatClient` lo expone mediante la interfaz común `IChatClient`.
3. `AsBuilder` inicia una cadena de componentes alrededor del cliente.

4. UseFunctionInvocation agrega el componente que detecta y ejecuta tool calls.
5. Build construye el cliente definitivo.

Sin UseFunctionInvocation, el modelo podría solicitar una tool, pero nuestro programa tendría que interpretar y ejecutar manualmente esa solicitud. Con este componente, la biblioteca implementa el ciclo automático. Microsoft documenta este patrón en su [guía de invocación de funciones .NET](#).

6. Escribir las tres funciones C#

Antes de convertirlas en tools, las herramientas son funciones C# normales. Se pueden comprender y probar sin inteligencia artificial.

6.1. Listar archivos

```
csharp
string ListarArchivos(
    [Description("Ruta relativa del directorio. Usar punto para la raíz.")] string ruta = ".") {
    var rutaCompleta = ResolverRuta(ruta);

    if (!Directory.Exists(rutaCompleta)) return $"No existe el directorio: {ruta}";

    var elementos = Directory.EnumerateFileSystemEntries(rutaCompleta).Select(Path.GetFileName);

    return string.Join(Environment.NewLine, elementos);
}
```

La función recibe una ruta relativa. El valor `.` representa la raíz del workspace. Si el directorio existe, devuelve un nombre por línea.

La función devuelve texto porque ese resultado será enviado al modelo. No imprime directamente en la consola.

6.2. Leer un archivo

```
csharp
string LeerArchivo(
    [Description("Ruta relativa del archivo que se quiere leer.")] string ruta) {
    var rutaCompleta = ResolverRuta(ruta);

    return File.Exists(rutaCompleta)
        ? File.ReadAllText(rutaCompleta)
        : $"No existe el archivo: {ruta}";
}
```

Si el archivo existe, devuelve su contenido completo. Si no existe, devuelve un mensaje que el modelo puede interpretar y explicar al usuario.

6.3. Escribir un archivo

csharp

```
string EscribirArchivo(
    [Description("Ruta relativa del archivo que se quiere escribir.")] string ruta,
    [Description("Contenido completo que se guardará en el archivo.")] string contenido) {
    var rutaCompleta = ResolverRuta(ruta);
    Directory.CreateDirectory(Path.GetDirectoryName(rutaCompleta)!);
    File.WriteAllText(rutaCompleta, contenido);
    return $"Archivo escrito: {ruta}";
}
```

Esta función crea los directorios intermedios, escribe el contenido completo y devuelve una confirmación.

`File.WriteAllText` reemplaza el contenido si el archivo ya existe. Por eso el prompt indicará que debe leer un archivo existente antes de modificarlo.

7. Resolver rutas de manera controlada

Las tres funciones llaman a un único helper:

csharp

```
string ResolverRuta(string ruta) {
    var rutaCompleta = Path.GetFullPath(Path.Combine(workspace, ruta));
    var prefijoValido = workspace.TrimEnd(Path.DirectorySeparatorChar)
        + Path.DirectorySeparatorChar;

    if (rutaCompleta != workspace &&
        !rutaCompleta.StartsWith(prefijoValido, StringComparison.Ordinal)) {
        throw new UnauthorizedAccessException("La ruta está fuera del workspace.");
    }

    return rutaCompleta;
}
```

Supongamos que el workspace es `/curso/workspace`:

Entrada	Ruta resuelta	Resultado
<code>programa.cs</code>	<code>/curso/workspace/programa.cs</code>	Permitida
<code>ejemplos/app.cs</code>	<code>/curso/workspace/ejemplos/app.cs</code>	Permitida
<code>../secreto.txt</code>	<code>/curso/secreto.txt</code>	Rechazada

El control se realiza después de normalizar la ruta. De esta forma, `..` no puede utilizarse para salir del workspace.

Esta verificación es suficiente para el ejemplo didáctico, pero no constituye un sandbox de producción. Una aplicación real también debe considerar enlaces simbólicos, permisos, tamaños máximos, extensiones permitidas y confirmación del usuario antes de sobrescribir información.

8. Convertir las funciones en tools

El modelo no recibe métodos C# directamente. Recibe el nombre, la descripción y un esquema de parámetros de cada tool.

`AIFunctionFactory.Create` toma una función C# y genera esa representación:

```
csharp
var opciones = new ChatOptions {
    Tools = [
        AIFunctionFactory.Create(ListarArchivos, new() {
            Name = "listar-archivos",
            Description = "Lista los archivos y directorios de una ruta del workspace."
        }),
        AIFunctionFactory.Create(LeerArchivo, new() {
            Name = "leer-archivo",
            Description = "Lee el contenido completo de un archivo del workspace."
        }),
        AIFunctionFactory.Create(EscribirArchivo, new() {
            Name = "escribir-archivo",
            Description = "Crea o reemplaza un archivo de texto dentro del workspace."
        })
    ]
};
```

Hay dos niveles de descripción:

- `Description` dentro de `AIFunctionFactoryOptions` explica para qué sirve la tool.
- `[Description]` sobre cada parámetro explica qué dato debe proporcionar el modelo.

`AIFunctionFactory` utiliza la firma de la función para generar el esquema JSON de sus argumentos. La [referencia de AIFunctionFactory](#) describe esta conversión.

Las tools no se colocan en el prompt. Se envían mediante `ChatOptions.Tools`, como capacidades estructuradas que el modelo puede seleccionar.

9. Crear el historial y el prompt mínimo

Una conversación es una lista ordenada de mensajes:

```
csharp
var mensajes = new List<ChatMessage> {
    new(ChatRole.System, """
        Sos un asistente de programación. Respondé en español.
        Usá las herramientas para listar, leer y escribir archivos cuando sea necesario.
        Antes de modificar un archivo existente, leelo.
        """)
};
```

El primer mensaje tiene rol `System`. Su función es definir reglas estables para toda la conversación.

El prompt está dentro del programa porque buscamos la versión más pequeña posible. En una aplicación mayor podría cargarse desde un archivo o desde configuración.

10. Implementar el ciclo de conversación

El chatbot repite cuatro operaciones: leer, validar, consultar y mostrar.

```
csharp
while (true) {
    Console.Write("vos> ");
    var texto = Console.ReadLine();

    if (texto is null || texto.Equals("/salir", StringComparison.OrdinalIgnoreCase)) break;
    if (string.IsNullOrWhiteSpace(texto)) continue;

    mensajes.Add(new(ChatRole.User, texto));

    try {
        var respuesta = await chat.GetResponseAsync(mensajes, opciones);
        mensajes.Add(new(ChatRole.Assistant, respuesta.Text ?? ""));
        Console.WriteLine($"asistente> {respuesta.Text}\n");
    } catch (Exception ex) {
        Console.WriteLine($"Error: {ex.Message}\n");
    }
}
```

La línea central es:

```
csharp
var respuesta = await chat.GetResponseAsync(mensajes, opciones);
```

`mensajes` contiene el contexto de la conversación. `opciones` contiene las tools disponibles.

No utilizamos streaming. `await` espera hasta que se completan todas las tool calls y se obtiene la respuesta final. Recién entonces se imprime el texto.

Después agregamos la respuesta al historial. Gracias a esto, la siguiente consulta puede hacer referencia a mensajes anteriores.

11. Ejecutar el programa

Desde el directorio `clases`:

```
bash
dotnet run 40.8-asistente-tools-minimo.cs
```

Al iniciar, el programa crea automáticamente el directorio `workspace`.

Probar las tools por separado con estas solicitudes:

```
texto
Creá saludo.txt con el texto Hola mundo.
```

```
texto
Leé saludo.txt y explicame qué contiene.
```


texto

Listá los archivos disponibles.

Luego probar una tarea que requiera varias tools:

texto

Creá operaciones.cs con una función Sumar. Después leé el archivo y explicame el código.

En esa última solicitud, el modelo puede llamar a escribir-archivo y luego a leer-archivo antes de presentar su respuesta final.

Para terminar:

texto

/salir

12. Qué conceptos deben quedar claros

1. El modelo no ejecuta C# ni accede directamente al disco.
2. Una tool es una función de nuestra aplicación expuesta mediante nombre, descripción y parámetros.
3. El modelo decide cuándo solicitar una tool; la aplicación conserva el control de su ejecución.
4. El resultado de una tool vuelve al modelo para que pueda continuar razonando y responder.
5. `UseFunctionInvocation` automatiza el intercambio de tool calls y resultados.
6. `ChatOptions.Tools` define las capacidades disponibles en una consulta.
7. La lista de `ChatMessage` conserva el contexto entre turnos.
8. Las rutas y argumentos generados por un modelo deben considerarse datos no confiables y validarse.

Con estos elementos ya existe un asistente de programación real, aunque deliberadamente pequeño. La interfaz gráfica, el streaming, la persistencia y las aprobaciones son mejoras posteriores; no son necesarias para comprender el mecanismo fundamental.

CAPITULO 51

Tutorial: bot de WhatsApp en C# con wacli y Microsoft.Extensions.AI

Este tutorial explica como construir una aplicacion C# simple que escucha mensajes de WhatsApp, consulta un modelo de IA y responde por WhatsApp.

La version final esta en WhatsAppBot .cs y fue pensada para ser explicada en clase: un solo archivo, pocas piezas y flujo lineal.

1. Objetivo

Queremos construir un bot que haga esto:

- 1. Mantener WhatsApp sincronizado usando wac li.
- 2. Leer periodicamente los ultimos mensajes de un contacto.
- 3. Detectar mensajes nuevos.
- 4. Enviar el texto nuevo a un asistente de IA.
- 5. Permitir que el asistente lea y escriba archivos.
- 6. Enviar la respuesta por WhatsApp.
- 7. Salir solamente cuando el contacto envíe /finalizar.

La idea central es esta:

texto

```
WhatsApp -> wac li -> C# -> Microsoft.Extensions.AI -> respuesta -> wac li -> WhatsApp
```

2. Arquitectura final

El programa esta dividido en piezas chicas:

Pieza	Responsabilidad
Flujo principal	Coordina el bot.
ClienteWhatsApp	Encapsula todos los comandos wac li.
AgenteSecretaria	Encapsula el cliente de IA y el historial.
ArchivosAgente	Herramientas de archivos para el asistente.
AGENTS.md	Prompt del asistente.
CONTACTOS.md	Archivo que administra el asistente.

Esta separación es importante: el flujo principal no conoce detalles de JSON, procesos externos ni tool calling. Solo llama métodos con nombres claros.

Una convención del código: los identificadores de lógica están en español (`RecibirTexto`, `ProcesarMensaje`, `IniciarSincronizacion`), y los métodos asíncronos no llevan el sufijo `Async` porque no existe una versión sincrónica de cada uno. Los tipos que mapean el JSON de `wac li` conservan sus nombres originales (`Wac liStoredMessage`, `InboundMessage`, `CommandResult`).

3. Requisitos previos

Necesitas:

- .NET 10 SDK.
- `wac li` instalado.
- WhatsApp autenticado con `wac li auth`.
- Variable de entorno `OPENAI_API_KEY`.
- Archivo `AGENTS.md`.
- Archivo `CONTACTOS.md`.

Comandos útiles:

```
bash
dotnet --version
wac li --help
printenv OPENAI_API_KEY
```

Para configurar la clave:

```
bash
export OPENAI_API_KEY="tu-clave"
```

4. Archivo C# autocontenido

La app usa file-based C#. El archivo `WhatsAppBot.cs` declara sus paquetes al inicio:

```
csharp
#!/usr/bin/env dotnet run
# :sdk Microsoft.NET.Sdk
# :property TargetFramework=net10.0
# :property LangVersion=preview
# :property Nullable=enable
# :package Microsoft.Extensions.AI@10.3.0
# :package Microsoft.Extensions.AI.OpenAI@10.3.0
# :package OpenAI@2.8.0
```

Esto permite ejecutar directamente:

```
bash
dotnet run WhatsAppBot.cs
```

No hace falta crear manualmente un .csproj.

5. Configuración del bot

Las constantes del comienzo son el panel de control de la app:

```
csharp
const string Contacto      = "+5493815343458";
const string Modelo        = "gpt-5.5";
const string ArchivoAgentes = "AGENTS.md";
const string ComandoFinalizar = "/finalizar";
const string PromptPorDefecto = "Sos un asistente claro, breve y util. Responde en el mismo idioma del usuario.";
const int IntervaloSondeoMs = 2_000;
```

Constante	Explicación
Contacto	Numero o identificador del contacto a escuchar.
Modelo	Modelo de OpenAI.
ArchivoAgentes	Archivo desde donde se carga el prompt.
ComandoFinalizar	Mensaje que detiene el bot.
PromptPorDefecto	Prompt de respaldo si falta AGENTS.md.
IntervaloSondeoMs	Pausa entre lecturas de mensajes.

La salida del bot se controla solo con /finalizar, para mantener el ejemplo simple.

Después de las constantes se valida la clave de OpenAI. Si falta, el bot termina con un mensaje claro:

```
csharp
string? claveApi = Environment.GetEnvironmentVariable("OPENAI_API_KEY");
if (string.IsNullOrEmpty(claveApi)) {
    Console.Error.WriteLine("Falta OPENAI_API_KEY. Defini la variable de entorno antes de iniciar el bot.");
    return 1;
}
```

6. Flujo principal

El flujo principal se puede leer como una receta:

```
csharp
Process? procesoSincronizacion = null;
try {
    procesoSincronizacion = ClienteWhatsApp.IniciarSincronizacion();

    string promptSistema = await LeerPromptSistema(ArchivoAgentes, PromptPorDefecto);
    AgenteSecretaria asistente = new(claveApi, Modelo, promptSistema);
    DateTimeOffset iniciadoEn = DateTimeOffset.Now;
```

csharp - continuacion

```

    Console.WriteLine($"Bot activo para {Contacto}. Envia {ComandoFinalizar} por WhatsApp para detenerlo.");

    while (true) {
        await Task.Delay(IntervaloSondeoMs);

        string? textoEntrante = await ClienteWhatsApp.RecibirTexto(Contacto, iniciadoEn);
        if (string.IsNullOrEmpty(textoEntrante)) { continue; }
        if (EsComandoFinalizar(textoEntrante)) { break; }

        string respuesta = await asistente.ProcesarMensaje(textoEntrante);
        if (string.IsNullOrEmpty(respuesta)) { continue; }

        await ClienteWhatsApp.EnviarTexto(Contacto, respuesta);
    }
} finally {
    ClienteWhatsApp.DetenerSincronizacion(procesoSincronizacion);
}

return 0;

```

En pseudocodigo:

texto

```

iniciar wacli sync
crear asistente
marcar la hora de arranque

mientras true:
    esperar 2 segundos
    pedir el texto nuevo del contacto
    si no hay texto, seguir esperando
    si el texto contiene /finalizar, salir del bucle
    pedir una respuesta al asistente
    enviar la respuesta por whatsapp

detener wacli sync

```

Esta es la parte mas importante para explicar en clase. Notar que `RecibirTexto` ya devuelve listo el texto nuevo (o `null` si no hay nada), asi que el bucle queda muy corto: leer, decidir, responder.

7. Como se detectan los mensajes nuevos

Cuando el bot arranca, `wacli` ya tiene mensajes guardados en su base local. Si el bot respondiera todo lo que lee, contestaria mensajes viejos.

En vez de recordar identificadores de mensajes, el bot usa dos lineas divisorias:

1. La hora de arranque (`iniciadoEn`): nada anterior a ese momento se responde.
2. El ultimo mensaje enviado por nosotros (`FromMe`): todo lo que esta antes o en ese mensaje ya fue atendido.

La logica vive en `FiltrarMensajesNuevos`:

```
csharp
static List<InboundMessage> FiltrarMensajesNuevos(IReadOnlyList<InboundMessage> mensajes,
DateTimeOffset? desde) {
    List<InboundMessage> mensajesCronologicos = mensajes
        .OrderBy(mensaje => mensaje.Timestamp ?? DateTimeOffset.MinValue)
        .ThenBy(mensaje => mensaje.Id, StringComparer.Ordinal)
        .ToList();

    int indiceUltimoMensajePropio = mensajesCronologicos.FindLastIndex(mensaje => mensaje.FromMe);
    int indiceUltimoMensajeAntesDeInicio = desde is null
        ? -1
        : mensajesCronologicos.FindLastIndex(mensaje => mensaje.Timestamp is not null &&
mensaje.Timestamp < desde);
    int indiceInicio = Math.Max(indiceUltimoMensajePropio, indiceUltimoMensajeAntesDeInicio);

    IEnumerable<InboundMessage> mensajesDespuesDeInicio = indiceInicio >= 0
        ? mensajesCronologicos.Skip(indiceInicio + 1)
        : mensajesCronologicos;

    return mensajesDespuesDeInicio
        .Where(EsTextoEntrante)
        .ToList();
}

static bool EsTextoEntrante(InboundMessage mensaje) => !mensaje.FromMe &&
!string.IsNullOrEmpty(mensaje.Text);
```

Pasos:

1. Ordena los mensajes en orden cronológico (por Timestamp, y por Id para desempatar).
2. Busca el índice del último mensaje propio y el del último mensaje anterior al arranque.
3. Se queda con el índice más avanzado de los dos: esa es la línea divisoria.
4. Toma todo lo que viene después de esa línea.
5. Filtra para quedarse solo con mensajes entrantes que tengan texto.

Detalle importante: por esto leemos todos los mensajes (no solo los del contacto), porque necesitamos ver también nuestros propios mensajes (FromMe) para ubicar la última respuesta enviada.

8. Recibir el texto nuevo

RecibirTexto junta todo lo anterior y devuelve un único texto listo para el asistente:

```
csharp
public static async Task<string?> RecibirTexto(string contacto, DateTimeOffset? desde = null) {
    CommandResult resultado = await EjecutarWacli("--json", "messages", "list", "--chat", contacto,
"--limit", "50");
    if (resultado.ExitCode != 0) {
        Console.Error.WriteLine($"No se pudieron leer mensajes con wacli: {resultado.Stderr}");
        return null;
    }

    IReadOnlyList<InboundMessage> mensajes = ParsearMensajes(resultado.Stdout, contacto);
    List<string> mensajesFiltrados = FiltrarMensajesNuevos(mensajes, desde)
        .Select(mensaje => mensaje.Text.Trim())
        .Where(texto => texto.Length > 0)
        .ToList();
    if (mensajesFiltrados.Count == 0) { return null; }

    return string.Join(Environment.NewLine, mensajesFiltrados);
}
```

Si el contacto mando varios mensajes seguidos, se combinan en un solo texto separado por saltos de linea. El asistente los lee como un unico turno. Si no hay nada nuevo, devuelve null y el bucle sigue esperando.

9. Comando de salida

La salida es muy simple:

```
csharp
static bool EsComandoFinalizar(string texto) => texto.Contains(ComandoFinalizar,
StringComparison.OrdinalIgnoreCase);
```

Si el texto nuevo contiene /finalizar (sin importar mayusculas), el bucle hace break y el bloque finally detiene wacli sync.

10. Modulo de IA: AgenteSecretaria

AgenteSecretaria encapsula la interaccion con Microsoft.Extensions.AI. Es una clase con estado: guarda el cliente, las opciones y el historial de la conversacion.

```
csharp
sealed class AgenteSecretaria {
    readonly IChatClient cliente;
    readonly ChatOptions opciones;
    readonly List<ChatMessage> conversacion;

    public AgenteSecretaria(string claveApi, string modelo, string promptSistema) {
        cliente = CrearCliente(claveApi, modelo);
        opciones = CrearOpciones();
        conversacion = [new(ChatRole.System, promptSistema)];
    }
    ...
}
```

El constructor arma todo de una vez: crea el cliente, registra las herramientas y deja el historial inicial con el mensaje System. Por eso el flujo principal puede escribir simplemente new(claveApi, Modelo, promptSistema).

10.1. Crear el cliente

```
csharp
static IChatClient CrearCliente(string claveApi, string modelo) => new ChatClientBuilder(
    new OpenAIClient(claveApi).GetChatClient(modelo).AsIChatClient())
    .UseFunctionInvocation()
    .Build();
```

Piezas:

Pieza	Que hace
OpenAIClient(claveApi)	Crea el cliente del proveedor.
GetChatClient(modelo)	Selecciona el modelo.
AsIChatClient()	Lo adapta a la abstraccion MEAI.

Pieza	Que hace
ChatClientBuilder	Permite agregar comportamiento.
UseFunctionInvocation()	Permite que el modelo invoque herramientas C#.

10.2. Registrar herramientas

```
csharp
static ChatOptions CrearOpciones() => new() {
    Tools =
    [
        AIFunctionFactory.Create(ArchivosAgente.LeerArchivo),
        AIFunctionFactory.Create(ArchivosAgente.EscribirArchivo),
        AIFunctionFactory.Create(ArchivosAgente.ListarArchivos)
    ]
};
```

`AIFunctionFactory.Create` convierte metodos C# en herramientas disponibles para el modelo. El nombre del metodo es el nombre con el que el modelo invoca la herramienta.

10.3. Responder

```
csharp
public async Task<string> ProcesarMensaje(string textoWhatsapp) {
    Console.WriteLine($"[{DateTimeOffset.Now:yyyy-MM-dd HH:mm:ss}] usuario: {textoWhatsapp}");

    conversacion.Add(new(ChatRole.User, textoWhatsapp));

    ChatResponse respuesta = await cliente.GetResponseAsync(conversacion, opciones);
    conversacion.AddMessages(respuesta);

    return respuesta.Text.Trim();
}
```

El historial (`conversacion`) se actualiza en cada turno. Sin esto, el modelo no recordaria mensajes anteriores. Como el historial es un campo de la clase, no hay que pasarlo de un lado a otro: el asistente lo conserva entre llamadas.

11. Herramientas del agente: ArchivosAgente

El asistente necesita manipular archivos porque administra `CONTACTOS.md`.

Las herramientas son:

```
csharp
public static async Task<string> LeerArchivo(string rutaRelativa)
public static async Task<string> EscribirArchivo(string rutaRelativa, string contenido)
public static string ListarArchivos(string patron = "*")
```

Usan atributos `Description` para que el modelo entienda como llamarlas:

csharp

```
[Description("Lee un archivo de texto dentro de la carpeta de trabajo.")]
public static async Task<string> LeerArchivo([Description("Ruta relativa del archivo a leer.")] string
rutaRelativa) {
    string ruta = ObtenerRutaSegura(rutaRelativa);
    if (!File.Exists(ruta)) { return $"No encuentre el archivo: {rutaRelativa}"; }
    return await File.ReadAllTextAsync(ruta);
}
```

11.1. Seguridad de rutas

El asistente no debe leer ni escribir fuera del directorio de trabajo. Por eso se valida la ruta:

csharp

```
static string ObtenerRutaSegura(string rutaRelativa) {
    string raiz = Path.GetFullPath(RaizTrabajo);
    string rutaCompleta = Path.GetFullPath(Path.Combine(raiz, rutaRelativa));

    if (!rutaCompleta.StartsWith(raiz + Path.DirectorySeparatorChar, StringComparison.Ordinal) &&
        rutaCompleta != raiz) {
        throw new InvalidOperationException("La ruta debe estar dentro de la carpeta de trabajo.");
    }

    return rutaCompleta;
}
```

Esto evita rutas como ../../otro-archivo.

12. Como funciona wacli

wacli es una CLI para WhatsApp. La aplicacion no implementa el protocolo de WhatsApp; delega esa parte a wacli.

Segun su ayuda:

texto

```
wacli is a WhatsApp CLI for syncing, searching, and sending from local scripts.
```

Conceptualmente, wacli hace tres cosas importantes para este bot:

1. Autentica una cuenta de WhatsApp.
2. Mantiene una base local de mensajes y contactos.
3. Permite leer y enviar mensajes desde comandos de terminal.

12.1. Autenticacion

Antes de correr el bot, la cuenta debe estar autenticada:

bash

```
wacli auth
```

Normalmente esto muestra un QR o inicia el flujo de autenticacion. Una vez autenticado, `wac li sync` puede sincronizar sin volver a mostrar QR.

12.2. Sincronizacion

El comando importante es:

```
bash
wac li sync --follow --refresh-contacts
```

Significado:

Parte	Funcion
sync	Sincroniza mensajes desde WhatsApp hacia la base local.
--follow	Mantiene la sincronizacion activa.
--refresh-contacts	Actualiza contactos en la base local.

`wac li sync` es el proceso que mantiene vivos los datos. El bot lo arranca en segundo plano al inicio.

12.3. Leer mensajes

Para leer mensajes usamos:

```
bash
wac li --json messages list --chat "+5493815343458" --limit 50
```

Significado:

Parte	Funcion
--json	Devuelve JSON en lugar de una tabla humana.
messages list	Lista mensajes desde la base local.
--chat	Filtra por chat/contacto.
--limit 50	Devuelve los ultimos 50.

No usamos `--from-them`: leemos tambien nuestros propios mensajes a proposito, porque el filtro en C# necesita ver el ultimo mensaje enviado por nosotros para ubicar la linea divisoria (ver seccion 7).

12.4. Enviar mensajes

Para responder usamos:

```
bash
wac li send text --to "+5493815343458" --message "Texto de respuesta"
```

Significado:

Parte	Funcion
send text	Envia un mensaje de texto.
--to	Destinatario: numero, JID, nombre de contacto o chat.
--message	Texto a enviar.

12.5. JSON de salida

La salida JSON de messages list tiene una estructura similar a:

```
json
{
  "success": true,
  "data": {
    "messages": [
      {
        "ChatJID": "5493815343458@s.whatsapp.net",
        "ChatName": "Alejandro Di Battista",
        "MsgID": "AC35583875E6E18ACAEB27005FCD14E8",
        "Timestamp": "2026-06-23T22:51:55Z",
        "FromMe": false,
        "Text": "Hola",
        "DisplayText": "Hola"
      }
    ]
  },
  "error": null
}
```

El bot solo necesita algunos campos:

Campo	Uso
MsgID	Identificar el mensaje y desempatar el orden.
ChatJID	Identificar contacto/chat.
Text	Texto que va al asistente.
DisplayText	Texto alternativo si Text esta vacio.
FromMe	Distinguir mensajes propios de los entrantes.
Timestamp	Ordenar mensajes y ubicar la linea divisoria.

13. Construir el modulo ClienteWhatsApp

ClienteWhatsApp es un adaptador. Su objetivo es esconder que estamos usando comandos externos.

El resto de la app no deberia saber si WhatsApp viene de wac li, una API HTTP o una base de datos. Solo necesita estos metodos:

```
csharp
public static Process IniciarSincronizacion()
public static void DetenerSincronizacion(Process? procesoSincronizacion)
public static Task<string?> RecibirTexto(string contacto, DateTimeOffset? desde = null)
public static Task EnviarTexto(string contacto, string mensaje)
```

13.1. Constante del modulo

```
csharp
static class ClienteWhatsApp {
    const string EjecutableWacli = "wacli";
    ...
}
```

Los argumentos de cada comando se arman en cada metodo como una lista de strings (ver 13.4 y 14). Si cambia la sintaxis de wacli, se toca este modulo y no el bot completo.

13.2. Iniciar wacli sync en segundo plano

```
csharp
public static Process IniciarSincronizacion() {
    ProcessStartInfo infoInicio = CrearInfoInicioWacli(redirigir: false, "sync", "--follow",
"--refresh-contacts");
    Process proceso = Process.Start(infoInicio) ?? throw new InvalidOperationException("No se pudo
iniciar la sincronizacion de WhatsApp con wacli.");
    Console.WriteLine($"Sincronizacion de WhatsApp iniciada con wacli. PID: {proceso.Id}");
    return proceso;
}
```

Punto clave: para sync no redirigimos la salida (redirigir: false). Es un proceso persistente que queda vivo mientras el bot corre.

13.3. Detener wacli sync

```
csharp
public static void DetenerSincronizacion(Process? procesoSincronizacion) {
    if (procesoSincronizacion is null) { return; }
    if (!procesoSincronizacion.HasExited) { procesoSincronizacion.Kill(entireProcessTree: true); }
    procesoSincronizacion.Dispose();
}
```

Como el bot solo termina con /finalizar, este metodo se llama desde finally para garantizar que wacli sync no quede vivo.

13.4. Enviar texto desde un comando externo

```
csharp
public static async Task EnviarTexto(string contacto, string mensaje) {
    CommandResult resultado = await EjecutarWacli("send", "text", "--to", contacto, "--message",
mensaje);
    if (resultado.ExitCode != 0) { throw new InvalidOperationException($"No se pudo enviar el mensaje
con wacli: {resultado.Stderr}"); }
}
```

Aca, si falla el envio, lanzamos excepcion porque el bot no pudo cumplir su accion principal. Notar que el contacto y el mensaje se pasan como argumentos sueltos, no concatenados en un string.

14. Ejecutar comandos externos desde C#

Esta es una parte clave del tutorial.

En C#, un comando externo se ejecuta con `Process` y `ProcessStartInfo`.

14.1. Configurar el proceso

```
csharp
static ProcessStartInfo CrearInfoInicioWacli(bool redirigir, params string[] argumentos) {
    ProcessStartInfo infoInicio = new(EjecutableWacli) {
        UseShellExecute = false,
        RedirectStandardOutput = redirigir,
        RedirectStandardError = redirigir,
        CreateNoWindow = true,
        WorkingDirectory = Environment.CurrentDirectory
    };
    foreach (string argumento in argumentos) { infoInicio.ArgumentList.Add(argumento); }
    return infoInicio;
}
```

Cada propiedad importa:

Propiedad	Explicacion
<code>UseShellExecute = false</code>	Ejecuta el proceso directamente, no a traves del shell.
<code>RedirectStandardOutput</code>	Si es true, permite leer la salida normal.
<code>RedirectStandardError</code>	Si es true, permite leer errores.
<code>CreateNoWindow = true</code>	No abre otra ventana de terminal.
<code>WorkingDirectory</code>	Ejecuta desde el directorio actual.

El parametro `redirigir` permite reusar el mismo metodo para sync (sin redirigir) y para los comandos que leen salida (redirigiendo).

14.2. Argumentos seguros con `ArgumentList`

En lugar de armar un unico string de argumentos y escapar comillas a mano, cada argumento se agrega por separado a `ArgumentList`:

```
csharp
infoInicio.ArgumentList.Add(argumento);
```

Esto es importante: los contactos y mensajes pueden tener espacios o comillas. Al pasar cada argumento por separado, el sistema operativo se encarga del quoting de forma segura. Por ejemplo, un mensaje como:

```
texto
Hola "Juan", como estas?
```

llega intacto al proceso, sin que tengamos que escapar nada manualmente. Por eso los metodos llaman a `EjecutarWaccli("send", "text", "--to", contacto, "--message", mensaje)` en vez de concatenar un comando.

14.3. Ejecutar y capturar salida

```
csharp
static async Task<CommandResult> EjecutarWaccli(params string[] argumentos) {
    ProcessStartInfo infoInicio = CrearInfoInicioWaccli(redirigir: true, argumentos);
    using Process proceso = Process.Start(infoInicio) ?? throw new InvalidOperationException("No se pudo ejecutar waccli.");

    Task<string> salidaEstandar = proceso.StandardOutput.ReadToEndAsync();
    Task<string> errorEstandar = proceso.StandardError.ReadToEndAsync();
    await proceso.WaitForExitAsync();

    return new(proceso.ExitCode, await salidaEstandar, await errorEstandar);
}
```

La funcion devuelve un record simple:

```
csharp
sealed record CommandResult(int ExitCode, string StdOut, string StdErr);
```

Esto es mejor que devolver solo un string porque distingue:

- Si el comando funciona (ExitCode).
- Que imprimio normalmente (StdOut).
- Que imprimio como error (StdErr).

14.4. Por que leer stdout y stderr

Un programa de terminal tiene dos salidas:

Salida	Uso
stdout	Resultado normal. En nuestro caso, JSON de mensajes.
stderr	Errores o advertencias.

Si `waccli messages list` funciona, el JSON viene por stdout. Si falla, el motivo suele venir por stderr.

15. Parsear la salida JSON de waccli

Primero se definen clases que representan el JSON. Estos tipos conservan los nombres originales de los campos de `waccli`:

```
csharp
sealed class WacliMessageList {
    public WacliMessageData? Data { get; set; }
}

sealed class WacliMessageData {
    public List<WacliStoredMessage> Messages { get; set; } = [];
}

sealed class WacliStoredMessage {
    public string? MsgID { get; set; }
    public string? ChatJID { get; set; }
    public string? ChatName { get; set; }
    public string? SenderJID { get; set; }
    public string? SenderName { get; set; }
    public string? Text { get; set; }
    public string? DisplayText { get; set; }
    public string? Snippet { get; set; }
    public bool FromMe { get; set; }
    public DateTimeOffset? Timestamp { get; set; }
}
```

Luego se deserializa dentro de ParsearMensajes:

```
csharp
static IReadOnlyList<InboundMessage> ParsearMensajes(string salida, string contactoPorDefecto) {
    try {
        WacliMessageList? respuesta = JsonSerializer.Deserialize(salida,
        WhatsAppJsonContext.Default.WacliMessageList);
        List<WacliStoredMessage> mensajes = respuesta?.Data?.Messages ?? [];

        return mensajes.Select(AMensajeEntrante).ToList();
    } catch (JsonException) {
        return salida
            .Split('\n', StringSplitOptions.RemoveEmptyEntries | StringSplitOptions.TrimEntries)
            .Select((linea, indice) => new InboundMessage($"line:{indice}:{linea.GetHashCode()}",
            contactoPorDefecto, linea, false, DateTimeOffset.UtcNow))
            .ToList();
    }
}
```

El catch (JsonException) es un respaldo: si la salida no fuera JSON valido, igual se arma una lista basica para no romper el bot.

Cada mensaje del JSON se transforma a un tipo interno mas simple:

```
csharp
static InboundMessage AMensajeEntrante(WacliStoredMessage mensaje) => new(
    Id: mensaje.MsgID ?? Guid.NewGuid().ToString("N"),
    Contact: mensaje.ChatJID ?? mensaje.SenderJID ?? mensaje.ChatName ?? mensaje.SenderName ??
    string.Empty,
    Text: PrimerNoVacio(mensaje.Text, mensaje.DisplayText, mensaje.Snippet) ?? string.Empty,
    FromMe: mensaje.FromMe,
    Timestamp: mensaje.Timestamp);
```

El tipo que usa el bot es:

```
csharp
sealed record InboundMessage(string Id, string Contact, string Text, bool FromMe, DateTimeOffset?
Timestamp);
```

PrimerNoVacio elige el primer texto disponible entre Text, DisplayText y Snippet:

csharp

```
static string? PrimerNoVacio(params string?[] valores) => valores.FirstOrDefault(valor =>
!string.IsNullOrEmpty(valor));
```

Esto mantiene simple al resto del programa.

16. JsonSerializerContext

El archivo usa source generation de JSON:

csharp

```
[JsonSourceGenerationOptions(PropertyNameCaseInsensitive = true)]
[JsonSerializable(typeof(WacLiMessageList))]
partial class WhatsAppJsonContext : JsonSerializerContext;
```

En una línea: le estamos diciendo a `System.Text.Json` que estructura esperamos recibir de `wacLi`.

`PropertyNameCaseInsensitive = true` permite que el parseo no dependa estrictamente de mayúsculas y minúsculas.

17. Prompt desde AGENTS.md

El prompt se carga desde archivo:

csharp

```
static async Task<string> LeerPromptSistema(string archivoAgentes, string promptPorDefecto) {
    if (!File.Exists(archivoAgentes)) { return promptPorDefecto; }
    string prompt = await File.ReadAllTextAsync(archivoAgentes);
    return string.IsNullOrEmpty(prompt) ? promptPorDefecto : prompt;
}
```

Esto permite cambiar el comportamiento del bot sin tocar código.

18. Ejecutar el bot

Primero configurar la clave:

bash

```
export OPENAI_API_KEY="tu-clave"
```

Luego correr:

bash

```
dotnet run WhatsAppBot.cs
```

Salida esperada:


```
texto
```

```
Sincronizacion de WhatsApp iniciada con wacli. PID: 12345  
Bot activo para +5493815343458. Envía /finalizar por WhatsApp para detenerlo.
```

Cuando llega un mensaje:

```
texto
```

```
[2026-06-23 20:10:00] usuario: Hola
```

Para detenerlo, enviar por WhatsApp:

```
texto
```

```
/finalizar
```

19. Probar por partes

Si algo falla, probar cada capa.

19.1. Probar lectura de WhatsApp

```
bash
```

```
wacli --json messages list --chat "+5493815343458" --limit 5
```

Si no devuelve mensajes, el problema está en `wac li`, la autenticación o el contacto.

19.2. Probar envío de WhatsApp

```
bash
```

```
wacli send text --to "+5493815343458" --message "Prueba desde wacli"
```

Si esto falla, el bot tampoco podrá responder.

19.3. Probar compilación

```
bash
```

```
dotnet build WhatsAppBot.cs
```

19.4. Probar clave de OpenAI

```
bash
```

```
printenv OPENAI_API_KEY
```

20. Errores comunes

Problema	Causa probable	Solucion
No responde	No llegan mensajes nuevos.	Probar <code>wacli messages list</code> .
Responde mensajes viejos	El reloj o la marca de arranque no coinciden.	Revisar <code>iniciadoEn</code> y <code>FiltrarMensajesNuevos</code> .
Falla IA	Falta <code>OPENAI_API_KEY</code> .	Exportar la variable.
Falla envio	<code>wacli send text</code> no resuelve el contacto.	Probar envio manual.
No modifica archivos	El modelo no invoca herramientas.	Revisar <code>UseFunctionInvocation</code> y <code>ChatOptions.Tools</code> .
Queda <code>wacli sync</code> vivo	No llego <code>/finalizar</code> .	Enviar <code>/finalizar</code> desde WhatsApp.

21. Resumen para explicar en clase

Una explicacion corta:

texto

El bot arranca `wacli sync`, lee los ultimos mensajes con `wacli messages list`, descarta lo viejo usando la hora de arranque y su ultimo mensaje enviado, usa `Microsoft.Extensions.AI` para generar una respuesta con herramientas de archivos y envia la respuesta con `wacli send text`. El ciclo sigue hasta recibir `/finalizar`.

Secuencia conceptual:

texto

1. `wacli sync` mantiene la base local actualizada.
2. C# consulta la base local con `wacli messages list`.
3. C# filtra mensajes nuevos por marca de tiempo y ultimo mensaje propio.
4. `Microsoft.Extensions.AI` genera la respuesta.
5. Las herramientas permiten leer/escribir archivos.
6. C# envia la respuesta usando `wacli send text`.
7. `/finalizar` corta el bucle.

22. Estructura final del archivo

texto

1. Directivas file-based y paquetes.
2. `using`.
3. Constantes.
4. Validacion de `OPENAI_API_KEY`.
5. Flujo principal (bucle `while`).
6. Funciones auxiliares: `EsComandoFinalizar`, `LeerPromptSistema`.
7. `AgenteSecretaria`.
8. `ArchivosAgente`.
9. `ClienteWhatsApp`.
10. Contexto JSON y tipos de datos (`WhatsAppJsonContext`, `Wacli*`, `InboundMessage`, `CommandResult`).

La clave pedagogica es que cada clase responde una pregunta:

Clase / modulo	Pregunta que responde
Flujo principal	Que hace el bot todo el tiempo?
AgenteSecretaria	Como hablo con la IA y recuerdo la conversacion?
ArchivosAgente	Que acciones puede hacer la IA?
ClienteWhatsApp	Como hablo con WhatsApp mediante wac li?

CAPITULO 52

Microsoft Agent Framework, desde los primeros principios

Microsoft.Agents.AI (MAF 1.0)

Antes de empezar: el porqué

Hay una pregunta que conviene contestar antes de tocar una sola línea de código: **¿qué problema resuelve un framework de agentes que no resuelva ya un cliente de chat?**

Cuando usás `IChatClient` (la abstracción de MEAI que ya vimos en clase), el flujo es lineal y vos sos el dueño del bucle:

texto

vos → armás mensajes → modelo responde → vos leés la respuesta → vos decidís qué hacer

Si el modelo quiere usar una herramienta, vos tenés que detectar el pedido, ejecutar la función, volver a meter el resultado en la conversación y reinvocar. Si querés que se acuerde de la charla anterior, vos acumulás los mensajes. Si querés que varios modelos colaboren, vos coordinás todo a mano.

Eso es exactamente lo que hicimos cuando construimos el loop ReAct "a mano": funciona, pero todo el andamiaje queda a tu cargo.

Un **agente** invierte esa relación. En vez de que vos manejes el bucle, le pasás un objetivo y *él* decide:

texto

vos → le das un objetivo → el agente razona, elige herramientas, las ejecuta, evalúa resultados, decide el próximo paso → te devuelve el resultado

El Microsoft Agent Framework (MAF de ahora en más) no es magia nueva: es **el bucle ReAct que ya escribiste, pero industrializado**. Te da, listos para usar, los cuatro andamiajes que antes hacías a mano:

1. **El bucle de ejecución** (modelo → herramienta → modelo → ...) hasta terminar.
2. **El estado de la conversación** (memoria de corto plazo, dentro de una sesión).
3. **La inyección de contexto** (memoria de largo plazo y RAG, entre sesiones).
4. **La orquestación de varios agentes** (workflows con grafos).

La clave pedagógica de todo el tutorial es esta: **MAF se construye literalmente encima de `IChatClient`**. No reemplaza lo que sabés; lo envuelve. El método estrella se llama `.AsIAgent()` y hace exactamente lo que su nombre dice: agarra un cliente de chat y lo convierte en un agente.

El linaje, para ubicarte

MAF es la fusión de dos proyectos previos de Microsoft:

- **Semantic Kernel** aportó lo "empresarial": manejo de estado, telemetría, middleware, type safety.
- **AutoGen** aportó las abstracciones simples para agentes y orquestación multi-agente.

MAF es el sucesor directo de ambos, escrito por los mismos equipos, y llegó a su versión 1.0 (APIs estables, soporte a largo plazo) en abril de 2026. Funciona igual en .NET y en Python; nosotros vamos en C#.

No te hace falta Azure para aprenderlo. Funciona con OpenAI, Azure OpenAI, GitHub Models, Anthropic (Claude), o modelos locales vía Ollama y Foundry Local. Cualquier cosa que exponga un `IChatClient` sirve.

El modelo mental central: cuatro piezas

Antes de los ejemplos, fijate este mapa. Todo lo que sigue es una de estas cuatro cosas:

Pieza	Qué es	El "a mano" que reemplaza
AIAgent	La unidad autónoma. Razona y actúa.	Tu loop ReAct entero.
AgentSession	El hilo de conversación. Memoria de corto plazo.	Tu <code>List<ChatMessage></code> que ibas acumulando.
AIContextProvider	Inyecta contexto antes/después de cada turno. Memoria de largo plazo, RAG.	El <code>string</code> de "datos del usuario" que pegabas al system prompt.
Workflow	Un grafo de ejecutores conectados por aristas. Orquesta varios agentes.	El <code>if/else</code> con el que decidías a qué modelo llamar.

Vamos a recorrerlas en orden de complejidad creciente.

Nivel 0 — Setup

Proyecto de consola y el paquete base:

```
bash
dotnet new console -n MiPrimerAgente
cd MiPrimerAgente
dotnet add package Microsoft.Agents.AI
```

Después agregás el paquete del proveedor que vayas a usar. Para OpenAI directo:

```
bash
dotnet add package OpenAI
```

Para Azure OpenAI:

```
bash
dotnet add package Azure.AI.OpenAI
dotnet add package Azure.Identity
```

Vamos a usar variables de entorno para las credenciales, nunca hardcodeadas. En el código las lees con `Environment.GetEnvironmentVariable(...)`.

Nivel 1 — Tu primer agente

El "hola mundo". Acá lo importante no es lo que hace (cuenta un chiste), sino **ver la cadena de transformación**: SDK del proveedor → cliente de chat → agente.

```
csharp
using OpenAI;
using Microsoft.Agents.AI;

var apiKey = Environment.GetEnvironmentVariable("OPENAI_API_KEY")
    ?? throw new InvalidOperationException("Falta OPENAI_API_KEY");

// 1. El SDK crudo del proveedor.
// 2. .GetChatClient(modelo) te da el sub-cliente del modelo.
// 3. .AsIAgent(...) lo envuelve en un agente.
IAgent agente = new OpenAIClient(apiKey)
    .GetChatClient("gpt-5.4-mini")
    .AsIAgent(
        instructions: "Sos bueno contando chistes. Respondé en español rioplatense.",
        name: "Chistoso");

Console.WriteLine(await agente.RunAsync("Contame un chiste de piratas."));
```

Tres ideas para no perder de vista:

`instructions` **es el system prompt**. Lo que antes ponías como primer mensaje con `ChatRole.System`, ahora vive en el agente. Define su personalidad y sus reglas, y se aplica en cada turno automáticamente.

`RunAsync` **es el reemplazo de tu bucle**. Una sola llamada. Adentro, MAF arma el mensaje, llama al modelo, y si hubiera herramientas las gestionaría (lo vemos en el Nivel 2). Devolvés y listo.

`IAgent` **es el tipo base abstracto**. Da igual si abajo hay OpenAI, Claude o un modelo local: el resto de tu código habla con `IAgent` y no se entera del proveedor. Eso es lo que después permite armar orquestaciones agnósticas.

Streaming

Si querés ver la respuesta token a token en vez de esperar el bloque completo:

```
csharp
await foreach (var update in agente.RunStreamingAsync("Contame un chiste de piratas. ")) {
    Console.Write(update);
}
```

Mismo agente, distinta forma de consumir. `RunAsync` espera todo; `RunStreamingAsync` te va dando pedazos.

Nivel 2 — Darle herramientas

Acá es donde un agente deja de ser un chatbot. Una **herramienta** es una función de C# que el modelo puede decidir llamar. Vos no escribís "si pregunta por el clima, llamá a tal función": el modelo lo decide solo.

El porqué: el modelo no sabe el clima de Tucumán hoy, pero puede *darse cuenta* de que tiene una función que sí lo sabe, llamarla con el parámetro correcto, y usar el resultado para responder.

```
csharp
using System.ComponentModel;
using OpenAI;
using Microsoft.Agents.AI;
using Microsoft.Extensions.AI;

// Una función común y corriente. Lo importante son los [Description].
[Description("Obtiene el clima para una ubicación dada.")]
static string ObtenerClima([Description("La ciudad para la que se quiere el clima.")] string ciudad)
    => $"En {ciudad} está nublado con una máxima de 15°C.";

var apiKey = Environment.GetEnvironmentVariable("OPENAI_API_KEY");

AIAgent agente = new OpenAIClient(apiKey)
    .GetChatClient("gpt-5.4-mini")
    .AsAIAgent( instructions: "Sos un asistente útil.",
        tools: [AIFunctionFactory.Create(ObtenerClima)]);

Console.WriteLine(await agente.RunAsync("¿Cómo está el clima en Yerba Buena?"));
```

Lo que pasa adentro de ese único RunAsync es **exactamente tu loop ReAct**, pero MAF lo corre por vos:

1. Manda tu pregunta + la lista de herramientas al modelo.
2. El modelo responde: "quiero llamar a ObtenerClima con ciudad = Yerba Buena".
3. MAF ejecuta tu función de C#.
4. MAF mete el resultado de vuelta en la conversación.
5. Vuelve a llamar al modelo, que ahora redacta la respuesta final.

Vos escribiste *cero* líneas de "detectar pedido de tool / ejecutar / reinyectar". Esa es la diferencia.

Por qué los [Description] no son opcionales

Los atributos [Description] son el manual de instrucciones de la herramienta *para el modelo*. El modelo no lee el cuerpo de tu función; lee el nombre, las descripciones y los tipos. Una descripción vaga lleva a que la herramienta se use mal o no se use. Pensalos como parte del prompt, no como documentación decorativa.

AIFunctionFactory.Create(...) es el puente: toma tu Func/método de C# y genera el esquema (nombre, parámetros, tipos, descripciones) que el modelo necesita para decidir. Es la misma fábrica de MEAI, así que cualquier herramienta que ya hayas definido para un IChatClient sirve acá tal cual.

Nivel 3 — Conversaciones multi-turno con sesiones

Hasta ahora cada RunAsync es amnésico: no recuerda nada del anterior. Para una charla real necesitás que el agente sepa de qué venían hablando.

El porqué: cuando el usuario dice "ahora agregale emojis a ese chiste", el "ese" solo tiene sentido si hay memoria del turno anterior. Esa memoria de corto plazo es la **sesión**.

```
csharp
AgentSession sesion = await agente.CreateSessionAsync();

Console.WriteLine( await agente.RunAsync("Contame un chiste de piratas.", sesion));

Console.WriteLine( await agente.RunAsync( "Ahora agregale emojis y contámelo con voz de loro pirata.",
sesion));
```

La diferencia con el Nivel 1 es un solo parámetro: le pasás la *sesion* a cada *RunAsync*. La sesión va guardando el historial entre llamadas. El segundo turno "sabe" cuál chiste, porque la sesión lo conserva.

Acá la sesión reemplaza tu vieja *List<ChatMessage>* que ibas llenando a mano. Pero hace algo más, clave para producción:

Serializar la sesión

Un servicio web es *stateless*: cada request puede caer en una instancia distinta del servidor, que no tiene la memoria en RAM. La solución es guardar el estado de la sesión (en una base, en Redis, donde sea) y recuperarlo después.

```
csharp
// Guardás el estado (por ejemplo, en tu base de datos)
JsonElement estado = await agente.SerializeSessionAsync(sesion);

// Más tarde, en otro request, lo recuperás
var sesionRestaurada = await agente.DeserializeSessionAsync(estado);

Console.WriteLine( await agente.RunAsync("¿De qué estábamos hablando?", sesionRestaurada));
```

Esto es lo que permite que un agente "recuerde" una conversación que empezó hace horas, en otra instancia del servidor. La memoria vive afuera del proceso, no en una variable.

Nivel 4 — Memoria de largo plazo con AIContextProvider

La sesión recuerda *esta* conversación. ¿Y si querés que el agente recuerde cosas del usuario **entre** conversaciones distintas? Su nombre, sus preferencias, lo que hablaron la semana pasada. O que, antes de responder, busque documentos relevantes en tu base vectorial (RAG).

Para eso está *AIContextProvider*. Es un objeto que se engancha en dos momentos del ciclo de vida de cada turno:

- *ProvideAIContextAsync* corre *antes* de cada interacción. Inyecta contexto extra (datos del usuario, documentos recuperados, instrucciones dinámicas).
- *StoreAIContextAsync* corre *después* de cada interacción. Es la oportunidad del agente de *aprender* de lo que acaba de pasar (extraer un dato y guardarlo).

El porqué de separar esto: desacoplás **qué recuerda el agente de cómo conversa**. La lógica de memoria vive en una clase aparte, y podés apilar varios providers (uno para preferencias, otro para historial, otro que

hace RAG contra tu VectorData).

Ejemplo: un provider que extrae y recuerda el nombre del usuario.

```
csharp
internal sealed class MemoriaUsuario : AIContextProvider {
    private readonly ProviderSessionState<InfoUsuario> _estado;
    private readonly IChatClient _chatClient;

    public MemoriaUsuario(IChatClient chatClient) {
        this._estado = new ProviderSessionState<InfoUsuario>(
            _ => new InfoUsuario(),
            GetType().Name);
        this._chatClient = chatClient;
    }

    // DESPUÉS del turno: aprender. ¿Apareció el nombre? Lo guardo.
    protected override async ValueTask StoreAIContextAsync(
        InvokedContext context,
        CancellationToken cancellationToken = default) {
        var info = this._estado.GetOrInitializeState(context.Session);

        if (info.Nombre is null && context.RequestMessages.Any(m => m.Role == ChatRole.User)) {
            var resultado = await this._chatClient.GetResponseAsync<InfoUsuario>(
                context.RequestMessages,
                new ChatOptions() {
                    Instructions = "Extraé el nombre del usuario del mensaje, si aparece."
                },
                cancellationToken: cancellationToken);

            info.Nombre ??= resultado.Result.Nombre;
        }

        this._estado.SaveState(context.Session, info);
    }

    // ANTES del turno: inyectar lo que ya sé.
    protected override ValueTask<AIContext> ProvideAIContextAsync(
        InvokingContext context,
        CancellationToken cancellationToken = default) {
        var info = this._estado.GetOrInitializeState(context.Session);

        var instrucciones = info.Nombre is null
            ? "Pedile el nombre al usuario."
            : $"El nombre del usuario es {info.Nombre}.";

        return new ValueTask<AIContext>(
            new AIContext { Instructions = instrucciones });
    }
}
```

Y lo enchufás al crear el agente:

```
csharp
IAgent agente = chatClient.AsIAgent(new ChatClientAgentOptions() {
    ChatOptions = new() {
        Instructions = "Sos un asistente amable. Siempre llamá al usuario por su nombre."
    },
    AIContextProviders = [new MemoriaUsuario(chatClient.AsIChatClient())]
});
```

Fijate el patrón: el provider *no* conversa. Solo decide qué meterle al contexto antes y qué guardar después. El agente conversa. Responsabilidades separadas.

Acá se cierra el círculo con lo que vimos de RAG y VectorData: un AIContextProvider que, en ProvideAIContextAsync, hace una búsqueda semántica en tu base vectorial y devuelve los documentos relevantes como contexto. Eso convierte el RAG manual en algo que pasa solo, en cada turno.

Nivel 5 — Workflows: orquestar varios agentes

Un solo agente alcanza para mucho. Pero muchos problemas reales se resuelven mejor repartiendo el trabajo entre **agentes especializados** que colaboran. Para eso MAF tiene un sistema de **workflows basado en grafos**.

El vocabulario, que es lo central:

- **Ejecutor (executor)**: una unidad de procesamiento. Puede ser un agente, o una función simple.
- **Arista (edge)**: una conexión que define el flujo de datos. "La salida de A entra a B."
- **WorkflowBuilder**: el constructor con el que armás el grafo.

El porqué de pensarlo como grafo: en vez de un `if/else` enredado decidiendo a mano quién llama a quién, **declarás la topología** (quién se conecta con quién y bajo qué condición) y dejás que el framework ejecute el recorrido.

El workflow más simple posible

Para entender el mecanismo sin distracciones, un workflow de dos ejecutores que ni siquiera usan IA: uno pasa a mayúsculas, otro invierte el texto.

```
csharp
using Microsoft.Agents.AI.Workflows;

Func<string, string> aMayusculas = s => s.ToUpperInvariant();
var mayus = aMayusculas.BindAsExecutor("Mayusculas");
var invertir = new InvertirTextoExecutor();

// El builder arranca en el ejecutor inicial.
WorkflowBuilder builder = new(mayus);

// Conectás: salida de 'mayus' → entra a 'invertir'.
// Y declarás que la salida final del workflow viene de 'invertir'.
builder.AddEdge(mayus, invertir).WithOutputFrom(invertir);

var workflow = builder.Build();

// Lo corrés. La entrada "Hello, World!" recorre el grafo.
await using Run run = await InProcessExecution.RunAsync(workflow, "Hello, World!");

foreach (WorkflowEvent evt in run.NewEvents) {
    if (evt is ExecutorCompletedEvent ec) {
        Console.WriteLine($"{ec.ExecutorId}: {ec.Data}");
    }
}
```

Lo que tenés que sacar de acá: **el dato fluye por las aristas**. Vos no llamás a cada ejecutor en orden; declarás el grafo (`AddEdge`) y `RunAsync` se encarga del recorrido. Los `WorkflowEvent` son cómo observás qué fue pasando paso a paso (útil para debugging y telemetría).

Patrones que el grafo habilita

Con ese mismo mecanismo de ejecutores + aristas, MAF soporta los patrones clásicos de multi-agente:

- **Secuencial**: los agentes procesan en fila, la salida de uno alimenta al siguiente.

- **Concurrente:** se abre en abanico a varios agentes en paralelo (*fan-out*) y después se juntan los resultados (*fan-in*).
- **Ruteo condicional (hand-off):** se decide dinámicamente a qué agente mandar el trabajo según la salida del paso anterior.
- **Bucles de feedback:** el patrón escritor-crítico, donde uno produce y otro evalúa, repitiendo hasta que la calidad sea suficiente.
- **Sub-workflows:** componés workflows metiendo uno dentro de otro, jerárquicamente.

El patrón escritor-crítico

Es uno de los más útiles y muestra el poder de las aristas condicionales. Un agente escribe (digamos, copy de marketing) y otro lo revisa. Si el crítico no lo aprueba, vuelve al escritor. Loop hasta que pase.

```
csharp
WorkflowBuilder builder = new(agenteEscritor);

builder
    // El escritor manda su borrador al crítico.
    .AddEdge(agenteEscritor, agenteCritico)
    // Si NO está aprobado, vuelve al escritor para revisión.
    .AddEdge(agenteCritico, agenteEscritor, condition: r => !r.IsApproved)
    // Si SÍ está aprobado, esa es la salida del workflow.
    .WithoutOutputFrom(agenteCritico, condition: r => r.IsApproved);

var workflow = builder.Build();
```

La elegancia está en las **condiciones sobre las aristas**: `condition: r => !r.IsApproved`. La topología del grafo es la lógica de control. No hay un `while` escrito a mano; el ciclo emerge de cómo conectaste los nodos.

Un consejo de producción: ponele siempre un tope de iteraciones a estos bucles. Un crítico demasiado exigente puede mandar a revisar para siempre (y eso son tokens y plata).

Nivel 6 — Humano en el loop

No todo lo que un agente quiere hacer debería hacerlo solo. Escribir en una base, mandar un mail, mover plata: ahí querés que un humano apruebe antes.

MAF soporta esto con un flujo de aprobación de herramientas, construido sobre dos tipos del modelo de contenido de MEAI: `FunctionApprovalRequestContent` y `FunctionApprovalResponseContent`.

El mecanismo conceptual:

1. El agente decide que quiere llamar a una herramienta sensible.
2. En vez de ejecutarla, **emite un pedido de aprobación y se frena**.
3. Tu código le muestra el pedido al usuario ("el agente quiere borrar 50 registros, ¿confirmás?").
4. La respuesta del usuario determina si la llamada procede o no.

Pensá los agentes como trabajadores especializados dirigidos por humanos a través del código. La autonomía es una herramienta, no un fin: la das donde es barata de equivocarse y la retenés donde es cara.

Nivel 7 — Herramientas externas con MCP

Volvé un momento al Nivel 2. Ahí una herramienta era una función de C# tuya, envuelta con `AIFunctionFactory.Create(...)` y pasada en `tools: [...]`. Funciona perfecto mientras la capacidad viva en tu propio código.

¿Pero qué pasa cuando la capacidad vive *afuera*? Un servidor que sabe leer el filesystem, otro que habla con GitHub, otro que consulta una base corporativa, quizás escritos en Python o Rust por otro equipo. Antes de MCP, cada uno de esos era un wrapper a mano: leías su API, escribías el cliente, lo adaptabas a tool. Una integración distinta por cada sistema.

El **Model Context Protocol (MCP)** es el estándar que elimina ese trabajo repetido. Pensalo como una API universal para herramientas de agentes: en vez de construir una conexión a medida para cada servicio, el agente se conecta a un *endpoint MCP* que expone una lista de herramientas, y el agente las usa.

La idea central, y el motivo por el que esto encaja tan limpio en lo que ya sabés: **en MAF un servidor MCP no es una categoría especial de cosa, es simplemente otra fuente de herramientas**. Las tools que descubre de un servidor MCP se convierten automáticamente en objetos `AITool`, el mismo tipo que produce `AIFunctionFactory`. El agente no distingue el origen: una `AITool` es una `AITool`, venga de una función tuya o de un proceso externo que habla el protocolo. Por eso el lugar del código donde las enchufás es idéntico: `tools: [...]`.

El paquete

MCP vive en su propio SDK, separado del de agentes. Es el SDK oficial de C#, mantenido en colaboración entre Microsoft, Anthropic y la organización del protocolo abierto MCP:

```
bash
dotnet add package ModelContextProtocol --prerelease
```

El patrón en tres pasos

El caso que más vas a usar en clase es un servidor **local por stdio**, porque no depende de Azure ni de ninguna nube: arrancás el servidor como un proceso hijo en tu propia máquina.

```
csharp
using Microsoft.Agents.AI;
using Microsoft.Extensions.AI;
using ModelContextProtocol.Client;

// 1. Crear el cliente MCP. Levanta el servidor de filesystem como
//    proceso local vía npx. El último argumento acota a qué carpeta
//    puede acceder: ya estás poniendo un límite de seguridad.
await using var mcpClient = await McpClientFactory.CreateAsync(
    new StdioClientTransport(new() {
        Name = "Filesystem",
        Command = "npx",
        Arguments = ["-y", "@modelcontextprotocol/server-filesystem", "/ruta/permitida"]
    }));
```

csharp - continuacion

```
// 2. Descubrir las herramientas que expone el servidor (el "handshake").
var mcpTools = await mcpClient.ListToolsAsync();

// 3. Crear el agente pasándole esas tools, igual que cualquier otra.
var apiKey = Environment.GetEnvironmentVariable("OPENAI_API_KEY");
IAgent agente = new OpenAIClient(apiKey)
    .GetChatClient("gpt-5.4-mini")
    .AsIAgent(
        instructions: "Usá las herramientas disponibles para responder.",
        tools: [.. mcpTools.Cast<AITool>()]);

Console.WriteLine(await agente.RunAsync("¿Qué archivos hay en la carpeta?"));
```

Los dos momentos del protocolo que importan conceptualmente:

- **Discovery** (`ListToolsAsync`): el cliente le pregunta al servidor "¿qué sabés hacer?" y recibe la lista de tools con sus descripciones. Es el equivalente externo de los `[Description]` del Nivel 2.
- **Invocación**: cuando el modelo decide usar una tool, MAF rutea la llamada al servidor por el canal stdio, espera el resultado y lo reinyecta. Es el mismo loop ReAct de siempre, pero el paso "ejecutar la tool" cruza un límite de proceso.

El `await using` no es decorativo: asegura que la conexión se cierre bien al salir de scope. Como el servidor local es un proceso hijo, querés que se termine prolijamente y no quede colgado.

Los dos transportes

Según dónde viva el servidor, cambia el transporte:

- **stdio** (`StdioClientTransport`): el servidor es un proceso local que arrancás vos. Ideal para filesystem, git, bases locales. Es el del ejemplo.
- **SSE / HTTP** (`SseClientTransport`): el servidor es remoto, ya corriendo en un endpoint. Por ejemplo, el endpoint MCP de Microsoft Learn (<https://learn.microsoft.com/api/mcp>), que le da al agente la capacidad de buscar en la documentación de Microsoft. Mismo patrón de tres pasos, solo cambia cómo construís el transporte.

MCP va en los dos sentidos

Hasta acá tu agente *consume* tools MCP. Pero también podés **exponer un agente MAF entero como una tool MCP**, para que cualquier cliente compatible lo use. El truco encadena dos ideas: primero convertís el agente en una `AIFunction` con `.AsAIFunction()` (que de paso es cómo un agente llama a otro agente como si fuera una función), y después envolvés esa función como tool MCP y la servís por HTTP. Construís el agente una vez, y cualquier cliente MCP del mundo puede usarlo.

Criterio de producción (no es solo sintaxis)

Dos cosas que conviene que los alumnos internalicen, porque son las que distinguen un juguete de algo desplegable:

Acotó la superficie de tools. Un servidor MCP puede exponer muchísimas herramientas, y eso *no* significa que tu agente deba recibirlas todas. Más tools significa más descripciones enviadas al modelo (más tokens), más chances de que elija la equivocada, y más superficie que auditar. La regla es la misma que con tools

locales: exponé el conjunto más chico que resuelva la tarea, y filtrá antes de que las tools lleguen al modelo.

MCP facilita las integraciones, no las hace seguras solo. El protocolo no es la capa de seguridad. Tratá a un servidor MCP como cualquier otra dependencia de integración: no conectes servidores random a un agente de producción, y no asumas que por hablar MCP ya es confiable. En particular, las credenciales van *fuera* del prompt: nunca metas API keys ni tokens OAuth en las instrucciones del agente. Para servidores remotos, lo correcto es pasarlas como headers por cada run, donde quedan disponibles solo para esa ejecución y no se persisten.

Nota de versión: el paquete `ModelContextProtocol` está en preview y su superficie de API se mueve rápido mientras madura la especificación. Pineá una versión concreta y seguí el repo oficial del SDK de MCP para .NET; esperá cambios de API entre previews.

Cómo encaja todo (el cierre)

La idea de "bloques que componen" es lo que hace valioso a MAF. Repasando la pila completa:

- 1. **MEAI (IChatClient)** es el cimiento: la interfaz universal para hablarle a cualquier modelo.
- 2. **VectorData** habilita RAG: tus agentes buscan en la base de conocimiento de tu organización con búsqueda semántica.
- 3. **Agent Framework** orquesta todo: los agentes usan `IChatClient` por debajo, incorporan búsqueda vectorial a través de context providers, y se coordinan con workflows.

Y la progresión que recorriste, que es la que conviene que tengan internalizada:

Nivel	Pieza	Qué agregó
1	AsAIAgent + RunAsync	El agente y su bucle de ejecución.
2	AIFunctionFactory + [Description]	Acción: el agente usa herramientas.
3	AgentSession	Memoria de corto plazo (una conversación).
4	AIContextProvider	Memoria de largo plazo y RAG (entre conversaciones).
5	WorkflowBuilder + ejecutores + aristas	Orquestación de varios agentes.
6	FunctionApprovalRequestContent	Control humano sobre acciones sensibles.
7	McpClientFactory + ListToolsAsync	Herramientas externas vía MCP (estándar abierto).

El hilo conductor, una vez más: **nada de esto es magia**. Cada pieza reemplaza un andamiaje que ya sabés escribir a mano. MAF te lo da hecho, probado y componible, para que vos pongas la atención en la *lógica del agente* y no en la plomería.

Para seguir

- Repositorio y samples en C#: github.com/microsoft/agent-framework/tree/main/dotnet
- Documentación oficial: learn.microsoft.com/agent-framework/
- DevUI: un depurador en el navegador para ver la ejecución del agente, los mensajes, las llamadas a herramientas y las decisiones de orquestación en tiempo real. Muy útil para enseñar qué pasa adentro del bucle.

CAPITULO 53

Model Context Protocol (MCP) en C#

Crear un servidor con el SDK oficial, consumirlo, e instalarlo en agentes de programación

Tutorial de Programación III — Tecnicatura Universitaria en Programación — UTN FRT .NET 10 · C# 14 ·
ModelContextProtocol 1.4.0

1. El porqué antes del qué: ¿qué problema resuelve MCP?

Antes de escribir una sola línea, conviene entender qué dolor vino a calmar este protocolo.

Un LLM, por sí solo, es una función pura: texto entra, texto sale. No puede leer un archivo, consultar una base de datos ni llamar a una API. La técnica para darle "manos" es el *function calling* (vos ya lo viste con `Microsoft.Extensions.AI`): el modelo, en lugar de responder, devuelve un pedido estructurado del estilo "llamá a `get_clima(ciudad=\"Tucumán\")`", el programa lo ejecuta, y le devuelve el resultado para que el modelo continúe.

El problema aparece cuando esto escala. Cada herramienta hay que cablearla a mano dentro de cada aplicación. Si tenés cinco asistentes (uno en tu IDE, uno en un bot de WhatsApp, uno en la terminal) y diez herramientas (sistema de archivos, GitHub, Postgres, etc.), terminás con **N×M integraciones**: cada cliente reimplementa cada herramienta. Es el mismo problema que tenían los periféricos antes del USB: cada impresora con su cable propietario.

MCP es el USB-C de las herramientas para LLMs. Define un estándar para que:

- un **servidor** exponga capacidades (herramientas, recursos, prompts) una sola vez, y
- cualquier **cliente** compatible (Claude Desktop, Cursor, opencode, Codex, tu propia app con MEAI) las consuma sin saber nada del servidor más allá del protocolo.

Pasás de N×M integraciones a **N+M**: cada servidor se escribe una vez, cada cliente habla el protocolo una vez.

Breve historia

MCP fue presentado por **Anthropic en noviembre de 2024** como especificación abierta. La idea prendió rápido: durante 2025 los principales clientes de IA (Claude, los editores tipo Cursor/Zed, y luego OpenAI con Codex, Google, etc.) adoptaron el protocolo, y se publicaron SDKs oficiales en TypeScript, Python, Java, Go y **C#**. El SDK de C# se mantiene **en colaboración con Microsoft** y es el que vamos a usar acá. Hoy MCP es, de hecho, el lenguaje franco de la integración de herramientas en el ecosistema de agentes.

2. En el fondo, es RPC. Nada más (y nada menos)

Acá viene la desmitificación importante, porque la sigla y el marketing hacen parecer que MCP es algo esotérico.

MCP es, esencialmente, una llamada a procedimiento remoto (RPC). El cliente invoca una función que vive en otro proceso, le pasa argumentos, y recibe un resultado. Exactamente la misma idea que un gRPC, un viejo XML-RPC de los 90, o tu typical API REST: ejecutar código en otro lado y recibir la respuesta.

Concretamente, MCP usa **JSON-RPC 2.0** como formato de mensajes. Un JSON-RPC se ve así:

```
jsonc
// El cliente pide ejecutar una herramienta:
{
  "jsonrpc": "2.0",
  "id": 7,
  "method": "tools/call",
  "params": {
    "name": "factorial",
    "arguments": { "n": 5 }
  }
}

// El servidor responde:
{
  "jsonrpc": "2.0",
  "id": 7,
  "result": {
    "content": [ { "type": "text", "text": "120" } ]
  }
}
```

Si entendés esto, entendiste el 80% de MCP. Todo lo demás (el SDK, los atributos, los `IChatClient`) es azúcar para no escribir ese JSON a mano. **El SDK es el mismo tipo de abstracción que un cliente HTTP por encima de los sockets TCP.** Vos no construís el JSON: anotás un método con un atributo y el SDK serializa, transporta y deserializa por vos.

¿Qué le agrega MCP a un RPC pelado? Tres cosas que un RPC genérico no estandariza:

1. **Descubrimiento (tools/list):** el cliente puede *preguntarle* al servidor qué herramientas tiene, con sus nombres, descripciones y esquemas de parámetros. Esto es clave porque el *consumidor real es un LLM*, que necesita leer esas descripciones en lenguaje natural para decidir cuándo llamar a cada una.
2. **Un handshake de capacidades (initialize)*:* cliente y servidor negocian versión del protocolo y qué soportan.
3. **Tres tipos de primitivas semánticas**, no solo "funciones" (las vemos en la sección 4).

3. Los dos transportes: cómo viajan los mensajes

JSON-RPC define el *formato* de los mensajes, pero no por dónde viajan. MCP estandariza dos transportes:

stdio (standard input/output). El cliente lanza al servidor como un **proceso hijo** y se comunican por las tuberías estándar: el cliente escribe pedidos en el *stdin* del servidor y lee respuestas de su *stdout*. El *stderr* queda para logs. Es local, sin red, sin puertos, sin autenticación. Es el transporte que vas a usar para herramientas que corren en tu máquina (como nuestro ejemplo) y el que esperan opencode y Codex por

defecto.

*Consecuencia práctica que muerde a todos la primera vez: **en un servidor stdio, el stdout es sagrado**. Si hacés `Console.WriteLine("debug")` contaminás el canal JSON-RPC y rompés el protocolo. Todo log va por `stderr` (o, con el host de .NET, configurás el logger para que vaya ahí).*

Streamable HTTP. El servidor es un proceso de larga vida que escucha en una URL. El cliente le habla por HTTP. Es el transporte para servidores remotos, multiusuario, con autenticación (OAuth, bearer tokens). En C# esto lo da el paquete `ModelContextProtocol.AspNetCore`.

Para este tutorial usamos **stdio**, que es lo más simple y lo correcto para una herramienta local.

4. Las tres primitivas: filosofía de diseño

MCP no expone solo "funciones". Distingue tres tipos de capacidad, y la distinción no es caprichosa: responde a *quién controla la acción*.

Primitiva	Qué es	Quién la controla	Analogía
Tools (herramientas)	Funciones con efectos: ejecutan código, calculan, escriben	El modelo decide invocarlas	Un POST / un verbo
Resources (recursos)	Datos de solo lectura identificados por URI: un archivo, una fila, un documento	La aplicación decide cargarlos al contexto	Un GET / un sustantivo
Prompts	Plantillas de interacción reutilizables, parametrizables	El usuario las dispara (ej. un slash-command)	Una macro

Esta separación es la *filosofía* central: **distinguir acciones controladas por el modelo, datos controlados por la app, y flujos controlados por el usuario**. En la práctica, el 90% de los servidores que vas a escribir (y el nuestro) exponen solo **tools**, así que en eso nos concentramos.

Una *tool* bien diseñada tiene tres partes que el LLM lee para decidir:

- un **nombre** claro (`calcular_factorial`),
- una **descripción** en lenguaje natural (acá se gana o se pierde: el modelo elige la herramienta leyendo esto),
- un **esquema de parámetros** (tipos y descripciones de cada argumento), que el SDK genera automáticamente desde la firma del método en C#.

5. El SDK oficial de C#: anatomía

El SDK se publica en NuGet bajo el perfil `ModelContextProtocol` y se reparte en tres paquetes:

- `ModelContextProtocol.Core` — cliente y APIs de servidor de bajo nivel, mínimas dependencias.

- `ModelContextProtocol` — el anterior + integración con el *Generic Host* e inyección de dependencias. **Es el que usamos para un servidor stdio.**
- `ModelContextProtocol.AspNetCore` — para servidores HTTP.

La pieza conceptual más linda del SDK es esta: del lado del cliente, `McpClientTool` **hereda de** `AIFunction` (la abstracción de `Microsoft.Extensions.AI`). Eso significa que una herramienta MCP remota se *enchufa directamente* como herramienta de un `IChatClient`, igual que una función local. El modelo no distingue entre una función que escribiste al lado y una que vive en un servidor MCP en otro proceso. Esa uniformidad es exactamente la ganancia de tener un estándar.

6. Ejemplo integrador: un servidor MCP "matemático"

Vamos a construir un servidor stdio que expone **dos herramientas**:

- `factorial(n)` — calcula $n!$ con `BigInteger` (para que no explote con números grandes).
- `primeros_primos(cantidad)` — devuelve los primeros k números primos.

Usamos el formato **file-based app de .NET 10** con directivas `#: package`, como en tus TPs.

6.1. El servidor

csharp

```
#:package ModelContextProtocol@1.4.0
#:package Microsoft.Extensions.Hosting@10.0.0

using System.ComponentModel;
using System.Numerics;
using System.Text;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
using ModelContextProtocol.Server;

// 1. Host genérico. Lo usamos por la DI y el ciclo de vida,
// pero NO es un servidor web: es un proceso de consola stdio.
var builder = Host.CreateApplicationBuilder(args);

// 2. CRÍTICO en stdio: todo log debe ir a stderr.
// Si va a stdout, contamina el canal JSON-RPC y rompe el protocolo.
builder.Logging.AddConsole(options => {
    options.LogToStandardErrorThreshold = LogLevel.Trace;
});

// 3. Registramos el servidor MCP:
// - WithStdioServerTransport: hablamos por stdin/stdout.
// - WithToolsFromAssembly: escanea el ensamblado y registra como
// herramientas todos los métodos marcados con [McpServerTool]
// dentro de clases marcadas con [McpServerToolType].
builder.Services
    .AddMcpServer()
    .WithStdioServerTransport()
    .WithToolsFromAssembly();

await builder.Build().RunAsync();

// 4. Las herramientas. Una clase marcada como tipo-de-tools,
// con métodos marcados como tools. El SDK genera el esquema
// JSON de parámetros a partir de la firma y los [Description].
[McpServerToolType]
public static class HerramientasMatematicas {
    [McpServerTool(Name = "factorial")]
    [Description("Calcula el factorial de un entero no negativo n (n!). " +
        "Devuelve el resultado exacto usando precisión arbitraria.")]
    public static string Factorial(
        [Description("Entero no negativo del cual calcular el factorial.")] int n) {
        if (n < 0) {
            return "Error: el factorial no está definido para números negativos.";
        }

        BigInteger resultado = BigInteger.One;
        for (int i = 2; i <= n; i++) {
            resultado *= i;
        }

        return resultado.ToString();
    }

    [McpServerTool(Name = "primeros_primos")]
    [Description("Devuelve los primeros 'cantidad' números primos, " +
        "separados por coma, en orden ascendente.")]
    public static string PrimerosPrimos(
        [Description("Cantidad de primos a generar (entre 1 y 1000).")] int cantidad) {
        if (cantidad < 1) {
            return "Error: la cantidad debe ser un entero positivo.";
        }
        if (cantidad > 1000) {
            return "Error: el máximo soportado es 1000 primos.";
        }

        List<int> primos = new();
        int candidato = 2;
```

csharp - continuación

```
while (primos.Count < cantidad) {
    if (EsPrimo(candidato)) {
        primos.Add(candidato);
    }
    candidato++;
}

return string.Join(" ", primos);
}

private static bool EsPrimo(int numero) {
    if (numero < 2) {
        return false;
    }
    if (numero % 2 == 0) {
        return numero == 2;
    }
    for (int divisor = 3; (long)divisor * divisor <= numero; divisor += 2) {
        if (numero % divisor == 0) {
            return false;
        }
    }
    return true;
}
}
```

Tres cosas para fijar la atención:

1. **No escribiste nada de JSON-RPC.** Anotaste métodos. El SDK arma el `tools/list` con los nombres, las descripciones y el esquema de parámetros (deducido de que `n` es `int`, que el `[Description]` lo explica, etc.), y resuelve los `tools/call`. Es exactamente "RPC con azúcar".
2. **Las descripciones son la interfaz con el LLM.** No son comentarios decorativos: son lo que el modelo lee para decidir cuándo y cómo llamar a cada herramienta. Una descripción pobre = una herramienta que el modelo no usa o usa mal.
3. **Devolvemos string.** El SDK lo envuelve en un bloque de contenido de texto. Para resultados estructurados podés devolver objetos y se serializan a JSON.

6.2. Probarlo a mano (opcional pero recomendado)

Antes de enchufarlo a un agente, conviene verificar que arranca. Guardá el archivo como `mcp-mate.cs` y corré:

```
bash
dotnet run mcp-mate.cs
```

El proceso queda "colgado" esperando en `stdin`: eso es correcto, está esperando mensajes JSON-RPC. Para una prueba real, el **MCP Inspector** (oficial) te da una UI:

```
bash
npx @modelcontextprotocol/inspector dotnet run --project mcp-mate.cs
```

Te abre un navegador donde ves la lista de herramientas y podés invocarlas con argumentos a mano. Es el equivalente al Postman de MCP.

7. Consumirlo desde código con el cliente oficial

Antes de instalarlo en un agente, veamos cómo lo consume *tu propia app*. Acá se ve la magia de que `McpClientTool` sea un `AIFunction`: las herramientas del servidor se enchufan directo a un `IChatClient` de MEAI.

```
csharp
#:package ModelContextProtocol@1.4.0
#:package Microsoft.Extensions.AI@10.0.0
#:package Microsoft.Extensions.AI.OpenAI@10.0.0-preview.1.25xxxxx

using System.ClientModel;
using Microsoft.Extensions.AI;
using ModelContextProtocol.Client;
using OpenAI;

// 1. Definimos cómo se LANZA el servidor (transporte stdio):
//     el cliente va a ejecutar 'dotnet run mcp-mate.cs' como proceso hijo.
var transport = new StdioClientTransport(new StdioClientTransportOptions {
    Name = "MateMCP",
    Command = "dotnet",
    Arguments = ["run", "mcp-mate.cs"],
});

// 2. Nos conectamos y hacemos el handshake.
await using var mcpClient = await McpClient.CreateAsync(transport);

// 3. Descubrimiento: pedimos la lista de herramientas (tools/list).
IList<McpClientTool> herramientas = await mcpClient.ListToolsAsync();
foreach (var t in herramientas) {
    Console.Error.WriteLine($"Herramienta disponible: {t.Name} – {t.Description}");
}

// 4a. Invocación DIRECTA, sin LLM (es RPC puro):
var resultado = await mcpClient.CallToolAsync(
    "factorial",
    new Dictionary<string, object?> { ["n"] = 20 });
Console.WriteLine(resultado.Content[0] is TextContentBlock tb ? tb.Text : "");

// 4b. Invocación a través de un LLM:
//     como McpClientTool : AIFunction, las pasamos tal cual en Tools.
IChatClient chatClient = new OpenAIClient(
    new ApiKeyCredential(Environment.GetEnvironmentVariable("OPENAI_API_KEY")))
    .GetChatClient("gpt-4o-mini")
    .AsIChatClient()
    .AsBuilder()
    .UseFunctionInvocation() // hace el loop de tool-calling por nosotros
    .Build();

var respuesta = await chatClient.GetResponseAsync(
    "¿Cuánto es el factorial de 15 y cuáles son los primeros 8 primos?",
    new ChatOptions { Tools =[.. herramientas] });

Console.WriteLine(respuesta.Text);
```

Fijate en el paralelo con el ReAct loop "a mano" que vimos en clase: ahí escribíamos el bucle de detectar-tool / ejecutar / devolver-resultado nosotros. Acá `UseFunctionInvocation()` hace ese mismo loop, y `ListToolsAsync()` reemplaza el catálogo de funciones que armábamos manualmente. Mismo patrón, otra capa de abstracción.

El paso 4a es la prueba más contundente de que "esto es RPC": llamamos a la herramienta sin ningún LLM en el medio. El modelo es solo un consumidor posible.

8. Instalarlo en un agente de programación

Ahora lo importante para tu día a día: que el servidor quede disponible en un agente de coding, y que **se invoque automáticamente** cuando hagas un pedido en lenguaje natural.

El mecanismo es siempre el mismo: le decís al agente *cómo lanzar el proceso del servidor* (comando + argumentos). El agente lo arranca por stdio, hace `tools/list`, y a partir de ahí el modelo del agente decide cuándo llamar a `factorial` o `primeros_primos`.

8.1. En opencode

opencode se configura con **JSON**. Podés usar `config global` (`~/.config/opencode/opencode.json`) o por proyecto (`opencode.json` en la raíz del repo). Los servidores locales van bajo mcp con `type: "local"`, y `command` es un **arreglo** [comando, arg, arg, ...]:

```
jsonc
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "mate": {
      "type": "local",
      "command": ["dotnet", "run", "/ruta/absoluta/a/mcp-mate.cs"],
      "enabled": true
    }
  }
}
```

Usá **ruta absoluta** al `.cs` para evitar sorpresas según el directorio de trabajo. Al iniciar opencode, el servidor `mate` aparece listado entre las herramientas. Podés confirmar con el comando `mcp` de opencode.

*Nota de seguridad que vale enseñar: el command de un MCP local **se ejecuta sin confirmación**. Por eso un opencode.json malicioso en un repo ajeno es un vector de ejecución de código. Regla: no abras con opencode un repo en el que no confiás sin revisar antes su opencode.json.*

8.2. En Codex (CLI de OpenAI)

Codex se configura con **TOML**, en `~/.codex/config.toml` (global) o `.codex/config.toml` por proyecto (solo en proyectos *trusted*). Codex solo soporta **stdio** para servidores locales. Tenés dos caminos.

Camino A — el helper de CLI (recomendado para la primera vez):

```
bash
codex mcp add mate -- dotnet run /ruta/absoluta/a/mcp-mate.cs
```

Todo lo que va después de `--` es el comando que lanza el servidor. Codex escribe el TOML con la forma correcta para tu versión instalada.

Camino B — editar el TOML a mano. A diferencia de opencode, acá `command` y `args` van **separados**: `command` es el ejecutable (string) y `args` es el arreglo de argumentos:

toml

```
[mcp_servers.mate]
command = "dotnet"
args = ["run", "/ruta/absoluta/a/mcp-mate.cs"]
startup_timeout_sec = 30
tool_timeout_sec = 60
enabled = true
```

Si Codex no encuentra dotnet, usá la ruta absoluta del ejecutable (which dotnet en Linux/macOS, where dotnet en Windows) como valor de command. Es la causa #1 de servidores que "no arrancan": el PATH que hereda el agente no es el de tu shell interactiva.

Dentro de la sesión de Codex, el comando `/mcp` te muestra los servidores activos y sus herramientas. Misma idea en opencode.

Comparación útil: opencode usa JSON con command como arreglo único; Codex usa TOML con command/args separados. La herramienta MCP es la misma; lo único que cambia es la sintaxis del archivo de configuración del cliente. Eso es, otra vez, el punto entero del estándar: escribís el servidor una vez, lo enchufás en cualquier cliente cambiando tres líneas de config.

9. Cómo se invoca automáticamente: el flujo completo

Esta es la pregunta del final y vale recorrerla paso a paso. Supongamos que ya configuraste el servidor mate en tu agente y, dentro de él, escribís:

"Necesito saber cuántas formas hay de ordenar un mazo de 52 cartas, y dame los primeros 10 números primos para una tabla hash."

Lo que pasa por debajo, en orden:

1. **Arranque.** Al iniciar la sesión, el agente lanzó `dotnet run mcp-mate.cs` como proceso hijo (stdio), envió `initialize`, y luego `tools/list`. Recibió el catálogo: `factorial` y `primeros_primos` con sus descripciones y esquemas.
1. **Inyección al contexto.** El agente le pasa ese catálogo al LLM como herramientas disponibles, junto con tu mensaje. El modelo ve las dos herramientas y sus descripciones en lenguaje natural.
1. **Decisión del modelo.** El LLM razona: "ordenar 52 cartas = 52!, tengo `factorial`; y necesita 10 primos, tengo `primeros_primos`". En vez de responder texto, **emite dos tool-calls**: `factorial(n=52)` y `primeros_primos(cantidad=10)`. Notó que el modelo tradujo "ordenar un mazo" a "factorial" *gracias a la descripción* — por eso insistimos tanto en escribirlas bien.
1. **Ejecución (el RPC).** El agente serializa cada llamada como un `tools/call` JSON-RPC, lo manda por `stdin` al proceso del servidor. Tu código C# corre `Factorial(52)` y `PrimerosPrimos(10)`, y devuelve los resultados por `stdout`.

1. **Vuelta al modelo.** El agente le entrega los resultados (806581751709... y 2, 3, 5, 7, 11, 13, 17, 19, 23, 29) al LLM como mensajes de rol *tool*.
1. **Respuesta final.** Con los datos ya calculados en el contexto, el modelo redacta la respuesta en prosa para vos, citando los números exactos que devolvió tu servidor.

El usuario nunca escribió "llamá a factorial". Solo describió un problema. **La invocación automática es la conjunción de tres cosas: el descubrimiento (`tools/list`), las descripciones bien escritas, y la capacidad de tool-calling del modelo.** El protocolo MCP estandariza las dos primeras; el modelo aporta la tercera.

10. Cierre: el mapa mental

- MCP resuelve el problema **N×M** de integrar herramientas con LLMs, volviéndolo **N+M**. Es el "USB-C de las herramientas".
- En el fondo es **JSON-RPC 2.0**: una llamada a procedimiento remoto. El SDK es azúcar sobre ese protocolo, igual que un cliente HTTP es azúcar sobre sockets.
- Le agrega al RPC genérico tres cosas: **descubrimiento** de capacidades, **handshake** de versión, y tres **primitivas** semánticas (`tools / resources / prompts`).
- Dos **transportes**: stdio (proceso local) y Streamable HTTP (remoto). Para herramientas locales, stdio.
- En C#: anotás métodos con `[McpServerTool]`, registrás con `AddMcpServer().WithStdioServerTransport().WithToolsFromAssembly()`. El SDK genera el esquema y resuelve las llamadas.
- Del lado cliente, `McpClientTool : AIFunction`, así que las herramientas remotas se enchufan a un `IChatClient` igual que funciones locales.
- En agentes: le das al cliente el **comando que lanza el servidor**. `opencode` usa JSON (`command` como arreglo); `Codex` usa TOML (`command + args`). Mismo servidor, distinta config.
- La **invocación automática** sale de: descubrimiento + buenas descripciones + tool-calling del modelo.

CAPITULO 54

REST API vs. MCP: Entendiendo la Revolución de las APIs para Inteligencia Artificial

Tanto **REST** como **MCP** son estándares para comunicar sistemas de software a través de una red o un límite de proceso. Sin embargo, nacieron en épocas distintas y fueron diseñados para **consumidores completamente diferentes**: REST fue hecho para programas escritos por humanos, mientras que MCP fue diseñado para Modelos de Lenguaje (LLMs) y Agentes de IA.

1. El Primer Principio (La Filosofía Detrás de Cada Uno)

El Principio de REST (Representational State Transfer)

"Determinismo y Contratos Estrictos"

- **Objetivo:** Permitir que una aplicación cliente ejecute operaciones específicas en un servidor remoto de la forma más predecible y estructurada posible.
- **El Enfoque:** El programador humano del cliente lee la documentación de la API (ej. Swagger/OpenAPI), escribe código explícito indicando cuándo y cómo llamar a cada endpoint (ej. `GET /clientes/5`), y define cómo procesar exactamente la respuesta JSON. Todo el flujo es determinista: si pasa A, el programa llama a B.

El Principio de MCP (Model Context Protocol)

"Descubrimiento Dinámico y Autonomía de la IA"

- **Objetivo:** Darle a un modelo de IA (LLM) acceso a datos, herramientas y prompts del mundo real de forma dinámica para que el modelo decida por sí mismo qué hacer.
- **El Enfoque:** El cliente MCP (el "cerebro" de la IA) no sabe de antemano qué funciones existen. Al conectarse, el servidor le provee una lista de capacidades descritas en lenguaje natural (ej. *"Esta herramienta lee archivos de texto"*). La IA evalúa la intención del usuario y **decide de forma autónoma** cuál herramienta llamar, en qué orden y qué argumentos pasarle.

2. Similitudes entre REST y MCP

- **Intercambio de Datos:** Ambos utilizan **JSON** como el formato estándar para enviar y recibir información debido a su ligereza y legibilidad.
- **Desacoplamiento:** Ambos separan la lógica del backend (donde están las bases de datos o el sistema de archivos) del cliente (la interfaz web o el agente inteligente).
- **Independencia Tecnológica:** Puedes implementar tanto servidores REST como MCP en cualquier lenguaje (C#, Python, Node.js, Go) e interactuar con clientes escritos en tecnologías distintas.
- **Transporte Web:** Ambos pueden funcionar sobre HTTP utilizando la infraestructura web tradicional.

3. Diferencias Clave

Característica	API REST	MCP (Model Context Protocol)
Consumidor Primario	Un programa informático (flujo rígido codificado por un humano).	Un Modelo de Lenguaje / Agente de IA (flujo dinámico por razonamiento).
Protocolo de Mensajería	HTTP crudo (métodos GET, POST, PUT, DELETE, códigos de estado 200, 404, 500).	JSON-RPC 2.0 (mensajes estructurados con campos fijos: <code>jsonrpc</code> , <code>id</code> , <code>method</code> , <code>params</code>).
Tipo de Conexión	Sin estado (Stateless): Cada petición es independiente. Se abre y se cierra la conexión HTTP.	Con estado / Persistente: Conexión continua bidireccional (vía <code>stdio</code> o HTTP con <code>streaming/SSE</code>).
Documentación	Estática (OpenAPI/Swagger) destinada a que la lea el programador .	Semántica (descripciones en lenguaje natural dentro del esquema JSON) para que la lea la IA .
Dirección del Flujo	Unidireccional: el cliente siempre inicia la petición al servidor.	Bidireccional: El cliente puede pedir herramientas al servidor, y el servidor puede pedirle al cliente que procese texto (<code>sampling</code>).

4. Un Ejemplo Comparativo Práctico

Imagina que quieres consultar el clima de una ciudad y mostrar una recomendación.

El Enfoque REST

1. El programador lee la documentación de OpenWeather y ve que el endpoint es `/weather?q={city}`.
2. El programador escribe en C#:

```
csharp
// El flujo está "hardcodeado" (cableado) por el desarrollador
if (usuarioPresionoBoton) {
    var clima = await httpClient.GetFromJsonAsync<Clima>("/weather?q=Rosario");
    if (clima.Temperatura > 30) {
        MostrarMensaje("Lleva agua!");
    }
}
```

```
csharp - continuacion
```

```
}
```

El Enfoque MCP

1. El programador crea un servidor MCP que expone una herramienta llamada `obtener_clima` y la describe: *"Devuelve la temperatura actual de una ciudad"*.
2. El programador conecta la IA a este servidor.
3. El usuario le escribe a la IA: *"Hola, voy a viajar a Rosario hoy, ¿qué me sugieres?"*.
4. La IA (no el código del programa) razona:
 - *La pregunta del usuario requiere saber el clima de Rosario.*
 - *Reviso mis herramientas MCP... ¡Ah! Tengo `obtener_clima`.*
 - *Llamo a `obtener_clima` pasándole "Rosario" en el parámetro.*
1. El servidor MCP responde con la temperatura (32°C).
2. La IA recibe el dato y razona: *"Hace calor en Rosario. Le sugeriré que use ropa ligera y lleve agua"*.

5. Conclusión: ¿Cuál elegir?

- Usa una **API REST** cuando quieras automatizar procesos rígidos, de alta velocidad y donde el flujo de control deba ser 100% predecible (ej. pasarelas de pago, registros de usuarios, endpoints móviles).
- Usa **MCP** cuando estés construyendo integraciones de Inteligencia Artificial (copilotos de código, agentes autónomos de soporte, análisis de datos interactivos) y necesites que el modelo interactúe dinámicamente con el entorno del usuario.

CAPITULO 55

Tutorial de CLI en C#: sortx en un solo archivo

¿Qué vamos a construir?

Una herramienta llamada `sortx` que lee un archivo de texto delimitado (CSV, TSV, PSV, etc.), ordena sus filas según uno o más criterios, y escribe el resultado.

```
texto
sortx [input [output]] [-b|--by campo[:tipo[:orden]]]...
      [-i|--input input] [-o|--output output]
      [-d|--delimiter delimitador]
      [-nh|--no-header] [-h|--help]
```

Ejemplos:

```
bash
# Ordenar por apellido (alfabético, ascendente)
sortx empleados.csv -b apellido

# Ordenar por salario descendente y guardar en archivo
sortx empleados.csv resultado.csv -b salario:num:desc

# Múltiples criterios: por departamento, dentro por salario desc
sortx empleados.csv -b departamento -b salario:num:desc

# TSV sin encabezado, ordenar por la segunda columna
sortx datos.tsv -d "\t" -nh -b 1:alpha:asc

# Encadenando con redirección
cat empleados.csv | sortx -b apellido > ordenado.csv

# Ayuda
sortx --help
```

Resultado esperado (con `-b departamento -b salario:num:desc`):

```
texto
nombre,apellido,edad,salario,departamento
Ana,Álvarez,28,72000,Diseño
Beatriz,Blanco,25,65000,Diseño
Carlos,Castro,38,95000,Gerencia
Diana,Díaz,42,120000,Gerencia
Eduardo,Espinosa,31,88000,Ingeniería
Fernanda,Fernández,35,85000,Ingeniería
```

La hoja de ruta del programa

Cada paso recibe datos, los transforma y los pasa al siguiente:

texto

1. ParseArgs → leer la configuración desde los argumentos
2. ReadInput → leer el texto desde el archivo o stdin
3. ParseDelimited → convertir el texto en filas (lista de diccionarios)
4. SortRows → ordenar las filas según los criterios
5. Serialize → convertir las filas ordenadas de vuelta a texto
6. WriteOutput → escribir en el archivo de salida o stdout

Cada paso es independiente: recibe su entrada, produce su salida, no sabe nada de los demás. El punto de entrada se limita a invocarlos en orden, dentro de un `try/catch`.

Paso 0: el archivo

A diferencia de un proyecto tradicional con `.csproj`, en C# 14 / .NET 10 se pueden ejecutar archivos `.cs` sueltos como **file-based programs**. Toda la herramienta vive en un único `sortx.cs`.

```
bash
# Crear el archivo
touch sortx.cs
code sortx.cs
```

Para ejecutar mientras se desarrolla:

```
bash
dotnet run sortx.cs -- empleados.csv -b apellido
```

El `--` separa los argumentos para `dotnet run` de los argumentos para `sortx`. Sin él, `dotnet` los interpretaría como propios.

Paso 1: el modelo de datos — dos record

Antes de parsear los argumentos, definimos *qué* vamos a parsear. En C# tenemos los **records**: tipos inmutables, compactos, perfectos para representar datos que no cambian una vez contruidos.

```
csharp
record SortField(string Name, bool Numeric, bool Descending);

record AppConfig(
    string? InputFile,
    string? OutputFile,
    string Delimiter,
    bool NoHeader,
    List<SortField> SortFields
);
```

`SortField` representa un criterio de ordenación: el nombre del campo (o el índice como `string` si no hay encabezado), si la comparación es numérica, y si es descendente. `AppConfig` agrupa toda la configuración de una corrida: archivos, delimitador, flag de encabezado, y la lista de criterios.

Dos records, no uno: AppConfig contiene una `List<SortField>` porque un solo comando puede pedir ordenar por varios campos a la vez (`-b departamento -b salario:num:desc`). Modelar cada criterio como su propio tipo hace que el código que los procesa quede mucho más legible.

record vs class: Un record es inmutable por defecto. Sus propiedades se asignan al construirlo y no cambian después. Para representar configuración, es la herramienta exacta.

Paso 2: parsear los argumentos

La sintaxis que debemos soportar:

Opción larga	Corta	Descripción
--help	-h	Muestra la ayuda y sale
--by	-b	Criterio de ordenación, repetible
--input	-i	Archivo de entrada
--output	-o	Archivo de salida
--delimiter	-d	Carácter delimitador (default: ,)
--no-header	-nh	El archivo no tiene fila de encabezado

La estrategia: recorreremos args con un índice `i` y consumimos uno o dos tokens según el caso.

```
csharp
AppConfig ParseArgs(string[] args) {
    string? inputFile = null;
    string? outputFile = null;
    string delimiter = ",";
    bool noHeader = false;
    var sortFields = new List<SortField>();
    var positional = new List<string>();

    int i = 0;
    while (i < args.Length) {
        string arg = args[i];
        switch (arg) {
            case "--help" or "-h":
                PrintHelp();
                Environment.Exit(0);
                break;

            case "--input" or "-i":
                inputFile = Next(args, ref i, arg);
                break;

            case "--output" or "-o":
                outputFile = Next(args, ref i, arg);
                break;

            case "--by" or "-b":
                sortFields.Add(ParseSortField(Next(args, ref i, arg)));
                break;

            case "--delimiter" or "-d":
                delimiter = NormalizeDelimiter(Next(args, ref i, arg));
        }
    }
}
```

csharp - continuación

```

        break;

        case "--no-header" or "-nh":
            noHeader = true;
            i++;
            break;

        case var option when option.StartsWith('-'):
            throw new Exception($"Opción desconocida: '{option}'. Use --help.");

        case var value:
            positional.Add(value);
            i++;
            break;
    }
}

if (positional.Count >= 1 && inputFile == null) inputFile = positional[0];
if (positional.Count >= 2 && outputFile == null) outputFile = positional[1];
if (positional.Count > 2)
    throw new Exception("Demasiados argumentos. Máximo: [input] [output]");
if (sortFields.Count == 0)
    throw new Exception("Debe indicar al menos un criterio con -b|--by.");

return new AppConfig(inputFile, outputFile, delimiter, noHeader, sortFields);
}

```

Next es una función auxiliar que avanza el índice y retorna el siguiente token:

csharp

```

string Next(string[] args, ref int i, string option) {
    i++;
    if (i >= args.Length)
        throw new Exception($"'{option}' requiere un valor.");
    return args[i++];
}

```

`ref int i`: Pasamos el índice por referencia para que Next pueda avanzarlo. Cuando retorna, `i` ya apunta al token siguiente al valor consumido. Así evitamos manejar el índice en dos lugares.

`-nh` **no consume valor**: A diferencia de las otras opciones, `-nh` es un flag booleano: no toma argumento. Por eso solo hace `i++` para pasar al siguiente token, sin llamar a Next.

Paso 2a: parsear cada - -by

Cada valor de `- -by` tiene la forma `campo[:tipo[:orden]]`:

Expresión	Significado
apellido	por apellido, alfabético, ascendente
salario:num	por salario, numérico, ascendente
salario:num:desc	por salario, numérico, descendente
2:num:asc	por columna 2, numérico, ascendente (sin enc)

Lo separamos por `:` y vamos asignando defaults para las partes ausentes:

csharp

```
SortField ParseSortField(string spec) {
    string[] parts = spec.Split(':');

    string name      = parts[0];
    bool  numeric    = false;
    bool  descending  = false;

    if (parts.Length >= 2) {
        switch (parts[1]) {
            case "num" : numeric = true; break;
            case "alpha": numeric = false; break;
            default:
                throw new Exception($"Tipo inválido en '{spec}': '{parts[1]}'. Use 'num' o 'alpha'.");
        }
    }

    if (parts.Length >= 3) {
        switch (parts[2]) {
            case "asc" : descending = false; break;
            case "desc": descending = true; break;
            default:
                throw new Exception($"Orden inválido en '{spec}': '{parts[2]}'. Use 'asc' o 'desc'.");
        }
    }

    if (parts.Length > 3)
        throw new Exception($"Especificación inválida: '{spec}'.");

    if (string.IsNullOrEmpty(name))
        throw new Exception($"Falta el nombre del campo en '{spec}'.");

    return new SortField(name, numeric, descending);
}
```

Validar al parsear, no al usar: Si el usuario escribe `salario: numerico: descendente`, queremos que el error se vea acá, junto al argumento que lo causó. Si lo dejáramos pasar y fallara más tarde durante la ordenación, el mensaje sería confuso.

Paso 2b: normalizar el delimitador

El usuario puede pasar `-d "\t"` para tabulación. Pero según la shell, `"\t"` puede llegar como dos caracteres literales (`\` y `t`) en lugar del carácter tab real. Lo convertimos uniformemente:

csharp

```
string NormalizeDelimiter(string raw) {
    return raw switch {
        "\\t" => "\t",
        "\\n" => "\n",
        "\\r" => "\r",
        ""    => throw new Exception("El delimitador no puede estar vacío."),
        _     => raw
    };
}
```

switch **como expresión:** En C# moderno, switch puede devolver un valor directamente sin `break` ni asignación intermedia. Es ideal para mapeos cortos como este.

Paso 3: leer y escribir

Dos funciones simétricas. Una lee desde archivo o stdin; la otra escribe hacia archivo o stdout. No cambian respecto a cualquier otra herramienta CLI.

```
csharp
string ReadInput(string? filePath) {
    if (filePath == null)
        return Console.In.ReadToEnd();

    if (!File.Exists(filePath))
        throw new FileNotFoundException($"Archivo no encontrado: '{filePath}'");

    return File.ReadAllText(filePath);
}

void WriteOutput(string? filePath, string content) {
    if (filePath == null) {
        Console.Write(content);
        return;
    }

    File.WriteAllText(filePath, content);
}
```

Redirección de consola: `Console.In.ReadToEnd()` y `Console.Write()` funcionan tanto en terminal como con redirección (`|`, `<`, `>`). El sistema operativo se encarga; el código no cambia.

Paso 4: parsear el archivo delimitado

Convertimos el texto en una lista de diccionarios. Cada diccionario representa una fila: campo → valor. El delimitador y la presencia o no de encabezado se reciben como parámetros.

```
csharp
(string[] headers, List<Dictionary<string, string>> rows) ParseDelimited(
    string content,
    string delimiter,
    bool noHeader)
{
    string[] split(string line) {
        string[] parts = line.Split(delimiter);
        for (int j = 0; j < parts.Length; j++)
            parts[j] = parts[j].Trim();
        return parts;
    }

    string[] lines = content.Split('\n');

    // Eliminar líneas vacías al final
    int last = lines.Length - 1;
    while (last >= 0 && lines[last].Trim() == "") last--;

    if (last < 0)
        return ([], []);

    // Determinar headers según haya o no encabezado
    string[] headers;
    int firstDataLine;

    if (noHeader) {
        // Sin encabezado: usamos índices "0", "1", "2"... como nombres
        string[] firstRow = split(lines[0]);
    }
```

csharp - continuacion

```

        headers = new string[firstRow.Length];
        for (int j = 0; j < firstRow.Length; j++)
            headers[j] = j.ToString();
        firstDataLine = 0;
    } else {
        headers = split(lines[0]);
        firstDataLine = 1;
    }

    // Parsear cada fila como diccionario campo → valor
    var rows = new List<Dictionary<string, string>>();
    for (int i = firstDataLine; i <= last; i++) {
        string line = lines[i];
        if (line == "") continue;

        string[] values = split(line);
        var row = new Dictionary<string, string>();
        for (int col = 0; col < headers.Length; col++)
            row[headers[col]] = col < values.Length ? values[col] : "";

        rows.Add(row);
    }

    return (headers, rows);
}

```

Sin encabezado → headers numéricos: El truco está en que, cuando no hay encabezado, generamos un array ["0", "1", "2", ...] que cumple el mismo rol. Así, todo el resto del programa puede seguir usando row["apellido"] o row["2"] sin distinguir el caso. La diferencia se concentra en este único punto.

Tuple return (T1, T2): ParseDelimited produce dos cosas: los encabezados y las filas. El tipo de retorno tupla expresa eso directamente, sin necesidad de crear una clase contenedora. Lo desestructuramos con var (headers, rows) = ParseDelimited(...).

Función local split: Es una función adentro de otra. Se define una sola vez y se reutiliza en este ParseDelimited sin contaminar el resto del archivo. Aplica también el delimitador correcto sin tener que pasarlo cada vez.

Paso 5: ordenar las filas

Acá vive la lógica del programa. Recibe las filas y la lista de criterios; devuelve las filas reordenadas. El truco es que cuando hay varios criterios, el segundo solo desempata cuando el primero da igual, y así sucesivamente.

csharp

```

List<Dictionary<string, string>> SortRows(
    List<Dictionary<string, string>> rows,
    string[] headers,
    List<SortField> sortFields)
{
    // 1. Validar que todos los campos pedidos existan
    foreach (var field in sortFields) {
        bool found = false;
        for (int i = 0; i < headers.Length; i++)
            if (headers[i] == field.Name) { found = true; break; }
    }
}

```

csharp - continuación

```

    if (!found)
        throw new Exception($"Campo no encontrado: '{field.Name}");
}

if (rows.Count == 0 || sortFields.Count == 0)
    return rows;

// 2. Construir el ordenamiento encadenando criterios
IOrderedEnumerable<Dictionary<string, string>>? ordered = null;

foreach (var field in sortFields) {
    IComparable Key(Dictionary<string, string> row) {
        string value = row[field.Name];
        if (field.Numeric)
            return double.TryParse(value, out double n) ? n : double.NaN;
        return value;
    }

    if (ordered == null)
        ordered = field.Descending
            ? rows.OrderByDescending(Key)
            : rows.OrderBy(Key);
    else
        ordered = field.Descending
            ? ordered.ThenByDescending(Key)
            : ordered.ThenBy(Key);
}

return ordered!.ToList();
}

```

Validar antes de ordenar: Si un campo del --by no existe entre los encabezados, queremos enterarnos antes de empezar a procesar. Validamos los campos primero, después ordenamos. Es el mismo principio que en sumx: primero verificar, después trabajar.

OrderBy vs ThenBy: El primer criterio usa OrderBy/OrderByDescending y arranca el ordenamiento desde cero. Los criterios siguientes usan ThenBy/ThenByDescending, que solo se aplican como desempate. Por eso el primer criterio se trata distinto: si lo tratáramos igual que los siguientes, perdería su rol de criterio principal.

IComparable como tipo de la clave: OrderBy puede ordenar por cualquier tipo que sepa compararse consigo mismo. string y double ambos implementan IComparable. Devolver IComparable desde la función local nos permite alternar entre comparación numérica y alfabética sin duplicar el código de ordenación.

Función local que captura field: Key está definida adentro del foreach y usa field. C# captura cada field correctamente para cada iteración (desde C# 5), así que cada ordenamiento usa su propio criterio sin interferir con los demás.

double.NaN para celdas no numéricas: Si una celda esperada como número no se puede parsear, devolvemos NaN. .NET lo ordena consistentemente al final y evita que el programa explote por una celda mal formada.

Paso 6: serializar de vuelta a texto

Lo opuesto exacto a ParseDelimited: tomamos las filas (ya ordenadas) y las convertimos a texto delimitado, respetando si hay encabezado y cuál es el delimitador.

csharp

```
string Serialize(
    string[] headers,
    List<Dictionary<string, string>> rows,
    string delimiter,
    bool noHeader)
{
    var sb = new StringBuilder();

    if (!noHeader)
        sb.AppendLine(string.Join(delimiter, headers));

    foreach (var row in rows) {
        string[] values = new string[headers.Length];
        for (int i = 0; i < headers.Length; i++)
            values[i] = row.TryGetValue(headers[i], out string? v) ? v : "";

        sb.AppendLine(string.Join(delimiter, values));
    }

    return sb.ToString();
}
```

StringBuilder **para construir texto**: Concatenar muchas veces con + crea un string nuevo cada vez (los strings son inmutables). Para archivos con miles de filas, eso degrada el rendimiento. StringBuilder mantiene un buffer interno y lo va completando, mucho más eficiente.

El orden de las columnas se respeta gracias a headers: Aunque las filas son diccionarios (sin orden definido), headers es un array y conserva el orden original de las columnas. Por eso recorremos headers para reconstruir cada fila.

Si no había encabezado, no lo hay en la salida: Cuando noHeader es true, los headers que generamos ("0", "1", ...) eran un detalle interno, no parte de los datos. La salida arranca directo con la primera fila.

Paso 7: la ayuda

```
csharp
void PrintHelp() {
    Console.WriteLine("""
sortx – ordena las filas de un archivo de texto delimitado

USO
    sortx [input [output]] [-b|--by campo[:tipo[:orden]]]...
        [-i|--input input] [-o|--output output]
        [-d|--delimiter delimitador]
        [-nh|--no-header] [-h|--help]

OPCIONES
    -b, --by          spec      Criterio de ordenación. Repetible.
                                spec = campo[:tipo[:orden]]
                                tipo  = alpha (default) | num
                                orden = asc  (default) | desc
    -i, --input       archivo   Archivo de entrada (o primer posicional)
    -o, --output       archivo   Archivo de salida (o segundo posicional)
    -d, --delimiter   caracter  Delimitador. Default ','. Use '\t' para tabulación.
    -nh, --no-header   El archivo no tiene fila de encabezado.
                                Los campos se referencian por índice (0, 1, ...).
    -h, --help         Muestra esta ayuda

EJEMPLOS
    sortx empleados.csv -b apellido
    sortx empleados.csv resultado.csv -b salario:num:desc
    sortx empleados.csv -b departamento -b salario:num:desc -o resultado.csv
    sortx datos.tsv -d "\t" -nh -b 1:alpha:asc
    cat empleados.csv | sortx -b apellido > ordenado.csv
""");
}
```

Raw string literal `""" ... """`: Permite escribir bloques multilínea sin escapar comillas ni saltos de línea. Ideal para ayuda, plantillas, JSON inline, SQL inline.

Paso 8: el punto de entrada

Conectamos las piezas. En C# *file-based programs*, las instrucciones al nivel raíz del archivo son el punto de entrada — no hace falta clase ni método `Main`.

```
csharp
try {
    AppConfig config = ParseArgs(args); // Paso 1
    string text = ReadInput(config.InputFile); // Paso 2
    var (headers, rows) = ParseDelimited(text, config.Delimiter, config.NoHeader); // Paso 3
    var sorted = SortRows(rows, headers, config.SortFields); // Paso 4
    string output = Serialize(headers, sorted, config.Delimiter, config.NoHeader); // Paso 5
    WriteOutput(config.OutputFile, output); // Paso 6
} catch (Exception ex) {
    Console.Error.WriteLine($"Error: {ex.Message}");
    Console.Error.WriteLine("Use --help para ver las opciones.");
    Environment.Exit(1);
}
```

Seis líneas, seis pasos. El punto de entrada no toma decisiones: solo conecta funciones.

`Console.Error` **para errores:** Si el usuario redirige la salida (`sortx ... > ordenado.csv`), los mensajes de error van a la terminal y no contaminan el archivo. Siempre usar `Console.Error` para mensajes que no son el output del programa.

`Environment.Exit(1)`: Convención universal de Unix: código 0 significa éxito, cualquier otro valor significa error. Permite encadenar el programa en pipelines y scripts (`sortx ... && echo OK`).

El archivo completo

```
csharp
// =====
// ■ sortx – ordena las filas de un archivo de texto delimitado ■
// ■ Uso: sortx [input [output]] [-b|--by campo[:tipo[:orden]]]... ■
// ■ [-i|--input input] [-o|--output output] ■
// ■ [-d|--delimiter delim] [-nh|--no-header] [-h|--help] ■
// =====

using System.Text;

record SortField(string Name, bool Numeric, bool Descending);

record AppConfig(
    string? InputFile,
    string? OutputFile,
    string Delimiter,
    bool NoHeader,
    List<SortField> SortFields
);

// ■ Punto de entrada ■=====

try {
    AppConfig config = ParseArgs(args);
    string text = ReadInput(config.InputFile);
    var (headers, rows) = ParseDelimited(text, config.Delimiter, config.NoHeader);
    var sorted = SortRows(rows, headers, config.SortFields);
    string output = Serialize(headers, sorted, config.Delimiter, config.NoHeader);
    WriteOutput(config.OutputFile, output);
} catch (Exception ex) {
    Console.Error.WriteLine($"Error: {ex.Message}");
    Console.Error.WriteLine("Use --help para ver las opciones.");
    Environment.Exit(1);
}

// ■ Paso 1: Parsear argumentos ■=====

AppConfig ParseArgs(string[] args) {
    string? inputFile = null;
    string? outputFile = null;
    string delimiter = ",";
    bool noHeader = false;
    var sortFields = new List<SortField>();
    var positional = new List<string>();

    int i = 0;
    while (i < args.Length) {
        string arg = args[i];
        switch (arg) {
            case "--help" or "-h":
                PrintHelp();
                Environment.Exit(0);
                break;

            case "--input" or "-i":
                inputFile = Next(args, ref i, arg);
                break;

            case "--output" or "-o":
                outputFile = Next(args, ref i, arg);
                break;

            case "--by" or "-b":
                sortFields.Add(ParseSortField(Next(args, ref i, arg)));
                break;

            case "--delimiter" or "-d":
                delimiter = NormalizeDelimiter(Next(args, ref i, arg));
                break;
        }
    }
}
```


csharp - continuacion

```

        case "--no-header" or "-nh":
            noHeader = true;
            i++;
            break;

        case var option when option.StartsWith('-'):
            throw new Exception($"Opción desconocida: '{option}'. Use --help.");

        case var value:
            positional.Add(value);
            i++;
            break;
    }
}

if (positional.Count >= 1 && inputFile == null) inputFile = positional[0];
if (positional.Count >= 2 && outputFile == null) outputFile = positional[1];
if (positional.Count > 2)
    throw new Exception("Demasiados argumentos. Máximo: [input] [output]");

if (sortFields.Count == 0)
    throw new Exception("Debe indicar al menos un criterio con -b|--by.");

return new AppConfig(inputFile, outputFile, delimiter, noHeader, sortFields);
}

string Next(string[] args, ref int i, string option) {
    i++;
    if (i >= args.Length)
        throw new Exception($"'{option}' requiere un valor.");
    return args[i++];
}

SortField ParseSortField(string spec) {
    string[] parts = spec.Split(':');

    string name      = parts[0];
    bool  numeric    = false;
    bool  descending = false;

    if (parts.Length >= 2) {
        switch (parts[1]) {
            case "num" : numeric = true; break;
            case "alpha": numeric = false; break;
            default:
                throw new Exception($"Tipo inválido en '{spec}': '{parts[1]}'. Use 'num' o 'alpha'.");
        }
    }

    if (parts.Length >= 3) {
        switch (parts[2]) {
            case "asc" : descending = false; break;
            case "desc": descending = true; break;
            default:
                throw new Exception($"Orden inválido en '{spec}': '{parts[2]}'. Use 'asc' o 'desc'.");
        }
    }

    if (parts.Length > 3)
        throw new Exception($"Especificación inválida: '{spec}'.");

    if (string.IsNullOrEmpty(name))
        throw new Exception($"Falta el nombre del campo en '{spec}'.");

    return new SortField(name, numeric, descending);
}

string NormalizeDelimiter(string raw) {
    return raw switch {
        "\\t" => "\t",
        "\\n" => "\n",
        "\\r" => "\r",
    };
}

```


csharp - continuacion

sortx – ordena las filas de un archivo de texto delimitado

USO

```
sortx [input [output]] [-b|--by campo[:tipo[:orden]]]...
      [-i|--input input] [-o|--output output]
      [-d|--delimiter delimitador]
      [-nh|--no-header] [-h|--help]
```

OPCIONES

-b, --by	spec	Criterio de ordenación. Repetible. spec = campo[:tipo[:orden]] tipo = alpha (default) num orden = asc (default) desc
-i, --input	archivo	Archivo de entrada (o primer posicional)
-o, --output	archivo	Archivo de salida (o segundo posicional)
-d, --delimiter	caracter	Delimitador. Default ','. Use '\t' para tabulación.
-nh, --no-header		El archivo no tiene fila de encabezado.
		Los campos se referencian por índice (0, 1, ...).
-h, --help		Muestra esta ayuda

EJEMPLOS

```
sortx empleados.csv -b apellido
sortx empleados.csv resultado.csv -b salario:num:desc
sortx empleados.csv -b departamento -b salario:num:desc -o resultado.csv
sortx datos.tsv -d "\t" -nh -b 1:alpha:asc
cat empleados.csv | sortx -b apellido > ordenado.csv
""");
}
```

Probando

Creá empleados.csv:

csv

```
nombre,apellido,edad,salario,departamento
Fernanda,Fernández,35,85000,Ingeniería
Ana,Álvarez,28,72000,Diseño
Diana,Díaz,42,120000,Gerencia
Eduardo,Espinosa,31,88000,Ingeniería
Beatriz,Blanco,25,65000,Diseño
Carlos,Castro,38,95000,Gerencia
```

bash

```
# Ordenar por apellido (alfabético, ascendente)
dotnet run sortx.cs -- empleados.csv -b apellido

# Ordenar por salario descendente
dotnet run sortx.cs -- empleados.csv -b salario:num:desc

# Ordenar por departamento, dentro por salario descendente
dotnet run sortx.cs -- empleados.csv -b departamento -b salario:num:desc

# Guardar en archivo
dotnet run sortx.cs -- empleados.csv resultado.csv -b apellido

# Leyendo desde stdin
cat empleados.csv | dotnet run sortx.cs -- -b apellido > ordenado.csv

# Sin encabezado, ordenar por columna 3 numérica descendente
tail -n +2 empleados.csv | dotnet run sortx.cs -- -nh -b 3:num:desc

# Campo inexistente → error claro
dotnet run sortx.cs -- empleados.csv -b columnaInexistente
# Error: Campo no encontrado: 'columnaInexistente'
```

bash - continuación

Sin criterios → error claro

dotnet run sortx.cs -- empleados.csv

Error: Debe indicar al menos un criterio con -b|--by.

Especificación inválida → error claro

dotnet run sortx.cs -- empleados.csv -b salario: numerico

Error: Tipo inválido en 'salario: numerico': 'numerico'. Use 'num' o 'alpha'.

Ver ayuda

dotnet run sortx.cs -- --help

¿Por qué -- antes de los argumentos? Le dice a dotnet run que todo lo que sigue son argumentos para tu programa, no para dotnet. Sin esa marca, opciones como -h o --help las consumiría dotnet antes de llegar a sortx.

Lo que aprendimos

Concepto	Dónde lo usamos
<i>File-based program</i>	Todo el programa en <code>sortx.cs</code> , sin <code>.csproj</code> ni clase ni <code>Main</code>
record con varios campos	<code>AppConfig</code> : 5 campos inmutables contruidos de una vez
record anidado en colección	<code>List<SortField></code> dentro de <code>AppConfig</code>
Funciones locales	Cada paso del pipeline es una función independiente
switch con patrones or	"-by" or "-b" en un solo case
switch como expresión	<code>NormalizeDelimiter</code> mapea valores con una expresión switch
<code>ref int i</code> en <code>Next</code>	El índice se avanza dentro de la función auxiliar
Tuple return (<code>T1</code> , <code>T2</code>)	<code>ParseDelimited</code> retorna headers y rows juntos
Función local anidada	<code>split</code> dentro de <code>ParseDelimited</code> reutiliza el delimitador
Headers numéricos sintéticos	Cuando no hay encabezado, generamos "0", "1", ... como nombres
<code>OrderBy + ThenBy</code>	El primer criterio inicia el orden; los siguientes desempatan
<code>IComparable</code> como key	Permite alternar entre orden numérico y alfabético sin duplicar código
Captura en función local	<code>key</code> captura <code>field</code> correctamente en cada iteración del <code>foreach</code>
<code>double.NaN</code> como caída suave	Una celda no numérica no rompe el ordenamiento
<code>StringBuilder</code>	<code>Serialize</code> evita concatenar strings inmutables uno por uno
Validar antes de operar	<code>SortRows</code> valida los campos antes de ordenar
Validar al parsear	<code>ParseSortField</code> valida tipo y orden donde el error tiene sentido
Raw string literal <code>"""..."""</code>	<code>PrintHelp</code> con bloque multilínea sin escapes
<code>Console.Error</code> para errores	Los errores no contaminan la salida redirigida
<code>Environment.Exit(código)</code>	0 para éxito, distinto de 0 para fallo: convención Unix

Resumen

texto

¿Necesito construir una CLI con varios criterios combinables?
 ■■■ pipeline de funciones + lista de records

¿Necesito modelar configuración inmutable con sub-elementos?
 ■■■ `record` que contiene `List<otroRecord>`

¿Necesito parsear opciones con sub-sintaxis (campo:tipo:orden)?
 ■■■ función dedicada `ParseSortField` con `Split(':')` y validación temprana

¿Necesito un flag booleano (sin valor)?
 ■■■ en el `switch` solo `i++`, sin llamar a `Next`

¿Necesito que el delimitador `\t` funcione siempre?
 ■■■ normalizar el string antes de usarlo

¿Necesito leer desde archivo o desde stdin?
 ■■■ `File.ReadAllText(...)` o `Console.In.ReadToEnd()`

¿Necesito procesar un archivo delimitado con o sin encabezado?
 ■■■ generar headers numéricos sintéticos cuando no hay encabezado

¿Necesito ordenar por varios criterios encadenados?
 ■■■ `OrderBy(...)` para el primero, `ThenBy(...)` para los siguientes

¿Necesito alternar comparación numérica y alfabética?
 ■■■ función local que devuelve `IComparable`, eligiendo `double` o `string`

¿Necesito construir texto eficientemente?
 ■■■ `StringBuilder` y `string.Join(delim, ...)`

¿Necesito mostrar ayuda clara y ejemplos?
 ■■■ `PrintHelp()` con raw string literal `"""..."""`

¿Necesito que el programa devuelva código de error a la shell?
 ■■■ `Environment.Exit(1)` desde el `catch`

Ejemplos clave:

`record SortField`	→ un criterio inmutable de ordenación
`record AppConfig`	→ toda la configuración de una corrida
`switch + or`	→ opciones cortas y largas en un solo case
`Split(':')`	→ sub-sintaxis de campo:tipo:orden
`Dictionary<,>`	→ filas con acceso por nombre o por índice
`OrderBy / ThenBy`	→ ordenamiento primario y desempate
`IComparable`	→ tipo unificado de clave de ordenación
`IOrderedEnumerable`	→ resultado intermedio de `OrderBy`
`StringBuilder`	→ construcción eficiente del texto de salida
`Console.Error`	→ mensajes de error separados del output

CAPITULO 56

Parser descendente recursivo y AST

En este tutorial vamos a construir un evaluador pequeño para expresiones aritméticas en C#.

La idea no es solamente calcular un número, sino separar con claridad dos problemas distintos:

1. **analizar** el texto de entrada;
2. **construir** una estructura que represente la expresión;
3. **evaluar** esa estructura tantas veces como queramos.

Ese enfoque aparece una y otra vez en compiladores, intérpretes, motores de reglas y hasta planillas.

La expresión de ejemplo será:

```
csharp
1 + 2 * (8 / 4)
```

y el resultado esperado será:

```
texto
5
```

Paso 1 — Pensar la gramática

Antes de escribir código conviene describir qué expresiones queremos aceptar.

En este caso vamos a soportar:

- números;
- operadores binarios +, -, *, /;
- operadores unarios + y -;
- paréntesis.

Una gramática simple para eso puede escribirse así:

```
texto
expresion := termino ('+' | '-' termino)*
termino   := factor ('*' | '/' factor)*
factor    := ('+' | '-') factor | '(' expresion ')' | numero
```

La parte importante es que la precedencia ya está codificada en la gramática:

- * y / se resuelven dentro de termino;

- + y - binarios se resuelven dentro de expresión;
- los operadores unarios viven dentro de factor.

Eso permite que una expresión como esta:

```
texto
1 + 2 * 3
```

se interprete como:

```
texto
1 + (2 * 3)
```

y no como:

```
texto
(1 + 2) * 3
```

Paso 2 — Parsear con un parser descendente recursivo

Un parser descendente recursivo traduce cada regla de la gramática a una función.

La estructura general queda así:

```
csharp
Nodo Parse(string s) {
    var pos = 0;

    char Current() => pos < s.Length ? s[pos] : '\0';
    char Consume() => s[pos++];

    bool Match(char c) {
        while (char.IsWhiteSpace(Current())) {
            Consume();
        }
        return Current() == c;
    }

    Nodo Expresion() {
        var izquierda = Termino();

        while (Match('+') || Match('-')) {
            var operador = Consume();
            var derecha = Termino();

            if (operador == '+') {
                izquierda = new Suma(izquierda, derecha);
            } else {
                izquierda = new Resta(izquierda, derecha);
            }
        }

        return izquierda;
    }

    return izquierda;
}

Nodo Termino() {
    var izquierda = Factor();
```

csharp - continuacion

```

while (Match('*') || Match('/')) {
    var operador = Consume();
    var derecha = Factor();

    if (operador == '*') {
        izquierda = new Multiplicacion(izquierda, derecha);
    } else {
        izquierda = new Division(izquierda, derecha);
    }
}

return izquierda;
}

Nodo Factor() {
    if (Match('+') || Match('-')) {
        var signo = Consume();
        var nodo = Factor();
        return signo == '+' ? new Positivo(nodo) : new Negativo(nodo);
    }

    if (Match('(')) {
        Consume();
        var nodo = Expresion();
        if (!Match('')) {
            throw new Exception("Se esperaba '')");
        }
        Consume();
        return nodo;
    }

    if (char.IsDigit(Current())) {
        var valor = 0f;
        while (char.IsDigit(Current())) {
            valor = valor * 10 + (Consume() - '0');
        }
        return new Numero(valor);
    }

    throw new Exception($"Carácter inesperado: '{Current()}'");
}

return Expresion();
}

```

Este parser no evalúa directamente. En lugar de eso, **construye un árbol**.

Ese detalle cambia por completo el diseño.

Paso 3 — Construir un AST

AST significa *Abstract Syntax Tree*, o árbol de sintaxis abstracta.

Por ejemplo, la expresión:

texto

```
1 + 2 * 3
```

puede representarse así:

texto



Ese árbol ya no es texto: es una estructura de objetos.

En nuestro caso la jerarquía base puede empezar así:

csharp

```
abstract class Nodo {
    public abstract float Evaluar();
}
```

Después aparecen dos familias muy naturales:

- nodos **unarios**;
- nodos **binarios**.

Podemos factorizar el comportamiento común en dos clases abstractas.

csharp

```
abstract class Unario(string operador, Nodo operando) : Nodo {
    protected string Operador { get; } = operador;
    protected Nodo Operando { get; } = operando;

    public override string ToString() => $"({Operador} {Operando})";
}

abstract class Binario(string operador, Nodo izquierda, Nodo derecha) : Nodo {
    protected string Operador { get; } = operador;
    protected Nodo Izquierda { get; } = izquierda;
    protected Nodo Derecha { get; } = derecha;

    public override string ToString() => $"({Izquierda} {Operador} {Derecha})";
}
```

Y luego definimos los nodos concretos:

csharp

```
class Numero(float valor) : Nodo {
    public override float Evaluar() => valor;
    public override string ToString() => valor.ToString();
}

class Positivo(Nodo operando) : Unario("+", operando) {
    public override float Evaluar() => Operando.Evaluar();
}

class Negativo(Nodo operando) : Unario("-", operando) {
    public override float Evaluar() => -Operando.Evaluar();
}

class Suma(Nodo izquierda, Nodo derecha) : Binario("+", izquierda, derecha) {
    public override float Evaluar() => Izquierda.Evaluar() + Derecha.Evaluar();
}

class Resta(Nodo izquierda, Nodo derecha) : Binario("-", izquierda, derecha) {
    public override float Evaluar() => Izquierda.Evaluar() - Derecha.Evaluar();
}
```

csharp - continuacion

```
class Multiplicacion(Nodo izquierda, Nodo derecha) : Binario("**", izquierda, derecha) {  
    public override float Evaluar() => Izquierda.Evaluar() * Derecha.Evaluar();  
}  
  
class Division(Nodo izquierda, Nodo derecha) : Binario("/", izquierda, derecha) {  
    public override float Evaluar() => Izquierda.Evaluar() / Derecha.Evaluar();  
}
```

Paso 4 — Evaluar el árbol

Una vez construido el AST, evaluarlo se vuelve simple: cada nodo sabe calcular su propio valor a partir de sus hijos.

Por ejemplo:

csharp

```
var expresion = Parse("1 + 2 * (8 / 4)");  
Console.WriteLine(expresion.Evaluar());
```

El resultado es:

texto

5

Y si además imprimimos el árbol:

csharp

```
Console.WriteLine(expresion);
```

obtenemos una forma parentizada que hace explícita la precedencia:

texto

```
(1 + (2 * (8 / 4)))
```

Eso es útil porque confirma que el parser entendió la expresión como queríamos.

Paso 5 — Construir el AST manualmente

Una ventaja importante del árbol es que no depende del texto original.

Podemos construir exactamente la misma expresión a mano:

```
csharp
var expresion = new Suma(
    new Numero(1),
    new Multiplicacion(
        new Numero(2),
        new Division(
            new Numero(8),
            new Numero(4)
        )
    )
);

Console.WriteLine($"{expresion} = {expresion.Evaluar()}");
```

La salida vuelve a ser:

```
texto
(1 + (2 * (8 / 4))) = 5
```

Eso demuestra que el AST es una representación independiente de cómo se escribió la expresión originalmente.

Paso 6 — Agregar azúcar sintáctico

Si el objetivo es trabajar con árboles, escribir siempre `new Suma(...)` o `new Multiplicacion(...)` puede resultar verboso.

Podemos hacer el código más natural agregando operadores a `Nodo`:

```
csharp
abstract class Nodo {
    public abstract float Evaluar();

    public static implicit operator Nodo(float valor) => new Numero(valor);
    public static Nodo operator +(Nodo a, Nodo b) => new Suma(a, b);
    public static Nodo operator -(Nodo a, Nodo b) => new Resta(a, b);
    public static Nodo operator *(Nodo a, Nodo b) => new Multiplicacion(a, b);
    public static Nodo operator /(Nodo a, Nodo b) => new Division(a, b);
    public static Nodo operator +(Nodo a) => new Positivo(a);
    public static Nodo operator -(Nodo a) => new Negativo(a);
}
```

Con eso ya podemos escribir:

```
csharp
Nodo expresion = 1 + 2 * (8 / 4);
Console.WriteLine($"{expresion} = {expresion.Evaluar()}");
```

La expresión se construye como árbol, no se calcula inmediatamente.

Ese detalle es importante: el resultado no es 5, sino un `Nodo` que representa la expresión completa.

Paso 7 — Qué ganamos con este diseño

Separar **parser** y **AST** trae varias ventajas:

- el texto se analiza una sola vez;
- la estructura resultante puede reutilizarse;
- `Evaluar()` y `ToString()` recorren el mismo árbol;
- más adelante podemos agregar otras operaciones sin cambiar el parser.

Por ejemplo, sobre ese mismo AST podríamos implementar:

- un optimizador;
- un contador de nodos;
- un visitante;
- un generador de código;
- una impresión con menos paréntesis.

Resumen final

En este tutorial seguimos esta secuencia:

1. definimos una gramática para expresiones aritméticas;
2. escribimos un parser descendente recursivo;
3. hicimos que el parser construya un AST;
4. evaluamos ese árbol;
5. vimos que también puede construirse manualmente;
6. agregamos operadores para escribirlo de forma más natural.

La idea central es esta:

texto → **parser** → **AST** → **operaciones sobre el árbol**

Ese modelo es pequeño, pero ya contiene varias ideas fundamentales de diseño de lenguajes.

Ejercicios sugeridos

1. Agregar soporte para espacios en blanco al parser y verificar que expresiones como `1 + 2 * 3` funcionen bien.
2. Extender `Factor()` para aceptar números decimales.
3. Hacer que `Division.Evaluar()` lance una excepción clara cuando el divisor sea cero.

4. Agregar una operación que cuente cuántos nodos tiene el árbol.
5. Reimplementar este mismo AST usando el patrón Visitante.

CAPITULO 57

Carrito de compras con EF Core y SQLite

Decisiones de diseño para construir sistemas mantenibles

Caso base: [clases/30.1-carrito.cs](#) C# / .NET / Entity Framework Core / SQLite

1. Qué problema resuelve este programa

El programa implementa un carrito de compras mínimo:

- Carga un catálogo de productos.
- Crea un carrito para un cliente.
- Agrega productos al carrito y reserva stock.
- Quita productos del carrito y repone stock.
- Muestra el total.
- Confirma o cancela el carrito.
- Persiste los cambios del carrito y del stock dentro de transacciones.
- Guarda todo en una base SQLite usando Entity Framework Core.

Aunque es un ejemplo chico, contiene decisiones que aparecen en sistemas reales:

- Cómo modelar entidades y relaciones.
- Dónde poner reglas de negocio.
- Cómo validar entradas.
- Cómo persistir datos sin mezclar SQL manual con la lógica principal.
- Cómo evitar duplicación y errores silenciosos.
- Cómo dejar una base clara para crecer después.

La idea central del diseño es esta:

El programa de consola muestra el flujo; el servicio aplica las reglas; las entidades representan el dominio; el DbContext sabe cómo guardar ese modelo en la base.

2. Arquitectura general

El archivo está organizado en cuatro capas simples:

```
texto
Programa principal
    Muestra el flujo de uso del carrito.

Funciones de presentación
    Imprimen catálogo, carrito y confirmación.

TiendaService
    Contiene las operaciones y reglas del sistema.

Entidades + DbContext
    Representan el modelo y su mapeo a SQLite.
```

Esta separación importa porque evita que todo quede mezclado. Si mañana el programa deja de ser una consola y pasa a ser una API, una app web o una app de escritorio, gran parte del código de dominio podría seguir sirviendo.

3. El programa principal como guion de uso

La primera parte del archivo funciona como un guion:

```
csharp
var dbPath = "30.1-carrito.sqlite";

using var context = new TiendaContext(dbPath);

context.Database.EnsureDeleted();
context.Database.EnsureCreated();

var tienda = new TiendaService(context);

CargarCatalogo(tienda);
MostrarCatalogo(tienda.ListarCatalogo());

var carrito = tienda.CrearCarrito("Juan Pérez");

tienda.AgregarProducto(carrito.Id, "NOTE-15", 1);
tienda.AgregarProducto(carrito.Id, "MOUSE-WL", 2);
tienda.AgregarProducto(carrito.Id, "USB-C", 3);
tienda.QuitarProducto(carrito.Id, "USB-C", 1);

MostrarCatalogo(tienda.ListarCatalogo());

var carritoActual = tienda.ObtenerCarrito(carrito.Id);
MostrarCarrito(carritoActual);

var carritoConfirmado = tienda.ConfirmarCarrito(carrito.Id);
MostrarConfirmacion(carritoConfirmado);

var carritoCancelado = tienda.CrearCarrito("Analía Gómez");
tienda.AgregarProducto(carritoCancelado.Id, "WEBCAM", 1);
tienda.AgregarProducto(carritoCancelado.Id, "TECL-WL", 1);

var carritoCanceladoFinal = tienda.CancelarCarrito(carritoCancelado.Id);
```

Esta parte no debería conocer detalles internos de la base. No hace INSERT, no calcula claves foráneas, no decide si un carrito está abierto o confirmado. Solo expresa el caso de uso.

Esa es una buena práctica general:

El código que coordina una pantalla, una consola o un endpoint debería leer como una historia de usuario, no como una mezcla de reglas, SQL y validaciones.

4. SQLite con una ruta local simple

La versión actual usa una ruta literal:

```
csharp
var dbPath = "30.1-carrito.sqlite";
```

La decisión es guardar la base SQLite en un archivo local, fácil de borrar y recrear en cada ejecución.

Ventajas para una clase o demo:

- Es fácil encontrar la base generada.
- Mantiene el ejemplo corto y fácil de leer.
- Evita incorporar configuración adicional antes de tiempo.
- Permite borrar y recrear el archivo sin afectar otras partes del proyecto.

Costo de esta decisión:

- La ubicación final depende del directorio de trabajo desde donde se ejecute el programa.

En un sistema real, esta decisión cambiaría. La ruta debería venir de configuración:

- appsettings.json
- variable de entorno
- argumentos de línea de comandos
- secretos del entorno de despliegue

Principio reutilizable:

Para un ejemplo chico, una ruta simple alcanza. Para producción, la ubicación de la base debería salir de configuración y no quedar fija en el código.

5. EnsureDeleted y EnsureCreated: bueno para aprender, no para producción

El programa hace esto:

```
csharp
context.Database.EnsureDeleted();
context.Database.EnsureCreated();
```

Eso borra la base y la crea de nuevo cada vez.

Es correcto para este caso porque queremos una demo repetible: cada ejecución arranca limpia, carga los mismos productos y muestra el mismo flujo.

Pero no es correcto para un sistema real. En producción, borrar la base significaría perder datos. Para proyectos reales se usan migraciones:

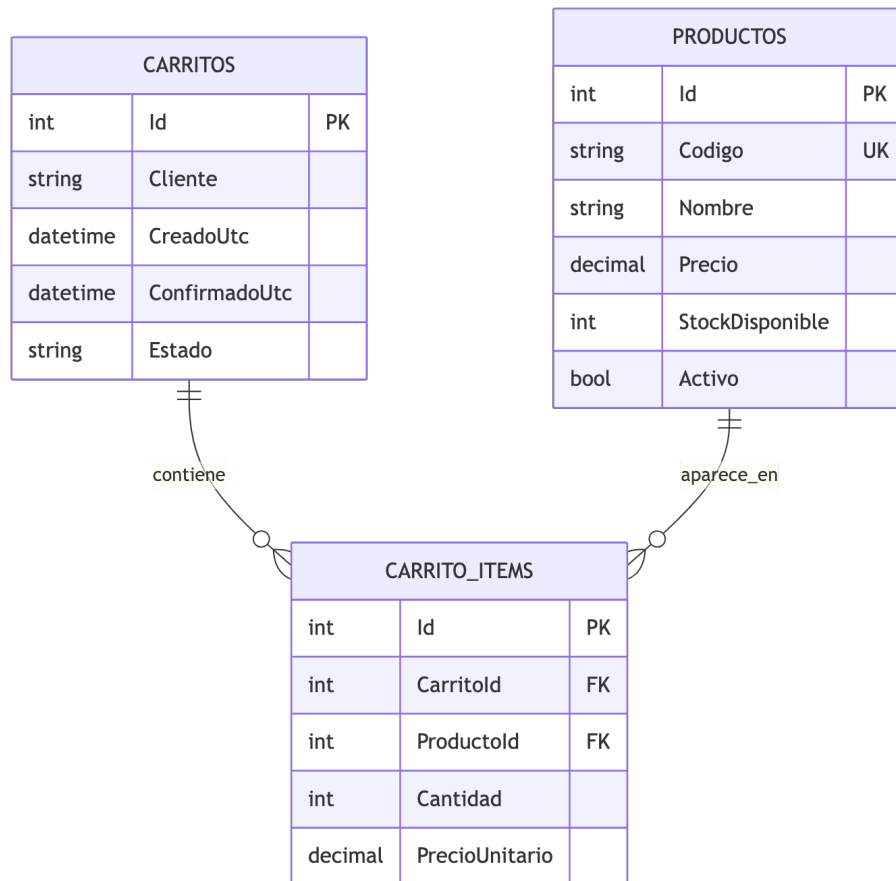
```
bash
dotnet ef migrations add Inicial
dotnet ef database update
```

Principio reutilizable:

EnsureCreated sirve para prototipos, pruebas simples y clases. Las migraciones sirven para sistemas que evolucionan sin perder datos.

6. Modelado del dominio

El modelo tiene cuatro piezas principales:



Diagrama

El diagrama muestra tres tablas persistidas:

- Productos: catálogo de productos disponibles.
- Carritos: encabezado del carrito, cliente, fecha y estado.
- CarritoItems: detalle del carrito; une un carrito con un producto y agrega cantidad y precio unitario.

Subtotal y Total no aparecen en el gráfico porque no son columnas de la base. Son propiedades calculadas en C# marcadas con [NotMapped].

```

csharp
enum EstadoCarrito {
    Abierto = 1,
    Confirmado = 2,
    Cancelado = 3
}
  
```

```

csharp
class Producto {
    public int Id { get; set; }
    public string Codigo { get; set; } = string.Empty;
    public string Nombre { get; set; } = string.Empty;
    public decimal Precio { get; set; }
    public int StockDisponible { get; set; }
    public bool Activo { get; set; } = true;

    public ICollection<CarritoItem> ItemsCarrito { get; set; } = [];
}
  
```

csharp

```
class Carrito {
    public int Id { get; set; }
    public string Cliente { get; set; } = string.Empty;
    public DateTime CreadoUtc { get; set; } = DateTime.UtcNow;
    public DateTime? ConfirmadoUtc { get; set; }
    public EstadoCarrito Estado { get; set; } = EstadoCarrito.Abierto;

    public ICollection<CarritoItem> Items { get; set; } = [];

    [NotMapped]
    public decimal Total => Items.Sum(item => item.Subtotal);
}
```

csharp

```
class CarritoItem {
    public int Id { get; set; }
    public int CarritoId { get; set; }
    public int ProductoId { get; set; }
    public int Cantidad { get; set; }
    public decimal PrecioUnitario { get; set; }

    public Carrito Carrito { get; set; } = null!;
    public Producto Producto { get; set; } = null!;

    [NotMapped]
    public decimal Subtotal => PrecioUnitario * Cantidad;
}
```

6.1 Por qué existe CarritoItem

Una primera idea podría ser que Carrito tenga una lista directa de Producto:

csharp

```
class Carrito {
    public List<Producto> Productos { get; set; }
}
```

Pero eso no alcanza. En un carrito no solo importa qué producto está incluido, también importa:

- cantidad
- precio unitario al momento de agregarlo
- subtotal
- relación entre un carrito específico y un producto específico

Por eso se modela una entidad intermedia: CarritoItem.

Principio reutilizable:

Cuando una relación necesita datos propios, deja de ser una lista directa y pasa a ser una entidad del dominio.

Ejemplos similares:

- FacturaDetalle: factura + producto + cantidad + precio.
- Inscripcion: alumno + curso + fecha + estado.

- `ReservaHabitacion`: `reserva + habitacion + noches + tarifa`.

6.2 Por qué `PrecioUnitario` está en el `item`

El producto ya tiene `Precio`, pero el `item` también guarda `PrecioUnitario`.

Esto parece duplicación, pero es una decisión correcta. El precio del producto puede cambiar mañana. Si el carrito o la compra ya fue confirmada, no queremos que el total histórico cambie porque alguien actualizó el catálogo.

Entonces:

- `Producto.Precio` representa el precio actual de catálogo.
- `CarritoItem.PrecioUnitario` representa el precio usado en ese carrito.

Principio reutilizable:

En operaciones históricas, guardar snapshots de datos importantes suele ser mejor que depender siempre del valor actual.

Esto aplica a precios, nombres legales, direcciones de envío, impuestos, descuentos y cotizaciones.

6.3 Por qué se usa `decimal` para dinero

Los precios son `decimal`, no `double`.

```
csharp
public decimal Precio { get; set; }
public decimal PrecioUnitario { get; set; }
```

`double` sirve para cálculos científicos aproximados. Para dinero necesitamos precisión decimal predecible.

Principio reutilizable:

Para dinero, usar `decimal`. Para mediciones físicas o cálculos aproximados, usar `double`.

6.4 Por qué hay propiedades calculadas

`Subtotal` y `Total` no se guardan como columnas:

```
csharp
[NotMapped]
public decimal Subtotal => PrecioUnitario * Cantidad;
```

```
csharp
[NotMapped]
public decimal Total => Items.Sum(item => item.Subtotal);
```

Se calculan a partir de datos existentes. Guardarlos en la base generaría riesgo de inconsistencia:

- cambia la cantidad pero no se actualiza el subtotal
- cambia un item pero no se recalcula el total
- se guarda un total distinto a la suma real

[NotMapped] le indica a EF Core que esa propiedad pertenece al modelo de objetos, pero no al esquema de la base.

Principio reutilizable:

Si un dato se puede calcular siempre desde otros datos confiables, no lo guardes salvo que tengas una razón concreta de rendimiento, auditoría o cierre contable.

7. Estados explícitos en vez de booleanos ambiguos

El carrito tiene un estado:

```
csharp
public EstadoCarrito Estado { get; set; } = EstadoCarrito.Abierto;
```

Podríamos haber usado:

```
csharp
public bool Confirmado { get; set; }
```

Pero un enum deja mejor preparada la evolución del sistema. Hoy hay tres estados:

- Abierto
- Confirmado
- Cancelado

Mañana podría haber más:

- Cancelado
- Expirado
- Pagado
- Enviado

Principio reutilizable:

Cuando una entidad atraviesa etapas de negocio, un enum suele expresar mejor el modelo que varios booleanos.

Evitar modelos como:

csharp

```
bool Confirmado;  
bool Cancelado;  
bool Pagado;
```

porque pueden aparecer combinaciones inválidas:

texto

```
Confirmado = true  
Cancelado = true  
Pagado = false
```

Un único estado reduce esa ambigüedad.

8. Reglas de negocio dentro del servicio

TiendaService concentra las operaciones importantes:

- crear producto
- listar catálogo
- crear carrito
- obtener carrito
- agregar producto
- quitar producto
- confirmar carrito
- cancelar carrito

Ejemplo:

csharp

```
public void AgregarProducto(int carritoId, string codigoProducto, int cantidad) {  
    var carrito = ObtenerCarrito(carritoId);  
    Verificar.Verdadero(carrito.Estado == EstadoCarrito.Abierto, "Solo se pueden agregar ítems a  
    carritos abiertos.");  
  
    var producto = ObtenerProductoPorCodigo(codigoProducto);  
    Verificar.Positivo(cantidad, "La cantidad debe ser mayor a cero.");  
    Verificar.Verdadero(producto.StockDisponible >= cantidad, $"El producto {producto.Codigo} no tiene  
    stock disponible.");  
  
    EjecutarEnTransaccion(() => {  
        var existente = carrito.Items.SingleOrDefault(item => item.ProductoId == producto.Id);  
        if (existente is null) {  
            carrito.Items.Add(new CarritoItem {  
                ProductoId = producto.Id,  
                Cantidad = cantidad,  
                PrecioUnitario = producto.Precio  
            });  
        } else {  
            existente.Cantidad += cantidad;  
        }  
  
        producto.StockDisponible -= cantidad;  
    });  
}
```



```
csharp - continuacion
```

```
    });  
}
```

Esta decisión evita que el programa principal tenga que saber:

- cómo se busca un producto
- cómo se valida la cantidad
- cómo se sabe si un carrito admite cambios
- cómo se agrega o actualiza un ítem
- cuándo hay que reservar o liberar stock
- cuándo hay que abrir una transacción y persistir cambios

Principio reutilizable:

Las pantallas, endpoints o comandos deberían pedir operaciones al servicio. El servicio debería proteger las reglas del sistema.

9. Validación y normalización centralizadas

La versión actual separa dos ideas:

- `NormalizarCodigo` unifica la identidad del producto.
- `Verificar` concentra precondiciones y mensajes de error.

```
csharp
```

```
static string NormalizarCodigo(string codigo) {  
    return codigo.Trim().ToUpperInvariant();  
}
```

```
csharp
```

```
public static void NoVacio(string valor, string mensaje = "") {  
    if (string.IsNullOrEmpty(valor)) {  
        throw new ArgumentException("El valor no puede ser vacío.", mensaje);  
    }  
}  
  
public static void Verdadero(bool condicion, string mensaje = "") {  
    if (!condicion) {  
        throw new ArgumentException(mensaje);  
    }  
}
```

Esto evita repetir en todos lados:

```
csharp
```

```
codigo.Trim().ToUpperInvariant()
```

También evita errores sutiles. Por ejemplo:

csharp

```
CrearProducto("note-15", "Notebook", 1000m);  
AgregarProducto(carrito.Id, "NOTE-15", 1);
```

Si no normalizamos de forma consistente, esos dos códigos podrían tratarse como distintos. Con `NormalizarCodigo`, ambos terminan siendo NOTE-15.

Principio reutilizable:

Cada dato con identidad de negocio necesita una regla única de normalización.

Ejemplos:

- códigos de producto
- emails
- CUIT/CUIL
- patentes
- nombres de usuario
- SKU

10. Errores claros en vez de excepciones accidentales

Cuando una búsqueda puede fallar, el programa evita depender de la excepción genérica de `Single`.

En vez de:

csharp

```
var producto = context.Productos.Single(producto => producto.Codigo == codigo);
```

usa:

csharp

```
var producto = context.Productos.SingleOrDefault(producto => producto.Codigo == codigoNormalizado);  
return producto ?? throw new InvalidOperationException($"No existe el producto {codigoNormalizado}.");
```

La diferencia es importante. El primer caso falla con un mensaje técnico como "Sequence contains no elements". El segundo caso dice qué regla de negocio no se pudo cumplir.

Principio reutilizable:

Las excepciones que llegan desde el dominio deben explicar el problema del dominio.

No es lo mismo:

texto

Sequence contains no elements.

que:

texto

No existe el producto NOTE-15.

11. Carga de datos relacionados con Include

Para mostrar el carrito necesitamos sus items y los productos de esos items.

csharp

```
public Carrito ObtenerCarrito(int carritoId) {
    var carrito = context.Carritos
        .Include(carrito => carrito.Items)
        .ThenInclude(item => item.Producto)
        .SingleOrDefault(carrito => carrito.Id == carritoId);

    return carrito ?? throw new InvalidOperationException($"No existe el carrito #{carritoId}.");
}
```

Include le dice a EF Core que cargue datos relacionados en la misma operación lógica.

Sin eso, dependiendo de la configuración, carrito.Items podría venir vacío o no estar cargado. En este ejemplo se elige ser explícito.

Principio reutilizable:

Cuando un caso de uso necesita un agregado completo, cargalo explícitamente con sus relaciones necesarias.

Para este carrito, el agregado útil es:

texto

```
Carrito
  Items
    Producto
```

12. Consultas de solo lectura con AsNoTracking

El catálogo se lista así:

csharp

```
public IReadOnlyList<Producto> ListarCatalogo() {
    return context.Productos
        .AsNoTracking()
        .OrderBy(producto => producto.Nombre)
        .ToList();
}
```

AsNoTracking le dice a EF Core:

No necesito modificar estas entidades después de leerlas.

Esto reduce trabajo interno del DbContext. En un sistema chico no se nota, pero es una buena costumbre en consultas de lectura.

Principio reutilizable:

Si una consulta solo muestra datos y no va a modificarlos, usar AsNoTracking.

13. Configuración del modelo con Fluent API

El DbContext define el mapeo:

```
csharp
class TiendaContext(string dbPath) : DbContext {
    public DbSet<Producto> Productos => Set<Producto>();
    public DbSet<Carrito> Carritos => Set<Carrito>();
    public DbSet<CarritoItem> CarritoItems => Set<CarritoItem>();

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder) {
        optionsBuilder.UseSqlite($"Data Source={dbPath}");
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder) {
        // configuracion de tablas, claves, indices, relaciones
    }
}
```

La Fluent API permite configurar la base sin llenar las entidades de atributos.

13.1 Índice único para código de producto

```
csharp
entity.HasIndex(producto => producto.Codigo).IsUnique();
```

Esto protege la regla:

No pueden existir dos productos con el mismo código.

No alcanza con validarlo en C#. La base también debe protegerlo.

Principio reutilizable:

Las reglas de unicidad importantes deben vivir también en la base de datos.

13.2 Longitudes máximas

csharp

```
entity.Property(producto => producto.Codigo).IsRequired().HasMaxLength(20);  
entity.Property(producto => producto.Nombre).IsRequired().HasMaxLength(120);
```

Esto documenta y limita el modelo. Si el código de producto no debería tener 500 caracteres, la base tampoco debería aceptarlo.

Principio reutilizable:

Si una propiedad tiene límites reales, exprésalos en el modelo y en la base.

13.3 Precisión decimal

csharp

```
entity.Property(producto => producto.Precio).HasPrecision(10, 2);  
entity.Property(item => item.PrecioUnitario).HasPrecision(10, 2);
```

Esto indica hasta 10 dígitos, con 2 decimales.

Para dinero, conviene ser explícito. No todos los motores tratan los decimales igual.

13.4 Estado como texto

csharp

```
entity.Property(carrito => carrito.Estado).HasConversion<string>().HasMaxLength(20);
```

EF Core podría guardar el enum como número:

texto

1
2

Pero acá se guarda como texto:

texto

Abierto
Confirmado

Ventajas:

- La base es más legible.
- Los datos son más fáciles de inspeccionar.
- Evita confusiones si cambia el valor numérico del enum.

Costo:

- Ocupa un poco más de espacio.

Para este sistema, la legibilidad vale más que ese costo.

13.5 Relación única entre carrito y producto

```
csharp
entity.HasIndex(item => new { item.CarritoId, item.ProductoId }).IsUnique();
```

Esta regla evita que el mismo producto aparezca dos veces en el mismo carrito.

En vez de tener:

```
texto
NOTE-15 x 1
NOTE-15 x 2
```

el sistema mantiene:

```
texto
NOTE-15 x 3
```

Principio reutilizable:

Si dos filas duplicadas significarían un error de negocio, exprésalo con un índice único.

13.6 Borrado en cascada y borrado restringido

```
csharp
entity.HasOne(item => item.Carrito)
    .WithMany(carrito => carrito.Items)
    .HasForeignKey(item => item.CarritoId)
    .OnDelete(DeleteBehavior.Cascade);
```

Si se borra un carrito, tiene sentido borrar sus items. Un item no existe por sí solo.

```
csharp
entity.HasOne(item => item.Producto)
    .WithMany(producto => producto.ItemsCarrito)
    .HasForeignKey(item => item.ProductoId)
    .OnDelete(DeleteBehavior.Restrict);
```

Si se intenta borrar un producto usado por carritos, se restringe el borrado. Esto protege el historial.

Principio reutilizable:

El comportamiento de borrado debe reflejar dependencia real entre entidades.

Regla práctica:

- Si una entidad hija no tiene sentido sin la padre, cascada.
- Si una entidad referenciada forma parte del historial, restringir.

14. Confirmar o cancelar un carrito

Confirmar no borra el carrito ni crea otra estructura. Cambia su estado dentro de una transacción:

```
csharp
public Carrito ConfirmarCarrito(int carritoId) {
    return EjecutarEnTransaccion(() => {
        var carrito = ObtenerCarrito(carritoId);
        Verificar.Verdadero(carrito.Estado == EstadoCarrito.Abierto, "Solo se pueden confirmar compras de carritos abiertos.");
        Verificar.NoVacio(carrito.Items, "No se puede confirmar una compra con el carrito vacío.");

        carrito.Estado = EstadoCarrito.Confirmado;
        carrito.ConfirmadoUtc = DateTime.UtcNow;

        return carrito;
    });
}
```

Hay tres reglas:

- Solo se puede confirmar un carrito abierto.
- No se puede confirmar un carrito vacío.
- Al confirmar, se guarda fecha UTC.

En esta versión, confirmar no descuenta stock porque la reserva ya ocurrió al agregar productos al carrito.

Cancelar también cambia el estado, pero además libera el stock reservado:

```
csharp
public Carrito CancelarCarrito(int carritoId) {
    var carrito = ObtenerCarrito(carritoId);
    Verificar.Verdadero(carrito.Estado == EstadoCarrito.Abierto, "Solo se pueden cancelar carritos abiertos.");

    return EjecutarEnTransaccion(() => {
        foreach (var item in carrito.Items) {
            item.Producto.StockDisponible += item.Cantidad;
        }

        carrito.Estado = EstadoCarrito.Cancelado;
        return carrito;
    });
}
```

La fecha se guarda en UTC porque es la forma más estable de persistir tiempos.

Principio reutilizable:

Guardar fechas en UTC y convertir a hora local solo al mostrar.

15. Invariantes del sistema

Una invariante es una regla que debe cumplirse siempre.

En este programa aparecen varias:

- Un producto debe tener código.
- Un producto debe tener nombre.
- Un producto debe tener precio mayor a cero.
- Un producto debe tener stock disponible mayor a cero al crearse.
- Un código de producto se guarda normalizado.
- Un carrito debe tener cliente.
- Un item debe tener cantidad mayor a cero.
- Un carrito confirmado no admite cambios.
- Un carrito cancelado no admite cambios.
- Un carrito vacío no se puede confirmar.
- No se puede agregar más cantidad que el stock disponible.
- Quitar o cancelar debe restituir el stock reservado.
- Un mismo producto no debe aparecer dos veces en el mismo carrito.

Algunas invariantes se protegen en C#:

```
csharp
if (cantidad <= 0) {
    throw new ArgumentOutOfRangeException(nameof(cantidad), "La cantidad debe ser mayor a cero.");
}
```

Otras se protegen en la base:

```
csharp
entity.HasIndex(item => new { item.CarritoId, item.ProductoId }).IsUnique();
```

Principio reutilizable:

Las reglas importantes deben estar cerca del dominio y, cuando corresponde, reforzadas por la base de datos.

16. Qué se podría mejorar en una versión real

Este ejemplo está pensado para aprender. Si lo lleváramos hacia un sistema real, agregaríamos varias piezas.

16.1 Migraciones

Reemplazar:

```
csharp
EnsureDeleted();
EnsureCreated();
```

por migraciones versionadas.

16.2 Inyección de dependencias

En una API o app web, `TiendaContext` y `TiendaService` se registrarían en el contenedor:

```
csharp
builder.Services.AddDbContext<TiendaContext>(...);
builder.Services.AddScoped<TiendaService>();
```

16.3 Concurrencia de stock

El ejemplo ya usa transacciones explícitas para agregar, quitar, confirmar y cancelar. En un sistema real faltaría resolver mejor la concurrencia entre usuarios:

- qué hacer si dos usuarios reservan el último ítem al mismo tiempo
- si conviene usar concurrencia optimista, bloqueos o reintentos
- cómo reportar conflictos de stock de manera amigable

16.4 Política de reservas

El ejemplo reserva stock al agregar al carrito y lo libera al quitar o cancelar. En un sistema real probablemente habría que definir además:

- vencimiento automático de carritos abandonados
- liberación diferida de stock
- auditoría de cambios de stock
- stock por sucursal o depósito

16.5 Usuarios reales

`Cliente` hoy es un string. En un sistema real probablemente sería una entidad:

```
texto
Cliente
  Id
  Nombre
  Email
  Direcciones
```

16.6 Pagos y auditoría

Confirmar carrito no es lo mismo que pagar. Un sistema real podría separar:

- carrito armado
- orden creada
- pago pendiente
- pago aprobado
- factura emitida

16.7 Tests

Las reglas de `TiendaService` son buenos candidatos para tests:

- agregar producto nuevo
- sumar cantidad si el producto ya existe en el carrito
- eliminar parcialmente
- eliminar completamente
- impedir modificar carrito confirmado
- impedir modificar carrito cancelado
- impedir confirmar carrito vacío
- liberar stock al cancelar

17. Checklist para futuros sistemas

Cuando modelas otro sistema, podés usar esta lista:

1. Identificar entidades principales.
2. Identificar relaciones entre entidades.
3. Detectar si alguna relación necesita datos propios.
4. Elegir tipos adecuados (`decimal` para dinero, `DateTime UTC` para timestamps).
5. Definir estados explícitos con `enum` cuando haya ciclo de vida.
6. Centralizar normalización de identificadores de negocio.
7. Validar entradas en el servicio o capa de aplicación.
8. Evitar que la UI o consola contenga reglas de negocio.
9. Usar `AsNoTracking` en consultas de lectura.
10. Usar `Include` cuando el caso de uso necesita relaciones cargadas.
11. Configurar índices únicos para reglas de unicidad.
12. Configurar precisión, longitudes y obligatoriedad.

13. Pensar el comportamiento de borrado antes de elegir cascada o restricción.
14. Guardar datos históricos importantes como snapshots.
15. Usar errores con mensajes del dominio.
16. Usar migraciones en sistemas persistentes.
17. Escribir tests para invariantes importantes.

18. Resumen de las decisiones más importantes

Decisión	Motivo
<code>CarritoItem</code> como entidad	La relación carrito-producto necesita cantidad y precio propio.
<code>PrecioUnitario</code> en el item	Congela el precio usado en ese carrito.
<code>decimal</code> para precios	Evita errores de precisión de <code>double</code> .
<code>EstadoCarrito</code> como enum	Modela el ciclo de vida del carrito, incluyendo cancelación, mejor que booleanos.
<code>Subtotal</code> y <code>Total</code> calculados	Evita duplicar datos derivados en la base.
<code>TiendaService</code>	Centraliza reglas y operaciones del sistema.
Reserva inmediata de stock	Hace visible el impacto del carrito sobre inventario desde el momento de agregar productos.
Normalización de código	Evita duplicados e inconsistencias por mayúsculas o espacios.
Errores explícitos	Hace que las fallas expliquen el problema real.
<code>AsNoTracking</code>	Mejora consultas de solo lectura.
<code>Include</code>	Carga relaciones necesarias de forma explícita.
Índices únicos	La base protege reglas de negocio.
Transacciones explícitas	Mantienen consistentes carrito y stock cuando una operación modifica varias entidades.
<code>DeleteBehavior.Cascade</code> para items	Un item no tiene sentido sin su carrito.
<code>DeleteBehavior.Restrict</code> para productos	Protege historial de carritos existentes.
UTC para fechas	Evita ambigüedades de zona horaria.

19. Idea final

Este programa no intenta ser un ecommerce completo. Su valor está en mostrar una estructura sana:

texto

```
Modelo claro
+ reglas centralizadas
+ persistencia configurada
+ errores comprensibles
+ datos históricos protegidos
= base reutilizable para sistemas más grandes
```

Cuando un sistema crece, los problemas no aparecen solamente por falta de tecnologías. Aparecen porque las reglas están mezcladas, los datos no tienen límites claros, las relaciones están mal modeladas o la base no protege lo que el dominio necesita.

El objetivo de este ejemplo es entrenar esa mirada: antes de escribir mucho código, entender qué cosas existen, qué reglas las gobiernan y dónde conviene ubicar cada responsabilidad.

CAPITULO 58

Construyendo un agente con shell en C#

Microsoft Agent Framework + OpenAI

C# 14 / .NET 10 / Microsoft.Agents.AI 1.0

¿Qué vamos a construir?

Una app de consola en C# que replica el comportamiento de un agente tipo "Nano": vos le pedís algo en lenguaje natural, y el agente decide cuándo ejecutar comandos de shell en tu máquina para cumplir la tarea. Mantiene contexto entre turnos y trabaja sobre un directorio específico (el workspace).

Es, en esencia, un *mini* Claude Code / Codex / Cursor CLI. Entender cómo se arma esto por dentro te da una ventaja enorme: dejás de ver a los agentes como magia y empezás a verlos como lo que son — un loop bien orquestado entre un LLM y un conjunto de herramientas.

El objetivo no es solo que termines con este programa funcionando, sino que entiendas **por qué cada pieza está donde está**, para que después puedas armar agentes distintos: uno que administre tu calendario, otro que analice logs, otro que consulte tu base de datos, etc.

Parte 1 — Primeros principios

Antes de tocar Visual Studio, necesitás tener claros cinco conceptos. Si te los saltás, vas a terminar copiando código sin entender qué hace.

1.1 — Un LLM por sí solo no hace nada

Un modelo de lenguaje (GPT, Claude, Gemini, lo que sea) es una función pura: le mandás texto, te devuelve texto. No tiene memoria, no tiene archivos, no tiene red, no puede ejecutar nada. Es literalmente un *stateless text transformer*.

Entonces, ¿cómo hace ChatGPT para "recordar" lo que le dijiste hace 5 mensajes? Le mandan **todos los mensajes anteriores** en cada request. La "memoria" vive fuera del modelo, en el cliente.

Y ¿cómo hace para "ejecutar código" o "buscar en internet"? No lo hace. Lo que hace es **pedir** que ejecutemos algo por él, y nosotros le devolvemos el resultado como si fuera más texto.

Eso es todo. Un agente es un loop alrededor de esta idea.

1.2 — El loop del agente

Todo agente, por sofisticado que parezca, es este loop:

texto

1. Usuario escribe algo
2. Cliente manda al LLM: historial + mensaje nuevo + lista de herramientas disponibles
3. LLM responde una de dos cosas:
 - a) Texto final para el usuario → mostramos y volvemos al paso 1
 - b) "Quiero llamar a la herramienta X con argumentos Y"
4. Si es (b): ejecutamos X(Y), agregamos el resultado al historial, volvemos al paso 2

Ese paso 4 es la magia. El LLM no ejecuta nada — solo *pide* que ejecutemos algo. Nosotros (el código C#) decidimos si le hacemos caso, cómo, y qué le devolvemos.

1.3 — Tool calling (la gramática del pedido)

Para que el modelo pueda "pedir" que ejecutemos algo, necesita dos cosas:

1. **Saber qué herramientas existen.** Le pasamos una lista con nombre, descripción y schema de parámetros de cada función. Esto va en cada request.
2. **Una forma estructurada de pedir.** Los modelos modernos devuelven un JSON con el nombre de la función y los argumentos cuando quieren invocarla, en lugar de texto plano.

OpenAI formalizó esto como *function calling*. Hoy es estándar.

1.4 — Por qué usar un framework

Podrías implementar todo esto a mano con `HttpClient` y la API REST de OpenAI. Serían unas 300 líneas de JSON parsing, manejo del loop, serialización de schemas, reintentos, streaming. Lo hicimos durante un tiempo. Es tedioso y repetitivo.

Microsoft Agent Framework (MAF) es la abstracción que te evita escribir ese plumbing. Te da:

- `AIAgent`: representa al agente, con sus instrucciones y herramientas.
- `AgentThread`: maneja el historial de la conversación.
- `AIFunctionFactory`: convierte cualquier método C# en una herramienta con su schema JSON autogenerado por reflexión.
- Un runner interno que ejecuta el loop 1-2-3-4 por vos.

MAF llegó a 1.0 en abril de 2026, reemplazando a Semantic Kernel y AutoGen como la recomendación oficial de Microsoft. Viene incluido en .NET 10.

1.5 — Por qué la shell como herramienta

Nuestro agente va a tener **una sola herramienta**: ejecutar un comando de shell.

Parece trampa — y en cierto sentido lo es. Una shell es una meta-herramienta: con `ls`, `cat`, `grep`, `sed`, `dotnet`, `git`, etc., el agente tiene acceso a todo el sistema. No necesitás darle 50 herramientas distintas; le das shell y listo.

Esta es la filosofía de Claude Code y Codex CLI. Es también el motivo por el cual son *peligrosos* si no se ejecutan en un directorio acotado: si le das `rm -rf` como opción, tarde o temprano lo va a usar mal.

Por eso vamos a restringirlo a un workspace. La decisión de diseño: **un agente con pocas herramientas poderosas es más flexible y más simple que uno con muchas herramientas específicas**, pero requiere más cuidado con los límites.

Parte 2 — Preparación del proyecto

2.1 — Requisitos

- .NET 10 SDK instalado (`dotnet --version` debería mostrar 10.x).
- Una API key de OpenAI. La conseguís en <https://platform.openai.com/api-keys>.
- Un editor: VS Code, Visual Studio 2026 o Rider.

2.2 — Crear el proyecto

Abrí una terminal en el directorio donde quieras el proyecto y:

```
bash
dotnet new console -n NanoAgent -f net10.0
cd NanoAgent
```

2.3 — Agregar los paquetes NuGet

Necesitás tres paquetes:

```
bash
dotnet add package Microsoft.Agents.AI --prerelease
dotnet add package Microsoft.Agents.AI.OpenAI --prerelease
dotnet add package Microsoft.Extensions.AI
```

Nota: al momento de escribir esto, los paquetes `Microsoft.Agents.AI.*` pueden seguir marcándose como `prerelease` en NuGet aunque la librería esté en 1.0. Si te da error, sacá el `--prerelease` y probá.

Revisá tu `NanoAgent.csproj`. Debería verse así:

```
xml
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <LangVersion>latest</LangVersion>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.Agents.AI" Version="1.0.*" />
    <PackageReference Include="Microsoft.Agents.AI.OpenAI" Version="1.0.*" />
    <PackageReference Include="Microsoft.Extensions.AI" Version="9.*" />
  </ItemGroup>
</Project>
```

```
xml - continuacion
</ItemGroup>
</Project>
```

2.4 — Configurar la API key

No hardcodeés la API key en el código. Nunca. Usá una variable de entorno:

Linux/macOS (~/.bashrc o ~/.zshrc):

```
bash
export OPENAI_API_KEY="sk-..."
```

Windows (PowerShell como admin, persistente):

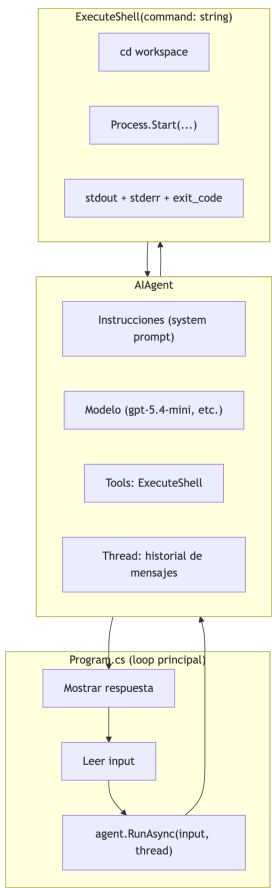
```
powershell
[System.Environment]::SetEnvironmentVariable("OPENAI_API_KEY", "sk-...", "User")
```

Cerrá y reabrí la terminal para que tome la variable.

Regla de oro: si subís una API key a GitHub, aunque sea un segundo, considerala comprometida y revocala. Los bots escanean el repo en segundos.

Parte 3 — Arquitectura del agente

Antes de escribir una línea, dibujemos el programa.



Diagrama

Tres piezas. Una sola herramienta. Un solo loop.

Decisiones de diseño que vale la pena hacer explícitas

Decisión	Por qué
Workspace fijo al directorio del ejecutable	Evita que el agente toque archivos fuera de ese scope. Mitigación simple, efectiva.
Una sola tool ExecuteShell	Menos código, más flexible. El LLM ya sabe usar shell.
Devolver stdout + stderr + exit_code juntos	El modelo necesita los tres para decidir qué hacer después (reintentar, ajustar, etc.).
Thread gestionado por MAF (no lo serializamos)	Para una app interactiva simple, la memoria en proceso alcanza. Si querés persistencia, se puede serializar después.
Instrucciones del sistema en un archivo externo	Así las podés editar sin recompilar. Mismo principio que tu archivo de prompt en Python.
Modelo configurable por constante	Fácil de cambiar entre gpt-5.4-mini, gpt-5.4, etc. según costo/calidad.

Parte 4 — Código, pieza por pieza

Vamos a armar `Program.cs` incrementalmente. Al final tenés el archivo completo.

4.1 — Imports y constantes

```
csharp
using System.ClientModel;
using System.ComponentModel;
using System.Diagnostics;
using System.Text;
using Microsoft.Agents.AI;
using Microsoft.Extensions.AI;
using OpenAI;

// --- Configuración ---
const string Model = "gpt-5.4-mini";
const string AgentName = "Nano";

string workspace = AppContext.BaseDirectory;
string promptFile = Path.Combine(workspace, "AGENTS.md");

string apiKey = Environment.GetEnvironmentVariable("OPENAI_API_KEY")
    ?? throw new InvalidOperationException(
        "Definí la variable de entorno OPENAI_API_KEY");
```

Qué está pasando:

- `AppContext.BaseDirectory` te da el directorio donde está corriendo el ejecutable. Es nuestro "workspace" — el sandbox donde el agente puede operar.
- Leemos la API key de la variable de entorno. Si no está, fallamos *temprano y ruidosamente*. Esto es un patrón importante: errores de configuración deben explotar al arrancar, no a los 10 minutos cuando el usuario haga algo.
- El modelo lo ponemos como constante. Mañana querés probar `gpt-5.4`, cambiás una línea.

4.2 — Las instrucciones del sistema

Las instrucciones (el "system prompt") son el ADN del agente. Definen quién es, qué puede hacer, qué estilo tiene, qué restricciones respeta.

```
csharp
static string BuildInstructions(string workspace, string promptFile) {
    string baseRules = $"""
    Reglas operativas:
    - Trabajás en {workspace}.
    - Antes de modificar, leé archivos relevantes.
    - Usá la herramienta ExecuteShell para leer, escribir y ejecutar.
    - No inventes resultados: si no sabés algo, ejecutá un comando para averiguarlo.
    - Respondé corto y en español.
    """;

    if (File.Exists(promptFile)) {
        string header = File.ReadAllText(promptFile)
            .Replace("{workspace}", workspace)
            .Trim();
        return $"{header}\n\n{baseRules}";
    }

    return baseRules;
}
```

Por qué este diseño:

- Hay dos niveles: un header opcional editable (desde `AGENTS.md`) y reglas base siempre presentes. El header sirve para personalizar la "personalidad" del agente sin tocar el código; las reglas base son el contrato operativo que no queremos perder.
- Usamos *raw string literals* (`"""..."""`, C# 11+) que respetan los saltos de línea y evitan el infierno de concatenaciones.
- El placeholder `{workspace}` lo reemplazamos manualmente. No usamos string interpolation dentro del archivo porque los archivos de texto no tienen interpolación — se reemplaza en tiempo de lectura.
- El nombre `AGENTS.md` no es casual: se está convirtiendo en una convención de facto en la industria (Cursor, Claude Code, Codex lo usan) para dejar instrucciones al agente que trabaja en un repositorio. Si ponés este agente en un proyecto que ya tiene un `AGENTS.md`, lo va a respetar automáticamente.

Primer principio oculto: las instrucciones son código. Tratalas como código. Versionalas. Probá cambios. Si cambiás el *system prompt* y el agente empieza a comportarse raro, es tu "bug".

4.3 — La herramienta: ExecuteShell

Acá está el corazón del asunto. Queremos una función C# que:

1. Reciba un comando como string.
2. Lo ejecute en el workspace.
3. Devuelva stdout, stderr y exit code como un único string estructurado.

csharp

```
[Description("Ejecuta un comando de shell en el workspace y devuelve su salida (stdout, stderr y exit code). Usala para listar archivos, leer contenido, correr programas, etc.")]
static string ExecuteShell(
    [Description("El comando exacto a ejecutar, tal como lo tipearías en una terminal.")]
    string command) {
    // En Windows usamos cmd.exe, en Unix bash
    bool isWindows = OperatingSystem.IsWindows();
    string shell = isWindows ? "cmd.exe" : "/bin/bash";
    string flag = isWindows ? "/c" : "-c";

    var psi = new ProcessStartInfo {
        FileName = shell,
        Arguments = $"{flag} \"{command.Replace("\"", "\\\"")}\"",
        WorkingDirectory = workspace,
        RedirectStandardOutput = true,
        RedirectStandardError = true,
        UseShellExecute = false,
        CreateNoWindow = true,
        StandardOutputEncoding = Encoding.UTF8,
        StandardErrorEncoding = Encoding.UTF8,
    };

    try {
        using var proc = Process.Start(psi)
            ?? throw new InvalidOperationException("No se pudo iniciar el proceso.");

        string stdout = proc.StandardOutput.ReadToEnd();
        string stderr = proc.StandardError.ReadToEnd();
        proc.WaitForExit();

        return $"""
            exit_code: {proc.ExitCode}
            --- stdout ---
        """
    }
}
```

csharp - continuacion

```

        {Truncate(stdout, 4000)}
        --- stderr ---
        {Truncate(stderr, 2000)}
        """;
    } catch (Exception ex) {
        return $"ERROR al ejecutar: {ex.Message}";
    }
}

static string Truncate(string s, int max) =>
    s.Length <= max ? s : s[..max] + $"\\n[... truncado, {s.Length - max} caracteres más ...]";

```

Puntos importantes (prestá atención, acá hay varios):

1. [Description(...)]. Este atributo es crítico. MAF lo lee con reflexión y lo incluye en el schema JSON que le manda al LLM. Esa descripción es lo que el modelo ve cuando decide si llamar esta función. Mala descripción → el modelo no sabe cuándo usarla. La descripción del parámetro también viaja al modelo.
1. **Workspace como directorio de trabajo del proceso.** WorkingDirectory = workspace hace que cualquier ruta relativa que use el comando se resuelva desde ahí. El agente puede hacer `cat mifile.txt` sin saber la ruta absoluta.
1. **Windows vs Unix.** Detectamos el SO y usamos la shell nativa. `cmd.exe /c "..."` en Windows, `/bin/bash -c "..."` en Linux/macOS. Esto ya hace tu agente multiplataforma.
1. **Truncado de la salida.** Un `ls -R /` podría devolver megabytes. Si mandás todo eso al modelo, (a) explota la ventana de contexto, (b) te sale carísimo. Truncamos a ~4KB stdout, ~2KB stderr, y le avisamos al modelo que truncamos. *Design decision: mejor un resultado truncado con aviso que un crash o una factura de USD 300.*
1. **Manejo de errores devuelto como string**, no como excepción. ¿Por qué? Porque si lanzás una excepción, MAF la propaga y el loop se rompe. Si devolvés un string "ERROR: ...", el modelo lo lee y puede reaccionar (probar otra cosa, reportarle al usuario, etc.). Los agentes deben manejar errores "conversacionalmente".
1. `using var proc.Process` implementa `IDisposable`. Sin `using` vas a tener process handles colgados — bug clásico.

Un comentario pedagógico: este método son 40 líneas. En Python con `subprocess.run` serían 5. C# es más verboso acá porque `Process.Start` está diseñado para casos generales, no para "correr un comando y leer su salida". La verbosidad vale la pena porque ganás control fino sobre timeouts, encoding, streaming, etc. — cosas que un alumno avanzado va a necesitar.

4.4 — Crear el agente

```
csharp
// 1. Cliente de OpenAI
var openAIClient = new OpenAIClient(new ApiKeyCredential(apiKey));

// 2. Chat client apuntado a un modelo específico
var chatClient = openAIClient.GetChatClient(Model);

// 3. Convertir nuestro método en un AIFunction (tool)
var shellTool = AIFunctionFactory.Create(ExecuteShell);

// 4. Armar el agente
IAgent agent = chatClient.CreateAIAgent(
    instructions: BuildInstructions(workspace, promptFile),
    name: AgentName,
    tools: [shellTool]
);

// 5. Crear el thread (historial de la conversación)
AgentThread thread = agent.GetNewThread();
```

Esto son 5 líneas reales de configuración. Mirá lo que cada una significa:

- `OpenAIClient` (del SDK oficial de OpenAI). Cliente HTTP base hacia `api.openai.com`.
- `GetChatClient(Model)`. Te da un cliente específico que apunta a ese modelo via la Chat Completions API. MAF soporta tres estilos: Chat Completions (el más simple, más universal), Responses (más rico), Assistants (estado en el servidor). Nosotros usamos Chat Completions porque es el más directo y portable — el mismo código funciona con Azure OpenAI, Ollama local, o cualquier endpoint compatible con OpenAI.
- `AIFunctionFactory.Create(ExecuteShell)`. Introspecciona el método con reflexión: lee el nombre, los parámetros, sus tipos, las anotaciones `[Description]`, y genera un `AIFunction` con todo el schema JSON que el LLM va a ver. Esto es lo que te ahorra escribir el schema a mano.
- `CreateAIAgent(...)`. Extension method que empaqueta chat client + instrucciones + tools en un `IAgent`.
- `GetNewThread()`. Crea un hilo de conversación vacío. El thread es donde vive la memoria de esta sesión.

4.5 — El loop principal (REPL)

```
csharp
Console.WriteLine($"Agente '{AgentName}' listo en {workspace}");
Console.WriteLine("Escribí 'salir' para terminar.\n");

while (true) {
    Console.Write("Vos> ");
    string? input = Console.ReadLine()?.Trim();

    if (string.IsNullOrEmpty(input))
        continue;

    if (input.Equals("salir", StringComparison.OrdinalIgnoreCase) ||
        input.Equals("exit", StringComparison.OrdinalIgnoreCase))
        break;

    try {
        AgentRunResponse response = await agent.RunAsync(input, thread);
        Console.WriteLine($"\\nAgente> {response.Text}\\n");
    } catch (Exception ex) {
        Console.WriteLine($"\\n[ERROR] {ex.Message}\\n");
    }
}

Console.WriteLine("Chau.");
```

Qué observar:

- `agent.RunAsync(input, thread)` es *toda* la lógica del loop 1-2-3-4 de la sección 1.2. Adentro, MAF:
 1. Agrega el input al thread.
 2. Manda el thread entero (+ instrucciones + tools) a OpenAI.
 3. Si la respuesta pide llamar a una tool, ejecuta `ExecuteShell` con los args que el modelo pidió.
 4. Mete el resultado en el thread.
 5. Vuelve a llamar al modelo.
 6. Repite hasta que el modelo no pida más tools y devuelva texto final.
 7. Devuelve ese texto.
- Puede hacer varias idas y vueltas en una sola `RunAsync`. Si vos pedís "listá los .cs y resumí el más grande", el agente puede llamar `ExecuteShell("ls *.cs")`, después `ExecuteShell("wc -l *.cs | sort")`, después `ExecuteShell("cat big.cs")`, y recién ahí respondernos. Todo eso pasa dentro de un `RunAsync`.
- El `try/catch` externo te protege de errores de red, rate limits, etc. Sin él, un hipo del modelo mata tu programa.

Parte 5 — El archivo completo

Acá va `Program.cs` entero, listo para copiar y pegar:

texto

```
Vos> listá los archivos del directorio
Vos> ¿cuántas líneas tiene Program.cs?
Vos> creá un archivo llamado hola.txt con el texto "Hola UTN"
Vos> leé hola.txt y decime qué dice
Vos> borrá hola.txt
```

Vas a ver cómo el agente *decide* cuándo llamar `ExecuteShell` y cuándo responder directamente. Preguntale "¿cuánto es 2+2?" y no debería ejecutar nada — el modelo sabe sumar. Preguntale "¿qué archivos .cs hay acá?" y vas a verlo ejecutar `ls *.cs` o `dir *.cs`.

Ejercicio mental

Antes de seguir, preguntate: **¿cómo sabe el agente cuándo usar la herramienta?**

La respuesta: no "sabe" en el sentido humano. Recibió un schema JSON que describe la tool y su descripción ("Ejecuta un comando de shell..."). Cuando procesa tu pedido, el modelo calcula que es más probable producir una llamada a esa función que producir texto directo. Es estadística, no razonamiento — pero con modelos grandes la estadística se parece sospechosamente a razonamiento.

Si cambiás la descripción de la tool a "usala solo cuando el usuario diga la palabra PLIM", el agente va a cambiar su comportamiento. Probalo. Es instructivo.

Parte 7 — Decisiones de diseño aplicables a otros agentes

Hasta acá tenés una app funcionando. Ahora viene lo importante: **los principios transferibles**. Estos son los que deberías llevarte a cualquier agente que construyas en el futuro.

7.1 — Granularidad de las herramientas

Una tool poderosa (shell) o muchas tools específicas (`ListFiles`, `ReadFile`, `WriteFile`, `DeleteFile`, `RunCommand`, ...). ¿Cuándo conviene cada una?

- **Poderosa y pocas:** cuando el dominio es abierto y el modelo es bueno (GPT-4 y superiores). El modelo improvisa. Más flexible, menos código.
- **Específicas y muchas:** cuando hay reglas de negocio que querés hacer cumplir *desde el código*. Ejemplo: un agente bancario no debería tener `ExecuteSQL` — debería tener `TransferirEntre(cuentaOrigen, cuentaDestino, monto)` que valida permisos, límites diarios, antifraude, y recién ahí toca la base. La herramienta es el control.

Regla: **cuanto más sensible la operación, más específica la tool.**

7.2 — Descripciones: el 80/20 del éxito

La diferencia entre un agente que funciona y uno que no suele estar en las descripciones de las tools y en el system prompt. No en el código.

Probá esto: con tu `ExecuteShell` funcionando, cambiá la descripción a simplemente "Ejecuta comandos ". Vas a ver que el agente se vuelve menos confiable — a veces la llama cuando no

corresponde, a veces no la llama cuando sí.

Las descripciones de tools no son comentarios decorativos. Son *parte del contrato con el modelo*. Tratalas con el mismo cuidado que tratás un endpoint de tu API.

7.3 — Errores como datos, no como excepciones

Regla general para agentes: **devolvé los errores como texto que el modelo pueda leer**. No lances excepciones que rompan el loop.

¿Por qué? Porque los modelos son buenísimos para reaccionar a errores. Si `ExecuteShell` devuelve `"exit_code: 1, stderr: file not found"`, el agente muy probablemente va a probar otra cosa — listar el directorio, chequear el nombre, preguntarte. Si en cambio lanzás una excepción, el loop muere y el usuario ve un stack trace.

Este patrón va más allá de los agentes. Es una forma de pensar: *¿qué información necesita quien me llama para recuperarse de este error?*

7.4 — Contexto acotado

Cada token que mandás al modelo cuesta plata. Y la ventana de contexto, aunque sea de 128K tokens, no es infinita. Dos mitigaciones básicas:

1. **Truncá los outputs largos.** Como hicimos con el shell. Si un comando devuelve 1 MB, mandarle todo eso al modelo es desperdicio.
2. **Considerá estrategias de memoria a largo plazo.** Si la conversación se alarga, podés resumir turnos viejos, guardar hechos importantes en una base, o empezar threads nuevos. MAF deja serializar threads (`AgentThread.SerializeAsync`) para persistirlos, pero la estrategia de gestión es tuya.

7.5 — El workspace como sandbox

Nuestro agente puede ejecutar `rm -rf` dentro del workspace. Podría borrarle todo el proyecto. ¿Cómo mitigamos?

Opciones, de menos a más seguro:

1. **Confiar en el usuario:** solo ponés el agente en un workspace que podés perder (ej: un git repo que podés resetear). Lo que hicimos.
2. **Blacklist de comandos:** antes de ejecutar, chequeás que el comando no contenga `rm -rf`, `dd`, `format`, etc. Frágil.
3. **Aprobación humana:** antes de cada comando, mostrás al usuario qué va a ejecutar y esperás confirmación. Es lo que hace Claude Code.
4. **Contenedor/VM:** corrés el agente adentro de un container sin red y con un volumen efímero. Máxima seguridad, más infra.

Para un ejercicio de clase, (1) alcanza. Para producción, casi siempre querés (3) o (4).

7.6 — El system prompt es infraestructura

El system prompt no es "un texto que le decís al modelo". Es la *configuración* del agente. Como cualquier config:

- Versionalo (commítealo a git).
- Probá cambios (un prompt distinto puede romper tests que antes pasaban).
- Documentá por qué cada línea está ahí.

El archivo externo `AGENTS.md` que propusimos es una forma simple de empezar. En proyectos más serios vas a querer templating, variantes por entorno (dev/prod), A/B testing de prompts, etc.

Cierre

Lo que acabamos de construir son ~100 líneas de C#. Hace dos años, armar algo con esta capacidad te llevaba una semana y mil líneas de plumbing. Ese es el valor de frameworks como Microsoft Agent Framework: bajan la barrera de entrada.

Pero — y este es el punto pedagógico que me importa — la baja barrera de entrada es un riesgo si no entendés los primeros principios. Un alumno que copia y pega código con un framework puede tener un agente funcionando sin entender qué es un tool call, qué es un thread, por qué la descripción importa, o por qué el error debe volver como dato. Cuando el agente falle (y va a fallar), no va a saber qué tocar.

Por eso este tutorial gasta más tiempo en el *por qué* que en el *cómo*. El cómo lo podés mirar en la documentación de Microsoft cuando lo necesites. El por qué es lo que te hace ingeniero.

CAPITULO 59

Tutorial: Sitio web con ASP.NET Core y Razor Pages

En este tutorial vamos a construir un sitio web completo usando ASP.NET Core y Razor Pages. Para que el aprendizaje sea concreto, vamos a implementar un CRUD de una agenda de contactos: crear, listar, editar y eliminar contactos.

El objetivo no es solo que funcione, sino entender qué hace cada pieza y por qué está ahí. Así vas a poder encarar tu propio proyecto con criterio.

Entender el problema

Una agenda de contactos es un problema clásico de CRUD. CRUD es el acrónimo de Create, Read, Update y Delete, las cuatro operaciones básicas sobre datos.

Queremos poder hacer esto:

```
texto
GET /Agenda          → listar todos los contactos
GET /Agenda/Create    → mostrar el formulario de nuevo contacto
POST /Agenda/Create   → guardar el nuevo contacto
GET /Agenda/Edit/5    → mostrar el formulario de edición del contacto 5
POST /Agenda/Edit/5   → guardar los cambios del contacto 5
GET /Agenda/Delete/5  → pedir confirmación antes de eliminar
POST /Agenda/Delete/5 → eliminar el contacto 5
GET /Agenda/Details/5 → ver el detalle del contacto 5
```

Eso es todo. Ocho rutas, cuatro operaciones. El resto es cómo ASP.NET Core nos ayuda a implementarlas de forma ordenada.

Cómo pensar la solución

Antes de escribir código conviene tener un modelo mental claro. En ASP.NET Core con Razor Pages, cada pantalla de la aplicación es un par de archivos:

- un archivo `.cshtml` que contiene la vista, es decir, el HTML que el usuario va a ver;
- un archivo `.cshtml.cs` que contiene el `PageModel`, es decir, la lógica de esa pantalla.

La vista conoce al `PageModel`. El `PageModel` no conoce a la vista. Esta separación hace que el código de lógica sea fácil de leer y de testear.

El servidor recibe una petición HTTP, la dirige al `PageModel` correspondiente, el `PageModel` hace su trabajo (consultar la base de datos, validar datos, etc.) y le pasa los resultados a la vista, que los renderiza como HTML y se lo envía al navegador.

Configuración del proyecto

Vamos a crear el proyecto desde la línea de comandos. Si preferís hacerlo desde Visual Studio, el resultado es el mismo.

texto

```
dotnet new webapp -n AgendaApp
cd AgendaApp
```

Esto genera una aplicación web mínima con la estructura de Razor Pages. Para agregar Entity Framework Core y el soporte para SQLite ejecutamos:

texto

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Tools
dotnet add package Microsoft.EntityFrameworkCore.Design
```

Con esto ya tenemos todo lo que necesitamos.

La estructura del proyecto

Al crear el proyecto vamos a ver esta estructura de carpetas:

texto

```
AgendaApp/
├── Pages/
│   ├── Index.cshtml
│   ├── Index.cshtml.cs
│   ├── Shared/
│   └── _Layout.cshtml
├── wwwroot/
├── appsettings.json
└── Program.cs
```

La carpeta Pages/ es donde viven todas las pantallas. La carpeta wwwroot/ contiene los archivos estáticos: CSS, JavaScript e imágenes. El archivo Program.cs es el punto de entrada de la aplicación, donde configuramos los servicios y el pipeline de middleware.

Modelado de los datos

Antes de tocar las páginas, necesitamos definir los datos con los que vamos a trabajar. Un contacto tiene nombre, teléfono, email y algunas propiedades opcionales.

csharp

```
// Models/Contacto.cs
using System.ComponentModel.DataAnnotations;

public class Contacto
{
    public int Id { get; set; }

    [Required(ErrorMessage = "El nombre es obligatorio")]
    [StringLength(100)]
    [Display(Name = "Nombre completo")]
}
```

csharp - continuacion

```

public string Nombre { get; set; } = string.Empty;

[Required(ErrorMessage = "El teléfono es obligatorio")]
[Display(Name = "Teléfono")]
public string Telefono { get; set; } = string.Empty;

[EmailAddress(ErrorMessage = "El formato del email no es válido")]
[Display(Name = "Correo electrónico")]
public string? Email { get; set; }

[Display(Name = "Dirección")]
public string? Direccion { get; set; }

[Display(Name = "Notas")]
public string? Notas { get; set; }

[Display(Name = "Favorito")]
public bool EsFavorito { get; set; }
}

```

Los atributos como [Required] o [EmailAddress] tienen doble función: Entity Framework los usa para definir las columnas de la base de datos, y ASP.NET los usa para validar los datos del formulario antes de guardarlos.

El DbContext

El DbContext es la clase que representa la sesión con la base de datos. Exponemos en él las colecciones de entidades que queremos persistir.

csharp

```

// Data/AgendaDbContext.cs
using Microsoft.EntityFrameworkCore;

public class AgendaDbContext : DbContext
{
    public AgendaDbContext(DbContextOptions<AgendaDbContext> options)
        : base(options) { }

    public DbSet<Contacto> Contactos { get; set; }
}

```

Cada DbSet<T> corresponde a una tabla en la base de datos. Entity Framework se encarga de crear esa tabla y de traducir nuestras consultas de C# a SQL.

Configurar los servicios

En Program.cs registramos el DbContext para que el sistema de inyección de dependencias pueda proveerlo a cualquier clase que lo necesite.

csharp

```

// Program.cs
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddRazorPages();
builder.Services.AddDbContext<AgendaDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("AgendaConnection")));

```

csharp - continuacion

```
var app = builder.Build();

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.MapRazorPages();

app.Run();
```

Y en `appsettings.json` agregamos la cadena de conexión:

json

```
{
  "ConnectionStrings": {
    "AgendaConnection": "Data Source=agenda.db"
  }
}
```

Migraciones

Las migraciones son la forma en que Entity Framework versiona el esquema de la base de datos. Cada vez que cambiamos el modelo, creamos una migración que describe cómo actualizar la base de datos.

texto

```
dotnet ef migrations add CreacionInicial
dotnet ef database update
```

El primer comando genera los archivos de migración. El segundo los aplica y crea el archivo `agenda.db`. A partir de acá, cada cambio en el modelo se refleja con una nueva migración.

La sintaxis Razor

Antes de construir las páginas conviene entender cómo funciona Razor. Es un motor de plantillas que mezcla HTML con C# usando el símbolo `@`.

html

```
@* Esto es un comentario Razor *@

@{
    var titulo = "Mi Agenda";
}

<h1>@titulo</h1>

@if (Model.Contactos.Any())
{
    <p>Hay @Model.Contactos.Count contactos.</p>
}
else
{
    <p>No hay contactos todavía.</p>
}

@foreach (var c in Model.Contactos)
{
```

```
html - continuación
<li>@c.Nombre</li>
}
```

El servidor procesa este archivo, ejecuta el C# y devuelve HTML puro al navegador. El navegador nunca ve la sintaxis @.

Los Tag Helpers son atributos HTML especiales que empiezan con asp-. Generan el HTML correcto a partir del modelo:

```
html
<a asp-page="/Edit" asp-route-id="@c.Id">Editar</a>
<input asp-for="Contacto.Nombre" class="form-control" />
<span asp-validation-for="Contacto.Nombre" class="text-danger"></span>
```

Usamos asp-page en lugar de escribir la URL a mano, asp-for para enlazar un input a una propiedad del modelo, y asp-validation-for para mostrar automáticamente el mensaje de error de validación.

El Layout

El archivo Pages/Shared/_Layout.cshtml es la plantilla base del sitio. Contiene el HTML común a todas las páginas: el <head>, la barra de navegación y el pie de página. El contenido específico de cada página se inserta donde aparece @RenderBody().

```
html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <title>@ViewData["Title"] - AgendaApp</title>
  <link rel="stylesheet" href="~/lib/bootstrap/css/bootstrap.min.css" />
</head>
<body>
  <nav class="navbar navbar-dark bg-primary px-3">
    <a class="navbar-brand" asp-page="/Index">Mi Agenda</a>
    <a asp-page="/Agenda/Index" class="nav-link text-white">Contactos</a>
  </nav>

  <div class="container mt-4">
    @if (TempData["Exito"] != null)
    {
      <div class="alert alert-success">@TempData["Exito"]</div>
    }
    @RenderBody()
  </div>

  <script src="~/lib/jquery/jquery.min.js"></script>
  @await RenderSectionAsync("Scripts", required: false)
</body>
</html>
```

El TempData["Exito"] es un diccionario que persiste exactamente una petición HTTP. Lo vamos a usar para mostrar mensajes de confirmación después de guardar o eliminar un contacto.

Página Index: listar los contactos

Esta es la pantalla principal. Muestra todos los contactos y permite buscar por nombre o teléfono.


```

csharp
// Pages/Agenda/Index.cshtml.cs
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;

public class IndexModel : PageModel
{
    private readonly AgendaDbContext _db;

    public IndexModel(AgendaDbContext db) => _db = db;

    public List<Contacto> Contactos { get; set; } = new();

    [BindProperty(SupportsGet = true)]
    public string? Buscar { get; set; }

    public async Task OnGetAsync()
    {
        var query = _db.Contactos.AsQueryable();

        if (!string.IsNullOrEmpty(Buscar))
        {
            query = query.Where(c =>
                c.Nombre.Contains(Buscar) ||
                c.Telefono.Contains(Buscar));
        }

        Contactos = await query.OrderBy(c => c.Nombre).ToListAsync();
    }
}

```

El atributo `[BindProperty(SupportsGet = true)]` le dice a ASP.NET que la propiedad `Buscar` puede recibir su valor tanto del formulario (POST) como de la URL (GET). Así la búsqueda funciona con un formulario de tipo `method="get"` y el término queda visible en la URL, lo que permite compartir el enlace.

La vista correspondiente muestra la barra de búsqueda y una tarjeta por cada contacto:

```

html
@page
@model IndexModel
@{ ViewData["Title"] = "Mi Agenda"; }

<div class="d-flex justify-content-between align-items-center mb-3">
    <h1>Contactos</h1>
    <a asp-page="/Create" class="btn btn-primary">+ Nuevo contacto</a>
</div>

<form method="get" class="mb-4">
    <div class="input-group">
        <input asp-for="Buscar" class="form-control" placeholder="Buscar..." />
        <button type="submit" class="btn btn-outline-secondary">Buscar</button>
    </div>
</form>

<p class="text-muted">@Model.Contactos.Count resultado(s)</p>

@foreach (var c in Model.Contactos)
{
    <div class="card mb-2">
        <div class="card-body d-flex justify-content-between align-items-center">
            <div>
                <strong>@c.Nombre</strong> – @c.Telefono
                @if (c.Email != null) { <span class="text-muted ms-2">@c.Email</span> }
            </div>
            <div class="d-flex gap-2">
                <a asp-page="/Details" asp-route-id="@c.Id" class="btn btn-sm btn-info">Ver</a>
                <a asp-page="/Edit" asp-route-id="@c.Id" class="btn btn-sm btn-warning">Editar</a>
                <a asp-page="/Delete" asp-route-id="@c.Id" class="btn btn-sm btn-danger">Eliminar</a>
            </div>
        </div>
    </div>
}

```

html - continuacion

```

    </div>
  </div>
}

```

Página Create: agregar un contacto

El flujo de creación tiene dos pasos. Primero el usuario pide el formulario (GET), lo completa y lo envía (POST). Si los datos son válidos, guardamos y redirigimos. Si no, mostramos los errores.

csharp

```

// Pages/Agenda/Create.cshtml.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

public class CreateModel : PageModel
{
    private readonly AgendaDbContext _db;

    public CreateModel(AgendaDbContext db) => _db = db;

    [BindProperty]
    public Contacto Contacto { get; set; } = new();

    public void OnGet() { }

    public async Task<IActionResult> OnPostAsync()
    {
        if (!ModelState.IsValid)
            return Page();

        _db.Contactos.Add(Contacto);
        await _db.SaveChangesAsync();

        TempData["Exito"] = $"Contacto {Contacto.Nombre} creado exitosamente.";
        return RedirectToPage("./Index");
    }
}

```

El atributo [BindProperty] hace que ASP.NET tome automáticamente los valores del formulario y los asigne a las propiedades del objeto Contacto. No necesitamos leer los campos del request a mano.

Cuando el POST es exitoso, redirigimos en lugar de devolver Page(). Esto se llama el patrón Post-Redirect-Get y evita que el navegador reenvíe el formulario si el usuario recarga la página.

La vista es un formulario estándar con Tag Helpers:

html

```

@page
@model CreateModel
@{ ViewData["Title"] = "Nuevo contacto"; }

<h1>Nuevo contacto</h1>
<a asp-page="./Index" class="btn btn-secondary mb-3">← Volver</a>

<div class="col-md-6">
<form method="post">
    <div class="mb-3">
        <label asp-for="Contacto.Nombre" class="form-label fw-bold"></label>
        <input asp-for="Contacto.Nombre" class="form-control" />
        <span asp-validation-for="Contacto.Nombre" class="text-danger"></span>
    </div>

```

html - continuacion

```

<div class="mb-3">
  <label asp-for="Contacto.Telefono" class="form-label fw-bold"></label>
  <input asp-for="Contacto.Telefono" class="form-control" />
  <span asp-validation-for="Contacto.Telefono" class="text-danger"></span>
</div>

<div class="mb-3">
  <label asp-for="Contacto.Email" class="form-label"></label>
  <input asp-for="Contacto.Email" class="form-control" type="email" />
  <span asp-validation-for="Contacto.Email" class="text-danger"></span>
</div>

<div class="mb-3">
  <label asp-for="Contacto.Direccion" class="form-label"></label>
  <input asp-for="Contacto.Direccion" class="form-control" />
</div>

<div class="mb-3">
  <label asp-for="Contacto.Notas" class="form-label"></label>
  <textarea asp-for="Contacto.Notas" class="form-control" rows="3"></textarea>
</div>

<div class="form-check mb-3">
  <input asp-for="Contacto.EsFavorito" class="form-check-input" type="checkbox" />
  <label asp-for="Contacto.EsFavorito" class="form-check-label">Marcar como favorito</label>
</div>

<button type="submit" class="btn btn-primary">Guardar</button>
</form>
</div>

@section Scripts {
    @{ await Html.RenderPartialAsync("_ValidationScriptsPartial"); }
}

```

La sección Scripts agrega los scripts de validación del lado del cliente. Estos leen los atributos `asp-validation-for` y muestran los errores en tiempo real, sin necesidad de enviar el formulario.

Página Edit: modificar un contacto existente

La edición es casi idéntica a la creación. La diferencia es que el GET recibe un `id`, carga el contacto de la base de datos y lo muestra prellenado en el formulario.

```

csharp
// Pages/Agenda/Edit.cshtml.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;

public class EditModel : PageModel
{
    private readonly AgendaDbContext _db;

    public EditModel(AgendaDbContext db) => _db = db;

    [BindProperty]
    public Contacto Contacto { get; set; } = default!;

    public async Task<IActionResult> OnGetAsync(int? id)
    {
        if (id == null) return NotFound();

        var contacto = await _db.Contactos.FindAsync(id);
    }
}

```

csharp - continuacion

```

        if (contacto == null) return NotFound();

        Contacto = contacto;
        return Page();
    }

    public async Task<IActionResult> OnPostAsync()
    {
        if (!ModelState.IsValid)
            return Page();

        _db.Attach(Contacto).State = EntityState.Modified;

        await _db.SaveChangesAsync();

        TempData["Exito"] = "Contacto actualizado exitosamente.";
        return RedirectToPage("./Index");
    }
}

```

El `_db.Attach(Contacto).State = EntityState.Modified` le dice a Entity Framework que el objeto ya existe en la base de datos y que todos sus campos fueron modificados. Así, al llamar a `SaveChangesAsync`, genera un UPDATE con todos los campos.

La vista es igual a la de creación, pero con el Id del contacto en un campo oculto para que el POST sepa cuál actualizar:

html

```

@page "{id:int}"
@model EditModel
@{ ViewData["Title"] = "Editar contacto"; }

<h1>Editor: @Model.Contacto.Nombre</h1>
<a asp-page="./Index" class="btn btn-secondary mb-3">← Volver</a>

<div class="col-md-6">
<form method="post">
    <input type="hidden" asp-for="Contacto.Id" />

    <div class="mb-3">
        <label asp-for="Contacto.Nombre" class="form-label fw-bold"></label>
        <input asp-for="Contacto.Nombre" class="form-control" />
        <span asp-validation-for="Contacto.Nombre" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Contacto.Telefono" class="form-label fw-bold"></label>
        <input asp-for="Contacto.Telefono" class="form-control" />
        <span asp-validation-for="Contacto.Telefono" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Contacto.Email" class="form-label"></label>
        <input asp-for="Contacto.Email" class="form-control" type="email" />
        <span asp-validation-for="Contacto.Email" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Contacto.Direccion" class="form-label"></label>
        <input asp-for="Contacto.Direccion" class="form-control" />
    </div>

    <div class="mb-3">
        <label asp-for="Contacto.Notas" class="form-label"></label>
        <textarea asp-for="Contacto.Notas" class="form-control" rows="3"></textarea>
    </div>

```

html - continuación

```

<div class="form-check mb-3">
  <input asp-for="Contacto.EsFavorito" class="form-check-input" type="checkbox" />
  <label asp-for="Contacto.EsFavorito" class="form-check-label">Favorito</label>
</div>

  <button type="submit" class="btn btn-warning">Guardar cambios</button>
</form>
</div>

@section Scripts {
    @{ await Html.RenderPartialAsync("_ValidationScriptsPartial"); }
}

```

La directiva `@page "{id:int}"` le dice al router que esta página espera un parámetro entero en la URL. Así `/Agenda/Edit/5` funciona directamente sin pasar el `id` como query string.

Página Details: ver el detalle de un contacto

La página de detalle es solo lectura. Cargamos el contacto y lo mostramos, sin formularios ni botones de guardado.

csharp

```

// Pages/Agenda/Details.cshtml.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

public class DetailsModel : PageModel
{
    private readonly AgendaDbContext _db;

    public DetailsModel(AgendaDbContext db) => _db = db;

    public Contacto Contacto { get; set; } = default!;

    public async Task<IActionResult> OnGetAsync(int? id)
    {
        if (id == null) return NotFound();

        var contacto = await _db.Contactos.FindAsync(id);

        if (contacto == null) return NotFound();

        Contacto = contacto;
        return Page();
    }
}

```

html

```

@page "{id:int}"
@model DetailsModel
@{ ViewData["Title"] = Model.Contacto.Nombre; }

<div class="d-flex justify-content-between align-items-center mb-3">
  <h1>@Model.Contacto.Nombre</h1>
  <div class="d-flex gap-2">
    <a asp-page="/Edit" asp-route-id="@Model.Contacto.Id" class="btn btn-warning">Editar</a>
    <a asp-page="/Index" class="btn btn-secondary">← Volver</a>
  </div>
</div>

<dl class="row">
  <dt class="col-sm-3">Teléfono</dt>
  <dd class="col-sm-9">@Model.Contacto.Telefono</dd>

```

html - continuación

```

@if (Model.Contacto.Email != null)
{
    <dt class="col-sm-3">Email</dt>
    <dd class="col-sm-9">@Model.Contacto.Email</dd>
}

@if (Model.Contacto.Direccion != null)
{
    <dt class="col-sm-3">Dirección</dt>
    <dd class="col-sm-9">@Model.Contacto.Direccion</dd>
}

@if (Model.Contacto.Notas != null)
{
    <dt class="col-sm-3">Notas</dt>
    <dd class="col-sm-9">@Model.Contacto.Notas</dd>
}

<dt class="col-sm-3">Favorito</dt>
<dd class="col-sm-9">@(Model.Contacto.EsFavorito ? "Sí" : "No")</dd>
</dl>

```

Página Delete: eliminar un contacto

La eliminación tiene una regla importante: nunca eliminemos datos como resultado de un GET. Si lo hiciéramos, cualquier enlace o bot que visitara la URL borraría el registro sin que el usuario lo confirme.

La solución es mostrar una página de confirmación en el GET y ejecutar la eliminación en el POST.

csharp

```

// Pages/Agenda/Delete.cshtml.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

public class DeleteModel : PageModel
{
    private readonly AgendaDbContext _db;

    public DeleteModel(AgendaDbContext db) => _db = db;

    [BindProperty]
    public Contacto Contacto { get; set; } = default!;

    public async Task<IActionResult> OnGetAsync(int? id)
    {
        if (id == null) return NotFound();

        var contacto = await _db.Contactos.FindAsync(id);

        if (contacto == null) return NotFound();

        Contacto = contacto;
        return Page();
    }

    public async Task<IActionResult> OnPostAsync(int id)
    {
        var contacto = await _db.Contactos.FindAsync(id);

        if (contacto != null)
        {
            _db.Contactos.Remove(contacto);
            await _db.SaveChangesAsync();
            TempData["Exito"] = "Contacto eliminado.";
        }
    }
}

```

csharp - continuacion

```

        return RedirectToPage("./Index");
    }
}

```

html

```

@page "{id:int}"
@model DeleteModel
@{ ViewData["Title"] = "Eliminar contacto"; }

<div class="alert alert-danger">
    <h4>¿Confirmar eliminación?</h4>
    <p>Esta acción no se puede deshacer.</p>
</div>

<div class="card mb-3">
    <div class="card-body">
        <h5>@Model.Contacto.Nombre</h5>
        <p>@Model.Contacto.Telefono</p>
        @if (Model.Contacto.Email != null) { <p>@Model.Contacto.Email</p> }
    </div>
</div>

<form method="post">
    <input type="hidden" asp-for="Contacto.Id" />
    <div class="d-flex gap-2">
        <button type="submit" class="btn btn-danger">Sí, eliminar</button>
        <a asp-page="./Index" class="btn btn-secondary">Cancelar</a>
    </div>
</form>

```

El flujo completo de una petición

Vale la pena detenerse a ver qué pasa exactamente cuando un usuario hace click en "Editar" un contacto.

El navegador envía GET /Agenda/Edit/5. ASP.NET Core recorre el pipeline de middleware y el router identifica que esa URL corresponde a Pages/Agenda/Edit.cshtml. El sistema de inyección de dependencias crea el EditModel y le pasa el AgendaDbContext. Se ejecuta OnGetAsync(5), que busca el contacto en la base de datos. Razor renderiza Edit.cshtml con los datos del EditModel y el servidor devuelve el HTML al navegador.

El usuario modifica el nombre y hace click en "Guardar". El navegador envía POST /Agenda/Edit/5 con los datos del formulario. ASP.NET Core crea un nuevo EditModel, [BindProperty] puebla el objeto Contacto con los datos del formulario y se ejecuta OnPostAsync(). Si los datos son válidos, actualizamos en la base de datos y retornamos RedirectToPage("./Index"), que es un HTTP 302. El navegador sigue la redirección y hace un GET /Agenda, donde ve el mensaje de éxito.

Este ciclo se repite para cada operación del CRUD.

Inyección de dependencias

Notamos que todos los PageModels reciben el AgendaDbContext en el constructor. Nunca lo crean ellos mismos. Esto se llama inyección de dependencias y es el mecanismo central de ASP.NET Core.

Cuando registramos AddDbContext<AgendaDbContext>() en Program.cs, le decimos al contenedor de servicios que él es responsable de crear y proveer instancias de ese tipo. Cada vez que un PageModel

declara un parámetro del tipo `AgendaDbContext` en su constructor, el contenedor se lo provee automáticamente.

Esto tiene dos ventajas prácticas: el `PageModel` no sabe cómo se construye el `DbContext` ni qué base de datos usa, y podemos reemplazar la implementación en cualquier momento sin tocar el código de los `PageModels`.

Validaciones

El sistema de validación funciona en dos capas. En el servidor, `ModelState.IsValid` verifica que todos los atributos del modelo se cumplan. Si alguno falla, `ModelState` contiene los mensajes de error y los `Tag Helpers asp-validation-for` los muestran en la vista.

En el cliente, el parcial `_ValidationScriptsPartial` carga los scripts de jQuery Validation que leen esos mismos atributos y muestran los errores mientras el usuario escribe, sin necesidad de hacer un round-trip al servidor.

Cuando necesitamos validaciones que van más allá de los atributos estándar, las agregamos directamente en el `PageModel`:

```
csharp
public async Task<IActionResult> OnPostAsync()
{
    var duplicado = await _db.Contactos
        .AnyAsync(c => c.Telefono == Contacto.Telefono && c.Id != Contacto.Id);

    if (duplicado)
        ModelState.AddModelError("Contacto.Telefono", "Ya existe un contacto con ese teléfono.");

    if (!ModelState.IsValid)
        return Page();

    // ... resto del guardado
}
```

Cómo pensar el próximo proyecto

Con este CRUD como base, el proceso para cualquier nuevo proyecto es siempre el mismo.

Primero identificamos las entidades: ¿con qué datos trabaja la aplicación? Pueden ser productos, pedidos, usuarios, artículos, lo que sea. Cada entidad se convierte en un modelo C# con sus propiedades y atributos de validación.

Después definimos las relaciones entre entidades. ¿Un pedido tiene muchos productos? ¿Un artículo pertenece a una categoría? Esas relaciones se expresan con propiedades de navegación en los modelos y con `Include()` en las consultas.

Luego listamos las pantallas. Por cada entidad, pensamos Index, Create, Edit, Details y Delete. No todas son necesarias siempre, pero es un buen punto de partida.

Creamos el `DbContext`, generamos las migraciones y empezamos a construir las páginas de adentro hacia afuera: primero el `PageModel` con la lógica, después la vista con el HTML. Si el `PageModel` está bien, la vista es casi mecánica.

El resto es acumular capas: autenticación con ASP.NET Identity, paginación con `Skip` y `Take`, búsqueda más sofisticada, carga de archivos con `IFormFile`, o tiempo real con SignalR. Todas esas funcionalidades se agregan sobre la misma base conceptual que vimos acá.

La clave es que el framework hace mucho trabajo pesado por nosotros. Si nombramos las cosas correctamente y seguimos las convenciones, el routing, el binding, la validación y la inyección de dependencias funcionan solos. Cuando algo no anda, la respuesta casi siempre está en los nombres o en el orden del pipeline de middleware.

CAPITULO 60

Blazor en .NET 10 / C# 14 — Tutorial completo en español argentino

TL;DR

- **Blazor es el framework de Microsoft para construir interfaces web interactivas usando C# en lugar de JavaScript**, y en .NET 10 unificó sus dos modelos históricos (Server y WebAssembly) bajo el template **Blazor Web App**, que permite elegir el modo de renderizado por componente con la directiva `@rendermode`.
- Para un curso de segundo año conviene arrancar con **Interactive Server** (`dotnet new blazor -int Server`): tiene tiempo de carga mínimo, debugging idéntico al de cualquier app .NET y oculta el código C# al navegador.
- En este tutorial vas a aprender la teoría (componentes Razor, ciclo de vida, binding, EventCallback, DI, ruteo, formularios) y vas a construir una **agenda de contactos CRUD con lista en memoria** paso a paso.

Parte 1 — Conceptual

1.1 ¿Qué es Blazor y para qué sirve?

Blazor es un framework de UI web que forma parte de ASP.NET Core. La documentación oficial lo define así:

"Blazor is a .NET frontend web framework that supports both server-side rendering and client interactivity in a single programming model." — Microsoft Learn

En criollo: Blazor te permite escribir **toda la app (UI + lógica)** en C# y Razor, y compartir clases, DTOs y validaciones entre cliente y servidor. La página final que ve el navegador sigue siendo HTML + CSS estándar; lo que cambia es **quién** la genera y **dónde** corre el código C# que reacciona a los clics.

¿Por qué te debería interesar como estudiante?

1. **Reutilizás conocimiento de C#/I.NET** (POO, LINQ, async/await, colecciones) en el frontend; no hace falta aprender JavaScript de cero.
2. **Tipado fuerte** end-to-end: si renombrás una propiedad del modelo, el compilador te avisa en la vista.
3. **Modelo de componentes** muy parecido al de React/Vue/Angular, pero con C# y sin Node.

■ ■ Blazor no reemplaza a JavaScript en el navegador: convive con él vía *interop* (IJSRuntime). Para tareas chicas tipo "scroll a un elemento" o "leer del localStorage" todavía necesitás JS.

1.2 Modelos de hosting: Server vs WebAssembly

Históricamente Blazor tuvo dos formas de correr el código C# que maneja la UI:

Blazor Server (Interactive Server)

El C# corre **en el servidor**. El navegador mantiene una conexión SignalR (WebSocket) abierta; cuando el usuario hace clic, el evento se envía al servidor, éste corre el handler, calcula el **diff** del DOM y manda solo los cambios.

- **Descarga inicial muy chica:** en .NET 10 el único script de framework, `blazor.web.js`, pesa 43 KB (Telerik / Jon Hilton, ".NET 10 Has Arrived—Here's What's Changed for Blazor", 13-nov-2025: "*blazor.web.js drops from 183 KB to 43 KB (76% size reduction)*") y **arranque casi instantáneo**.
- Acceso directo a recursos de servidor (DB, sistema de archivos, secrets) sin exponer nada al cliente.
- Debugging idéntico a cualquier app .NET (breakpoints en Visual Studio, logs en consola).
- **Requiere conexión permanente.** Si se cae el WebSocket, la UI se queda colgada (en .NET 10 se mitigó con persistencia de circuito, pero el problema base sigue).
- **Latencia:** cada interacción es un round-trip de red.
- **Escalabilidad:** cada usuario activo consume memoria del servidor para mantener su "circuit".

Blazor WebAssembly (Interactive WebAssembly)

El **runtime de .NET completo se compila a WebAssembly** y se descarga al navegador, junto con tus DLLs. Una vez cargado, todo el C# corre en la máquina del usuario.

- **App offline-capable** (después de la carga inicial puede funcionar sin internet).
- Escalabilidad enorme: el servidor solo sirve archivos estáticos o una Web API.
- Rendimiento de UI superior una vez cargada (sin latencia de red por cada clic).
- **Descarga inicial pesada** (varios MB con el runtime; en .NET 10 mejoró con preloading y compresión).
- **Sin acceso directo a DB:** necesitás una Web API intermedia.
- El código C# **se descarga al cliente** — no es buen lugar para secretos ni lógica de negocio sensible.

.NET 8+: Render modes y el template "Blazor Web App"

Desde .NET 8 Microsoft unificó los dos mundos. Ahora hay un único template (`dotnet new blazor`) que puede elegir el modo de renderizado **por componente** mediante la directiva `@rendermode`:

Render mode	Dónde corre	Cuándo usarlo
Static SSR	Servidor, una sola vez	Páginas marketing/landing, contenido SEO, sin interactividad
InteractiveServer	Servidor + SignalR	Apps internas, dashboards, lo que antes era "Blazor Server"
InteractiveWebAssembly	Navegador (WASM)	SPAs offline, alta interactividad cliente
InteractiveAuto	Empieza Server, después WASM	Lo mejor de los dos: arranque rápido + cliente luego

Decisión para este curso

Vamos a usar **InteractiveServer**. Razones pedagógicas:

- Setup más simple (un solo proyecto, sin `.Client`).
- Debugging directo desde Visual Studio / Rider / VS Code.
- No hay que armar Web API aparte para acceder a la "base" (la lista en memoria).
- Los conceptos de componentes/binding/lifecycle son **idénticos** a WASM, así que lo aprendido transfiere.

1.3 Blazor en el ecosistema .NET 10 / C# 14

.NET 10 (lanzado el 11 de noviembre de 2025) es una versión **LTS** con soporte hasta el **10 de noviembre de 2028**, según el blog oficial de Microsoft (.NET Blog, "Announcing .NET 10", 11-nov-2025: *".NET 10 is a Long Term Support (LTS) release and will be supported for three years, until November 10, 2028"*). Es el target recomendado para producción ahora.

Lo más relevante para Blazor en .NET 10:

- `[PersistentState]` — atributo nuevo (clase `PersistentStateAttribute`) que persiste estado entre el prerender y el render interactivo, eliminando el "doble fetch" de datos. La documentación oficial lo describe así: *"To preserve prerendered state, use the `[PersistentState]` attribute to persist state in properties. Properties with this attribute are automatically persisted using the `PersistentComponentState` service during prerendering."*
- **Reducción del 76% en `blazor.web.js`** — de 183 KB a 43 KB gracias a compresión y fingerprinting automáticos (Telerik / Jon Hilton, ".NET 10 Has Arrived—Here's What's Changed for Blazor", 13-nov-2025: *"the compression takes the blazor.web.js script down by about 76%"*).
- `<ReconnectModal />` — componente built-in en el template que reemplaza el viejo UI de reconexión y respeta políticas de CSP estrictas.
- `NotFound.razor` + parámetro `Router.NotFoundPage` para manejar 404 de forma uniforme.
- `NavigationManager.NotFound()` método nuevo para disparar 404 desde código.
- **Validación con `[ValidatableType]`** (experimental) y `AddValidation()` para formularios anidados.
- **Hot Reload habilitado por defecto para builds Debug de Blazor WebAssembly** (Telerik / Jon Hilton, 13-nov-2025: *"Hot Reload is now enabled by default for Blazor WebAssembly Debug builds. Edit a .razor file, save, and see changes instantly without rebuilding or refreshing the browser."*).

Del lado de **C# 14**, lo que más vamos a usar en este tutorial:

- **field keyword** — accede al backing field auto-generado dentro de un property accessor, sin declarar el campo a mano:

```
csharp
public string Nombre {
    get;
    set => field = value?.Trim() ?? throw new ArgumentNullException(nameof(value));
}
```

- **Null-conditional assignment** — `cliente?.Pedido = ...` ahora compila.

- **nameof con generics no contruidos** — `nameof(List<>)` devuelve "List".

■ **Estilo de código del tutorial:** usamos llaves K&R (la { va en la misma línea que la declaración), y para acceder a campos de instancia escribimos `this.campo` siempre. Es una convención para que el código quede prolijo y para distinguir visualmente lo que es estado del objeto.

1.4 Componentes Razor

Un **componente Razor** es la unidad de composición de Blazor. Vive en un archivo `.razor` y es, debajo, una clase de C# que hereda de `ComponentBase`. La documentación oficial lo expresa así: *"Components are .NET C# classes built into .NET assemblies that: Define flexible UI rendering logic. Handle user events. Can be nested and reused."*

Estructura básica de un `.razor`

```
csharp
@* Saludo.razor *@
@page "/saludo"
@using MiApp.Modelos
@inject ILogger<Saludo> Logger

<h3>Hola, @this.nombre</h3>
<button @onclick="Cambiar">Cambiar</button>

@code {
    private string nombre = "Mundo";

    private void Cambiar() {
        this.nombre = "Blazor";
        this.Logger.LogInformation("Nombre cambiado");
    }
}
```

Partes:

1. **Directivas** (`@page`, `@using`, `@inject`, etc.) — siempre arriba.
2. **Markup HTML + Razor** — el cuerpo, con expresiones `@variable` y atributos `@onclick`, `@bind...`
3. **Bloque `@code { ... }`** — el "code-behind" inline: campos, propiedades, métodos.

Directivas más usadas

Directiva	Para qué sirve
@page "/ruta"	Convierte al componente en una página accesible vía URL
@using Namespace	Igual que using en C#
@inject Tipo nombre	Inyecta una dependencia
@code { ... }	Bloque de código C# del componente
@bind, @bind-Value	Data binding bidireccional
@onclick, @onchange	Handlers de eventos del DOM
@ref	Capturar referencia al elemento o componente
@rendermode	Especificar render mode (Interactive Server / WebAssembly / Auto)
@layout	Override del layout para esta página
@attribute [Authorize]	Aplicar un atributo a la clase generada

1.5 Ciclo de vida de un componente

El "por qué" antes de la sintaxis: cuando Blazor renderiza un componente no lo hace todo de una vez. Hay momentos diferenciados (creación, recepción de parámetros, render del DOM real) y necesitás engancharte al correcto. Si cargás datos en el momento equivocado, vas a tener bugs raros tipo "se duplica la llamada al servicio" o "el @ref es null".

El flujo, simplificado, es:



Cuándo overridear cada uno:

- `OnInitialized / OnInitializedAsync` → inicialización única (cargar dropdowns estáticos, subscribirse a un evento global). **Se llama UNA sola vez por instancia del componente.**
- `OnParametersSet / OnParametersSetAsync` → reaccionar a parámetros que cambian (ej. el padre te pasa un Id diferente y tenés que recargar el detalle).
- `OnAfterRender(bool firstRender) / OnAfterRenderAsync` → cosas que necesitan el DOM ya construido: @ref, interop con JavaScript, foco a un input. La sobrecarga toma un bool firstRender que

es true solo la primera vez.

- `Dispose()` (si implementás `IDisposable`) → liberar timers, unsubscribe de eventos.

■ **Trampa clásica con prerendering:** si tu app tiene prerender activado (es el default), `OnInitializedAsync` se llama **dos veces**: una en el servidor durante el prerender, otra cuando arranca el modo interactivo. Para evitar duplicar fetches, usá `[PersistentState]` en .NET 10.

```
csharp
@code {
    [PersistentState]
    public List<Contacto>? Contactos { get; set; }

    protected override async Task OnInitializedAsync() {
        // Esto solo se ejecuta si Contactos viene null
        this.Contactos ??= await this.ContactoService.ListarAsync();
    }
}
```

1.6 Data binding

Blazor distingue tres patrones:

Unidireccional (one-way): C# → UI

```
csharp
<p>Total: @this.total</p>
```

Cualquier cambio de `this.total` que dispare un re-render se refleja. La UI no puede modificarlo desde acá.

Bidireccional (two-way) con `@bind`

```
csharp
<input @bind="this.nombre" />
<p>Hola @this.nombre</p>
```

`@bind="campo"` es azúcar para `value="@campo" @onchange="e => campo = e.Value"`. Por defecto se actualiza con `onchange` (cuando el input pierde foco). Si querés que se actualice en cada tecla:

```
csharp
<input @bind="this.nombre" @bind:event="oninput" />
```

`@bind:after` (.NET 7+) para correr lógica después

```
csharp
<input @bind="this.filtro" @bind:after="Buscar" />
```

Binding a componentes hijos

Si tu componente expone una propiedad [Parameter] `public string Texto { get; set; },` para que sea bindeable bidireccionalmente tiene que también exponer [Parameter] `public EventCallback<string> TextoChanged { get; set; }` (la convención de nombre `<Prop>Changed` es obligatoria). Después el padre puede hacer:

```
csharp
<MiInput @bind-Texto="this.busqueda" />
```

Eventos DOM

```
csharp
<button @onclick="Guardar">Guardar</button>
<input @onkeydown="OnTecla" />
```

Los handlers pueden ser void, async Task, recibir o no un EventArgs (MouseEventArgs, KeyboardEventArgs, etc.).

1.7 Parámetros entre componentes

Padre → Hijo: [Parameter]

```
csharp
@* TarjetaContacto.razor *@
<div class="card">
    <h5>@this.Contacto.Nombre</h5>
    <p>@this.Contacto.Email</p>
</div>

@code {
    [Parameter, EditorRequired]
    public Contacto Contacto { get; set; } = default!;
}
```

Uso desde el padre:

```
csharp
<TarjetaContacto Contacto="@this.contactoActual" />
```

Hijo → Padre: EventCallback<T>

El hijo expone un callback como parámetro, y lo invoca cuando algo pasa. Blazor llama automáticamente a `StateHasChanged()` en el padre cuando el callback termina, así que la UI se actualiza sola.

```
csharp
@* BotonEliminar.razor *@
<button class="btn btn-danger" @onclick="OnClick">■</button>

@code {
    [Parameter] public int Id { get; set; }
    [Parameter] public EventCallback<int> OnEliminar { get; set; }

    private Task OnClick() => this.OnEliminar.InvokeAsync(this.Id);
}
```



```
csharp - continuacion
}
```

Padre:

```
csharp
<BotonEliminar Id="@C.Id" OnEliminar="Eliminar" />

@code {
    private void Eliminar(int id) { /* ... */ }
}
```

■ **Por qué *EventCallback* y no *Action/Func***: a diferencia de un *delegate normal*, *EventCallback* está atado al renderer del componente que lo recibió, así que dispara el re-render automático del padre **sin** tener que llamar `StateHasChanged()` a mano.

1.8 Inyección de dependencias

Blazor usa el mismo contenedor de DI que ASP.NET Core. Registrás servicios en `Program.cs`:

```
csharp
builder.Services.AddSingleton<IContactoService, ContactoService>();
builder.Services.AddScoped<UsuarioActual>();
builder.Services.AddTransient<IGeneradorDeId, GeneradorDeId>();
```

Los tres lifetimes:

Lifetime	Blazor Server	Blazor WASM
Singleton	Una sola instancia para toda la app, compartida entre usuarios	Una instancia por sesión de navegador
Scoped	Una instancia por circuito (≈ por usuario conectado)	Equivalente a Singleton
Transient	Una nueva instancia por cada <code>@inject</code>	Igual

■ **Trampa importante**: en *Blazor Server*, *Scoped* **no es por petición HTTP** como en *MVC*; es **por circuito *SignalR***, o sea por sesión de usuario. Si guardás estado en un servicio *Scoped*, ese estado dura mientras el usuario tenga la pestaña abierta y conectada.

Para nuestro CRUD en memoria conviene **Singleton** (queremos que todos los usuarios vean la misma agenda).

Cómo se inyecta en un componente

Dos formas equivalentes:

```
csharp
@Inject IContactoService Contactos
```

o en `@code`:

```
csharp
[Inject]
private IContactoService Contactos { get; set; } = default!;
```

■ *Atajo argentino: en componentes solemos usar la primera porque queda más compacta. La segunda es útil cuando tenés code-behind separado.*

1.9 Routing

Definir rutas

Cada página se hace ruteable con `@page`:

```
csharp
@page "/contactos"
@page "/agenda"    @* podés tener varias rutas para la misma página *@
```

Parámetros de ruta con tipo

```
csharp
@page "/contactos/{Id:int}"

@code {
    [Parameter] public int Id { get; set; }
}
```

Los **constraints** soportados según la documentación oficial de routing en .NET 10 son: `bool`, `datetime`, `decimal`, `double`, `float`, `guid`, `int`, `long`, `nonfile`. La doc aclara: "Route constraints that verify the URL and are converted to a CLR type (such as `int` or `System.DateTime`) always use the invariant culture."

Parámetros opcionales y query strings

```
csharp
@page "/buscar"

@code {
    [SupplyParameterFromQuery]
    public string? Termino { get; set; }

    [SupplyParameterFromQuery(Name = "pagina")]
    public int Pagina { get; set; } = 1;
}
```

Navegación programática

Se inyecta `NavigationManager`:

```
csharp
@Inject NavigationManager Nav

private void IrAEditar(int id) {
    this.Nav.NavigateTo($"/contactos/editar/{id}");
}
```

NavLink vs <a>

<NavLink> agrega automáticamente la clase CSS active cuando la URL coincide. Es preferible para menús de navegación:

```
csharp
<NavLink href="/contactos" Match="NavLinkMatch.All">Contactos</NavLink>
```

Página 404 en .NET 10

La doc oficial muestra el patrón:

```
csharp
@page "/not-found"
@layout MainLayout

<h3>Not Found</h3>
<p>Sorry, the content you are looking for does not exist.</p>
```

Y en Routes.razor:

```
csharp
<Router AppAssembly="@typeof(Program).Assembly"
    NotFoundPage="typeof(Pages.NotFound)">
    <Found Context="routeData">
        <RouteView RouteData="@routeData" />
        <FocusOnNavigate RouteData="@routeData" Selector="h1" />
    </Found>
</Router>
```

1.10 Formularios y validación

Blazor incluye un componente EditForm que envuelve un <form> HTML y le agrega un EditContext que trackea cambios y validaciones.

Componentes input built-in

La doc oficial lista estos wrappers (todos heredan de InputBase<T>):

- InputText → <input>
- InputTextArea → <textarea>
- InputNumber<T> → <input type="number">
- InputCheckbox → <input type="checkbox">
- InputDate<T> → <input type="date">
- InputSelect<T> → <select>

- `InputRadio<T>` / `InputRadioGroup<T>` → radio buttons
- `InputFile` → upload de archivos

Validación con DataAnnotations

```
csharp
public class Contacto {
    public int Id { get; set; }

    [Required(ErrorMessage = "El nombre es obligatorio")]
    [StringLength(80, MinimumLength = 2)]
    public string Nombre { get; set; } = "";

    [Required, EmailAddress]
    public string Email { get; set; } = "";

    [Phone]
    public string? Telefono { get; set; }
}
```

Y la vista:

```
csharp
<EditForm Model="this.modelo" OnValidSubmit="Guardar" FormName="ContactoForm">
    <DataAnnotationsValidator />
    <ValidationSummary />

    <div class="mb-3">
        <label>Nombre</label>
        <InputText @bind-Value="this.modelo.Nombre" class="form-control" />
        <ValidationMessage For="@(() => this.modelo.Nombre)" />
    </div>

    <button type="submit" class="btn btn-primary">Guardar</button>
</EditForm>
```

Los tres callbacks de `EditForm`:

- `OnValidSubmit` → solo si la validación pasa.
- `OnInvalidSubmit` → solo si hay errores.
- `OnSubmit` → siempre, vos manejás la validación a mano.

Parte 2 — Proyecto práctico: Agenda de contactos CRUD

2.1 Lo que vamos a construir

Una pequeña aplicación de gestión de contactos con:

- Lista paginada de contactos.
- Buscador por nombre.
- Alta de contacto (formulario validado).
- Edición de contacto existente.

- Eliminación con confirmación.
- Datos persistidos en una `List<Contacto>` en memoria (singleton) — sin EF, sin DB.

2.2 Crear el proyecto

Abrí una terminal en la carpeta donde querés trabajar y ejecutá:

```
bash
dotnet new blazor -o Agenda -int Server
cd Agenda
dotnet run
```

¿Qué hicieron esos comandos?

Según la documentación oficial de `dotnet new`, el template `blazor` acepta estas opciones:

- `-int|--interactivity <None|Server|WebAssembly|Auto>` — *"Specifies which interactive render mode to use for interactive components"*. Le dimos `Server`.
- `-ai|--all-interactive` — *"Makes every page interactive by applying an interactive render mode at the top level. If false, pages use static server-side rendering by default and can be marked interactive on a per-page or per-component basis."* No lo usamos: vamos a marcar la interactividad página por página.
- `--empty` — *"Omits sample pages and styling that demonstrate basic usage patterns"*. No lo usamos para tener el layout y CSS de arranque.
- `-au|--auth <None|Individual>` — autenticación (`None` por defecto).
- `--use-program-main` — *"If specified, an explicit Program class and Main method is generated instead of top-level statements"*. No lo usamos: top-level statements queda más limpio.

2.3 Estructura del proyecto generado

```
texto
Agenda/
  Components/
    App.razor          ← html root (<html>, <head>, <body>)
    Routes.razor       ← <Router> con la tabla de rutas
    _Imports.razor     ← @usings compartidos
    Layout/
      MainLayout.razor
      MainLayout.razor.css
      NavMenu.razor
      NavMenu.razor.css
    Pages/
      Home.razor
      Counter.razor
      Weather.razor
      Error.razor
      NotFound.razor   ← nuevo en .NET 10
  wwwroot/
    app.css
  Program.cs
  Agenda.csproj
```

2.4 Revisión de Program.cs

El archivo generado por `dotnet new blazor -int Server` en .NET 10 contiene las siguientes llamadas clave (la doc oficial las describe así: *"Services for Razor components are added by calling AddRazorComponents. AddInteractiveServerComponents adds services to support rendering Interactive Server components."*):

```
csharp
var builder = WebApplication.CreateBuilder(args);

// 1. Registrar Blazor + servicios de interactividad
builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();

// 2. ■■■■ ACÁ vamos a registrar nuestro servicio
// builder.Services.AddSingleton<IContactoService, ContactoServiceEnMemoria>();

var app = builder.Build();

if (!app.Environment.IsDevelopment()) {
    app.UseExceptionHandler("/Error", createScopeForErrors: true);
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseAntiforgery(); // protección CSRF para formularios
app.MapStaticAssets(); // sirve wwwroot/ con fingerprinting

app.MapRazorComponents<App>()
    .AddInteractiveServerRenderMode();

app.Run();
```

2.5 Modelo de dominio

Creá una carpeta `Modelos/` y dentro `Contacto.cs`:

```
csharp
using System.ComponentModel.DataAnnotations;

namespace Agenda.Modelos;

public class Contacto {
    public int Id { get; set; }

    [Required(ErrorMessage = "El nombre es obligatorio.")]
    [StringLength(80, MinimumLength = 2,
        ErrorMessage = "El nombre debe tener entre 2 y 80 caracteres.")]
    public string Nombre {
        get;
        // C# 14: usamos 'field' para acceder al backing field generado
        // y normalizar el valor (trim) antes de guardarlo.
        set => field = (value ?? "").Trim();
    } = "";

    [Required, EmailAddress(ErrorMessage = "Email inválido.")]
    public string Email { get; set; } = "";

    [Phone(ErrorMessage = "Teléfono inválido.")]
    public string? Telefono { get; set; }

    [DataType(DataType.Date)]
    public DateOnly? Cumpleaños { get; set; }

    public Categoria Categoria { get; set; } = Categoria.Personal;
}
```

csharp - continuacion

```
public enum Categoria { Personal, Trabajo, Familia, Otro }
```

Notá el uso del **keyword** **field de C# 14**: nos permite ejecutar lógica en el setter (un `Trim()`) sin tener que declarar a mano un `private string _nombre;`. La sintaxis previa era más verbosa.

2.6 Servicio en memoria con inyección de dependencias

Creá Servicios/IContactoService.cs:

csharp

```
using Agenda.Modelos;

namespace Agenda.Servicios;

public interface IContactoService {
    IReadOnlyList<Contacto> Listar(string? filtro = null);
    Contacto? ObtenerPorId(int id);
    Contacto Crear(Contacto nuevo);
    bool Actualizar(Contacto modificado);
    bool Eliminar(int id);
}
```

Implementación Servicios/ContactoServiceEnMemoria.cs:

csharp

```
using Agenda.Modelos;

namespace Agenda.Servicios;

public class ContactoServiceEnMemoria : IContactoService {
    // Usamos this. para acceder a campos de instancia, como pide el estilo
    // del curso.
    private readonly List<Contacto> contactos;
    private int proximoId;
    private readonly Lock candado = new(); // sincronización: varios usuarios

    public ContactoServiceEnMemoria() {
        // Sembramos algunos contactos de ejemplo.
        this.contactos = [
            new Contacto { Id = 1, Nombre = "Diego Maradona",
                Email = "d10@argentina.ar", Categoria = Categoria.Familia },
            new Contacto { Id = 2, Nombre = "Lionel Messi",
                Email = "messi@psg.fr", Categoria = Categoria.Trabajo },
            new Contacto { Id = 3, Nombre = "Charly García",
                Email = "charly@sui.generis", Categoria = Categoria.Personal },
        ];
        this.proximoId = this.contactos.Count + 1;
    }

    public IReadOnlyList<Contacto> Listar(string? filtro = null) {
        lock (this.candado) {
            if (string.IsNullOrEmpty(filtro)) {
                return this.contactos
                    .OrderBy(c => c.Nombre)
                    .ToList();
            }
            // LINQ: ya lo conocen del año pasado
            return this.contactos
                .Where(c => c.Nombre.Contains(filtro,
                    StringComparison.OrdinalIgnoreCase))
                .OrderBy(c => c.Nombre)
                .ToList();
        }
    }
}
```

csharp - continuacion

```

    }

    public Contacto? ObtenerPorId(int id) {
        lock (this.candado) {
            return this.contactos.FirstOrDefault(c => c.Id == id);
        }
    }

    public Contacto Crear(Contacto nuevo) {
        lock (this.candado) {
            nuevo.Id = this.proximoId++;
            this.contactos.Add(nuevo);
            return nuevo;
        }
    }

    public bool Actualizar(Contacto modificado) {
        lock (this.candado) {
            var existente = this.contactos.FirstOrDefault(
                c => c.Id == modificado.Id);
            if (existente is null) {
                return false;
            }
            existente.Nombre = modificado.Nombre;
            existente.Email = modificado.Email;
            existente.Telefono = modificado.Telefono;
            existente.Cumpleaños = modificado.Cumpleaños;
            existente.Categoria = modificado.Categoria;
            return true;
        }
    }

    public bool Eliminar(int id) {
        lock (this.candado) {
            var existente = this.contactos.FirstOrDefault(c => c.Id == id);
            if (existente is null) {
                return false;
            }
            return this.contactos.Remove(existente);
        }
    }
}

```

¿Por qué lock? Porque en Blazor Server hay un solo proceso atendiendo a múltiples usuarios. Sin el lock, dos usuarios creando contactos al mismo tiempo podrían pisarse el proximoId. Es buena costumbre incluso si para el curso no importa mucho. Lock es el tipo nuevo (.NET 9) para Monitor; si no lo tenés, usá private readonly object candado = new();.

Registrá el servicio en Program.cs:

csharp

```
builder.Services.AddSingleton<IContactoService, ContactoServiceEnMemoria>();
```

Lo registramos como **Singleton** porque queremos que la "base" sea única y se comparta entre todos los usuarios. Si fuera *Scoped*, cada usuario tendría su propia agenda (no es lo que queremos).

2.7 Página 1: Listado

Creá Components/Pages/Contactos.razor:


```

csharp
@page "/contactos"
@rendermode InteractiveServer
@inject IContactoService Contactos
@inject NavigationManager Nav
@using Agenda.Modelos
@using Agenda.Servicios

<PageTitle>Contactos</PageTitle>

<h1>■ Mis contactos</h1>

<div class="d-flex gap-2 mb-3">
    <input class="form-control" placeholder="Buscar por nombre..."
        @bind="this.filtro" @bind:event="oninput" @bind:after="Refrescar" />
    <a class="btn btn-primary" href="/contactos/nuevo">+ Nuevo</a>
</div>

@if (this.lista is null) {
    <p><em>Cargando...</em></p>
} else if (this.lista.Count == 0) {
    <div class="alert alert-info">No hay contactos que coincidan.</div>
} else {
    <table class="table table-striped">
        <thead>
            <tr>
                <th>Nombre</th>
                <th>Email</th>
                <th>Categoría</th>
                <th class="text-end">Acciones</th>
            </tr>
        </thead>
        <tbody>
            @foreach (var c in this.lista) {
                <tr>
                    <td>@c.Nombre</td>
                    <td>@c.Email</td>
                    <td><span class="badge bg-secondary">@c.Categoría</span></td>
                    <td class="text-end">
                        <a class="btn btn-sm btn-outline-primary"
                            href="@($"/contactos/editar/{c.Id}")">Editar</a>
                        <button class="btn btn-sm btn-outline-danger"
                            @onclick="() => Confirmar(c)">
                            Eliminar
                        </button>
                    </td>
                </tr>
            }
        </tbody>
    </table>
}

@if (this.aEliminar is not null) {
    <div class="modal d-block" tabindex="-1" style="background:rgba(0,0,0,.4)">
        <div class="modal-dialog">
            <div class="modal-content">
                <div class="modal-header">
                    <h5 class="modal-title">¿Borrar contacto?</h5>
                </div>
                <div class="modal-body">
                    <p>Estás por borrar a <strong>@this.aEliminar.Nombre</strong>.
                        Esta acción no se puede deshacer.</p>
                </div>
                <div class="modal-footer">
                    <button class="btn btn-secondary"
                        @onclick="() => this.aEliminar = null">
                        Cancelar
                    </button>
                    <button class="btn btn-danger" @onclick="EliminarConfirmado">
                        Sí, borrar
                    </button>
                </div>
            </div>
        </div>
    </div>
}

```

```

csharp - continuacion
    </div>
</div>
}

@code {
    private string filtro = "";
    private IReadOnlyList<Contacto>? lista;
    private Contacto? aEliminar;

    protected override void OnInitialized() {
        // Carga inicial: ejecutar una sola vez.
        this.Refrescar();
    }

    private void Refrescar() {
        this.lista = this.Contactos.Listar(this.filtro);
    }

    private void Confirmar(Contacto c) => this.aEliminar = c;

    private void EliminarConfirmado() {
        if (this.aEliminar is not null) {
            this.Contactos.Eliminar(this.aEliminar.Id);
            this.aEliminar = null;
            this.Refrescar();
        }
    }
}

```

Conceptos que aparecen acá

1. `@rendermode InteractiveServer` — porque la página tiene botones que ejecutan código C# en respuesta a clics. Sin esto, los botones serían HTML estático y los `@onlick` no harían nada.
2. `@bind="filtro" @bind:event="oninput" @bind:after="Refrescar"` — binding bidireccional que se dispara en cada tecla y, después de actualizar la variable, llama `Refrescar()`. Esto da búsqueda en vivo.
3. `OnInitialized` **vs hacer el `Listar()` en línea**: lo hacemos en el lifecycle para que se ejecute una sola vez cuando el componente se monta.
4. **Lambda inline en el `@onclick="() => Confirmar(c)"`** — necesaria porque queremos pasar `c` como argumento.

2.8 Página 2: Crear / editar contacto

Components/Pages/ContactoFormulario.razor:

```

csharp
@page "/contactos/nuevo"
@page "/contactos/editar/{Id:int}"
@rendermode InteractiveServer
@inject IContactoService Contactos
@inject NavigationManager Nav
@using Agenda.Modelos
@using Agenda.Servicios

<PageTitle>@(this.EsNuevo ? "Nuevo contacto" : "Editar contacto")</PageTitle>

<h1>@(this.EsNuevo ? "■ Nuevo contacto" : $"=■ Editar: {this.modelo.Nombre}")</h1>

@if (this.cargando) {
    <p><em>Cargando...</em></p>

```

csharp - continuacion

```

} else {
    <EditForm Model="this.modelo" OnValidSubmit="Guardar"
        FormName="ContactoFormulario">
        <DataAnnotationsValidator />
        <ValidationSummary class="alert alert-danger" />

        <div class="mb-3">
            <label class="form-label">Nombre</label>
            <InputText @bind-Value="this.modelo.Nombre" class="form-control" />
            <ValidationMessage For="@(() => this.modelo.Nombre)"
                class="text-danger" />
        </div>

        <div class="mb-3">
            <label class="form-label">Email</label>
            <InputText @bind-Value="this.modelo.Email" class="form-control"
                type="email" />
            <ValidationMessage For="@(() => this.modelo.Email)"
                class="text-danger" />
        </div>

        <div class="mb-3">
            <label class="form-label">Teléfono</label>
            <InputText @bind-Value="this.modelo.Telefono"
                class="form-control" />
            <ValidationMessage For="@(() => this.modelo.Telefono)"
                class="text-danger" />
        </div>

        <div class="mb-3">
            <label class="form-label">Cumpleaños</label>
            <InputDate @bind-Value="this.modelo.Cumpleaños"
                class="form-control" />
        </div>

        <div class="mb-3">
            <label class="form-label">Categoría</label>
            <InputSelect @bind-Value="this.modelo.Categoria"
                class="form-select">
                @foreach (var cat in Enum.GetValues<Categoria>()) {
                    <option value="@cat">@cat</option>
                }
            </InputSelect>
        </div>

        <div class="d-flex gap-2">
            <button type="submit" class="btn btn-primary">■ Guardar</button>
            <a class="btn btn-secondary" href="/contactos">Cancelar</a>
        </div>
    </EditForm>
}

@code {
    [Parameter] public int? Id { get; set; }

    [SupplyParameterFromForm]
    public Contacto modelo { get; set; } = new();

    private bool cargando = true;
    private bool EsNuevo => this.Id is null;

    protected override void OnParametersSet() {
        // Si el Id cambia (porque navegaron a otro /contactos/editar/X),
        // volvemos a cargar.
        if (this.Id is int id) {
            var existente = this.Contactos.ObtenerPorId(id);
            if (existente is null) {
                this.Nav.NavigateTo("/contactos");
                return;
            }
            // Clonamos para no mutar el original hasta que se guarde.
            this.modelo = new Contacto {

```

csharp - continuacion

```

        Id = existente.Id,
        Nombre = existente.Nombre,
        Email = existente.Email,
        Telefono = existente.Telefono,
        Cumpleaños = existente.Cumpleaños,
        Categoria = existente.Categoria,
    };
}
this.cargando = false;
}

private void Guardar() {
    if (this.EsNuevo) {
        this.Contactos.Crear(this.modelo);
    } else {
        this.Contactos.Actualizar(this.modelo);
    }
    this.Nav.NavigateTo("/contactos");
}
}

```

Por qué dos @page en el mismo componente

Es una técnica muy útil: el **mismo componente** sirve tanto para crear (/contactos/nuevo) como para editar (/contactos/editar/123). La diferencia es que en el caso de edición llega un Id como parámetro de ruta; en el caso de creación, llega null.

Por qué OnParametersSet y no OnInitialized

Si el usuario navega de /contactos/editar/1 a /contactos/editar/2, Blazor **reutiliza el mismo componente** (no lo destruye). OnInitialized no se vuelve a llamar, pero OnParametersSet **sí** porque cambió el parámetro Id. Por eso cargamos los datos ahí.

2.9 Componente reutilizable: tarjeta de contacto

Para mostrar el uso de [Parameter] y EventCallback, creemos Components/TarjetaContacto.razor:

csharp

@using Agenda.Modelos

```

<div class="card mb-2">
    <div class="card-body d-flex justify-content-between align-items-center">
        <div>
            <h5 class="card-title mb-0">@this.Contacto.Nombre</h5>
            <small class="text-muted">@this.Contacto.Email</small>
        </div>
        <div>
            @if (this.OnVer.HasDelegate) {
                <button class="btn btn-sm btn-outline-info"
                    @onclick="() => this.OnVer.InvokeAsync(this.Contacto)">
                    Ver
                </button>
            }
            @if (this.OnEliminar.HasDelegate) {
                <button class="btn btn-sm btn-outline-danger"
                    @onclick="() => this.OnEliminar.InvokeAsync(this.Contacto.Id)">
                    ■
                </button>
            }
        </div>
    </div>
</div>

```

csharp - continuacion

```
@code {
    [Parameter, EditorRequired]
    public Contacto Contacto { get; set; } = default!;

    [Parameter]
    public EventCallback<Contacto> OnVer { get; set; }

    [Parameter]
    public EventCallback<int> OnEliminar { get; set; }
}
```

Uso desde otra página:

csharp

```
@foreach (var c in this.lista) {
    <TarjetaContacto Contacto="c"
        OnVer="VerDetalle"
        OnEliminar="Eliminar" />
}

@code {
    private void VerDetalle(Contacto c) {
        this.Nav.NavigateTo($"/contactos/editar/{c.Id}");
    }

    private void Eliminar(int id) {
        this.Contactos.Eliminar(id);
    }
}
```

HasDelegate permite que el componente sea flexible: si el padre no pasa el callback, el botón no se muestra.

2.10 Agregar el ítem al menú

Editá Components/Layout/NavMenu.razor y agregá:

csharp

```
<div class="nav-item px-3">
    <NavLink class="nav-link" href="/contactos">
        <span class="bi bi-person-lines-fill" aria-hidden="true"></span>
        Contactos
    </NavLink>
</div>
```

2.11 Ejecutar

bash

```
dotnet watch run
```

dotnet watch rebuilddea y refresca el navegador automáticamente cuando guardás un archivo .razor o .cs. Es tu mejor amigo durante el desarrollo.

Abrí <https://localhost:5001/contactos> (el puerto exacto te lo dice la consola) y deberías ver la lista con los tres contactos sembrados.

2.12 Resumen de archivos que tocaste

Archivo	Rol
Program.cs	Registrar el servicio singleton
Modelos/Contacto.cs	Entidad + DataAnnotations
Servicios/IContactoService.cs	Interfaz del repositorio
Servicios/ContactoServiceEnMemoria.cs	Implementación con List<T>
Components/Pages/Contactos.razor	Listado + búsqueda + delete
Components/Pages/ContactoFormulario.razor	Crear/editar
Components/TarjetaContacto.razor	Componente reutilizable
Components/Layout/NavMenu.razor	Link al menú

Recomendaciones (próximos pasos)

Etapas 1 — Consolidar lo aprendido (1-2 semanas):

1. Agregá una página de **detalle** (solo lectura) y un botón "Ver" en la lista.
2. Implementá **paginación** real en la lista (parámetros `pagina` y `tamaño` como query string).
3. Extraé el modal de confirmación a un componente genérico `<ConfirmDialog Titulo="..." OnConfirmar="..." />` con `RenderFragment` para el cuerpo.

Etapas 2 — Persistencia real (2-3 semanas):

1. Reemplazá `ContactoServiceEnMemoria` por una versión con **Entity Framework Core 10 + SQLite** (interfaz `IContactoService` permanece igual: ése es el valor de la abstracción).
2. Agregá manejo de errores con `ILogger<T>` y `try/catch` en el servicio.

Etapas 3 — Características avanzadas (cuando estés cómodo):

1. Probá cambiar el render mode de la página `Contactos` a `InteractiveAuto` y observá cómo arranca en server y migra a WASM.
2. Agregá autenticación con `dotnet new blazor -au Individual` en un proyecto nuevo.
3. Investigá la directiva `[StreamRendering]` para listas que cargan en streaming.

Cuándo cambiar de `InteractiveServer` a `InteractiveWebAssembly`: cuando necesites trabajar offline, cuando el costo de mantener circuits para muchos usuarios concurrentes se vuelva un problema (la guía oficial "Host and deploy ASP.NET Core server-side Blazor apps" de Microsoft Learn estima *"approximately 250 KB of memory for a minimal Hello World-style app"* por circuito, o sea ~1,3 GB de RAM para soportar 5.000 usuarios concurrentes), o cuando quieras hosting estático puro (GitHub Pages, Azure Static Web Apps).

Caveats

- **Render modes son sutiles:** `@rendermode InteractiveServer` debe aplicarse a la página o a un ancestro. Si te olvidás, los `@onclick` no harán nada y no vas a tener error explícito — solo botones inertes.
- **Prerender + servicios stateful:** si tu app prerenderiza (default), `OnInitializedAsync` corre dos veces (servidor + cliente). En .NET 10 usá `[PersistentState]` para no duplicar trabajo.
- `Scoped` ≠ **"por request"**: en Blazor Server, una instancia `Scoped` dura toda la sesión del usuario, no una petición HTTP. Confundir esto causa fugas de memoria y datos cruzados.
- **Lista en memoria:** el contenido se pierde al reiniciar el servidor. Sirve para enseñar; **no** lo uses para nada real. Para un cuatrimestre completo, agregá EF Core con SQLite (un solo archivo `.db`, cero configuración).
- **Versiones:** este tutorial asume .NET 10 (LTS, lanzado el 11 de noviembre de 2025) y C# 14. Algunos detalles (el atributo `[PersistentState]`, el `<ReconnectModal>`, la unificación `Router.NotFoundPage`) **no existen** en .NET 8/9. Si te toca una versión más vieja, ajustá. Hasta el final del soporte LTS (10 de noviembre de 2028) este código debería seguir compilando sin cambios.
- **C# 14 keyword field:** si tu IDE marca error, asegurate de que el `<LangVersion>preview</LangVersion>` o `<LangVersion>14.0</LangVersion>` esté en el `.csproj`, o de estar usando el SDK 10.0 que lo tiene como default.
- **El template `dotnet new blazor` cambió bastante entre .NET 7, 8 y 10.** Si seguís un tutorial viejo, es muy probable que la estructura de carpetas y `Program.cs` no coincida; mirá siempre la fecha del material.

CAPITULO 61

Anexo: Terminal y Bash para desarrollo

¿Qué es la terminal?

La **terminal** es una aplicación de texto. Su trabajo es mostrar un prompt, dejarte escribir comandos y mostrar la salida.

La terminal no interpreta los comandos por sí sola. El programa que los interpreta es el **shell**.

¿Qué es Bash?

Bash es un shell. Lee lo que escribís, lo procesa y ejecuta el comando correspondiente.

Cuando escribís algo como:

```
bash
ls -la
```

Bash hace varias tareas antes de ejecutar `ls`:

- separa el texto en partes,
- interpreta comillas y barras invertidas,
- expande variables como `$HOME`,
- expande atajos como `~`,
- resuelve comodines como `*.txt`,
- aplica redirecciones y pipes,
- y recién después ejecuta el programa.

Por eso, aprender Bash no es solo memorizar comandos: también es entender cómo el shell transforma lo que escribís.

Terminal, shell y sistema operativo

- **Terminal**: la aplicación o ventana donde escribís.
- **Shell**: el intérprete de comandos.
- **Bash**: un shell concreto.

En Linux, Bash es muy común. En macOS moderno, el shell por defecto suele ser **zsh**, no Bash, pero casi todos los ejemplos básicos de este apunte funcionan igual en ambos. En Windows, el shell por defecto es

CMD o **PowerShell**, pero también podés usar Bash a través de herramientas como Git Bash o WSL.

¿Para qué sirve?

Bash se usa para:

- navegar por carpetas y archivos,
- crear, copiar, mover y borrar contenido,
- ejecutar programas,
- automatizar tareas repetitivas,
- combinar herramientas pequeñas para resolver trabajos grandes,
- trabajar más rápido que con una interfaz gráfica en muchas tareas técnicas.

Usar Bash en Windows

En Windows hay dos opciones comunes:

- **Git Bash**: simple y suficiente para empezar.
- **WSL**: más parecido a un Linux real.

Para una primera práctica, **Git Bash** suele alcanzar.

Con **Git Bash** tenés acceso a comandos típicos de Bash como:

- `ls`
- `cp`
- `grep`
- `cat`
- `pwd`
- `mkdir`
- `rm`

En **CMD** o **PowerShell** no siempre vas a tener exactamente esos mismos comandos con comportamiento Bash. En PowerShell, por ejemplo, algunos nombres existen como alias, pero no son necesariamente los comandos GNU originales.

Instalar Git Bash

1. Instalá **Git for Windows**.
2. Abrí **Git Bash** desde el menú Inicio.
3. Verificá la instalación con:

```
bash
git --version
bash --version
which git
which bash
```

Si probás desde **CMD** o **PowerShell**, en lugar de which usá:

```
bash
where git
where bash
```

Cómo usarlo en VS Code

1. Abrí VS Code.
2. Presioná Ctrl + Shift + P y escribí "Terminal: Select Default Profile".
3. Elegí "Git Bash" como terminal por defecto.
4. Abrí una terminal nueva dentro de VS Code.

De esa forma, seguís trabajando dentro de VS Code, pero el shell que corre en la terminal integrada es Git Bash.

Rutas en Git Bash sobre Windows

En **Git Bash**, lo normal es usar / para separar directorios.

Ejemplos:

```
bash
/c/Users/Ana/Documentos
./carpeta/archivo.txt
../otra-carpeta/archivo.txt
```

Aunque Windows suele usar \, en Bash ese carácter normalmente se interpreta como escape. Por eso, para escribir rutas en Git Bash conviene usar /.

Cómo leer un comando

La forma general es esta:

```
texto
comando [opciones] [argumentos]
```

Ejemplos:

```
bash
ls -la
cp origen.txt copia.txt
grep -i "bash" notas.txt
dotnet run programa.cs
```

- `ls`, `cp`, `grep`, `dotnet` son comandos.
- `-l`, `-a`, `-i` son opciones.
- `origen.txt`, `copia.txt`, `notas.txt`, `programa.cs` son argumentos.

Directorio actual y rutas

Directorio actual

Es la carpeta en la que estás parado en este momento. Muchos comandos trabajan sobre esa ubicación.

Para verla:

```
bash
pwd
```

Ruta

Una ruta indica dónde está un archivo o una carpeta.

Puede ser:

- **absoluta**: empieza desde la raíz del sistema.
- **relativa**: se interpreta desde el directorio actual.

Ejemplos:

```
bash
/Users/usuario/Documentos
proyecto/archivo.txt
```

Atajos importantes:

- `.`: directorio actual.
- `..`: directorio padre.
- `~`: directorio personal del usuario actual.

Ejemplos:

```
bash
cd .
cd ..
cd ~
ls .
ls ..
```

Comillas y espacios en rutas

Si una ruta tiene espacios, tenés que protegerla.

Esto falla porque Bash separa por espacios:

```
bash
cd Mis Documentos
```

Esto funciona:

```
bash
cd "Mis Documentos"
cd Mis\ Documentos
```

Regla útil:

- comillas dobles `" . . . "`: mantienen el texto junto, pero permiten expandir variables.
- comillas simples `' . . . '`: mantienen el texto literal, sin expandir variables.

Ejemplos:

```
bash
echo "$HOME"
echo '$HOME'
```

La primera línea muestra el valor de la variable. La segunda imprime literalmente \$HOME.

Primeros comandos importantes

pwd

Muestra la ruta del directorio actual.

```
bash
pwd
```

ls

Lista el contenido de una carpeta.

```
bash
ls
ls -l
ls -a
ls -la
```

Opciones comunes:

- `-l`: formato largo, con permisos, tamaño y fecha.

- -a: incluye archivos ocultos.

En Unix, un archivo oculto suele empezar con `.`

También podés usar patrones:

```
bash
ls *.txt
ls a*
ls ??-*.md
```

Estos patrones no los interpreta `ls`: los interpreta Bash antes de ejecutar el comando. Se llaman **globs** o **comodines**.

- `*.txt`: nombres que terminan en `.txt`
- `a*`: nombres que empiezan con `a`
- `??-*.md`: dos caracteres, luego `-`, luego cualquier texto y al final `.md`

cd

Cambia de directorio.

```
bash
cd Documentos
cd ..
cd ~
cd
cd -
```

- `cd ..`: sube un nivel.
- `cd ~`: va al directorio personal.
- `cd` sin argumentos: también va al directorio personal.
- `cd -`: vuelve al directorio anterior.

mkdir

Crea carpetas.

```
bash
mkdir proyecto
mkdir docs src
mkdir -p proyecto/src/modelos
```

- -p: crea también carpetas intermedias si faltan.

touch

Si el archivo no existe, lo crea vacío. Si ya existe, actualiza su fecha de modificación.

```
bash
touch notas.txt
touch informe.md
```

cp

Copia archivos o carpetas.

```
bash
cp origen.txt copia.txt
cp archivo.txt Documentos/
cp -r carpeta1 carpeta2
```

- Si el destino es una carpeta existente, copia adentro.
- Si el destino no existe, crea una copia con ese nombre.
- -r copia carpetas de forma recursiva.

mv

Mueve o renombra.

```
bash
mv viejo.txt nuevo.txt
mv archivo.txt Documentos/
mv carpeta_vieja carpeta_nueva
```

mv no duplica: cambia la ubicación o el nombre.

rm

Elimina archivos o carpetas.

```
bash
rm archivo.txt
rm -r carpeta_vieja
rm -ri carpeta_vieja
```

- -r: borra de forma recursiva.
- -i: pide confirmación.

Importante:

- rm no manda a la papelera.
- si escribís mal la ruta, podés borrar otra cosa.
- conviene revisar con pwd y ls antes de usarlo.

cat

Muestra el contenido completo de un archivo en la salida estándar.

```
bash
cat archivo.txt
```

Para archivos largos, suele ser más cómodo usar `less`:

```
bash
less archivo.txt
```

Con `less` podés desplazarte. Salís con `q`.

head y tail

Sirven para ver el principio o el final de un archivo.

```
bash
head archivo.txt
tail archivo.txt
head -n 5 archivo.txt
tail -n 20 archivo.txt
```

echo

Imprime texto.

```
bash
echo Hola mundo
echo "$HOME"
```

`echo` es útil para casos simples. Si necesitás más control sobre el formato, `printf` suele ser más predecible:

```
bash
printf "Hola %s\n" "mundo"
```

clear

Limpia la pantalla de la terminal.

```
bash
clear
```

Ejecutar programas y scripts

Para ejecutar un programa instalado, escribís su nombre:

```
bash
git --version
dotnet --info
```

Para ejecutar un archivo del directorio actual, normalmente necesitás anteponer `./`:

```
bash
./programa.sh
```

Eso pasa porque, por seguridad, el directorio actual no suele estar en el PATH.

Si el archivo no tiene permiso de ejecución, podés hacerlo ejecutable con:

```
bash
chmod +x programa.sh
```

O ejecutarlo invocando Bash de forma explícita:

```
bash
bash programa.sh
```

Flujos de entrada, salida y error

Todo comando trabaja, como mínimo, con estos tres flujos:

- **entrada estándar** (`stdin`): lo que el programa recibe.
- **salida estándar** (`stdout`): resultado normal.
- **salida de error** (`stderr`): mensajes de error.

En Bash se suelen representar con estos números:

- 0: `stdin`
- 1: `stdout`
- 2: `stderr`

Redirecciones

La redirección permite cambiar de dónde lee un comando o a dónde escribe.

>

Redirige la salida estándar a un archivo y lo sobrescribe.

```
bash
ls > listado.txt
```


>>

Agrega al final del archivo.

```
bash
date >> historial.txt
```

<

Hace que el comando lea desde un archivo.

```
bash
sort < nombres.txt
```

2>

Redirige los errores a un archivo.

```
bash
ls carpeta-que-no-existe 2> errores.txt
```

2>&1

Hace que stderr vaya al mismo destino que stdout.

```
bash
comando > salida.txt 2>&1
```

Ese orden importa. En esta forma, tanto la salida normal como los errores terminan en `salida.txt`.

Ejemplos útiles

Guardar un listado:

```
bash
ls -la > contenido.txt
```

Guardar errores sin mezclar con la salida normal:

```
bash
grep "hola" archivo-inexistente.txt > salida.txt 2> errores.txt
```

Descartar mensajes de error:

```
bash
ls carpeta-que-no-existe 2> /dev/null
```

Conductos (|)

El símbolo | conecta la salida estándar de un comando con la entrada estándar del siguiente.

```
bash
comando1 | comando2
```

Eso no conecta los errores: `stderr` sigue aparte, salvo que lo redirijas.

Ejemplos:

```
bash
ls | wc -l
cat nombres.txt | sort
history | grep cd
```

Filtros comunes

grep

Busca líneas que coincidan con un patrón.

```
bash
grep "bash" archivo.txt
grep -i "bash" archivo.txt
ls | grep '\.md$'
```

- `-i`: ignora mayúsculas y minúsculas.

Importante:

- los globs como `*.txt` son cosa del shell;
- `grep` usa **expresiones regulares**, no globs.

Por eso:

```
bash
ls *.txt
```

usa un patrón de Bash, mientras que:

```
bash
ls | grep '\.txt$'
```

usa una expresión regular de `grep`.

sort

Ordena líneas.

```
bash
sort nombres.txt
sort < nombres.txt
```

WC -l

Cuenta líneas.

```
bash
wc -l alumnos.txt
ls | wc -l
```

Redirecciones y pipes juntos

Se pueden combinar:

```
bash
ls -la | grep '\.md$' > markdowns.txt
grep -i "bash" apuntes.txt | sort
```

En la primera línea:

1. `ls -la` genera la salida,
2. `grep` filtra solo líneas que terminan en `.md`,
3. `>` guarda el resultado final en `markdowns.txt`.

Variables de entorno

Una variable de entorno guarda información que Bash y otros programas pueden usar.

Ejemplos frecuentes:

- HOME: carpeta personal.
- PATH: lista de carpetas donde se buscan comandos.
- USER: nombre del usuario.

Ver una variable:

```
bash
echo "$HOME"
echo "$PATH"
```

Definir una variable para la sesión actual:

```
bash
NOMBRE="Ana"
echo "$NOMBRE"
```

Exportarla para que la vean otros procesos:

```
bash
export NOMBRE="Ana"
```

Definir una variable solo para un comando:

```
bash
NOMBRE="Ana" printenv NOMBRE
```

Cómo pedir ayuda

Dos herramientas muy útiles son:

```
bash
man ls
type cd
```

- `man ls`: abre el manual del comando `ls`.
- `type cd`: indica si algo es un comando externo, un builtin, un alias o una función.

También es común usar:

```
bash
which bash
```

`which` muestra qué ejecutable se está usando, aunque `type` suele ser más exacto dentro del shell.

En muchos programas también funciona:

```
bash
comando --help
```

Pero no es universal. En macOS varios comandos del sistema no aceptan `--help` y usan `man`.

Historial y edición de línea

Historial

Podés recuperar comandos anteriores con:

- `↑` y `↓`
- `history`

- `Ctrl + R` para buscar en el historial

Atajos útiles

- `Tab`: autocompleta nombres si puede.
- `Ctrl + C`: interrumpe el comando actual.
- `Ctrl + D`: cierra la entrada actual; si la línea está vacía, suele cerrar la sesión.
- `Ctrl + A`: va al inicio de la línea.
- `Ctrl + E`: va al final de la línea.
- `Ctrl + L`: limpia la pantalla.

Código de salida

Cuando un comando termina, devuelve un **código de salida**.

Por convención:

- `0` significa éxito.
- un valor distinto de `0` indica algún error o condición especial.

Podés ver el código del último comando con:

```
bash
echo $?
```

Ejemplo:

```
bash
ls carpeta-que-no-existe
echo $?
```

Buenas prácticas

- Revisá `pwd` antes de borrar o mover cosas.
- Usá comillas si hay espacios en rutas o nombres.
- Probá primero con `ls` o `echo` cuando no estés seguro de una expansión.
- Preferí `rm -ri` si querés más seguridad.
- Hacé copias antes de cambios grandes.
- No uses `sudo` si no entendés exactamente qué va a hacer el comando.

Casos habituales

Ver dónde estás y qué hay

```
bash
pwd
ls -la
```

Entrar a un proyecto

```
bash
cd Documentos
cd mi-proyecto
ls
```

Crear una estructura mínima

```
bash
mkdir -p proyecto/src proyecto/docs
cd proyecto
touch README.md
```

Hacer un respaldo rápido

```
bash
cp datos.csv datos-respaldo.csv
```

Buscar archivos Markdown y guardar el resultado

```
bash
ls -la | grep '\.md$' > markdowns.txt
```

Contar cuántas líneas contienen una palabra

```
bash
grep -i "bash" apuntes.txt | wc -l
```

Resumen

Si querés ubicarte rápido, pensá primero qué problema querés resolver y después elegí el comando que mejor encaja.

texto

```

¿Quiero saber en qué carpeta estoy?
    ■■■ `pwd`

¿Quiero ver qué hay en una carpeta?
    ■■■ `ls`, `ls -l`, `ls -a`, `ls -la`

¿Quiero moverme entre carpetas?
    ■■■ `cd`, `cd ..`, `cd ~`, `cd -`

¿Quiero crear carpetas o archivos?
    ■■■ `mkdir`, `mkdir -p`, `touch`

¿Quiero copiar, mover o borrar algo?
    ■■■ `cp`, `mv`, `rm`

¿Quiero ver el contenido de un archivo?
    ■■■ `cat`, `less`, `head`, `tail`

¿Quiero imprimir texto o una variable?
    ■■■ `echo`, `printf`, `$HOME`, `NOMBRE="Ana"`

¿Quiero buscar, ordenar o contar líneas?
    ■■■ `grep`, `sort`, `wc -l`

¿Quiero conectar comandos entre sí?
    ■■■ `|`

¿Quiero guardar la salida o los errores?
    ■■■ `>`, `>>`, `<`, `2>`, `2>&1`

¿Quiero pedir ayuda sobre un comando?
    ■■■ `man`, `type`, `--help`

¿Quiero saber si el último comando salió bien?
    ■■■ `echo $?`

¿Quiero ejecutar un programa o un script?
    ■■■ `git`, `dotnet`, `./programa.sh`, `bash programa.sh`

Ejemplos clave:
    `pwd`           → ruta actual
    `ls -la`        → lista detallada, incluyendo ocultos
    `cd ..`         → subir un nivel
    `mkdir -p`      → crear carpetas intermedias
    `cp`, `mv`, `rm` → copiar, mover y eliminar
    `grep`          → filtrar líneas por patrón
    `|`             → encadenar comandos
    `>` / `2>`      → redirigir salida normal o de error
    `man`           → consultar documentación
    `echo $?`       → ver el código de salida

```

La terminal no es una pantalla negra que hay que memorizar: es una forma precisa de hablarle al sistema.

Si entendés qué hace el shell, cómo se interpretan rutas y comillas, y cómo funcionan la entrada, la salida, los errores y los pipes, ya tenés una base sólida para trabajar con Bash con confianza y sin depender solo de prueba y error.

CAPITULO 62

Anexo: Use de Git y GitHub

1. ¿Qué es Git?

Git es un *sistema de control de versiones distribuido*. Traducido: es un programa que lleva un historial completo de todos los cambios que ocurren en una carpeta de archivos (típicamente, el código de un proyecto).

Para entender por qué existe, pensemos en el problema que resuelve. Sin Git, cuando varias personas trabajan sobre el mismo código, el flujo habitual termina siendo algo como:

```
texto
proyecto_final.cs
proyecto_final_v2.cs
proyecto_final_v2_corregido.cs
proyecto_final_v2_corregido_juan.cs
proyecto_final_ESTESIANDA.cs
```

Git elimina ese caos. Guarda cada versión del proyecto como una *foto* (un **commit**) y permite:

- Ver qué cambió, cuándo, y quién lo hizo.
- Volver a cualquier versión anterior.
- Trabajar en paralelo sobre la misma base sin pisarse.
- Combinar el trabajo de varias personas de forma controlada.

Distribuido

La parte de "distribuido" significa que **cada persona tiene una copia completa del repositorio en su máquina**, con todo el historial. No depende de un servidor central para trabajar. Se puede hacer commits sin internet.

Conceptos fundamentales

Estos son los términos que hay que internalizar. El resto se construye sobre ellos.

Repositorio (repo)

Es una carpeta gestionada por Git. Dentro tiene una subcarpeta oculta `.git/` donde se almacena todo el historial. Cuando decimos "el repositorio", nos referimos al proyecto completo con su historial.

Working directory (directorio de trabajo)

Los archivos como los ves en el explorador de archivos: lo que estás editando en este momento.

Staging area (área de preparación) / Index

Una zona intermedia donde se *preparan* los cambios antes de guardarlos definitivamente. Sirve para elegir **qué cambios** entran en el próximo commit. Esto permite, por ejemplo, hacer dos commits separados con cambios distintos aunque hayas editado varios archivos a la vez.

Commit

Es una *foto* del proyecto en un momento específico. Cada commit tiene:

- Un identificador único (un hash tipo a3f9b2c . . .).
- Un autor y una fecha.
- Un mensaje explicando qué cambió.
- Una referencia al commit anterior (su "padre").

Los commits forman una cadena histórica. Este historial es el corazón de Git.

Branch (rama)

Una **rama** es una línea de desarrollo independiente. Técnicamente es solo un puntero movable a un commit. La rama principal tradicionalmente se llama `main` (antes `master`).

Las ramas permiten trabajar en una funcionalidad nueva sin tocar la versión estable. Cuando la funcionalidad está lista, se *fusiona* (merge) con la rama principal.

Merge

Combinar los cambios de una rama en otra. Git intenta hacerlo automáticamente; cuando dos ramas modificaron la misma línea de manera distinta, aparece un **conflicto** que hay que resolver a mano.

Remote (remoto)

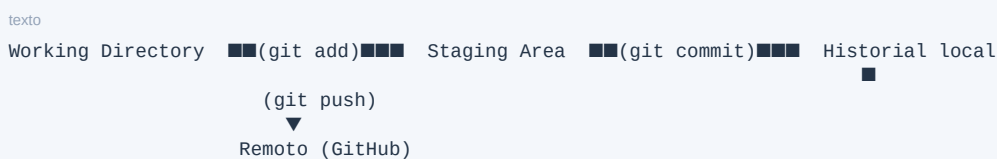
Una copia del repositorio que vive en otro lado — típicamente en un servidor como GitHub. El remoto por defecto se llama `origin`.

Push y Pull

- **Push**: subir tus commits locales al remoto.
- **Pull**: traer los commits del remoto a tu repo local (equivale a `fetch + merge`).

El flujo mental

Visualmente, el ciclo básico de trabajo con Git es:



2. ¿Qué es GitHub?

GitHub es un servicio web que hospeda repositorios Git. Es un servidor en la nube donde los equipos suben sus repositorios para compartirlos, colaborar y gestionarlos.

Diferencia clave: Git vs GitHub

Confundir ambos es el error más común al empezar. La distinción es:

	Git	GitHub
Qué es	Un programa (herramienta)	Un sitio web (servicio)
Dónde corre	En tu computadora	En los servidores de Microsoft
Quién lo hizo	Linus Torvalds, 2005	GitHub Inc. (hoy de Microsoft), 2008
Necesita internet	No	Sí
Alternativas	Mercurial, SVN	GitLab, Bitbucket, Azure DevOps

Git funciona perfectamente sin GitHub. Podés usar Git en una carpeta local de tu máquina y nunca subir nada a ningún lado. GitHub es un lugar donde *alojar* repositorios Git para que otros los puedan ver y colaborar.

Qué agrega GitHub sobre Git

GitHub no solo hospeda: añade funcionalidades que no son parte de Git:

- **Interfaz web** para navegar el código y el historial.
- **Pull Requests (PR)**: mecanismo para proponer cambios y revisarlos antes de fusionarlos.
- **Issues**: sistema de tickets para reportar bugs o proponer funcionalidades.
- **Actions**: automatización (ejecutar tests al subir código, por ejemplo).
- **Releases, Wiki, Projects**, control de permisos, etc.

Cómo trabajan juntos

El modelo típico es:

1. El código vive en GitHub (el remoto central compartido).
2. Cada desarrollador **clona** el repo a su máquina.
3. Trabaja localmente con Git (ramas, commits).
4. **Sube** sus cambios a GitHub con push.
5. Abre un **pull request** para integrar sus cambios a la rama principal.
6. El resto del equipo revisa, comenta y aprueba.
7. Se fusiona el PR.

3. Instalación en Windows

Vamos a instalar dos herramientas:

- **Git for Windows:** el programa Git en sí (te da el comando `git`).
- **GitHub CLI:** un programa oficial de GitHub (comando `gh`) que facilita autenticación, crear PRs, clonar repos privados, etc.

3.1 Instalar Git for Windows

Opción A — Instalador gráfico (recomendado la primera vez):

1. Descargá desde <<https://git-scm.com/download/win>>. La descarga debería empezar automáticamente.
 2. Ejecutá el .exe descargado.
 3. Durante el asistente, podés aceptar casi todos los valores por defecto. Dos pantallas a tener en cuenta:
 - **Default editor used by Git:** si no conocés Vim, cambialo a **Visual Studio Code** (o **Notepad**). Vim atrapa a muchos principiantes.
 - **Adjusting the name of the initial branch:** elegí **"Override the default branch name"** → `main`. Es el estándar actual.
1. El resto: Next, Next, Install.

Opción B — Por línea de comandos con winget (Windows 10/11):

Abrí PowerShell y ejecutá:

```
powershell  
winget install --id Git.Git -e
```

Verificar la instalación:

Cerrá y volvé a abrir la terminal (PowerShell o CMD), y escribí:

```
bash  
git --version
```

Debería responder algo como `git version 2.47.0.windows.1`.

Configuración inicial obligatoria:

Git necesita saber quién sos para registrar los commits. Ejecutá una vez:

```
bash  
git config --global user.name "Tu Nombre Apellido"  
git config --global user.email "tu.email@alumnos.utn.edu.ar"
```

Usá el mismo email que vas a usar en GitHub. Opcionalmente:

```
bash
git config --global init.defaultBranch main
git config --global core.autocrlf true
```

(autocrlf true evita problemas con saltos de línea entre Windows y otros sistemas.)

3.2 Instalar GitHub CLI

Opción A — Instalador gráfico:

Descargá el .msi desde <<https://cli.github.com/>> y ejecutalo.

Opción B — Con winget:

```
powershell
winget install --id GitHub.cli -e
```

Verificar:

```
bash
gh --version
```

Autenticarse con tu cuenta de GitHub:

```
bash
gh auth login
```

Te va a hacer una serie de preguntas. Respuestas recomendadas:

- *Where do you use GitHub?* → **GitHub.com**
- *What is your preferred protocol for Git operations?* → **HTTPS**
- *Authenticate Git with your GitHub credentials?* → **Yes**
- *How would you like to authenticate GitHub CLI?* → **Login with a web browser**

Te mostrará un código de un solo uso (tipo ABCD-1234), copialo, apretá Enter y se abre el navegador. Pegás el código, aceptás los permisos, y listo. A partir de ahora `git push` no te va a pedir usuario ni contraseña.

4. Flujo de trabajo: resolver un ejercicio y hacer un Pull Request

Este es el escenario típico del curso: el profesor tiene un repositorio con ejercicios en GitHub, y cada alumno resuelve su ejercicio en una rama y lo entrega mediante un PR.

Supongamos que el repo del curso es `alejandrodibattista/tup26-p3` y el ejercicio asignado es `tp1-65000`.

4.1 Clonar el repositorio (solo la primera vez)

Abrí una terminal en la carpeta donde querés guardar los proyectos (por ejemplo `C:\Users\ale\Dev\`) y clonalo:

```
bash
gh repo clone https://github.com/alejandrodibattista/tup26-p3
cd tup26-p3
```

`gh repo clone` es equivalente a `git clone https://github.com/alejandrodibattista/tup26-p3`, pero más corto y maneja la autenticación automáticamente.

4.2 Actualizar el repositorio local

Esto hay que hacerlo siempre antes de empezar a trabajar. Garantiza que partís de la versión más reciente, no de lo que tenías ayer.

```
bash
git switch main
git pull origin main
```

- `git switch main` te posiciona en la rama principal.
- `git pull origin main` trae los commits nuevos del remoto.

Si es la primera vez después de clonar, `pull` no trae nada — ya estás al día. Pero cuando el profesor publique nuevos ejercicios o correcciones, este es el comando que los trae.

4.3 Crear una rama para el ejercicio

Regla de oro: nunca trabajes directamente sobre main. Cada trabajo va en su propia rama.

```
bash
git switch -c tp1-65000
```

El flag `-c` (de *create*) crea la rama y te mueve a ella en un solo paso. El nombre de la rama conviene que describa qué hace — convenciones habituales:

- `tp1-65000`
- `feature/login-con-jwt`
- `fix/bug-division-por-cero`

Para verificar en qué rama estás:

```
bash
git branch
```

El asterisco (*) marca la rama actual.

4.4 Resolver el ejercicio

Ahora trabajás normalmente: abrís tu editor (VS Code, Rider, Visual Studio), editás los archivos, escribís el código, lo compilás, lo probás.

Mientras tanto, podés ir viendo qué cambió con:

```
bash
git status
```

Te muestra archivos modificados, agregados, eliminados, y cuáles están en el staging area y cuáles no.

```
bash
git diff
```

Te muestra exactamente qué líneas cambiaron.

4.5 Hacer commits

Cuando tengas una parte funcionando — no hace falta que el ejercicio esté terminado —, guardá un commit. **Commits pequeños y frecuentes son mejores que un commit gigante al final.**

```
bash
git add .
git commit -m "Implementar interfaz IComparable en clase Alumno"
```

- `git add .` pone *todos* los archivos modificados en el staging area. Si querés agregar solo uno: `git add Program.cs`.
- `git commit -m "..."` crea el commit con ese mensaje.

Buenas prácticas para el mensaje de commit:

- Empezar con un verbo en infinitivo o imperativo: "Agregar...", "Corregir...", "Refactorizar...".
- Que describa *qué* cambia, no *cómo*. El código ya muestra el cómo.
- Menos de 72 caracteres en la primera línea.

Repetí editar → add → commit tantas veces como haga falta hasta terminar el ejercicio.

4.6 Subir la rama a GitHub

Hasta ahora, todos los commits viven solo en tu máquina. Para subirlos:

```
bash
git push -u origin tp1-65000
```

El flag `-u` (de *upstream*) establece el vínculo entre tu rama local y la remota. Solo lo necesitás la primera vez. A partir del segundo push en la misma rama, alcanza con `git push`.

4.7 Crear el Pull Request

Ahora que la rama está en GitHub, abrimos el PR. Con GitHub CLI es un solo comando:

```
bash
gh pr create --base main --head tp1-65000
```

Te va a abrir un editor para escribir el título y la descripción del PR. Un buen PR contiene:

- **Título** claro: "Ejercicio 5 — Interfaces — Alejandro Pérez".
- **Descripción** con:
 - Qué resuelve el PR.
 - Decisiones de diseño relevantes.
 - Cómo probarlo (si aplica).
 - Cualquier duda que quieras que el profesor mire.

Ejemplo:

```
markdown
## Qué hace
Resuelve el ejercicio 5. Implementa `IComparable<Alumno>` ordenando por
promedio descendente, y `IEquatable<Alumno>` comparando por DNI.

## Decisiones
- Usé `record class` en vez de `class` para aprovechar la igualdad estructural
  automática, pero sobrescribí `Equals` para usar solo el DNI.
- El método `CompareTo` devuelve 0 cuando los promedios son iguales,
  sin desempatar por nombre.

## Dudas
¿Está bien que `IEquatable` y la igualdad de `record` convivan, o debería
volver a `class` común?
```

Alternativamente, sin CLI: gh te va a dar una URL; o entrás a GitHub en el navegador, tu rama aparecerá con un botón amarillo "Compare & pull request".

4.8 Revisión y ciclo de comentarios

El profesor revisa el PR en GitHub. Puede:

- **Aprobarlo** y fusionarlo.
- **Pedir cambios** con comentarios sobre líneas específicas.

Si pide cambios, el ciclo es:

1. Editás en tu máquina (en la misma rama).
2. `git add . → git commit -m "Ajustar CompareTo según revisión".`
3. `git push` (ya no hace falta `-u`).
4. El PR se actualiza automáticamente con los commits nuevos.

Cuando el profesor aprueba y mergea, tu rama se integra a main. Ese es el final del ciclo.

4.9 Limpieza después del merge

Una vez que el PR fue fusionado, la rama ya no hace falta:

```
bash
git switch main
git pull origin main
git branch -d tp1-65000
```

Y listo — estás limpio y listo para el próximo ejercicio.

5. Resumen del ciclo en 10 comandos

```
bash
# Una sola vez
git clone <url>

# Al empezar a trabajar en algo nuevo
git switch main
git pull origin main
git switch -c mi-rama

# Mientras trabajás
git status
git diff
git add .
git commit -m "mensaje"

# Para entregar
git push -u origin mi-rama
gh pr create

# Después del merge
git switch main
git pull origin main
git branch -d mi-rama
```

Con estos comandos se resuelve el 90% del trabajo diario. El resto (resolver conflictos, reescribir historia, cherry-pick, rebase, etc.) viene cuando haya necesidad real de aprenderlo.

6. Errores típicos a evitar

- **Trabajar directamente en main.** Siempre crear una rama.
- **Olvidarse de `git pull` antes de empezar.** Terminás con conflictos evitables.
- **Hacer un único commit gigante al final.** Perdés la capacidad de deshacer cambios parciales.
- **Commits con mensajes inútiles** ("cambios", "asdf", "fix"). En seis meses no vas a entender qué hiciste.
- **Subir archivos que no corresponden** (bin/, obj/, .vs/, node_modules/). Para eso existe .gitignore. Para proyectos .NET, GitHub genera uno automáticamente al crear el repo.
- **Hacer `git push --force`** sin entender qué hace. Puede borrar el trabajo de otros del servidor.

7. Circuito con GitHub Desktop

GitHub Desktop es la aplicación oficial de GitHub para trabajar con repositorios Git desde una interfaz gráfica. Sirve para clonar repositorios, crear ramas, revisar cambios, hacer commits, subir la rama al remoto y abrir pull requests sin usar la terminal todo el tiempo.

7.1 Instalar GitHub Desktop

1. Entrá a <<https://desktop.github.com/>>.
2. Descargá el instalador para Windows.
3. Ejecutalo y seguí el asistente.
4. Iniciá sesión con tu cuenta de GitHub.

GitHub Desktop usa Git por debajo, así que conviene tener Git for Windows instalado igualmente. No reemplaza Git: lo simplifica con una interfaz visual.

7.2 Circuito de trabajo con GitHub Desktop

1. Abrí GitHub Desktop e iniciá sesión.
2. Elegí **Clone a repository from the Internet** y clona el repositorio del curso.
3. Antes de empezar, asegurate de estar en **main** y hacé **Fetch origin** o **Pull origin** para actualizarte.
4. Creá una rama nueva con **Branch > New Branch**.
5. Editá el código en tu editor habitual.
6. Volvé a GitHub Desktop y revisá los archivos modificados.
7. Seleccioná los cambios que querés incluir, escribí el mensaje del commit y hacé clic en **Commit to <rama>**.
8. Hacé clic en **Push origin** para subir la rama a GitHub.
9. Usá **Create Pull Request** o **View on GitHub** para abrir el PR.
10. Si te piden cambios, repetí el ciclo: editar, commit y push.
11. Cuando el PR se fusione, hacé **Pull origin**, volvé a **main** y borrá la rama si ya no la necesitás.

7.3 Equivalencia con la terminal

- **Clone** = `git clone`
- **New Branch** = `git switch -c`
- **Commit** = `git add + git commit`
- **Push origin** = `git push`
- **Pull origin** = `git pull`
- **Create Pull Request** = abrir el PR en GitHub